Learn Being Forward

COURSES · CAMPUS PARTNERSHIPS · PLACEMENTS

COURSE NOTES · BASIC TO ADVANCED

Python Programming

Complete notes for the Learn Being Forward Python Programming syllabus: every module, topic and subtopic explained step by step, with diagrams, verified examples and a practice checkpoint for each module.

25	3	70	682
modules	levels	minimum days	Python programs

How to use these notes

These notes follow the **Learn Being Forward Python Programming syllabus** exactly: the same 25 modules, in the same order, with every topic and subtopic explained step by step. Each subtopic number (for example **3.7.2**) matches the syllabus, so you can always find where you are.

Level	Modules	Minimum	Outcome
1. Beginner	1 to 10	20 days	Write Python programs with variables, operators, control flow, strings, lists, tuples, sets, dictionaries and functions
2. Intermediate	11 to 17	19 days	Use decorators, modules and packages, handle exceptions and files, design classes with OOP, and work with iterators and generators
3. Advanced	18 to 25	31 days	Standard library, type hints, concurrency, internals, testing and packaging, databases and web, data analysis, DSA and interview readiness
Total	1 to 25	70 days	About 12 weeks at 6 study days per week

Read
Read the concept for each subtopic and note the key points in your own words.

Type & run
Type out and run every example. Do not copy and paste code. Compare with the OUTPUT box.

Checkpoint
Finish the practice checkpoint at the end of each module before moving on.

Revise
At the end of each level, revise its modules and build one small project.

Every program in these notes was executed to produce the output shown: core-language examples with **Python 3.14**, and examples that use third-party libraries (NumPy, pandas, Matplotlib, Flask, requests, Beautiful Soup) with the Python installation that has those libraries. Where a program reads keyboard input, the text typed by the user is shown in gold. Examples that need a network service or a package that could not be installed show the output those programs produce, and say so. Values that depend on the moment or the machine, such as dates, timings, memory addresses and random numbers, will differ when you run the programs.

HOW TO READ THE BOXES

Dark boxes with a gold top line are **Python code**. Dashed cream boxes labelled **OUTPUT** show what the program prints. Gold-bordered boxes are notes and tips, red boxes warn about common mistakes, and navy boxes hold the key idea to remember.

Course plan at a glance

Module	Topic	Level	Min days
01	Introduction to Python	Beginner	1
02	Setup and your first program	Beginner	1
03	Variables and data types	Beginner	2
04	Operators and expressions	Beginner	1

Module	Topic	Level	Min days
05	Input, output and type conversion	Beginner	1
06	Control flow	Beginner	3
07	Strings	Beginner	3
08	Lists and tuples	Beginner	3
09	Sets and dictionaries	Beginner	2
10	Functions	Beginner	3
11	Functional programming and decorators	Intermediate	3
12	Modules, packages and virtual environments	Intermediate	2
13	Exception handling	Intermediate	2
14	File handling and data formats	Intermediate	2
15	Object-oriented programming: classes and objects	Intermediate	3
16	Object-oriented programming: inheritance, polymorphism and special methods	Intermediate	4
17	Iterators, generators and context managers	Intermediate	3
18	Standard library essentials	Advanced	4
19	Type hints and modern Python features	Advanced	2
20	Concurrency and parallelism	Advanced	4
21	Python internals and performance	Advanced	2
22	Testing, debugging, code quality and packaging	Advanced	3
23	Databases, web scraping and web development	Advanced	5
24	Data analysis and visualisation	Advanced	5
25	Data structures, projects, interviews and career paths	Advanced	6

Table of contents

LEVEL 1 · BEGINNER LEVEL · MODULES 1 TO 10

Module 01 · Introduction to Python 9

1.1 What is Python? 9
1.2 History of Python 12
1.3 Why do we need Python? 14
1.4 Features of Python 18
1.5 Where is Python used? 20
1.6 Python implementations 25
1.7 Limitations of Python 25
1.8 Python compared with other languages 27

Module 02 · Setup and your first program 31

2.1 Installing Python 31
2.2 Ways to run Python 33
2.3 Editors and IDEs 35
2.4 Hello World 36
2.5 Basic syntax 38
2.6 Comments and docstrings 42
2.7 Errors 44

Module 03 · Variables and data types 50

3.1 Variables 50
3.2 Object identity 54
3.3 Variable scope (introduction) 56
3.4 Built-in data types overview 57
3.5 Numbers in depth 59
3.6 Booleans and None 64
3.7 Constants 66

Module 04 · Operators and expressions 69

4.1 Arithmetic operators 69
4.2 Comparison operators 72
4.3 Logical operators 74
4.4 Assignment operators 75
4.5 Bitwise operators 77
4.6 Identity and membership operators 78
4.7 Operator overloading by type 80
4.8 Precedence and associativity 80
4.9 Built-in math functions 82

Module 05 · Input, output and type conversion 87

5.1 Output with print() 87
5.2 String formatting 89
5.3 Input with input() 93
5.4 Type conversion 95
5.5 Command-line arguments 98

Module 06 · Control flow 101

6.1 Conditional statements 101

6.2 Structural pattern matching (3.10) 105
6.3 while loop 110
6.4 for loop 111
6.5 Loop control 115
6.6 Nested loops and patterns 116
6.7 Classic loop problems 120

Module 07 · Strings 127

7.1 Creating strings 127
7.2 Accessing characters 130
7.3 String operations 133
7.4 String methods 134
7.5 Building strings efficiently 138
7.6 Regular expressions (introduction) 139
7.7 Classic string problems 141

Module 08 · Lists and tuples 146

8.1 Lists 146
8.2 List methods 149
8.3 Copying in depth 154
8.4 List comprehensions 156
8.5 Built-in functions with lists 158
8.6 Lists as stacks and queues 159
8.7 Tuples 161
8.8 List problems 167

Module 09 · Sets and dictionaries 173

9.1 Sets 173
9.2 Dictionaries 177
9.3 Specialised dictionaries (collections) 184
9.4 How dictionaries and sets work 187
9.5 Choosing a collection 188
9.6 Problems 189

Module 10 · Functions 194

10.1 Function basics 194
10.2 Types of arguments 199
10.3 Scope and namespaces 205
10.4 Pass by object reference 209
10.5 Recursion 211
10.6 Built-in functions tour 218

LEVEL 2 · INTERMEDIATE LEVEL · MODULES 11 TO 17

Module 11 · Functional programming and decorators 223

11.1 Functions as first-class objects 223
11.2 Lambda functions 227
11.3 Higher-order functions 229
11.4 Closures 232
11.5 Decorators 235
11.6 functools 244

Module 12 · Modules, packages and virtual environments 250

12.1 Modules 250

12.2 Packages 256
12.3 Standard library tour 260
12.4 Installing packages with pip 263
12.5 Virtual environments 267

Module 13 · Exception handling 272

13.1 Basics 272
13.2 Built-in exceptions 274
13.3 Handling exceptions 277
13.4 Raising exceptions 282
13.5 Custom exceptions 284
13.6 Advanced topics 286
13.7 Best practices 290

Module 14 · File handling and data formats 295

14.1 File basics 295
14.2 Reading files 298
14.3 Writing files 300
14.4 File positions 302
14.5 Binary files 302
14.6 Paths and the file system 304
14.7 Data formats 308

Module 15 · Object-oriented programming: classes and objects 319

15.1 OOP concepts 319
15.2 Classes and objects 321
15.3 Attributes 324
15.4 Methods 328
15.5 Encapsulation 331
15.6 Object representation 335
15.7 Object lifecycle 335
15.8 Relationships between classes 337

Module 16 · Object-oriented programming: inheritance, polymorphism and special methods 341

16.1 Inheritance 341
16.2 Polymorphism 347
16.3 Abstraction 350
16.4 Special (dunder) methods 352
16.5 Data classes and enums 358
16.6 Mixins and composition over inheritance 361
16.7 Advanced class topics (overview) 363

Module 17 · Iterators, generators and context managers 368

17.1 Iterables and iterators 368
17.2 Generators 371
17.3 itertools 378
17.4 Comprehensions review 380
17.5 Context managers 381

LEVEL 3 · ADVANCED LEVEL · MODULES 18 TO 25

Module 18 · Standard library essentials 389

18.1 collections 389
18.2 Date and time 391

18.3 Operating system and files 393
18.4 Regular expressions in depth (re) 397
18.5 Maths and data 403
18.6 Logging 406
18.7 Command-line tools 409
18.8 Other useful modules 412

Module 19 · Type hints and modern Python features 418

19.1 Type hints 418
19.2 Static type checking 426
19.3 Runtime validation 429
19.4 Modern syntax recap 430
19.5 Recent CPython improvements 434

Module 20 · Concurrency and parallelism 440

20.1 Concepts 440
20.2 Threading 442
20.3 Multiprocessing 447
20.4 concurrent.futures 450
20.5 asyncio 452
20.6 Choosing the right tool 459

Module 21 · Python internals and performance 463

21.1 How Python executes code 463
21.2 Memory management 466
21.3 Performance measurement 470
21.4 Optimisation techniques 473
21.5 Going faster than CPython 478

Module 22 · Testing, debugging, code quality and packaging 482

22.1 Debugging 482
22.2 Testing 485
22.3 Code quality 493
22.4 Packaging 496
22.5 Version control 498
22.6 Design principles and patterns in Python 501

Module 23 · Databases, web scraping and web development 508

23.1 Databases 508
23.2 Working with web APIs 515
23.3 Web scraping 519
23.4 Web development 523
23.5 Automation 528

Module 24 · Data analysis and visualisation 535

24.1 NumPy 535
24.2 pandas 540
24.3 Visualisation 548
24.4 Exploratory data analysis workflow 554
24.5 Next steps into data science and AI (overview) 556

Module 25 · Data structures, projects, interviews and career paths 561

25.1 Data structures in Python 561
25.2 Algorithms 576
25.3 Capstone projects 591

25.4 Interview preparation 603
25.5 Career paths with Python 610

LEVEL 1

Beginner level

Outcome: write Python programs with variables, operators, control flow, strings, lists, tuples, sets, dictionaries and functions.

10 MODULES · 20 DAYS MINIMUM

Module 01 · Introduction to Python	**1 day**
Module 02 · Setup and your first program	**1 day**
Module 03 · Variables and data types	**2 days**
Module 04 · Operators and expressions	**1 day**
Module 05 · Input, output and type conversion	**1 day**
Module 06 · Control flow	**3 days**
Module 07 · Strings	**3 days**
Module 08 · Lists and tuples	**3 days**
Module 09 · Sets and dictionaries	**2 days**
Module 10 · Functions	**3 days**

MODULE 01 · BEGINNER

Introduction to Python

Minimum duration **1 day** | Level **Beginner**

Before writing any code, this module explains what Python is, how it runs, where it came from, why it became one of the most popular programming languages in the world, where it is used, which implementations exist, what its limitations are, and how it compares with C, C++, Java and JavaScript.

IN THIS MODULE

1.1	What is Python?	1.5	Where is Python used?
1.2	History of Python	1.6	Python implementations
1.3	Why do we need Python?	1.7	Limitations of Python
1.4	Features of Python	1.8	Python compared with other languages

1.1 What is Python?

1.1.1 Definition: a high-level, interpreted, general-purpose programming language

Python is a **high-level**, **interpreted**, **general-purpose** programming language. Each word in that definition matters. **High-level** means you write instructions in terms close to human thinking (lists, text, files, "for each student") rather than in terms of the machine (memory addresses, registers); Python manages memory for you. **Interpreted** means you do not compile a program into a separate executable before running it: you give the source file to the Python interpreter, which runs it directly. **General-purpose** means it is not tied to one field: the same language is used for websites, data analysis, artificial intelligence, automation scripts, games and scientific research.

A FIRST LOOK AT PYTHON CODE

main.py

```
# A complete Python program: no main function, no class, no semicolons needed
students = ["Asha", "Ravi", "Meera"]
marks = [91, 78, 85]

for name, mark in zip(students, marks):
    grade = "A" if mark >= 85 else "B"
    print(f"{name} scored {mark} -> grade {grade}")

print("Class average:", sum(marks) / len(marks))
```

OUTPUT

```
Asha scored 91 -> grade A
Ravi scored 78 -> grade B
Meera scored 85 -> grade A
Class average: 84.66666666666667
```

High-level
Work with ideas such as lists, text and files; memory is managed automatically by the interpreter.

Interpreted
Run the source file directly with `python file.py`; no separate compile-and-link step for you.

General-purpose
One language for web, data, AI, automation, testing, scripting, games and science.

1.1.2 Dynamically typed and strongly typed

Python is **dynamically typed**: you never declare the type of a variable. A variable is just a name for an object, and the **object** carries its type; the same name can later refer to an object of a different type. Types are checked while the program runs, not before. Python is also **strongly typed**: it does not silently convert between unrelated types. Adding a number to a text string raises a `TypeError` instead of guessing (JavaScript would produce `"52"`). Dynamic typing makes code short and flexible; strong typing prevents a whole family of hidden bugs. (Optional **type hints**, covered in Module 19, let tools check types before running.)

DYNAMIC TYPING, BUT STRONG TYPING

main.py

```
x = 42
print(x, type(x))
x = "forty-two"              # the same name now refers to a str object
print(x, type(x))
x = [4, 2]
print(x, type(x))

print("5" + "2")             # str + str: joining text
print(5 + 2)                 # int + int: arithmetic
print("5" + 2)               # str + int: Python refuses to guess
```

OUTPUT

```
42 <class 'int'>
forty-two <class 'str'>
[4, 2] <class 'list'>
52
7
Traceback (most recent call last):
  File "main.py", line 10, in <module>
    print("5" + 2)               # str + int: Python refuses to guess
          ~~~~^~~
TypeError: can only concatenate str (not "int") to str
```

	Static typing	Dynamic typing
Types checked	before running (compile time)	while running
Declaring types	`int age = 21;`	`age = 21`
Examples	C, C++, Java, C#, TypeScript	Python, JavaScript, Ruby, PHP

	Weak typing	Strong typing
Mixing types	silently converts: `"5" + 2` gives `"52"`	raises an error unless you convert explicitly
Examples	JavaScript, PHP, C (some conversions)	Python, Java, C#, Ruby

1.1.3 Multi-paradigm: procedural, object-oriented and functional

A **paradigm** is a style of organising programs. Python supports several and lets you mix them. **Procedural**: a sequence of steps grouped into functions. **Object-oriented**: data and behaviour bundled into classes and objects (Modules 15 and 16). **Functional**: building programs from functions that transform data, using `map`, `filter`, comprehensions and lambdas (Module 11). Most real Python code combines them: classes for the main concepts, functions for operations, comprehensions for data transformations.

THE SAME TASK IN THREE PARADIGMS main.py

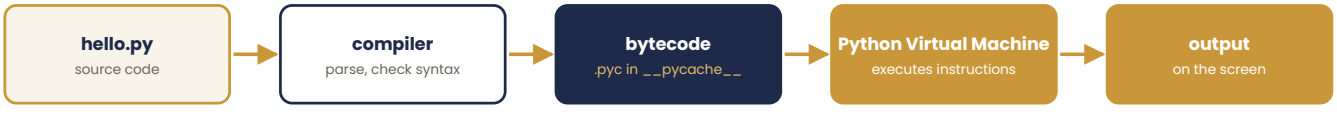
```
marks = [91, 78, 85, 64]

# 1. Procedural: step-by-step instructions
total = 0
for m in marks:
    total += m
print("procedural average:", total / len(marks))

# 2. Object-oriented: data and behaviour together in a class
class Batch:
    def __init__(self, marks):
        self.marks = marks
    def average(self):
        return sum(self.marks) / len(self.marks)
print("object-oriented average:", Batch(marks).average())

# 3. Functional: transform data with functions, no changing variables
from functools import reduce
print("functional average:", reduce(lambda a, b: a + b, marks) / len(marks))
print("toppers (comprehension):", [m for m in marks if m >= 80])
```

OUTPUT

```
procedural average: 79.5
object-oriented average: 79.5
functional average: 79.5
toppers (comprehension): [91, 85]
```

1.1.4 How Python runs: source (.py) → bytecode (.pyc) → Python Virtual Machine

When you run `python hello.py`, the standard interpreter, **CPython**, works in two stages. First it **compiles** your source code into **bytecode**: compact, low-level instructions for an imaginary machine. This step is automatic and fast, and it is where `SyntaxError`s are detected. For imported modules, the bytecode is cached in `.pyc` files inside a `__pycache__` folder, so the next run can skip compilation if the source has not changed. Then the **Python Virtual Machine** (PVM), a loop inside the interpreter, executes the bytecode instructions one by one. So "interpreted" in Python really means "compiled to bytecode, then interpreted"; Python 3.13 and 3.14 also include an experimental just-in-time (JIT) compiler that can turn hot bytecode into machine code.



FROM SOURCE CODE TO RUNNING PROGRAM IN CPYTHON

LOOKING AT THE BYTECODE PYTHON GENERATES

main.py

```
import dis

def greet(name):
    return "Hello, " + name

print(greet("Asha"))
print("--- bytecode of greet ---")
for ins in dis.get_instructions(greet):
    if ins.opname not in ("RESUME", "CACHE"):
        print(f"{ins.opname:<14} {ins.argrepr}")
```

OUTPUT

```
Hello, Asha
--- bytecode of greet ---
LOAD_CONST     'Hello, '
LOAD_FAST_BORROW name
BINARY_OP      +
RETURN_VALUE
```

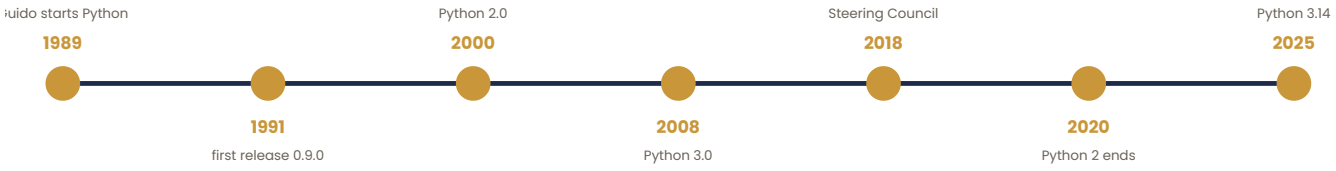
NOTE

You never write bytecode yourself; the `dis` module simply lets you see it. Instruction names vary slightly between Python versions, which is why `.pyc` files are specific to one version.

1.2 History of Python

1.2.1 Guido van Rossum, started in 1989 and first released in 1991

Python was created by **Guido van Rossum**, a Dutch programmer at the Centrum Wiskunde & Informatica (CWI) in Amsterdam. He began it as a hobby project over the Christmas holidays of **December 1989**, wanting a language that was as easy to read as the teaching language ABC but practical for real system tasks. The first public release, version 0.9.0, came in **February 1991**. Python 1.0 followed in 1994, Python 2.0 in 2000 (list comprehensions, garbage collection of cycles) and Python 3.0 in 2008. Guido led the language as its "Benevolent Dictator For Life" until 2018, when he stepped down and a five-person elected **Steering Council** took over.



MILESTONES IN THE HISTORY OF PYTHON

1.2.2 Named after "Monty Python's Flying Circus"

The name has nothing to do with snakes. Guido was reading the scripts of **Monty Python's Flying Circus**, a British comedy series from the 1970s, and wanted a short, slightly mysterious name. The humour lives on in the community: the documentation uses `spam` and `eggs` as example names (from a famous sketch) instead of the usual `foo` and `bar`, and the IDLE editor is named after Monty Python member Eric Idle. The snake logo came later.

1.2.3 Python 2 vs Python 3 and the end of Python 2 (2020)

Python 3 (2008) deliberately broke backward compatibility to fix long-standing design problems. `print` became a function; text became Unicode by default with a clear split between `str` (text) and `bytes` (binary data); `/` between integers became true division; many functions return lazy views and iterators instead of lists; and old syntax was removed. Because old code had to be changed, migration took more than a decade, and both versions coexisted. **Python 2 reached its end of life on 1 January 2020**: it receives no more fixes, even for security problems. All new code must be Python 3, and everything in these notes is Python 3.

Feature	Python 2	Python 3
print	`print "Hi"` (a statement)	`print("Hi")` (a function)
Integer division	`7 / 2` gives 3	`7 / 2` gives 3.5; `7 // 2` gives 3
Text	ASCII `str`, separate `unicode`	Unicode `str`, separate `bytes`
input	`raw_input()`; `input()` evaluated code	`input()` always returns a string
range	`range()` built a list; `xrange()` was lazy	`range()` is lazy
Status	end of life since 2020	actively developed

1.2.4 Annual release cycle (a new 3.x version every October)

Since Python 3.9 (2020), a new feature version is released every year in **October** (PEP 602): 3.12 in October 2023, 3.13 in October 2024, 3.14 in October 2025, and so on. Each version receives about two years of bug-fix releases (3.14.1, 3.14.2…) followed by three years of security fixes, five years in total. Recent versions have brought major improvements: better error messages (3.10-3.13), large speed-ups (3.11 was 10-60% faster than 3.10), a new interactive shell with colours and multi-line editing (3.13), an optional free-threaded build without the GIL (3.13, officially supported in 3.14), and template strings (3.14).

Version	Released	Highlights
3.10	Oct 2021	structural pattern matching (match-case), better error messages
3.11	Oct 2022	much faster, exception groups, fine-grained error locations
3.12	Oct 2023	type parameter syntax, more flexible f-strings
3.13	Oct 2024	new interactive shell, experimental free-threading and JIT
3.14	Oct 2025	template strings (t-strings), deferred annotations, free-threading supported, multiple interpreters
3.15	Oct 2026	the next yearly release

1.2.5 The Python Software Foundation and PEPs (Python Enhancement Proposals)

The **Python Software Foundation** (PSF), a non-profit organisation founded in 2001, owns the rights to Python, manages the open-source licence, runs the Python Package Index (PyPI) and the PyCon US conference, and funds community work. Changes to the language are proposed and discussed through **PEPs**, Python Enhancement Proposals: numbered design documents that describe a new feature or process, its motivation and its specification. The Steering Council accepts or rejects them. Famous examples: **PEP 8** (the style guide), **PEP 20** (the Zen of Python), **PEP 257** (docstring conventions), **PEP 484**

(type hints), **PEP 572** (the walrus operator), **PEP 634** (pattern matching), **PEP 703** (making the GIL optional) and **PEP 750** (template strings).

1.2.6 The Zen of Python (import this)

Tim Peters summarised Python's design philosophy in 19 short aphorisms, the **Zen of Python** (PEP 20). They are built into the interpreter as an Easter egg: `import this` prints them. They guide how Pythonic code is written: readable, explicit and simple. "There should be one obvious way to do it" is a direct contrast with languages that offer many equivalent styles.

THE ZEN OF PYTHON

main.py

```
import this
```

OUTPUT

```
The Zen of Python, by Tim Peters

Beautiful is better than ugly.
Explicit is better than implicit.
Simple is better than complex.
Complex is better than complicated.
Flat is better than nested.
Sparse is better than dense.
Readability counts.
Special cases aren't special enough to break the rules.
Although practicality beats purity.
Errors should never pass silently.
Unless explicitly silenced.
In the face of ambiguity, refuse the temptation to guess.
There should be one-- and preferably only one --obvious way to do it.
Although that way may not be obvious at first unless you're Dutch.
Now is better than never.
Although never is often better than *right* now.
If the implementation is hard to explain, it's a bad idea.
If the implementation is easy to explain, it may be a good idea.
Namespaces are one honking great idea -- let's do more of those!
```

1.3 Why do we need Python?

1.3.1 Simple, readable syntax close to English

Python code reads almost like English: `if name in students:`, `for line in file:`, `not done`, `x is None`. Blocks are marked by indentation instead of braces, so the visual layout always matches the structure. There are few symbols, no semicolons and no type declarations. This makes Python the most popular first language in universities and schools, and it also matters for professionals: code is read far more often than it is written.

CODE THAT READS LIKE A DESCRIPTION OF THE TASK main.py

```
attendance = {"Asha": 18, "Ravi": 12, "Meera": 20}
minimum = 15

for student, days in attendance.items():
    if days >= minimum:
        print(student, "is eligible for the exam")
    else:
        print(student, "needs", minimum - days, "more days")
```

OUTPUT

```
Asha is eligible for the exam
Ravi needs 3 more days
Meera is eligible for the exam
```

1.3.2 Faster development with less code

Python programs are typically three to five times shorter than equivalent Java or C++ programs. Built-in high-level types (lists, dictionaries, sets), automatic memory management, and powerful one-liners such as comprehensions mean you spend your time on the problem, not on boilerplate. That makes Python ideal for prototypes, start-ups, research and automation, where getting a working result quickly matters most.

COUNTING THE MOST COMMON WORDS IN FIVE LINES main.py

```
text = """Python is easy to learn. Python is powerful.
Many companies use Python for data and AI."""

from collections import Counter
words = text.lower().replace(".", "").split()
for word, count in Counter(words).most_common(3):
    print(f"{word:<8} {count}")
```

OUTPUT

```
python   3
is       2
easy     1
```

NOTE

The same word-frequency program in Java needs a class, a main method, a HashMap, explicit loops for counting and sorting, and roughly 25-30 lines.

1.3.3 "Batteries included" standard library

Python ships with a large **standard library**: hundreds of modules available without installing anything. They cover files and folders (`pathlib`, `shutil`), dates (`datetime`), maths and statistics, random numbers, regular expressions (`re`), data formats (`json`, `csv`), databases (`sqlite3`), the internet (`urllib`, `http`, `email`), compression, concurrency, testing (`unittest`) and more. This philosophy is called "batteries included".

SIX STANDARD-LIBRARY MODULES, NOTHING INSTALLED

main.py

```
import datetime, json, math, random, statistics, sqlite3

random.seed(7)
marks = [random.randint(50, 100) for _ in range(5)]
print("marks:", marks, "| mean:", statistics.mean(marks), "| median:", statistics.median(marks))
print("square root of 2:", math.sqrt(2))
print("as JSON:", json.dumps({"student": "Asha", "marks": marks}))
print("days until 31 Dec 2026 from 9 Mar 2026:", (datetime.date(2026, 12, 31) - datetime.date(2026, 3,
9)).days)

db = sqlite3.connect(":memory:")                    # a real SQL database, built in
db.execute("CREATE TABLE s (name TEXT, marks INT)")
db.executemany("INSERT INTO s VALUES (?, ?)", [("Asha", 91), ("Ravi", 78)])
print("from SQLite:", db.execute("SELECT name FROM s WHERE marks > 80").fetchall())
```

OUTPUT

```
marks: [70, 59, 75, 91, 53] | mean: 69.6 | median: 70
square root of 2: 1.4142135623730951
as JSON: {"student": "Asha", "marks": [70, 59, 75, 91, 53]}
days until 31 Dec 2026 from 9 Mar 2026: 297
from SQLite: [('Asha',)]
```

1.3.4 Huge ecosystem of third-party packages (PyPI)

Beyond the standard library, the **Python Package Index** (PyPI, pypi.org) hosts more than 600,000 open-source packages, installed with one command: `pip install requests`. Whatever you need, someone has probably published a well-tested package for it: web frameworks (Django, Flask, FastAPI), data (NumPy, pandas, Polars), visualisation (Matplotlib, Plotly), machine learning (scikit-learn, PyTorch, TensorFlow), automation (Selenium, Playwright), images (Pillow, OpenCV) and thousands more.

TWO FAMOUS THIRD-PARTY PACKAGES: NUMPY AND PANDAS

main.py

```
import numpy as np
import pandas as pd

scores = pd.DataFrame({
    "student": ["Asha", "Ravi", "Meera", "John"],
    "maths": [91, 64, 85, 77],
    "science": [88, 72, 90, 69],
})
scores["average"] = scores[["maths", "science"]].mean(axis=1)
print(scores.sort_values("average", ascending=False).to_string(index=False))
print("correlation between subjects:", np.corrcoef(scores["maths"], scores["science"])[0, 1].round(2))
```

OUTPUT

```
student  maths  science  average
   Asha     91       88     89.5
  Meera     85       90     87.5
   John     77       69     73.0
   Ravi     64       72     68.0
correlation between subjects: 0.79
```

1.3.5 Cross-platform

The same Python program runs unchanged on Windows, macOS and Linux, and also on Raspberry Pi boards, servers and in the cloud, as long as it avoids system-specific features. The standard library hides

operating-system differences (for example, `pathlib` builds file paths with the right separator), and the `sys` and `platform` modules let a program detect where it is running when it needs to.

ONE PROGRAM, ANY OPERATING SYSTEM main.py

```
import os, platform, sys
from pathlib import Path

print("operating system:", platform.system())
print("Python version:", ".".join(map(str, sys.version_info[:2])))
print("path separator here:", repr(os.sep))
report = Path("reports") / "2026" / "march.csv"       # works on every OS
print("portable path:", report, "| suffix:", report.suffix, "| parent:", report.parent)
```

OUTPUT

```
operating system: Linux
Python version: 3.14
path separator here: '/'
portable path: reports/2026/march.csv | suffix: .csv | parent: reports/2026
```

1.3.6 Large community and job demand

Python has topped popularity rankings such as the TIOBE index and the IEEE Spectrum ranking for several years, and is the most used language on GitHub. A huge community means answers to almost any question on Stack Overflow, free courses and books, conferences (PyCon India, PyCon US, local meetups), and packages for every task. Companies such as Google, Netflix, Instagram, Spotify, Dropbox, NASA, Reddit and, in India, Swiggy, Zomato, Flipkart and Razorpay use Python extensively, and job roles from Python developer and data analyst to machine-learning engineer and automation tester list it as a core skill.

1.3.7 First choice for data science, AI and automation

Python became the standard language of **data science** and **artificial intelligence** because researchers could write simple code on top of very fast libraries written in C, C++ and CUDA (NumPy, pandas, PyTorch, TensorFlow). Today almost every machine-learning course, research paper and AI framework, including the tools behind large language models, uses Python. It is equally dominant in **automation**: renaming thousands of files, filling spreadsheets, sending reports by email, scraping websites and controlling browsers, often in a few lines.

A TASTE OF AUTOMATION main.py

```
# Automation: rename messy file names to a clean, consistent pattern (shown as a dry run)
files = ["Invoice March FINAL.PDF", "invoice-april (2).pdf", "INVOICE_may.Pdf"]
months = ["march", "april", "may"]

for old, month in zip(files, months):
    new = f"invoice_2026_{month}.pdf"
    print(f"{old:<28} -> {new}")
```

OUTPUT

```
Invoice March FINAL.PDF      -> invoice_2026_march.pdf
invoice-april (2).pdf        -> invoice_2026_april.pdf
INVOICE_may.Pdf              -> invoice_2026_may.pdf
```

1.4 Features of Python

1.4.1 Interpreted

As described in 1.1.4, Python source code is run directly by the interpreter (after automatic compilation to bytecode). There is no build step for you: change a line, run again. You can also type code into the interactive shell and see results instantly, which makes experimenting and learning very fast. Errors in one part of a program are only discovered when that part runs (syntax errors excepted), so testing matters.

1.4.2 Dynamically typed

Variables have no fixed type; objects do (1.1.2). You can write functions that work with any object that supports the operations they use, an idea called **duck typing** ("if it walks like a duck and quacks like a duck, it is a duck").

DUCK TYPING: ONE FUNCTION, MANY TYPES main.py

```
def describe(collection):
    return f"{type(collection).__name__} with {len(collection)} items"

print(describe([1, 2, 3]))            # works with anything that has a length
print(describe("Python"))
print(describe({"a": 1, "b": 2}))
print(describe(range(10)))
```

OUTPUT

```
list with 3 items
str with 6 items
dict with 2 items
range with 10 items
```

1.4.3 Object-oriented (everything is an object)

In Python, **everything is an object**: numbers, strings, lists, functions, classes, modules and even types themselves. Every object has an identity (`id()`), a type (`type()`) and a value, and most have methods. You do not have to write classes to use Python, but objects are everywhere.

EVERYTHING IS AN OBJECT main.py

```
values = [42, 3.14, "hello", [1, 2], len, print, int, None]
for v in values:
    print(f"{repr(v):<26} type: {type(v).__name__}")

print("even an int has methods:", (255).bit_length(), (10).to_bytes(2, "big"))
print("functions have attributes:", len.__name__, "| a type is an object too:", type(int))
```

OUTPUT

```
42                         type: int
3.14                       type: float
'hello'                    type: str
[1, 2]                     type: list
<built-in function len>    type: builtin_function_or_method
<built-in function print>  type: builtin_function_or_method
<class 'int'>              type: type
None                       type: NoneType
even an int has methods: 8 b'\x00\n'
functions have attributes: len | a type is an object too: <class 'type'>
```

1.4.4 Automatic memory management

You never allocate or free memory yourself, as in C. When you create an object, Python allocates memory for it; when no name refers to it any more, Python frees it automatically. CPython uses **reference counting** (each object counts how many references point to it and is freed when the count reaches zero), plus a **garbage collector** that finds groups of objects that refer only to each other (cycles). Module 21 looks at this in detail.

REFERENCE COUNTING AND THE GARBAGE COLLECTOR main.py

```
import sys, gc

data = [1, 2, 3]
alias = data
print("references to the list:", sys.getrefcount(data) - 1)   # -1 for getrefcount's own argument
del alias
print("after del alias:", sys.getrefcount(data) - 1)

class Node:
    pass
a, b = Node(), Node()
a.partner, b.partner = b, a            # a reference cycle
del a, b
print("cycle objects collected by gc:", gc.collect() >= 2)
```

OUTPUT

```
references to the list: 2
after del alias: 1
cycle objects collected by gc: True
```

1.4.5 Extensible with C/C++

Performance-critical parts of a program can be written in C, C++ or Rust and called from Python as if they were normal Python modules. This is how NumPy, pandas, PyTorch and many other fast libraries work: Python provides the friendly interface, compiled code does the heavy lifting. Tools such as Cython, pybind11, PyO3 (Rust) and the built-in `ctypes` module make this possible.

CALLING C FUNCTIONS DIRECTLY FROM PYTHON main.py

```
import ctypes, ctypes.util

libc = ctypes.CDLL(ctypes.util.find_library("c"))     # load the C standard library
print("C strlen('Learn Being Forward') =", libc.strlen(b"Learn Being Forward"))
print("C abs(-42) =", libc.abs(-42))
```

OUTPUT

```
C strlen('Learn Being Forward') = 19
C abs(-42) = 42
```

1.4.6 Embeddable

The reverse also works: the Python interpreter can be **embedded** inside programs written in other languages, so users can write scripts or plug-ins for them in Python. Blender (3D graphics), GIMP, Inkscape, LibreOffice, Autodesk Maya, many games and scientific instruments embed Python as their scripting language.

1.4.7 Indentation-based blocks

Python uses **indentation** (leading spaces) to group statements into blocks, where C, Java and JavaScript use braces `{ }`. A colon `:` starts a block, and every line in the block must be indented by the same amount, conventionally **four spaces**. This forces neatly formatted code: what you see is exactly the structure Python executes. Module 2 covers the rules and the errors you get when indentation is wrong.

INDENTATION DEFINES THE BLOCKS main.py

```
for n in range(1, 6):
    if n % 2 == 0:
        print(n, "is even")      # inside the if, inside the for
    else:
        print(n, "is odd")
print("done")                    # not indented: runs once, after the loop
```

OUTPUT

```
1 is odd
2 is even
3 is odd
4 is even
5 is odd
done
```

Feature	What it means for you
Interpreted	run immediately, experiment in the shell
Dynamically typed	no declarations; types checked at run time
Strongly typed	no surprising automatic conversions
Everything is an object	consistent model; methods on every value
Automatic memory management	no manual allocation; fewer crashes and leaks
Extensible and embeddable	fast C/C++ libraries; Python inside other apps
Indentation-based	readable, consistently formatted code
Large standard library + PyPI	solutions for almost every task
Free and open source	PSF licence, usable commercially

1.5 Where is Python used?

Python is used in almost every area of computing. The examples below are tiny, but each uses the same libraries that professionals use in that field.

1.5.1 Data science and analytics (pandas, NumPy)

Analysts load data from spreadsheets, databases and APIs, clean it, compute statistics and find patterns with **pandas** (tables called DataFrames) and **NumPy** (fast numeric arrays), usually in Jupyter notebooks. Module 24 covers them.

SUMMARISING SALES DATA WITH PANDAS main.py

```
import pandas as pd

sales = pd.DataFrame({
    "city": ["Mysuru", "Pune", "Mysuru", "Delhi", "Pune", "Delhi"],
    "month": ["Jan", "Jan", "Feb", "Feb", "Feb", "Jan"],
    "amount": [12000, 18500, 15500, 22000, 17250, 9800],
})
print(sales.groupby("city")["amount"].agg(["sum", "mean", "count"]))
print("best month:", sales.groupby("month")["amount"].sum().idxmax())
```

OUTPUT

```
          sum     mean  count
city
Delhi   31800  15900.0      2
Mysuru  27500  13750.0      2
Pune    35750  17875.0      2
best month: Feb
```

1.5.2 Machine learning and AI (scikit-learn, TensorFlow, PyTorch)

Machine learning finds patterns in data and makes predictions. **scikit-learn** covers classic algorithms (regression, decision trees, clustering); **TensorFlow**/Keras and **PyTorch** build deep neural networks for images, speech and language; Hugging Face Transformers provides ready-made language models.

A TINY PREDICTION MODEL WITH SCIKIT-LEARN main.py

```
from sklearn.linear_model import LinearRegression

hours_studied = [[1], [2], [3], [4], [5], [6]]
exam_marks = [35, 45, 52, 61, 70, 78]

model = LinearRegression().fit(hours_studied, exam_marks)
for hours in (2.5, 7):
    print(f"predicted marks for {hours} hours of study: {model.predict([[hours]])[0]:.1f}")
print(f"each extra hour adds about {model.coef_[0]:.1f} marks")
```

OUTPUT

```
predicted marks for 2.5 hours of study: 48.3
predicted marks for 7 hours of study: 86.7
each extra hour adds about 8.5 marks
```

1.5.3 Web development (Django, Flask, FastAPI)

Python powers the back end of many websites and APIs: **Django** (a full framework with an admin panel and ORM; Instagram and Pinterest), **Flask** (a lightweight framework) and **FastAPI** (modern, fast, with automatic API documentation). Module 23 builds web applications.

A FLASK WEB API, TESTED WITHOUT A SERVER

main.py

```
from flask import Flask, jsonify

app = Flask(__name__)
COURSES = [{"id": 1, "title": "Python"}, {"id": 2, "title": "Data Science"}]

@app.get("/api/courses")
def list_courses():
    return jsonify(COURSES)

# Flask's test client sends a request without starting a real server
response = app.test_client().get("/api/courses")
print(response.status_code, response.get_json())
```

OUTPUT

```
200 [{'id': 1, 'title': 'Python'}, {'id': 2, 'title': 'Data Science'}]
```

1.5.4 Automation and scripting

System administrators, testers and office workers use Python scripts to automate repetitive tasks: organising files, processing Excel sheets, generating reports, sending emails, backing up data and calling APIs. The book *Automate the Boring Stuff with Python* made this use famous.

SORTING FILES INTO FOLDERS BY EXTENSION

main.py

```
from pathlib import Path
import tempfile

folder = Path(tempfile.mkdtemp())
for name in ["report.pdf", "photo.jpg", "notes.txt", "scan.pdf", "logo.png"]:
    (folder / name).touch()

# organise files into sub-folders by type
for file in folder.iterdir():
    target = folder / file.suffix[1:].upper()
    target.mkdir(exist_ok=True)
    file.rename(target / file.name)

for sub in sorted(folder.iterdir()):
    print(sub.name, "->", sorted(f.name for f in sub.iterdir()))
```

OUTPUT

```
JPG -> ['photo.jpg']
PDF -> ['report.pdf', 'scan.pdf']
PNG -> ['logo.png']
TXT -> ['notes.txt']
```

1.5.5 Web scraping

Scraping extracts information from web pages automatically: prices, news headlines, job listings, research data. **requests** downloads pages, **Beautiful Soup** and lxml parse the HTML, and Scrapy, Selenium or Playwright handle large crawls and JavaScript-heavy sites. Always respect a site's terms of service and robots.txt.

EXTRACTING DATA FROM HTML WITH BEAUTIFUL SOUP

main.py

```
from bs4 import BeautifulSoup

html = """
<ul id="courses">
  <li class="course"><span class="name">Python</span> <span class="fee">14,999</span></li>
  <li class="course"><span class="name">Java</span> <span class="fee">16,499</span></li>
  <li class="course"><span class="name">SQL</span> <span class="fee">6,999</span></li>
</ul>"""
soup = BeautifulSoup(html, "html.parser")          # in real scraping, html comes from
requests.get(url).text
for li in soup.select("li.course"):
    print(li.select_one(".name").text, "costs Rs", li.select_one(".fee").text)
```

OUTPUT

```
Python costs Rs 14,999
Java costs Rs 16,499
SQL costs Rs 6,999
```

1.5.6 DevOps and cloud tooling

Infrastructure and cloud engineers use Python for deployment scripts and tools: **Ansible** (configuration management) is written in Python, the AWS SDK **boto3**, Google Cloud and Azure libraries automate cloud resources, and many monitoring, CI/CD and container tools offer Python APIs.

1.5.7 Testing and QA automation

Quality-assurance teams write automated tests with **pytest** and the built-in **unittest**, drive browsers with **Selenium** and **Playwright**, and test APIs with requests. Robot Framework, a keyword-driven test tool popular in Indian IT companies, is built on Python. Module 22 covers testing.

AUTOMATED TESTS WITH UNITTEST

main.py

```
import unittest

def discount(price, percent):
    if not 0 <= percent <= 100:
        raise ValueError("percent must be 0-100")
    return round(price * (1 - percent / 100), 2)

class TestDiscount(unittest.TestCase):
    def test_normal(self):
        self.assertEqual(discount(1000, 10), 900)
    def test_zero(self):
        self.assertEqual(discount(499, 0), 499)
    def test_invalid(self):
        with self.assertRaises(ValueError):
            discount(100, 150)

unittest.main(argv=["tests"], verbosity=2, exit=False)
```

OUTPUT

```
test_invalid (__main__.TestDiscount.test_invalid) ... ok
test_normal (__main__.TestDiscount.test_normal) ... ok
test_zero (__main__.TestDiscount.test_zero) ... ok

----------------------------------------------------------------------
Ran 3 tests in 0.009s

OK
```

1.5.8 Desktop apps, games (Pygame) and IoT (MicroPython, Raspberry Pi)

Python also builds **desktop applications** (Tkinter, included with Python; PyQt; Kivy), **games** (Pygame, Arcade, and scripting inside game engines), and **IoT and electronics** projects: the Raspberry Pi was designed around Python, and **MicroPython** and CircuitPython run on tiny microcontrollers such as the ESP32 to read sensors and control LEDs and motors.

Area	Key libraries and tools	Used by
Data science	pandas, NumPy, Jupyter, Polars	analysts at banks, e-commerce, consultancies
Machine learning / AI	scikit-learn, PyTorch, TensorFlow, Hugging Face	Google, Meta, OpenAI, Netflix recommendations
Web back ends	Django, Flask, FastAPI	Instagram, Pinterest, Spotify, Swiggy
Automation	pathlib, openpyxl, smtplib, schedule	IT teams, offices everywhere
Web scraping	requests, Beautiful Soup, Scrapy, Playwright	price comparison, research
DevOps / cloud	Ansible, boto3, Fabric	AWS, infrastructure teams
Testing	pytest, unittest, Selenium, Robot Framework	QA teams
Desktop / games / IoT	Tkinter, PyQt, Pygame, MicroPython	hobbyists, education, embedded devices

1.6 Python implementations

"Python" is a **language specification**; an **implementation** is a program that runs Python code. Several exist, each with different strengths.

1.6.1 CPython (the standard)

CPython is the reference implementation, written in C and maintained by the core developers. It is what you download from python.org and what almost everyone uses; "Python" normally means CPython. New language features appear in CPython first, and the C extension libraries (NumPy, pandas) are built for it.

WHICH IMPLEMENTATION IS RUNNING THIS CODE?

main.py

```
import platform, sys
print("implementation:", platform.python_implementation())
print("version:", ".".join(map(str, sys.version_info[:2])))
print("compiled with:", platform.python_compiler().split()[0])
```

OUTPUT

```
implementation: CPython
version: 3.14
compiled with: Clang
```

1.6.2 PyPy, Jython, IronPython, MicroPython

Implementation	Written in / runs on	Strengths
CPython	C	the standard; best library compatibility
PyPy	RPython, with a JIT compiler	often 4-10x faster for long-running pure-Python code
Jython	Java, runs on the JVM	uses Java libraries directly (Python 2.7 only, slow development)
IronPython	C#, runs on .NET	uses .NET libraries
MicroPython / CircuitPython	C, for microcontrollers	runs in kilobytes of memory on ESP32, Raspberry Pi Pico
GraalPy	Java (GraalVM)	JIT-compiled Python on the JVM with Java interop
Pyodide	CPython compiled to WebAssembly	Python in the web browser

1.7 Limitations of Python

1.7.1 Slower than compiled languages

Pure Python code typically runs 10 to 100 times slower than equivalent C, C++, Rust or Java code, because the interpreter executes bytecode instruction by instruction and every value is a full object with type checks at run time. For most programs this does not matter: they spend their time waiting for networks, disks or users, or they call fast libraries written in C (NumPy is close to C speed). For CPU-heavy loops, the usual remedies are using such libraries, PyPy, Cython or Numba, or rewriting the hot spot in a compiled language.

PURE PYTHON VERSUS COMPILED NUMPY CODE

main.py

```
import time
import numpy as np

numbers = list(range(1_000_000))
t = time.perf_counter()
total_py = sum(n * n for n in numbers)                    # pure Python loop
py_time = time.perf_counter() - t

arr = np.arange(1_000_000, dtype=np.int64)
t = time.perf_counter()
total_np = int((arr * arr).sum())                         # loop runs in compiled C
np_time = time.perf_counter() - t

print("same result:", total_py == total_np)
print(f"NumPy was faster: {np_time < py_time} (roughly {py_time / np_time:.0f}x on this machine)")
```

OUTPUT

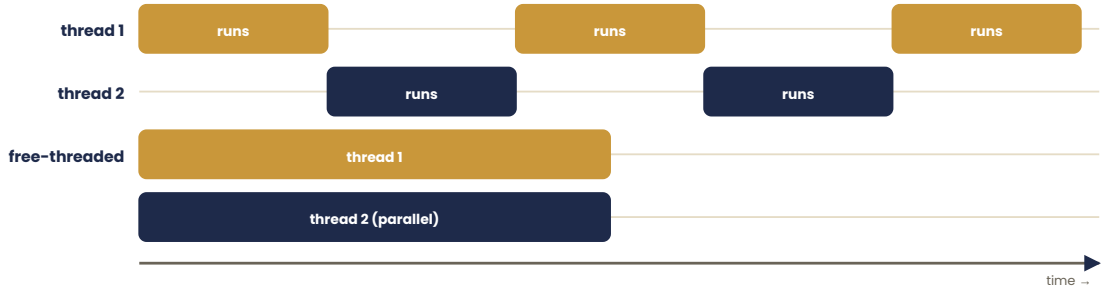
```
same result: True
NumPy was faster: True (roughly 8x on this machine)
```

NOTE

The exact speed-up varies by machine; the point is that heavy number crunching belongs in vectorised libraries, not in Python loops.

1.7.2 The Global Interpreter Lock (GIL) and the free-threaded build

Standard CPython has a **Global Interpreter Lock** (GIL): a lock that allows only one thread to execute Python bytecode at a time, even on a multi-core processor. It keeps the interpreter's memory management simple and safe, but it means that threads cannot speed up CPU-bound Python code (they still work well for waiting on I/O). The traditional workarounds are multiprocessing (separate processes, each with its own GIL) and C extensions that release the GIL. Python 3.13 introduced an optional **free-threaded build** without the GIL (PEP 703), and Python 3.14 made it officially supported (installed as `python3.14t`), so threads can finally run Python code in parallel. Module 20 explains this in detail.



WITH THE GIL, THREADS TAKE TURNS; THE FREE-THREADED BUILD RUNS THEM IN PARALLEL

1.7.3 Higher memory use

Because everything is an object with a header (type pointer, reference count) and dynamic features, Python data uses more memory than the same data in C. An integer that needs 4 bytes in C takes 28 bytes in CPython, and a list stores pointers to separate objects. For large data sets, specialised structures help: NumPy arrays, the `array` module, `__slots__` in classes, and generators that produce values one at a time instead of storing them.

PYTHON OBJECTS CARRY OVERHEAD

main.py

```
import sys, array

print("an int object:", sys.getsizeof(1), "bytes")
print("a float object:", sys.getsizeof(1.0), "bytes")
print("an empty list:", sys.getsizeof([]), "bytes; an empty dict:", sys.getsizeof({}), "bytes")

million = list(range(1_000_000))
packed = array.array("i", range(1_000_000))              # C-style array of 4-byte ints
list_total = sys.getsizeof(million) + sum(sys.getsizeof(n) for n in million[:1000]) * 1000
print(f"list of 1M ints: about {list_total / 1e6:.0f} MB | array of 1M ints: about
{sys.getsizeof(packed) / 1e6:.0f} MB")
```

OUTPUT

```
an int object: 28 bytes
a float object: 24 bytes
an empty list: 56 bytes; an empty dict: 64 bytes
list of 1M ints: about 36 MB | array of 1M ints: about 4 MB
```

1.7.4 Weak in mobile development

Python is rarely used for mobile apps. Android apps are mainly written in Kotlin or Java, iOS apps in Swift, and cross-platform apps in Flutter (Dart) or React Native (JavaScript). Python options exist (Kivy, BeeWare, and official iOS and Android support added to CPython 3.13), but they are niche. Python is also not used in web browsers (JavaScript runs there), apart from WebAssembly projects such as Pyodide and PyScript.

Limitation	Why	Common remedies
Speed	interpreted, dynamic typing	NumPy, PyPy, Cython, Numba, C extensions
GIL	one thread runs bytecode at a time	multiprocessing, asyncio for I/O, free-threaded 3.14t
Memory	every value is a full object	NumPy arrays, generators, __slots__
Mobile and browser	platforms favour other languages	Kivy/BeeWare; Pyodide; use Python for the back end
Run-time errors	types checked only when code runs	type hints with mypy, tests

1.8 Python compared with other languages

1.8.1 Python vs C/C++

C and C++ are compiled, statically typed and close to the hardware: you manage memory yourself (C) or with careful rules (C++), and programs run extremely fast. They are used for operating systems, embedded systems, game engines, browsers and the internals of Python itself. Python trades raw speed for development speed, safety and simplicity. The two complement each other: Python as the friendly front end, C/C++ for the performance-critical core.

Aspect	C / C++	Python
Execution	compiled to machine code	compiled to bytecode, interpreted (plus JIT)
Typing	static, weak-ish (C) / static (C++)	dynamic, strong
Memory	manual (malloc/free, new/delete)	automatic (reference counting + GC)

Aspect	C / C++	Python
Speed	very fast	10-100x slower for pure Python loops
Code length	long: headers, types, braces	short and readable
Hello World	`#include <stdio.h>` + `main()` + `printf`	`print("Hello")`
Typical use	systems, embedded, games, performance	data, AI, web, automation, scripting

1.8.2 Python vs Java

Java is compiled to JVM bytecode, statically typed and strictly object-oriented (every function lives in a class). It is very popular for large enterprise systems, banking back ends and Android. Java is faster than CPython and catches type errors before running; Python is far more concise, quicker to write, and dominant in data science and AI.

Aspect	Java	Python
Typing	static: `int count = 0;`	dynamic: `count = 0`
Structure	everything inside classes; `public static void main`	scripts, functions and classes as needed
Blocks	braces and semicolons	indentation
Speed	faster (JIT-compiled JVM)	slower, but fast libraries
Strengths	enterprise, Android, large teams	data, AI, scripting, rapid development

1.8.3 Python vs JavaScript

JavaScript is the language of web browsers and, with Node.js, also of servers. Both are dynamic, interpreted (JIT-compiled) and multi-paradigm. JavaScript is weakly typed (implicit coercions) and built around an asynchronous event loop; Python is strongly typed and synchronous by default, with `asyncio` for asynchronous code. For a web application, a common split is JavaScript (React) in the browser and Python (Django, FastAPI) on the server, or the same team using JavaScript on both sides.

Aspect	JavaScript	Python
Runs in browsers	yes, natively	no (only via WebAssembly)
Typing	dynamic, weak: `"5" + 2` is `"52"`	dynamic, strong: TypeError
Blocks	braces	indentation
Concurrency model	event loop, async by default	synchronous by default; threads, processes, asyncio
Main strengths	front end, full-stack web, real-time apps	data, AI, back end, automation, science

The same task, the sum of the squares of the even numbers from 1 to 10, in four languages:

Language	Code
C	`int s = 0; for (int i = 1; i <= 10; i++) if (i % 2 == 0) s += i * i;`
Java	`int s = IntStream.rangeClosed(1, 10).filter(i -> i % 2 == 0).map(i -> i * i).sum();`

Language	Code
JavaScript	`const s = [...Array(10).keys()].map(i => i + 1).filter(i => i % 2 === 0).reduce((a, i) => a + i * i, 0);`
Python	`s = sum(i * i for i in range(1, 11) if i % 2 == 0)`

THE PYTHON VERSION, RUN

main.py

```
s = sum(i * i for i in range(1, 11) if i % 2 == 0)
print("sum of squares of even numbers 1-10:", s)
```

OUTPUT

```
sum of squares of even numbers 1-10: 220
```

Python is a high-level, interpreted, general-purpose language that is dynamically typed (objects, not names, have types) and strongly typed (no silent conversions), and supports procedural, object-oriented and functional styles. CPython compiles source to bytecode (cached in .pyc files) and runs it on the Python Virtual Machine. Guido van Rossum started it in 1989 (first release 1991), naming it after Monty Python; Python 3 replaced Python 2, which ended in 2020; a new 3.x arrives every October; the PSF and PEPs govern it; import this shows the Zen of Python. Its readable syntax, short code, batteries-included standard library, huge PyPI ecosystem, cross-platform support and community make it the first choice for data science, AI and automation, and it is used for web back ends, scraping, DevOps, testing, desktop apps, games and IoT. Implementations include CPython, PyPy, Jython, IronPython and MicroPython. Its limitations are speed, the GIL (now optional in the free-threaded build), memory use and weak mobile support.

PRACTICE CHECKPOINT

Explain in your own words what **"interpreted"** and **"dynamically typed"** mean, and list **five areas** where Python is used with **one library** for each.

SAMPLE SOLUTION

main.py

```
# Sample answer, printed as a small report
answers = {
    "interpreted": "The Python interpreter runs my source file directly: it compiles it to bytecode "
                   "automatically and executes it, so there is no separate compile step for me.",
    "dynamically typed": "I do not declare types; each object knows its own type, a variable can later
"
                         "refer to an object of another type, and types are checked while the program
runs.",
}
for term, meaning in answers.items():
    print(f"{term.upper()}:\n  {meaning}\n")

areas = [
    ("Data analysis", "pandas"),
    ("Machine learning", "scikit-learn"),
    ("Web development", "Django"),
    ("Web scraping", "Beautiful Soup"),
    ("Test automation", "pytest"),
]
print(f"{'Area':<18}Library")
for area, library in areas:
    print(f"{area:<18}{library}")

# proof of "dynamically typed": the same name, three types
value = 10; print(type(value).__name__, end=" -> ")
value = "ten"; print(type(value).__name__, end=" -> ")
value = [10]; print(type(value).__name__)
```

OUTPUT

```
INTERPRETED:
  The Python interpreter runs my source file directly: it compiles it to bytecode automatically and
executes it, so there is no separate compile step for me.

DYNAMICALLY TYPED:
  I do not declare types; each object knows its own type, a variable can later refer to an object of
another type, and types are checked while the program runs.

Area              Library
Data analysis     pandas
Machine learning  scikit-learn
Web development   Django
Web scraping      Beautiful Soup
Test automation   pytest
int -> str -> list
```

MODULE 02 · BEGINNER

Setup and your first program

Minimum duration **1 day** | Level **Beginner**

By the end of this module you will have Python and an editor installed and will be able to write and run Python code in several ways: in the interactive shell, as script files, in IDLE and in notebooks. You will also learn the basic syntax rules (indentation, statements, keywords and names), how to write comments and docstrings, and how to read the errors Python reports.

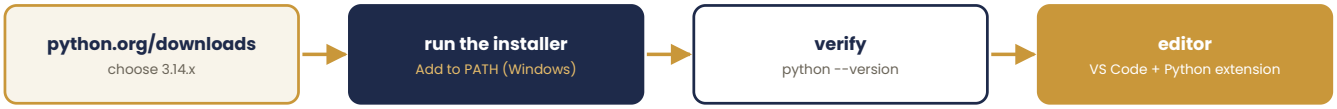
IN THIS MODULE

2.1 Installing Python
2.2 Ways to run Python
2.3 Editors and IDEs
2.4 Hello World
2.5 Basic syntax
2.6 Comments and docstrings
2.7 Errors

2.1 Installing Python

2.1.1 Downloading the latest stable 3.x release from python.org

Download Python from the official website, **python.org/downloads**. The page detects your operating system and offers the latest stable release (Python 3.14.x at the time of writing). Avoid downloading Python from unofficial sites. Choose a stable release, not one marked alpha, beta or release candidate; and if a project or course requires a specific version, the "All releases" list has older ones. Python is free and open source, also for commercial use.



GETTING STARTED IN FOUR STEPS

2.1.2 Installation on Windows (Add to PATH), macOS and Linux

Windows: run the downloaded installer (`python-3.14.x-amd64.exe`). On the first screen, tick **"Add python.exe to PATH"**, which lets you type `python` in any terminal, then click "Install Now". At the end, click "Disable path length limit" if offered. Python is also available from the Microsoft Store, and the new **Python install manager** (`pymanager`) can install and switch versions. **macOS**: run the `.pkg` installer from python.org (or use Homebrew: `brew install python@3.14`); afterwards, run the "Install Certificates" script in the Python folder so that HTTPS downloads work. The command is `python3`, because macOS reserves `python`. **Linux**: most distributions include Python 3 already; install a newer version with the package manager (`sudo apt install python3.14` via the deadsnakes PPA on Ubuntu, `sudo dnf install python3.14` on Fedora) or with a version manager such as `uv` or `pyenv`. Never remove or replace the system Python on Linux, because the operating system depends on it.

OS	How to install	Command to run
Windows	python.org installer, tick "Add python.exe to PATH" (or Microsoft Store / pymanager)	`python` or `py`
macOS	python.org .pkg installer or `brew install python@3.14`	`python3`
Linux	usually pre-installed; `apt` / `dnf` / `uv python install 3.14`	`python3`

WATCH OUT

The most common Windows problem is forgetting to tick "Add python.exe to PATH": typing `python` then opens the Microsoft Store or says "not recognized". Re-run the installer, choose "Modify", and enable "Add Python to environment variables", or use the `py` launcher instead.

2.1.3 Verifying: python --version / python3 --version

Open a terminal (Command Prompt or PowerShell on Windows, Terminal on macOS and Linux) and check the installed version. On Windows the command is usually `python`; on macOS and Linux it is `python3`. pip, the package installer, comes with Python; check it with `pip --version` (or `python -m pip --version`, which guarantees you use the pip of that exact Python).

CHECKING THE INSTALLATION main.py

```
python --version          # Windows
python3 --version         # macOS / Linux
python -m pip --version
```

TYPICAL OUTPUT

```
Python 3.14.0
Python 3.14.0
pip 25.2 from C:\Users\asha\AppData\Local\Programs\Python\Python314\Lib\site-packages\pip (python
3.14)
```

ASKING PYTHON ABOUT ITSELF main.py

```
python -c "import sys; print(f'running Python {sys.version_info.major}.{sys.version_info.minor}')"
python -c "import sys; print(sys.executable.endswith(('python', 'python3', 'python3.14')))"
python -m pip --version | cut -d' ' -f1,2
```

TERMINAL

```
$ python -c "import sys; print(f'running Python {sys.version_info.major}.{sys.version_info.minor}')"
running Python 3.14
$ python -c "import sys; print(sys.executable.endswith(('python', 'python3', 'python3.14')))"
True
$ python -m pip --version | cut -d' ' -f1,2
pip 24.3.1
```

2.1.4 Multiple versions and the py launcher

It is normal to have several Python versions installed, for example 3.12 for an old project and 3.14 for new work. On **Windows**, the **py launcher** (installed with Python) chooses between them: `py` runs the newest, `py -3.12` runs a specific one, and `py --list` shows all installed versions. A script can also request a version in its first line (a "shebang" such as `#!/usr/bin/env python3.12`). On macOS and Linux, versions are called by name (`python3.12`, `python3.14`), and tools such as **pyenv** or **uv** install and switch versions per project. Each project should also use its own **virtual environment** (Module 12), so packages for one project never conflict with another.

THE PY LAUNCHER ON WINDOWS

main.py

```
py --list
py -3.12 --version
py -3.14 hello.py
```

TYPICAL OUTPUT

```
 -V:3.14 *        Python 3.14 (64-bit)
 -V:3.12          Python 3.12 (64-bit)
Python 3.12.10
Hello, World!
```

2.2 Ways to run Python

2.2.1 Interactive shell (REPL)

Typing `python` (or `python3`, or `py`) with nothing after it starts the **interactive shell**, also called the **REPL**: it **R**eads a line, **E**valuates it, **P**rints the result and **L**oops. The prompt `>>>` waits for input; `...` means a statement continues on the next line. You do not need `print()` in the REPL: the value of an expression is shown automatically, and `_` holds the last result. The new shell in Python 3.13+ has colours, multi-line editing, history (up arrow) and commands such as `help`, `exit` and `clear`. Leave with `exit()`, `quit()`, Ctrl+D (macOS/Linux) or Ctrl+Z then Enter (Windows). The REPL is perfect for trying things out; programs you want to keep belong in files.

AN INTERACTIVE SESSION

main.py

```
$ python
Python 3.14.0 (main, Oct  7 2025, 10:00:00) [MSC v.1944 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> 2 + 3
5
>>> _ * 10
50
>>> name = "Asha"
>>> f"Hello, {name}"
'Hello, Asha'
>>> for i in range(3):
...     print(i)
...
0
1
2
>>> exit()
```

NOTE

Lines starting with >>> or ... are what you type; the other lines are Python's replies.

THE SAME SESSION, REALLY RUN (INPUT PIPED IN) main.py

```
printf '2 + 3\n_ * 10\nname = "Asha"\nf"Hello, {name}"\nlen(name)\n' | python -q -i 2>&1 | sed 's/>>> //g'
```

TERMINAL

```
$ printf '2 + 3\n_ * 10\nname = "Asha"\nf"Hello, {name}"\nlen(name)\n' | python -q -i 2>&1 | sed
's/>>> //g'
5
50
'Hello, Asha'
4
```

2.2.2 Running script files: python hello.py

Real programs are saved in **script files** with the `.py` extension and run from the terminal with `python filename.py` (from the folder that contains the file, or with its full path). Python runs the file from top to bottom and exits when it reaches the end. Unlike the REPL, a script only shows what you explicitly `print()`. Extra words after the file name are passed to the program as command-line arguments (Module 5). On macOS and Linux a script can be made executable with a shebang line and `chmod +x`.

GREET.PY Python

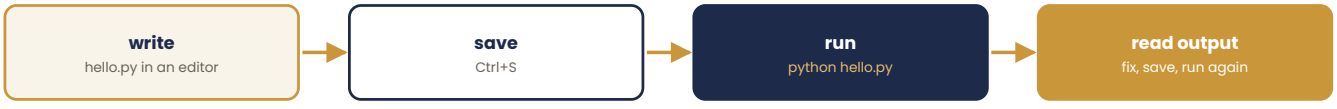
```
import sys
name = sys.argv[1] if len(sys.argv) > 1 else "student"
print(f"Hi {name}, your first script works!")
```

CREATING AND RUNNING SCRIPTS main.py

```
cat hello.py
python hello.py
python greet.py Ravi
chmod +x hello.py && ./hello.py
```

TERMINAL

```
$ cat hello.py
#!/usr/bin/env python3
print("Hello, World!")
print("Welcome to Learn Being Forward")
2 + 3          # computed, but not shown: scripts only display what you print
$ python hello.py
Hello, World!
Welcome to Learn Being Forward
$ python greet.py Ravi
Hi Ravi, your first script works!
$ chmod +x hello.py && ./hello.py
Hello, World!
Welcome to Learn Being Forward
```



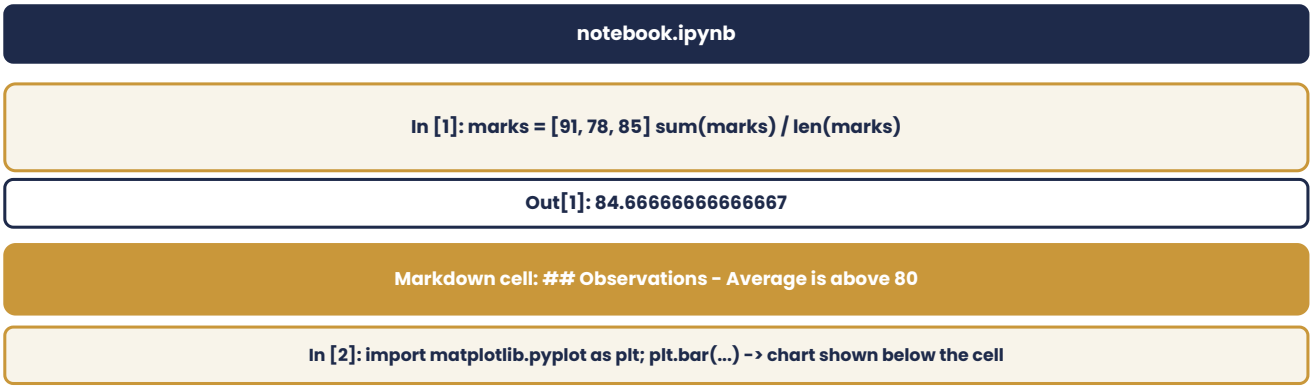
THE EDIT-RUN CYCLE

2.2.3 IDLE

IDLE (Integrated Development and Learning Environment) is a simple editor that is installed with Python on Windows and macOS (on Linux, install `idle3`). Open it from the Start menu or Applications. It starts with a Python shell window; **File → New File** opens an editor with syntax highlighting and automatic indentation, and **Run → Run Module** (F5) saves and runs the file, showing output in the shell. IDLE is fine for first steps, but most developers soon move to VS Code or PyCharm.

2.2.4 Jupyter Notebook and Google Colab

A **notebook** mixes code, its output, charts and formatted text in one document made of **cells**. You run one cell at a time (Shift+Enter), and the variables stay in memory between cells, which is ideal for exploring data step by step, teaching and reports. **Jupyter** (JupyterLab or Jupyter Notebook) runs locally: install it with `pip install jupyterlab` and start it with `jupyter lab`, which opens in the browser; VS Code can also open `.ipynb` files directly. **Google Colab** (colab.research.google.com) is a free online Jupyter service: nothing to install, many data and AI libraries pre-installed, free GPU time, and notebooks saved in Google Drive. Kaggle Notebooks are similar.



A NOTEBOOK: CODE CELLS, OUTPUTS AND TEXT CELLS

Way to run	Best for	How
REPL	quick experiments, checking a function	`python`, then type
Script file	real programs, automation	`python file.py`
IDLE	absolute beginners	installed with Python; F5 to run
Jupyter / Colab	data analysis, teaching, reports	cells, Shift+Enter; Colab in the browser
IDE (VS Code, PyCharm)	projects of any size	run button, debugger, terminal

2.3 Editors and IDEs

2.3.1 VS Code with the Python extension

Visual Studio Code (free, from Microsoft) is the most popular editor for Python. Install it from code.visualstudio.com, open the Extensions panel (Ctrl+Shift+X) and install the official **Python** extension (it brings Pylance for autocompletion and type checking, and the debugger). Open your project folder (File → Open Folder), create a `.py` file, choose the interpreter with **Python: Select Interpreter** in the Command Palette (Ctrl+Shift+P), and run the file with the ▶ button or in the integrated terminal (Ctrl+`). The **Jupyter** extension adds notebook support, and Ruff or Black can format code on save (Module 22).

VS Code feature	Shortcut / where
Run the current file	▶ (top right) or right-click → Run Python File
Integrated terminal	Ctrl+` (backtick)
Command Palette	Ctrl+Shift+P → "Python: Select Interpreter"
Debug with breakpoints	F9 to set a breakpoint, F5 to start debugging
Rename a variable everywhere	F2
Go to definition	F12 or Ctrl+click
Comment / uncomment lines	Ctrl+/

2.3.2 PyCharm

PyCharm, from JetBrains, is a full IDE designed only for Python. Since 2025 it is a single product with a free core (the former Community Edition features) and paid Pro features (web frameworks, databases, remote development, Jupyter), free for students and teachers. It offers excellent code analysis, refactoring, a powerful debugger, test runners, virtual-environment management and version control, at the cost of being heavier than VS Code. Many professional Python teams use it.

2.3.3 Online editors

When you cannot install anything (on a library computer, a tablet, or for a quick demonstration), online editors run Python in the browser: **Google Colab** and **Kaggle** notebooks, **Replit** and **GitHub Codespaces** (full development environments), **Programiz** and **OnlineGDB** (simple editors), and **Python Tutor** (pythontutor.com), which visualises step by step how variables and objects change, excellent for understanding programs. Do not paste passwords or private data into public online tools.

Tool	Type	Highlights
VS Code + Python extension	editor	free, light, huge extension library, notebooks
PyCharm	IDE	deep Python analysis, refactoring, debugger
IDLE	basic editor	ships with Python
Jupyter / Colab / Kaggle	notebooks	data science, free GPUs (Colab)
Replit / Codespaces	online IDE	nothing to install, share a link
Python Tutor	visualiser	see memory and execution step by step

2.4 Hello World

2.4.1 print("Hello, World!")

By tradition, the first program in a new language prints "Hello, World!". In Python it is one line. `print` is a built-in **function**; the parentheses contain its **argument**, here a **string** (text) written between quotes. Single or double quotes both work. Python runs the line and displays the text. `print` can take several arguments and other values, which Module 5 explores.

HELLO WORLD AND A FEW VARIATIONS main.py

```
print("Hello, World!")
print('Single quotes work too')
print("Learn", "Being", "Forward")          # several arguments, separated by spaces
print("2 + 3 =", 2 + 3)                      # expressions are evaluated first
print()                                      # an empty line
print("Done")
```

OUTPUT

```
Hello, World!
Single quotes work too
Learn Being Forward
2 + 3 = 5

Done
```

2.4.2 Comparison with Hello World in C and Java

The same program in C and Java shows how much ceremony Python removes: no header files or imports, no class, no `main` function, no type declarations, no semicolons and no compilation step for you. That is one reason Python is such a popular first language.

HELLO.C C

```c
#include <stdio.h>

int main(void) {
    printf("Hello, World!\n");
    return 0;
}
/* compile: gcc hello.c -o hello    run: ./hello */
```

HELLO.JAVA Java

```java
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
// compile: javac Hello.java    run: java Hello
```

HELLO.PY Python

```
print("Hello, World!")
# run: python hello.py
```

	C	Java	Python
Lines of code	5	5	1
Needs a main function / class	yes / no	yes / yes	no / no
Separate compile step	yes (gcc)	yes (javac)	no
Semicolons and braces	yes	yes	no

2.5 Basic syntax

2.5.1 Indentation instead of braces

A **block** is a group of statements that belong together: the body of an `if`, a loop, a function or a class. The line that opens a block ends with a **colon** `:`, and the statements inside are **indented** further than that line. The block ends when the indentation returns to the previous level. Use **4 spaces** per level (PEP 8); editors insert them when you press Tab. All statements in one block must have exactly the same indentation, and you must not mix tabs and spaces. Indentation is not decoration in Python: it changes what the program does.

BLOCKS MARKED BY INDENTATION main.py

```
marks = 72
if marks >= 40:
    print("Passed")             # inside the if-block
    if marks >= 75:
        print("Distinction")    # nested block: 8 spaces
    print("Well done")          # back to the if-block
print("Result processed")       # outside: always runs
```

OUTPUT

```
Passed
Well done
Result processed
```

ONE LINE MOVED LEFT CHANGES THE LOGIC main.py

```
marks = 30
if marks >= 40:
    print("Passed")
print("Well done")              # moved out of the block: now it runs even for a fail
print("Result processed")
```

OUTPUT

```
Well done
Result processed
```

EACH INDENTATION LEVEL IS A NESTED BLOCK

2.5.2 Statements and line continuation (\ and brackets)

Normally one line holds one **statement**, and the end of the line ends the statement; no semicolon is needed. When a statement is too long (PEP 8 suggests at most 79 characters, many teams allow 88-100), it can continue on the next line in two ways. **Implicit continuation** (preferred): any expression inside parentheses `( )`, square brackets `[ ]` or braces `{ }` may span several lines. **Explicit continuation**: a

backslash `\` at the very end of a line joins it with the next; it is fragile (a space after it breaks it) and generally discouraged.

SPLITTING LONG STATEMENTS OVER SEVERAL LINES main.py

```
total = (1000 + 2500 + 750 +
         1200 + 3100)                    # implicit continuation inside ( )

courses = [
    "Python",
    "Java",
    "Data Science",                      # a trailing comma is allowed
]

fee = 14999 + \
      1800                               # explicit continuation with a backslash

message = ("Python is easy to read "
           "and quick to write.")        # adjacent strings are joined automatically
print(total, courses, fee)
print(message)
```

OUTPUT

```
8550 ['Python', 'Java', 'Data Science'] 16799
Python is easy to read and quick to write.
```

2.5.3 Multiple statements with ;

A semicolon can separate several simple statements on one line: `a = 1; b = 2; print(a + b)`. It works, but PEP 8 discourages it because it hides structure and makes code harder to read and debug. Use one statement per line. A trailing semicolon at the end of a line is simply ignored (in Jupyter it also suppresses the display of a value).

SEMICOLONS ARE ALLOWED BUT DISCOURAGED main.py

```
a = 10; b = 20; print(a + b)             # legal, but not recommended

a = 10
b = 20
print(a + b)                             # preferred: one statement per line
```

OUTPUT

```
30
30
```

2.5.4 Case sensitivity

Python is **case-sensitive**: `name`, `Name` and `NAME` are three different identifiers. Keywords must be written exactly as defined: `True`, `False` and `None` start with a capital letter, while `if`, `for`, `def` and the other keywords are all lower case. Built-in function names are lower case too: `print`, not `Print`.

UPPER AND LOWER CASE MATTER main.py

```
name = "asha"
Name = "Ravi"
NAME = "MEERA"
print(name, Name, NAME)                    # three different variables
is_ready = True                            # True, not true
print(is_ready)
Print("hello")                             # NameError: Print is not print
```

OUTPUT

```
asha Ravi MEERA
True
Traceback (most recent call last):
  File "main.py", line 7, in <module>
    Print("hello")                                # NameError: Print is not print
    ^^^^^
NameError: name 'Print' is not defined. Did you mean: 'print'?
```

2.5.5 Keywords (import keyword; keyword.kwlist)

Keywords are reserved words with a special meaning in the language: `if`, `else`, `for`, `while`, `def`, `class`, `return`, `import` and so on. They cannot be used as variable or function names. Python 3.14 has 35 keywords, listed by the `keyword` module. There are also four **soft keywords** (`match`, `case`, `type` and `_`) that are keywords only in specific places and can still be used as normal names elsewhere.

LISTING THE KEYWORDS main.py

```
import keyword

print("number of keywords:", len(keyword.kwlist))
for i in range(0, len(keyword.kwlist), 7):
    print("  ".join(f"{k:<9}" for k in keyword.kwlist[i:i + 7]))
print("soft keywords:", keyword.softkwlist)
print("is 'class' a keyword?", keyword.iskeyword("class"), "| is 'print'?",
keyword.iskeyword("print"))
```

OUTPUT

```
number of keywords: 35
False      None       True       and        as         assert     async
await      break      class      continue   def        del        elif
else       except     finally    for        from       global     if
import     in         is         lambda     nonlocal   not        or
pass       raise      return     try        while      with       yield
soft keywords: ['_', 'case', 'match', 'type']
is 'class' a keyword? True | is 'print'? False
```

Group	Keywords
Values	`True False None`
Logic and comparison	`and or not is in`
Conditions	`if elif else` (and soft `match case`)
Loops	`for while break continue`
Functions	`def return lambda yield global nonlocal`
Classes	`class`

Group	Keywords
Exceptions	`try except finally raise assert`
Modules	`import from as`
Context managers and misc.	`with pass del`
Asynchronous code	`async await`

2.5.6 Identifiers and naming rules

An **identifier** is a name you give to a variable, function, class or module. The rules: it may contain letters, digits and underscores; it must **not start with a digit**; it must not be a keyword; it cannot contain spaces or symbols such as `-`, `@` or `$`; and it is case-sensitive. Unicode letters are allowed (`naam` or even Hindi or Kannada letters), but English names are the convention. Avoid using names of built-in functions such as `list`, `str`, `sum` or `print` for your variables: it is legal, but it hides the built-in.

Name	Valid?	Reason
`student_name`	yes	letters and underscore
`marks2`	yes	digit not at the start
`_total`	yes	may start with an underscore
`2nd_place`	no	starts with a digit
`first-name`	no	`-` is the minus operator
`class`	no	keyword
`total marks`	no	contains a space
`list`	valid but bad	hides the built-in `list()`

WHY YOU SHOULD NOT REUSE BUILT-IN NAMES

main.py

```
sum = 10 + 20                # legal, but now the built-in sum() is hidden
print(sum)
try:
    print(sum([1, 2, 3]))
except TypeError as e:
    print("TypeError:", e)
del sum                      # remove our variable; the built-in is visible again
print(sum([1, 2, 3]))
```

OUTPUT

```
30
TypeError: 'int' object is not callable
6
```

2.5.7 PEP 8 naming conventions (snake_case, PascalCase, UPPER_CASE)

PEP 8, the official style guide, defines naming conventions that almost every Python project follows, so names alone tell you what kind of thing they refer to. Following them makes your code look professional and easy for others to read.

Convention	Used for	Examples
`snake_case`	variables, functions, methods, modules	`student_name`, `calculate_total()`, `file_utils.py`
`PascalCase` (CapWords)	classes, exceptions, type aliases	`BankAccount`, `ValidationError`
`UPPER_CASE`	constants	`MAX_STUDENTS`, `PI`, `API_URL`
`_single_leading`	"internal" names, by convention private	`_cache`, `_helper()`
`__double_leading`	name-mangled class attributes (Module 15)	`__balance`
`__dunder__`	special methods defined by Python	`__init__`, `__str__`
`trailing_`	avoid clashing with a keyword	`class_`, `type_`

PEP 8 NAMES IN A SMALL PROGRAM

main.py

```
MAX_MARKS = 100                                 # constant: UPPER_CASE

class StudentRecord:                            # class: PascalCase
    def __init__(self, full_name, total_marks):   # method and parameters: snake_case
        self.full_name = full_name
        self.total_marks = total_marks

    def percentage(self):
        return self.total_marks / MAX_MARKS * 100

record = StudentRecord("Asha Rao", 91)          # variable: snake_case
print(f"{record.full_name}: {record.percentage():.1f}%")
```

OUTPUT

```
Asha Rao: 91.0%
```

2.6 Comments and docstrings

2.6.1 Single-line comments

A **comment** starts with `#` and continues to the end of the line; Python ignores it completely. Comments explain **why** code does something, give context, or temporarily disable a line. A comment can occupy a whole line (indented like the code it describes) or follow code on the same line (an **inline comment**, separated by at least two spaces). Good comments explain intent; the code itself should show what happens.

COMMENTS

main.py

```
# Calculate the fee after the early-bird discount
fee = 14999
discount = 0.10        # 10% for enrolling before March
final_fee = fee * (1 - discount)
# print("debug:", final_fee)     <- commented out: Python skips this line
print(f"Final fee: Rs {final_fee:.2f}")
```

OUTPUT

```
Final fee: Rs 13499.10
```

TIP

Write comments for the reader who arrives next month (possibly you). `x = x + 1 # add one to x` adds nothing; `retries += 1 # the API sometimes fails on the first call` explains a decision.

2.6.2 Multi-line comments with triple quotes

Python has no special multi-line comment syntax. Either put `#` at the start of every line (the official recommendation; editors do it with Ctrl+/), or use a **triple-quoted string** `"""..."""` that is not assigned to anything: Python evaluates it and throws it away, so it behaves like a comment. Triple-quoted strings are really string literals, which can span many lines and contain both kinds of quotes.

MULTI-LINE COMMENTS AND MULTI-LINE STRINGS

main.py

```
# This program prints a simple report card.
# Author: Learn Being Forward
# Version: 1.0

"""
This triple-quoted string is not assigned to any variable,
so Python simply ignores it. It is sometimes used as a
multi-line comment.
"""

poem = """Roses are red,
Python is great,
indentation matters,
so don't be late."""          # here the triple quotes create a real multi-line string
print(poem)
```

OUTPUT

```
Roses are red,
Python is great,
indentation matters,
so don't be late.
```

2.6.3 Docstrings and help()

A **docstring** is a string literal written as the **first statement** in a module, function, class or method. Unlike a comment, Python keeps it: it is stored in the object's `__doc__` attribute and shown by `help()`, by editors when you hover over a name, and by documentation tools. By convention (PEP 257) docstrings use triple double quotes; a one-line summary comes first, followed, for longer docstrings, by a blank line and details about arguments, return value and errors. `help()` also works on every built-in function and module.

WRITING A DOCSTRING AND READING IT WITH HELP()

main.py

```
def bmi(weight_kg, height_m):
    """Return the body-mass index rounded to one decimal place.

    Args:
        weight_kg: weight in kilograms.
        height_m: height in metres.

    Raises:
        ValueError: if height_m is not positive.
    """
    if height_m <= 0:
        raise ValueError("height must be positive")
    return round(weight_kg / height_m ** 2, 1)

print(bmi(68, 1.72))
print("first line of the docstring:", bmi.__doc__.splitlines()[0])
help(bmi)
```

OUTPUT

```
23.0
first line of the docstring: Return the body-mass index rounded to one decimal place.
Help on function bmi in module __main__:

bmi(weight_kg, height_m)
    Return the body-mass index rounded to one decimal place.

    Args:
        weight_kg: weight in kilograms.
        height_m: height in metres.

    Raises:
        ValueError: if height_m is not positive.
```

BUILT-IN FUNCTIONS HAVE DOCSTRINGS TOO

main.py

```
print(len.__doc__)
print()
print(str.upper.__doc__)
```

OUTPUT

```
Return the number of items in a container.

Return a copy of the string converted to uppercase.
```

2.7 Errors

Every programmer makes errors; what matters is reading them calmly. Python errors fall into three groups: **syntax errors** (the code is not valid Python, so nothing runs), **run-time errors** (exceptions raised while the program runs) and **logical errors** (the program runs but gives the wrong answer).

2.7.1 SyntaxError and IndentationError

A **SyntaxError** means the code breaks the grammar of Python: a missing colon, an unclosed bracket or quote, a misspelt keyword, using `=` where `==` is needed. Python detects it while compiling the file, before running anything, so not even the first line executes. An **IndentationError** is a kind of SyntaxError caused

by wrong indentation: a block that is not indented, inconsistent indentation, or unexpected indentation. Modern Python versions give helpful messages that point to the exact place and often suggest the fix.

A MISSING COLON

main.py

```
print("This line never runs, because the file cannot be compiled")
marks = 75
if marks > 40
    print("Passed")
```

OUTPUT

```
  File "main.py", line 3
    if marks > 40
                ^
SyntaxError: expected ':'
```

A BLOCK THAT IS NOT INDENTED

main.py

```
marks = 75
if marks > 40:
print("Passed")
```

OUTPUT

```
  File "main.py", line 3
    print("Passed")
    ^^^^^
IndentationError: expected an indented block after 'if' statement on line 2
```

AN UNCLOSED BRACKET

main.py

```
courses = ["Python", "Java"
print(courses)
```

OUTPUT

```
  File "main.py", line 1
    courses = ["Python", "Java"
              ^
SyntaxError: '[' was never closed
```

ASSIGNMENT INSTEAD OF COMPARISON

main.py

```
marks = 75
if marks = 75:
    print("Exactly 75")
```

OUTPUT

```
  File "main.py", line 2
    if marks = 75:
       ^^^^^^^^^^
SyntaxError: invalid syntax. Maybe you meant '==' or ':=' instead of '='?
```

2.7.2 Runtime errors (NameError, TypeError, ZeroDivisionError)

A **run-time error**, called an **exception**, happens while the program is running: the lines before it have already run. If nothing handles it (Module 13), the program stops and Python prints a traceback. Common ones: **NameError** (a name that does not exist, often a typo or a variable used before it is assigned); **TypeError** (an operation on the wrong type, such as adding a string and a number, or calling a function

with the wrong number of arguments); **ZeroDivisionError**; **ValueError** (right type, wrong value, such as `int("abc")`); **IndexError** and **KeyError** (missing list position or dictionary key); **AttributeError** (an object without that attribute or method).

NAMEERROR, WITH A SUGGESTION main.py

```
student = "Asha"
print("Line 2 runs")
print(studnet)
```

OUTPUT

```
Line 2 runs
Traceback (most recent call last):
  File "main.py", line 3, in <module>
    print(studnet)
          ^^^^^^^
NameError: name 'studnet' is not defined. Did you mean: 'student'?
```

TYPEERROR main.py

```
age = 21
print("Age next year: " + age + 1)
```

OUTPUT

```
Traceback (most recent call last):
  File "main.py", line 2, in <module>
    print("Age next year: " + age + 1)
          ~~~~~~~~~~~~~~~~~~^~~~~
TypeError: can only concatenate str (not "int") to str
```

ZERODIVISIONERROR main.py

```
total, count = 450, 0
print("Average:", total / count)
```

OUTPUT

```
Traceback (most recent call last):
  File "main.py", line 2, in <module>
    print("Average:", total / count)
                      ~~~~~~^~~~~~~
ZeroDivisionError: division by zero
```

MORE RUN-TIME ERRORS (CAUGHT SO THAT ALL OF THEM CAN BE SHOWN)

main.py

```
errors = [
    ("ValueError", lambda: int("abc")),
    ("IndexError", lambda: [1, 2, 3][5]),
    ("KeyError", lambda: {"a": 1}["b"]),
    ("AttributeError", lambda: "text".push("x")),
    ("TypeError (arguments)", lambda: len()),
]
for label, action in errors:
    try:
        action()
    except Exception as e:
        print(f"{label:<22} -> {type(e).__name__}: {e}")
```

OUTPUT

```
ValueError             -> ValueError: invalid literal for int() with base 10: 'abc'
IndexError             -> IndexError: list index out of range
KeyError               -> KeyError: 'b'
AttributeError         -> AttributeError: 'str' object has no attribute 'push'
TypeError (arguments)  -> TypeError: len() takes exactly one argument (0 given)
```

2.7.3 Logical errors

A **logical error** (a bug) is the hardest kind: the program runs without any error message but produces the wrong result, because the instructions do not express what you meant. Typical causes: wrong operator precedence, an off-by-one range, a condition with `and` instead of `or`, integer versus float division, or code at the wrong indentation level. Python cannot detect them; you find them by testing with inputs whose correct answer you know, by printing intermediate values, and with a debugger.

TWO PROGRAMS THAT RUN BUT GIVE WRONG ANSWERS

main.py

```
# Intended: the average of three marks
a, b, c = 80, 90, 70
average = a + b + c / 3                # bug: / has higher precedence than +
print("buggy average:", average)
average = (a + b + c) / 3              # fixed with parentheses
print("correct average:", average)

# Intended: print the numbers 1 to 5
print("buggy range:", list(range(1, 5)))    # bug: the end is excluded
print("correct range:", list(range(1, 6)))
```

OUTPUT

```
buggy average: 193.33333333333334
correct average: 80.0
buggy range: [1, 2, 3, 4]
correct range: [1, 2, 3, 4, 5]
```

2.7.4 Reading a traceback

When an exception is not handled, Python prints a **traceback**: the chain of function calls that led to the error. Read it **from the bottom up**. The **last line** gives the exception type and message, the most important information. The lines above it show where it happened: each entry has the file name, line number and function name, followed by the line of code, with `~~~^^^` markers under the exact part that failed (Python 3.11+). The bottom-most entry is where the error occurred; the entries above it are the calls that led there. Copying the last line into a search engine usually finds an explanation quickly.

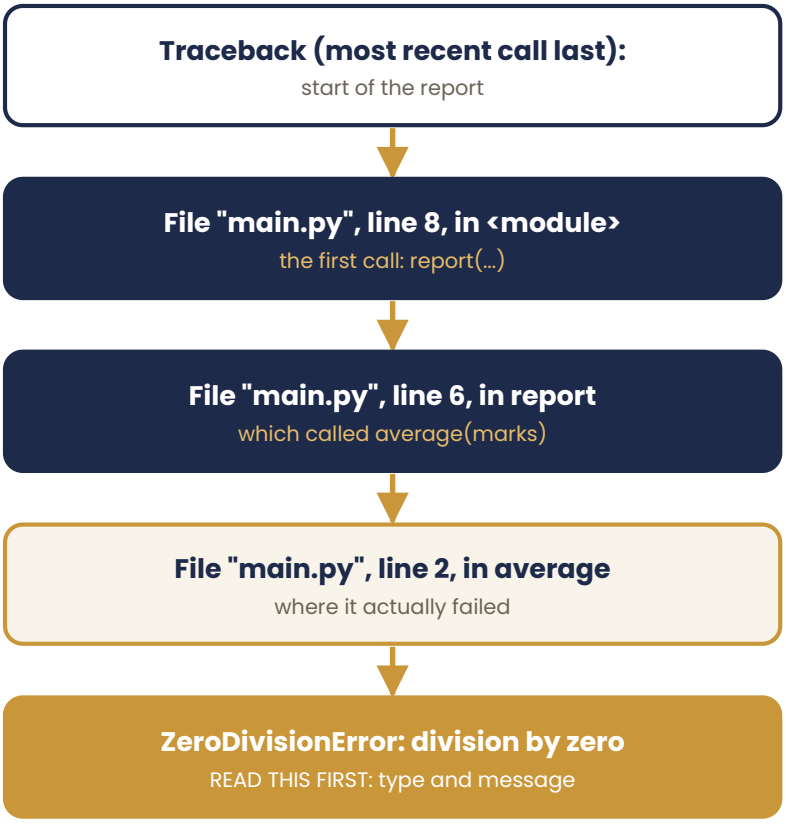
A TRACEBACK THROUGH TWO FUNCTIONS

main.py

```
def average(marks):
    return sum(marks) / len(marks)

def report(students):
    for name, marks in students.items():
        print(f"{name}: {average(marks):.1f}")

report({"Asha": [91, 88], "Ravi": []})
```

OUTPUT

```
Asha: 89.5
Traceback (most recent call last):
  File "main.py", line 8, in <module>
    report({"Asha": [91, 88], "Ravi": []})
    ~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "main.py", line 6, in report
    print(f"{name}: {average(marks):.1f}")
                     ~~~~~~~^^^^^^^
  File "main.py", line 2, in average
    return sum(marks) / len(marks)
           ~~~~~~~~~~~^~~~~~~~~~~~
ZeroDivisionError: division by zero
```



HOW TO READ A TRACEBACK: START AT THE LAST LINE, THEN LOOK JUST ABOVE IT

NOTE

Here the last line says division by zero, the entry above it points to `average`, line 2, and the call chain shows it was reached through `report` for Ravi, whose list of marks is empty. The fix is to handle empty lists in `average`.

Install Python 3.14 from python.org (tick "Add python.exe to PATH" on Windows; use python3 on macOS and Linux), verify with python --version, and manage versions with the py launcher, pyenv or uv. Run code in the REPL (>>> prompt, values shown automatically), as script files (python file.py), in IDLE, or in Jupyter and Colab notebooks; write code in VS Code with the Python extension, PyCharm or an online editor. print("Hello, World!") is a complete program. Blocks are defined by a colon and 4-space indentation; statements end at the end of the line, continue inside brackets or after \, and can be separated by ; (discouraged). Python is case-sensitive, has 35 keywords (keyword.kwlist) plus soft keywords, and names must start with a letter or underscore; PEP 8 uses snake_case, PascalCase and UPPER_CASE. Comments start with #; docstrings are the first string in a module, function or class and are shown by help(). Errors are syntax errors (nothing runs), run-time exceptions such as NameError, TypeError and ZeroDivisionError, and logical errors; read tracebacks from the last line upwards.

PRACTICE CHECKPOINT

Run a program that prints your **name and course** from the REPL, from a **.py file** in the terminal, and from a **Jupyter notebook**.

The same two lines work in all three places. In the REPL, type them after the `>>>` prompt; in a file, save them as `about_me.py` and run `python about_me.py`; in Jupyter or Colab, put them in a cell and press Shift+Enter. The runs below show the REPL and the script; in a notebook the output appears directly under the cell.

ABOUT_ME.PY Python

```python
"""Print my name and course."""
name = "Asha Rao"
course = "Python Programming"
print(f"My name is {name} and I am learning {course}.")
print("Running from a .py file in the terminal")
```

REPL AND SCRIPT VERSIONS main.py

```
printf 'name = "Asha Rao"\ncourse = "Python Programming"\nprint(f"My name is {name} and I am learning
{course}.")\n' | python -q -i 2>&1 | sed 's/>>> //g'
python about_me.py
```

TERMINAL

```
$ printf 'name = "Asha Rao"\ncourse = "Python Programming"\nprint(f"My name is {name} and I am
learning {course}.")\n' | python -q -i 2>&1 | sed 's/>>> //g'
My name is Asha Rao and I am learning Python Programming.
$ python about_me.py
My name is Asha Rao and I am learning Python Programming.
Running from a .py file in the terminal
```

MODULE 03 · BEGINNER

Variables and data types

Minimum duration **2 days** | Level **Beginner**

In Python, a variable is a name that refers to an object in memory; the object, not the name, has the type. This module explains how variables are created and re-bound, object identity and mutability, a first look at scope, every built-in data type, numbers in depth (integers of any size, floating point and its pitfalls, Decimal, Fraction and complex numbers), Booleans, truthiness and None, and how Python handles constants.

IN THIS MODULE

3.1 Variables
3.2 Object identity
3.3 Variable scope (introduction)
3.4 Built-in data types overview
3.5 Numbers in depth
3.6 Booleans and None
3.7 Constants

3.1 Variables

3.1.1 Creating variables by assignment (no declaration)

A **variable** is created the first time you assign to it with `=`; there is no declaration and no type to write. `name = "Asha"` means: create (or find) the string object `"Asha"`, then make the name `name` refer to it. The right side is always evaluated first, so `count = count + 1` reads the old value, adds one, and re-binds the name to the result. Using a name before it has been assigned raises a `NameError`.

VARIABLES ARE CREATED BY ASSIGNMENT

main.py

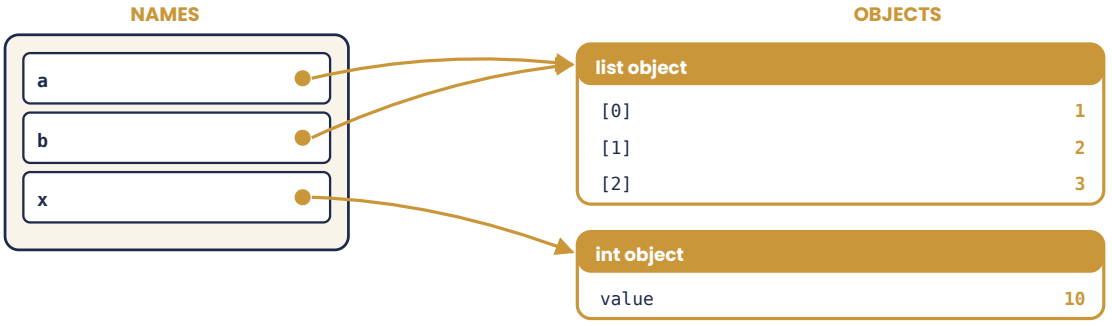
```
student = "Asha Rao"          # str
age = 21                      # int
height_m = 1.62               # float
is_enrolled = True            # bool
courses = ["Python", "SQL"]   # list

print(student, age, height_m, is_enrolled, courses)
age = age + 1                 # right side first: 21 + 1, then re-bind age
print("next year:", age)
print(grade)                  # never assigned
```

OUTPUT

```
Asha Rao 21 1.62 True ['Python', 'SQL']
next year: 22
Traceback (most recent call last):
  File "main.py", line 10, in <module>
    print(grade)                  # never assigned
          ^^^^^
NameError: name 'grade' is not defined
```

3.1.2 Names as references to objects

It helps to think of a Python variable not as a box that contains a value, but as a **label** (a reference) attached to an object. Objects live in memory; names point to them. Assigning one variable to another, `b = a`, copies the **reference**, not the object: both names now point to the same object. For immutable objects such as numbers and strings this makes no visible difference, but for mutable objects such as lists, a change made through one name is visible through the other.



B = A MAKES TWO NAMES REFER TO THE SAME LIST

TWO NAMES, ONE OBJECT

main.py

```
a = [1, 2, 3]
b = a                       # b refers to the SAME list object
b.append(4)                 # change it through b...
print("a:", a)              # ...and a sees the change
print("same object?", a is b)

c = a.copy()                # a NEW list with the same contents
c.append(5)
print("a:", a, "| c:", c, "| same object?", a is c)

x = 10
y = x
y = y + 1                   # ints are immutable: y is re-bound to a new object
print("x:", x, "| y:", y)
```

OUTPUT

```
a: [1, 2, 3, 4]
same object? True
a: [1, 2, 3, 4] | c: [1, 2, 3, 4, 5] | same object? False
x: 10 | y: 11
```

3.1.3 Dynamic typing and re-binding to a different type

Because the type belongs to the object, the same name can be **re-bound** to an object of a completely different type at any time. This is legal, but doing it on purpose usually makes code confusing; in real programs keep one meaning per variable. `type(x)` always tells you the type of the object a name currently refers to.

ONE NAME, FOUR TYPES OVER TIME main.py

```
value = 100
print(value, type(value))
value = "one hundred"
print(value, type(value))
value = 100.0
print(value, type(value))
value = None
print(value, type(value))
```

OUTPUT

```
100 <class 'int'>
one hundred <class 'str'>
100.0 <class 'float'>
None <class 'NoneType'>
```

3.1.4 Multiple assignment a, b, c = 1, 2, 3

Several variables can be assigned in one statement by separating names and values with commas. The right side builds a tuple, and Python **unpacks** it into the names in order. The number of names must match the number of values, otherwise a `ValueError` is raised. Unpacking works with any iterable (lists, strings, ranges), and a starred name collects the rest (`first, *rest = ...`, covered in Module 8).

ASSIGNING SEVERAL VARIABLES AT ONCE main.py

```
name, age, city = "Ravi", 22, "Pune"
print(name, age, city)

x, y, z = [10, 20, 30]          # any iterable can be unpacked
a, b, c = "XYZ"
print(x, y, z, "|", a, b, c)

first, *others = [91, 78, 85, 64]
print("first:", first, "| others:", others)

try:
    p, q = 1, 2, 3
except ValueError as e:
    print("ValueError:", e)
```

OUTPUT

```
Ravi 22 Pune
10 20 30 | X Y Z
first: 91 | others: [78, 85, 64]
ValueError: too many values to unpack (expected 2, got 3)
```

3.1.5 Same value to many variables a = b = c = 0

A chained assignment `a = b = c = 0` makes all three names refer to the **same** object. With immutable values such as numbers and strings this is safe and convenient for initialising counters. With a mutable object it is a trap: `x = y = []` creates one list shared by both names, so appending through one changes the other. Create separate lists with `x, y = [], []`.

CHAINED ASSIGNMENT AND THE SHARED-LIST TRAP main.py

```
total = count = errors = 0
count += 1
print(total, count, errors)              # ints: count got a new object, others unaffected

passed = failed = []                     # ONE list shared by two names
passed.append("Asha")
print("passed:", passed, "| failed:", failed)

passed, failed = [], []                  # two separate lists
passed.append("Asha")
print("passed:", passed, "| failed:", failed)
```

OUTPUT

```
0 1 0
passed: ['Asha'] | failed: ['Asha']
passed: ['Asha'] | failed: []
```

3.1.6 Swapping a, b = b, a

In most languages swapping two variables needs a temporary third variable. In Python, `a, b = b, a` does it in one line: the right side builds the tuple `(b, a)` from the current values first, then unpacks it into `a` and `b`. The same idea rotates several values.

SWAPPING WITHOUT A TEMPORARY VARIABLE main.py

```
a, b = 5, 9
temp = a; a = b; b = temp                # the long way
print("after temp swap:", a, b)

a, b = b, a                              # the Pythonic way
print("after tuple swap:", a, b)

x, y, z = "x", "y", "z"
x, y, z = y, z, x                        # rotate left
print("rotated:", x, y, z)
```

OUTPUT

```
after temp swap: 9 5
after tuple swap: 5 9
rotated: y z x
```

3.1.7 Deleting variables with del

`del name` removes the **name**, not necessarily the object: if no other name refers to the object, Python frees it automatically. After `del`, using the name raises a `NameError`. `del` also removes list items, slices and dictionary keys (Modules 8 and 9). It is rarely needed for plain variables; it is useful to drop references to large objects or to remove temporary names.

DEL REMOVES NAMES (AND ITEMS) main.py

```
data = [1, 2, 3]
alias = data
del data                        # removes the name 'data' only
print("alias still works:", alias)

scores = [10, 20, 30, 40]
del scores[1]                   # delete an item
print("scores:", scores)

try:
    print(data)
except NameError as e:
    print("NameError:", e)
```

OUTPUT

```
alias still works: [1, 2, 3]
scores: [10, 30, 40]
NameError: name 'data' is not defined
```

3.2 Object identity

3.2.1 id() and type()

Every object has three properties. Its **identity** is a unique integer that never changes during the object's life, returned by `id()` (in CPython it is the memory address). Its **type** determines what operations it supports, returned by `type()`; `isinstance(obj, type)` checks it, also accepting subclasses. Its **value** is the data it holds. The `is` operator compares identities (same object), while `==` compares values (equal contents).

IDENTITY, TYPE AND VALUE main.py

```
a = [1, 2, 3]
b = a
c = [1, 2, 3]

print("type(a):", type(a), "| isinstance(a, list):", isinstance(a, list))
print("id(a) == id(b):", id(a) == id(b), "-> same object")
print("id(a) == id(c):", id(a) == id(c), "-> different objects")
print("a == c:", a == c, "(equal values) | a is c:", a is c, "(not the same object)")
print("isinstance(True, int):", isinstance(True, int), "(bool is a subclass of int)")
```

OUTPUT

```
type(a): <class 'list'> | isinstance(a, list): True
id(a) == id(b): True -> same object
id(a) == id(c): False -> different objects
a == c: True (equal values) | a is c: False (not the same object)
isinstance(True, int): True (bool is a subclass of int)
```

3.2.2 Mutable vs immutable objects

An object is **mutable** if its value can change after it is created, and **immutable** if it cannot. **Immutable**: `int`, `float`, `complex`, `bool`, `str`, `tuple`, `frozenset`, `bytes`, `None`. **Mutable**: `list`, `dict`, `set`, `bytearray` and most objects of your own classes. "Changing" an immutable value, as in `s = s + "!"` or `n += 1`, really creates a **new** object and re-binds the name, so `id()` changes; changing a mutable object in place (`lst.append(4)`) keeps the same identity. Mutability explains why shared lists change

unexpectedly, why tuples can be dictionary keys but lists cannot, and why you should not use a mutable default argument (Module 10).

IMMUTABLE OBJECTS ARE REPLACED, MUTABLE OBJECTS ARE MODIFIED

main.py

```
n = 10
print("n:", n, "id before:", id(n) % 100000)
n += 1                                      # a NEW int object
print("n:", n, "id after: ", id(n) % 100000, "-> changed")

s = "Py"
old = id(s)
s += "thon"                                  # a NEW str object
print("str changed identity:", id(s) != old)

lst = [1, 2, 3]
old = id(lst)
lst.append(4)                                # same list object, modified in place
print("list kept identity:", id(lst) == old, lst)

try:
    s[0] = "J"                               # strings cannot be changed in place
except TypeError as e:
    print("TypeError:", e)
t = (1, 2, 3)
try:
    t[0] = 99
except TypeError as e:
    print("TypeError:", e)
```

OUTPUT

```
n: 10 id before: 26744
n: 11 id after:  26776 -> changed
str changed identity: True
list kept identity: True [1, 2, 3, 4]
TypeError: 'str' object does not support item assignment
TypeError: 'tuple' object does not support item assignment
```

Immutable	Mutable
int, float, complex, bool	list
str	dict
tuple	set
frozenset	bytearray
bytes, None, range	objects of most user-defined classes

3.2.3 Integer caching and string interning

To save memory and time, CPython **reuses** some immutable objects. Small integers from **-5 to 256** are created once at start-up and shared, so every `100` in a program is the same object. Short strings that look like identifiers are **interned** (stored once), and `sys.intern()` forces interning. These are implementation details that can change between versions and depend on how the code is written, which is exactly why you must **never use `is` to compare numbers or strings**: use `==`. Use `is` only for singletons such as `None`, `True` and `False`.

CACHING MAKES "IS" UNRELIABLE FOR VALUES

main.py

```
import sys

a = 256
b = int("256")                # created at run time, still the cached object
print("256 is 256:", a is b)

c = 257
d = int("257")                # outside the cache: a new object
print("257 is 257:", c is d, "| but 257 == 257:", c == d)

s1 = "python"
s2 = "".join(["py", "thon"])  # built at run time: a different object
print("strings: ==", s1 == s2, "| is", s1 is s2)
print("after sys.intern: is", sys.intern(s1) is sys.intern(s2))
```

OUTPUT

```
256 is 256: True
257 is 257: False | but 257 == 257: True
strings: == True | is False
after sys.intern: is True
```

WATCH OUT

`if x is 100:` or `if name is "admin":` may work in a quick test and fail in real use. Python 3.8+ even prints a `SyntaxWarning: "is" with a literal`. Always compare values with `==`.

3.3 Variable scope (introduction)

3.3.1 Local and global variables (detailed in Module 10)

The **scope** of a variable is the part of the program where its name can be used. A variable assigned at the top level of a file is **global**: visible everywhere in that file (module). A variable assigned inside a function is **local**: it exists only while the function runs and is invisible outside it. A function can **read** a global variable, but assigning to a name inside a function creates a new local variable, unless the name is declared `global`. Note that `if` and `for` blocks do **not** create a new scope in Python, unlike C and Java. Module 10 covers the full LEGB rules.

GLOBAL AND LOCAL VARIABLES

main.py

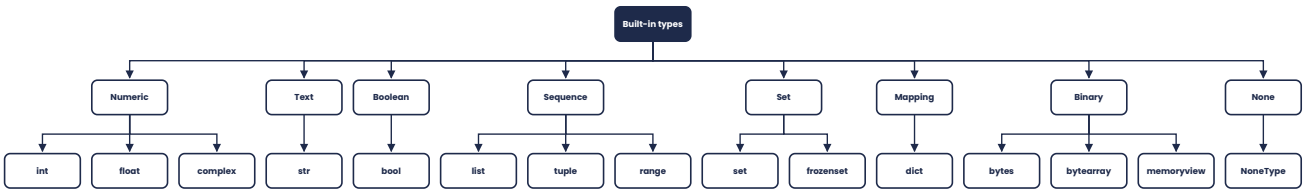
```
college = "LBF Academy"          # global

def show_student():
    name = "Asha"                # local to this function
    print(name, "studies at", college)   # reading a global is fine

show_student()
print("college outside:", college)

for i in range(3):
    last = i                     # loops do NOT create a new scope
print("i and last after the loop:", i, last)

try:
    print(name)                  # local names do not exist outside
except NameError as e:
    print("NameError:", e)
```

OUTPUT

```
Asha studies at LBF Academy
college outside: LBF Academy
i and last after the loop: 2 2
NameError: name 'name' is not defined
```

3.4 Built-in data types overview

Python has a rich set of built-in types, grouped into categories. Each one is covered in depth later; this overview shows them side by side.



THE BUILT-IN DATA TYPES BY CATEGORY

3.4.1 Numeric: int, float, complex

int holds whole numbers of any size (`42`, `-7`, `10**100`); **float** holds real numbers with a fractional part in double precision (`3.14`, `2.5e-3`); **complex** holds numbers with a real and imaginary part (`3+4j`). Section 3.5 covers them in depth.

3.4.2 Text: str

str holds text: an immutable sequence of Unicode characters, written in single, double or triple quotes. It supports indexing, slicing and dozens of methods (Module 7).

3.4.3 Boolean: bool

bool has exactly two values, `True` and `False`, produced by comparisons and used in conditions. It is a subclass of `int` (section 3.6).

3.4.4 Sequence: list, tuple, range

Sequences are ordered collections accessed by position (index from 0). A **list** `[1, 2, 3]` is mutable and can grow and shrink; a **tuple** `(1, 2, 3)` is immutable; a **range** `range(0, 10, 2)` is an immutable, memory-efficient sequence of integers generated on demand (Module 8).

3.4.5 Set: set, frozenset

A **set** `{1, 2, 3}` is an unordered collection of **unique** items with fast membership tests and mathematical operations (union, intersection); a **frozenset** is its immutable version (Module 9).

3.4.6 Mapping: dict

A **dict** `{"name": "Asha", "marks": 91}` maps unique **keys** to values, with very fast lookup by key; it remembers insertion order (Module 9).

3.4.7 Binary: bytes, bytearray, memoryview

Binary types hold raw bytes, numbers from 0 to 255, used for files, images, network data and encodings. **bytes** `b"abc"` is immutable, **bytearray** is mutable, and **memoryview** gives a view onto the memory of another binary object without copying it (useful for large data). Text becomes bytes with `str.encode()` and back with `bytes.decode()`.

BYTES, BYTEARRAY AND MEMORYVIEW main.py

```
text = "Namaste ₹"
data = text.encode("utf-8")              # str -> bytes
print(data, "| length in bytes:", len(data), "| characters:", len(text))
print("back to text:", data.decode("utf-8"))
print("first byte as a number:", data[0], "->", chr(data[0]))

buf = bytearray(b"hello")                # mutable
buf[0] = ord("J")
print(buf, "|", buf.decode())

big = bytearray(b"ABCDEFGH")
view = memoryview(big)[2:5]              # no copy: a window onto big
view[0] = ord("x")
print("memoryview changed the original:", big)
```

OUTPUT

```
b'Namaste \xe2\x82\xb9' | length in bytes: 11 | characters: 9
back to text: Namaste ₹
first byte as a number: 78 -> N
bytearray(b'Jello') | Jello
memoryview changed the original: bytearray(b'ABxDEFGH')
```

3.4.8 NoneType

None is the single value of type **NoneType**, meaning "no value" or "nothing yet". Functions without a `return` statement return `None` (section 3.6.3).

ONE VALUE OF EVERY BUILT-IN TYPE

main.py

```
samples = [42, 3.14, 2 + 3j, "LBF", True, [1, 2], (1, 2), range(5),
           {1, 2}, frozenset({3}), {"k": "v"}, b"hi", bytearray(b"hi"), memoryview(b"hi"), None]
for value in samples:
    print(f"{type(value).__name__:<11} {'immutable' if type(value) not in (list, set, dict, bytearray)
else 'mutable':<10} {value!r}")
```

OUTPUT

```
int         immutable  42
float       immutable  3.14
complex     immutable  (2+3j)
str         immutable  'LBF'
bool        immutable  True
list        mutable    [1, 2]
tuple       immutable  (1, 2)
range       immutable  range(0, 5)
set         mutable    {1, 2}
frozenset   immutable  frozenset({3})
dict        mutable    {'k': 'v'}
bytes       immutable  b'hi'
bytearray   mutable    bytearray(b'hi')
memoryview  immutable  <memory at 0x7f90dcea2740>
NoneType    immutable  None
```

Type	Example	Mutable	Ordered	Duplicates
int / float / complex	`42`, `3.14`, `2+3j`	no	-	-
str	`"LBF"`	no	yes	yes
bool	`True`	no	-	-
list	`[1, 2, 2]`	yes	yes	yes
tuple	`(1, 2, 2)`	no	yes	yes
range	`range(5)`	no	yes	no
set / frozenset	`{1, 2}`	yes / no	no	no
dict	`{"a": 1}`	yes	yes (insertion)	unique keys
bytes / bytearray	`b"hi"`	no / yes	yes	yes
NoneType	`None`	no	-	-

3.5 Numbers in depth

3.5.1 int: unlimited precision

Python integers have **arbitrary precision**: they grow as large as memory allows, so they never overflow. In C or Java, an `int` stops at about 2.1 billion (2^{31} - 1) and a `long` at about 9.2 quintillion, and larger results wrap around silently. In Python, factorials, powers and cryptographic numbers with hundreds of digits just work. (Very large integers are slower, and converting a number with more than 4,300 digits to a string is limited by default for security reasons.)

INTEGERS NEVER OVERFLOW main.py

```
import math, sys

print("2 ** 100 =", 2 ** 100)
print("25! =", math.factorial(25))
print("digits in 1000!:", len(str(math.factorial(1000))))
c_int_max = 2 ** 31 - 1
print("C int max:", c_int_max, "| Python adds 1 without overflow:", c_int_max + 1)
print("a googol has", len(str(10 ** 100)), "digits")
print("int max digits for str():", sys.get_int_max_str_digits())
```

OUTPUT

```
2 ** 100 = 1267650600228229401496703205376
25! = 15511210043330985984000000
digits in 1000!: 2568
C int max: 2147483647 | Python adds 1 without overflow: 2147483648
a googol has 101 digits
int max digits for str(): 4300
```

3.5.2 Binary 0b, octal 0o, hexadecimal 0x

Integer literals can be written in other bases with a prefix: `0b` for binary (base 2), `0o` for octal (base 8) and `0x` for hexadecimal (base 16). They are ordinary `int` values; only the way you write them differs. `bin()`, `oct()` and `hex()` convert an integer to a string in those bases, and `int(text, base)` parses one. Hex is common for colours and memory, binary for flags and bit masks, octal for Unix file permissions.

WRITING NUMBERS IN OTHER BASES main.py

```
print(0b1010, 0o17, 0xFF, 0xff)            # all plain ints
print(bin(10), oct(15), hex(255))           # int -> text in another base
print(int("1010", 2), int("17", 8), int("ff", 16), int("z", 36))
print("LBF navy colour #1B2A4A as RGB:", int("1B", 16), int("2A", 16), int("4A", 16))
print("file permission 0o755 in binary:", bin(0o755))
```

OUTPUT

```
10 15 255 255
0b1010 0o17 0xff
10 15 255 35
LBF navy colour #1B2A4A as RGB: 27 42 74
file permission 0o755 in binary: 0b111101101
```

3.5.3 Underscores in numbers 1_000_000

Underscores may be placed between digits to make long numbers readable: `1_000_000`, `1_00_00_000` (Indian grouping), `0b1111_0000`. They are ignored by Python. Underscores can also be produced in output with the format specifier `:_`.

READABLE NUMERIC LITERALS main.py

```
population = 1_40_00_00_000          # 140 crore, Indian grouping
salary = 12_50_000
print(population, salary, population == 1400000000)
print(f"{population:,} | {population:_}")
print(int("1_000"))                   # int() accepts underscores too
```

OUTPUT

```
1400000000 1250000 True
1,400,000,000 | 1_400_000_000
1000
```

3.5.4 float: double precision, scientific notation, inf, nan

A **float** is a 64-bit IEEE 754 double-precision floating-point number: about 15-17 significant decimal digits, with values from about 2.2×10^{-308} to 1.8×10^{308}. A number with a decimal point or an exponent is a float: `3.0`, `.5`, `2e3` (2×10^3), `1.5e-4`. Special values: `float("inf")` (infinity, from overflow or by choice, useful as a starting maximum) and `float("nan")` ("not a number", from undefined operations); NaN is not equal to anything, not even itself, so test it with `math.isnan()`. `sys.float_info` describes the limits.

FLOATS, SCIENTIFIC NOTATION AND SPECIAL VALUES main.py

```
import math, sys

print(3.0, .5, 2e3, 1.5e-4, 6.022e23)
print("max float:", sys.float_info.max, "| epsilon:", sys.float_info.epsilon)
print("overflow becomes inf:", 1e308 * 10)
big = float("inf")
print("inf > 10**100:", big > 10 ** 100, "| -inf:", -big)
nan = float("nan")
print("nan == nan:", nan == nan, "| math.isnan(nan):", math.isnan(nan))
print("inf - inf:", big - big)
print("int division by zero raises; but", math.inf, "is a float constant")
```

OUTPUT

```
3.0 0.5 2000.0 0.00015 6.022e+23
max float: 1.7976931348623157e+308 | epsilon: 2.220446049250313e-16
overflow becomes inf: inf
inf > 10**100: True | -inf: -inf
nan == nan: False | math.isnan(nan): True
inf - inf: nan
int division by zero raises; but inf is a float constant
```

3.5.5 Floating-point precision issues and round()

Floats are stored in **binary**, and most decimal fractions such as 0.1 cannot be represented exactly in binary, just as 1/3 cannot be written exactly in decimal. The stored value is extremely close, but not exact, so tiny errors appear: `0.1 + 0.2` gives `0.30000000000000004`. This is not a Python bug; every language using IEEE 754 does the same. Consequences: never test floats for exact equality, use `math.isclose()`; format results for display (`f"{x:.2f}"`); use `math.fsum()` (or `sum()`, which compensates for rounding errors since Python 3.12) when adding many floats; and use `Decimal` or integer paise for money. `round(x, n)` rounds to n decimal places using **banker's rounding** (round half to even: `round(2.5)` is 2, `round(3.5)` is 4), and results like `round(2.675, 2) == 2.67` happen because 2.675 is really stored as 2.67499999...

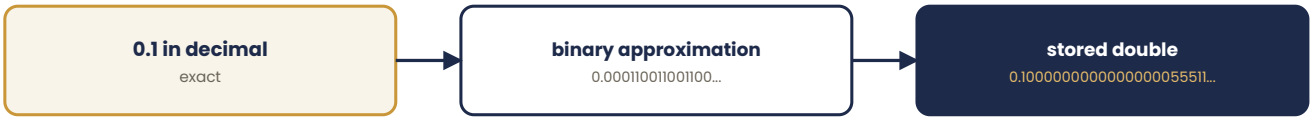
WHY 0.1 + 0.2 IS NOT 0.3

main.py

```
import math

print("0.1 + 0.2 =", 0.1 + 0.2)
print("0.1 + 0.2 == 0.3:", 0.1 + 0.2 == 0.3)
print("math.isclose(0.1 + 0.2, 0.3):", math.isclose(0.1 + 0.2, 0.3))
print("stored value of 0.1:", f"{0.1:.25f}")
total = 0.0
for _ in range(10):
    total += 0.1                                  # the error accumulates step by step
print("adding 0.1 ten times in a loop:", total, "| math.fsum:", math.fsum([0.1] * 10))
print("sum() (compensated since Python 3.12):", sum([0.1] * 10))

print("round(2.5), round(3.5), round(-2.5):", round(2.5), round(3.5), round(-2.5))   # banker's
rounding
print("round(2.675, 2):", round(2.675, 2), "because 2.675 is", f"{2.675:.20f}")
print("for display:", f"{0.1 + 0.2:.2f}")
```

OUTPUT

```
0.1 + 0.2 = 0.30000000000000004
0.1 + 0.2 == 0.3: False
math.isclose(0.1 + 0.2, 0.3): True
stored value of 0.1: 0.1000000000000000055511151
adding 0.1 ten times in a loop: 0.9999999999999999 | math.fsum: 1.0
sum() (compensated since Python 3.12): 1.0
round(2.5), round(3.5), round(-2.5): 2 4 -2
round(2.675, 2): 2.67 because 2.675 is 2.67499999999999982236
for display: 0.30
```



DECIMAL FRACTIONS BECOME REPEATING BINARY FRACTIONS

3.5.6 decimal.Decimal and fractions.Fraction

The standard library offers two exact alternatives. **decimal.Decimal** stores numbers in base 10, so 0.1 is exactly 0.1; you control precision and rounding (`ROUND_HALF_UP`, the school rule), which makes it the right type for **money**, billing and tax. Create Decimals from **strings** (`Decimal("0.1")`), not from floats, or the float's error comes along. **fractions.Fraction** stores exact rational numbers as numerator/denominator, so 1/3 stays exactly 1/3; it is useful in maths and education. Both are slower than float.

EXACT ARITHMETIC WITH DECIMAL AND FRACTION main.py

```
from decimal import Decimal, ROUND_HALF_UP, getcontext
from fractions import Fraction

print("Decimal:", Decimal("0.1") + Decimal("0.2"), "| == 0.3:", Decimal("0.1") + Decimal("0.2") ==
Decimal("0.3"))
print("Decimal from a float carries the error:", Decimal(0.1))

price = Decimal("499.99")
gst = (price * Decimal("0.18")).quantize(Decimal("0.01"), rounding=ROUND_HALF_UP)
print("price:", price, "| GST 18%:", gst, "| total:", price + gst)
print("school rounding of 2.675:", Decimal("2.675").quantize(Decimal("0.01"), rounding=ROUND_HALF_UP))

getcontext().prec = 40
print("1/7 to 40 digits:", Decimal(1) / Decimal(7))

third = Fraction(1, 3)
print("Fraction:", third + third + third, "|", Fraction(3, 4) * Fraction(2, 3), "|",
Fraction("0.125"))
print("Fraction from 0.1 shows the binary truth:", Fraction(0.1))
```

OUTPUT

```
Decimal: 0.3 | == 0.3: True
Decimal from a float carries the error: 0.1000000000000000055511151231257827021181583404541015625
price: 499.99 | GST 18%: 90.00 | total: 589.99
school rounding of 2.675: 2.68
1/7 to 40 digits: 0.1428571428571428571428571428571428571429
Fraction: 1 | 1/2 | 1/8
Fraction from 0.1 shows the binary truth: 3602879701896397/36028797018963968
```

Type	Exact?	Speed	Use for
float	no (binary)	fastest	science, measurements, general calculations
Decimal	yes for decimal values	slower	money, billing, tax, when decimal rounding rules matter
Fraction	yes for rationals	slowest	exact ratios, maths teaching
int (in paise)	yes	fast	money stored as whole paise or cents

3.5.7 complex numbers

A **complex** number has a real part and an imaginary part, written with `j` (engineers' notation) for the square root of -1: `3 + 4j`, or `complex(3, 4)`. The attributes `.real` and `.imag` give the parts, `abs()` the magnitude, and `.conjugate()` the conjugate; the `cmath` module provides complex versions of mathematical functions. Complex numbers are used in electrical engineering (AC circuits), signal processing and physics.

COMPLEX NUMBERS

main.py

```
import cmath

z = 3 + 4j
print(z, type(z), "| real:", z.real, "| imag:", z.imag)
print("magnitude abs(z):", abs(z), "| conjugate:", z.conjugate())
print("(1+2j) * (3-1j) =", (1 + 2j) * (3 - 1j))
print("square root of -1:", cmath.sqrt(-1))
print("polar form (r, angle):", cmath.polar(z))
```

OUTPUT

```
(3+4j) <class 'complex'> | real: 3.0 | imag: 4.0
magnitude abs(z): 5.0 | conjugate: (3-4j)
(1+2j) * (3-1j) = (5+5j)
square root of -1: 1j
polar form (r, angle): (5.0, 0.9272952180016122)
```

3.6 Booleans and None

3.6.1 True and False as 1 and 0

`bool` is a subclass of `int`: `True` behaves as 1 and `False` as 0 in arithmetic. This is handy for counting: `sum(mark >= 40 for mark in marks)` counts how many students passed. `bool(x)` converts any value to a Boolean using the truthiness rules below.

BOOLEANS ARE INTEGERS

main.py

```
print(True + True, True * 10, False * 10)
print("isinstance(True, int):", isinstance(True, int), "| True == 1:", True == 1)
marks = [91, 35, 78, 22, 64]
print("students passed:", sum(m >= 40 for m in marks))
print("bool(1), bool(0):", bool(1), bool(0))
```

OUTPUT

```
2 10 0
isinstance(True, int): True | True == 1: True
students passed: 3
bool(1), bool(0): True False
```

3.6.2 Truthy and falsy values

Every object can be tested in a condition. The **falsy** values, treated as false, are: `False`, `None`, zero of any numeric type (`0`, `0.0`, `0j`, `Decimal(0)`), and every **empty** collection (`""`, `[]`, `()`, `{}`, `set()`, `range(0)`). Everything else is **truthy**, including the string `"0"`, `"False"`, `" "`, `[0]` and negative numbers. Pythonic code relies on this: `if names:` rather than `if len(names) > 0:`.

WHICH VALUES COUNT AS FALSE?

main.py

```
values = [False, None, 0, 0.0, 0j, "", [], (), {}, set(), range(0),
          True, 1, -5, 0.001, "0", "False", " ", [0], (None,), {"": 0}]
falsy = [repr(v) for v in values if not v]
truthy = [repr(v) for v in values if v]
print("falsy: ", ", ".join(falsy))
print("truthy:", ", ".join(truthy))

cart = []
if not cart:
    print("Your cart is empty")
```

OUTPUT

```
falsy:  False, None, 0, 0.0, 0j, '', [], (), {}, set(), range(0, 0)
truthy: True, 1, -5, 0.001, '0', 'False', ' ', [0], (None,), {'': 0}
Your cart is empty
```

Falsy	Truthy (examples)
`False`, `None`	`True`, any object not listed on the left
`0`, `0.0`, `0j`	`1`, `-5`, `0.001`
`""`	`"0"`, `"False"`, `" "`
`[]`, `()`, `{}`, `set()`, `range(0)`	`[0]`, `(None,)`, `{"": 0}`

3.6.3 None and is None

`None` represents the absence of a value: a result that is not available, an optional argument that was not given, or a variable that will be set later. A function without `return` returns `None`. There is exactly one `None` object, so test for it with **`is None`** or **`is not None`**, not `== None` (which a class could override) and not plain truthiness when 0 or an empty string are valid values.

TESTING FOR NONE CORRECTLY main.py

```
def find_student(roll_no):
    students = {101: "Asha", 102: "Ravi"}
    return students.get(roll_no)          # None if not found

for roll in (101, 999):
    result = find_student(roll)
    if result is None:
        print(roll, "-> not found")
    else:
        print(roll, "->", result)

def greet(name):
    print("Hi", name)                      # no return statement
print("greet returns:", greet("Meera"))

marks = 0
print("truthiness check wrongly says no marks:", not marks, "| is None check:", marks is None)
```

OUTPUT

```
101 -> Asha
999 -> not found
Hi Meera
greet returns: None
truthiness check wrongly says no marks: True | is None check: False
```

3.7 Constants

3.7.1 UPPER_CASE convention (no true constants)

Python has no `const` keyword: any name can be re-bound. By convention, names written in **UPPER_CASE** at the top of a module are treated as **constants** that must not be changed: `MAX_STUDENTS = 60`, `PI = 3.14159`, `API_URL = "..."`. Everyone reading the code understands the intention, and linters can warn about re-assignment. Group related constants in a module (`config.py`) or use an `Enum` (Module 18) for fixed sets of values.

CONSTANTS BY CONVENTION main.py

```
GST_RATE = 0.18
MAX_STUDENTS_PER_BATCH = 60
COURSES = ("Python", "Java", "SQL")      # a tuple: the collection itself cannot change

def fee_with_gst(fee):
    return round(fee * (1 + GST_RATE), 2)

print(fee_with_gst(14999), MAX_STUDENTS_PER_BATCH, COURSES)
GST_RATE = 0.5                           # Python allows this; the convention says "don't"
print("re-bound anyway:", GST_RATE)
```

OUTPUT

```
17698.82 60 ('Python', 'Java', 'SQL')
re-bound anyway: 0.5
```

3.7.2 typing.Final (overview)

`typing.Final` marks a name as a constant for **type checkers**: `MAX_SIZE: Final = 100`. Python itself still allows re-assignment at run time, but tools such as mypy and Pyright (and editors using them) report an

error if the code assigns to it again. It is also used for class attributes that subclasses must not override. Module 19 covers type hints in detail.

FINAL FOR TYPE CHECKERS

main.py

```
from typing import Final

PASS_MARK: Final = 40
MAX_MARK: Final[int] = 100

def result(mark: int) -> str:
    return "pass" if mark >= PASS_MARK else "fail"

print(result(55), result(30))
PASS_MARK = 35        # runs, but mypy reports: Cannot assign to final name "PASS_MARK"
print("at run time Python did not stop the change:", PASS_MARK)
```

OUTPUT

```
pass fail
at run time Python did not stop the change: 35
```

A variable is a name bound to an object by assignment; the object has the type, names are references, and the same name can be re-bound to any type. Multiple assignment unpacks values, a = b = c shares one object (dangerous for lists), a, b = b, a swaps, and del removes a name. Every object has an identity (id), a type (type, isinstance) and a value; is compares identity, == compares values. Immutable objects (int, float, str, tuple, frozenset, bytes) are replaced when "changed"; mutable ones (list, dict, set, bytearray) change in place. Small ints and some strings are cached, so never use is for values. Top-level names are global, names assigned in functions are local, and blocks do not create scopes. The built-in types are numeric (int, float, complex), str, bool, sequences (list, tuple, range), sets, dict, binary types and None. ints have unlimited precision and can be written in binary, octal, hex and with underscores; floats are binary doubles with precision errors (use math.isclose, round, Decimal for money, Fraction for exact ratios), plus inf and nan; complex numbers use j. True and False are 1 and 0; empty and zero values are falsy; test None with is None. Constants are UPPER_CASE by convention, and typing.Final lets type checkers enforce them.

PRACTICE CHECKPOINT

Create one variable of **each built-in type**, print its value, `type()` and `id()`, and show that re-assigning an int creates a **new object**.

SAMPLE SOLUTION

main.py

```
variables = {
    "an_int": 42,
    "a_float": 3.14,
    "a_complex": 2 + 3j,
    "a_str": "Learn Being Forward",
    "a_bool": True,
    "a_list": [1, 2, 3],
    "a_tuple": (1, 2, 3),
    "a_range": range(5),
    "a_set": {1, 2, 3},
    "a_frozenset": frozenset({4, 5}),
    "a_dict": {"name": "Asha"},
    "a_bytes": b"LBF",
    "a_bytearray": bytearray(b"LBF"),
    "a_memoryview": memoryview(b"LBF"),
    "a_none": None,
}
print(f"{'name':<14}{'type':<12}{'id (last 6 digits)':<20}value")
for name, value in variables.items():
    print(f"{name:<14}{type(value).__name__:<12}{id(value) % 1_000_000:<20}{value!r}")

print()
count = 1000
first_id = id(count)
count += 1                                  # ints are immutable: a new object is created
print(f"count = {count}: id changed after +=1 ->", id(count) != first_id)
numbers = [1000]
list_id = id(numbers)
numbers.append(1001)                        # lists are mutable: same object
print(f"numbers = {numbers}: id unchanged after append ->", id(numbers) == list_id)
```

OUTPUT

```
name          type        id (last 6 digits)  value
an_int        int         127768              42
a_float       float       43536               3.14
a_complex     complex     901232              (2+3j)
a_str         str         92080               'Learn Being Forward'
a_bool        bool        681640              True
a_list        list        839744              [1, 2, 3]
a_tuple       tuple       849712              (1, 2, 3)
a_range       range       865632              range(0, 5)
a_set         set         734144              {1, 2, 3}
a_frozenset   frozenset   52832               frozenset({4, 5})
a_dict        dict        711744              {'name': 'Asha'}
a_bytes       bytes       101536              b'LBF'
a_bytearray   bytearray   92912               bytearray(b'LBF')
a_memoryview  memoryview  218880              <memory at 0x7fd1f1f46800>
a_none        NoneType    773216              None

count = 1001: id changed after +=1 -> True
numbers = [1000, 1001]: id unchanged after append -> True
```

MODULE 04 · BEGINNER

Operators and expressions

Minimum duration **1 day** | Level **Beginner**

Operators combine values into expressions. Python's operators work on objects, and many change their behaviour with the type of their operands: + adds numbers but joins strings and lists. This module covers arithmetic (including floor division and modulus with negative numbers), comparison and chained comparisons, logical operators and short-circuit evaluation, assignment and the walrus operator, bitwise operators, identity and membership, operator overloading by type, precedence and associativity, and the built-in and math module functions.

IN THIS MODULE

4.1 Arithmetic operators
4.2 Comparison operators
4.3 Logical operators
4.4 Assignment operators
4.5 Bitwise operators
4.6 Identity and membership operators
4.7 Operator overloading by type
4.8 Precedence and associativity
4.9 Built-in math functions

4.1 Arithmetic operators

An **expression** is any combination of values, names, operators and function calls that Python evaluates to produce a value: `2 + 3`, `price * qty`, `len(name) > 3`. An **operator** is the symbol that performs the operation, and the values it works on are its **operands**.

4.1.1 + - * /

The four basic operators work as in mathematics: addition `+`, subtraction `-`, multiplication `*` and division `/`. Unary minus negates (`-x`) and unary plus leaves a number unchanged. When an `int` and a `float` are combined, the result is a `float`.

THE BASIC ARITHMETIC OPERATORS

main.py

```
a, b = 17, 5
print("a + b =", a + b)
print("a - b =", a - b)
print("a * b =", a * b)
print("a / b =", a / b)
print("-a =", -a, "| +a =", +a)
print("int + float:", 10 + 2.5, type(10 + 2.5).__name__)
```

OUTPUT

```
a + b = 22
a - b = 12
a * b = 85
a / b = 3.4
-a = -17 | +a = 17
int + float: 12.5 float
```

4.1.2 Floor division //

`//` divides and then rounds **down** to the nearest whole number ("floor"). With two ints the result is an int; if either operand is a float the result is a float with the fractional part removed (`7.5 // 2` is `3.0`). For negative numbers, "down" means towards minus infinity, so `-7 // 2` is `-4`, not -3 (C and Java truncate towards zero and give -3). Floor division is used to split things into whole groups: hours from minutes, pages needed, rows in a grid.

FLOOR DIVISION

main.py

```
print("17 // 5 =", 17 // 5)
print("7.5 // 2 =", 7.5 // 2)
print("-7 // 2 =", -7 // 2, "(rounded down, not towards zero)")
print("int(-7 / 2) =", int(-7 / 2), "(truncation, like C/Java)")

minutes = 135
print(f"{minutes} minutes = {minutes // 60} h {minutes % 60} min")
students, per_bench = 47, 3
print("benches needed:", -(-students // per_bench))      # ceiling division trick
```

OUTPUT

```
17 // 5 = 3
7.5 // 2 = 3.0
-7 // 2 = -4 (rounded down, not towards zero)
int(-7 / 2) = -3 (truncation, like C/Java)
135 minutes = 2 h 15 min
benches needed: 16
```

4.1.3 Modulus % with negative numbers

`%` gives the **remainder** after floor division, satisfying `a == (a // b) * b + (a % b)`. Common uses: testing divisibility (`n % 2 == 0` for even), cycling through a range (`(i + 1) % 7` for days of the week), extracting digits (`n % 10`). In Python the result of `%` always has the **same sign as the divisor**, so `-7 % 3` is `2` (in C and Java it is -1). This makes wrap-around arithmetic work naturally for negative numbers. `math.fmod()` gives the C-style result.

MODULUS AND ITS SIGN RULE main.py

```
import math

print("17 % 5 =", 17 % 5)
print("-7 % 3 =", -7 % 3, "| 7 % -3 =", 7 % -3)
print("math.fmod(-7, 3) =", math.fmod(-7, 3), "(C-style)")
a, b = -7, 3
print("check:", (a // b) * b + (a % b) == a)

days = ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"]
print("3 days before Tue is", days[(1 - 3) % 7])        # wraps around correctly
n = 4729
print("last digit:", n % 10, "| even?", n % 2 == 0, "| divisible by 3?", n % 3 == 0)
print("divmod(135, 60) =", divmod(135, 60))
```

OUTPUT

```
17 % 5 = 2
-7 % 3 = 2 | 7 % -3 = -2
math.fmod(-7, 3) = -1.0 (C-style)
check: True
3 days before Tue is Sat
last digit: 9 | even? False | divisible by 3? False
divmod(135, 60) = (2, 15)
```

Expression	Python	C / Java
`7 // 2`	3	3
`-7 // 2`	-4 (floor)	-3 (truncate)
`7 % 3`	1	1
`-7 % 3`	2 (sign of divisor)	-1 (sign of dividend)
`7 % -3`	-2	1

4.1.4 Exponent **

`**` raises to a power: `2 ** 10` is 1024. With integer operands and a non-negative exponent, the result is an exact integer of any size. A negative exponent gives a float (`2 ** -1` is 0.5), and a fractional exponent computes roots (`81 ** 0.5` is 9.0). `pow(base, exp, mod)` computes powers modulo a number efficiently, which cryptography relies on.

POWERS AND ROOTS main.py

```
print("2 ** 10 =", 2 ** 10)
print("2 ** 100 =", 2 ** 100)
print("2 ** -1 =", 2 ** -1)
print("81 ** 0.5 =", 81 ** 0.5, "| 27 ** (1/3) =", 27 ** (1 / 3))
print("(-8) ** (1/3) =", (-8) ** (1 / 3), "(complex result)")
print("pow(3, 200, 1000) =", pow(3, 200, 1000), "(last three digits of 3**200)")
```

OUTPUT

```
2 ** 10 = 1024
2 ** 100 = 1267650600228229401496703205376
2 ** -1 = 0.5
81 ** 0.5 = 9.0 | 27 ** (1/3) = 3.0
(-8) ** (1/3) = (1.0000000000000002+1.7320508075688772j) (complex result)
pow(3, 200, 1000) = 1 (last three digits of 3**200)
```

4.1.5 / always returns float

In Python 3, true division `/` **always** returns a `float`, even when the operands divide exactly: `10 / 2` is `5.0`. This avoids the old Python 2 surprise where `7 / 2` was 3. Use `//` when you want an integer result. Dividing by zero, with `/`, `//` or `%`, raises `ZeroDivisionError`.

TRUE DIVISION AND DIVISION BY ZERO main.py

```
print("10 / 2 =", 10 / 2, type(10 / 2).__name__)
print("7 / 2 =", 7 / 2)
print("10 // 2 =", 10 // 2, type(10 // 2).__name__)
for op in ("10 / 0", "10 // 0", "10 % 0"):
    try:
        eval(op)
    except ZeroDivisionError as e:
        print(f"{op:<8} -> ZeroDivisionError: {e}")
```

OUTPUT

```
10 / 2 = 5.0 float
7 / 2 = 3.5
10 // 2 = 5 int
10 / 0   -> ZeroDivisionError: division by zero
10 // 0  -> ZeroDivisionError: division by zero
10 % 0   -> ZeroDivisionError: division by zero
```

Operator	Name	Example	Result
`+`	addition	`17 + 5`	22
`-`	subtraction	`17 - 5`	12
`*`	multiplication	`17 * 5`	85
`/`	true division (float)	`17 / 5`	3.4
`//`	floor division	`17 // 5`	3
`%`	modulus (remainder)	`17 % 5`	2
`**`	exponent	`2 ** 5`	32
`-x`	negation	`-17`	-17

4.2 Comparison operators

4.2.1 == != > < >= <=

Comparison operators compare two values and return a `bool`. `==` tests equality of **values** (not to be confused with assignment `=`), `!=` inequality, and `< > <= >=` order. Numbers of different types compare by value (`1 == 1.0` is True). Strings compare **lexicographically** by Unicode code point, character by character, so `"apple" < "banana"` and `"Z" < "a"` (upper-case letters come first). Lists and tuples compare element by element. Comparing incompatible types for order, such as `5 < "5"`, raises a `TypeError` (equality simply returns False).

COMPARING NUMBERS, STRINGS AND SEQUENCES main.py

```
x, y = 10, 20
print(x == y, x != y, x < y, x > y, x <= 10, y >= 25)
print("1 == 1.0:", 1 == 1.0, "| 0.1 + 0.2 == 0.3:", 0.1 + 0.2 == 0.3)
print("'apple' < 'banana':", "apple" < "banana", "| 'Zebra' < 'apple':", "Zebra" < "apple")
print("ord('Z'), ord('a'):", ord("Z"), ord("a"))
print("'10' < '9':", "10" < "9", "(text comparison!)")
print("[1, 2, 3] < [1, 3]:", [1, 2, 3] < [1, 3], "| (2, 'b') > (2, 'a'):", (2, "b") > (2, "a"))
print("5 == '5':", 5 == "5")
print(5 < "5")
```

OUTPUT

```
False True True False True False
1 == 1.0: True | 0.1 + 0.2 == 0.3: False
'apple' < 'banana': True | 'Zebra' < 'apple': True
ord('Z'), ord('a'): 90 97
'10' < '9': True (text comparison!)
[1, 2, 3] < [1, 3]: True | (2, 'b') > (2, 'a'): True
5 == '5': False
Traceback (most recent call last):
  File "main.py", line 9, in <module>
    print(5 < "5")
          ^^^^^^^
TypeError: '<' not supported between instances of 'int' and 'str'
```

4.2.2 Chained comparisons 1 < x < 10

Python lets you chain comparisons as in mathematics: `1 < x < 10` means `1 < x and x < 10`, with `x` evaluated only once. Any comparison operators can be chained (`a == b == c`, `0 <= marks <= 100`), and evaluation stops at the first False. This is more readable than the long form and is not possible in C or Java, where `1 < x < 10` means something quite different.

CHAINED COMPARISONS main.py

```
for marks in (-5, 0, 55, 100, 120):
    print(f"{marks:>4} valid: {0 <= marks <= 100}")

age = 19
print("teenager:", 13 <= age <= 19)
a = b = c = 7
print("all equal:", a == b == c)
print("1 < 3 > 2:", 1 < 3 > 2, "(means 1 < 3 and 3 > 2)")
```

OUTPUT

```
  -5 valid: False
   0 valid: True
  55 valid: True
 100 valid: True
 120 valid: False
teenager: True
all equal: True
1 < 3 > 2: True (means 1 < 3 and 3 > 2)
```

4.3 Logical operators

4.3.1 and, or, not

`and`, `or` and `not` combine conditions, written as English words rather than symbols (C and Java use `&&`, `||`, `!`). `a and b` is true only if both are true; `a or b` is true if at least one is true; `not a` inverts. Precedence: `not` binds tightest, then `and`, then `or`; use parentheses when mixing them for clarity.

LOGICAL OPERATORS AND THEIR TRUTH TABLE

main.py

```
age, has_id, is_member = 20, True, False
print("can enter:", age >= 18 and has_id)
print("gets discount:", is_member or age < 21)
print("not a member:", not is_member)
print("not (a and b) == (not a) or (not b):", (not (True and False)) == ((not True) or (not False)))

print(f"{'a':<6}{'b':<6}{'a and b':<9}{'a or b':<8}not a")
for a in (True, False):
    for b in (True, False):
        print(f"{a!s:<6}{b!s:<6}{(a and b)!s:<9}{(a or b)!s:<8}{not a}")
```

OUTPUT

```
can enter: True
gets discount: True
not a member: True
not (a and b) == (not a) or (not b): True
a     b     a and b  a or b  not a
True  True  True     True    False
True  False False    True    False
False True  False    True    True
False False False    False   True
```

4.3.2 Short-circuit evaluation and returned values (x or default)

Python evaluates `and` and `or` from left to right and stops as soon as the result is known: **short-circuit evaluation**. In `a and b`, if `a` is falsy, `b` is never evaluated; in `a or b`, if `a` is truthy, `b` is skipped. This protects code such as `count > 0 and total / count > 50` from dividing by zero. Importantly, these operators return one of their **operands**, not necessarily `True` or `False`: `a or b` returns the first truthy operand (or the last one), and `a and b` returns the first falsy operand (or the last one). That gives the idiom `name = user_input or "Guest"` for defaults; be careful when 0 or an empty string are valid values (then use `if x is None`).

SHORT-CIRCUITING AND THE VALUES AND/OR RETURN

main.py

```
def check(label, result):
    print(f"  evaluated {label}")
    return result

print("False and ...:", check("left", False) and check("right", True))
print("True or ...:", check("left", True) or check("right", False))

count, total = 0, 0
print("safe average check:", count > 0 and total / count > 50)     # no ZeroDivisionError

print("'' or 'Guest' ->", repr("" or "Guest"))
print("'Asha' or 'Guest' ->", repr("Asha" or "Guest"))
print("0 or 10 or 20 ->", 0 or 10 or 20)
print("'a' and 'b' and 'c' ->", repr("a" and "b" and "c"))
print("[] and 'never' ->", repr([] and "never"))
volume = 0
print("pitfall: volume or 50 ->", volume or 50, "(0 was a valid volume!)")
print("better:", 50 if volume is None else volume)
```

OUTPUT

```
  evaluated left
False and ...: False
  evaluated left
True or ...: True
safe average check: False
'' or 'Guest' -> 'Guest'
'Asha' or 'Guest' -> 'Asha'
0 or 10 or 20 -> 10
'a' and 'b' and 'c' -> 'c'
[] and 'never' -> []
pitfall: volume or 50 -> 50 (0 was a valid volume!)
better: 0
```

4.4 Assignment operators

4.4.1 = and compound += -= *= /= //= %= **=

`=` binds a name to the value of an expression. **Compound assignment** operators combine an operation with assignment: `x += 5` means `x = x + 5`, and likewise `-=`, `*=`, `/=`, `//=`, `%=`, `**=` and the bitwise ones (`&= |= ^= <<= >>=`). Python has no `++` or `--` operators: write `count += 1`. For immutable objects `+=` creates a new object; for a list, `lst += [4]` modifies the list in place (like `extend`), which matters when the list is shared.

COMPOUND ASSIGNMENT

main.py

```
x = 10
x += 5;  print("+= 5 ->", x)
x -= 3;  print("-= 3 ->", x)
x *= 2;  print("*= 2 ->", x)
x /= 4;  print("/= 4 ->", x)
x //= 2; print("//= 2 ->", x)
x **= 3; print("**= 3 ->", x)
x %= 5;  print("%= 5 ->", x)

name = "Py"; name += "thon"; print(name)
a = [1, 2]; b = a
a += [3]                          # in place for lists: b sees it
print("a:", a, "| b:", b)
a = a + [4]                       # creates a NEW list: b does not see it
print("a:", a, "| b:", b)
```

OUTPUT

```
+= 5 -> 15
-= 3 -> 12
*= 2 -> 24
/= 4 -> 6.0
//= 2 -> 3.0
**= 3 -> 27.0
%= 5 -> 2.0
Python
a: [1, 2, 3] | b: [1, 2, 3]
a: [1, 2, 3, 4] | b: [1, 2, 3]
```

4.4.2 Walrus operator := (3.8)

The **assignment expression** `:=`, nicknamed the **walrus operator** because of its shape, assigns a value to a name **and** returns that value, inside a larger expression. It avoids computing something twice or writing an extra line, typically in `while` loops that read input, in `if` conditions, and in comprehensions. Use it where it makes code clearer, not everywhere. It cannot be used as a plain statement at the top level without parentheses.

ASSIGNMENT EXPRESSIONS WITH :=

main.py

```
import re

text = "Order ID: LBF-20931, amount Rs 4,999"
if (match := re.search(r"LBF-(\d+)", text)):          # assign and test in one step
    print("order number:", match.group(1))

data = [12, 7, 33, 4, 19]
if (n := len(data)) > 3:
    print(f"list is long ({n} items)")

values = [3, 8, 1, 9]
squares_over_10 = [sq for v in values if (sq := v * v) > 10]   # compute once, use twice
print(squares_over_10)
```

OUTPUT

```
order number: 20931
list is long (5 items)
[64, 81]
```

A WALRUS IN A WHILE LOOP THAT READS INPUT main.py

```
commands = []
while (line := input("command (blank to stop): ")) != "":
    commands.append(line.upper())
print("received:", commands)
```

CONSOLE (user input in gold)

```
command (blank to stop): start
command (blank to stop): status
command (blank to stop): stop
command (blank to stop):
received: ['START', 'STATUS', 'STOP']
```

4.5 Bitwise operators

4.5.1 & | ^ ~ << >>

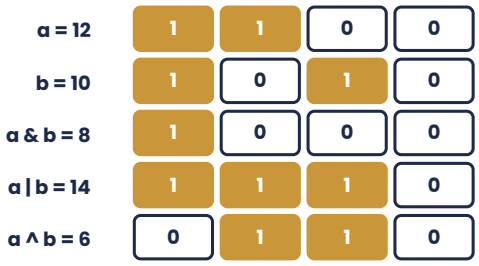
Bitwise operators work on the individual **bits** of integers. `&` (AND) sets a bit where both operands have it; `|` (OR) where either has it; `^` (XOR) where exactly one has it; `~` (NOT) inverts every bit, which for Python's integers gives `-x - 1`; `<< n` shifts left (multiplies by 2^n); `>> n` shifts right (floor-divides by 2^n). Uses: flags and permissions packed into one number, fast checks (`n & 1` is 1 for odd numbers), masks, networking, graphics and some interview problems. On sets, `& | ^ -` mean intersection, union, symmetric difference and difference (Module 9).

BITWISE OPERATORS AND FLAGS main.py

```
a, b = 12, 10                   # 1100 and 1010
print(f"a      = {a:>3} = {a:04b}")
print(f"b      = {b:>3} = {b:04b}")
print(f"a & b  = {a & b:>3} = {a & b:04b}")
print(f"a | b  = {a | b:>3} = {a | b:04b}")
print(f"a ^ b  = {a ^ b:>3} = {a ^ b:04b}")
print(f"~a     = {~a:>3}  (that is -a - 1)")
print(f"a << 2 = {a << 2:>3} = {a << 2:06b}  (x 4)")
print(f"a >> 2 = {a >> 2:>3} = {a >> 2:04b}    (// 4)")

READ, WRITE, EXECUTE = 4, 2, 1              # permission flags
perms = READ | WRITE
print("can write?", bool(perms & WRITE), "| can execute?", bool(perms & EXECUTE))
perms ^= WRITE                              # toggle write off
print("after toggling write:", perms, f"{perms:03b}")
```

OUTPUT

```
a      =  12 = 1100
b      =  10 = 1010
a & b  =   8 = 1000
a | b  =  14 = 1110
a ^ b  =   6 = 0110
~a     = -13  (that is -a - 1)
a << 2 =  48 = 110000  (x 4)
a >> 2 =   3 = 0011    (// 4)
can write? True | can execute? False
after toggling write: 4 100
```



bit by bit:
& both, | either, ^ exactly one

AND, OR AND XOR WORK COLUMN BY COLUMN

4.5.2 bin(), hex(), oct()

`bin()`, `hex()` and `oct()` return a string representation of an integer in base 2, 16 or 8, with the prefix `0b`, `0x` or `0o`. Format specifiers give more control: `f"{n:08b}"` pads to 8 binary digits without the prefix, `:x`/`:X` give lower/upper-case hex, and `#` adds the prefix. `int.bit_count()` counts the 1 bits and `int.bit_length()` the bits needed.

SHOWING NUMBERS IN BINARY, HEX AND OCTAL main.py

```
n = 200
print(bin(n), hex(n), oct(n))
print(f"{n:08b} | {n:x} | {n:X} | {n:#x} | {n:o}")
print("bits set:", n.bit_count(), "| bits needed:", n.bit_length())
print("negative:", bin(-5), "| byte pattern of -5:", f"{-5 & 0xFF:08b}")
print("IPv4 192.168.1.10 as one number:", (192 << 24) | (168 << 16) | (1 << 8) | 10)
```

OUTPUT

```
0b11001000 0xc8 0o310
11001000 | c8 | C8 | 0xc8 | 310
bits set: 3 | bits needed: 8
negative: -0b101 | byte pattern of -5: 11111011
IPv4 192.168.1.10 as one number: 3232235786
```

4.6 Identity and membership operators

4.6.1 is, is not

`a is b` is True when both names refer to the **same object** (the same `id()`); `is not` is its negation. Use them to test for singletons: `x is None`, `x is not None`, and occasionally `is True`/`is False` or sentinel objects. Do not use `is` for comparing numbers, strings or other values (Module 3, 3.2.3).

4.6.2 in, not in

`x in collection` tests **membership**: True if `x` is an element of a list, tuple, set or range, a **key** of a dictionary, or a **substring** of a string. `not in` is its negation. Membership in a list or tuple checks items one by one (slow for large lists); in a set or dict it uses hashing and is almost instant, so convert to a set when you test membership many times.

MEMBERSHIP TESTS main.py

```
courses = ["Python", "Java", "SQL"]
print("'Java' in courses:", "Java" in courses, "| 'C++' not in courses:", "C++" not in courses)
print("'thon' in 'Python':", "thon" in "Python", "| 'py' in 'Python':", "py" in "Python")
marks = {"Asha": 91, "Ravi": 78}
print("'Asha' in marks (keys):", "Asha" in marks, "| 91 in marks:", 91 in marks, "| 91 in values:", 91
in marks.values())
print("7 in range(0, 20, 7):", 7 in range(0, 20, 7))

import time
big_list = list(range(1_000_000)); big_set = set(big_list)
t = time.perf_counter(); 999_999 in big_list; t_list = time.perf_counter() - t
t = time.perf_counter(); 999_999 in big_set; t_set = time.perf_counter() - t
print("set lookup faster than list scan:", t_set < t_list)
```

OUTPUT

```
'Java' in courses: True | 'C++' not in courses: True
'thon' in 'Python': True | 'py' in 'Python': False
'Asha' in marks (keys): True | 91 in marks: False | 91 in values: True
7 in range(0, 20, 7): True
set lookup faster than list scan: True
```

4.6.3 is vs ==

`==` asks "do these have the same **value**?" and `is` asks "are these the very **same object**?". Two different lists with equal contents are `==` but not `is`; two names bound to one list are both. Classes decide what `==` means (by defining `__eq__`), while `is` can never be overridden.

EQUALITY VERSUS IDENTITY main.py

```
a = [1, 2, 3]
b = [1, 2, 3]
c = a
print("a == b:", a == b, "| a is b:", a is b)
print("a == c:", a == c, "| a is c:", a is c)

x = None
print("x is None:", x is None)
print("1000 == 1000.0:", 1000 == 1000.0, "| 1000 is 1000.0:", 1000 is 1000.0)
```

OUTPUT

```
main.py:9: SyntaxWarning: "is" with 'int' literal. Did you mean "=="?
  print("1000 == 1000.0:", 1000 == 1000.0, "| 1000 is 1000.0:", 1000 is 1000.0)
a == b: True | a is b: False
a == c: True | a is c: True
x is None: True
1000 == 1000.0: True | 1000 is 1000.0: False
```

Question	Operator	Example
same value?	`==`, `!=`	`a == [1, 2, 3]`
same object?	`is`, `is not`	`result is None`
contained?	`in`, `not in`	`"py" in name`, `key in dict`

4.7 Operator overloading by type

4.7.1 + and * on strings and lists

Operators mean different things for different types: this is **operator overloading**. `+` adds numbers but **concatenates** (joins) strings, lists and tuples; `*` multiplies numbers but **repeats** a sequence when combined with an integer. The operand types must make sense together: `"a" + 1` and `[1] + (2,)` raise `TypeError`. Beware that `[[0] * 3] * 3` repeats references to the **same** inner list. Your own classes can define what operators do by implementing special methods such as `__add__` (Module 16).

THE SAME OPERATOR, DIFFERENT MEANINGS

main.py

```
print("Py" + "thon", "|", "ab" * 3, "|", "-" * 20)
print([1, 2] + [3, 4], "|", [0] * 5, "|", (1, 2) * 2)
print("concatenating mixed types fails:")
for expr in ('"Total: " + 50', '[1, 2] + (3,)', '"abc" * 2.5'):
    try:
        eval(expr)
    except TypeError as e:
        print(f"  {expr:<15} -> TypeError: {e}")
print("Total: " + str(50))

grid = [[0] * 3] * 3                  # three references to ONE row
grid[0][0] = 9
print("shared rows:", grid)
grid = [[0] * 3 for _ in range(3)]    # three separate rows
grid[0][0] = 9
print("separate rows:", grid)
```

OUTPUT

```
Python | ababab | --------------------
[1, 2, 3, 4] | [0, 0, 0, 0, 0] | (1, 2, 1, 2)
concatenating mixed types fails:
  "Total: " + 50  -> TypeError: can only concatenate str (not "int") to str
  [1, 2] + (3,)   -> TypeError: can only concatenate list (not "tuple") to list
  "abc" * 2.5     -> TypeError: can't multiply sequence by non-int of type 'float'
Total: 50
shared rows: [[9, 0, 0], [9, 0, 0], [9, 0, 0]]
separate rows: [[9, 0, 0], [0, 0, 0], [0, 0, 0]]
```

4.8 Precedence and associativity

4.8.1 Precedence table

Precedence decides which operator is applied first in an expression with several operators: `2 + 3 * 4` is 14, because `*` has higher precedence than `+`. **Associativity** decides the order for operators of the same precedence: most are left-to-right (`10 - 4 - 3` is `(10 - 4) - 3` = 3). Parentheses override everything and make intentions clear, so use them whenever an expression is not obvious. The table lists the main operators from highest to lowest precedence.

Precedence	Operators	Description
1 (highest)	`( )`	parentheses, grouping
2	`x[i]`, `x.attr`, `f(x)`	subscription, attribute, call

Precedence	Operators	Description
3	`await`	await expression
4	`**`	exponent (right-associative)
5	`+x`, `-x`, `~x`	unary plus, minus, bitwise NOT
6	`* / // % @`	multiplication, division, remainder, matrix multiply
7	`+ -`	addition, subtraction
8	`<< >>`	shifts
9	`&`	bitwise AND
10	`^`	bitwise XOR
11	`|`	bitwise OR
12	`== != < > <= >= is in` and negations	comparisons, identity, membership (chainable)
13	`not`	logical NOT
14	`and`	logical AND
15	`or`	logical OR
16	`x if c else y`	conditional expression
17	`lambda`	lambda expression
18 (lowest)	`:=`	assignment expression

PRECEDENCE IN PRACTICE

main.py

```
print("2 + 3 * 4 =", 2 + 3 * 4, "| (2 + 3) * 4 =", (2 + 3) * 4)
print("10 - 4 - 3 =", 10 - 4 - 3, "(left to right)")
print("100 / 10 / 5 =", 100 / 10 / 5)
print("-3 ** 2 =", -3 ** 2, "| (-3) ** 2 =", (-3) ** 2)
print("not True == False:", not True == False, "(not applies after ==)")
print("True or False and False:", True or False and False, "(and before or)")
print("5 & 3 == 1:", 5 & 3 == 1, "(bitwise & before ==, unlike C)")
```

OUTPUT

```
2 + 3 * 4 = 14 | (2 + 3) * 4 = 20
10 - 4 - 3 = 3 (left to right)
100 / 10 / 5 = 2.0
-3 ** 2 = -9 | (-3) ** 2 = 9
not True == False: True (not applies after ==)
True or False and False: True (and before or)
5 & 3 == 1: True (bitwise & before ==, unlike C)
```

WATCH OUT

`-3 ** 2` is **-9**, because `**` binds tighter than unary minus: it means `-(3 ** 2)`. Also note a difference from C, Java and JavaScript: in Python the bitwise operators `& | ^` bind **tighter** than comparisons, so `5 & 3 == 1` means `(5 & 3) == 1` (True), whereas in C it means `5 & (3 == 1)` (0). When in doubt, add parentheses.

4.8.2 Right associativity of **

The exponent operator is **right-associative**: `2 ** 3 ** 2` is evaluated as `2 ** (3 ** 2)` = 2^9 = 512, not `(2 ** 3) ** 2` = 64. This matches mathematical notation, where a tower of exponents is evaluated from the top down. Comparison operators are not associative in the usual sense: they chain (4.2.2).

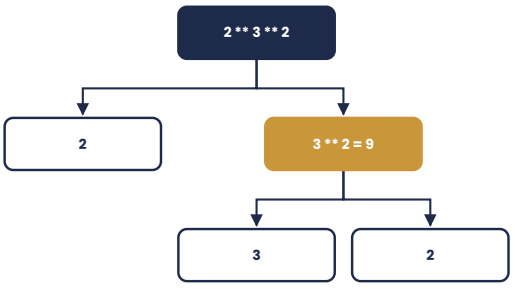
**** GROUPS FROM THE RIGHT**

main.py

```
print("2 ** 3 ** 2 =", 2 ** 3 ** 2)
print("(2 ** 3) ** 2 =", (2 ** 3) ** 2)
print("2 ** (3 ** 2) =", 2 ** (3 ** 2))
```

OUTPUT

```
2 ** 3 ** 2 = 512
(2 ** 3) ** 2 = 64
2 ** (3 ** 2) = 512
```



RIGHT ASSOCIATIVITY: THE UPPER POWER IS COMPUTED FIRST (RESULT 512)

4.9 Built-in math functions

4.9.1 abs, round, pow, divmod, min, max, sum

Several mathematical functions are built in, with no import needed. `abs(x)` gives the absolute value (and the magnitude of a complex number); `round(x, n)` rounds to n decimal places (banker's rounding, 3.5.5), or to the nearest integer when n is omitted (negative n rounds to tens, hundreds…); `pow(x, y[, mod])` raises to a power; `divmod(a, b)` returns the pair `(a // b, a % b)`; `min()` and `max()` take several arguments or one iterable, with optional `key=` and `default=`; `sum(iterable, start=0)` adds items.

BUILT-IN NUMERIC FUNCTIONS

main.py

```
print("abs(-7.5):", abs(-7.5), "| abs(3+4j):", abs(3 + 4j))
print("round(3.14159, 2):", round(3.14159, 2), "| round(1234.5):", round(1234.5), "| round(1234,
-2):", round(1234, -2))
print("pow(2, 8):", pow(2, 8), "| pow(2, 8, 10):", pow(2, 8, 10))
print("divmod(17, 5):", divmod(17, 5), "| divmod(-17, 5):", divmod(-17, 5))
marks = [72, 95, 88, 61]
print("min, max, sum:", min(marks), max(marks), sum(marks), "| min(4, 9, 2):", min(4, 9, 2))
names = ["Asha", "Meera", "Jo"]
print("longest name:", max(names, key=len), "| empty with default:", max([], default=0))
print("sum with a start value:", sum([0.5, 0.25], start=10))
```

OUTPUT

```
abs(-7.5): 7.5 | abs(3+4j): 5.0
round(3.14159, 2): 3.14 | round(1234.5): 1234 | round(1234, -2): 1200
pow(2, 8): 256 | pow(2, 8, 10): 6
divmod(17, 5): (3, 2) | divmod(-17, 5): (-4, 3)
min, max, sum: 61 95 316 | min(4, 9, 2): 2
longest name: Meera | empty with default: 0
sum with a start value: 10.75
```

4.9.2 The math module: sqrt, ceil, floor, factorial, gcd, pi

The standard `math` module provides more mathematical functions and constants for real numbers: `sqrt` and `isqrt` (integer square root), `ceil` (round up) and `floor` (round down), `trunc`, `factorial`, `gcd` and `lcm`, `comb` and `perm` (combinations and permutations), `log`, `log10`, `log2` and `exp`, trigonometry in radians (`sin`, `cos`, `radians`, `degrees`), `hypot`, `dist`, `prod`, `isclose`, and constants `pi`, `e`, `tau`, `inf` and `nan`. Import it with `import math` and call functions as `math.sqrt(2)`.

A TOUR OF THE MATH MODULE

main.py

```
import math

print("sqrt(2):", math.sqrt(2), "| isqrt(17):", math.isqrt(17))
print("ceil(4.1):", math.ceil(4.1), "| floor(4.9):", math.floor(4.9), "| floor(-4.1):",
math.floor(-4.1), "| trunc(-4.9):", math.trunc(-4.9))
print("factorial(10):", math.factorial(10))
print("gcd(48, 36):", math.gcd(48, 36), "| lcm(4, 6):", math.lcm(4, 6))
print("comb(5, 2):", math.comb(5, 2), "| perm(5, 2):", math.perm(5, 2))
print("pi:", math.pi, "| e:", math.e)
print("log10(1000):", math.log10(1000), "| log2(1024):", math.log2(1024), "| log(e):",
math.log(math.e))
print("sin(30 deg):", round(math.sin(math.radians(30)), 4), "| hypot(3, 4):", math.hypot(3, 4))
print("prod([1, 2, 3, 4]):", math.prod([1, 2, 3, 4]), "| dist((0,0), (3,4)):", math.dist((0, 0), (3,
4)))

radius = 7
print(f"circle of radius {radius}: area {math.pi * radius ** 2:.2f}, circumference {2 * math.pi *
radius:.2f}")
```

OUTPUT

```
sqrt(2): 1.4142135623730951 | isqrt(17): 4
ceil(4.1): 5 | floor(4.9): 4 | floor(-4.1): -5 | trunc(-4.9): -4
factorial(10): 3628800
gcd(48, 36): 12 | lcm(4, 6): 12
comb(5, 2): 10 | perm(5, 2): 20
pi: 3.141592653589793 | e: 2.718281828459045
log10(1000): 3.0 | log2(1024): 10.0 | log(e): 1.0
sin(30 deg): 0.5 | hypot(3, 4): 5.0
prod([1, 2, 3, 4]): 24 | dist((0,0), (3,4)): 5.0
circle of radius 7: area 153.94, circumference 43.98
```

Function	Result	Example
`math.sqrt(x)`	square root (float)	`sqrt(16)` → 4.0
`math.ceil(x)` / `floor(x)`	round up / down to int	`ceil(4.1)` → 5
`math.factorial(n)`	n!	`factorial(5)` → 120
`math.gcd(a, b)` / `lcm`	greatest common divisor / least common multiple	`gcd(48, 36)` → 12
`math.pi`, `math.e`	constants	3.14159..., 2.71828...
`math.log(x, base)`	logarithm	`log(8, 2)` → 3.0
`math.isclose(a, b)`	float comparison	`isclose(0.1+0.2, 0.3)` → True

Arithmetic operators are + - * /, // (floor division, rounding towards minus infinity), % (remainder with the sign of the divisor) and ** (powers, exact for ints); / always returns a float and dividing by zero raises ZeroDivisionError. Comparisons return bools, compare strings by code point and sequences element by element, and can be chained (0 <= x <= 100). and, or, not short-circuit and return one of their operands, giving the x or default idiom (careful with valid zeros). Compound assignment (+= etc.) replaces ++; the walrus := assigns inside expressions. Bitwise & | ^ ~ << >> work on bits; bin, hex and oct show them. is tests identity (use it for None), == tests values, in tests membership (fast for sets and dicts). + and * concatenate and repeat strings, lists and tuples. Precedence runs from ** (right-associative) through unary, multiplicative, additive, shifts, bitwise, comparisons, not, and, or; use parentheses. abs, round, pow, divmod, min, max and sum are built in; the math module adds sqrt, ceil, floor, factorial, gcd, pi and more.

PRACTICE CHECKPOINT

Build a **calculator** using all arithmetic operators, an **even/odd check** with `%` and `&`, and a demonstration of **is vs ==**.

SAMPLE SOLUTION

main.py

```
a = float(input("First number: "))
b = float(input("Second number: "))

print(f"\n{'Operation':<16}Result")
print(f"{'a + b':<16}{a + b}")
print(f"{'a - b':<16}{a - b}")
print(f"{'a * b':<16}{a * b}")
print(f"{'a ** b':<16}{a ** b}")
if b != 0:
    print(f"{'a / b':<16}{a / b:.4f}")
    print(f"{'a // b':<16}{a // b}")
    print(f"{'a % b':<16}{a % b}")
else:
    print("division operations skipped: the second number is zero")

print()
for n in (7, 10, -3, 0):
    by_mod = "even" if n % 2 == 0 else "odd"
    by_bit = "even" if n & 1 == 0 else "odd"
    print(f"{n:>3}: % says {by_mod:<4} | & says {by_bit:<4} | agree: {by_mod == by_bit}")

print()
x = [1, 2, 3]
y = [1, 2, 3]
z = x
print("x == y:", x == y, "-> same values")
print("x is y:", x is y, "-> different list objects")
print("x is z:", x is z, "-> z is another name for x")
value = None
print("value is None:", value is None, "-> the right way to test for None")
```

CONSOLE (user input in gold)

```
First number: 17
Second number: 5

Operation       Result
a + b           22.0
a - b           12.0
a * b           85.0
a ** b          1419857.0
a / b           3.4000
a // b          3.0
a % b           2.0

  7: % says odd  | & says odd  | agree: True
 10: % says even | & says even | agree: True
 -3: % says odd  | & says odd  | agree: True
  0: % says even | & says even | agree: True

x == y: True -> same values
x is y: False -> different list objects
x is z: True -> z is another name for x
value is None: True -> the right way to test for None
```

MODULE 05 · BEGINNER

Input, output and type conversion

Minimum duration **1 day** | Level **Beginner**

This module makes programs interactive. It covers printing with print() and its sep, end and file parameters, every way of formatting strings (f-strings with format specifiers and debug output, str.format(), %-formatting and the new template strings of Python 3.14), reading keyboard input with input(), converting between types implicitly and explicitly (including ord() and chr()), handling conversion errors, and reading command-line arguments with sys.argv.

IN THIS MODULE

5.1 Output with print()
5.2 String formatting
5.3 Input with input()
5.4 Type conversion
5.5 Command-line arguments

NOTE
In examples that read from the keyboard, the text typed by the user is shown in **gold**.

5.1 Output with print()

5.1.1 Printing multiple values

`print()` writes text to the screen (more precisely, to standard output). It accepts any number of arguments of any type, converts each to a string with `str()`, and prints them separated by a space, followed by a newline. With no arguments it prints an empty line. Its full signature is `print(*objects, sep=" ", end="\n", file=sys.stdout, flush=False)`.

PRINT() WITH SEVERAL ARGUMENTS — main.py

```
name, marks, passed = "Asha", 91.5, True
print("Student:", name)
print(name, marks, passed, [1, 2], None)       # any types, separated by spaces
print()                                         # empty line
print("Total =", 45 + 46)
```

OUTPUT

```
Student: Asha
Asha 91.5 True [1, 2] None

Total = 91
```

5.1.2 sep and end parameters

The keyword argument `sep` sets the separator printed **between** arguments (default a space), and `end` sets what is printed **after** the last argument (default a newline `"\n"`). `sep=""` joins values with nothing

between them, `sep="\n"` prints each on its own line, and `end=""` or `end=" "` keeps the next `print` on the same line. Escape sequences such as `\n` (newline) and `\t` (tab) work inside strings.

CONTROLLING SEPARATORS AND LINE ENDINGS main.py

```
print("2026", "03", "09", sep="-")
print("Python", "Java", "SQL", sep=" | ")
print("a", "b", "c", sep="")
print("one", "two", "three", sep="\n")

for i in range(1, 6):
    print(i, end=" ")              # stay on the same line
print()                            # finish the line
print("Loading", end="")
for _ in range(3):
    print(".", end="")
print(" done")
print("Name\tMarks\nAsha\t91")
```

OUTPUT

```
2026-03-09
Python | Java | SQL
abc
one
two
three
1 2 3 4 5
Loading... done
Name    Marks
Asha    91
```

5.1.3 Printing to a file with file=

The `file` parameter sends output somewhere other than the screen: an open file, or `sys.stderr`, the standard error stream used for error messages (it is shown on screen but kept separate from normal output, so it is not captured when output is redirected to a file). `flush=True` forces the text out immediately, useful for progress messages. Module 14 covers files in depth.

PRINTING INTO A FILE AND TO STANDARD ERROR main.py

```
import sys

with open("report.txt", "w") as f:             # open a file for writing
    print("REPORT CARD", file=f)
    print("Asha", 91, sep=",", file=f)
    print("Ravi", 78, sep=",", file=f)

print("Warning: 2 students absent", file=sys.stderr)
print("report.txt contains:")
print(open("report.txt").read(), end="")
```

OUTPUT

```
Warning: 2 students absent
report.txt contains:
REPORT CARD
Asha,91
Ravi,78
```

5.2 String formatting

Formatting means building a string from text and values, with control over how each value looks: decimal places, alignment, thousands separators. Python has four ways, from the modern recommended one to the legacy ones you will meet in older code.

5.2.1 f-strings (3.6+)

A **formatted string literal**, or **f-string**, has the prefix `f` and contains expressions in braces: `f"Hello {name}"`. Python evaluates each expression and inserts its string form. Any expression works: arithmetic, function and method calls, indexing, conditional expressions. f-strings are the fastest, most readable formatting method and should be your default. Since Python 3.12 the expressions can even contain the same quote character and span several lines. To print a literal brace, double it: `{{` and `}}`.

F-STRINGS WITH EXPRESSIONS main.py

```
name, course, fee = "Asha", "Python", 14999
print(f"{name} enrolled in {course}.")
print(f"Fee with 18% GST: {fee * 1.18}")
print(f"Upper case: {name.upper()} | length: {len(name)} | first letter: {name[0]}")
print(f"Result: {'pass' if fee > 0 else 'free'}")
scores = {"maths": 91}
print(f"Maths: {scores["maths"]}")              # same quotes inside allowed since 3.12
print(f"Braces: {{literal}} and value {fee}")
```

OUTPUT

```
Asha enrolled in Python.
Fee with 18% GST: 17698.82
Upper case: ASHA | length: 4 | first letter: A
Result: pass
Maths: 91
Braces: {literal} and value 14999
```

5.2.2 Format specifiers: width, alignment, precision, :,, :%, :b, :x

After the expression, a colon introduces a **format specifier** that controls the appearance: `f"{value:spec}"`. Its parts, in order, are optional: **fill** character and **alignment** (`<` left, `>` right, `^` centre), **sign** (`+` to always show it), **width** (minimum characters), **grouping** (`,` or `_` for thousands), **.precision** (digits after the decimal point), and **type**: `d` integer, `f` fixed-point, `e` scientific, `%` percentage, `b` binary, `o` octal, `x`/`X` hex, `s` string. Width and precision can themselves be expressions in nested braces.

THE FORMAT SPECIFICATION MINI-LANGUAGE

main.py

```
price = 1234567.891
print(f"[{price:.2f}]")             # 2 decimal places
print(f"[{price:,.2f}]")            # thousands separator
print(f"[{price:_.1f}]")
print(f"[{price:15,.2f}]")          # width 15, right-aligned numbers by default
print(f"[{price:e}] [{0.000123:.2e}]")

name = "Asha"
print(f"[{name:<10}] [{name:>10}] [{name:^10}] [{name:*^10}]")
print(f"[{42:05d}] [{42:+d}] [{-42:+d}] [{42:>6}]")
print(f"[{0.8567:.1%}] [{0.05:%}]")
print(f"[{255:b}] [{255:o}] [{255:x}] [{255:X}] [{255:#x}] [{5:08b}]")
width, decimals = 12, 3
print(f"[{3.14159265:{width}.{decimals}f}]")    # nested: width and precision from variables
```

OUTPUT

```
[1234567.89]
[1,234,567.89]
[1_234_567.9]
[   1,234,567.89]
[1.234568e+06] [1.23e-04]
[Asha      ] [      Asha] [   Asha   ] [***Asha***]
[00042] [+42] [-42] [    42]
[85.7%] [5.000000%]
[11111111] [377] [ff] [FF] [0xff] [00000101]
[       3.142]
```

Specifier	Meaning	Example	Result
`:.2f`	2 decimal places	`f"{3.14159:.2f}"`	3.14
`:10`	minimum width 10	`f"{42:10}"`	␣␣␣␣␣␣␣␣42
`:<10` `:>10` `:^10`	left / right / centre	`f"{"ab":^6}"`	␣␣ab␣␣
`:*^10`	fill character	`f"{"ab":*^6}"`	**ab**
`:,` `:_`	thousands separator	`f"{1234567:,}"`	1,234,567
`:05d`	zero padding	`f"{42:05d}"`	00042
`:+`	always show the sign	`f"{5:+}"`	+5
`:%` `:.1%`	percentage	`f"{0.856:.1%}"`	85.6%
`:e`	scientific notation	`f"{1234.5:e}"`	1.234500e+03
`:b` `:o` `:x`	binary, octal, hex	`f"{255:x}"`	ff

A NEATLY ALIGNED TABLE WITH FORMAT SPECIFIERS

main.py

```
students = [("Asha Rao", 91.456, 14999), ("Ravi Kumar", 78.2, 12999), ("Meera N", 85.0, 9999)]
print(f"{'Name':<12}{'Marks':>8}{'Fee (Rs)':>12}")
print("-" * 32)
for name, marks, fee in students:
    print(f"{name:<12}{marks:>8.1f}{fee:>12,}")
print("-" * 32)
total = sum(f for *_, f in students)
print(f"{'Total':<12}{'':>8}{total:>12,}")
```

OUTPUT

```
Name           Marks    Fee (Rs)
--------------------------------
Asha Rao        91.5      14,999
Ravi Kumar      78.2      12,999
Meera N         85.0       9,999
--------------------------------
Total                     37,997
```

5.2.3 Debug f-strings f"{x=}"

Adding `=` after an expression in an f-string (Python 3.8+) prints the expression text, an equals sign and its value: `f"{marks=}"` gives `marks=91`. It is a quick way to inspect variables while debugging. It combines with format specifiers (`f"{avg=:.2f}"`) and uses `repr()` by default, so strings appear in quotes. Spaces around `=` are kept in the output.

SELF-DOCUMENTING DEBUG OUTPUT

main.py

```
marks = [91, 78, 85]
avg = sum(marks) / len(marks)
name = "Asha"
print(f"{name=}")
print(f"{avg=}")
print(f"{avg=:.2f}")
print(f"{len(marks) = }")
print(f"{max(marks) - min(marks)=}")
```

OUTPUT

```
name='Asha'
avg=84.66666666666667
avg=84.67
len(marks) = 3
max(marks) - min(marks)=13
```

5.2.4 str.format()

Before f-strings, `"...".format()` was the standard method, and it is still useful when the template is stored separately from the data (for example read from a file or configuration, or reused many times). Braces mark replacement fields, filled by position (`{}`, `{0}`) or by keyword (`{name}`), and the same format specifiers work after a colon. `format_map(dict)` fills fields from a dictionary.

THE FORMAT() METHOD

main.py

```
print("{} scored {} marks".format("Asha", 91))
print("{1} before {0}".format("second", "first"))
print("{name} paid Rs {fee:,.2f}".format(name="Ravi", fee=12999))

template = "Dear {student}, your {course} class starts on {date}."      # reusable template
for row in [{"student": "Asha", "course": "Python", "date": "9 March"},
            {"student": "Meera", "course": "SQL", "date": "16 March"}]:
    print(template.format_map(row))
```

OUTPUT

```
Asha scored 91 marks
first before second
Ravi paid Rs 12,999.00
Dear Asha, your Python class starts on 9 March.
Dear Meera, your SQL class starts on 16 March.
```

5.2.5 % formatting (legacy)

The oldest style, borrowed from C's `printf`, uses the `%` operator: `"%s scored %d" % (name, marks)`, with conversion codes such as `%s` (string), `%d` (integer), `%f` (float, `%.2f` for 2 decimals), `%x` and `%%` for a literal percent sign. You will see it in older code and in the `logging` module, but do not use it in new code: it is less readable and error-prone.

LEGACY %-FORMATTING

main.py

```
name, marks, avg = "Asha", 91, 87.456
print("%s scored %d marks" % (name, marks))
print("Average: %.2f | width 8: [%8.1f] | hex: %x | percent: %d%%" % (avg, avg, 255, 95))
```

OUTPUT

```
Asha scored 91 marks
Average: 87.46 | width 8: [    87.5] | hex: ff | percent: 95%
```

Method	Syntax	When to use
f-string	`f"{name} has {marks:.1f}"`	default choice: readable and fast
str.format()	`"{} has {:.1f}".format(name, marks)`	templates kept separately from data
% formatting	`"%s has %.1f" % (name, marks)`	legacy code, logging messages
string.Template	`Template("$name has $marks").substitute(...)`	simple user-supplied templates
t-string (3.14)	`t"{name} has {marks}"`	safe processing of interpolations (HTML, SQL, logs)

5.2.6 Template strings (t-strings, 3.14, overview)

Python 3.14 adds **template string literals** (PEP 750), written with the prefix `t`: `t"Hello {name}"`. They look like f-strings, but instead of immediately producing a `str`, they produce a `string.templatelib.Template` object that keeps the literal text parts and the interpolated values **separate**. Code can then process the values before combining them: escaping user input for HTML (preventing cross-site scripting), building parameterised SQL queries (preventing SQL injection), structured logging, or translation. A t-string is not a string itself; you pass it to a function that knows how to render it. (This is different from the older `string.Template` class with `$name` placeholders.)

A T-STRING KEEPS VALUES SEPARATE SO THEY CAN BE ESCAPED

main.py

```
from string.templatelib import Template, Interpolation
import html

def render_html(template: Template) -> str:
    parts = []
    for item in template:                          # iterate: strings and Interpolation objects
        if isinstance(item, Interpolation):
            parts.append(html.escape(str(item.value)))   # escape only the inserted values
        else:
            parts.append(item)
    return "".join(parts)

comment = "<script>alert('hacked')</script>"
user = "Asha"
t = t"<p><b>{user}</b> wrote: {comment}</p>"
print(type(t).__name__, "| static parts:", t.strings)
print("values:", [i.value for i in t.interpolations])
print("f-string (unsafe):", f"<p><b>{user}</b> wrote: {comment}</p>")
print("t-string rendered safely:", render_html(t))
```

OUTPUT

```
Template | static parts: ('<p><b>', '</b> wrote: ', '</p>')
values: ['Asha', "<script>alert('hacked')</script>"]
f-string (unsafe): <p><b>Asha</b> wrote: <script>alert('hacked')</script></p>
t-string rendered safely: <p><b>Asha</b> wrote: &lt;script&gt;alert(&#x27;hacked&#x27;)&lt;/script&gt;
</p>
```

5.3 Input with input()

5.3.1 input() always returns a string

`input(prompt)` displays the prompt, waits for the user to type a line and press Enter, and returns what they typed as a **string**, without the newline. It always returns a `str`, even if the user types digits: `"25"`, not `25`. Forgetting this is the most common beginner bug: adding two "numbers" read with `input()` joins them as text. Include a space or colon at the end of the prompt for readability.

INPUT() RETURNS TEXT

main.py

```
name = input("Enter your name: ")
age = input("Enter your age: ")
print("Hello,", name)
print("type of age:", type(age))
print("age + 1 as text:", age + "1")         # joins strings!
```

CONSOLE (user input in gold)

```
Enter your name: Asha
Enter your age: 21
Hello, Asha
type of age: <class 'str'>
age + 1 as text: 211
```

5.3.2 Reading numbers with int(input())

To read a number, convert the string with `int()` or `float()`, usually in the same line: `age = int(input("Age: "))`. `int()` accepts surrounding spaces and a sign but not a decimal point; `float()`

accepts decimals and scientific notation. Use `float()` for prices, heights and marks that can have decimals.

CONVERTING INPUT TO NUMBERS main.py

```
age = int(input("Age: "))
height = float(input("Height in metres: "))
print("Next year you will be", age + 1)
print(f"Height in cm: {height * 100:.0f}")
print("types:", type(age).__name__, type(height).__name__)
```

CONSOLE (user input in gold)

```
Age: 21
Height in metres: 1.62
Next year you will be 22
Height in cm: 162
types: int float
```

5.3.3 Reading multiple values with split() and map()

To read several values typed on one line, `split()` breaks the string into a list of words (on any whitespace by default, or on a given separator such as `","`). Unpack them into variables, converting each one, or apply a conversion to all of them with `map(int, ...)`, which calls `int` on every item. A list comprehension (`[int(x) for x in ...]`) does the same and is often clearer.

SEVERAL VALUES ON ONE LINE main.py

```
a, b = input("Two numbers separated by a space: ").split()
print("as strings:", a, b, "| sum:", int(a) + int(b))

x, y, z = map(int, input("Three integers: ").split())
print("product:", x * y * z)

marks = list(map(float, input("Marks separated by commas: ").split(",")))
print("marks:", marks, "| average:", sum(marks) / len(marks))

name, city = [part.strip() for part in input("name, city: ").split(",")]
print(f"{name} lives in {city}")
```

CONSOLE (user input in gold)

```
Two numbers separated by a space: 4 9
as strings: 4 9 | sum: 13
Three integers: 2 3 7
product: 42
Marks separated by commas: 91.5,78,85
marks: [91.5, 78.0, 85.0] | average: 84.83333333333333
name, city: Ravi Kumar , Pune
Ravi Kumar lives in Pune
```

5.3.4 Handling invalid input (preview of try-except)

If the user types something that cannot be converted, `int()` raises a `ValueError` and the program stops. Real programs must expect bad input. The `try` / `except` statement (fully covered in Module 13) catches the error so the program can show a message and ask again, usually in a loop. String methods such as `isdigit()` can also check input before converting it.

ASKING AGAIN UNTIL THE INPUT IS VALID main.py

```
while True:
    text = input("Enter your age: ")
    try:
        age = int(text)
    except ValueError:
        print(f"  '{text}' is not a whole number, please try again")
        continue
    if not 1 <= age <= 120:
        print("  age must be between 1 and 120")
        continue
    break
print("Thank you, age recorded as", age)
```

CONSOLE (user input in gold)

```
Enter your age: twenty
  'twenty' is not a whole number, please try again
Enter your age: 25.5
  '25.5' is not a whole number, please try again
Enter your age: 150
  age must be between 1 and 120
Enter your age: 25
Thank you, age recorded as 25
```

WHAT HAPPENS WITHOUT ERROR HANDLING main.py

```
age = int(input("Age: "))
```

CONSOLE (user input in gold)

```
Age: abc
Traceback (most recent call last):
  File "main.py", line 1, in <module>
    age = int(input("Age: "))
ValueError: invalid literal for int() with base 10: 'abc'
```

5.4 Type conversion

5.4.1 Implicit conversion (int + float)

Implicit conversion (coercion) happens automatically only in a few safe cases: in arithmetic mixing `int` and `float`, the int is converted to float; mixing with `complex` gives complex; and `bool` acts as int. Python never converts between strings and numbers implicitly (it is strongly typed), so `"5" + 2` is an error, not 7 or "52".

AUTOMATIC CONVERSIONS AND THEIR LIMITS

main.py

```
print(3 + 4.5, type(3 + 4.5).__name__)
print(2 * (1 + 2j), type(2 * (1 + 2j)).__name__)
print(True + 1.5, "| 10 / 4 =", 10 / 4)
print("5" * 2, "(repetition, not arithmetic)")
print("5" + 2)
```

OUTPUT

```
7.5 float
(2+4j) complex
2.5 | 10 / 4 = 2.5
55 (repetition, not arithmetic)
Traceback (most recent call last):
  File "main.py", line 5, in <module>
    print("5" + 2)
          ~~~~^~~
TypeError: can only concatenate str (not "int") to str
```

5.4.2 Explicit conversion: int(), float(), str(), bool(), list(), tuple(), set(), dict()

Explicit conversion (casting) uses the type's name as a function. `int()` converts strings of digits and truncates floats towards zero (`int(3.99)` is 3; use `round()` to round), and accepts a base (`int("ff", 16)`). `float()` converts strings and ints. `str()` converts anything to text. `bool()` applies the truthiness rules. `list()`, `tuple()` and `set()` build collections from any iterable (a string becomes its characters; `set` removes duplicates). `dict()` builds a dictionary from key-value pairs or keyword arguments.

CONVERTING BETWEEN TYPES

main.py

```
print(int("42"), int(" -7 "), int(3.99), int(-3.99), int("ff", 16), int("101", 2), int(True))
print(float("3.14"), float(7), float("1e3"), float("  2.5 "))
print(str(42) + str(3.5) + str([1, 2]) + str(None))
print(bool(0), bool(""), bool("False"), bool([0]), bool(None))
print(list("hello"), tuple([1, 2, 3]), set("banana"))
print(list(range(5)), list({"a": 1, "b": 2}))
print(dict([("Asha", 91), ("Ravi", 78)]), dict(name="Meera", marks=85))
print(dict(zip(["x", "y"], [10, 20])))
```

OUTPUT

```
42 -7 3 -3 255 5 1
3.14 7.0 1000.0 2.5
423.5[1, 2]None
False False True True False
['h', 'e', 'l', 'l', 'o'] (1, 2, 3) {'n', 'a', 'b'}
[0, 1, 2, 3, 4] ['a', 'b']
{'Asha': 91, 'Ravi': 78} {'name': 'Meera', 'marks': 85}
{'x': 10, 'y': 20}
```

Function	Converts	Examples
`int(x, base=10)`	str of digits, float (truncates), bool	`int("42")` =42, `int(3.9)` =3, `int("ff", 16)` =255
`float(x)`	str, int	`float("3.5")` =3.5, `float("inf")`
`str(x)`	anything	`str(42)` ="42"
`bool(x)`	anything (truthiness)	`bool("")` =False, `bool("0")` =True

Function	Converts	Examples
`list(x)` / `tuple(x)`	any iterable	`list("ab")` =["a", "b"]
`set(x)`	any iterable (removes duplicates)	`set([1, 1, 2])` ={1, 2}
`dict(x)`	pairs or keywords	`dict([("a", 1)])` ={"a": 1}
`round(x, n)`	float to nearest	`round(3.9)` =4

5.4.3 ord() and chr()

Every character has a numeric **Unicode code point**. `ord(ch)` returns the code point of a single character, and `chr(n)` returns the character for a code point. ASCII characters have codes 0-127 (`"A"` is 65, `"a"` is 97, `"0"` is 48), and Unicode covers every writing system and emoji. They are used for simple ciphers, generating letter sequences and understanding string comparisons.

CHARACTERS AND THEIR CODE POINTS main.py

```
print(ord("A"), ord("a"), ord("0"), ord(" "), ord("₹"), ord("अ"))
print(chr(65), chr(97), chr(8377), chr(2309), chr(128512))
print("alphabet:", "".join(chr(c) for c in range(ord("A"), ord("Z") + 1)))
print("next letter after 'm':", chr(ord("m") + 1))

def caesar(text, shift):
    return "".join(chr((ord(c) - 97 + shift) % 26 + 97) if c.islower() else c for c in text)
secret = caesar("learn being forward", 3)
print("Caesar cipher:", secret, "->", caesar(secret, -3))
```

OUTPUT

```
65 97 48 32 8377 2309
A a ₹ अ 😀
alphabet: ABCDEFGHIJKLMNOPQRSTUVWXYZ
next letter after 'm': n
Caesar cipher: ohduq ehlqj iruzdug -> learn being forward
```

5.4.4 Conversion errors (ValueError)

Converting a value that does not represent the target type raises **ValueError**: `int("12.5")`, `int("abc")`, `float("1,000")`, an empty string. Converting the wrong **type** of object raises **TypeError**: `int([1, 2])`, `int(None)`. Always validate or catch errors when converting data that comes from users, files or the network.

WHEN CONVERSION FAILS main.py

```
tests = ["42", "12.5", "abc", "", "1,000", "  7  ", "٣"]
for text in tests:
    try:
        print(f"int({text!r}) = {int(text)}")
    except ValueError as e:
        print(f"int({text!r}) -> ValueError: {e}")

for value in ([1, 2], None):
    try:
        int(value)
    except TypeError as e:
        print(f"int({value!r}) -> TypeError: {e}")
print("workaround for '12.5':", int(float("12.5")), "| for '1,000':", int("1,000".replace(",", "")))
```

OUTPUT

```
int('42') = 42
int('12.5') -> ValueError: invalid literal for int() with base 10: '12.5'
int('abc') -> ValueError: invalid literal for int() with base 10: 'abc'
int('') -> ValueError: invalid literal for int() with base 10: ''
int('1,000') -> ValueError: invalid literal for int() with base 10: '1,000'
int('  7  ') = 7
int('٣') = 3
int([1, 2]) -> TypeError: int() argument must be a string, a bytes-like object or a real number, not
'list'
int(None) -> TypeError: int() argument must be a string, a bytes-like object or a real number, not
'NoneType'
workaround for '12.5': 12 | for '1,000': 1000
```

5.5 Command-line arguments

5.5.1 sys.argv

Programs can also receive input from the **command line**: `python grade.py Asha 91 78 85`. The list `sys.argv` holds the words: `sys.argv[0]` is the script name, and the arguments follow, always as strings. Check `len(sys.argv)` before using them and print a usage message if they are missing. Command-line arguments let scripts run without interaction, for example from scheduled tasks. For anything beyond a couple of arguments, use the `argparse` module (Module 18), which generates help text and validates input.

READING ARGUMENTS: PYTHON GRADE.PY ASHA 91 78 85 grade.py

```
import sys

print("sys.argv =", sys.argv)
if len(sys.argv) < 3:
    print(f"usage: python {sys.argv[0]} NAME MARK [MARK ...]")
    sys.exit(1)

name = sys.argv[1]
marks = [int(m) for m in sys.argv[2:]]
print(f"{name}: total {sum(marks)}, average {sum(marks) / len(marks):.2f}")
```

OUTPUT

```
sys.argv = ['grade.py', 'Asha', '91', '78', '85']
Asha: total 254, average 84.67
```

RUNNING THE SAME SCRIPT WITH AND WITHOUT ARGUMENTS

main.py

```
python grade.py; echo "exit code: $?"
python grade.py Ravi 64 72
```

TERMINAL

```
$ python grade.py; echo "exit code: $?"
usage: python grade.py NAME MARK [MARK ...]
exit code: 1
$ python grade.py Ravi 64 72
Ravi: total 136, average 68.00
```

print() prints any number of values separated by sep (default space) and followed by end (default newline), to the screen or to a file= such as sys.stderr. f-strings f"{expr:spec}" are the standard way to format: the format specification controls fill, alignment (< > ^), sign, width, grouping (, _), precision (.2f) and type (d f e % b o x); f"{x=}" prints debug output; str.format() suits stored templates; % formatting is legacy; Python 3.14 t-strings produce Template objects whose values can be escaped safely. input() always returns a string: convert with int() or float(), read several values with split() and map(), and catch ValueError for invalid input. Python converts implicitly only between numeric types; explicit conversions use int(), float(), str(), bool(), list(), tuple(), set() and dict(); ord() and chr() map characters to code points. sys.argv lists command-line arguments as strings, starting with the script name.

PRACTICE CHECKPOINT

Read a student's **name and three marks on one line**, then print the **total and average to two decimal places** in a formatted report using f-strings.

SAMPLE SOLUTION

main.py

```
line = input("Enter name and three marks (e.g. Asha 91 78 85): ")
parts = line.split()
name = parts[0]
marks = [float(m) for m in parts[1:4]]

total = sum(marks)
average = total / len(marks)
grade = "A" if average >= 85 else "B" if average >= 70 else "C" if average >= 50 else "F"

width = 30
print()
print("=" * width)
print(f"{'REPORT CARD':^{width}}")
print("=" * width)
print(f"{'Student':<16}{name:>14}")
for i, m in enumerate(marks, start=1):
    print(f"{'Subject ' + str(i):<16}{m:>14.2f}")
print("-" * width)
print(f"{'Total':<16}{total:>14.2f}")
print(f"{'Average':<16}{average:>14.2f}")
print(f"{'Percentage':<16}{average / 100:>14.2%}")
print(f"{'Grade':<16}{grade:>14}")
print("=" * width)
```

CONSOLE (user input in gold)

```
Enter name and three marks (e.g. Asha 91 78 85): Meera 91.5 78 86

==============================
         REPORT CARD
==============================
Student                 Meera
Subject 1               91.50
Subject 2               78.00
Subject 3               86.00
------------------------------
Total                  255.50
Average                 85.17
Percentage             85.17%
Grade                       A
==============================
```

MODULE 06 · BEGINNER

Control flow

Minimum duration **3 days** | Level **Beginner**

Control flow decides which statements run and how many times. This module covers decisions with if, elif and else, the conditional expression and pass; structural pattern matching with match and case (Python 3.10+); while loops and for loops with range, enumerate, zip, reversed and sorted; the else clause on loops; break, continue and pass; nested loops with star, number and character patterns; and the classic loop problems asked in every exam and interview.

IN THIS MODULE

6.1 Conditional statements
6.2 Structural pattern matching (3.10)
6.3 while loop
6.4 for loop
6.5 Loop control
6.6 Nested loops and patterns
6.7 Classic loop problems

6.1 Conditional statements

Programs normally run from top to bottom. **Conditional statements** choose between different paths depending on whether a condition is true. A **condition** is any expression; Python evaluates its truthiness (Module 3): comparisons, logical combinations, or values such as a list (true if non-empty).

6.1.1 if

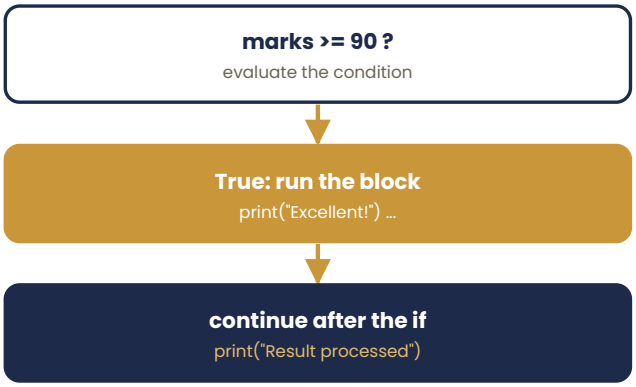
`if condition:` runs the indented block only when the condition is truthy; otherwise Python skips the block and continues after it. Do not forget the colon, and indent the block consistently.

A SIMPLE IF — main.py

```
marks = 92
if marks >= 90:
    print("Excellent!")
    print("You earned a scholarship.")
print("Result processed")

cart = ["pen"]
if cart:                                  # truthiness: a non-empty list
    print(f"{len(cart)} item(s) in your cart")
```

OUTPUT

```
Excellent!
You earned a scholarship.
Result processed
1 item(s) in your cart
```



FLOW OF AN IF STATEMENT

6.1.2 if-else

`else:` adds a block that runs when the condition is falsy, so exactly one of the two blocks runs. The `else` line is aligned with its `if`.

TWO-WAY DECISION main.py

```
n = int(input("Enter a number: "))
if n % 2 == 0:
    print(n, "is even")
else:
    print(n, "is odd")
```

CONSOLE (user input in gold)

```
Enter a number: 17
17 is odd
```

6.1.3 if-elif-else

To choose between more than two paths, add `elif` ("else if") clauses. Python tests the conditions **from top to bottom** and runs only the block of the **first** true condition; the optional final `else` runs if none matched. Because the order matters, check the most specific or highest range first. Python has no `switch` statement; `elif` chains or `match` (6.2) take its place.

AN ELIF LADDER

main.py

```
def grade(marks):
    if marks >= 90:
        return "A+"
    elif marks >= 75:
        return "A"
    elif marks >= 60:
        return "B"
    elif marks >= 40:
        return "C"
    else:
        return "F"

for m in (95, 75, 62, 40, 12):
    print(m, "->", grade(m))
```

OUTPUT

```
95 -> A+
75 -> A
62 -> B
40 -> C
12 -> F
```

AN IF-ELIF-ELSE LADDER

6.1.4 Nested if

An `if` statement can contain another `if` inside its block: a **nested** condition, checked only when the outer one is true. Nesting more than two or three levels quickly becomes hard to read; combine conditions with `and`, or use early returns (guard clauses) to flatten the code.

NESTED CONDITIONS AND A FLATTER ALTERNATIVE main.py

```
age, has_licence, has_insurance = 20, True, False

if age >= 18:
    if has_licence:
        if has_insurance:
            print("You may drive")
        else:
            print("Get insurance first")
    else:
        print("Get a licence first")
else:
    print("Too young to drive")

# flatter version with combined conditions
can_drive = age >= 18 and has_licence and has_insurance
print("can drive:", can_drive)
```

OUTPUT

```
Get insurance first
can drive: False
```

6.1.5 Conditional expression (ternary) a if cond else b

The **conditional expression** `value_if_true if condition else value_if_false` chooses between two values in a single expression, the Python equivalent of C's `cond ? a : b`. It is useful in assignments, return statements, f-strings and comprehensions. Keep it simple: nested ternaries are hard to read.

ONE-LINE DECISIONS main.py

```
age = 17
status = "adult" if age >= 18 else "minor"
print(status)

n = 7
print(f"{n} is {'even' if n % 2 == 0 else 'odd'}")
fee = 1000
print("fee:", fee * 0.9 if age < 18 else fee)
labels = ["pass" if m >= 40 else "fail" for m in [91, 35, 60]]
print(labels)
```

OUTPUT

```
minor
7 is odd
fee: 900.0
['pass', 'fail', 'pass']
```

6.1.6 pass statement

Python does not allow an empty block, so `pass` is a statement that does nothing. It serves as a **placeholder** where syntax requires a statement but you have nothing to write yet: an unfinished function or class, an `except` clause that deliberately ignores an error, or a branch that intentionally does nothing. The ellipsis `...` is sometimes used the same way.

PLACEHOLDERS WITH PASS main.py

```
def send_email(to, subject):
    pass                        # TODO: implement later

class Report:
    pass                        # an empty class for now

marks = 55
if marks < 40:
    pass                        # nothing to do for failures yet
else:
    print("send congratulations")
send_email("asha@lbf.in", "Hi")
print("program finished without errors")
```

OUTPUT

```
send congratulations
program finished without errors
```

6.2 Structural pattern matching (3.10)

6.2.1 match and case

Python 3.10 added **structural pattern matching** (PEP 634): the `match` statement compares a value (the **subject**) against a series of `case` **patterns** and runs the block of the first one that matches. In its simplest form it replaces long `if-elif` chains that compare one value with constants, like `switch` in other languages, but it is much more powerful: patterns can check the **structure** of data (lists, tuples, dictionaries, objects) and extract parts of it into variables in the same step. Unlike C's switch, there is no fall-through and no `break` is needed.

MATCH AS A CLEARER SWITCH main.py

```
def http_status(code):
    match code:
        case 200:
            return "OK"
        case 404:
            return "Not Found"
        case 500:
            return "Internal Server Error"
        case _:
            return "Unknown status"

for c in (200, 404, 418):
    print(c, http_status(c))
```

OUTPUT

```
200 OK
404 Not Found
418 Unknown status
```

6.2.2 Literal, wildcard _ and OR patterns

A **literal pattern** (a number, string, `True`, `False` or `None`) matches equal values. The **wildcard** `_` matches anything without binding it, so `case _:` acts as the default branch and must come last. An **OR pattern** `p1 | p2 | p3` matches if any alternative matches.

LITERAL, OR AND WILDCARD PATTERNS main.py

```
def day_type(day):
    match day.lower():
        case "saturday" | "sunday":
            return "weekend"
        case "monday" | "tuesday" | "wednesday" | "thursday" | "friday":
            return "weekday"
        case _:
            return "not a day"

for d in ("Sunday", "wednesday", "Funday"):
    print(d, "->", day_type(d))
```

OUTPUT

```
Sunday -> weekend
wednesday -> weekday
Funday -> not a day
```

6.2.3 Capture patterns and guards

A bare name in a pattern is a **capture pattern**: it matches anything and binds the value to that name, so the block can use it. (Because of this, `case LIMIT:` does not compare with a variable named LIMIT; it captures. Compare with constants through dotted names such as `Color.RED`, or use a guard.) A **guard** is an `if` condition after the pattern, `case x if x < 0:`; the case matches only if the pattern matches **and** the guard is true. `as` binds a sub-pattern to a name.

CAPTURE PATTERNS, GUARDS AND AS main.py

```
def describe(n):
    match n:
        case 0:
            return "zero"
        case x if x < 0:
            return f"negative ({x})"
        case x if x % 2 == 0:
            return f"positive even ({x})"
        case (1 | 3 | 5 | 7 | 9) as digit:
            return f"odd digit {digit}"
        case other:
            return f"positive odd ({other})"

for n in (0, -4, 10, 7, 21):
    print(n, "->", describe(n))
```

OUTPUT

```
0 -> zero
-4 -> negative (-4)
10 -> positive even (10)
7 -> odd digit 7
21 -> positive odd (21)
```

6.2.4 Sequence and mapping patterns

Sequence patterns match lists and tuples by length and contents, and capture items: `[x, y]` matches a two-element sequence, `[first, *rest]` one or more elements, and literals can be mixed in (`["go", direction]`). Strings are not matched as sequences. **Mapping patterns** match dictionaries that contain certain keys, capturing their values: `{"type": "click", "x": x, "y": y}`; extra keys are allowed, and

`**rest` captures them. Together they make processing commands, JSON messages and parsed data very concise.

SEQUENCE PATTERNS FOR A TEXT-ADVENTURE COMMAND PARSER

main.py

```
def run(command):
    match command.split():
        case ["quit"] | ["exit"]:
            return "Goodbye"
        case ["go", ("north" | "south" | "east" | "west") as direction]:
            return f"Moving {direction}"
        case ["go", _]:
            return "Unknown direction"
        case ["take", item]:
            return f"Picked up {item}"
        case ["drop", *items] if items:
            return f"Dropped {', '.join(items)}"
        case []:
            return "Say something"
        case [verb, *_]:
            return f"I don't know how to {verb}"

for cmd in ("go north", "go up", "take lamp", "drop key coin map", "", "dance now", "quit"):
    print(f"{cmd!r:<20} -> {run(cmd)}")
```

OUTPUT

```
'go north'           -> Moving north
'go up'              -> Unknown direction
'take lamp'          -> Picked up lamp
'drop key coin map'  -> Dropped key, coin, map
''                   -> Say something
'dance now'          -> I don't know how to dance
'quit'               -> Goodbye
```

MAPPING PATTERNS FOR JSON-LIKE EVENTS

main.py

```
events = [
    {"type": "click", "x": 120, "y": 45, "button": "left"},
    {"type": "key", "key": "Enter"},
    {"type": "scroll", "delta": -3},
    {"type": "resize", "size": [1280, 720]},
    {"user": "asha"},
]
for event in events:
    match event:
        case {"type": "click", "x": x, "y": y, **rest}:
            print(f"click at ({x}, {y}), other data: {rest}")
        case {"type": "key", "key": "Enter" | "Return"}:
            print("submit form")
        case {"type": "scroll", "delta": int(d)} if d < 0:
            print(f"scroll up {-d} lines")
        case {"type": "resize", "size": [w, h]}:
            print(f"resize to {w}x{h}")
        case _:
            print("ignored:", event)
```

OUTPUT

```
click at (120, 45), other data: {'button': 'left'}
submit form
scroll up 3 lines
resize to 1280x720
ignored: {'user': 'asha'}
```

6.2.5 Class patterns (after OOP)

Class patterns match objects by type and attributes: `case Point(x=0, y=y):` matches a `Point` whose `x` is 0 and captures its `y`. Built-in types work too: `case int(n):` matches integers, `case str():` any string. Classes can define `__match_args__` to allow positional sub-patterns (`Point(0, y)`), and dataclasses do this automatically. Classes are explained in Module 15; this preview shows the idea.

MATCHING OBJECTS BY TYPE AND ATTRIBUTES

main.py

```
from dataclasses import dataclass

@dataclass
class Point:
    x: float
    y: float

@dataclass
class Circle:
    centre: Point
    radius: float

def where(shape):
    match shape:
        case Point(0, 0):
            return "the origin"
        case Point(0, y):
            return f"on the y-axis at {y}"
        case Point(x=x, y=0):
            return f"on the x-axis at {x}"
        case Circle(Point(0, 0), r):
            return f"a circle of radius {r} around the origin"
        case Point() | Circle():
            return "somewhere else"
        case int(n) | float(n):
            return f"just a number: {n}"

for s in (Point(0, 0), Point(0, 5), Point(3, 0), Circle(Point(0, 0), 2), Point(1, 1), 42):
    print(s, "->", where(s))
```

OUTPUT

```
Point(x=0, y=0) -> the origin
Point(x=0, y=5) -> on the y-axis at 5
Point(x=3, y=0) -> on the x-axis at 3
Circle(centre=Point(x=0, y=0), radius=2) -> a circle of radius 2 around the origin
Point(x=1, y=1) -> somewhere else
42 -> just a number: 42
```

Pattern	Example	Matches
Literal	`case 404:`	equal value
Wildcard	`case _:`	anything (default)
OR	`case "y" | "yes":`	any alternative
Capture	`case x:`	anything, binds x
Guard	`case x if x > 0:`	pattern + condition
Sequence	`case [a, b, *rest]:`	list/tuple of 2 or more
Mapping	`case {"id": i}:`	dict with key "id"
Class	`case Point(x=0):`	Point object with x == 0
as	`case [1, *_] as seq:`	binds the whole match

6.3 while loop

6.3.1 Syntax and counters

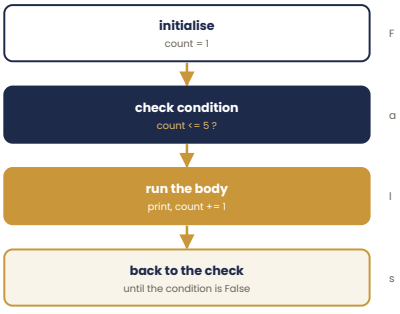
A `while` loop repeats its block **as long as** its condition is true. The condition is checked before every iteration; when it becomes false, the loop ends and execution continues after it. If the condition is false at the start, the body never runs. Something inside the loop must eventually make the condition false, typically a **counter** that changes each time. Use `while` when the number of repetitions is not known in advance (until the user quits, until a value converges); use `for` to go through a known collection or range.

COUNTING LOOPS AND AN UNKNOWN NUMBER OF ITERATIONS main.py

```
count = 1
while count <= 5:
    print("iteration", count)
    count += 1                  # without this, the loop would never end
print("loop finished, count =", count)

balance, target, years = 10000, 20000, 0      # how long until the money doubles at 8%?
while balance < target:
    balance *= 1.08
    years += 1
print(f"doubled after {years} years: Rs {balance:,.2f}")
```

OUTPUT

```
iteration 1
iteration 2
iteration 3
iteration 4
iteration 5
loop finished, count = 6
doubled after 10 years: Rs 21,589.25
```



HOW A WHILE LOOP RUNS

6.3.2 Infinite loops and while True with break

A loop whose condition never becomes false runs forever: an **infinite loop**. Accidentally, it is a bug (stop it with Ctrl+C). Deliberately, `while True:` with a `break` inside is a common pattern when the exit condition is best checked in the middle of the body, such as menus and input validation: "repeat forever; when the user chooses Quit, break out".

WHILE TRUE WITH BREAK

main.py

```
total = 0
while True:
    entry = input("Enter an amount (or 'done'): ")
    if entry == "done":
        break                          # leave the loop immediately
    total += float(entry)
print(f"Total: Rs {total:.2f}")
```

CONSOLE (user input in gold)

```
Enter an amount (or 'done'): 250
Enter an amount (or 'done'): 99.50
Enter an amount (or 'done'): 1200
Enter an amount (or 'done'): done
Total: Rs 1549.50
```

6.3.3 while-else

A `while` loop can have an `else` block, which runs once when the loop ends **normally** because its condition became false, but **not** when the loop is left with `break`. It answers the question "did the loop finish without finding what it was looking for?" without needing a flag variable. Think of it as "no break".

THE ELSE CLAUSE OF WHILE

main.py

```
def find_first_multiple(numbers, k):
    i = 0
    while i < len(numbers):
        if numbers[i] % k == 0:
            print(f"found {numbers[i]} at index {i}")
            break
        i += 1
    else:
        print(f"no multiple of {k} found")       # runs only if no break happened

find_first_multiple([4, 9, 14, 21], 7)
find_first_multiple([4, 9, 15], 7)
```

OUTPUT

```
found 14 at index 2
no multiple of 7 found
```

6.4 for loop

6.4.1 Iterating over sequences

A `for` loop takes each item of an **iterable** in turn, binds it to the loop variable, and runs the block: `for item in collection:`. Iterables include lists, tuples, strings (character by character), ranges, sets, dictionaries (keys), files (line by line) and many more. There is no index to manage and no risk of running past the end. Python's `for` is a "for each" loop; counting loops are written with `range()`.

LOOPING OVER DIFFERENT ITERABLES main.py

```
for course in ["Python", "Java", "SQL"]:
    print("Course:", course)

for ch in "LBF":
    print(ch, end=" ")
print()

marks = {"Asha": 91, "Ravi": 78}
for name in marks:                      # iterating a dict gives its keys
    print(name, marks[name])
for name, mark in marks.items():        # key-value pairs
    print(f"{name}: {mark}")
```

OUTPUT

```
Course: Python
Course: Java
Course: SQL
L B F
Asha 91
Ravi 78
Asha: 91
Ravi: 78
```

6.4.2 range(start, stop, step)

`range()` produces a sequence of integers lazily, without building a list. `range(stop)` counts from 0 up to but **not including** stop; `range(start, stop)` from start; `range(start, stop, step)` in steps, which can be negative for counting down. The excluded end means `range(n)` gives exactly n numbers and `range(len(s))` gives every valid index. Convert to a list to see the values.

COUNTING WITH RANGE main.py

```
print(list(range(5)))
print(list(range(2, 8)))
print(list(range(0, 20, 5)))
print(list(range(10, 0, -2)))
print(list(range(5, 0, -1)), "| empty:", list(range(5, 0)))

for i in range(1, 11):
    print(f"7 x {i:>2} = {7 * i:>3}")
r = range(0, 1_000_000_000, 3)
print("len:", len(r), "| 999 in r:", 999 in r, "| r[10]:", r[10])     # no huge list is built
```

OUTPUT

```
[0, 1, 2, 3, 4]
[2, 3, 4, 5, 6, 7]
[0, 5, 10, 15]
[10, 8, 6, 4, 2]
[5, 4, 3, 2, 1] | empty: []
7 x  1 =   7
7 x  2 =  14
7 x  3 =  21
7 x  4 =  28
7 x  5 =  35
7 x  6 =  42
7 x  7 =  49
7 x  8 =  56
7 x  9 =  63
7 x 10 =  70
len: 333333334 | 999 in r: True | r[10]: 30
```

6.4.3 enumerate()

When you need both the **position** and the **item**, use `enumerate(iterable, start=0)`, which yields pairs `(index, item)`. It is cleaner and less error-prone than `for i in range(len(items))` followed by `items[i]`. `start=1` numbers from 1 for display.

INDEX AND ITEM TOGETHER main.py

```
toppers = ["Asha", "Meera", "Ravi"]

for i in range(len(toppers)):              # works, but not Pythonic
    print(i, toppers[i])

for rank, name in enumerate(toppers, start=1):
    print(f"#{rank} {name}")
print(list(enumerate("abc")))
```

OUTPUT

```
0 Asha
1 Meera
2 Ravi
#1 Asha
#2 Meera
#3 Ravi
[(0, 'a'), (1, 'b'), (2, 'c')]
```

6.4.4 zip()

`zip(a, b, ...)` walks through several iterables **in parallel**, yielding tuples of corresponding items: first with first, second with second. It stops at the end of the shortest iterable; pass `strict=True` to raise an error if the lengths differ (a useful safety check). `zip` also builds dictionaries (`dict(zip(keys, values))`) and "unzips" pairs with `zip(*pairs)`.

LOOPING OVER SEVERAL LISTS TOGETHER

main.py

```
names = ["Asha", "Ravi", "Meera"]
maths = [91, 64, 85]
science = [88, 72, 90]

for name, m, s in zip(names, maths, science):
    print(f"{name:<6} maths {m}  science {s}  total {m + s}")

print(dict(zip(names, maths)))
pairs = [("a", 1), ("b", 2), ("c", 3)]
letters, numbers = zip(*pairs)                # unzip
print(letters, numbers)
print(list(zip([1, 2, 3], "ab")))             # stops at the shortest
try:
    list(zip([1, 2, 3], "ab", strict=True))
except ValueError as e:
    print("strict=True:", e)
```

OUTPUT

```
Asha   maths 91  science 88  total 179
Ravi   maths 64  science 72  total 136
Meera  maths 85  science 90  total 175
{'Asha': 91, 'Ravi': 64, 'Meera': 85}
('a', 'b', 'c') (1, 2, 3)
[(1, 'a'), (2, 'b')]
strict=True: zip() argument 2 is shorter than argument 1
```

6.4.5 reversed() and sorted() in loops

`reversed(seq)` iterates over a sequence backwards without copying it; `sorted(iterable, key=..., reverse=...)` returns a **new** sorted list and leaves the original unchanged. Both combine naturally with `for`. The `key` function decides what to sort by: `key=len`, `key=str.lower` for case-insensitive order, or a lambda selecting a field.

LOOPING BACKWARDS AND IN SORTED ORDER main.py

```
for n in reversed(range(1, 6)):
    print(n, end=" ")
print("lift-off!")

words = ["banana", "Apple", "cherry", "date"]
print("sorted:", sorted(words))
print("case-insensitive:", sorted(words, key=str.lower))
print("by length, longest first:", sorted(words, key=len, reverse=True))
print("original unchanged:", words)

students = [("Asha", 91), ("Ravi", 78), ("Meera", 85)]
for name, mark in sorted(students, key=lambda s: s[1], reverse=True):
    print(f"{name}: {mark}")
```

OUTPUT

```
5 4 3 2 1 lift-off!
sorted: ['Apple', 'banana', 'cherry', 'date']
case-insensitive: ['Apple', 'banana', 'cherry', 'date']
by length, longest first: ['banana', 'cherry', 'Apple', 'date']
original unchanged: ['banana', 'Apple', 'cherry', 'date']
Asha: 91
Meera: 85
Ravi: 78
```

6.4.6 for-else

Like `while`, a `for` loop can have an `else` block that runs only if the loop finished **without** a `break`. The classic use is searching: break when found, and handle "not found" in the `else`.

FOR-ELSE IN A SEARCH main.py

```
def is_prime(n):
    if n < 2:
        return False
    for d in range(2, int(n ** 0.5) + 1):
        if n % d == 0:
            print(f"  {n} is divisible by {d}")
            break
    else:
        return True              # loop completed: no divisor found
    return False

for n in (29, 91, 97):
    print(n, "prime?", is_prime(n))
```

OUTPUT

```
29 prime? True
  91 is divisible by 7
91 prime? False
97 prime? True
```

6.5 Loop control

6.5.1 break

`break` exits the **innermost** enclosing loop immediately; execution continues after the loop, and any loop `else` is skipped. Use it when the result has been found or a stop signal received. To leave several nested

loops at once, put them in a function and `return`, or use a flag.

6.5.2 continue

`continue` skips the **rest of the current iteration** and jumps to the next one (in a `while` loop, back to the condition check). Use it to skip items that need no processing, keeping the main logic un-nested.

6.5.3 pass

`pass` does nothing (6.1.6); inside a loop it is just a placeholder and, unlike `continue`, does not skip the remaining statements of the iteration.

BREAK, CONTINUE AND PASS TOGETHER

main.py

```
readings = [12, -1, 15, 0, 18, 999, 20]          # -1 = sensor error, 999 = end marker
total = count = 0
for r in readings:
    if r == 999:
        print("end marker: stop reading")
        break                    # leave the loop
    if r < 0:
        print("skip invalid reading", r)
        continue                 # next iteration
    if r == 0:
        pass                     # nothing special, but keep going below
    total += r
    count += 1
print(f"valid readings: {count}, average: {total / count:.2f}")
```

OUTPUT

```
skip invalid reading -1
end marker: stop reading
valid readings: 4, average: 11.25
```

Statement	Effect	Typical use
`break`	exit the innermost loop; skip loop else	found it, stop signal
`continue`	skip to the next iteration	ignore invalid or uninteresting items
`pass`	do nothing	placeholder in empty blocks
loop `else`	runs if no break occurred	"not found" handling

6.6 Nested loops and patterns

A loop inside another loop is a **nested loop**: for every iteration of the outer loop, the inner loop runs completely. With an outer loop of n iterations and an inner loop of m, the body runs n × m times. Printing patterns is the classic way to master nested loops: the **outer** loop usually controls the **rows**, the **inner** loop the **columns** (characters in a row). Python's string repetition (`"*" * n`) often replaces the inner loop.

THE ORDER IN WHICH NESTED LOOPS RUN

main.py

```
for i in range(1, 4):
    for j in range(1, 4):
        print(f"({i},{j})", end=" ")
    print()
```

OUTPUT

```
(1,1) (1,2) (1,3)
(2,1) (2,2) (2,3)
(3,1) (3,2) (3,3)
```

6.6.1 Star patterns (square, triangles, pyramid, diamond)

SQUARE AND TRIANGLES

main.py

```
n = 4
print("Square:")
for i in range(n):
    for j in range(n):
        print("*", end=" ")
    print()

print("Right triangle:")
for i in range(1, n + 1):
    print("* " * i)

print("Inverted triangle:")
for i in range(n, 0, -1):
    print("* " * i)

print("Right-aligned triangle:")
for i in range(1, n + 1):
    print("  " * (n - i) + "* " * i)
```

OUTPUT

```
Square:
* * * *
* * * *
* * * *
* * * *
Right triangle:
*
* *
* * *
* * * *
Inverted triangle:
* * * *
* * *
* *
*
Right-aligned triangle:
      *
    * *
  * * *
* * * *
```

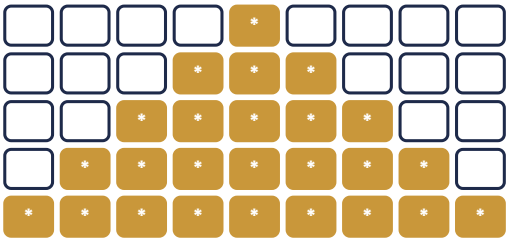
PYRAMID, DIAMOND AND HOLLOW SQUARE main.py

```
n = 5
print("Pyramid:")
for i in range(1, n + 1):
    print(" " * (n - i) + "*" * (2 * i - 1))

print("Diamond:")
for i in list(range(1, n + 1)) + list(range(n - 1, 0, -1)):
    print(" " * (n - i) + "*" * (2 * i - 1))

print("Hollow square:")
for i in range(n):
    print("".join("*" if i in (0, n - 1) or j in (0, n - 1) else " " for j in range(n)))
```

OUTPUT

```
Pyramid:
    *
   ***
  *****
 *******
*********
Diamond:
    *
   ***
  *****
 *******
*********
 *******
  *****
   ***
    *
Hollow square:
*****
*   *
*   *
*   *
*****
```

row 1	4 spaces + 1 stars
row 2	3 spaces + 3 stars
row 3	2 spaces + 5 stars
row 4	1 spaces + 7 stars
row 5	0 spaces + 9 stars



THE PYRAMID: ROW I HAS N − I SPACES AND 2I − 1 STARS

6.6.2 Number patterns (Floyd's triangle, Pascal's triangle)

Floyd's triangle fills rows of increasing length with consecutive numbers. **Pascal's triangle** starts with 1, and each number is the sum of the two numbers above it; row n contains the binomial coefficients C(n, k), used in probability and algebra.

NUMBER PATTERNS

main.py

```
print("Number triangle:")
for i in range(1, 6):
    print(" ".join(str(j) for j in range(1, i + 1)))

print("Floyd's triangle:")
num = 1
for i in range(1, 6):
    for _ in range(i):
        print(f"{num:3}", end="")
        num += 1
    print()

print("Pascal's triangle:")
rows = 6
row = [1]
for i in range(rows):
    print(" " * (rows - i) * 2 + "".join(f"{v:4}" for v in row))
    row = [1] + [row[k] + row[k + 1] for k in range(len(row) - 1)] + [1]
```

OUTPUT

```
Number triangle:
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
Floyd's triangle:
  1
  2  3
  4  5  6
  7  8  9 10
 11 12 13 14 15
Pascal's triangle:
                1
              1   1
            1   2   1
          1   3   3   1
        1   4   6   4   1
      1   5  10  10   5   1
```

6.6.3 Character patterns

CHARACTER PATTERNS

main.py

```
n = 5
print("Alphabet triangle:")
for i in range(n):
    print(" ".join(chr(65 + j) for j in range(i + 1)))

print("Same letter per row:")
for i in range(n):
    print(chr(65 + i) * (i + 1))

print("Palindromic pyramid:")
for i in range(n):
    left = "".join(chr(65 + j) for j in range(i + 1))
    print(" " * (n - i - 1) + left + left[-2::-1])
```

OUTPUT

```
Alphabet triangle:
A
A B
A B C
A B C D
A B C D E
Same letter per row:
A
BB
CCC
DDDD
EEEEE
Palindromic pyramid:
    A
   ABA
  ABCBA
 ABCDCBA
ABCDEDCBA
```

6.7 Classic loop problems

These problems appear in college exams, coding tests and interviews because they combine loops, conditions and arithmetic. Practise writing each one without looking.

6.7.1 Sum of digits, reverse a number, palindrome

The digit-by-digit technique: `n % 10` gives the last digit and `n // 10` removes it; repeat while `n > 0`. A **palindrome number** reads the same backwards (121, 1331).

WORKING WITH THE DIGITS OF A NUMBER main.py

```
def digit_sum(n):
    total = 0
    while n > 0:
        total += n % 10        # last digit
        n //= 10               # remove it
    return total

def reverse_number(n):
    rev = 0
    while n > 0:
        rev = rev * 10 + n % 10
        n //= 10
    return rev

for n in (12345, 121, 1331, 2024):
    print(f"{n}: digit sum {digit_sum(n)}, reversed {reverse_number(n)}, palindrome {n ==
reverse_number(n)}")
print("Pythonic one-liners:", sum(int(d) for d in "12345"), int(str(12345)[::-1]))
```

OUTPUT

```
12345: digit sum 15, reversed 54321, palindrome False
121: digit sum 4, reversed 121, palindrome True
1331: digit sum 8, reversed 1331, palindrome True
2024: digit sum 8, reversed 4202, palindrome False
Pythonic one-liners: 15 54321
```

6.7.2 Factorial and Fibonacci

The **factorial** n! is the product 1 × 2 × … × n (0! = 1). The **Fibonacci sequence** starts 0, 1 and each term is the sum of the previous two: 0, 1, 1, 2, 3, 5, 8, 13…

FACTORIAL AND FIBONACCI WITH LOOPS main.py

```
def factorial(n):
    result = 1
    for i in range(2, n + 1):
        result *= i
    return result

def fibonacci(count):
    a, b = 0, 1
    terms = []
    for _ in range(count):
        terms.append(a)
        a, b = b, a + b          # tuple assignment does the update in one step
    return terms

print("5! =", factorial(5), "| 20! =", factorial(20))
print("first 12 Fibonacci numbers:", fibonacci(12))
import math
print("check with math.factorial:", math.factorial(20) == factorial(20))
```

OUTPUT

```
5! = 120 | 20! = 2432902008176640000
first 12 Fibonacci numbers: [0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89]
check with math.factorial: True
```

6.7.3 Prime check and primes in a range

A **prime** number is greater than 1 and divisible only by 1 and itself. It is enough to test divisors up to the square root of n, because any larger divisor pairs with a smaller one. To find all primes up to a limit efficiently, the **Sieve of Eratosthenes** crosses out the multiples of each prime.

PRIME NUMBERS

main.py

```
def is_prime(n):
    if n < 2:
        return False
    if n % 2 == 0:
        return n == 2
    d = 3
    while d * d <= n:          # only up to the square root
        if n % d == 0:
            return False
        d += 2                 # skip even divisors
    return True

print("primes between 1 and 50:", [n for n in range(1, 51) if is_prime(n)])

def sieve(limit):
    is_p = [True] * (limit + 1)
    is_p[0] = is_p[1] = False
    for i in range(2, int(limit ** 0.5) + 1):
        if is_p[i]:
            for multiple in range(i * i, limit + 1, i):
                is_p[multiple] = False
    return [i for i, p in enumerate(is_p) if p]

primes = sieve(100_000)
print("count of primes below 100,000:", len(primes), "| largest:", primes[-1])
```

OUTPUT

```
primes between 1 and 50: [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]
count of primes below 100,000: 9592 | largest: 99991
```

6.7.4 Armstrong, perfect and strong numbers

An **Armstrong** (narcissistic) number equals the sum of its digits each raised to the power of the number of digits: $153 = 1^3 + 5^3 + 3^3$. A **perfect** number equals the sum of its proper divisors: $28 = 1 + 2 + 4 + 7 + 14$. A **strong** number equals the sum of the factorials of its digits: $145 = 1! + 4! + 5!$.

SPECIAL NUMBERS main.py

```
import math

def is_armstrong(n):
    digits = str(n)
    return n == sum(int(d) ** len(digits) for d in digits)

def is_perfect(n):
    return n > 1 and n == sum(d for d in range(1, n // 2 + 1) if n % d == 0)

def is_strong(n):
    return n == sum(math.factorial(int(d)) for d in str(n))

print("Armstrong numbers below 10,000:", [n for n in range(1, 10_000) if is_armstrong(n)])
print("perfect numbers below 10,000:", [n for n in range(1, 10_000) if is_perfect(n)])
print("strong numbers below 100,000:", [n for n in range(1, 100_000) if is_strong(n)])
```

OUTPUT

```
Armstrong numbers below 10,000: [1, 2, 3, 4, 5, 6, 7, 8, 9, 153, 370, 371, 407, 1634, 8208, 9474]
perfect numbers below 10,000: [6, 28, 496, 8128]
strong numbers below 100,000: [1, 2, 145, 40585]
```

6.7.5 GCD, LCM, number guessing game

The **greatest common divisor** (GCD) of two numbers is found with **Euclid's algorithm**: replace the pair (a, b) with (b, a % b) until b is 0; then a is the GCD. The **least common multiple** is a × b / GCD. A **number guessing game** combines a loop, input, conditions and random numbers.

EUCLID'S ALGORITHM main.py

```
def gcd(a, b):
    while b:
        print(f"  gcd({a}, {b})")
        a, b = b, a % b
    return a

a, b = 252, 105
g = gcd(a, b)
print(f"GCD({a}, {b}) = {g} | LCM = {a * b // g}")
import math
print("check:", math.gcd(a, b), math.lcm(a, b))
```

OUTPUT

```
  gcd(252, 105)
  gcd(105, 42)
  gcd(42, 21)
GCD(252, 105) = 21 | LCM = 1260
check: 21 1260
```

A NUMBER GUESSING GAME

main.py

```python
import random

random.seed(42)                          # fixed seed so this run is repeatable; remove it to play
secret = random.randint(1, 100)
attempts = 0
print("I am thinking of a number between 1 and 100.")
while True:
    guess = int(input("Your guess: "))
    attempts += 1
    if guess < secret:
        print("  Too low!")
    elif guess > secret:
        print("  Too high!")
    else:
        print(f"  Correct! The number was {secret}. You took {attempts} attempts.")
        break
```

CONSOLE (user input in gold)

```
I am thinking of a number between 1 and 100.
Your guess: 50
  Too low!
Your guess: 75
  Too low!
Your guess: 88
  Too high!
Your guess: 81
  Too low!
Your guess: 84
  Too high!
Your guess: 82
  Correct! The number was 82. You took 6 attempts.
```

if, if-else and if-elif-else choose a path by testing conditions from top to bottom; nested ifs can often be flattened with and or guard clauses; a if cond else b is the conditional expression; pass is an empty placeholder. match-case (3.10+) compares a subject with patterns: literals, the wildcard _, OR patterns, captures with guards, sequence patterns ([x, *rest]), mapping patterns ({"key": v}) and class patterns (Point(x=0)). while repeats while a condition is true (use while True with break for mid-loop exits); for iterates over any iterable, with range(start, stop, step) for counting, enumerate for index and item, zip for parallel iteration, and reversed and sorted for order. Both loops accept an else block that runs only if no break occurred. break leaves the loop, continue skips to the next iteration. Nested loops print patterns (outer loop rows, inner loop columns) and solve classic problems: digit sums and reversal with % 10 and // 10, factorial, Fibonacci, primes up to the square root and the sieve, Armstrong, perfect and strong numbers, Euclid's GCD, and guessing games.

PRACTICE CHECKPOINT

Build a **menu-driven calculator** using `match-case` inside a `while True` loop, plus **five pattern programs**.

PART 1: THE MENU-DRIVEN CALCULATOR

main.py

```
def show_menu():
    print("\n1) Add  2) Subtract  3) Multiply  4) Divide  5) Power  6) Quit")

while True:
    show_menu()
    choice = input("Choose an option: ").strip()
    match choice:
        case "6" | "q" | "quit":
            print("Goodbye!")
            break
        case "1" | "2" | "3" | "4" | "5":
            a = float(input("  first number: "))
            b = float(input("  second number: "))
            match choice:
                case "1": result = a + b
                case "2": result = a - b
                case "3": result = a * b
                case "4" if b == 0:
                    print("  cannot divide by zero")
                    continue
                case "4": result = a / b
                case "5": result = a ** b
            print(f"  result = {result:g}")
        case _:
            print(f"  '{choice}' is not a valid option")
```

CONSOLE (user input in gold)

```
1) Add  2) Subtract  3) Multiply  4) Divide  5) Power  6) Quit
Choose an option: 1
  first number: 12
  second number: 30
  result = 42

1) Add  2) Subtract  3) Multiply  4) Divide  5) Power  6) Quit
Choose an option: 4
  first number: 7
  second number: 0
  cannot divide by zero

1) Add  2) Subtract  3) Multiply  4) Divide  5) Power  6) Quit
Choose an option: 9
  '9' is not a valid option

1) Add  2) Subtract  3) Multiply  4) Divide  5) Power  6) Quit
Choose an option: 5
  first number: 2
  second number: 10
  result = 1024

1) Add  2) Subtract  3) Multiply  4) Divide  5) Power  6) Quit
Choose an option: q
Goodbye!
```

PART 2: FIVE PATTERN PROGRAMS

main.py

```
n = 4
print("1) Right triangle")
for i in range(1, n + 1): print("*" * i)
print("2) Inverted right triangle")
for i in range(n, 0, -1): print("*" * i)
print("3) Pyramid")
for i in range(1, n + 1): print(" " * (n - i) + "*" * (2 * i - 1))
print("4) Floyd's triangle")
k = 1
for i in range(1, n + 1):
    print(" ".join(f"{k + j:2}" for j in range(i))); k += i
print("5) Alphabet pyramid")
for i in range(n):
    s = "".join(chr(65 + j) for j in range(i + 1))
    print(" " * (n - i - 1) + s + s[-2::-1])
```

OUTPUT

```
1) Right triangle
*
**
***
****
2) Inverted right triangle
****
***
**
*
3) Pyramid
   *
  ***
 *****
*******
4) Floyd's triangle
 1
 2  3
 4  5  6
 7  8  9 10
5) Alphabet pyramid
   A
  ABA
 ABCBA
ABCDCBA
```

MODULE 07 · BEGINNER

Strings

Minimum duration **3 days** Level **Beginner**

A Python string is an immutable sequence of Unicode characters with a rich set of built-in methods. This module covers creating strings with every kind of quote, escape sequences, raw strings and encodings; indexing, slicing and immutability; string operators; the full family of string methods for case, searching, checking, cleaning, splitting, joining, replacing and alignment; building strings efficiently; an introduction to regular expressions; and the classic string problems asked in exams and interviews.

IN THIS MODULE

7.1 Creating strings
7.2 Accessing characters
7.3 String operations
7.4 String methods
7.5 Building strings efficiently
7.6 Regular expressions (introduction)
7.7 Classic string problems

7.1 Creating strings

7.1.1 Single, double and triple quotes

A string literal is text between quotes. Single quotes `'...'` and double quotes `"..."` are equivalent; choose one style and use the other when the text contains a quote, avoiding escapes (`"It's"`, `'She said "hi"'`). Triple quotes, `"""..."""` or `'''...'''`, allow both kinds of quote and line breaks. `str(x)` converts any object to a string, and `""` is the empty string. Python has no separate character type: a single character is a string of length 1.

QUOTES AND STRING CREATION

main.py

```
a = 'single quotes'
b = "double quotes"
c = "It's easy"
d = 'She said "Python is fun"'
e = """He said: "It's great!" """
empty = ""
print(a, "|", b, "|", c, "|", d, "|", e)
print("type:", type(a).__name__, "| empty length:", len(empty), "| one char:", type("x").__name__)
print(str(3.14) + str(True) + str([1, 2]))
```

OUTPUT

```
single quotes | double quotes | It's easy | She said "Python is fun" | He said: "It's great!"
type: str | empty length: 0 | one char: str
3.14True[1, 2]
```

7.1.2 Multi-line strings

A triple-quoted string keeps line breaks and indentation exactly as typed, which suits long messages, SQL queries, HTML templates and docstrings. Because it also keeps leading spaces, text inside an indented function picks up that indentation; `textwrap.dedent()` removes the common indentation. Adjacent string literals are joined automatically, a way to write one long line over several source lines inside parentheses.

MULTI-LINE STRINGS main.py

```
import textwrap

poem = """Twinkle, twinkle, little star,
How I wonder what you are."""
print(poem)
print("lines:", len(poem.splitlines()))

def email(name):
    return textwrap.dedent(f"""\
        Dear {name},
            Your class starts on Monday.
        Regards, LBF""")
print(email("Asha"))

long_line = ("This is one long sentence "
             "written across two lines of code.")
print(long_line)
```

OUTPUT

```
Twinkle, twinkle, little star,
How I wonder what you are.
lines: 2
Dear Asha,
    Your class starts on Monday.
Regards, LBF
This is one long sentence written across two lines of code.
```

7.1.3 Escape sequences and raw strings r"..."

Inside a normal string, the backslash `\` starts an **escape sequence** for characters that are hard to type: `\n` newline, `\t` tab, `\\` a backslash, `\'` and `\"` quotes, `\r` carriage return, `\u20b9` a Unicode character by code (here ₹), `\N{...}` by name, and `\x41` a byte value. A **raw string**, prefixed with `r`, treats backslashes as ordinary characters, which is ideal for Windows paths and regular expressions: `r"C:\new\table"`. (A raw string cannot end with an odd number of backslashes.)

ESCAPE SEQUENCES AND RAW STRINGS

main.py

```
print("Line 1\nLine 2")
print("Name:\tAsha\nCity:\tMysuru")
print("Backslash: \\ | quote: \" | apostrophe: \'")
print("Rupee: \u20b9 | Snake: \N{SNAKE} | A as hex: \x41")
path = "C:\new\table"                # \n and \t are interpreted!
print(path)
raw = r"C:\new\table"                # raw: backslashes kept
print(raw, "| length:", len(raw))
print(repr("tab\there"), "| repr shows escapes")
```

OUTPUT

```
Line 1
Line 2
Name:   Asha
City:   Mysuru
Backslash: \ | quote: " | apostrophe: '
Rupee: ₹ | Snake: 🐍 | A as hex: A
C:
ew      able
C:\new\table | length: 12
'tab\there' | repr shows escapes
```

Escape	Meaning
`\n`	newline
`\t`	tab
`\\`	backslash
`\'` `\"`	single / double quote
`\r`	carriage return
`\uXXXX` `\UXXXXXXXX`	Unicode character by hex code
`\N{name}`	Unicode character by name
`\xhh`	character with hex value hh
`\0`	null character

7.1.4 Unicode and encode() / decode()

Python 3 strings are sequences of **Unicode code points**, so every language and emoji works: Kannada, Hindi, Tamil, Chinese, ₹, 😀. `len()` counts code points, not bytes. When text is saved to a file or sent over a network it must be turned into **bytes** with an **encoding**, almost always **UTF-8**, which uses 1 byte for ASCII characters and 2-4 bytes for others. `str.encode()` converts text to bytes and `bytes.decode()` converts back; decoding with the wrong encoding raises `UnicodeDecodeError` or produces garbled text ("mojibake").

UNICODE TEXT AND UTF-8 BYTES

main.py

```
for text in ("Hello", "नमस्ते", "ಕನ್ನಡ", "₹500", "😀"):
    data = text.encode("utf-8")
    print(f"{text:<8} chars: {len(text):<2} UTF-8 bytes: {len(data):<3} {data}")

encoded = "Café".encode("utf-8")
print("decoded correctly:", encoded.decode("utf-8"))
print("decoded with the wrong encoding:", encoded.decode("latin-1"))
try:
    "₹".encode("ascii")
except UnicodeEncodeError as e:
    print("UnicodeEncodeError:", e.reason)
print("replace unencodable characters:", "Price ₹500".encode("ascii", errors="replace"))
```

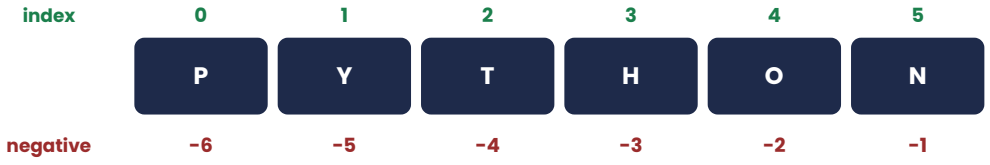
OUTPUT

```
Hello    chars: 5  UTF-8 bytes: 5   b'Hello'
नमस्ते   chars: 6  UTF-8 bytes: 18
b'\xe0\xa4\xa8\xe0\xa4\xae\xe0\xa4\xb8\xe0\xa5\x8d\xe0\xa4\xa4\xe0\xa5\x87'
ಕನ್ನಡ    chars: 5  UTF-8 bytes: 15  b'\xe0\xb2\x95\xe0\xb2\xa8\xe0\xb3\x8d\xe0\xb2\xa8\xe0\xb2\xa1'
₹500     chars: 4  UTF-8 bytes: 6   b'\xe2\x82\xb9500'
😀        chars: 1  UTF-8 bytes: 4   b'\xf0\x9f\x98\x80'
decoded correctly: Café
decoded with the wrong encoding: CafÃ©
UnicodeEncodeError: ordinal not in range(128)
replace unencodable characters: b'Price ?500'
```

7.2 Accessing characters

7.2.1 Positive and negative indexing

Each character has a position, its **index**, starting at **0**. `s[0]` is the first character and `s[len(s) - 1]` the last. **Negative indices** count from the end: `s[-1]` is the last character, `s[-2]` the second last. An index outside the string raises `IndexError`.



POSITIVE AND NEGATIVE INDICES OF "PYTHON"

INDEXING

main.py

```
s = "PYTHON"
print(s[0], s[1], s[5], "|", s[-1], s[-2], s[-6])
print("last character:", s[len(s) - 1], "=", s[-1])
s[10]
```

OUTPUT

```
P Y N | N O P
last character: N = N
Traceback (most recent call last):
  File "main.py", line 4, in <module>
    s[10]
    ~^^^^
IndexError: string index out of range
```

7.2.2 Slicing s[start:stop:step]

A **slice** extracts a substring: `s[start:stop]` takes characters from `start` up to but **not including** `stop`, so its length is stop - start. Omitted `start` means the beginning, omitted `stop` the end. An optional `step` takes every n-th character; a negative step goes backwards. Negative indices work in slices too. Slicing never raises `IndexError`: out-of-range bounds are clipped, so `s[2:100]` is fine. A slice always returns a **new** string.

SLICING

main.py

```
s = "LEARN BEING FORWARD"
print(repr(s[0:5]), repr(s[6:11]), repr(s[12:]))
print(repr(s[:5]), repr(s[-7:]), repr(s[:-8]))
print(repr(s[::2]), "| every 2nd character")
print(repr(s[1:10:3]))
print(repr(s[2:100]), "| out-of-range bounds are clipped")
print(repr(s[5:2]), "| empty when start >= stop with a positive step")
date = "2026-03-09"
print("year", date[:4], "| month", date[5:7], "| day", date[8:])
email = "asha.rao@learnbeingforward.in"
print("user:", email[:email.index("@")], "| domain:", email[email.index("@") + 1:])
```

OUTPUT

```
'LEARN' 'BEING' 'FORWARD'
'LEARN' 'FORWARD' 'LEARN BEING'
'LANBIGFRAD' | every 2nd character
'ENE'
'ARN BEING FORWARD' | out-of-range bounds are clipped
'' | empty when start >= stop with a positive step
year 2026 | month 03 | day 09
user: asha.rao | domain: learnbeingforward.in
```

Slice	Meaning	"PYTHON" gives
`s[1:4]`	index 1 up to 4 (excluded)	"YTH"
`s[:3]`	first 3 characters	"PYT"
`s[3:]`	from index 3 to the end	"HON"
`s[-2:]`	last 2 characters	"ON"
`s[:-2]`	all but the last 2	"PYTH"

Slice	Meaning	"PYTHON" gives
`s[::2]`	every second character	"PTO"
`s[::-1]`	reversed	"NOHTYP"
`s[:]`	a full copy	"PYTHON"

7.2.3 Reversing with s[::-1]

A slice with step -1 and no bounds walks the whole string backwards: `s[::-1]` is the idiomatic way to reverse a string. `"".join(reversed(s))` does the same. Reversal is the basis of palindrome checks.

REVERSING A STRING main.py

```
word = "Python"
print(word[::-1])
print("".join(reversed(word)))
print("racecar"[::-1] == "racecar", "| level is a palindrome:", "level" == "level"[::-1])
print("every second character, backwards:", "ABCDEFGH"[::-2])
```

OUTPUT

```
nohtyP
nohtyP
True | level is a palindrome: True
every second character, backwards: HFDB
```

7.2.4 Immutability

Strings are **immutable**: once created, their characters cannot be changed. `s[0] = "J"` raises `TypeError`. Every operation that seems to modify a string (concatenation, `replace`, `upper`) returns a **new** string, and you must keep the result: `s.upper()` alone does nothing to `s`. Immutability makes strings safe to share and usable as dictionary keys.

STRINGS CANNOT BE CHANGED IN PLACE main.py

```
s = "Python"
try:
    s[0] = "J"
except TypeError as e:
    print("TypeError:", e)

s.upper()                   # returns a new string, which is thrown away
print("unchanged:", s)
s = s.upper()               # keep the result
print("re-bound:", s)
s = "J" + s[1:]             # build a new string from pieces
print("new string:", s)
```

OUTPUT

```
TypeError: 'str' object does not support item assignment
unchanged: Python
re-bound: PYTHON
new string: JYTHON
```

7.3 String operations

7.3.1 Concatenation + and repetition *

`+` joins two strings into a new one (both operands must be strings: convert numbers with `str()` or use an f-string), and `*` repeats a string a whole number of times.

JOINING AND REPEATING main.py

```
first, last = "Asha", "Rao"
full = first + " " + last
print(full)
print("Marks: " + str(91))
print("=" * 25)
print("ha" * 3, "| zero times:", repr("ha" * 0))
```

OUTPUT

```
Asha Rao
Marks: 91
=========================
hahaha | zero times: ''
```

7.3.2 Membership in

`sub in s` tests whether one string occurs anywhere inside another (a **substring** test), and `not in` the opposite. It is case-sensitive.

SUBSTRING TESTS main.py

```
sentence = "Python is popular for data science"
print("data" in sentence, "Data" in sentence, "java" not in sentence)
print("case-insensitive:", "Data".lower() in sentence.lower())
email = "asha@lbf.in"
print("looks like an email:", "@" in email and "." in email.split("@")[-1])
```

OUTPUT

```
True False True
case-insensitive: True
looks like an email: True
```

7.3.3 Comparison and lexicographic order

Strings compare **lexicographically**, like words in a dictionary but by Unicode code point: the first differing character decides, and a prefix is smaller than the longer string (`"app" < "apple"`). All upper-case letters come before all lower-case ones (`"Z" < "a"`) and digits before letters, so for human ordering compare lower-cased (or `casefold()`ed) versions. Numbers stored as text compare character by character: `"10" < "9"`.

ORDERING STRINGS

main.py

```
print("apple" < "banana", "app" < "apple", "Zoo" < "apple", "abc" == "abc")
words = ["banana", "Apple", "cherry", "apple"]
print(sorted(words))
print(sorted(words, key=str.lower))
print(max("Asha", "Ravi", "Meera"), min("Asha", "Ravi", "Meera"))
print("'10' < '9':", "10" < "9", "| as numbers:", 10 < 9)
print(sorted(["file10.txt", "file2.txt", "file1.txt"]))
```

OUTPUT

```
True True True True
['Apple', 'apple', 'banana', 'cherry']
['Apple', 'apple', 'banana', 'cherry']
Ravi Asha
'10' < '9': True | as numbers: False
['file1.txt', 'file10.txt', 'file2.txt']
```

7.3.4 len(), iterating over a string

`len(s)` returns the number of characters. A `for` loop visits the characters one by one; `enumerate` adds positions. Strings are sequences, so `sorted()`, `list()`, `set()`, `max()`, `reversed()` and unpacking all work on them.

LENGTH AND ITERATION

main.py

```
word = "banana"
print("length:", len(word))
for ch in word:
    print(ch, end=" ")
print()
for i, ch in enumerate(word):
    if ch == "a":
        print("'a' at index", i)
print(sorted(word), set(word), list("abc"))
a, b, c = "XYZ"
print(a, b, c)
```

OUTPUT

```
length: 6
b a n a n a
'a' at index 1
'a' at index 3
'a' at index 5
['a', 'a', 'a', 'b', 'n', 'n'] {'b', 'n', 'a'} ['a', 'b', 'c']
X Y Z
```

7.4 String methods

Strings have more than 40 methods, called as `s.method(...)`. None of them changes the original string; they return new strings, numbers, booleans or lists. `dir(str)` lists them and `help(str.method)` explains one.

7.4.1 Case: upper(), lower(), title(), capitalize(), swapcase(), casefold()

`upper()` and `lower()` convert the whole string; `title()` capitalises the first letter of every word; `capitalize()` only the first character of the string (and lower-cases the rest); `swapcase()` inverts each

letter. `casefold()` is a more aggressive `lower()` designed for case-insensitive comparison across languages (the German ß becomes ss).

CHANGING CASE main.py

```
s = "learn BEING forward"
print(s.upper())
print(s.lower())
print(s.title())
print(s.capitalize())
print(s.swapcase())
print("Straße".lower(), "|", "Straße".casefold(), "| equal ignoring case:", "STRASSE".casefold() ==
"Straße".casefold())
print("title() pitfall:", "o'neil's python class".title())
```

OUTPUT

```
LEARN BEING FORWARD
learn being forward
Learn Being Forward
Learn being forward
LEARN being FORWARD
straße | strasse | equal ignoring case: True
title() pitfall: O'Neil'S Python Class
```

7.4.2 Searching: find(), rfind(), index(), count(), startswith(), endswith()

`find(sub)` returns the index of the first occurrence of a substring, or **-1** if absent; `rfind()` searches from the right. `index()` and `rindex()` do the same but raise `ValueError` when not found. All accept optional start and end positions. `count(sub)` counts non-overlapping occurrences. `startswith()` and `endswith()` test the beginning and end, and accept a tuple of alternatives.

SEARCHING INSIDE STRINGS main.py

```
s = "banana bandana"
print("find('an'):", s.find("an"), "| rfind('an'):", s.rfind("an"), "| find('x'):", s.find("x"))
print("find('an', 3):", s.find("an", 3), "| index('band'):", s.index("band"))
print("count('an'):", s.count("an"), "| count('a'):", s.count("a"))
try:
    s.index("x")
except ValueError as e:
    print("index -> ValueError:", e)

file = "report_2026.PDF"
print(file.lower().endswith((".pdf", ".docx")), file.startswith("report"), file.startswith("2026", 7))
```

OUTPUT

```
find('an'): 1 | rfind('an'): 11 | find('x'): -1
find('an', 3): 3 | index('band'): 7
count('an'): 4 | count('a'): 6
index -> ValueError: substring not found
True True True
```

7.4.3 Checking: isalpha(), isdigit(), isalnum(), isspace(), isupper(), islower()

The `is...()` methods return True if **all** characters (and at least one) satisfy a condition: `isalpha()` letters only; `isdigit()` digits only (also `isdecimal()` and `isnumeric()`, which differ for special digits such as ² and Roman numerals); `isalnum()` letters or digits; `isspace()` whitespace only; `isupper()` / `islower()` cased letters all upper/lower; `istitle()` title case; `isidentifier()` a valid Python name. They are handy for validating input before converting it.

TESTING THE KIND OF CHARACTERS main.py

```
tests = ["Python", "2026", "Py3", "  ", "PYTHON", "python", "-5", "3.14", "", "²"]
print(f"{'text':<9}{'alpha':<7}{'digit':<7}{'alnum':<7}{'space':<7}{'upper':<7}lower")
for t in tests:
    print(f"{t!r:<9}{t.isalpha()!s:<7}{t.isdigit()!s:<7}{t.isalnum()!s:<7}{t.isspace()!s:<7}
{t.isupper()!s:<7}{t.islower()}")

pin = "560001"
print("valid PIN code:", len(pin) == 6 and pin.isdigit())
```

OUTPUT

```
text     alpha  digit  alnum  space  upper  lower
'Python' True   False  True   False  False  False
'2026'   False  True   True   False  False  False
'Py3'    False  False  True   False  False  False
'  '     False  False  False  True   False  False
'PYTHON' True   False  True   False  True   False
'python' True   False  True   False  False  True
'-5'     False  False  False  False  False  False
'3.14'   False  False  False  False  False  False
''       False  False  False  False  False  False
'²'      False  True   True   False  False  False
valid PIN code: True
```

7.4.4 Cleaning: strip(), lstrip(), rstrip(), removeprefix(), removesuffix()

`strip()` removes leading and trailing whitespace (spaces, tabs, newlines), the first thing to do with user input and lines read from files; `lstrip()` and `rstrip()` work on one side. Given an argument, they remove any of the listed **characters** (not a substring). `removeprefix()` and `removesuffix()` (Python 3.9+) remove an exact substring at the start or end if present.

CLEANING TEXT main.py

```
raw = "   Asha Rao \n"
print(repr(raw.strip()), repr(raw.lstrip()), repr(raw.rstrip()))
print(repr("***SALE***".strip("*")), repr("www.lbf.in".strip("w.")))
print(repr("$1,299.00".lstrip("$").replace(",", "")))

print("https://learnbeingforward.in".removeprefix("https://"))
print("report.pdf".removesuffix(".pdf"), "|", "report.docx".removesuffix(".pdf"))
print("strip('.pdf') removes characters, not the suffix:", repr("report.pdf".strip(".pdf")),
repr("ppd.pdf".strip(".pdf")))
```

OUTPUT

```
'Asha Rao' 'Asha Rao \n' '   Asha Rao'
'SALE' 'lbf.in'
'1299.00'
learnbeingforward.in
report | report.docx
strip('.pdf') removes characters, not the suffix: 'report' ''
```

7.4.5 Splitting and joining: split(), rsplit(), splitlines(), join()

`split(sep=None, maxsplit=-1)` breaks a string into a list. With no separator it splits on any run of whitespace and ignores leading and trailing spaces; with a separator such as `","` it splits exactly there (empty fields included). `maxsplit` limits the number of splits, and `rsplit()` counts them from the right. `splitlines()` splits into lines. `partition(sep)` splits once into (before, sep, after). The opposite,

`sep.join(iterable)`, concatenates strings with a separator; it is called on the **separator**, and all items must be strings.

SPLITTING AND JOINING main.py

```
print("  Python   is   fun ".split())
print("Asha,91,,Mysuru".split(","))
print("2026-03-09".split("-"))
print("a b c d".split(" ", 2), "|", "a b c d".rsplit(" ", 1))
print("line 1\nline 2\r\nline 3".splitlines())
print("name=Asha".partition("="))

words = ["Learn", "Being", "Forward"]
print(" ".join(words), "|", "-".join(words), "|", "".join(words))
print(", ".join(str(n) for n in [1, 2, 3]))        # items must be strings
path_parts = "home/asha/docs/notes.txt".split("/")
print("file name:", path_parts[-1], "| folder:", "/".join(path_parts[:-1]))
```

OUTPUT

```
['Python', 'is', 'fun']
['Asha', '91', '', 'Mysuru']
['2026', '03', '09']
['a', 'b', 'c d'] | ['a b c', 'd']
['line 1', 'line 2', 'line 3']
('name', '=', 'Asha')
Learn Being Forward | Learn-Being-Forward | LearnBeingForward
1, 2, 3
file name: notes.txt | folder: home/asha/docs
```

7.4.6 Replacing: replace(), translate(), maketrans()

`replace(old, new, count=-1)` returns a copy with every occurrence of `old` replaced (or only the first `count`). For replacing or deleting many **single characters** at once, build a translation table with `str.maketrans()` and apply it with `translate()`: much faster than chained `replace` calls.

REPLACING TEXT main.py

```
s = "I like Java. Java is great."
print(s.replace("Java", "Python"))
print(s.replace("Java", "Python", 1))
print("1,23,456".replace(",", ""))

leet = str.maketrans("aeios", "43105")
print("leet speak:", "python is awesome".translate(leet))
no_punct = str.maketrans("", "", ".,!?;:")              # third argument: characters to delete
print("Hello, world! How are you?".translate(no_punct))
vowels_to_star = str.maketrans({c: "*" for c in "aeiouAEIOU"})
print("Learn Being Forward".translate(vowels_to_star))
```

OUTPUT

```
I like Python. Python is great.
I like Python. Java is great.
123456
leet speak: pyth0n 15 4w350m3
Hello world How are you
L**rn B**ng F*rw*rd
```

7.4.7 Alignment: center(), ljust(), rjust(), zfill()

`center(width, fill=" ")`, `ljust()` and `rjust()` pad a string to a given width, centred, left-aligned or right-aligned, with an optional fill character. `zfill(width)` pads a number string with leading zeros, keeping a sign at the front. f-string alignment (`:^20`, `:<10`) does the same inside formatting.

PADDING AND ALIGNMENT

main.py

```
print("[" + "LBF".center(11) + "]", "[" + "LBF".center(11, "*") + "]")
print("[" + "Name".ljust(10) + "]", "[" + "Marks".rjust(8) + "]")
print("42".zfill(6), "-42".zfill(6), "3.5".zfill(6))
for item, price in [("Pen", 10), ("Notebook", 120), ("Bag", 1499)]:
    print(item.ljust(12, ".") + str(price).rjust(6))
```

OUTPUT

```
[    LBF    ] [****LBF****]
[Name      ] [   Marks]
000042 -00042 0003.5
Pen.........    10
Notebook....   120
Bag.........  1499
```

Category	Methods
Case	`upper lower title capitalize swapcase casefold`
Search	`find rfind index rindex count startswith endswith`
Check	`isalpha isdigit isdecimal isnumeric isalnum isspace isupper islower istitle isidentifier`
Clean	`strip lstrip rstrip removeprefix removesuffix`
Split / join	`split rsplit splitlines partition rpartition join`
Replace	`replace translate maketrans expandtabs`
Align	`center ljust rjust zfill`
Format / encode	`format format_map encode`

7.5 Building strings efficiently

7.5.1 Why += in loops is slow

Because strings are immutable, `result += piece` in a loop has to create a **new** string every time, copying everything built so far. For n pieces this can take time proportional to n^2: fine for 10 pieces, very slow for 100,000. (CPython sometimes optimises `+=` when no other reference to the string exists, but you should not rely on it.)

7.5.2 "".join(list)

The efficient pattern is to collect the pieces in a **list** (appending is cheap) and join them once at the end with `"".join(pieces)`, which calculates the final size and copies each piece only once. A generator expression inside `join()` is even shorter. `io.StringIO` is another option for building large text.

COLLECT, THEN JOIN main.py

```
import time

def build_with_plus(n):
    s = ""
    for i in range(n):
        s = s + str(i) + ","          # a new string each time (no in-place optimisation)
    return s

def build_with_join(n):
    return ",".join(str(i) for i in range(n)) + ","

n = 50_000
t = time.perf_counter(); a = build_with_plus(n); t_plus = time.perf_counter() - t
t = time.perf_counter(); b = build_with_join(n); t_join = time.perf_counter() - t
print("same result:", a == b, "| length:", len(a))
print("join() was faster:", t_join < t_plus)

rows = [("Asha", 91), ("Ravi", 78), ("Meera", 85)]
report = "\n".join(f"{name:<6}{mark:>4}" for name, mark in rows)
print(report)
```

OUTPUT

```
same result: True | length: 288890
join() was faster: True
Asha    91
Ravi    78
Meera   85
```

7.6 Regular expressions (introduction)

A **regular expression** (regex) is a pattern that describes a set of strings, for searching, validating and extracting text far more flexibly than `find()`: "a 10-digit phone number", "every email address", "a date like 09-03-2026". Python's `re` module implements them. Write patterns as **raw strings** (`r"\d+"`) so backslashes reach the regex engine unchanged.

Pattern	Matches	Example
`.`	any character except newline	`a.c` matches abc, a7c
`\d` / `\D`	a digit / a non-digit	`\d\d` matches 42
`\w` / `\W`	word character (letter, digit, _) / other	`\w+` matches Asha_1
`\s` / `\S`	whitespace / non-whitespace	
`[abc]` `[a-z]` `[^0-9]`	one of / a range / not in	`[A-Z][a-z]+` matches Asha
`*` `+` `?`	0 or more / 1 or more / 0 or 1	`colou?r`
`{n}` `{n,m}`	exactly n / n to m times	`\d{6}` a PIN code
`^` `$`	start / end of the string	`^\d+$` digits only
`( )`	group (capture a part)	`(\d{2})-(\d{2})`
`|`	alternation (or)	`cat|dog`
`\b`	word boundary	`\bis\b`

7.6.1 re.search, re.match, re.findall, re.sub

`re.search(pattern, s)` finds the **first** match anywhere in the string and returns a **match object** (or `None`); `.group()` gives the matched text and `.group(1)` the first parenthesised group. `re.match()` matches only at the **beginning**, and `re.fullmatch()` requires the whole string to match (ideal for validation). `re.findall()` returns a list of **all** matches (or of the groups if the pattern has groups), `re.finditer()` yields match objects, `re.sub(pattern, replacement, s)` replaces matches, and `re.split()` splits on a pattern. Flags such as `re.IGNORECASE` change behaviour, and `re.compile()` prepares a pattern for repeated use.

THE MAIN RE FUNCTIONS main.py

```
import re

text = "Order LBF-1029 placed on 09-03-2026 for Rs 4,999. Order LBF-1030 on 10-03-2026."
m = re.search(r"LBF-(\d+)", text)
print("search:", m.group(), "| group 1:", m.group(1), "| position:", m.span())
print("match at start:", re.match(r"Order", text) is not None, "|", re.match(r"LBF", text))
print("findall order ids:", re.findall(r"LBF-\d+", text))
print("findall dates (groups):", re.findall(r"(\d{2})-(\d{2})-(\d{4})", text))
print("sub (mask ids):", re.sub(r"LBF-\d+", "LBF-****", text))
print("sub with groups (dd-mm-yyyy -> yyyy-mm-dd):", re.sub(r"(\d{2})-(\d{2})-(\d{4})", r"\3-\2-\1",
text))
print("split on punctuation and spaces:", re.split(r"[,.\s]+", "one, two.  three"))
print("ignore case:", re.findall(r"order", text, flags=re.IGNORECASE))
for m in re.finditer(r"Rs ([\d,]+)", text):
    print("amount:", int(m.group(1).replace(",", "")))
```

OUTPUT

```
search: LBF-1029 | group 1: 1029 | position: (6, 14)
match at start: True | None
findall order ids: ['LBF-1029', 'LBF-1030']
findall dates (groups): [('09', '03', '2026'), ('10', '03', '2026')]
sub (mask ids): Order LBF-**** placed on 09-03-2026 for Rs 4,999. Order LBF-**** on 10-03-2026.
sub with groups (dd-mm-yyyy -> yyyy-mm-dd): Order LBF-1029 placed on 2026-03-09 for Rs 4,999. Order
LBF-1030 on 2026-03-10.
split on punctuation and spaces: ['one', 'two', 'three']
ignore case: ['Order', 'Order']
amount: 4999
```

7.6.2 Common patterns (digits, emails, phone numbers)

Practical patterns combine these building blocks. Real-world validation is often stricter or looser than a simple regex (email addresses in particular), so use regexes for extraction and basic checks, and libraries for full validation. Test patterns interactively on sites such as regex101.com.

EXTRACTING COMMON DATA WITH REGULAR EXPRESSIONS

main.py

```
import re

patterns = {
    "Indian mobile": r"(?:\+91[\s-]?)?[6-9]\d{9}",
    "email": r"[\w.+-]+@[\w-]+(?:\.[\w-]+)+",
    "PIN code": r"\b\d{6}\b",
    "date dd/mm/yyyy": r"\b\d{2}/\d{2}/\d{4}\b",
    "price in rupees": r"(?:Rs\.?|₹)\s?[\d,]+(?:\.\d{2})?",
    "hashtag": r"#\w+",
}
text = """Contact Asha at asha.rao@lbf.in or +91 9845012345, office 080-23456789.
Ship to Mysuru 570001 by 15/04/2026. Fee ₹14,999.00 (Rs. 12,999 for alumni). #Python #LBF"""
for name, pat in patterns.items():
    print(f"{name:<16} {re.findall(pat, text)}")

def is_valid_mobile(number):
    return re.fullmatch(r"[6-9]\d{9}", number) is not None
print([is_valid_mobile(n) for n in ("9845012345", "1234567890", "98450")])
```

OUTPUT

```
Indian mobile    ['+91 9845012345']
email            ['asha.rao@lbf.in']
PIN code         ['570001']
date dd/mm/yyyy  ['15/04/2026']
price in rupees  ['₹14,999.00', 'Rs. 12,999']
hashtag          ['#Python', '#LBF']
[True, False, False]
```

7.7 Classic string problems

7.7.1 Palindrome and anagram check

A **palindrome** reads the same forwards and backwards (madam, "Never odd or even" ignoring spaces and case). Two words are **anagrams** if they contain the same letters in a different order (listen and silent): compare their sorted letters, or their letter counts.

PALINDROMES AND ANAGRAMS

main.py

```
from collections import Counter

def is_palindrome(text):
    cleaned = "".join(ch.lower() for ch in text if ch.isalnum())
    return cleaned == cleaned[::-1]

def is_palindrome_two_pointers(text):
    s = [ch.lower() for ch in text if ch.isalnum()]
    i, j = 0, len(s) - 1
    while i < j:
        if s[i] != s[j]:
            return False
        i, j = i + 1, j - 1
    return True

for t in ("madam", "Never odd or even", "A man, a plan, a canal: Panama", "python"):
    print(f"{t!r:<34} {is_palindrome(t)} {is_palindrome_two_pointers(t)}")

def is_anagram(a, b):
    clean = lambda s: s.replace(" ", "").lower()
    return sorted(clean(a)) == sorted(clean(b))

for a, b in [("listen", "silent"), ("Dormitory", "dirty room"), ("hello", "world")]:
    print(f"{a} / {b}: {is_anagram(a, b)} | Counter check: {Counter(a.replace(' ', '').lower()) ==
Counter(b.replace(' ', '').lower())}")
```

OUTPUT

```
'madam'                          True True
'Never odd or even'              True True
'A man, a plan, a canal: Panama'  True True
'python'                         False False
listen / silent: True | Counter check: True
Dormitory / dirty room: True | Counter check: True
hello / world: False | Counter check: False
```

7.7.2 Count vowels, consonants, words

COUNTING KINDS OF CHARACTERS

main.py

```
text = "Learn Being Forward makes Python easy to learn!"
vowels = consonants = digits = spaces = others = 0
for ch in text:
    if ch.isalpha():
        if ch.lower() in "aeiou":
            vowels += 1
        else:
            consonants += 1
    elif ch.isdigit():
        digits += 1
    elif ch.isspace():
        spaces += 1
    else:
        others += 1
print(f"vowels {vowels}, consonants {consonants}, digits {digits}, spaces {spaces}, others {others}")
print("words:", len(text.split()), "| using sum():", sum(ch.lower() in "aeiou" for ch in text),
"vowels")
```

OUTPUT

```
vowels 14, consonants 25, digits 0, spaces 7, others 1
words: 8 | using sum(): 14 vowels
```

7.7.3 Character frequency and first non-repeating character

FREQUENCIES AND THE FIRST UNIQUE CHARACTER

main.py

```
from collections import Counter

s = "programming"
freq = {}
for ch in s:
    freq[ch] = freq.get(ch, 0) + 1          # manual counting with a dict
print("frequencies:", freq)
print("Counter:", Counter(s).most_common(3))

def first_non_repeating(s):
    counts = Counter(s)
    for ch in s:                             # keep the original order
        if counts[ch] == 1:
            return ch
    return None

for word in ("programming", "swiss", "aabb"):
    print(word, "->", first_non_repeating(word))
```

OUTPUT

```
frequencies: {'p': 1, 'r': 2, 'o': 1, 'g': 2, 'a': 1, 'm': 2, 'i': 1, 'n': 1}
Counter: [('r', 2), ('g', 2), ('m', 2)]
programming -> p
swiss -> w
aabb -> None
```

7.7.4 Remove duplicates, capitalise each word, string compression

MORE CLASSIC MANIPULATIONS

main.py

```
def remove_duplicates(s):
    seen, out = set(), []
    for ch in s:
        if ch not in seen:
            seen.add(ch)
            out.append(ch)
    return "".join(out)

def capitalise_words(s):
    return " ".join(word[:1].upper() + word[1:].lower() for word in s.split())

def compress(s):
    """aaabccdddd -> a3b1c2d4 (returned only if shorter)."""
    if not s:
        return s
    out, count = [], 1
    for prev, ch in zip(s, s[1:]):
        if ch == prev:
            count += 1
        else:
            out.append(f"{prev}{count}")
            count = 1
    out.append(f"{s[-1]}{count}")
    result = "".join(out)
    return result if len(result) < len(s) else s

print(remove_duplicates("programming"), "|", "".join(dict.fromkeys("programming")))
print(capitalise_words("o'neil's   PYTHON class"))
for s in ("aaabccdddd", "abc", "aabbbcccc"):
    print(f"{s} -> {compress(s)}")
```

OUTPUT

```
progamin | progamin
O'neil's Python Class
aaabccdddd -> a3b1c2d4
abc -> abc
aabbbcccc -> a2b3c4
```

Strings are immutable sequences of Unicode characters, written with single, double or triple quotes; escapes such as \n and \t, raw strings r"..." for paths and regexes, and encode()/decode() to convert to UTF-8 bytes. Index from 0 (negative indices count from the end) and slice with s[start:stop:step] (stop excluded, bounds clipped, s[::-1] reverses); every "change" creates a new string. + concatenates, * repeats, in tests substrings, comparison is lexicographic by code point. Methods cover case (upper, lower, title, casefold), searching (find returns -1, index raises, count, startswith/endswith), checking (isalpha, isdigit, isalnum, isspace), cleaning (strip, removeprefix/removesuffix), splitting and joining (split, splitlines, partition, sep.join), replacing (replace, translate with maketrans) and alignment (center, ljust, rjust, zfill). Build long strings with "".join, not += in loops. The re module searches (search, match, fullmatch), extracts (findall, finditer), replaces (sub) and splits with patterns such as \d, \w, [a-z], +, {n} and groups.

PRACTICE CHECKPOINT

Write a **word counter**, a **palindrome checker** that ignores case and spaces, and a **regex** that extracts all **email addresses** from a paragraph.

SAMPLE SOLUTION

main.py

```
import re
from collections import Counter

paragraph = """Python is easy to learn. Python is also powerful, and Python is everywhere!
For admissions write to admissions@learnbeingforward.in, for fees to accounts.team@lbf.in,
and the trainer Asha can be reached at Asha.Rao+python@gmail.com. Not an email: info@ or @lbf."""

# 1. Word counter
words = re.findall(r"[a-z']+", paragraph.lower())
counts = Counter(words)
print(f"total words: {len(words)}, distinct: {len(counts)}")
for word, n in counts.most_common(4):
    print(f"  {word:<6} {n}")

# 2. Palindrome checker ignoring case, spaces and punctuation
def is_palindrome(text):
    cleaned = "".join(ch for ch in text.casefold() if ch.isalnum())
    return bool(cleaned) and cleaned == cleaned[::-1]

for phrase in ["Madam", "Was it a car or a cat I saw", "Step on no pets!", "Python"]:
    print(f"{phrase!r:<32} palindrome: {is_palindrome(phrase)}")

# 3. Extract every email address
emails = re.findall(r"[\w.+-]+@[\w-]+(?:\.[\w-]+)+", paragraph)
print("emails found:", emails)
print("domains:", sorted({e.split("@")[1].lower() for e in emails}))
```

OUTPUT

```
total words: 46, distinct: 33
  python 4
  is     3
  to     3
  and    2
'Madam'                          palindrome: True
'Was it a car or a cat I saw'    palindrome: True
'Step on no pets!'               palindrome: True
'Python'                         palindrome: False
emails found: ['admissions@learnbeingforward.in', 'accounts.team@lbf.in', 'Asha.Rao+python@gmail.com']
domains: ['gmail.com', 'lbf.in', 'learnbeingforward.in']
```

MODULE 08 · BEGINNER

Lists and tuples

Minimum duration **3 days** | Level **Beginner**

Lists are ordered, mutable collections; tuples are ordered, immutable collections. Together they are Python's main sequence types. This module covers creating, indexing, slicing and modifying lists, nested lists and matrices, every list method, copying (assignment, shallow and deep copies and the [[0]*3]*3 trap), list comprehensions, built-in functions that work on lists, lists as stacks and queues, tuples with packing, unpacking and named tuples, and classic list problems: searching, sorting and matrix operations.

IN THIS MODULE

8.1 Lists
8.2 List methods
8.3 Copying in depth
8.4 List comprehensions
8.5 Built-in functions with lists
8.6 Lists as stacks and queues
8.7 Tuples
8.8 List problems

8.1 Lists

8.1.1 Creating lists (literal, list(), from range)

A **list** is an ordered collection of items written in square brackets and separated by commas: `[91, 78, 85]`. Items can be of any type, including mixed types and other lists, and duplicates are allowed. `[]` or `list()` creates an empty list; `list(iterable)` builds a list from any iterable (a string, a range, a tuple, a set); `[value] * n` repeats a value; and list comprehensions (8.4) build lists from expressions. Internally a list is a dynamic array of references to objects.

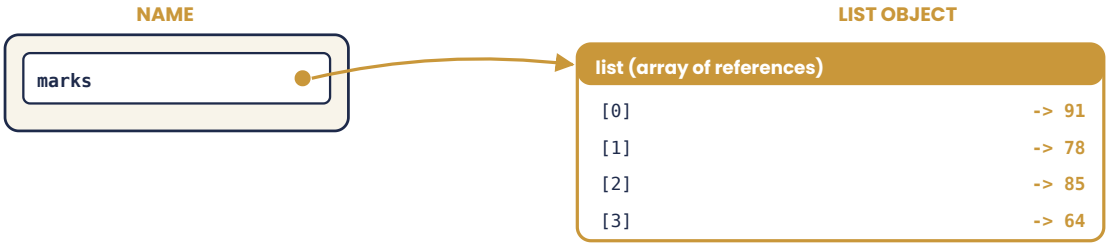
WAYS TO CREATE LISTS

main.py

```
marks = [91, 78, 85, 64]
mixed = ["Asha", 21, 8.7, True, None, [1, 2]]
empty1, empty2 = [], list()
letters = list("python")
numbers = list(range(1, 11))
evens = list(range(0, 21, 2))
zeros = [0] * 5
squares = [n * n for n in range(1, 6)]
print(marks, mixed, empty1, empty2, sep="\n")
print(letters, numbers, evens, zeros, squares, sep="\n")
print("length:", len(marks), "| type:", type(marks).__name__)
```

OUTPUT

```
[91, 78, 85, 64]
['Asha', 21, 8.7, True, None, [1, 2]]
[]
[]
['p', 'y', 't', 'h', 'o', 'n']
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
[0, 2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
[0, 0, 0, 0, 0]
[1, 4, 9, 16, 25]
length: 4 | type: list
```



A LIST STORES REFERENCES TO ITS ITEMS

8.1.2 Indexing, negative indexing and slicing

Lists use the same indexing and slicing as strings (Module 7): `lst[0]` is the first item, `lst[-1]` the last, `lst[1:3]` a new list with items 1 and 2, `lst[::-1]` a reversed copy. An index out of range raises `IndexError`, while slices are clipped.

ACCESSING ITEMS AND SLICES

main.py

```
courses = ["Python", "Java", "C", "SQL", "Git"]
print(courses[0], courses[-1], courses[2])
print(courses[1:3], courses[:2], courses[3:], courses[-2:])
print(courses[::2], courses[::-1])
print(courses[1:100], "| last index:", len(courses) - 1)
courses[5]
```

OUTPUT

```
Python Git C
['Java', 'C'] ['Python', 'Java'] ['SQL', 'Git'] ['SQL', 'Git']
['Python', 'C', 'Git'] ['Git', 'SQL', 'C', 'Java', 'Python']
['Java', 'C', 'SQL', 'Git'] | last index: 4
Traceback (most recent call last):
  File "main.py", line 6, in <module>
    courses[5]
    ~~~~~~~^^^
IndexError: list index out of range
```

8.1.3 Lists are mutable; modifying elements and slices

Unlike strings, lists are **mutable**: you can change items in place by assigning to an index, and even replace, insert or delete whole **slices**. Assigning to a slice can change the list's length: `lst[1:3] = ["a", "b", "c"]` replaces two items with three; `lst[1:1] = [...]` inserts; `lst[1:3] = []` deletes. The list keeps its identity, so all names referring to it see the changes.

CHANGING ITEMS AND SLICES

main.py

```
nums = [10, 20, 30, 40, 50]
print("id before:", id(nums) % 10000)
nums[0] = 11                    # change one item
nums[-1] *= 2                   # 50 -> 100
print(nums)
nums[1:3] = [21, 22, 23]        # replace 2 items with 3
print(nums)
nums[1:1] = [15]                # insert at index 1
print(nums)
nums[2:5] = []                  # delete a slice
print(nums)
nums[::2] = [0, 0]              # extended slice: same length required
print(nums, "| id after:", id(nums) % 10000)
```

OUTPUT

```
id before: 2256
[11, 20, 30, 40, 100]
[11, 21, 22, 23, 40, 100]
[11, 15, 21, 22, 23, 40, 100]
[11, 15, 40, 100]
[0, 15, 0, 100] | id after: 2256
```

8.1.4 Nested lists and 2D lists (matrices)

A list can contain lists, giving **nested** structures such as a 2D table or **matrix**: a list of rows, each row a list of values. `m[i]` is row i and `m[i][j]` the item in row i, column j. Loop over rows and then over the items in each row. Create matrices with a comprehension, never with `[[0] * cols] * rows` (8.3.3).

MATRICES AS LISTS OF LISTS

main.py

```
matrix = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9],
]
print("row 1:", matrix[1], "| item [1][2]:", matrix[1][2], "| rows:", len(matrix), "| cols:",
len(matrix[0]))
for row in matrix:
    print(" ".join(f"{v:3}" for v in row))
print("diagonal:", [matrix[i][i] for i in range(3)])
print("column 0:", [row[0] for row in matrix])
print("sum of all:", sum(sum(row) for row in matrix))

grid = [[0] * 4 for _ in range(3)]       # 3 rows x 4 columns
grid[1][2] = 5
print(grid)
students = [["Asha", [91, 88]], ["Ravi", [78, 72]]]
print(students[0][1][1], "-> Asha's second mark")
```

OUTPUT

```
row 1: [4, 5, 6] | item [1][2]: 6 | rows: 3 | cols: 3
  1   2   3
  4   5   6
  7   8   9
diagonal: [1, 5, 9]
column 0: [1, 4, 7]
sum of all: 45
[[0, 0, 0, 0], [0, 0, 5, 0], [0, 0, 0, 0]]
88 -> Asha's second mark
```

MATRIX[I][J]: ROW I, COLUMN J (DIAGONAL HIGHLIGHTED)

8.2 List methods

8.2.1 Adding: append(), extend(), insert()

`append(x)` adds one item at the end, the most common way to grow a list (fast, O(1)). `extend(iterable)` adds every item of an iterable at the end (`+=` does the same). Appending a list with `append` adds it as a **single** nested item, a common mistake. `insert(i, x)` inserts before index i (slow for large lists because later items shift). These methods change the list in place and return `None`, so never write `lst = lst.append(x)`.

ADDING ITEMS

main.py

```
cart = ["pen"]
cart.append("book")
print("append:", cart)
cart.extend(["bag", "ruler"])
print("extend:", cart)
cart.append(["eraser", "sharpener"])        # one nested item!
print("append a list:", cart, "| len", len(cart))
cart.pop()
cart.insert(0, "pencil")
cart.insert(2, "glue")
print("insert:", cart)
cart += "ab"                                # extends with each character
print("+= a string:", cart)
result = cart.append("x")
print("append returns:", result)
```

OUTPUT

```
append: ['pen', 'book']
extend: ['pen', 'book', 'bag', 'ruler']
append a list: ['pen', 'book', 'bag', 'ruler', ['eraser', 'sharpener']] | len 5
insert: ['pencil', 'pen', 'glue', 'book', 'bag', 'ruler']
+= a string: ['pencil', 'pen', 'glue', 'book', 'bag', 'ruler', 'a', 'b']
append returns: None
```

8.2.2 Removing: remove(), pop(), clear(), del

`remove(x)` deletes the **first** item equal to x (ValueError if absent). `pop()` removes and **returns** the last item; `pop(i)` the item at index i (IndexError on an empty list). `clear()` empties the list. The `del` statement deletes by index or slice. Do not remove items from a list while looping over it, because indices shift and items get skipped; loop over a copy or build a new list with a comprehension.

REMOVING ITEMS SAFELY main.py

```
nums = [10, 20, 30, 20, 40, 50]
nums.remove(20)                    # first 20 only
print("remove(20):", nums)
last = nums.pop()
print("pop():", last, nums)
second = nums.pop(1)
print("pop(1):", second, nums)
del nums[0]
print("del nums[0]:", nums)
nums.extend([1, 2, 3, 4])
del nums[1:3]
print("del slice:", nums)
nums.clear()
print("clear():", nums)
try:
    nums.remove(99)
except ValueError as e:
    print("ValueError:", e)

scores = [35, 20, 80, 15, 90]
for s in scores:                   # wrong: modifying while iterating
    if s < 40:
        scores.remove(s)
print("buggy removal:", scores)
scores = [35, 20, 80, 15, 90]
scores = [s for s in scores if s >= 40]
print("correct filter:", scores)
```

OUTPUT

```
remove(20): [10, 30, 20, 40, 50]
pop(): 50 [10, 30, 20, 40]
pop(1): 30 [10, 20, 40]
del nums[0]: [20, 40]
del slice: [20, 2, 3, 4]
clear(): []
ValueError: list.remove(x): x not in list
buggy removal: [20, 80, 90]
correct filter: [80, 90]
```

8.2.3 Searching: index(), count(), in

`x in lst` tests membership; `index(x)` returns the position of the first occurrence (with optional start and end, ValueError if absent); `count(x)` counts occurrences. All three scan the list item by item, which is O(n); for frequent lookups in large collections use a set or dict.

FINDING ITEMS

main.py

```
marks = [78, 91, 64, 91, 85]
print(91 in marks, 100 in marks, 100 not in marks)
print("index(91):", marks.index(91), "| next 91:", marks.index(91, 2), "| count(91):",
marks.count(91))
print("all positions of 91:", [i for i, m in enumerate(marks) if m == 91])
target = 100
print("position:", marks.index(target) if target in marks else "not found")
```

OUTPUT

```
True False True
index(91): 1 | next 91: 3 | count(91): 2
all positions of 91: [1, 3]
position: not found
```

8.2.4 Ordering: sort(), reverse(), sorted(), reversed()

`lst.sort()` sorts the list **in place** and returns `None`; the built-in `sorted(iterable)` returns a **new** sorted list and works on any iterable. `lst.reverse()` reverses in place; `reversed(lst)` returns an iterator for looping backwards. Python's sort (Timsort) is stable and O(n log n). Mixed types that cannot be compared (numbers and strings) raise `TypeError`.

SORTING AND REVERSING

main.py

```
nums = [42, 7, 19, 3, 25]
new = sorted(nums)
print("sorted():", new, "| original:", nums)
nums.sort()
print("sort():", nums)
nums.reverse()
print("reverse():", nums)
print("reversed():", list(reversed(nums)))
print("sort() returns:", nums.sort())
print(sorted("python"), sorted((3, 1, 2)), sorted({"b": 1, "a": 2}))
try:
    [3, "a", 1].sort()
except TypeError as e:
    print("TypeError:", e)
```

OUTPUT

```
sorted(): [3, 7, 19, 25, 42] | original: [42, 7, 19, 3, 25]
sort(): [3, 7, 19, 25, 42]
reverse(): [42, 25, 19, 7, 3]
reversed(): [3, 7, 19, 25, 42]
sort() returns: None
['h', 'n', 'o', 'p', 't', 'y'] [1, 2, 3] ['a', 'b']
TypeError: '<' not supported between instances of 'str' and 'int'
```

8.2.5 Sorting with key= and reverse=True

Both `sort()` and `sorted()` accept `reverse=True` for descending order and a `key` function that is called once per item to produce the value to sort by: `key=len`, `key=str.lower`, `key=abs`, or a lambda that picks a field, such as `key=lambda s: s["marks"]`. To sort by several criteria, return a tuple; to sort one criterion descending and another ascending, negate the number. Because the sort is **stable**, items with equal keys keep their original order. `operator.itemgetter` and `attrgetter` are fast alternatives to lambdas.

CUSTOM SORT ORDERS

main.py

```
from operator import itemgetter

words = ["banana", "Kiwi", "apple", "Cherry", "fig"]
print(sorted(words))
print(sorted(words, key=str.lower))
print(sorted(words, key=len, reverse=True))
print(sorted([-7, 3, -1, 5], key=abs))

students = [{"name": "Asha", "marks": 91, "age": 21},
            {"name": "Ravi", "marks": 78, "age": 22},
            {"name": "Meera", "marks": 91, "age": 20},
            {"name": "John", "marks": 64, "age": 21}]
by_marks = sorted(students, key=lambda s: s["marks"], reverse=True)
print([s["name"] for s in by_marks], "(Asha before Meera: stable)")
by_marks_then_age = sorted(students, key=lambda s: (-s["marks"], s["age"]))
print([s["name"] for s in by_marks_then_age])
print([s["name"] for s in sorted(students, key=itemgetter("age", "name"))])
```

OUTPUT

```
['Cherry', 'Kiwi', 'apple', 'banana', 'fig']
['apple', 'banana', 'Cherry', 'fig', 'Kiwi']
['banana', 'Cherry', 'apple', 'Kiwi', 'fig']
[-1, 3, 5, -7]
['Asha', 'Meera', 'Ravi', 'John'] (Asha before Meera: stable)
['Meera', 'Asha', 'Ravi', 'John']
['Meera', 'Asha', 'John', 'Ravi']
```

8.2.6 Copying: copy(), slicing, list()

Assignment never copies a list (Module 3). To get an independent list with the same items, use `lst.copy()`, the full slice `lst[:]` or `list(lst)`: all three create a new **shallow** copy (8.3).

THREE WAYS TO COPY A LIST

main.py

```
original = [1, 2, 3]
alias = original                # not a copy
c1 = original.copy()
c2 = original[:]
c3 = list(original)
original.append(4)
print("alias:", alias, "| copies:", c1, c2, c3)
```

OUTPUT

```
alias: [1, 2, 3, 4] | copies: [1, 2, 3] [1, 2, 3] [1, 2, 3]
```

Method	Effect	Returns
`append(x)`	add x at the end	None
`extend(it)`	add all items of an iterable	None
`insert(i, x)`	insert x before index i	None
`remove(x)`	remove first x (ValueError if absent)	None
`pop(i=-1)`	remove and return item i	the item
`clear()`	remove all items	None

Method	Effect	Returns
`index(x)`	position of first x	int
`count(x)`	number of occurrences	int
`sort(key, reverse)`	sort in place	None
`reverse()`	reverse in place	None
`copy()`	shallow copy	new list

8.3 Copying in depth

8.3.1 Assignment vs shallow copy vs deep copy

There are three levels of "copying". **Assignment** (`b = a`) copies only the reference: one object, two names. A **shallow copy** (`a.copy()`, `a[:]`, `copy.copy(a)`) creates a new outer list, but its items are the **same** objects as in the original; for a list of numbers or strings that is a perfect copy, but for a list of lists, the inner lists are shared. A **deep copy** (`copy.deepcopy(a)`) recursively copies everything, so the result is completely independent.

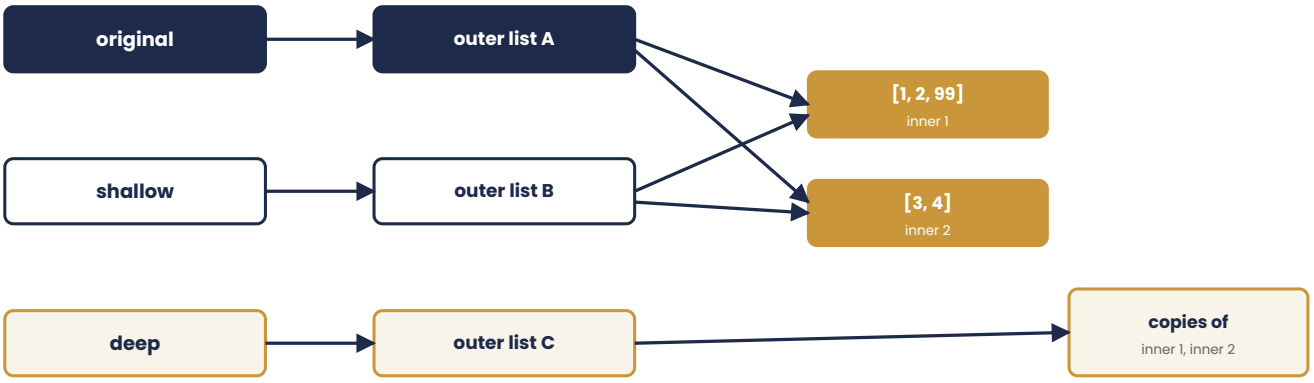
ASSIGNMENT, SHALLOW COPY AND DEEP COPY COMPARED

main.py

```
import copy

original = [[1, 2], [3, 4]]
assigned = original
shallow = original.copy()
deep = copy.deepcopy(original)

original[0].append(99)            # modify an INNER list
original.append([5, 6])           # modify the OUTER list

print("original:", original)
print("assigned:", assigned, "<- same object, sees everything")
print("shallow: ", shallow, "<- new outer list, shares inner lists")
print("deep:    ", deep, "<- fully independent")
print("shallow[0] is original[0]:", shallow[0] is original[0], "| deep[0] is original[0]:", deep[0] is
original[0])
```

OUTPUT

```
original: [[1, 2, 99], [3, 4], [5, 6]]
assigned: [[1, 2, 99], [3, 4], [5, 6]] <- same object, sees everything
shallow:  [[1, 2, 99], [3, 4]] <- new outer list, shares inner lists
deep:     [[1, 2], [3, 4]] <- fully independent
shallow[0] is original[0]: True | deep[0] is original[0]: False
```



A SHALLOW COPY SHARES THE INNER LISTS; A DEEP COPY DOES NOT

8.3.2 copy.deepcopy()

`copy.deepcopy(obj)` copies an object and, recursively, everything it contains: nested lists, dictionaries, sets and objects of your own classes. It handles shared references and cycles correctly by remembering what it has already copied. Use it when you need to modify nested data without affecting the original, for example to try a move in a game board or to keep a snapshot of a configuration. It is slower than a shallow copy, so use it only when needed. `copy.copy()` makes a shallow copy of any object.

INDEPENDENT COPIES OF NESTED DATA main.py

```
import copy

timetable = {"Mon": ["Python", "SQL"], "Tue": ["Java"]}
draft = copy.deepcopy(timetable)
draft["Mon"].append("Git")
draft["Wed"] = ["DSA"]
print("original:", timetable)
print("draft:   ", draft)

board = [["X", "", ""], ["", "O", ""], ["", "", ""]]
trial = copy.deepcopy(board)
trial[2][2] = "X"                          # try a move on the copy
print("board unchanged:", board[2], "| trial:", trial[2])
```

OUTPUT

```
original: {'Mon': ['Python', 'SQL'], 'Tue': ['Java']}
draft:    {'Mon': ['Python', 'SQL', 'Git'], 'Tue': ['Java'], 'Wed': ['DSA']}
board unchanged: ['', '', ''] | trial: ['', '', 'X']
```

8.3.3 The [[0]*3]*3 trap

`[0] * 3` is fine: it repeats references to the immutable number 0. But `[[0] * 3] * 3` creates **one** inner list and repeats a reference to it three times, so all three "rows" are the same object, and changing one cell changes the whole column. Build 2D lists with a comprehension, which creates a new inner list for each row.

REPEATING A LIST OF LISTS main.py

```
bad = [[0] * 3] * 3
bad[0][0] = 1
print("bad: ", bad)
print("rows are the same object:", bad[0] is bad[1] is bad[2])

good = [[0] * 3 for _ in range(3)]
good[0][0] = 1
print("good:", good)
print("rows are different objects:", good[0] is not good[1])
```

OUTPUT

```
bad:  [[1, 0, 0], [1, 0, 0], [1, 0, 0]]
rows are the same object: True
good: [[1, 0, 0], [0, 0, 0], [0, 0, 0]]
rows are different objects: True
```

[[0]*3]*3 REPEATS ONE ROW; A COMPREHENSION CREATES THREE

8.4 List comprehensions

8.4.1 Basic syntax [x*x for x in range(10)]

A **list comprehension** builds a new list from an iterable in one readable expression: `[expression for item in iterable]`. It is equivalent to creating an empty list and appending in a loop, but shorter, clearer and usually faster. Read it as "the list of *expression* for each *item* in *iterable*". The loop variable is local to the comprehension.

FROM LOOP TO COMPREHENSION main.py

```
squares = []
for x in range(10):                # the loop version
    squares.append(x * x)
print(squares)

squares = [x * x for x in range(10)]     # the comprehension
print(squares)
print([name.upper() for name in ["asha", "ravi"]])
print([len(w) for w in "Learn Being Forward".split()])
print([float(m) for m in "91 78.5 85".split()])
```

OUTPUT

```
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
['ASHA', 'RAVI']
[5, 5, 7]
[91.0, 78.5, 85.0]
```

8.4.2 With conditions

An `if` clause at the end **filters** items: `[x for x in data if condition]` keeps only the items for which the condition is true. Several `if` clauses act like `and`.

FILTERING WITH IF main.py

```
marks = [91, 35, 78, 22, 64, 88]
passed = [m for m in marks if m >= 40]
print("passed:", passed)
print("even squares:", [x * x for x in range(1, 11) if x % 2 == 0])
words = ["Python", "is", "a", "great", "language"]
print("long words:", [w for w in words if len(w) > 3])
print("multiples of 3 and 5:", [n for n in range(1, 50) if n % 3 == 0 if n % 5 == 0])
```

OUTPUT

```
passed: [91, 78, 64, 88]
even squares: [4, 16, 36, 64, 100]
long words: ['Python', 'great', 'language']
multiples of 3 and 5: [15, 30, 45]
```

8.4.3 With if-else

To **transform** each item differently instead of filtering, put a conditional expression in the expression part, **before** `for`: `[a if cond else b for x in data]`. The result has the same length as the input. (An `if` after `for` filters; an `if-else` before `for` chooses a value. Both can be combined.)

CHOOSING VALUES WITH IF-ELSE main.py

```
marks = [91, 35, 78, 22]
print(["pass" if m >= 40 else "fail" for m in marks])
print([n if n >= 0 else 0 for n in [5, -3, 8, -1]])          # clamp negatives to 0
print(["Fizz" * (n % 3 == 0) + "Buzz" * (n % 5 == 0) or n for n in range(1, 16)])
print([("A" if m >= 85 else "B") for m in marks if m >= 40])  # filter, then transform
```

OUTPUT

```
['pass', 'fail', 'pass', 'fail']
[5, 0, 8, 0]
[1, 2, 'Fizz', 4, 'Buzz', 'Fizz', 7, 8, 'Fizz', 'Buzz', 11, 'Fizz', 13, 14, 'FizzBuzz']
['A', 'B']
```

8.4.4 Nested comprehensions

A comprehension can have several `for` clauses, which work like nested loops in the same order (outer first): `[(x, y) for x in a for y in b]`. This flattens lists of lists and builds combinations. A comprehension can also contain another comprehension as its expression, which builds nested lists such as matrices. Keep them readable: if a comprehension needs more than two levels, write ordinary loops.

COMPREHENSIONS WITH SEVERAL LOOPS main.py

```
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
flat = [v for row in matrix for v in row]           # for row ... then for v
print("flatten:", flat)
print("pairs:", [(c, n) for c in "AB" for n in range(1, 3)])
transposed = [[row[j] for row in matrix] for j in range(3)]
print("transpose:", transposed)
table = [[i * j for j in range(1, 6)] for i in range(1, 4)]
for row in table:
    print(row)
print("coordinates without the diagonal:", [(i, j) for i in range(3) for j in range(3) if i != j])
```

OUTPUT

```
flatten: [1, 2, 3, 4, 5, 6, 7, 8, 9]
pairs: [('A', 1), ('A', 2), ('B', 1), ('B', 2)]
transpose: [[1, 4, 7], [2, 5, 8], [3, 6, 9]]
[1, 2, 3, 4, 5]
[2, 4, 6, 8, 10]
[3, 6, 9, 12, 15]
coordinates without the diagonal: [(0, 1), (0, 2), (1, 0), (1, 2), (2, 0), (2, 1)]
```

8.5 Built-in functions with lists

8.5.1 len, min, max, sum, any, all

`len` counts items; `min` and `max` find the smallest and largest (with optional `key=` and `default=`); `sum` adds numbers. `any(iterable)` is True if **at least one** item is truthy, `all(iterable)` if **every** item is truthy (and True for an empty iterable). With generator expressions they express checks such as "any student failed" in one line, and they stop as soon as the answer is known.

AGGREGATES AND CHECKS main.py

```
marks = [91, 35, 78, 64]
print("len:", len(marks), "| min:", min(marks), "| max:", max(marks), "| sum:", sum(marks), "| avg:",
sum(marks) / len(marks))
print("any failed?", any(m < 40 for m in marks), "| all passed?", all(m >= 40 for m in marks))
print("any([]):", any([]), "| all([]):", all([]))
names = ["Asha", "Meera", "Jo"]
print("shortest:", min(names, key=len), "| max of empty:", max([], default=None))
print("all non-empty:", all(["a", "b", ""]), "| any truthy:", any([0, "", None, 3]))
```

OUTPUT

```
len: 4 | min: 35 | max: 91 | sum: 268 | avg: 67.0
any failed? True | all passed? False
any([]): False | all([]): True
shortest: Jo | max of empty: None
all non-empty: False | any truthy: True
```

8.5.2 enumerate, zip, map, filter

`enumerate` and `zip` (Module 6) pair items with indices or with items of other lists. `map(function, iterable)` applies a function to every item, and `filter(function, iterable)` keeps the items for which the function returns a truthy value; both return lazy iterators, so wrap them in `list()` to see the results. Comprehensions usually express the same ideas more readably; `map` is concise when the function already exists (`map(int, parts)`). Module 11 covers functional programming in depth.

ENUMERATE, ZIP, MAP AND FILTER main.py

```
names = ["asha", "ravi", "meera"]
marks = [91, 35, 78]
print(list(enumerate(names, 1)))
print(list(zip(names, marks)))
print(list(map(str.title, names)))
print(list(map(int, "10 20 30".split())))
print(list(filter(lambda m: m >= 40, marks)))
print(list(filter(None, [0, 1, "", "a", None, [2]])))      # None: keep truthy items
print("map is lazy:", map(str.upper, names))
```

OUTPUT

```
[(1, 'asha'), (2, 'ravi'), (3, 'meera')]
[('asha', 91), ('ravi', 35), ('meera', 78)]
['Asha', 'Ravi', 'Meera']
[10, 20, 30]
[91, 78]
[1, 'a', [2]]
map is lazy: <map object at 0x7fe806293780>
```

8.6 Lists as stacks and queues

8.6.1 Stack with append / pop

A **stack** is last-in, first-out (LIFO), like a pile of plates: you add (**push**) and remove (**pop**) at the same end. A list is an efficient stack: `append()` pushes and `pop()` pops at the end, both O(1), and `stack[-1]` peeks at the top. Stacks are used for undo history, browser back buttons, matching brackets, evaluating expressions and depth-first search.

A LIST AS A STACK

main.py

```
history = []
for page in ["home", "courses", "python", "fees"]:
    history.append(page)                # push
    print("visit", page, "->", history)
print("back from", history.pop(), "| now on", history[-1])
print("back from", history.pop(), "| now on", history[-1])

def balanced(expr):
    pairs = {")": "(", "]": "[", "}": "{"}
    stack = []
    for ch in expr:
        if ch in "([{":
            stack.append(ch)
        elif ch in pairs:
            if not stack or stack.pop() != pairs[ch]:
                return False
    return not stack
for e in ["(a + b) * [c - {d}]", "(a + b))", "{[}]"]:
    print(e, "->", "balanced" if balanced(e) else "not balanced")
```

OUTPUT

```
visit home -> ['home']
visit courses -> ['home', 'courses']
visit python -> ['home', 'courses', 'python']
visit fees -> ['home', 'courses', 'python', 'fees']
back from fees | now on python
back from python | now on courses
(a + b) * [c - {d}] -> balanced
(a + b)) -> not balanced
{[}] -> not balanced
```

8.6.2 Queue with collections.deque

A **queue** is first-in, first-out (FIFO), like a line at a counter: add at the back, remove from the front. A list is a poor queue, because `pop(0)` shifts every remaining item (O(n)). Use `collections.deque` ("deck", double-ended queue), which adds and removes at **both** ends in O(1): `append()` and `popleft()` for a queue, plus `appendleft()`, `pop()` and `rotate()`. `deque(maxlen=n)` keeps only the last n items, perfect for "recent items" lists.

A DEQUE AS A QUEUE

main.py

```
from collections import deque

queue = deque()
for customer in ["Asha", "Ravi", "Meera"]:
    queue.append(customer)              # join at the back
print("queue:", queue)
print("served:", queue.popleft(), "| next:", queue[0], "| remaining:", list(queue))

recent = deque(maxlen=3)
for page in ["home", "courses", "python", "fees", "contact"]:
    recent.append(page)
print("last 3 pages:", list(recent))
d = deque([1, 2, 3, 4, 5])
d.rotate(2)
print("rotated:", list(d))

import time
big_list, big_deque = list(range(100_000)), deque(range(100_000))
t = time.perf_counter(); [big_list.pop(0) for _ in range(20_000)]; t_list = time.perf_counter() - t
t = time.perf_counter(); [big_deque.popleft() for _ in range(20_000)]; t_deque = time.perf_counter() -
t
print("deque.popleft faster than list.pop(0):", t_deque < t_list)
```

OUTPUT

```
queue: deque(['Asha', 'Ravi', 'Meera'])
served: Asha | next: Ravi | remaining: ['Ravi', 'Meera']
last 3 pages: ['python', 'fees', 'contact']
rotated: [4, 5, 1, 2, 3]
deque.popleft faster than list.pop(0): True
```

STACKS AND QUEUES

8.7 Tuples

8.7.1 Creating tuples and the single-element tuple (1,)

A **tuple** is an ordered, **immutable** sequence, written with commas, usually inside parentheses: `(21, "Asha", 8.7)`. In fact it is the **comma** that makes a tuple, not the parentheses: `t = 1, 2, 3` is a tuple. Therefore a single-element tuple needs a trailing comma, `(1,)`, because `(1)` is just the number 1 in parentheses. `()` is the empty tuple, and `tuple(iterable)` converts. Indexing, slicing, `len`, `in`, iteration, `+` and `*` work as for lists.

CREATING TUPLES main.py

```
point = (3, 4)
person = ("Asha", 21, "Mysuru")
no_parens = 1, 2, 3
single = (1,)
not_a_tuple = (1)
empty = ()
from_list = tuple([1, 2, 3])
print(point, person, no_parens, single, empty, from_list)
print(type(single).__name__, type(not_a_tuple).__name__)
print(person[0], person[-1], person[1:], len(person), "Asha" in person)
print(point + (5,), point * 2)
```

OUTPUT

```
(3, 4) ('Asha', 21, 'Mysuru') (1, 2, 3) (1,) () (1, 2, 3)
tuple int
Asha Mysuru (21, 'Mysuru') 3 True
(3, 4, 5) (3, 4, 3, 4)
```

8.7.2 Immutability and why tuples are useful

A tuple cannot be changed after creation: no item assignment, no `append` or `remove`. That is a feature. Tuples express **fixed records** whose parts belong together (a coordinate, an RGB colour, a date, a database row); they protect data that must not change; they are **hashable** (when their items are), so they can be dictionary keys and set members; and they are slightly smaller and faster than lists. Note that a tuple containing a mutable object (a list) cannot be re-bound item by item, but that inner list can still change.

TUPLES ARE IMMUTABLE (BUT MAY HOLD MUTABLE OBJECTS) main.py

```
rgb = (27, 42, 74)
try:
    rgb[0] = 255
except TypeError as e:
    print("TypeError:", e)
print("hashable, so usable as a key:", {rgb: "LBF navy"})

import sys
print("size of list vs tuple:", sys.getsizeof([1, 2, 3]), sys.getsizeof((1, 2, 3)), "bytes")

record = ("Asha", [91, 88])          # the tuple is fixed, the list inside is not
record[1].append(95)
print(record)
try:
    hash(record)
except TypeError as e:
    print("a tuple holding a list is not hashable:", e)
```

OUTPUT

```
TypeError: 'tuple' object does not support item assignment
hashable, so usable as a key: {(27, 42, 74): 'LBF navy'}
size of list vs tuple: 88 72 bytes
('Asha', [91, 88, 95])
a tuple holding a list is not hashable: unhashable type: 'list'
```

8.7.3 Packing and unpacking

Packing puts several values into a tuple: `t = "Asha", 21`. **Unpacking** assigns the items of a tuple (or any iterable) to separate names: `name, age = t`; the numbers must match. Unpacking makes functions that return several values natural (they return a tuple), lets loops unpack pairs (`for name, mark in pairs:`), and powers the swap `a, b = b, a`. Nested structures unpack with nested patterns.

PACKING AND UNPACKING main.py

```
student = "Asha", 21, "Mysuru"          # packing
name, age, city = student                # unpacking
print(name, age, city)

def min_max(values):
    return min(values), max(values)        # returns a tuple
low, high = min_max([7, 2, 9, 4])
print("low:", low, "| high:", high)

for name, (maths, science) in [("Asha", (91, 88)), ("Ravi", (78, 72))]:   # nested unpacking
    print(name, maths + science)
try:
    a, b = (1, 2, 3)
except ValueError as e:
    print("ValueError:", e)
```

OUTPUT

```
Asha 21 Mysuru
low: 2 | high: 9
Asha 179
Ravi 150
ValueError: too many values to unpack (expected 2, got 3)
```

8.7.4 Extended unpacking with *

A starred name in an unpacking target collects **all remaining** items into a **list**: `first, *rest = values`, `*init, last = values`, `head, *middle, tail = values`. Only one starred name is allowed per level, and it may receive an empty list. `_` is the conventional name for values you want to ignore (`*_` for several).

COLLECTING THE REST WITH * main.py

```
marks = [91, 78, 85, 64, 72]
first, *rest = marks
*init, last = marks
head, *middle, tail = marks
print(first, rest, "|", init, last, "|", head, middle, tail)

name, *_, city = ("Ravi", 22, "B.Tech", "Pune")
print(name, city)
a, *b = [1]
print("empty rest:", a, b)
date, *times = "2026-03-09 10:00 14:30 16:15".split()
print(date, times)
```

OUTPUT

```
91 [78, 85, 64, 72] | [91, 78, 85, 64] 72 | 91 [78, 85, 64] 72
Ravi Pune
empty rest: 1 []
2026-03-09 ['10:00', '14:30', '16:15']
```

8.7.5 count() and index()

Because tuples are immutable, they have only two methods: `count(x)` and `index(x)`, which work as for lists.

TUPLE METHODS main.py

```
results = ("pass", "fail", "pass", "pass", "absent")
print("passes:", results.count("pass"), "| first fail at:", results.index("fail"))
print("methods of tuple:", [m for m in dir(tuple) if not m.startswith("_")])
```

OUTPUT

```
passes: 3 | first fail at: 1
methods of tuple: ['count', 'index']
```

8.7.6 Tuples as dictionary keys

Dictionary keys must be **hashable** (immutable). Lists cannot be keys, but tuples of hashable items can, which is ideal for compound keys: coordinates on a grid, (city, date) pairs, (student, subject) pairs.

COMPOUND DICTIONARY KEYS main.py

```
distances = {("Mysuru", "Bengaluru"): 145, ("Bengaluru", "Chennai"): 350}
print(distances[("Mysuru", "Bengaluru")], "km")

board = {}
board[(0, 0)] = "X"
board[(1, 1)] = "O"
print(board, "| cell (2, 2):", board.get((2, 2), "empty"))
marks = {("Asha", "maths"): 91, ("Asha", "science"): 88, ("Ravi", "maths"): 78}
print("Asha's subjects:", [subj for (name, subj) in marks if name == "Asha"])
try:
    {[0, 0]: "X"}
except TypeError as e:
    print("list as key -> TypeError:", e)
```

OUTPUT

```
145 km
{(0, 0): 'X', (1, 1): 'O'} | cell (2, 2): empty
Asha's subjects: ['maths', 'science']
list as key -> TypeError: cannot use 'list' as a dict key (unhashable type: 'list')
```

8.7.7 Named tuples (collections.namedtuple)

A **named tuple** is a tuple whose fields can also be accessed by **name**: `p.x` instead of `p[0]`, which makes code self-documenting. `collections.namedtuple("Point", ["x", "y"])` creates the new type; instances are immutable, lightweight, print readably and still unpack and index like tuples. `_replace()` returns a modified copy and `_asdict()` converts to a dictionary. `typing.NamedTuple` offers the same with type hints and class syntax; dataclasses (Module 16) are the mutable, more flexible alternative.

TUPLES WITH NAMED FIELDS

main.py

```
from collections import namedtuple
from typing import NamedTuple

Point = namedtuple("Point", ["x", "y"])
p = Point(3, 4)
print(p, "| p.x:", p.x, "| p[1]:", p[1], "| distance:", (p.x ** 2 + p.y ** 2) ** 0.5)
x, y = p                                  # still unpacks
print(p._replace(x=10), p._asdict())

class Student(NamedTuple):                # the typed, class-based form
    name: str
    marks: int
    city: str = "Mysuru"

students = [Student("Asha", 91), Student("Ravi", 78, "Pune")]
for s in sorted(students, key=lambda s: s.marks, reverse=True):
    print(f"{s.name} ({s.city}): {s.marks}")
try:
    students[0].marks = 100
except AttributeError as e:
    print("immutable:", e)
```

OUTPUT

```
Point(x=3, y=4) | p.x: 3 | p[1]: 4 | distance: 5.0
Point(x=10, y=4) {'x': 3, 'y': 4}
Asha (Mysuru): 91
Ravi (Pune): 78
immutable: can't set attribute
```

8.7.8 List vs tuple comparison

	list	tuple
Syntax	`[1, 2, 3]`	`(1, 2, 3)` or `1, 2, 3`
Mutable	yes	no
Methods	many (append, remove, sort…)	count, index
Hashable (dict key, set member)	no	yes, if all items are hashable
Memory and speed	slightly larger, a bit slower	smaller, slightly faster
Typical use	a collection of similar items that changes: students, tasks	a fixed record of related values: (x, y), (name, age), RGB
Single element	`[1]`	`(1,)`

SIZE AND SPEED

main.py

```
import sys, timeit
print("list :", sys.getsizeof(list(range(100))), "bytes | tuple:", sys.getsizeof(tuple(range(100))),
"bytes")
t_list = timeit.timeit("[1, 2, 3, 4, 5]", number=1_000_000)
t_tuple = timeit.timeit("(1, 2, 3, 4, 5)", number=1_000_000)
print("creating a constant tuple is faster:", t_tuple < t_list)
```

OUTPUT

```
list : 856 bytes | tuple: 848 bytes
creating a constant tuple is faster: True
```

8.8 List problems

8.8.1 Second largest, remove duplicates preserving order, rotate, flatten

FOUR CLASSIC LIST PROBLEMS

main.py

```
def second_largest(nums):
    first = second = float("-inf")
    for n in nums:
        if n > first:
            first, second = n, first
        elif first > n > second:
            second = n
    return None if second == float("-inf") else second

def dedupe(items):
    seen, out = set(), []
    for x in items:
        if x not in seen:
            seen.add(x)
            out.append(x)
    return out

def rotate(lst, k):
    if not lst:
        return lst
    k %= len(lst)
    return lst[-k:] + lst[:-k]          # rotate right by k

def flatten(nested):
    out = []
    for item in nested:
        if isinstance(item, list):
            out.extend(flatten(item))    # recursion for any depth
        else:
            out.append(item)
    return out

print("second largest:", second_largest([12, 45, 2, 41, 45]), second_largest([5, 5]))
print("dedupe:", dedupe([3, 1, 3, 2, 1, 4]), "| dict.fromkeys:", list(dict.fromkeys([3, 1, 3, 2, 1,
4])))
print("rotate right 2:", rotate([1, 2, 3, 4, 5], 2), "| left 1:", rotate([1, 2, 3, 4, 5], -1))
print("flatten:", flatten([1, [2, [3, [4, 5]], 6], [], 7]))
```

OUTPUT

```
second largest: 41 None
dedupe: [3, 1, 2, 4] | dict.fromkeys: [3, 1, 2, 4]
rotate right 2: [4, 5, 1, 2, 3] | left 1: [2, 3, 4, 5, 1]
flatten: [1, 2, 3, 4, 5, 6, 7]
```

8.8.2 Linear and binary search, bubble and selection sort

Linear search checks each item in turn: O(n), works on unsorted data. **Binary search** works on **sorted** data: compare with the middle item and discard half the list each step, O(log n): about 20 steps for a million items. The `bisect` module implements it. **Bubble sort** repeatedly swaps adjacent items that are out of order, so large values "bubble" to the end; **selection sort** repeatedly selects the smallest remaining item and puts it next. Both are $O(n^2)$ and only for learning; real code uses `sort()`.

SEARCHING

main.py

```
import bisect

def linear_search(items, target):
    for i, item in enumerate(items):
        if item == target:
            return i
    return -1

def binary_search(sorted_items, target):
    lo, hi, steps = 0, len(sorted_items) - 1, 0
    while lo <= hi:
        steps += 1
        mid = (lo + hi) // 2
        if sorted_items[mid] == target:
            return mid, steps
        if sorted_items[mid] < target:
            lo = mid + 1
        else:
            hi = mid - 1
    return -1, steps

data = [4, 8, 15, 16, 23, 42, 57, 61, 79, 88]
print("linear search 42:", linear_search(data, 42), "| 5:", linear_search(data, 5))
print("binary search 61 (index, steps):", binary_search(data, 61))
million = list(range(0, 2_000_000, 2))
print("binary search in 1,000,000 items:", binary_search(million, 1_234_566))
print("bisect: insert position for 20:", bisect.bisect_left(data, 20))
```

OUTPUT

```
linear search 42: 5 | 5: -1
binary search 61 (index, steps): (7, 2)
binary search in 1,000,000 items: (617283, 20)
bisect: insert position for 20: 4
```

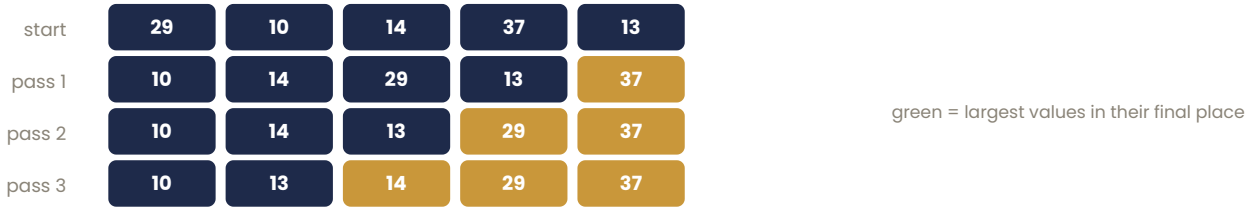
BUBBLE AND SELECTION SORT

main.py

```
def bubble_sort(a):
    a = a.copy()
    n = len(a)
    for i in range(n - 1):
        swapped = False
        for j in range(n - 1 - i):          # the last i items are already in place
            if a[j] > a[j + 1]:
                a[j], a[j + 1] = a[j + 1], a[j]
                swapped = True
        print(f"  pass {i + 1}: {a}")
        if not swapped:                      # already sorted: stop early
            break
    return a

def selection_sort(a):
    a = a.copy()
    for i in range(len(a)):
        smallest = min(range(i, len(a)), key=a.__getitem__)
        a[i], a[smallest] = a[smallest], a[i]
    return a

data = [29, 10, 14, 37, 13]
print("bubble sort:")
print("result:", bubble_sort(data))
print("selection sort:", selection_sort(data), "| built-in:", sorted(data))
```

OUTPUT

```
bubble sort:
  pass 1: [10, 14, 29, 13, 37]
  pass 2: [10, 14, 13, 29, 37]
  pass 3: [10, 13, 14, 29, 37]
  pass 4: [10, 13, 14, 29, 37]
result: [10, 13, 14, 29, 37]
selection sort: [10, 13, 14, 29, 37] | built-in: [10, 13, 14, 29, 37]
```



BUBBLE SORT: EACH PASS MOVES THE LARGEST REMAINING VALUE TO THE END

8.8.3 Matrix addition, multiplication and transpose

For matrices stored as lists of lists: **addition** adds corresponding items (same dimensions required); the **transpose** swaps rows and columns; **multiplication** of an m×n matrix A by an n×p matrix B gives an m×p matrix C, where C[i][j] is the sum over k of A[i][k] × B[k][j] (row of A times column of B). For real numerical work use NumPy (Module 24), which does this in compiled code with `A @ B`.

MATRIX OPERATIONS WITH NESTED LOOPS

main.py

```
A = [[1, 2, 3],
     [4, 5, 6]]
B = [[7, 8],
     [9, 10],
     [11, 12]]

def show(name, m):
    print(name)
    for row in m:
        print("  " + " ".join(f"{v:4}" for v in row))

add = [[A[i][j] + A[i][j] for j in range(len(A[0]))] for i in range(len(A))]
transpose = [[A[i][j] for i in range(len(A))] for j in range(len(A[0]))]
transpose_zip = [list(col) for col in zip(*A)]

product = [[0] * len(B[0]) for _ in range(len(A))]          # loop version
for i in range(len(A)):
    for j in range(len(B[0])):
        for k in range(len(B)):
            product[i][j] += A[i][k] * B[k][j]

show("A + A", add)
show("transpose of A", transpose)
print("zip(*A) gives the same:", transpose == transpose_zip)
show("A x B (2x3 times 3x2 = 2x2)", product)
```

OUTPUT

```
A + A
     2    4    6
     8   10   12
transpose of A
     1    4
     2    5
     3    6
zip(*A) gives the same: True
A x B (2x3 times 3x2 = 2x2)
    58   64
   139  154
```

Lists are ordered, mutable sequences created with [], list() or comprehensions; they support indexing, slicing, item and slice assignment, and nesting for matrices (m[i][j]). Methods: append, extend, insert add; remove, pop, clear and del delete; index, count and in search; sort (in place) and sorted (new list) order items, with key= and reverse=True and stable sorting; copy, [:] and list() make shallow copies, and copy.deepcopy copies nested data. Never build 2D lists with [[0]*n]*m. List comprehensions [expr for x in it if cond] build, filter and transform lists; if-else goes before for; several for clauses nest. len, min, max, sum, any and all aggregate; enumerate, zip, map and filter combine and transform. Lists make efficient stacks (append/pop); collections.deque makes efficient queues (append/popleft). Tuples are immutable sequences made by commas ((1,) for one item), used for fixed records, multiple return values, unpacking (with * for the rest) and dictionary keys; namedtuple adds field names. Classic problems: second largest, dedupe, rotate, flatten, linear and binary search, bubble and selection sort, matrix addition, transpose and multiplication.

PRACTICE CHECKPOINT

Build a **to-do list manager** (add, remove, list, sort) using a list, and write **matrix multiplication** once with loops and once with a comprehension.

PART 1: A TO-DO LIST MANAGER

main.py

```
todos = []

def show():
    if not todos:
        print("  (no tasks)")
    for i, (task, priority) in enumerate(todos, start=1):
        print(f"  {i}. [{priority}] {task}")

while True:
    command = input("todo> ").strip()
    if command == "quit":
        break
    action, _, rest = command.partition(" ")
    match action:
        case "add":
            task, _, priority = rest.rpartition(" ")
            todos.append((task, int(priority)))
            print(f"  added '{task}' with priority {priority}")
        case "remove":
            removed = todos.pop(int(rest) - 1)
            print(f"  removed '{removed[0]}'")
        case "list":
            show()
        case "sort":
            todos.sort(key=lambda t: (t[1], t[0].lower()))
            print("  sorted by priority, then name")
            show()
        case _:
            print("  commands: add TASK PRIORITY | remove N | list | sort | quit")
```

CONSOLE (user input in gold)

```
todo> add Revise lists 2
  added 'Revise lists' with priority 2
todo> add Submit assignment 1
  added 'Submit assignment' with priority 1
todo> add Buy notebook 3
  added 'Buy notebook' with priority 3
todo> list
  1. [2] Revise lists
  2. [1] Submit assignment
  3. [3] Buy notebook
todo> sort
  sorted by priority, then name
  1. [1] Submit assignment
  2. [2] Revise lists
  3. [3] Buy notebook
todo> remove 3
  removed 'Buy notebook'
todo> list
  1. [1] Submit assignment
  2. [2] Revise lists
todo> help
  commands: add TASK PRIORITY | remove N | list | sort | quit
todo> quit
```

PART 2: MATRIX MULTIPLICATION TWO WAYS

main.py

```
A = [[1, 2], [3, 4], [5, 6]]          # 3 x 2
B = [[7, 8, 9], [10, 11, 12]]         # 2 x 3

def matmul_loops(A, B):
    rows, inner, cols = len(A), len(B), len(B[0])
    if len(A[0]) != inner:
        raise ValueError("columns of A must equal rows of B")
    C = [[0] * cols for _ in range(rows)]
    for i in range(rows):
        for j in range(cols):
            for k in range(inner):
                C[i][j] += A[i][k] * B[k][j]
    return C

def matmul_comprehension(A, B):
    return [[sum(a * b for a, b in zip(row, col)) for col in zip(*B)] for row in A]

C1 = matmul_loops(A, B)
C2 = matmul_comprehension(A, B)
for row in C1:
    print(" ".join(f"{v:4}" for v in row))
print("both versions agree:", C1 == C2)
```

OUTPUT

```
  27   30   33
  61   68   75
  95  106  117
both versions agree: True
```

MODULE 09 · BEGINNER

Sets and dictionaries

Minimum duration **2 days** | Level **Beginner**

Sets store unique, unordered items with very fast membership tests; dictionaries map keys to values and are the most important data structure in Python. This module covers creating and modifying sets, set algebra, frozensets, creating and using dictionaries, merging and comprehensions, nested dictionaries and sorting, the specialised dictionaries in collections (defaultdict, Counter, OrderedDict, ChainMap), how hashing makes lookups fast, how to choose the right collection, and classic dictionary problems.

IN THIS MODULE

9.1 Sets
9.2 Dictionaries
9.3 Specialised dictionaries (collections)
9.4 How dictionaries and sets work
9.5 Choosing a collection
9.6 Problems

9.1 Sets

9.1.1 Creating sets and the empty set()

A **set** is an unordered collection of **unique** items, written in braces: `{"python", "java", "sql"}`. `set(iterable)` builds a set from any iterable, automatically dropping duplicates, which makes `set()` the quickest way to find distinct values. The empty set must be written `set()`: `{}` is an empty **dictionary**. Sets have no order and no indexing; print order may differ from insertion order.

CREATING SETS — main.py

```
skills = {"python", "sql", "git", "python"}       # duplicate dropped
print(skills, len(skills))
print(set([3, 1, 3, 2, 1]), set("mississippi"), set(range(5)))
empty = set()
not_a_set = {}
print(type(empty).__name__, type(not_a_set).__name__)
try:
    skills[0]
except TypeError as e:
    print("no indexing:", e)
emails = ["a@x.in", "b@x.in", "a@x.in", "c@x.in", "b@x.in"]
print("distinct emails:", len(set(emails)), "of", len(emails))
```

OUTPUT

```
{'git', 'python', 'sql'} 3
{1, 2, 3} {'s', 'i', 'p', 'm'} {0, 1, 2, 3, 4}
set dict
no indexing: 'set' object is not subscriptable
distinct emails: 3 of 5
```

9.1.2 Uniqueness and hashable elements

A set decides uniqueness by equality (`==`) and stores items by their **hash** (9.4), so every element must be **hashable**: immutable built-in types such as numbers, strings, tuples of hashable items and frozensets. Lists, dictionaries and sets cannot be set elements. Note that `1`, `1.0` and `True` are equal and have the same hash, so a set keeps only one of them.

WHAT CAN GO INTO A SET main.py

```
print({1, 1.0, True}, {0, False, 0.0})
print({(1, 2), (1, 2), (2, 1)})              # tuples are hashable
print({frozenset({1, 2}), frozenset({2, 1})})
try:
    {[1, 2], [3]}
except TypeError as e:
    print("TypeError:", e)
```

OUTPUT

```
{1} {0}
{(1, 2), (2, 1)}
{frozenset({1, 2})}
TypeError: cannot use 'list' as a set element (unhashable type: 'list')
```

9.1.3 Adding and removing: add(), update(), remove(), discard(), pop(), clear()

Sets are mutable. `add(x)` adds one item (no effect if already present); `update(iterable)` adds many. `remove(x)` deletes an item and raises `KeyError` if it is missing, while `discard(x)` silently does nothing; `pop()` removes and returns an **arbitrary** item; `clear()` empties the set.

CHANGING A SET main.py

```
tags = {"python"}
tags.add("data")
tags.add("python")                      # already present: no change
tags.update(["ai", "ml"], {"stats"})
print(sorted(tags))
tags.remove("ml")
tags.discard("java")                    # missing: no error
try:
    tags.remove("java")
except KeyError as e:
    print("remove missing -> KeyError:", e)
item = tags.pop()
print("popped one item; remaining", len(tags))
tags.clear()
print("after clear:", tags)
```

OUTPUT

```
['ai', 'data', 'ml', 'python', 'stats']
remove missing -> KeyError: 'java'
popped one item; remaining 3
after clear: set()
```

9.1.4 Set operations: union |, intersection &, difference -, symmetric difference ^

Sets support the operations of mathematical set theory, as operators (both operands must be sets) or as methods (which accept any iterable). **Union** `a | b`: items in either. **Intersection** `a & b`: items in both. **Difference** `a - b`: items in a but not in b. **Symmetric difference** `a ^ b`: items in exactly one. In-place

forms (`|=`, `&=`, `-=`, `^=`, `update`, `intersection_update`…) modify the left set. These operations answer questions such as "which students take both courses?" in one line.

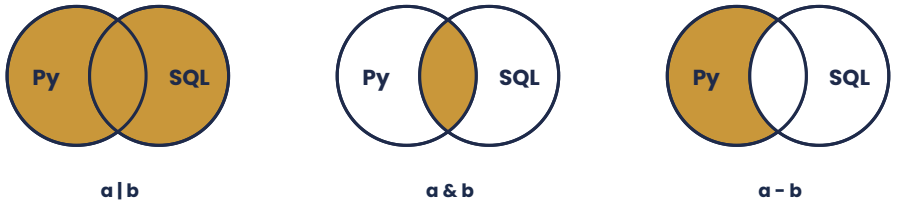
SET ALGEBRA

main.py

```
python_batch = {"Asha", "Ravi", "Meera", "John"}
sql_batch = {"Ravi", "Zoya", "John", "Kiran"}

print("either (union):       ", sorted(python_batch | sql_batch))
print("both (intersection):  ", sorted(python_batch & sql_batch))
print("only Python (diff):   ", sorted(python_batch - sql_batch))
print("only SQL (diff):      ", sorted(sql_batch - python_batch))
print("exactly one (sym diff):", sorted(python_batch ^ sql_batch))
print("method forms accept lists:", sorted(python_batch.union(["Neha"])),
sorted(python_batch.intersection(["Asha", "X"])))
```

OUTPUT

```
either (union):        ['Asha', 'John', 'Kiran', 'Meera', 'Ravi', 'Zoya']
both (intersection):   ['John', 'Ravi']
only Python (diff):    ['Asha', 'Meera']
only SQL (diff):       ['Kiran', 'Zoya']
exactly one (sym diff): ['Asha', 'Kiran', 'Meera', 'Zoya']
method forms accept lists: ['Asha', 'John', 'Meera', 'Neha', 'Ravi'] ['Asha']
```



UNION, INTERSECTION AND DIFFERENCE

Operation	Operator	Method	Result for {1,2,3} and {2,3,4}
union	`a | b`	`a.union(b)`	{1, 2, 3, 4}
intersection	`a & b`	`a.intersection(b)`	{2, 3}
difference	`a - b`	`a.difference(b)`	{1}
symmetric difference	`a ^ b`	`a.symmetric_difference(b)`	{1, 4}
subset	`a <= b`	`a.issubset(b)`	False
superset	`a >= b`	`a.issuperset(b)`	False
disjoint	-	`a.isdisjoint(b)`	False

9.1.5 Subset, superset and disjoint checks

`a <= b` (or `a.issubset(b)`) is True if every item of a is in b; `a < b` is a **proper** subset (a subset but not equal). `a >= b` and `a > b` test supersets. `a.isdisjoint(b)` is True if they share no items. Two sets are equal (`==`) if they contain the same items, regardless of order.

SUBSET AND SUPERSET TESTS main.py

```
required = {"python", "sql"}
asha = {"python", "sql", "git", "excel"}
ravi = {"python", "java"}
print("Asha has all required skills:", required <= asha, "| Ravi:", required.issubset(ravi))
print("Asha's skills are a superset:", asha >= required, "| proper superset:", asha > required)
print("Ravi and {'sql', 'excel'} disjoint:", ravi.isdisjoint({"sql", "excel"}))
print("{1, 2, 3} == {3, 2, 1}:", {1, 2, 3} == {3, 2, 1})
print("missing for Ravi:", required - ravi)
```

OUTPUT

```
Asha has all required skills: True | Ravi: False
Asha's skills are a superset: True | proper superset: True
Ravi and {'sql', 'excel'} disjoint: True
{1, 2, 3} == {3, 2, 1}: True
missing for Ravi: {'sql'}
```

9.1.6 Set comprehensions

A **set comprehension** `{expr for x in iterable if cond}` works like a list comprehension but produces a set, so duplicates disappear automatically.

SET COMPREHENSIONS main.py

```
words = "the cat and the hat and the bat".split()
print({w for w in words})
print({len(w) for w in words})
print({w[0].upper() for w in words if len(w) == 3})
emails = ["Asha@LBF.in", "ravi@gmail.com", "meera@lbf.in", "john@GMAIL.com"]
print("domains:", {e.split("@")[1].lower() for e in emails})
```

OUTPUT

```
{'hat', 'bat', 'the', 'cat', 'and'}
{3}
{'T', 'A', 'B', 'C', 'H'}
domains: {'lbf.in', 'gmail.com'}
```

9.1.7 frozenset

A **frozenset** is an immutable set: same operations except those that modify it. Because it is hashable, it can be an element of another set or a dictionary key, for example to represent a group of items whose order does not matter (a pair of friends, a combination of toppings).

IMMUTABLE SETS AS KEYS main.py

```
pair1 = frozenset({"Asha", "Ravi"})
pair2 = frozenset({"Ravi", "Asha"})
print(pair1 == pair2, hash(pair1) == hash(pair2))
friendships = {pair1, frozenset({"Meera", "John"})}
print(frozenset({"Ravi", "Asha"}) in friendships)
combo_price = {frozenset({"burger", "fries"}): 199, frozenset({"pizza", "coke"}): 249}
print(combo_price[frozenset({"fries", "burger"})])
try:
    pair1.add("Zoya")
except AttributeError as e:
    print("AttributeError:", e)
```

OUTPUT

```
True True
True
199
AttributeError: 'frozenset' object has no attribute 'add'
```

9.1.8 Fast membership testing

Checking `x in some_set` takes about the same time whether the set has ten items or ten million (average O(1)), because the set jumps straight to the right place using the hash. The same check on a list scans item by item (O(n)). Whenever you test membership many times, such as checking words against a dictionary of valid words, or IDs against a list of banned users, convert the collection to a set first.

SET MEMBERSHIP VERSUS LIST MEMBERSHIP main.py

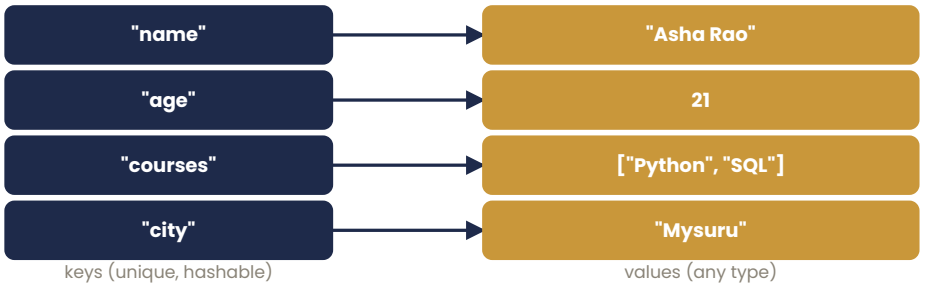
```
import time

banned_list = [f"user{i}" for i in range(200_000)]
banned_set = set(banned_list)
queries = [f"user{i}" for i in range(199_000, 201_000)]

t = time.perf_counter()
hits_list = sum(q in banned_list for q in queries)
t_list = time.perf_counter() - t
t = time.perf_counter()
hits_set = sum(q in banned_set for q in queries)
t_set = time.perf_counter() - t
print("same answers:", hits_list == hits_set, "| matches:", hits_set)
print(f"set was faster: {t_set < t_list} (about {t_list / t_set:,.0f} times on this machine)")
```

OUTPUT

```
same answers: True | matches: 1000
set was faster: True (about 10,319 times on this machine)
```

9.2 Dictionaries

A **dictionary** (`dict`) stores **key-value pairs**: each unique key maps to a value, and you look values up by key rather than by position, like a word in a real dictionary or a contact in a phone book. Dictionaries are everywhere in Python: JSON data, configuration, counting, caching, database rows, function keyword arguments, and even the namespaces that hold your variables are dictionaries.



A DICTIONARY MAPS KEYS TO VALUES

9.2.1 Creating dictionaries (literal, dict(), zip, fromkeys)

Write a literal with braces and `key: value` pairs. `dict()` also accepts keyword arguments (when keys are valid identifiers), a list of pairs, or `zip(keys, values)`. `dict.fromkeys(keys, value)` creates a dictionary with all keys mapped to the same value (careful: a mutable default value would be shared). `{}` is an empty dictionary. Comprehensions (9.2.7) are another way.

CREATING DICTIONARIES main.py

```
student = {"name": "Asha", "age": 21, "courses": ["Python", "SQL"]}
from_kwargs = dict(name="Ravi", age=22)
from_pairs = dict([("maths", 91), ("science", 88)])
from_zip = dict(zip(["Mon", "Tue", "Wed"], [3, 5, 2]))
attendance = dict.fromkeys(["Asha", "Ravi", "Meera"], 0)
empty = {}
print(student, from_kwargs, from_pairs, from_zip, attendance, empty, sep="\n")
print("length:", len(student))
```

OUTPUT

```
{'name': 'Asha', 'age': 21, 'courses': ['Python', 'SQL']}
{'name': 'Ravi', 'age': 22}
{'maths': 91, 'science': 88}
{'Mon': 3, 'Tue': 5, 'Wed': 2}
{'Asha': 0, 'Ravi': 0, 'Meera': 0}
{}
length: 3
```

9.2.2 Keys must be hashable; insertion order is preserved (3.7+)

Keys must be **hashable** (strings, numbers, tuples, frozensets, None); values can be anything. Keys are unique: assigning to an existing key replaces its value, and a literal with a repeated key keeps the last value. Since Python 3.7 dictionaries remember **insertion order**: iteration and printing follow the order in which keys were first added (updating a value keeps the position).

HASHABLE, UNIQUE KEYS IN INSERTION ORDER main.py

```
d = {"b": 1, "a": 2, "c": 3, "a": 99}        # repeated key: last value wins
print(d)
d["z"] = 0                                    # new key goes to the end
d["b"] = 100                                  # existing key keeps its position
print(d, list(d))
mixed_keys = {1: "int", "1": "str", (1, 2): "tuple", None: "none", 2.5: "float"}
print(mixed_keys)
try:
    {["list"]: 1}
except TypeError as e:
    print("TypeError:", e)
```

OUTPUT

```
{'b': 1, 'a': 99, 'c': 3}
{'b': 100, 'a': 99, 'c': 3, 'z': 0} ['b', 'a', 'c', 'z']
{1: 'int', '1': 'str', (1, 2): 'tuple', None: 'none', 2.5: 'float'}
TypeError: cannot use 'list' as a dict key (unhashable type: 'list')
```

9.2.3 Accessing values with [] and get()

`d[key]` returns the value for a key and raises `KeyError` if the key is missing. `d.get(key)` returns `None` instead, and `d.get(key, default)` returns a default of your choice: the safe way when keys may be absent. `key in d` tests whether a key exists (it checks keys, not values).

READING VALUES SAFELY main.py

```
marks = {"Asha": 91, "Ravi": 78}
print(marks["Asha"])
print(marks.get("Meera"), marks.get("Meera", 0), marks.get("Ravi", 0))
print("'Asha' in marks:", "Asha" in marks, "| 91 in marks:", 91 in marks)
try:
    marks["Meera"]
except KeyError as e:
    print("KeyError:", e)
config = {"theme": "dark"}
font_size = config.get("font_size", 14)
print("font size:", font_size)
```

OUTPUT

```
91
None 0 78
'Asha' in marks: True | 91 in marks: False
KeyError: 'Meera'
font size: 14
```

9.2.4 Adding, updating and deleting: update(), pop(), popitem(), del, setdefault()

Assign to a key to add or replace: `d[key] = value`. `update(other)` merges another dictionary (or pairs or keyword arguments), overwriting existing keys. `pop(key[, default])` removes a key and returns its value (KeyError without a default if missing); `popitem()` removes and returns the **last inserted** pair; `del d[key]` deletes; `clear()` empties. `setdefault(key, default)` returns the value if the key exists, otherwise inserts the default and returns it, a handy way to build lists of values per key.

MODIFYING DICTIONARIES

main.py

```
stock = {"pen": 120, "book": 45}
stock["bag"] = 12                       # add
stock["pen"] = 100                      # update
stock.update({"book": 50, "ruler": 30}, eraser=75)
print(stock)
removed = stock.pop("ruler")
missing = stock.pop("glue", "not stocked")
print("popped:", removed, missing)
print("popitem:", stock.popitem(), "->", stock)
del stock["bag"]
print("after del:", stock)

groups = {}
for name, city in [("Asha", "Mysuru"), ("Ravi", "Pune"), ("Meera", "Mysuru")]:
    groups.setdefault(city, []).append(name)
print("grouped with setdefault:", groups)
```

OUTPUT

```
{'pen': 100, 'book': 50, 'bag': 12, 'ruler': 30, 'eraser': 75}
popped: 30 not stocked
popitem: ('eraser', 75) -> {'pen': 100, 'book': 50, 'bag': 12}
after del: {'pen': 100, 'book': 50}
grouped with setdefault: {'Mysuru': ['Asha', 'Meera'], 'Pune': ['Ravi']}
```

9.2.5 Iterating with keys(), values(), items()

Looping over a dictionary gives its **keys**. `d.keys()`, `d.values()` and `d.items()` return **views**: live, iterable windows onto the dictionary that reflect later changes. `items()` yields `(key, value)` pairs, unpacked directly in the loop: the most common pattern. Key views support set operations (`d1.keys() & d2.keys()`). Do not add or remove keys while iterating over a dictionary (RuntimeError); iterate over `list(d)` if you must.

LOOPING OVER DICTIONARIES main.py

```
prices = {"pen": 10, "book": 250, "bag": 899}
for item in prices:
    print(item, end=" ")
print()
for item, price in prices.items():
    print(f"{item:<5} Rs {price}")
print("keys:", list(prices.keys()), "| values:", list(prices.values()), "| total:",
sum(prices.values()))
view = prices.keys()
prices["ruler"] = 15
print("the view updated itself:", view)
other = {"book": 240, "pencil": 5}
print("common items:", prices.keys() & other.keys())
try:
    for k in prices:
        if prices[k] < 20:
            del prices[k]
except RuntimeError as e:
    print("RuntimeError:", e)
for k in list(prices):
    if prices[k] < 20:
        del prices[k]
print(prices)
```

OUTPUT

```
pen book bag
pen   Rs 10
book  Rs 250
bag   Rs 899
keys: ['pen', 'book', 'bag'] | values: [10, 250, 899] | total: 1159
the view updated itself: dict_keys(['pen', 'book', 'bag', 'ruler'])
common items: {'book'}
RuntimeError: dictionary changed size during iteration
{'book': 250, 'bag': 899}
```

9.2.6 Merging with | and |= (3.9)

Since Python 3.9, `d1 | d2` returns a **new** merged dictionary, with values from `d2` winning for duplicate keys, and `d1 |= d2` updates `d1` in place (like `update`). Before 3.9 the idiom was `{**d1, **d2}`, which still works and also lets you add extra keys inline.

MERGING DICTIONARIES main.py

```
defaults = {"theme": "light", "font": 14, "lang": "en"}
user = {"font": 16, "lang": "kn"}
settings = defaults | user                    # user values win
print(settings)
print({**defaults, **user, "beta": True})
defaults |= {"theme": "dark"}
print(defaults)
```

OUTPUT

```
{'theme': 'light', 'font': 16, 'lang': 'kn'}
{'theme': 'light', 'font': 16, 'lang': 'kn', 'beta': True}
{'theme': 'dark', 'font': 14, 'lang': 'en'}
```

9.2.7 Dictionary comprehensions

A **dictionary comprehension** `{key_expr: value_expr for x in iterable if cond}` builds a dictionary in one expression: transforming values, filtering items, inverting a dictionary, or building lookups from lists.

BUILDING DICTIONARIES WITH COMPREHENSIONS main.py

```
names = ["Asha", "Ravi", "Meera"]
print({n: len(n) for n in names})
print({n: n.upper() for n in names if len(n) == 4})
marks = {"Asha": 91, "Ravi": 35, "Meera": 78}
print({n: ("pass" if m >= 40 else "fail") for n, m in marks.items()})
print("inverted:", {m: n for n, m in marks.items()})
print("squares:", {x: x * x for x in range(1, 6)})
prices_usd = {"pen": 1.2, "book": 8}
print("in rupees:", {k: round(v * 83.5) for k, v in prices_usd.items()})
```

OUTPUT

```
{'Asha': 4, 'Ravi': 4, 'Meera': 5}
{'Asha': 'ASHA', 'Ravi': 'RAVI'}
{'Asha': 'pass', 'Ravi': 'fail', 'Meera': 'pass'}
inverted: {91: 'Asha', 35: 'Ravi', 78: 'Meera'}
squares: {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
in rupees: {'pen': 100, 'book': 668}
```

9.2.8 Nested dictionaries

Values can themselves be dictionaries or lists, giving nested structures that model real records, exactly like JSON. Access inner values by chaining keys (`data["Asha"]["marks"]["maths"]`), and use `get()` with an empty dictionary as default to navigate safely when levels may be missing.

RECORDS AS NESTED DICTIONARIES

main.py

```
students = {
    "Asha": {"age": 21, "city": "Mysuru", "marks": {"maths": 91, "science": 88}},
    "Ravi": {"age": 22, "city": "Pune", "marks": {"maths": 78, "science": 72}},
}
print(students["Asha"]["marks"]["maths"])
students["Ravi"]["marks"]["english"] = 81           # add deep inside
students["Meera"] = {"age": 20, "city": "Mysuru", "marks": {}}

for name, info in students.items():
    avg = sum(info["marks"].values()) / len(info["marks"]) if info["marks"] else 0
    print(f"{name:<6} {info['city']:<7} average {avg:.1f}")
print("safe deep access:", students.get("John", {}).get("marks", {}).get("maths", "n/a"))

import json
print(json.dumps(students["Asha"], indent=2))
```

OUTPUT

```
91
Asha   Mysuru  average 89.5
Ravi   Pune    average 77.0
Meera  Mysuru  average 0.0
safe deep access: n/a
{
  "age": 21,
  "city": "Mysuru",
  "marks": {
    "maths": 91,
    "science": 88
  }
}
```

9.2.9 Sorting a dictionary by key or value

A dictionary itself is not sorted, but `sorted()` on its items produces a sorted list of pairs, which can be turned back into a dictionary (which keeps that order). Sort by key with `sorted(d.items())`, by value with `key=lambda kv: kv[1]` (or `operator.itemgetter(1)`), and use `reverse=True` for descending order. `sorted(d, key=d.get)` lists keys ordered by their values.

ORDERING DICTIONARY ITEMS

main.py

```
marks = {"Ravi": 78, "Asha": 91, "Meera": 85, "John": 64}
print("by key:", dict(sorted(marks.items())))
print("by value:", dict(sorted(marks.items(), key=lambda kv: kv[1])))
print("by value, descending:", dict(sorted(marks.items(), key=lambda kv: kv[1], reverse=True)))
print("top 2 names:", sorted(marks, key=marks.get, reverse=True)[:2])
for rank, (name, m) in enumerate(sorted(marks.items(), key=lambda kv: -kv[1]), 1):
    print(f"{rank}. {name} {m}")
```

OUTPUT

```
by key: {'Asha': 91, 'John': 64, 'Meera': 85, 'Ravi': 78}
by value: {'John': 64, 'Ravi': 78, 'Meera': 85, 'Asha': 91}
by value, descending: {'Asha': 91, 'Meera': 85, 'Ravi': 78, 'John': 64}
top 2 names: ['Asha', 'Meera']
1. Asha 91
2. Meera 85
3. Ravi 78
4. John 64
```

Operation	Syntax
create	`{"k": v}`, `dict(k=v)`, `dict(zip(ks, vs))`, `dict.fromkeys(ks, v)`
read	`d[k]` (KeyError), `d.get(k, default)`
test key	`k in d`
add / update	`d[k] = v`, `d.update(other)`, `d |= other`
delete	`del d[k]`, `d.pop(k, default)`, `d.popitem()`, `d.clear()`
default insert	`d.setdefault(k, default)`
views	`d.keys()`, `d.values()`, `d.items()`
merge	`d1 | d2`, `{**d1, **d2}`
copy	`d.copy()` (shallow), `copy.deepcopy(d)`

9.3 Specialised dictionaries (collections)

9.3.1 defaultdict

`collections.defaultdict(factory)` is a dictionary that automatically creates a value for a missing key by calling the factory: `defaultdict(list)` starts each new key with an empty list, `defaultdict(int)` with 0, `defaultdict(set)` with an empty set. It removes the "if key not in d: d[key] = []" boilerplate from grouping and counting code. Only reading with `d[key]` triggers the factory; `get()` and `in` do not.

GROUPING AND COUNTING WITH DEFAULTDICT

main.py

```
from collections import defaultdict

orders = [("Mysuru", "pen"), ("Pune", "book"), ("Mysuru", "bag"), ("Delhi", "pen"), ("Pune", "pen")]
by_city = defaultdict(list)
for city, item in orders:
    by_city[city].append(item)          # no need to check whether the key exists
print(dict(by_city))

counts = defaultdict(int)
for _, item in orders:
    counts[item] += 1
print(dict(counts))

graph = defaultdict(set)
for a, b in [("A", "B"), ("A", "C"), ("B", "C")]:
    graph[a].add(b); graph[b].add(a)
print({k: sorted(v) for k, v in graph.items()})
print("get() does not create keys:", counts.get("ruler"), "| 'ruler' in counts:", "ruler" in counts)
```

OUTPUT

```
{'Mysuru': ['pen', 'bag'], 'Pune': ['book', 'pen'], 'Delhi': ['pen']}
{'pen': 3, 'book': 1, 'bag': 1}
{'A': ['B', 'C'], 'B': ['A', 'C'], 'C': ['A', 'B']}
get() does not create keys: None | 'ruler' in counts: False
```

9.3.2 Counter

`collections.Counter` counts hashable items: `Counter(iterable)` builds a dictionary from items to counts. `most_common(n)` returns the n most frequent items, missing items count as 0 (no KeyError), `update()` adds more counts, `total()` sums them, and counters support arithmetic: `+`, `-`, `&` (minimum) and `|` (maximum). It is the standard tool for frequencies: words, votes, characters, log entries.

COUNTING WITH COUNTER

main.py

```
from collections import Counter

votes = ["Python", "Java", "Python", "SQL", "Python", "Java"]
c = Counter(votes)
print(c)
print("most common:", c.most_common(2), "| C++ votes:", c["C++"], "| total:", c.total())
c.update(["SQL", "SQL", "Go"])
print("after update:", c)
print("letters:", Counter("mississippi").most_common(3))
morning, evening = Counter(apples=3, mangoes=5), Counter(apples=2, bananas=4)
print("sum:", morning + evening, "| difference:", morning - evening)
```

OUTPUT

```
Counter({'Python': 3, 'Java': 2, 'SQL': 1})
most common: [('Python', 3), ('Java', 2)] | C++ votes: 0 | total: 6
after update: Counter({'Python': 3, 'SQL': 3, 'Java': 2, 'Go': 1})
letters: [('i', 4), ('s', 4), ('p', 2)]
sum: Counter({'apples': 5, 'mangoes': 5, 'bananas': 4}) | difference: Counter({'mangoes': 5, 'apples':
1})
```

9.3.3 OrderedDict

`collections.OrderedDict` was the way to keep insertion order before regular dictionaries did (3.7). It is still useful for its extra features: `move_to_end(key, last=True)` and `popitem(last=False)` to remove the **oldest** item, which makes it a natural building block for an LRU (least recently used) cache; and its equality comparison considers order.

AN LRU CACHE WITH ORDEREDDICT

main.py

```
from collections import OrderedDict

class LRUCache:
    def __init__(self, capacity):
        self.capacity, self.data = capacity, OrderedDict()
    def get(self, key):
        if key not in self.data:
            return None
        self.data.move_to_end(key)             # mark as recently used
        return self.data[key]
    def put(self, key, value):
        self.data[key] = value
        self.data.move_to_end(key)
        if len(self.data) > self.capacity:
            oldest = self.data.popitem(last=False)
            print("  evicted", oldest)

cache = LRUCache(2)
cache.put("a", 1); cache.put("b", 2)
cache.get("a")                                  # a is now the most recent
cache.put("c", 3)                               # evicts b
print(list(cache.data.items()))
print(OrderedDict(a=1, b=2) == OrderedDict(b=2, a=1), "|", dict(a=1, b=2) == dict(b=2, a=1))
```

OUTPUT

```
  evicted ('b', 2)
[('a', 1), ('c', 3)]
False | True
```

9.3.4 ChainMap (overview)

`collections.ChainMap(d1, d2, ...)` groups several dictionaries into one view that searches them in order, without copying them. Lookups return the first match; writes go to the first dictionary. The classic use is layered configuration: command-line options, then environment settings, then defaults.

LAYERED SETTINGS WITH CHAINMAP

main.py

```
from collections import ChainMap

defaults = {"theme": "light", "font": 14, "lang": "en"}
env = {"lang": "hi"}
cli = {"theme": "dark"}
config = ChainMap(cli, env, defaults)
print(config["theme"], config["lang"], config["font"])
config["font"] = 18                          # written to the first map (cli)
print(cli, "| defaults untouched:", defaults)
print(dict(config))
```

OUTPUT

```
dark hi 14
{'theme': 'dark', 'font': 18} | defaults untouched: {'theme': 'light', 'font': 14, 'lang': 'en'}
{'theme': 'dark', 'font': 18, 'lang': 'hi'}
```

9.4 How dictionaries and sets work

9.4.1 Hashing and hash()

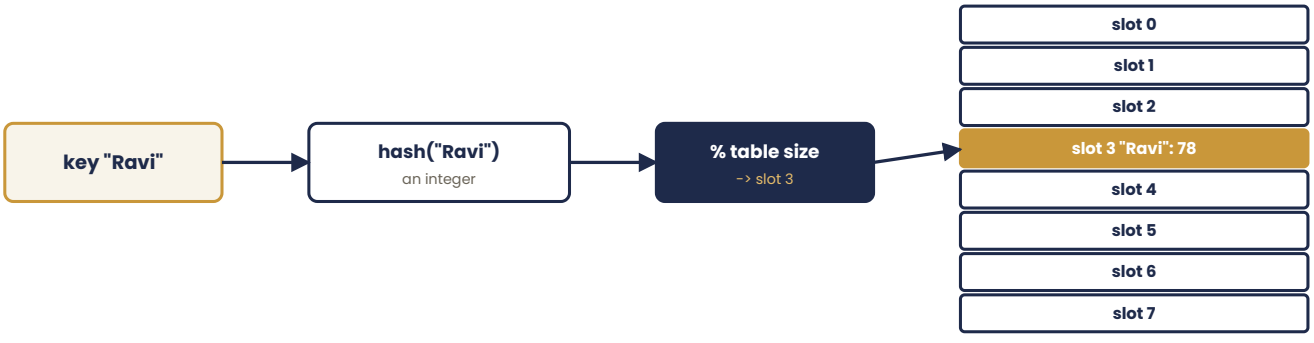
Dictionaries and sets are **hash tables**. When you store a key, Python calls `hash(key)`, which turns the object into an integer; that number decides which **slot** of an internal array the entry goes into. To look a key up, Python computes the same hash, goes straight to that slot, and confirms with `==`. Two rules make this work: equal objects must have equal hashes, and a key's hash must never change, which is why keys must be immutable. Different keys can occasionally land on the same slot (a **collision**); Python then probes other slots. String hashes are randomised per process for security (so they differ between runs), while small integers hash to themselves.

HASHES DECIDE WHERE ENTRIES LIVE main.py

```
print("hash(42):", hash(42), "| hash(42.0):", hash(42.0), "| hash(True):", hash(True))
print("hash of a tuple is an int:", isinstance(hash((1, 2)), int))
print("equal objects, equal hashes:", hash("python") == hash("py" + "thon"))
try:
    hash([1, 2])
except TypeError as e:
    print("TypeError:", e)

slots = 8
for key in ["Asha", "Ravi", "Meera", 42, (1, 2)]:
    print(f"{key!r:<8} -> slot {hash(key) % slots} of {slots} (this run)")
```

OUTPUT

```
hash(42): 42 | hash(42.0): 42 | hash(True): 1
hash of a tuple is an int: True
equal objects, equal hashes: True
TypeError: unhashable type: 'list'
'Asha'   -> slot 2 of 8 (this run)
'Ravi'   -> slot 2 of 8 (this run)
'Meera'  -> slot 0 of 8 (this run)
42       -> slot 2 of 8 (this run)
(1, 2)   -> slot 3 of 8 (this run)
```



A KEY GOES DIRECTLY TO ITS SLOT: NO SCANNING

9.4.2 Average O(1) lookup

Because the slot is computed rather than searched for, looking up, inserting and deleting a key take **constant time on average**, O(1), no matter how many entries there are. Python keeps the table less than about two-thirds full, resizing it as it grows, so collisions stay rare. The cost is extra memory. Lists, by contrast, need O(n) to find a value. This is why converting lookups to dictionaries or sets is one of the most effective optimisations in Python.

LOOKUP TIME DOES NOT GROW WITH SIZE

main.py

```
import timeit

times = {}
for n in (1_000, 1_000_000):
    d = {i: i for i in range(n)}
    times[n] = min(timeit.repeat(lambda: (n - 1) in d, number=100_000, repeat=5))
    print(f"dict of {n:>9,} keys: built and searched 100,000 times")

ratio = times[1_000_000] / times[1_000]
print("1000x more keys, but lookups less than 3x slower:", ratio < 3)
print("(lookup cost stays roughly constant: O(1))")
```

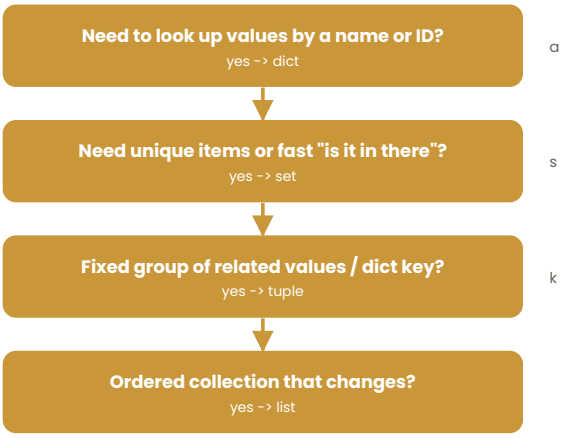
OUTPUT

```
dict of     1,000 keys: built and searched 100,000 times
dict of 1,000,000 keys: built and searched 100,000 times
1000x more keys, but lookups less than 3x slower: True
(lookup cost stays roughly constant: O(1))
```

9.5 Choosing a collection

9.5.1 Comparison of list, tuple, set and dict

	list	tuple	set	dict
Syntax	`[1, 2]`	`(1, 2)`	`{1, 2}`	`{"a": 1}`
Ordered	yes	yes	no	yes (insertion)
Mutable	yes	no	yes (frozenset: no)	yes
Duplicates	allowed	allowed	no	unique keys
Access	by index	by index	membership only	by key
Membership test	O(n)	O(n)	O(1)	O(1) for keys
Hashable	no	yes (if items are)	no (frozenset: yes)	no
Use when	ordered, changing items	fixed record, dict key	uniqueness, fast lookup, set algebra	look up values by a key



CHOOSING THE RIGHT COLLECTION

9.6 Problems

9.6.1 Word frequency count

COUNTING WORDS TWO WAYS

main.py

```
import re
from collections import Counter

text = """Python is easy. Python is powerful. Learning Python at LBF is fun,
and practice makes Python easier."""
words = re.findall(r"[a-z]+", text.lower())

freq = {}
for w in words:                          # 1. plain dictionary
    freq[w] = freq.get(w, 0) + 1
top = sorted(freq.items(), key=lambda kv: (-kv[1], kv[0]))[:3]
print("with a dict:", top)
print("with Counter:", Counter(words).most_common(3))
```

OUTPUT

```
with a dict: [('python', 4), ('is', 3), ('and', 1)]
with Counter: [('python', 4), ('is', 3), ('easy', 1)]
```

9.6.2 Group anagrams

Words are anagrams when their sorted letters are the same, so the sorted letters make a perfect dictionary key for grouping.

GROUPING ANAGRAMS BY A CANONICAL KEY main.py

```
from collections import defaultdict

words = ["listen", "silent", "enlist", "google", "gooegl", "cat", "act", "tac", "dog"]
groups = defaultdict(list)
for w in words:
    groups["".join(sorted(w))].append(w)
for key, group in groups.items():
    print(f"{key:<8} {group}")
```

OUTPUT

```
eilnst   ['listen', 'silent', 'enlist']
eggloo   ['google', 'gooegl']
act      ['cat', 'act', 'tac']
dgo      ['dog']
```

9.6.3 Two-sum with a dictionary

Two-sum: given a list of numbers and a target, find two positions whose numbers add up to the target. The brute-force way checks every pair ($O(n^2)$). With a dictionary that remembers each number's index, one pass is enough: for each number, check whether its complement (target - number) has already been seen ($O(n)$). This pattern, "remember what you have seen in a dict", solves many interview problems.

TWO-SUM IN ONE PASS main.py

```
def two_sum(nums, target):
    seen = {}                                # number -> index
    for i, n in enumerate(nums):
        complement = target - n
        if complement in seen:
            return seen[complement], i
        seen[n] = i
    return None

print(two_sum([2, 7, 11, 15], 9))
print(two_sum([3, 2, 4], 6))
print(two_sum([1, 2, 3], 100))
```

OUTPUT

```
(0, 1)
(1, 2)
None
```

9.6.4 Student records stored as a list of dictionaries

A list of dictionaries is the natural shape for tabular records in plain Python (and exactly what JSON APIs return): each dictionary is a row, keys are column names. Comprehensions, `sorted` with a key, and grouping with `defaultdict` answer typical queries.

QUERYING A LIST OF RECORDS

main.py

```
from collections import defaultdict
from statistics import mean

students = [
    {"roll": 101, "name": "Asha", "city": "Mysuru", "marks": 91},
    {"roll": 102, "name": "Ravi", "city": "Pune", "marks": 78},
    {"roll": 103, "name": "Meera", "city": "Mysuru", "marks": 85},
    {"roll": 104, "name": "John", "city": "Delhi", "marks": 64},
    {"roll": 105, "name": "Zoya", "city": "Pune", "marks": 95},
]
print("toppers:", [s["name"] for s in students if s["marks"] >= 85])
print("ranked:", [f'{s["name"]}({s["marks"]})' for s in sorted(students, key=lambda s: s["marks"],
reverse=True)])
by_roll = {s["roll"]: s for s in students}            # index for O(1) lookup by roll number
print("roll 103:", by_roll[103]["name"])
per_city = defaultdict(list)
for s in students:
    per_city[s["city"]].append(s["marks"])
for city, marks in sorted(per_city.items()):
    print(f"{city:<7} students {len(marks)}  average {mean(marks):.1f}")
best = max(students, key=lambda s: s["marks"])
print("best student:", best["name"], best["marks"])
```

OUTPUT

```
toppers: ['Asha', 'Meera', 'Zoya']
ranked: ['Zoya(95)', 'Asha(91)', 'Meera(85)', 'Ravi(78)', 'John(64)']
roll 103: Meera
Delhi   students 1  average 64.0
Mysuru  students 2  average 88.0
Pune    students 2  average 86.5
best student: Zoya 95
```

Sets are unordered collections of unique, hashable items: create them with {…} or set() (never {} for an empty set), modify them with add, update, remove (KeyError), discard, pop and clear, and combine them with | & - ^ (or methods), subset/superset tests (<= >=) and isdisjoint; frozenset is the immutable, hashable version; membership tests are O(1). Dictionaries map unique hashable keys to any values and keep insertion order: create with literals, dict(), zip or fromkeys; read with [] (KeyError) or get(key, default); change with assignment, update, |=, pop, popitem, del and setdefault; iterate with keys(), values() and items() views; merge with |; build with comprehensions; nest them like JSON; and sort their items with sorted and a key. collections adds defaultdict (automatic default values), Counter (frequencies and most_common), OrderedDict (move_to_end, LRU caches) and ChainMap (layered lookups). Hash tables make lookups average O(1). Choose dict for lookups by key, set for uniqueness and membership, tuple for fixed records and keys, and list for ordered changing data.

PRACTICE CHECKPOINT

Build a **phone book** (add, search, delete, list sorted) using a dictionary, and a **word-frequency report** using `Counter`.

PART 1: A DICTIONARY-BASED PHONE BOOK

main.py

```
phone_book = {}

while True:
    command = input("phonebook> ").strip()
    action, _, rest = command.partition(" ")
    match action.lower():
        case "add":
            name, _, number = rest.rpartition(" ")
            if not number.isdigit() or len(number) != 10:
                print("  number must have 10 digits")
                continue
            existed = name.title() in phone_book
            phone_book[name.title()] = number
            print(f"  {'updated' if existed else 'added'} {name.title()}")
        case "search":
            matches = {n: p for n, p in phone_book.items() if rest.lower() in n.lower()}
            for n, p in matches.items():
                print(f"  {n}: {p}")
            if not matches:
                print("  no match")
        case "delete":
            number = phone_book.pop(rest.title(), None)
            print(f"  deleted {rest.title()}" if number else f"  {rest.title()} not found")
        case "list":
            for n in sorted(phone_book, key=str.lower):
                print(f"  {n:<14}{phone_book[n]}")
        case "quit":
            break
        case _:
            print("  commands: add NAME NUMBER | search TEXT | delete NAME | list | quit")
```

CONSOLE (user input in gold)

```
phonebook> add Ravi Kumar 9845012345
  added Ravi Kumar
phonebook> add asha rao 9900112233
  added Asha Rao
phonebook> add Meera 98765
  number must have 10 digits
phonebook> add Meera N 9123456780
  added Meera N
phonebook> search ra
  Ravi Kumar: 9845012345
  Asha Rao: 9900112233
  Meera N: 9123456780
phonebook> list
  Asha Rao      9900112233
  Meera N       9123456780
  Ravi Kumar    9845012345
phonebook> delete ravi kumar
  deleted Ravi Kumar
phonebook> delete John
  John not found
phonebook> list
  Asha Rao      9900112233
  Meera N       9123456780
phonebook> quit
```

PART 2: A WORD-FREQUENCY REPORT WITH COUNTER

main.py

```
from collections import Counter
import re

text = """Learn Being Forward teaches Python. Python is simple, Python is powerful,
and learning Python opens doors. Practice Python every day and you will learn fast."""
STOP_WORDS = {"is", "and", "you", "will", "every"}

words = [w for w in re.findall(r"[a-z]+", text.lower()) if w not in STOP_WORDS]
counts = Counter(words)

print(f"{'WORD FREQUENCY REPORT':^30}")
print(f"total words: {len(words)} | distinct: {len(counts)}")
print("-" * 30)
for word, n in counts.most_common(5):
    print(f"{word:<10} {n:>3}  {'#' * n}")
print("-" * 30)
print("words used once:", sorted(w for w, n in counts.items() if n == 1))
```

OUTPUT

```
    WORD FREQUENCY REPORT
total words: 18 | distinct: 13
------------------------------
python       5  #####
learn        2  ##
being        1  #
forward      1  #
teaches      1  #
------------------------------
words used once: ['being', 'day', 'doors', 'fast', 'forward', 'learning', 'opens', 'powerful',
'practice', 'simple', 'teaches']
```

MODULE 10 · BEGINNER

Functions

Minimum duration **3 days** | Level **Beginner**

Functions package reusable logic under a name. Python functions are first-class objects with very flexible parameters. This module covers defining and calling functions, parameters and arguments, return values (including several at once), docstrings, every kind of argument (positional, keyword, default, *args, **kwargs, positional-only and keyword-only) and unpacking at call time, scope and the LEGB rule with global and nonlocal, how arguments are passed by object reference, recursion from base cases to Tower of Hanoi and permutations, memoization, and a tour of the most useful built-in functions.

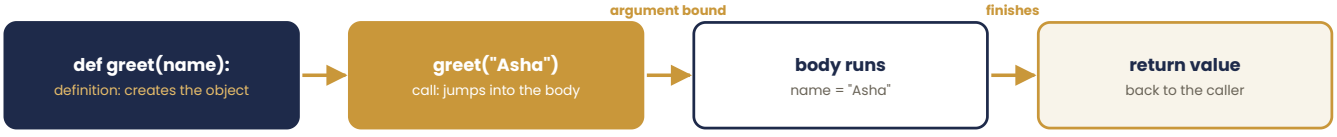
IN THIS MODULE

10.1 Function basics
10.2 Types of arguments
10.3 Scope and namespaces
10.4 Pass by object reference
10.5 Recursion
10.6 Built-in functions tour

10.1 Function basics

10.1.1 Defining with def and calling

A **function** is a named block of code that runs only when it is **called**. You define it with `def name(parameters):` followed by an indented body; you call it by writing its name followed by parentheses. Defining a function does not run its body: Python just creates a function object and binds it to the name. Functions remove repetition (write once, call many times), give a name to an idea (`is_prime(n)` reads better than ten lines of loop), and let you test small pieces separately.



DEFINING A FUNCTION ONLY CREATES IT; CALLING IT RUNS THE BODY AND RETURNS TO THE CALLER

DEFINING AND CALLING FUNCTIONS

main.py

```
def greet(name):
    print(f"Hello, {name}! Welcome to LBF.")

print("before any call")
greet("Asha")
greet("Ravi")
print(type(greet).__name__, greet.__name__)

def line():
    print("-" * 24)

line(); print("REPORT".center(24)); line()
```

OUTPUT

```
before any call
Hello, Asha! Welcome to LBF.
Hello, Ravi! Welcome to LBF.
function greet
------------------------
         REPORT
------------------------
```

A function must be defined **before the line that calls it runs**. Inside another function, though, you can refer to a function defined later in the file, because the lookup happens only when that outer function is actually called.

ORDER OF DEFINITION MATTERS ONLY AT CALL TIME

main.py

```
def main():
    print("total:", add(2, 3))     # add is defined below, but main runs later

def add(a, b):
    return a + b

main()
try:
    subtract(5, 1)
except NameError as e:
    print("NameError:", e)

def subtract(a, b):
    return a - b
```

OUTPUT

```
total: 5
NameError: name 'subtract' is not defined
```

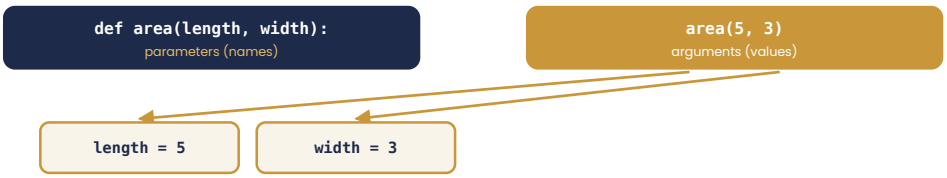
TIP

The usual layout of a Python script is: imports, constants, function definitions, and finally the code that calls them, often inside `main()` guarded by `if __name__ == "__main__":` (Module 13).

10.1.2 Parameters vs arguments

Parameters are the names listed in the definition; they act as local variables of the function. **Arguments** are the actual values passed in a call. When the call happens, each parameter is bound to its argument:

in `def area(length, width)` and the call `area(5, 3)`, `length` and `width` are parameters and `5` and `3` are arguments. Passing the wrong number of arguments raises `TypeError`.



EACH ARGUMENT IS BOUND TO ITS PARAMETER WHEN THE FUNCTION IS CALLED

PARAMETERS, ARGUMENTS AND A WRONG COUNT

main.py

```
def area(length, width):
    return length * width

print(area(5, 3))
print(area(2.5, 4))
try:
    area(5)
except TypeError as e:
    print("TypeError:", e)
try:
    area(1, 2, 3)
except TypeError as e:
    print("TypeError:", e)
```

OUTPUT

```
15
10.0
TypeError: area() missing 1 required positional argument: 'width'
TypeError: area() takes 2 positional arguments but 3 were given
```

10.1.3 return and returning multiple values (tuples)

`return value` ends the function immediately and sends the value back to the caller, where the call expression evaluates to it. A function can have several `return` statements, for example one per branch, and the first one reached wins. To return **several values**, separate them with commas: Python packs them into a **tuple**, which the caller usually unpacks straight into variables.

RETURN, EARLY RETURN AND MULTIPLE RETURN VALUES

main.py

```
def grade(marks):
    if marks >= 90:
        return "A"
    if marks >= 75:
        return "B"
    if marks >= 50:
        return "C"
    return "F"                      # reached only if no earlier return ran

for m in (95, 80, 62, 30):
    print(m, "->", grade(m))

def min_max(values):
    return min(values), max(values)  # packs a tuple

result = min_max([4, 9, 1, 7])
print(result, type(result).__name__)
low, high = min_max([4, 9, 1, 7])    # unpack
print("low:", low, "| high:", high)

q, r = divmod(17, 5)                 # built-ins do the same
print(q, r)
```

OUTPUT

```
95 -> A
80 -> B
62 -> C
30 -> F
(1, 9) tuple
low: 1 | high: 9
3 2
```

10.1.4 Functions without return (None)

A function that ends without a `return`, or with a bare `return`, returns `None`. Such functions are used for their **side effects**: printing, writing a file, modifying a list. A common beginner mistake is to print inside a function and then try to use its "result", or to assign the result of a method like `list.sort()`, which also returns `None`.

FUNCTIONS THAT RETURN NONE main.py

```
def show(x):
    print("showing", x)              # no return

value = show(5)
print("value is", value)

def check_age(age):
    if age < 0:
        print("invalid age")
        return                       # bare return: exit early, gives None
    print("age ok:", age)

print(check_age(-3))
print(check_age(20))

nums = [3, 1, 2]
result = nums.sort()                 # sorts in place, returns None
print(result, nums)
```

OUTPUT

```
showing 5
value is None
invalid age
None
age ok: 20
None
None [1, 2, 3]
```

WATCH OUT

`print` shows a value to a human; `return` hands it to the program. A function that prints its answer cannot be reused in a calculation. Prefer returning values and printing at the edge of the program.

10.1.5 Docstrings

A **docstring** is a string literal placed as the first statement of a function (or module or class). Python stores it in the function's `__doc__` attribute, and `help()`, editors and documentation tools display it. Convention (PEP 257): a one-line summary in the imperative mood ("Return…", not "Returns…"), and for longer docstrings, a blank line followed by details of arguments, return value and errors. Use triple double quotes.

WRITING AND READING A DOCSTRING main.py

```
def bmi(weight_kg, height_m):
    """Return the body-mass index for a weight in kg and height in metres.

    Raises ValueError if the height is not positive.
    """
    if height_m <= 0:
        raise ValueError("height must be positive")
    return round(weight_kg / height_m ** 2, 1)

print(bmi(68, 1.72))
print(bmi.__doc__.splitlines()[0])
help(bmi)
```

OUTPUT

```
23.0
Return the body-mass index for a weight in kg and height in metres.
Help on function bmi in module __main__:

bmi(weight_kg, height_m)
    Return the body-mass index for a weight in kg and height in metres.

    Raises ValueError if the height is not positive.
```

Docstrings document *what* the function does for its users; comments explain *how* or *why* for people reading the code. Type hints such as `def bmi(weight_kg: float, height_m: float) -> float:` (Module 18) complement docstrings by stating the expected types.

10.2 Types of arguments

Python offers very flexible ways to pass arguments. The full shape of a parameter list, in the order the parts must appear, is shown below; most functions use only one or two of these parts.

```
def f(a, b, /, c, d=1, *args, e, f=2, **kwargs)
```

THE ORDER OF EVERY KIND OF PARAMETER IN A DEF LINE

10.2.1 Positional arguments

Positional arguments are matched to parameters by position: the first argument to the first parameter, and so on. They are short to write but the order matters, and swapping two arguments of the same type is a silent bug.

POSITIONAL ARGUMENTS main.py

```
def describe(name, city, age):
    print(f"{name} from {city}, age {age}")

describe("Asha", "Mysuru", 21)
describe("Mysuru", 21, "Asha")        # wrong order: no error, wrong output
```

OUTPUT

```
Asha from Mysuru, age 21
Mysuru from 21, age Asha
```

10.2.2 Keyword arguments

Keyword arguments name the parameter in the call: `describe(age=21, name="Asha", city="Mysuru")`. Their order does not matter and the call documents itself. Positional and keyword arguments can be mixed, but all positional ones must come first, and no parameter may receive two values.

KEYWORD ARGUMENTS AND THE RULES FOR MIXING main.py

```
def describe(name, city, age):
    print(f"{name} from {city}, age {age}")

describe(age=21, name="Asha", city="Mysuru")
describe("Ravi", age=22, city="Pune")          # positional first, then keywords
print("one", "two", sep=" | ", end=" <end>\n")  # print uses keyword arguments
for bad in ('describe(name="Asha", "Mysuru", 21)', 'describe("Asha", name="Ravi", city="Pune",
age=1)'):
    try:
        eval(bad)
    except (SyntaxError, TypeError) as e:
        print(type(e).__name__ + ":", e.msg if isinstance(e, SyntaxError) else e)
```

OUTPUT

```
Asha from Mysuru, age 21
Ravi from Pune, age 22
one | two <end>
SyntaxError: positional argument follows keyword argument
TypeError: describe() got multiple values for argument 'name'
```

10.2.3 Default arguments and the mutable default trap

A parameter with a **default value**, `def power(base, exp=2)`, becomes optional: if the caller leaves it out the default is used. Parameters with defaults must come after those without. Defaults make functions convenient while keeping the common case short.

DEFAULT ARGUMENT VALUES

main.py

```
def power(base, exp=2):
    return base ** exp

print(power(5), power(5, 3), power(exp=0, base=9))

def make_greeting(name, greeting="Hello", punctuation="!"):
    return f"{greeting}, {name}{punctuation}"

print(make_greeting("Asha"))
print(make_greeting("Ravi", "Namaste"))
print(make_greeting("Meera", punctuation="."))
```

OUTPUT

```
25 125 1
Hello, Asha!
Namaste, Ravi!
Hello, Meera.
```

The trap: a default value is evaluated **once**, when the `def` line runs, not on every call. If the default is a mutable object such as a list or dictionary, every call that relies on the default shares the **same** object, so changes leak from one call into the next. The standard fix is to use `None` as the default and create a fresh object inside the function.

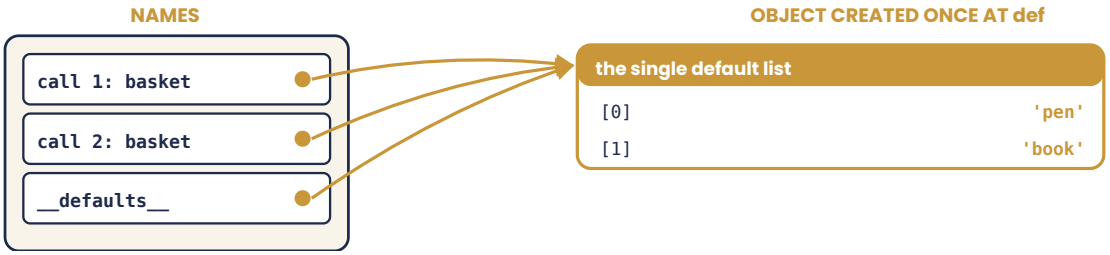
THE MUTABLE DEFAULT TRAP AND THE NONE FIX

main.py

```
def add_item_bad(item, basket=[]):          # ONE list created at def time
    basket.append(item)
    return basket

print(add_item_bad("pen"))
print(add_item_bad("book"))                    # surprise: pen is still there
print(add_item_bad.__defaults__)               # the shared default

def add_item(item, basket=None):               # the fix
    if basket is None:
        basket = []                            # fresh list on every call
    basket.append(item)
    return basket

print(add_item("pen"))
print(add_item("book"))
mine = ["bag"]
print(add_item("ruler", mine), mine)
```

OUTPUT

```
['pen']
['pen', 'book']
(['pen', 'book'],)
['pen']
['book']
['bag', 'ruler'] ['bag', 'ruler']
```



EVERY CALL THAT USES THE DEFAULT RECEIVES THE SAME LIST OBJECT

WATCH OUT

Never use `[]`, `{}` or `set()` as a default value. Linters flag it (ruff rule B006). Immutable defaults such as numbers, strings, tuples and `None` are always safe.

10.2.4 Variable positional *args

A parameter written `*args` collects **any number of extra positional arguments** into a tuple. The name `args` is only a convention; the star is what matters. This is how `print()`, `max()` and `os.path.join()` accept as many values as you give them.

COLLECTING EXTRA POSITIONAL ARGUMENTS WITH *ARGS

main.py

```
def total(*numbers):
    print("received:", numbers, type(numbers).__name__)
    return sum(numbers)

print(total(1, 2, 3))
print(total(10))
print(total())

def tag(label, *items):                        # normal parameters come first
    return f"{label}: " + ", ".join(items)

print(tag("Fruits", "mango", "banana", "apple"))
```

OUTPUT

```
received: (1, 2, 3) tuple
6
received: (10,) tuple
10
received: () tuple
0
Fruits: mango, banana, apple
```

10.2.5 Variable keyword **kwargs

`**kwargs` collects **any number of extra keyword arguments** into a dictionary of name to value. It is used for options, for building records with arbitrary fields, and for functions that pass their options on to another function. A function can take both: `def f(*args, **kwargs)` accepts any call at all.

COLLECTING EXTRA KEYWORD ARGUMENTS WITH **KWARGS

main.py

```
def profile(name, **details):
    print(name, "->", details)
    for key, value in details.items():
        print(f"  {key:<6}= {value}")

profile("Asha", city="Mysuru", age=21, course="Python")
profile("Ravi")

def anything(*args, **kwargs):
    print("args:", args, "| kwargs:", kwargs)

anything(1, 2, x=3, y=4)
```

OUTPUT

```
Asha -> {'city': 'Mysuru', 'age': 21, 'course': 'Python'}
  city  = Mysuru
  age   = 21
  course= Python
Ravi -> {}
args: (1, 2) | kwargs: {'x': 3, 'y': 4}
```

A useful pattern is a **wrapper** that forwards whatever it receives to another function, adding behaviour around it. Decorators (Module 11) are built on exactly this.

FORWARDING *ARGS AND **KWARGS

main.py

```
def logged_print(*args, **kwargs):
    print("[log]", *args, **kwargs)            # forward everything to print

logged_print("saving", "file.txt", sep=" -> ")
logged_print("done", end=" (ok)\n")
```

OUTPUT

```
[log] -> saving -> file.txt
[log] done (ok)
```

10.2.6 Positional-only / and keyword-only * parameters

Two markers control **how** parameters may be passed. Parameters before a `/` are **positional-only** (3.8): callers cannot use their names, which leaves you free to rename them later and is how many built-ins such as `len(obj, /)` work. Parameters after a `*` (or after `*args`) are **keyword-only**: callers must name them, which makes calls with flags and options readable and prevents mistakes.

POSITIONAL-ONLY AND KEYWORD-ONLY PARAMETERS

main.py

```
def divide(a, b, /):                    # positional-only
    return a / b

print(divide(10, 4))
try:
    divide(a=10, b=4)
except TypeError as e:
    print("TypeError:", e)

def connect(host, *, port=5432, timeout=10, ssl=True):   # keyword-only after *
    return f"{host}:{port} timeout={timeout} ssl={ssl}"

print(connect("db.lbf.in", port=6543, ssl=False))
try:
    connect("db.lbf.in", 6543, 30)
except TypeError as e:
    print("TypeError:", e)

def f(a, b, /, c, *, d):
    return a, b, c, d

print(f(1, 2, 3, d=4), f(1, 2, c=3, d=4))
```

OUTPUT

```
2.5
TypeError: divide() got some positional-only arguments passed as keyword arguments: 'a, b'
db.lbf.in:6543 timeout=10 ssl=False
TypeError: connect() takes 1 positional argument but 3 were given
(1, 2, 3, 4) (1, 2, 3, 4)
```

Marker	Effect	Typical use
`/`	parameters before it cannot be passed by name	built-in style APIs, names that may change
`*`	parameters after it must be passed by name	flags and options: `sorted(data, *, key, reverse)`
`*args`	also makes the following parameters keyword-only	`print(*objects, sep, end)`

10.2.7 Argument unpacking with * and ** at call time

The stars also work in the **opposite direction**, in a call: `f(*sequence)` spreads the items of a list, tuple or any iterable into separate positional arguments, and `f(**mapping)` spreads a dictionary into keyword arguments whose names are the keys.

UNPACKING ITERABLES AND DICTIONARIES INTO A CALL

main.py

```
def describe(name, city, age):
    return f"{name} from {city}, age {age}"

row = ["Asha", "Mysuru", 21]
record = {"name": "Ravi", "city": "Pune", "age": 22}
print(describe(*row))
print(describe(**record))

point = (3, 4)
print(pow(*point))                      # pow(3, 4)
print(*range(5), sep="-")
settings = {"sep": " | ", "end": "\n"}
print("a", "b", "c", **settings)

def total(*numbers):
    return sum(numbers)
marks = [91, 78, 85]
print(total(*marks, 100))
```

OUTPUT

```
Asha from Mysuru, age 21
Ravi from Pune, age 22
81
0-1-2-3-4
a | b | c
354
```

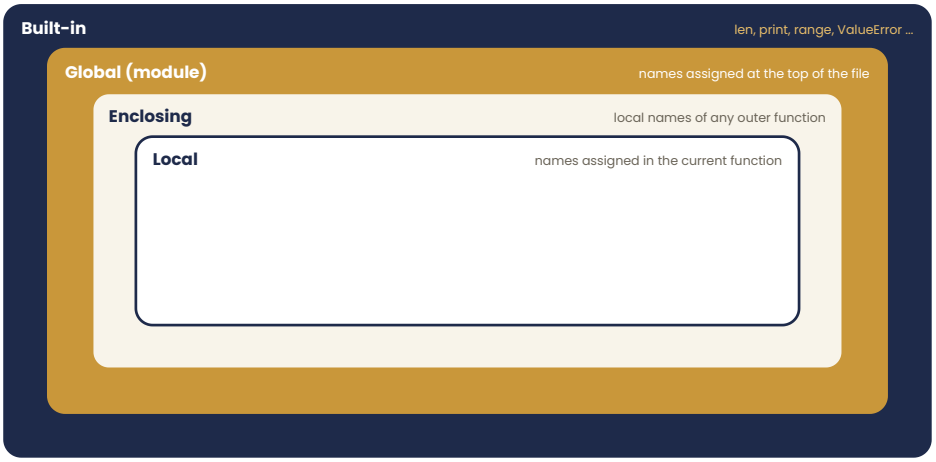
*ROW SPREADS THE LIST INTO THREE POSITIONAL ARGUMENTS

10.3 Scope and namespaces

A **namespace** is a mapping from names to objects; a **scope** is the region of code where a namespace is directly visible. Each function call gets a fresh **local** namespace that disappears when the call ends. Variables assigned inside a function are local to it, which is why two functions can each have their own `total` without interfering.

10.3.1 LEGB rule: Local, Enclosing, Global, Built-in

When Python meets a name, it searches four scopes in order and uses the first match: **L**ocal (the current function), **E**nclosing (any outer functions, for nested functions), **G**lobal (the module) and **B**uilt-in (names such as `len` and `print`). If none has the name, it raises `NameError`.



LEGB: PYTHON LOOKS FROM THE INNERMOST SCOPE OUTWARDS

NAME LOOKUP FOLLOWS LEGB

main.py

```
x = "global x"

def outer():
    x = "enclosing x"
    def inner():
        x = "local x"
        print("inner sees:", x)
    inner()
    print("outer sees:", x)

outer()
print("module sees:", x)

def uses_outer_names():
    print(len("abc"), x)          # len: built-in, x: global
uses_outer_names()
```

OUTPUT

```
inner sees: local x
outer sees: enclosing x
module sees: global x
3 global x
```

A key rule: **if a function assigns to a name anywhere in its body, that name is local in the whole function**. Reading it before the assignment then raises `UnboundLocalError`, even if a global of the same name exists. Python decides this when it compiles the function, not line by line.

ASSIGNMENT MAKES A NAME LOCAL: UNBOUNDLOCALERROR main.py

```
count = 10

def show():
    print(count)               # only reads: finds the global

def broken():
    print(count)               # count is local here, because of the next line
    count = 5

show()
try:
    broken()
except UnboundLocalError as e:
    print("UnboundLocalError:", e)

total = 0
def add_local(n):
    total = n                  # creates a NEW local; the global is unchanged
    return total
add_local(99)
print("global total still", total)
```

OUTPUT

```
10
UnboundLocalError: cannot access local variable 'count' where it is not associated with a value
global total still 0
```

Shadowing a built-in is legal but confusing: after `list = [1, 2]` at module level, `list("abc")` fails because the global `list` is found before the built-in one. Avoid names like `list`, `dict`, `str`, `sum`, `max` and `id` for variables.

SHADOWING A BUILT-IN NAME main.py

```
list = [1, 2, 3]               # shadows the built-in list
try:
    list("abc")
except TypeError as e:
    print("TypeError:", e)
del list                       # remove the global; the built-in is visible again
print(list("abc"))
print(dir(__builtins__)[-6:])
```

OUTPUT

```
TypeError: 'list' object is not callable
['a', 'b', 'c']
['sum', 'super', 'tuple', 'type', 'vars', 'zip']
```

10.3.2 global keyword

`global name` inside a function declares that assignments to `name` should change the **module-level** variable instead of creating a local one. It is occasionally useful for module-wide settings or counters in small scripts, but heavy use makes code hard to follow and test; prefer passing values in and returning results.

USING GLOBAL (AND AN ALTERNATIVE) main.py

```
calls = 0

def tracked():
    global calls
    calls += 1                  # changes the module variable
    return calls

tracked(); tracked(); tracked()
print("calls:", calls)

# better: return the new value and let the caller decide
def increment(n):
    return n + 1
score = 0
score = increment(score)
print("score:", score)
```

OUTPUT

```
calls: 3
score: 1
```

NOTE

Mutating a global object, for example `config["debug"] = True` or `items.append(x)`, does not need `global`, because the name itself is not reassigned. `global` is needed only to rebind the name.

10.3.3 nonlocal keyword

`nonlocal name` is the equivalent for **nested functions**: it makes assignments rebind a variable of the nearest **enclosing** function. Combined with the fact that an inner function keeps access to the enclosing variables after the outer function returns (a **closure**, Module 11), it lets a function carry private state.

NONLOCAL AND A COUNTER WITH PRIVATE STATE

main.py

```
def make_counter():
    count = 0
    def counter():
        nonlocal count          # rebind the enclosing count
        count += 1
        return count
    return counter

c1 = make_counter()
c2 = make_counter()
print(c1(), c1(), c1())        # each counter has its own count
print(c2())

def outer():
    n = 1
    def without_nonlocal():
        n = 100                 # a new local; outer's n unchanged
    def with_nonlocal():
        nonlocal n
        n = 100
    without_nonlocal(); print("after without:", n)
    with_nonlocal();    print("after with:", n)
outer()
```

OUTPUT

```
1 2 3
1
after without: 1
after with: 100
```

Situation	What happens	Needed keyword
Read a global or enclosing name	found by LEGB	none
Assign to a name in a function	creates a local	none
Rebind a module variable	changes the global	`global`
Rebind an outer function's variable	changes the enclosing one	`nonlocal`
Mutate an object (append, item set)	object changes everywhere	none

10.4 Pass by object reference

Python passes arguments by **object reference** (also called "call by sharing"): the parameter becomes a new name for the **same object** the caller passed. Nothing is copied. Two consequences follow, and they explain every "why did my list change?" surprise.

10.4.1 Passing mutable vs immutable objects

If the function **mutates** a mutable object (appending to a list, setting a dictionary key), the caller sees the change, because both names refer to one object. If the function **rebinds** its parameter (`x = ...`), only the local name moves to a new object; the caller's variable is untouched. Immutable objects (int, str, tuple) cannot be mutated, so a function can never change the caller's value; operations like `n += 1` rebind the local name.

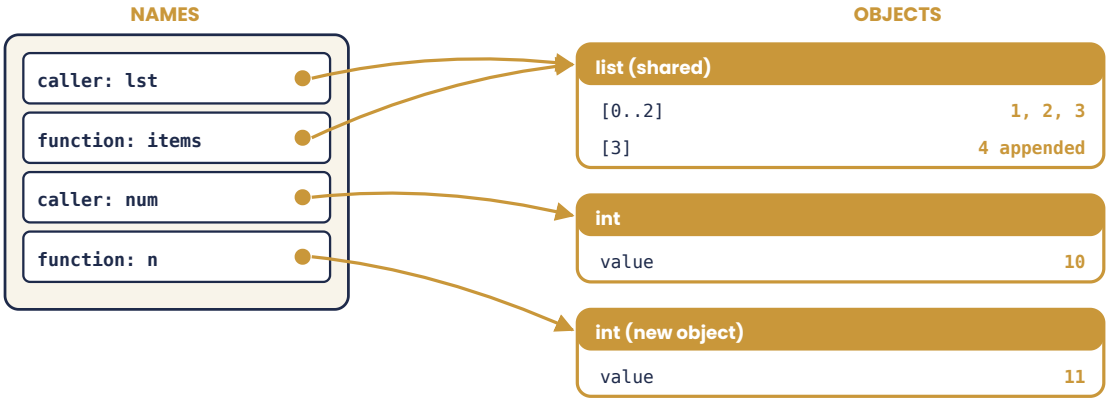
MUTATION IS VISIBLE TO THE CALLER; REBINDING IS NOT

main.py

```
def try_change(n, s, items, data):
    n += 1                      # rebinds local n (ints are immutable)
    s += "!"                    # rebinds local s
    items.append(4)             # mutates the shared list
    data["seen"] = True         # mutates the shared dict
    print("inside:", n, s, items, data)

num, text, lst, d = 10, "hi", [1, 2, 3], {}
try_change(num, text, lst, d)
print("outside:", num, text, lst, d)

def replace(items):
    items = [99]                # rebinding: caller unaffected
def clear_in_place(items):
    items[:] = []               # slice assignment mutates

a = [1, 2]; replace(a); print("after replace:", a)
clear_in_place(a); print("after clear_in_place:", a)
```

OUTPUT

```
inside: 11 hi! [1, 2, 3, 4] {'seen': True}
outside: 10 hi [1, 2, 3, 4] {'seen': True}
after replace: [1, 2]
after clear_in_place: []
```



THE LIST IS SHARED AND MUTATED; N += 1 MAKES THE LOCAL NAME POINT TO A NEW INT

If a function must not change its input, either make a copy first (`items = list(items)`, `copy.deepcopy` for nested data) or design it to **return a new object**. Functions that do not modify their arguments and have no other side effects are called **pure**; they are the easiest to test and reason about.

PURE FUNCTIONS RETURN NEW OBJECTS main.py

```
def with_tax_bad(prices):
    for i in range(len(prices)):
        prices[i] = round(prices[i] * 1.18, 2)   # changes the caller's list
    return prices

def with_tax(prices):                             # pure: returns a new list
    return [round(p * 1.18, 2) for p in prices]

cart = [100, 250]
print(with_tax(cart), "| original:", cart)
print(with_tax_bad(cart), "| original:", cart)
```

OUTPUT

```
[118.0, 295.0] | original: [100, 250]
[118.0, 295.0] | original: [118.0, 295.0]
```

10.5 Recursion

10.5.1 Base case and recursive case

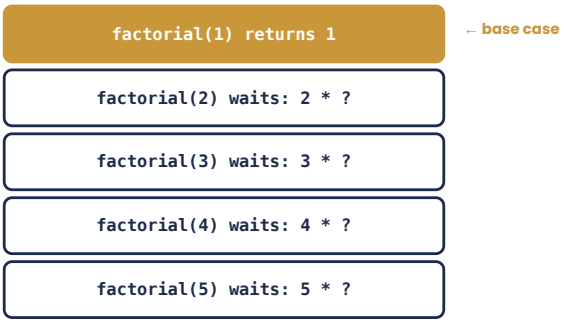
A **recursive** function calls itself to solve a smaller version of the same problem. Every recursive function needs two parts: a **base case** that is answered directly without recursion, and a **recursive case** that reduces the problem and moves it towards the base case. Missing or unreachable base cases cause infinite recursion. Each call gets its own frame on the **call stack** with its own local variables; frames are removed as calls return.

A FIRST RECURSIVE FUNCTION main.py

```
def countdown(n):
    if n == 0:                        # base case
        print("lift off!")
        return
    print(n)
    countdown(n - 1)                  # recursive case: smaller problem

countdown(3)

def factorial(n):
    if n <= 1:
        return 1
    return n * factorial(n - 1)

print("5! =", factorial(5))
```

OUTPUT

```
3
2
1
lift off!
5! = 120
```



THE CALL STACK AT THE DEEPEST POINT OF FACTORIAL(5); RESULTS THEN FLOW BACK DOWN: 1, 2, 6, 24, 120

Tracing a recursion with indentation makes the winding and unwinding visible:

TRACING THE CALLS AND RETURNS

main.py

```
def factorial(n, depth=0):
    pad = "  " * depth
    print(f"{pad}factorial({n}) called")
    if n <= 1:
        print(f"{pad}-> returns 1")
        return 1
    result = n * factorial(n - 1, depth + 1)
    print(f"{pad}-> returns {result}")
    return result

factorial(4)
```

OUTPUT

```
factorial(4) called
  factorial(3) called
    factorial(2) called
      factorial(1) called
      -> returns 1
    -> returns 2
  -> returns 6
-> returns 24
```

10.5.2 Recursion limit and sys.setrecursionlimit

Python limits the depth of the call stack to protect the interpreter; the default limit is 1000 frames. Exceeding it raises `RecursionError`. `sys.getrecursionlimit()` reads the limit and `sys.setrecursionlimit(n)` changes it, but raising it a lot risks crashing the process, and CPython does not optimise tail calls. For deep problems, rewrite the recursion as a loop.

HITTING AND RAISING THE RECURSION LIMIT main.py

```
import sys

print("limit:", sys.getrecursionlimit())

def depth(n):
    return depth(n + 1)              # no base case

try:
    depth(0)
except RecursionError as e:
    print("RecursionError:", e)

def total(n):
    return 0 if n == 0 else n + total(n - 1)

try:
    total(5000)
except RecursionError:
    print("total(5000) is too deep with the default limit")
sys.setrecursionlimit(10_000)
print("after raising the limit:", total(5000))

def total_loop(n):                   # the iterative version has no limit
    return sum(range(n + 1))
print("loop version:", total_loop(1_000_000))
```

OUTPUT

```
limit: 1000
RecursionError: maximum recursion depth exceeded
total(5000) is too deep with the default limit
after raising the limit: 12502500
loop version: 500000500000
```

10.5.3 Classic problems: factorial, Fibonacci, power, sum of digits, reverse a string, Tower of Hanoi, binary search, permutations

These problems all follow the same recipe: decide the smallest input you can answer immediately, then express the answer for `n` in terms of the answer for a smaller input.

Problem	Base case	Recursive case
factorial(n)	n <= 1 → 1	n * factorial(n-1)
fib(n)	n < 2 → n	fib(n-1) + fib(n-2)
power(b, e)	e == 0 → 1	square power(b, e//2), times b if e odd
digit_sum(n)	n < 10 → n	n % 10 + digit_sum(n // 10)
reverse(s)	len(s) <= 1 → s	reverse(s[1:]) + s[0]
hanoi(n)	n == 0 → nothing	move n-1 aside, move 1, move n-1 back on top
binary_search	empty range → not found	search the half that can contain the target
permutations(s)	len(s) <= 1 → [s]	each char + permutations of the rest

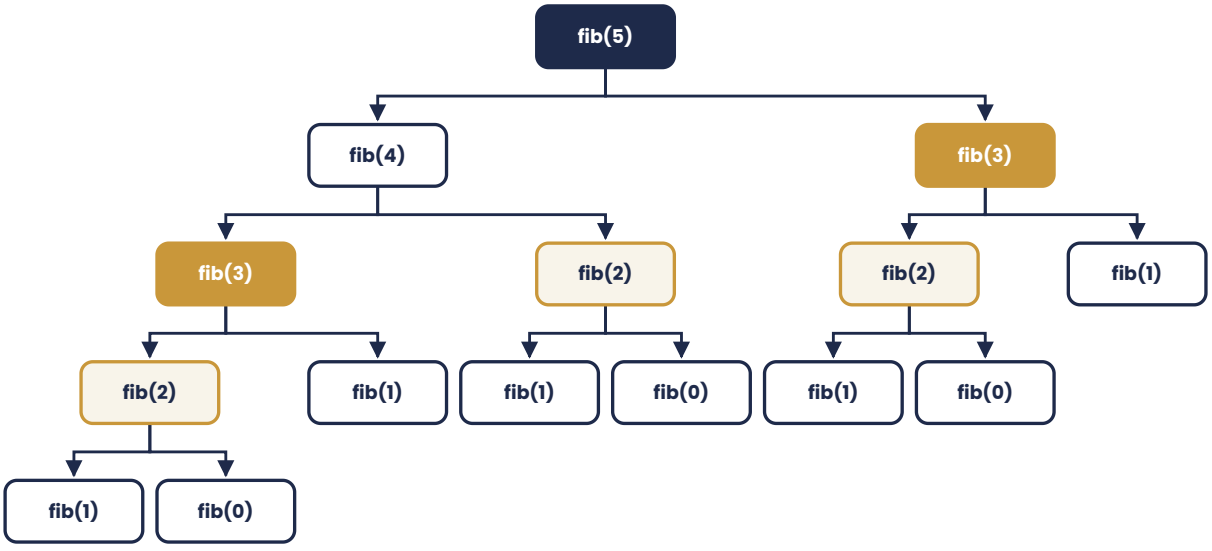
FIBONACCI, FAST POWER, DIGIT SUM AND STRING REVERSAL

main.py

```
def fib(n):
    return n if n < 2 else fib(n - 1) + fib(n - 2)

def power(base, exp):
    if exp == 0:
        return 1
    half = power(base, exp // 2)      # fast power: O(log exp) calls
    return half * half if exp % 2 == 0 else half * half * base

def digit_sum(n):
    return n if n < 10 else n % 10 + digit_sum(n // 10)

def reverse(s):
    return s if len(s) <= 1 else reverse(s[1:]) + s[0]

print("fib 0-10:", [fib(i) for i in range(11)])
print("2^10 =", power(2, 10), "| 3^13 =", power(3, 13))
print("digit_sum(98765) =", digit_sum(98765))
print("reverse('python') =", reverse("python"))
```

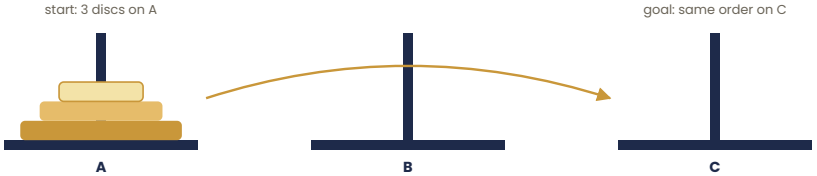
OUTPUT

```
fib 0-10: [0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55]
2^10 = 1024 | 3^13 = 1594323
digit_sum(98765) = 35
reverse('python') = nohtyp
```



THE NAIVE FIB(5) RECOMPUTES FIB(3) TWICE (GOLD) AND FIB(2) THREE TIMES (CREAM): EXPONENTIAL WORK

Tower of Hanoi. Move `n` discs from peg A to peg C using peg B, moving one disc at a time and never placing a larger disc on a smaller one. The recursive insight: to move `n` discs, move the top `n−1` to the spare peg, move the largest disc to the target, then move the `n−1` discs on top of it. It takes exactly $2^n - 1$ moves.



TOWER OF HANOI WITH THREE DISCS

TOWER OF HANOI main.py

```
def hanoi(n, source="A", target="C", spare="B", moves=None):
    if moves is None:
        moves = []
    if n == 0:
        return moves
    hanoi(n - 1, source, spare, target, moves)    # clear the way
    moves.append((n, source, target))             # move the largest disc
    hanoi(n - 1, spare, target, source, moves)    # stack the rest on top
    return moves

for disc, frm, to in hanoi(3):
    print(f"move disc {disc} from {frm} to {to}")
for n in (1, 2, 3, 4, 10, 20):
    print(f"{n:>2} discs: {len(hanoi(n)):>9,} moves (2^n - 1 = {2**n - 1:,})")
```

OUTPUT

```
move disc 1 from A to C
move disc 2 from A to B
move disc 1 from C to B
move disc 3 from A to C
move disc 1 from B to A
move disc 2 from B to C
move disc 1 from A to C
 1 discs:         1 moves (2^n - 1 = 1)
 2 discs:         3 moves (2^n - 1 = 3)
 3 discs:         7 moves (2^n - 1 = 7)
 4 discs:        15 moves (2^n - 1 = 15)
10 discs:     1,023 moves (2^n - 1 = 1,023)
20 discs: 1,048,575 moves (2^n - 1 = 1,048,575)
```

Recursive binary search searches a sorted list by comparing with the middle item and recursing into the half that can contain the target. Passing `lo` and `hi` indices avoids copying slices.

RECURSIVE BINARY SEARCH main.py

```
def binary_search(items, target, lo=0, hi=None):
    if hi is None:
        hi = len(items) - 1
    if lo > hi:
        return -1                                  # base case: empty range
    mid = (lo + hi) // 2
    print(f"  looking at [{lo}..{hi}], middle {items[mid]}")
    if items[mid] == target:
        return mid
    if target < items[mid]:
        return binary_search(items, target, lo, mid - 1)
    return binary_search(items, target, mid + 1, hi)

data = [3, 8, 15, 21, 34, 42, 57, 66, 79, 91]
print("index of 57:", binary_search(data, 57))
print("index of 10:", binary_search(data, 10))
```

OUTPUT

```
  looking at [0..9], middle 34
  looking at [5..9], middle 66
  looking at [5..6], middle 42
  looking at [6..6], middle 57
index of 57: 6
  looking at [0..9], middle 34
  looking at [0..3], middle 8
  looking at [2..3], middle 15
index of 10: -1
```

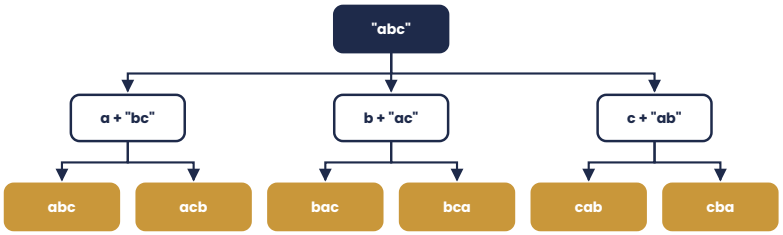
Permutations. The permutations of a string are each character followed by every permutation of the remaining characters. A string of `n` distinct characters has `n!` permutations. The standard library provides `itertools.permutations`, which is faster and lazy.

GENERATING PERMUTATIONS RECURSIVELY main.py

```
def permutations(s):
    if len(s) <= 1:
        return [s]
    result = []
    for i, ch in enumerate(s):
        rest = s[:i] + s[i + 1:]
        for p in permutations(rest):
            result.append(ch + p)
    return result

print(permutations("abc"))
print(len(permutations("abcd")), "permutations of 4 characters")

from itertools import permutations as perms
print(["".join(p) for p in perms("abc")])
```

OUTPUT

```
['abc', 'acb', 'bac', 'bca', 'cab', 'cba']
24 permutations of 4 characters
['abc', 'acb', 'bac', 'bca', 'cab', 'cba']
```



FIX ONE CHARACTER PER LEVEL; THE LEAVES ARE THE 3! = 6 PERMUTATIONS

10.5.4 Memoization with a dictionary

Memoization stores the results of calls in a dictionary so that each distinct input is computed only once. It turns the exponential naive Fibonacci into linear time. The standard library does this for you with the `functools.lru_cache` or `functools.cache` decorator (Module 11).

MEMOIZATION TURNS EXPONENTIAL INTO LINEAR

main.py

```
import time

calls = 0
def fib_naive(n):
    global calls
    calls += 1
    return n if n < 2 else fib_naive(n - 1) + fib_naive(n - 2)

memo = {}
def fib_memo(n):
    if n in memo:
        return memo[n]                    # already computed
    result = n if n < 2 else fib_memo(n - 1) + fib_memo(n - 2)
    memo[n] = result                      # remember it
    return result

print("naive fib(25):", fib_naive(25), "using", f"{calls:,}", "calls")
print("memo  fib(25):", fib_memo(25), "using", len(memo), "stored results")
print("memo fib(200):", fib_memo(200))

from functools import cache
@cache
def fib(n):
    return n if n < 2 else fib(n - 1) + fib(n - 2)
print("cached fib(90):", fib(90))
```

OUTPUT

```
naive fib(25): 75025 using 242,785 calls
memo  fib(25): 75025 using 26 stored results
memo fib(200): 280571172992510140037611932413038677189525
cached fib(90): 2880067194370816120
```

FUNCTION CALLS NEEDED FOR FIB(25)

TIP

Recursion is natural for problems that are recursive by nature: trees, nested data, divide-and-conquer (merge sort, quick sort), and backtracking. For simple linear repetition, a loop is usually clearer and faster in Python.

10.6 Built-in functions tour

10.6.1 len, range, type, isinstance, sorted, round, abs, help, dir

Python ships with about 70 built-in functions that are always available without an import. The ones below are used in almost every program.

Function	What it does	Example
`len(x)`	number of items in a sequence or collection	`len("hello")` → 5
`range(start, stop, step)`	lazy sequence of integers	`list(range(1, 10, 3))` → [1, 4, 7]
`type(x)`	the exact type of an object	`type(3.0)` → float
`isinstance(x, T)`	is x an instance of T (or a subclass)?	`isinstance(True, int)` → True
`sorted(it, key, reverse)`	new sorted list from any iterable	`sorted("cab")` → ['a', 'b', 'c']
`round(x, n)`	round to n digits (banker's rounding at .5)	`round(2.675, 2)` → 2.67
`abs(x)`	absolute value	`abs(-7)` → 7
`help(obj)`	show documentation	`help(len)`
`dir(obj)`	list the attributes of an object	`dir(str)`

THE MOST COMMON BUILT-INS

main.py

```
print(len([1, 2, 3]), len({"a": 1}), len(range(0, 100, 5)))
print(list(range(5)), list(range(10, 0, -3)), range(1_000_000)[-1])
print(type(42), type("hi"), type([]) is list)
print(isinstance(42, int), isinstance(True, int), isinstance(3.0, (int, float)))
print(sorted([3, 1, 2], reverse=True), sorted(["banana", "Apple", "cherry"], key=str.lower))
print(round(3.14159, 2), round(2.5), round(3.5), round(1234, -2))
print(abs(-7), abs(-2.5), abs(3 + 4j))
```

OUTPUT

```
3 1 20
[0, 1, 2, 3, 4] [10, 7, 4, 1] 999999
<class 'int'> <class 'str'> True
True True True
[3, 2, 1] ['Apple', 'banana', 'cherry']
3.14 2 4 1200
7 2.5 5.0
```

`type()` versus `isinstance()`: prefer `isinstance` for checks, because it accepts subclasses and a tuple of types. `help()` and `dir()` are your interactive documentation: use them in the REPL whenever you forget a method name or its parameters.

EXPLORING WITH DIR() AND HELP()

main.py

```
print([name for name in dir(str) if not name.startswith("_")][:12])
print(callable(len), callable(42))
help(abs)
```

OUTPUT

```
['capitalize', 'casefold', 'center', 'count', 'encode', 'endswith', 'expandtabs', 'find', 'format',
'format_map', 'index', 'isalnum']
True False
Help on built-in function abs in module builtins:

abs(x, /)
    Return the absolute value of the argument.
```

Other built-ins met throughout this book include `print`, `input`, `int` / `float` / `str` / `bool`, `list` / `tuple` / `set` / `dict`, `min` / `max` / `sum`, `any` / `all`, `enumerate` / `zip` / `map` / `filter`, `reversed`, `open`, `id`, `hash`, `ord` / `chr`, `divmod` / `pow`, `iter` / `next` and `getattr` / `setattr` / `hasattr`.

HOW MANY BUILT-INS ARE THERE?

main.py

```
import builtins
funcs = [n for n in dir(builtins) if callable(getattr(builtins, n)) and n.islower() and not
n.startswith("_")]
print(len(funcs), "lower-case built-in callables, for example:")
print(", ".join(funcs[:24]))
```

OUTPUT

```
75 lower-case built-in callables, for example:
abs, aiter, all, anext, any, ascii, bin, bool, breakpoint, bytearray, bytes, callable, chr,
classmethod, compile, complex, copyright, credits, delattr, dict, dir, divmod, enumerate, eval
```

Define functions with def and call them with parentheses; parameters are the names in the definition, arguments are the values in the call. return ends the call and sends back a value (commas return a tuple); without return a function gives None. Document functions with docstrings. Arguments can be positional or keyword; defaults make parameters optional but must never be mutable (use None); *args collects extra positional arguments into a tuple and **kwargs extra keyword arguments into a dict; / makes earlier parameters positional-only and * makes later ones keyword-only; * and ** in a call unpack iterables and dicts. Names are resolved by LEGB (Local, Enclosing, Global, Built-in); assigning to a name makes it local, global rebinds a module variable and nonlocal an enclosing one. Arguments are passed by object reference: mutations are visible to the caller, rebinding is not. Recursion needs a base case and a recursive case, is limited to about 1000 frames by default, and can be sped up with memoization. Built-ins such as len, range, type, isinstance, sorted, round, abs, help and dir are always available.

PRACTICE CHECKPOINT

Write a function **stats(*numbers)** that returns the minimum, maximum and average, a function that uses **keyword-only arguments**, and a recursive **Tower of Hanoi**.

PART 1: STATS(*NUMBERS)

main.py

```
def stats(*numbers):
    """Return (minimum, maximum, average) of any number of values."""
    if not numbers:
        raise ValueError("stats() needs at least one number")
    return min(numbers), max(numbers), sum(numbers) / len(numbers)

low, high, avg = stats(91, 78, 85, 64, 99)
print(f"min {low} | max {high} | average {avg:.2f}")
marks = [72, 88, 95]
print("from a list:", stats(*marks))
try:
    stats()
except ValueError as e:
    print("ValueError:", e)
```

OUTPUT

```
min 64 | max 99 | average 83.40
from a list: (72, 95, 85.0)
ValueError: stats() needs at least one number
```

PART 2: KEYWORD-ONLY ARGUMENTS

main.py

```
def format_price(amount, *, currency="Rs", decimals=2, thousands=True):
    """Format an amount; options must be passed by keyword."""
    number = f"{amount:,.{decimals}f}" if thousands else f"{amount:.{decimals}f}"
    return f"{currency} {number}"

print(format_price(1234567.891))
print(format_price(1234567.891, currency="USD", decimals=0))
print(format_price(99.5, thousands=False, decimals=1))
try:
    format_price(10, "USD")
except TypeError as e:
    print("TypeError:", e)
```

OUTPUT

```
Rs 1,234,567.89
USD 1,234,568
Rs 99.5
TypeError: format_price() takes 1 positional argument but 2 were given
```

PART 3: RECURSIVE TOWER OF HANOI

main.py

```
def hanoi(n, source, target, spare):
    """Print the moves to shift n discs from source to target; return the count."""
    if n == 0:
        return 0
    count = hanoi(n - 1, source, spare, target)
    print(f"  disc {n}: {source} -> {target}")
    count += 1
    count += hanoi(n - 1, spare, target, source)
    return count

discs = int(input("How many discs? "))
moves = hanoi(discs, "A", "C", "B")
print(f"{discs} discs solved in {moves} moves (2^{discs} - 1 = {2**discs - 1})")
```

CONSOLE (user input in gold)

```
How many discs? 4
  disc 1: A -> B
  disc 2: A -> C
  disc 1: B -> C
  disc 3: A -> B
  disc 1: C -> A
  disc 2: C -> B
  disc 1: A -> B
  disc 4: A -> C
  disc 1: B -> C
  disc 2: B -> A
  disc 1: C -> A
  disc 3: B -> C
  disc 1: A -> B
  disc 2: A -> C
  disc 1: B -> C
4 discs solved in 15 moves (2^4 - 1 = 15)
```

LEVEL 2

Intermediate level

Outcome: use decorators, modules and packages, handle exceptions and files, design classes with OOP, and work with iterators and generators.

7 MODULES · 19 DAYS MINIMUM

Module 11 · Functional programming and decorators	**3 days**
Module 12 · Modules, packages and virtual environments	**2 days**
Module 13 · Exception handling	**2 days**
Module 14 · File handling and data formats	**2 days**
Module 15 · Object-oriented programming: classes and objects	**3 days**
Module 16 · Object-oriented programming: inheritance, polymorphism and special methods	**4 days**
Module 17 · Iterators, generators and context managers	**3 days**

MODULE 11 · INTERMEDIATE

Functional programming and decorators

Minimum duration **3 days** | Level **Intermediate**

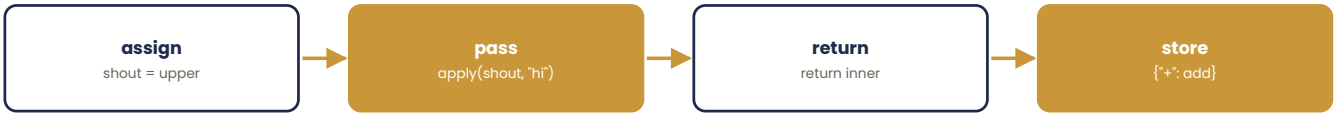
Python functions are objects that can be stored in variables, passed to other functions, returned from functions and wrapped. This module covers functions as first-class objects, lambda expressions, the higher-order functions map, filter and reduce (and when comprehensions are better), closures and the variables they capture, decorators from first principles to decorators with arguments, stacking and class-based decorators, practical decorators (timer, logger, retry, access check), and the functools tools lru_cache, cache, partial and singledispatch.

IN THIS MODULE

11.1 Functions as first-class objects
11.2 Lambda functions
11.3 Higher-order functions
11.4 Closures
11.5 Decorators
11.6 functools

11.1 Functions as first-class objects

In Python, a function is an **object** like any other: it has a type (`function`), attributes (`__name__`, `__doc__`, `__defaults__`) and an identity. Being **first-class** means a function can be used anywhere a value can: assigned to a variable, passed as an argument, returned from another function, and stored in lists and dictionaries. This single idea is the foundation of everything in this module.



FOUR THINGS YOU CAN DO WITH A FUNCTION BECAUSE IT IS AN OBJECT

11.1.1 Assigning functions to variables

Writing a function name **without parentheses** refers to the function object itself; adding parentheses calls it. Assigning `say = greet` does not call `greet`: it makes a second name for the same function object.

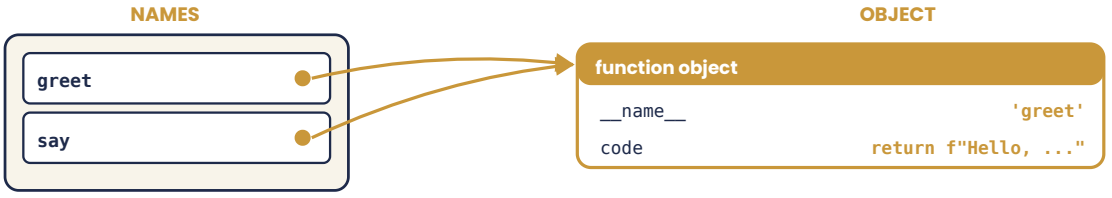
A FUNCTION IS AN OBJECT WITH NAMES

main.py

```
def greet(name):
    return f"Hello, {name}"

say = greet                          # no parentheses: the function itself
print(say("Asha"))
print(say is greet, say.__name__)
print(type(greet).__name__, callable(greet))
print(greet)                         # the object, not a call

del greet                            # removes one name only
print(say("Ravi"))
```

OUTPUT

```
Hello, Asha
True greet
function True
<function greet at 0x7f150780f530>
Hello, Ravi
```



TWO NAMES REFERRING TO ONE FUNCTION OBJECT

11.1.2 Passing functions as arguments

A function that takes another function as an argument can leave part of its behaviour to the caller. You have already used this: `sorted(words, key=len)` passes the built-in `len` to `sorted`, which calls it on every item. Functions that accept or return other functions are called **higher-order functions**.

PASSING FUNCTIONS TO OTHER FUNCTIONS

main.py

```
def apply(func, value):
    return func(value)

def double(x):
    return x * 2

print(apply(double, 21))
print(apply(str.upper, "lbf"))
print(apply(len, [1, 2, 3]))

def apply_to_all(func, items):
    return [func(item) for item in items]

print(apply_to_all(double, [1, 2, 3]))
print(apply_to_all(abs, [-3, 4, -5]))
print(sorted(["banana", "kiwi", "apple"], key=len))
```

OUTPUT

```
42
LBF
3
[2, 4, 6]
[3, 4, 5]
['kiwi', 'apple', 'banana']
```

Passing a function is a clean way to separate *what to do* from *how to iterate*. The function below handles retries, validation or reporting once, and the caller supplies only the specific action.

BEHAVIOUR SUPPLIED BY THE CALLER

main.py

```
def report(title, items, formatter):
    print(title)
    for item in items:
        print("  " + formatter(item))

def as_money(x):
    return f"Rs {x:>8,.2f}"

def as_percent(x):
    return f"{x:>6.1%}"

report("Prices", [99.5, 1250, 18999], as_money)
report("Growth", [0.051, 0.125, -0.02], as_percent)
```

OUTPUT

```
Prices
  Rs    99.50
  Rs 1,250.00
  Rs 18,999.00
Growth
    5.1%
   12.5%
   -2.0%
```

11.1.3 Returning functions

A function can create another function inside itself and return it. Each call can return a differently configured function. Such a function is often called a **factory**.

RETURNING A FUNCTION FROM A FUNCTION main.py

```
def make_multiplier(factor):
    def multiply(x):
        return x * factor
    return multiply                  # return the function, not a call

double = make_multiplier(2)
triple = make_multiplier(3)
print(double(10), triple(10))
print(double.__name__, double is triple)
print(make_multiplier(5)(4))         # create and call at once
```

OUTPUT

```
20 30
multiply False
20
```

11.1.4 Storing functions in data structures

Functions can be stored in lists, tuples and dictionaries. A dictionary mapping names to functions, a **dispatch table**, replaces long `if-elif` chains: you look up the function by key and call it.

A DISPATCH TABLE AND A LIST OF FUNCTIONS main.py

```
import operator

operations = {
    "+": operator.add,
    "-": operator.sub,
    "*": operator.mul,
    "/": operator.truediv,
    "^": pow,
}

for expr in ("8 + 2", "8 - 2", "8 * 2", "8 / 2", "2 ^ 10", "8 % 2"):
    a, op, b = expr.split()
    func = operations.get(op)
    if func is None:
        print(f"{expr:<7} -> unknown operator {op!r}")
    else:
        print(f"{expr:<7} -> {func(float(a), float(b)):g}")

pipeline = [str.strip, str.lower, str.title]
text = "   hELLO wORLD   "
for step in pipeline:
    text = step(text)
print(repr(text))
```

OUTPUT

```
8 + 2   -> 10
8 - 2   -> 6
8 * 2   -> 16
8 / 2   -> 4
2 ^ 10  -> 1024
8 % 2   -> unknown operator '%'
'Hello World'
```

TIP

The `operator` module provides function versions of all operators (`add` , `mul` , `itemgetter` , `attrgetter`), which are handy whenever a function argument is needed.

11.2 Lambda functions

11.2.1 Syntax lambda args: expression

A **lambda** creates a small anonymous function in a single expression: `lambda parameters: expression`. The expression's value is returned automatically; there is no `return` keyword and the body cannot contain statements (no assignment with `=`, no loops, no `if` statements, though a conditional expression is fine). Lambdas support defaults, `*args` and `**kwargs` just like `def`.

def	Equivalent lambda
`def square(x): return x * x`	`lambda x: x * x`
`def add(a, b=1): return a + b`	`lambda a, b=1: a + b`
`def sign(n): return "+" if n >= 0 else "-"`	`lambda n: "+" if n >= 0 else "-"`
`def total(*xs): return sum(xs)`	`lambda *xs: sum(xs)`

LAMBDA SYNTAX

main.py

```
square = lambda x: x * x            # works, but PEP 8 prefers def here
print(square(7), square.__name__)

add = lambda a, b=1: a + b
print(add(5), add(5, 10))

sign = lambda n: "positive" if n > 0 else "zero" if n == 0 else "negative"
print([sign(n) for n in (5, 0, -3)])

print((lambda *xs: max(xs) - min(xs))(4, 9, 1, 7))    # called immediately
```

OUTPUT

```
49 <lambda>
6 15
['positive', 'zero', 'negative']
8
```

11.2.2 Lambdas with sorted, min, max

The natural home of a lambda is a `key=` argument, where a small throwaway function is needed for one call. `sorted` , `list.sort` , `min` , `max` and many library functions accept a key function that maps each item to the value to compare.

LAMBDAS AS KEY FUNCTIONS main.py

```
students = [("Asha", 91, "Mysuru"), ("Ravi", 78, "Pune"), ("Meera", 85, "Mysuru"), ("John", 91,
"Delhi")]

print(sorted(students, key=lambda s: s[1]))                  # by marks
print(sorted(students, key=lambda s: (-s[1], s[0])))         # marks desc, then name
print(max(students, key=lambda s: s[1])[0])
print(min(students, key=lambda s: len(s[0]))[0])

words = ["banana", "Apple", "cherry", "date"]
print(sorted(words, key=lambda w: w.lower()))
print(sorted(words, key=lambda w: w[-1]))                    # by last letter

prices = {"pen": 10, "bag": 899, "book": 250}
print(sorted(prices.items(), key=lambda kv: kv[1], reverse=True))
```

OUTPUT

```
[('Ravi', 78, 'Pune'), ('Meera', 85, 'Mysuru'), ('Asha', 91, 'Mysuru'), ('John', 91, 'Delhi')]
[('Asha', 91, 'Mysuru'), ('John', 91, 'Delhi'), ('Meera', 85, 'Mysuru'), ('Ravi', 78, 'Pune')]
Asha
Asha
['Apple', 'banana', 'cherry', 'date']
['banana', 'Apple', 'date', 'cherry']
[('bag', 899), ('book', 250), ('pen', 10)]
```

For simple item and attribute access, `operator.itemgetter` and `operator.attrgetter` are faster and arguably clearer than a lambda: `sorted(students, key=itemgetter(1))`.

ITEMGETTER INSTEAD OF A LAMBDA main.py

```
from operator import itemgetter

students = [("Asha", 91, "Mysuru"), ("Ravi", 78, "Pune"), ("Meera", 85, "Mysuru")]
print(sorted(students, key=itemgetter(1)))
print(sorted(students, key=itemgetter(2, 0)))     # by city, then name
```

OUTPUT

```
[('Ravi', 78, 'Pune'), ('Meera', 85, 'Mysuru'), ('Asha', 91, 'Mysuru')]
[('Asha', 91, 'Mysuru'), ('Meera', 85, 'Mysuru'), ('Ravi', 78, 'Pune')]
```

11.2.3 When not to use lambdas

Lambdas are for short, obvious functions used once. Use `def` instead when:

Assigning to a name

square = lambda x: x*x gives a function called <lambda> in tracebacks. Use def square(x).

Logic is complex

Nested conditional expressions in a lambda are hard to read. A def with if statements is clearer.

It needs statements

Loops, try/except, assignments and multiple steps are impossible in a lambda.

It needs a docstring or tests

A named def can be documented, tested and reused.

A classic trap is creating lambdas in a loop: each lambda looks up the loop variable **when it is called**, not when it is created, so they all see its final value (late binding). Binding the current value as a default argument fixes it.

LATE BINDING AND NAMING PROBLEMS main.py

```
funcs = [lambda: i for i in range(3)]
print([f() for f in funcs])            # all see the final i

funcs = [lambda i=i: i for i in range(3)]   # capture the current value
print([f() for f in funcs])

bad = lambda x: x * 2
print(bad.__name__)                     # unhelpful name in tracebacks
def double(x):
    return x * 2
print(double.__name__)
```

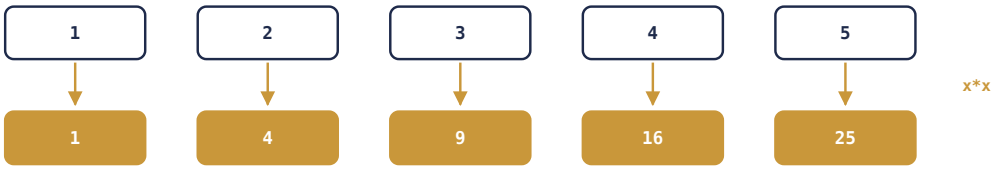
OUTPUT

```
[2, 2, 2]
[0, 1, 2]
<lambda>
double
```

11.3 Higher-order functions

11.3.1 map()

`map(function, iterable, ...)` applies a function to every item and returns a lazy **iterator** of the results; wrap it in `list()` to see them all. With several iterables, the function receives one item from each, stopping at the shortest.



MAP(LAMBDA X: X * X, [1, 2, 3, 4, 5]) TRANSFORMS EVERY ITEM

MAP() main.py

```
nums = [1, 2, 3, 4, 5]
squares = map(lambda x: x * x, nums)
print(squares)                          # a lazy map object
print(list(squares))
print(list(squares))                    # exhausted after one pass

print(list(map(str.upper, ["asha", "ravi"])))
print(list(map(int, "7 3 9".split())))
print(list(map(pow, [2, 3, 4], [5, 2, 1])))           # two iterables
print(list(map(lambda a, b: a + b, [1, 2, 3], [10, 20])))
```

OUTPUT

```
<map object at 0x7f0210293a40>
[1, 4, 9, 16, 25]
[]
['ASHA', 'RAVI']
[7, 3, 9]
[32, 9, 4]
[11, 22]
```

11.3.2 filter()

`filter(function, iterable)` keeps only the items for which the function returns a truthy value, again lazily. Passing `None` as the function keeps the truthy items themselves, a quick way to drop empty strings, zeros and `None`.

FILTER()

main.py

```
nums = range(1, 21)
print(list(filter(lambda n: n % 2 == 0, nums)))
print(list(filter(str.isdigit, ["12", "abc", "7", "4.5"])))
print(list(filter(None, [0, 1, "", "hi", None, [], [0], False])))

def is_prime(n):
    return n > 1 and all(n % d for d in range(2, int(n ** 0.5) + 1))
print(list(filter(is_prime, range(50))))
```

OUTPUT

```
[2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
['12', '7']
[1, 'hi', [0]]
[2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]
```

11.3.3 functools.reduce()

`functools.reduce(function, iterable, initial)` combines the items into **one value** by repeatedly applying a two-argument function: first to the first two items (or to `initial` and the first item), then to that result and the next item, and so on. Sum, product, maximum and "merge all these dictionaries" are reductions.

REDUCE(LAMBDA A, B: A + B, [1, 2, 3, 4, 5]) FOLDS THE LIST INTO A SINGLE VALUE

REDUCE() AND TRACING ITS STEPS

main.py

```
from functools import reduce
import operator

nums = [1, 2, 3, 4, 5]
print(reduce(lambda a, b: a + b, nums))
print(reduce(operator.mul, nums))                    # 5! = 120
print(reduce(lambda a, b: a if a > b else b, [4, 9, 2, 7]))
print(reduce(operator.add, [], 0))                   # initial value for empty input

def trace(acc, x):
    print(f"  acc={acc:<3} x={x}")
    return acc + x
print("total:", reduce(trace, nums, 0))

configs = [{"theme": "light"}, {"font": 14}, {"theme": "dark"}]
print(reduce(operator.or_, configs))                  # merge dicts with |
```

OUTPUT

```
15
120
9
0
  acc=0   x=1
  acc=1   x=2
  acc=3   x=3
  acc=6   x=4
  acc=10  x=5
total: 15
{'theme': 'dark', 'font': 14}
```

NOTE

Guido van Rossum moved `reduce` out of the built-ins in Python 3 because explicit loops or specialised functions are usually clearer. Prefer `sum`, `math.prod`, `max`, `min`, `any`, `all` and `"".join` when they fit.

11.3.4 Comprehensions vs map/filter

In modern Python, comprehensions are usually preferred over `map` and `filter` with lambdas because they read left to right and combine both steps. `map` remains neat when you already have a named function, such as `map(int, line.split())`.

Task	map / filter	Comprehension
Transform	`list(map(lambda x: x * 2, xs))`	`[x * 2 for x in xs]`
Select	`list(filter(lambda x: x > 0, xs))`	`[x for x in xs if x > 0]`
Both	`list(map(f, filter(g, xs)))`	`[f(x) for x in xs if g(x)]`
Named function	`list(map(int, parts))` (neat)	`[int(p) for p in parts]`
Lazy	`map(...)` is lazy	`(x * 2 for x in xs)` generator expression

SAME RESULT, TWO STYLES

main.py

```
xs = [-3, 5, 0, 8, -1, 4]
a = list(map(lambda x: x * x, filter(lambda x: x > 0, xs)))
b = [x * x for x in xs if x > 0]
print(a, b, a == b)

import timeit
setup = "xs = list(range(10_000))"
t_map = min(timeit.repeat("list(map(lambda x: x * 2, xs))", setup, number=200, repeat=5))
t_comp = min(timeit.repeat("[x * 2 for x in xs]", setup, number=200, repeat=5))
print("comprehension faster than map+lambda:", t_comp < t_map)
```

OUTPUT

```
[25, 64, 16] [25, 64, 16] True
comprehension faster than map+lambda: True
```

11.4 Closures

11.4.1 Inner functions

A function defined inside another function is an **inner** (nested) function. It exists only while the outer function runs, unless the outer function returns it. Inner functions are useful as private helpers that should not be visible at module level.

AN INNER HELPER FUNCTION

main.py

```
def print_receipt(items):
    def line(name, price):                # helper visible only inside
        return f"{name:<10}{price:>8.2f}"
    print("\n".join(line(n, p) for n, p in items))
    print(line("TOTAL", sum(p for _, p in items)))

print_receipt([("pen", 10), ("book", 250.5), ("bag", 899)])
try:
    line("x", 1)
except NameError as e:
    print("NameError:", e)
```

OUTPUT

```
pen          10.00
book        250.50
bag         899.00
TOTAL      1159.50
NameError: name 'line' is not defined
```

11.4.2 Capturing variables from the enclosing scope

When an inner function uses variables of the enclosing function and is returned, it keeps those variables alive even after the outer function has finished. The function together with the captured environment is a **closure**. Python stores the captured variables in **cells** reachable through the function's `__closure__` attribute.

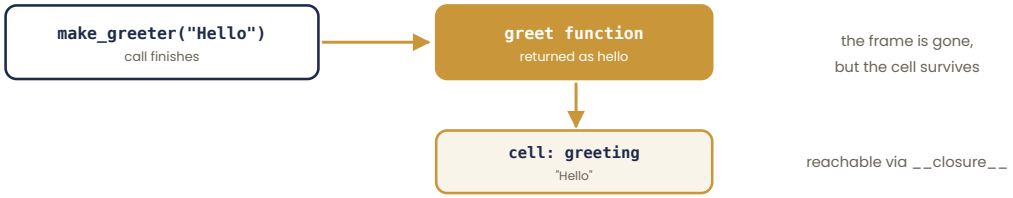
A CLOSURE REMEMBERS ITS ENVIRONMENT

main.py

```
def make_greeter(greeting):
    def greet(name):
        return f"{greeting}, {name}!"      # greeting comes from the enclosing scope
    return greet

hello = make_greeter("Hello")
namaste = make_greeter("Namaste")
print(hello("Asha"), "|", namaste("Ravi"))

print(hello.__code__.co_freevars)          # names captured
print(hello.__closure__[0].cell_contents)
print(namaste.__closure__[0].cell_contents)
```

OUTPUT

```
Hello, Asha! | Namaste, Ravi!
('greeting',)
Hello
Namaste
```



THE RETURNED FUNCTION KEEPS A REFERENCE TO THE CAPTURED VARIABLE

Closures capture **variables**, not values: if the captured variable changes later, the closure sees the new value. That is the late-binding effect seen with lambdas in loops (11.2.3). To **rebind** a captured variable from inside the closure you need `nonlocal` (10.3.3).

CLOSURES CAPTURE VARIABLES, NOT VALUES

main.py

```
def outer():
    message = "first"
    def show():
        return message
    message = "changed later"              # the closure sees this
    return show

print(outer()())
```

OUTPUT

```
changed later
```

11.4.3 Practical uses (counters, function factories)

Closures give a function **private state** without a class or a global variable, and they let you build families of related functions from one template.

COUNTERS AND FUNCTION FACTORIES

main.py

```
def make_counter(start=0, step=1):
    count = start
    def counter():
        nonlocal count
        count += step
        return count
    return counter

ticket = make_counter(100)
evens = make_counter(0, 2)
print([ticket() for _ in range(3)], [evens() for _ in range(4)])

def make_validator(min_len, must_have_digit):
    def validate(password):
        if len(password) < min_len:
            return f"too short (need {min_len})"
        if must_have_digit and not any(c.isdigit() for c in password):
            return "needs a digit"
        return "ok"
    return validate

basic = make_validator(6, False)
strict = make_validator(10, True)
for pw in ("abc", "secret", "longsecretpw", "longsecret99"):
    print(f"{pw:<14} basic: {basic(pw):<20} strict: {strict(pw)}")
```

OUTPUT

```
[101, 102, 103] [2, 4, 6, 8]
abc            basic: too short (need 6)   strict: too short (need 10)
secret         basic: ok                   strict: too short (need 10)
longsecretpw   basic: ok                   strict: needs a digit
longsecret99   basic: ok                   strict: ok
```

A RUNNING AVERAGE WITH CAPTURED STATE

main.py

```
def make_averager():
    values = []                        # mutated, not rebound: no nonlocal needed
    def add(x):
        values.append(x)
        return sum(values) / len(values)
    return add

avg = make_averager()
for mark in (80, 90, 70, 100):
    print(f"added {mark}: running average {avg(mark):.1f}")
```

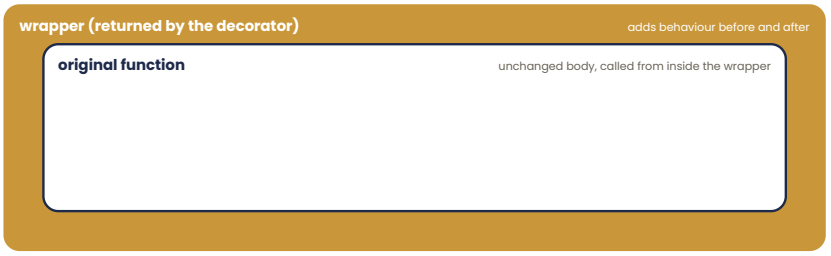
OUTPUT

```
added 80: running average 80.0
added 90: running average 85.0
added 70: running average 80.0
added 100: running average 85.0
```

11.5 Decorators

11.5.1 What a decorator is

A **decorator** is a function that takes a function and returns a (usually new) function that adds behaviour before and/or after calling the original. It lets you add timing, logging, caching, validation or access control to many functions without editing their bodies. Decorators combine the three ideas of this module: functions passed as arguments, functions returned, and closures.



A DECORATOR WRAPS THE ORIGINAL FUNCTION IN A NEW ONE

11.5.2 Writing a simple decorator

A decorator defines an inner `wrapper` function that does something extra, calls the original function captured in its closure, and returns the result. The decorator returns `wrapper`, and you rebind the original name to it.

A DECORATOR APPLIED BY HAND main.py

```
def announce(func):
    def wrapper():
        print(f"-> calling {func.__name__}")
        result = func()
        print(f"<- {func.__name__} finished")
        return result
    return wrapper

def say_hello():
    print("Hello!")

say_hello = announce(say_hello)        # decorate by hand
say_hello()
print("the name now refers to:", say_hello.__name__)
```

OUTPUT

```
-> calling say_hello
Hello!
<- say_hello finished
the name now refers to: wrapper
```

11.5.3 The @ syntax

Writing `@decorator` on the line above a `def` is exactly the same as `func = decorator(func)` immediately after the definition, only clearer, because the decoration is visible where the function is defined. The decorator runs **once, at definition time**; the wrapper runs on every call.

THE @ LINE IS SHORTHAND FOR REBINDING THE NAME

THE @ SYNTAX AND WHEN THE DECORATOR RUNS

main.py

```
def announce(func):
    print(f"[decorating {func.__name__} at definition time]")
    def wrapper():
        print(f"-> calling {func.__name__}")
        return func()
    return wrapper

@announce
def backup():
    print("backing up files")

print("--- program starts calling ---")
backup()
backup()
```

OUTPUT

```
[decorating backup at definition time]
--- program starts calling ---
-> calling backup
backing up files
-> calling backup
backing up files
```

11.5.4 Decorating functions with arguments using *args, **kwargs

The wrapper above accepts no arguments, so it fails for functions that take any. A general-purpose wrapper takes `*args, **kwargs` and forwards them unchanged, and returns whatever the original returns. This works for every function signature.

A WRAPPER THAT ACCEPTS ANY ARGUMENTS

main.py

```
def log_calls(func):
    def wrapper(*args, **kwargs):
        arg_text = ", ".join([repr(a) for a in args] + [f"{k}={v!r}" for k, v in kwargs.items()])
        print(f"{func.__name__}({arg_text})", end=" ")
        result = func(*args, **kwargs)
        print(f"-> {result!r}")
        return result
    return wrapper

@log_calls
def add(a, b):
    return a + b

@log_calls
def greet(name, greeting="Hello"):
    return f"{greeting}, {name}"

add(2, 3)
greet("Asha")
greet("Ravi", greeting="Namaste")
total = add(10, 20) * 2
print("total:", total)
```

OUTPUT

```
add(2, 3) -> 5
greet('Asha') -> 'Hello, Asha'
greet('Ravi', greeting='Namaste') -> 'Namaste, Ravi'
add(10, 20) -> 30
total: 60
```

11.5.5 functools.wraps

The wrapper replaces the original function, so by default its name, docstring and other metadata are lost: `add.__name__` becomes `"wrapper"`, `help(add)` shows nothing useful and debugging tools get confused. Decorating the wrapper with `@functools.wraps(func)` copies `__name__`, `__doc__`, `__module__`, `__qualname__` and the annotations from the original, and sets `__wrapped__` to the original function. Always use it.

PRESERVING METADATA WITH FUNCTOOLS.WRAPS main.py

```
import functools

def plain(func):
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs)
    return wrapper

def proper(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs)
    return wrapper

@plain
def area(r):
    """Return the area of a circle of radius r."""
    return 3.14159 * r * r

@proper
def volume(r):
    """Return the volume of a sphere of radius r."""
    return 4 / 3 * 3.14159 * r ** 3

print(area.__name__, "|", area.__doc__)
print(volume.__name__, "|", volume.__doc__)
print(volume.__wrapped__(1) == volume(1))
```

OUTPUT

```
wrapper | None
volume | Return the volume of a sphere of radius r.
True
```

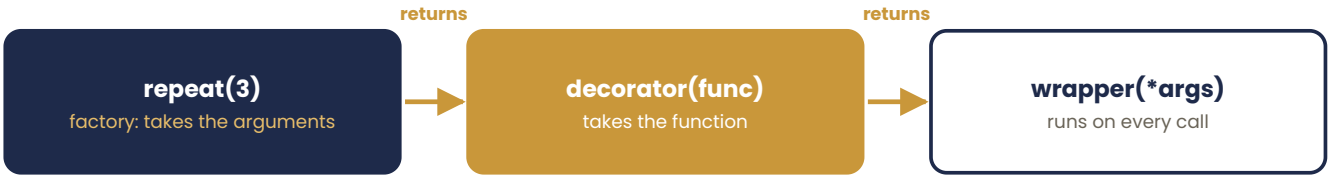
The standard template for every decorator is therefore:

THE DECORATOR TEMPLATE main.py

```
import functools

def decorator(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        # ... before ...
        result = func(*args, **kwargs)
        # ... after ...
        return result
    return wrapper
```

11.5.6 Decorators with arguments

To configure a decorator, as in `@repeat(3)` or `@retry(times=3)`, you need **one more level**: a function that takes the arguments and returns the actual decorator. `@repeat(3)` first calls `repeat(3)`, which returns a decorator, which is then applied to the function.



THREE LEVELS: ARGUMENT FACTORY, DECORATOR, WRAPPER

DECORATORS THAT TAKE ARGUMENTS

main.py

```
import functools

def repeat(times):
    def decorator(func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            result = None
            for i in range(times):
                result = func(*args, **kwargs)
            return result
        return wrapper
    return decorator

@repeat(3)
def cheer(name):
    print(f"Go {name}!")

cheer("LBF")

def tag(name, css_class=None):
    def decorator(func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            attr = f' class="{css_class}"' if css_class else ""
            return f"<{name}{attr}>{func(*args, **kwargs)}</{name}>"
        return wrapper
    return decorator

@tag("p", css_class="note")
def message(text):
    return text

print(message("Decorators can take arguments"))
```

OUTPUT

```
Go LBF!
Go LBF!
Go LBF!
<p class="note">Decorators can take arguments</p>
```

11.5.7 Stacking decorators

Several decorators can be stacked on one function. They are applied **bottom-up** (the one nearest the `def` first), so the topmost decorator becomes the outermost wrapper and runs **first** on each call. `@a @b def f` means `f = a(b(f))`.

ORDER OF STACKED DECORATORS

main.py

```
import functools

def bold(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        return "<b>" + func(*args, **kwargs) + "</b>"
    return wrapper

def italic(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        return "<i>" + func(*args, **kwargs) + "</i>"
    return wrapper

@bold
@italic
def hello():
    return "hello"

@italic
@bold
def world():
    return "world"

print(hello())        # bold(italic(hello))
print(world())        # italic(bold(world))
```

OUTPUT

```
<b><i>hello</i></b>
<i><b>world</b></i>
```

STACKED DECORATORS WRAP FROM THE BOTTOM UP: BOLD(ITALIC(HELLO))

11.5.8 Practical decorators: timer, logger, retry, access check

These four patterns appear constantly in real code. Each uses the same template.

A TIMER DECORATOR

main.py

```
import functools, time

def timer(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        result = func(*args, **kwargs)
        elapsed = time.perf_counter() - start
        print(f"[timer] {func.__name__} took {'under' if elapsed < 0.5 else 'over'} 0.5 s")
        return result
    return wrapper

@timer
def slow_sum(n):
    return sum(i * i for i in range(n))

print(slow_sum(100_000))
```

OUTPUT

```
[timer] slow_sum took under 0.5 s
333328333350000
```

A LOGGER DECORATOR

main.py

```
import functools, logging, sys

logging.basicConfig(stream=sys.stdout, level=logging.INFO, format="%(levelname)s %(message)s")

def logger(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        logging.info("calling %s args=%s kwargs=%s", func.__name__, args, kwargs)
        try:
            result = func(*args, **kwargs)
        except Exception as e:
            logging.error("%s raised %r", func.__name__, e)
            raise
        logging.info("%s returned %r", func.__name__, result)
        return result
    return wrapper

@logger
def divide(a, b):
    return a / b

divide(10, 4)
try:
    divide(1, 0)
except ZeroDivisionError:
    print("caller handled the error")
```

OUTPUT

```
INFO calling divide args=(10, 4) kwargs={}
INFO divide returned 2.5
INFO calling divide args=(1, 0) kwargs={}
ERROR divide raised ZeroDivisionError('division by zero')
caller handled the error
```

A RETRY DECORATOR WITH ARGUMENTS

main.py

```
import functools, time

def retry(times=3, delay=0.01, exceptions=(Exception,)):
    def decorator(func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            for attempt in range(1, times + 1):
                try:
                    return func(*args, **kwargs)
                except exceptions as e:
                    print(f"  attempt {attempt} failed: {e}")
                    if attempt == times:
                        raise
                    time.sleep(delay)
        return wrapper
    return decorator

responses = iter([ConnectionError("timeout"), ConnectionError("reset"), "200 OK"])

@retry(times=3, exceptions=(ConnectionError,))
def fetch():
    r = next(responses)
    if isinstance(r, Exception):
        raise r
    return r

print("result:", fetch())
```

OUTPUT

```
  attempt 1 failed: timeout
  attempt 2 failed: reset
result: 200 OK
```

AN ACCESS-CHECK DECORATOR main.py

```
import functools

current_user = {"name": "ravi", "roles": {"student"}}

def requires_role(role):
    def decorator(func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            if role not in current_user["roles"]:
                raise PermissionError(f"{current_user['name']} needs role {role!r} for
{func.__name__}")
            return func(*args, **kwargs)
        return wrapper
    return decorator

@requires_role("student")
def view_notes():
    return "notes for module 11"

@requires_role("admin")
def delete_course(course):
    return f"{course} deleted"

print(view_notes())
try:
    delete_course("Python")
except PermissionError as e:
    print("PermissionError:", e)
current_user["roles"].add("admin")
print(delete_course("Python"))
```

OUTPUT

```
notes for module 11
PermissionError: ravi needs role 'admin' for delete_course
Python deleted
```

NOTE

Web frameworks are built on exactly these ideas: Flask's `@app.route("/")` and FastAPI's `@app.get("/")` are decorators with arguments that register functions, and authentication libraries provide `@login_required` access-check decorators.

11.5.9 Class-based decorators (overview)

Anything callable can be a decorator, including a class. A class whose `__init__` receives the function and whose `__call__` method acts as the wrapper is convenient when the decorator needs to keep **state** visible as attributes, such as a call count. `functools.update_wrapper(self, func)` plays the role of `wraps`. Decorators can also be applied to classes themselves (`@dataclass`, Module 15).

A CLASS USED AS A DECORATOR main.py

```
import functools

class CountCalls:
    def __init__(self, func):
        functools.update_wrapper(self, func)
        self.func = func
        self.calls = 0

    def __call__(self, *args, **kwargs):
        self.calls += 1
        return self.func(*args, **kwargs)

@CountCalls
def ping(host):
    return f"pong from {host}"

print(ping("lbf.in"))
ping("python.org"); ping("example.com")
print(ping.__name__, "called", ping.calls, "times")
print(type(ping).__name__)
```

OUTPUT

```
pong from lbf.in
ping called 3 times
CountCalls
```

11.6 functools

11.6.1 lru_cache and cache

`@functools.lru_cache(maxsize=128)` memoizes a function: results are stored in a dictionary keyed by the arguments, and repeated calls return the stored result. When the cache is full, the **least recently used** entry is discarded. `@functools.cache` (3.9) is the same with no size limit. Arguments must be hashable. `cache_info()` reports hits and misses and `cache_clear()` empties the cache.

MEMOIZATION WITH LRU_CACHE AND CACHE

main.py

```
import functools, time

@functools.lru_cache(maxsize=None)
def fib(n):
    return n if n < 2 else fib(n - 1) + fib(n - 2)

print(fib(100))
print(fib.cache_info())

@functools.cache
def slow_square(n):
    time.sleep(0.05)              # pretend this is expensive
    return n * n

t = time.perf_counter(); slow_square(12); first = time.perf_counter() - t
t = time.perf_counter(); slow_square(12); second = time.perf_counter() - t
print("second call at least 10x faster:", second * 10 < first)

@functools.lru_cache(maxsize=2)
def square(n):
    print(f"  computing {n}")
    return n * n
for n in (1, 2, 1, 3, 2):
    square(n)
print(square.cache_info())
```

OUTPUT

```
354224848179261915075
CacheInfo(hits=98, misses=101, maxsize=None, currsize=101)
second call at least 10x faster: True
  computing 1
  computing 2
  computing 3
  computing 2
CacheInfo(hits=1, misses=4, maxsize=2, currsize=2)
```

WATCH OUT

Only cache **pure** functions whose result depends solely on their arguments. Caching a function that reads the clock, a file or a database returns stale data. Unbounded caches can also grow without limit in long-running programs.

11.6.2 partial

`functools.partial(func, *args, **kwargs)` creates a new callable with some arguments of `func` already filled in. It is a quick alternative to writing a small wrapper function or lambda, and keeps the original function introspectable via `.func`, `.args` and `.keywords`.

PRE-FILLING ARGUMENTS WITH PARTIAL

main.py

```
from functools import partial

def power(base, exp):
    return base ** exp

square = partial(power, exp=2)
cube = partial(power, exp=3)
two_to = partial(power, 2)
print(square(9), cube(3), two_to(10))

parse_binary = partial(int, base=2)
print(parse_binary("101101"))

log_error = partial(print, "[ERROR]", sep=" | ")
log_error("disk full", "retry later")

print(square.func.__name__, square.args, square.keywords)
```

OUTPUT

```
81 27 1024
45
[ERROR] | disk full | retry later
power () {'exp': 2}
```

11.6.3 singledispatch (overview)

`@functools.singledispatch` turns a function into a **generic function** that chooses an implementation based on the **type of its first argument**. You register implementations with `@func.register`; type hints on the first parameter select the type. It replaces chains of `isinstance` checks and lets other modules add support for new types.

CHOOSING AN IMPLEMENTATION BY TYPE

main.py

```
from functools import singledispatch

@singledispatch
def describe(value):
    return f"something: {value!r}"

@describe.register
def _(value: int):
    return f"an integer with {len(str(abs(value)))} digits"

@describe.register
def _(value: list):
    return f"a list of {len(value)} items"

@describe.register(str)
def _(value):
    return f"a string of length {len(value)}"

for v in (12345, [1, 2, 3], "hello", 3.5, True):
    print(f"{v!r:<10} -> {describe(v)}")
```

OUTPUT

```
12345      -> an integer with 5 digits
[1, 2, 3]  -> a list of 3 items
'hello'    -> a string of length 5
3.5        -> something: 3.5
True       -> an integer with 1 digits
```

Note that `True` was handled by the `int` implementation, because `bool` is a subclass of `int`. For methods inside classes, use `functools.singledispatchmethod`.

Functions are first-class objects: they can be assigned, passed, returned and stored (dispatch tables). Lambdas are one-expression anonymous functions, ideal for key= arguments but not for named or complex logic, and they bind loop variables late. map applies a function lazily, filter keeps truthy results, reduce folds items into one value; comprehensions usually read better. An inner function that captures enclosing variables is a closure, giving private state and function factories (use nonlocal to rebind). A decorator takes a function and returns a wrapper: @dec means f = dec(f) at definition time; use *args, **kwargs and functools.wraps in every wrapper; decorators with arguments add a third level; stacked decorators apply bottom-up; classes with __call__ can be decorators. functools provides lru_cache and cache for memoizing pure functions, partial for pre-filling arguments and singledispatch for type-based dispatch.

PRACTICE CHECKPOINT

Write a **@timer** decorator, a **@retry(times=3)** decorator that takes arguments, and speed up Fibonacci with **@lru_cache**.

PART 1: @TIMER AND @LRU_CACHE

main.py

```
import functools, time

def timer(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        result = func(*args, **kwargs)
        ms = (time.perf_counter() - start) * 1000
        wrapper.last_ms = ms
        print(f"[timer] {func.__name__}{args} finished")
        return result
    wrapper.last_ms = 0.0
    return wrapper

def fib_plain(n):
    return n if n < 2 else fib_plain(n - 1) + fib_plain(n - 2)

@functools.lru_cache(maxsize=None)
def fib_cached(n):
    return n if n < 2 else fib_cached(n - 1) + fib_cached(n - 2)

@timer
def run_plain(n):
    return fib_plain(n)

@timer
def run_cached(n):
    return fib_cached(n)

a = run_plain(27); slow = run_plain.last_ms
b = run_cached(27); fast = run_cached.last_ms
print("same answer:", a == b, a)
print("lru_cache version at least 100x faster:", fast * 100 < slow)
print(fib_cached.cache_info())
```

OUTPUT

```
[timer] run_plain(27,) finished
[timer] run_cached(27,) finished
same answer: True 196418
lru_cache version at least 100x faster: True
CacheInfo(hits=25, misses=28, maxsize=None, currsize=28)
```

PART 2: @RETRY(TIMES=3)

main.py

```
import functools, random

def retry(times=3):
    def decorator(func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            last_error = None
            for attempt in range(1, times + 1):
                try:
                    result = func(*args, **kwargs)
                    print(f"  succeeded on attempt {attempt}")
                    return result
                except Exception as e:
                    last_error = e
                    print(f"  attempt {attempt}/{times} failed: {e}")
            raise RuntimeError(f"{func.__name__} failed after {times} attempts") from last_error
        return wrapper
    return decorator

random.seed(46)

@retry(times=3)
def unreliable_upload(filename):
    if random.random() < 0.6:
        raise ConnectionError("network glitch")
    return f"{filename} uploaded"

for name in ("notes.pdf", "slides.pptx", "video.mp4"):
    print(name)
    try:
        print("  ->", unreliable_upload(name))
    except RuntimeError as e:
        print("  ->", e)
```

OUTPUT

```
notes.pdf
  succeeded on attempt 1
  -> notes.pdf uploaded
slides.pptx
  attempt 1/3 failed: network glitch
  attempt 2/3 failed: network glitch
  succeeded on attempt 3
  -> slides.pptx uploaded
video.mp4
  attempt 1/3 failed: network glitch
  attempt 2/3 failed: network glitch
  attempt 3/3 failed: network glitch
  -> unreliable_upload failed after 3 attempts
```

MODULE 12 · INTERMEDIATE

Modules, packages and virtual environments

Minimum duration **2 days** | Level **Intermediate**

Modules and packages organise code into files and folders, so that programs can grow without becoming one enormous script. pip and virtual environments manage third-party libraries for each project separately. This module covers what a module is and every form of import, writing your own modules, the `__name__ == "__main__"` idiom, how Python finds modules (sys.path), module attributes and reloading, packages, `__init__.py`, absolute and relative imports, sub-packages and `__all__`, a quick tour of the standard library, PyPI and pip, requirements files and version specifiers, and virtual environments with venv, plus the modern tools uv, Poetry and conda.

IN THIS MODULE

12.1 Modules
12.2 Packages
12.3 Standard library tour
12.4 Installing packages with pip
12.5 Virtual environments

12.1 Modules

12.1.1 What a module is

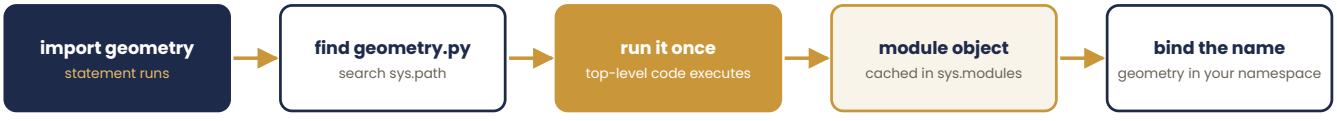
A **module** is simply a file containing Python code, with the extension `.py`. Its name is the file name without the extension: the file `geometry.py` is the module `geometry`. Any functions, classes and variables defined at the top level of the file become the module's **attributes**, which other files can use by importing it. Modules give you:

Organisation
Related code lives together: all the file-handling helpers in one module, all the maths in another.

Reuse
Write a function once and import it from every script that needs it.

Namespaces
math.sqrt and numpy.sqrt can coexist: each module has its own namespace, so names do not clash.

Python comes with hundreds of modules in its **standard library** ("batteries included"), and hundreds of thousands more are available from PyPI (12.4). Modules can also be written in C (such as `math`) or be whole folders, called packages (12.2).



WHAT HAPPENS ON THE FIRST IMPORT OF A MODULE

A module's code runs only on the **first** import in a program. The resulting module object is cached in `sys.modules`; later imports, even from other files, reuse it without running the code again.

12.1.2 import module, from module import name, import ... as ...

There are four common forms of the `import` statement:

Form	What it binds	Use as
`import math`	the module name	`math.sqrt(16)`
`import math as m`	the module under an alias	`m.sqrt(16)`
`from math import sqrt, pi`	specific names from the module	`sqrt(16)`
`from math import sqrt as root`	a name under an alias	`root(16)`
`from math import *`	all public names (avoid)	`sqrt(16)`

THE FORMS OF IMPORT main.py

```
import math
print(math.sqrt(16), math.pi)

import statistics as stats
print(stats.mean([91, 78, 85]))

from random import choice, seed
seed(1)
print(choice(["Asha", "Ravi", "Meera"]))

from datetime import date as Date
print(Date(2026, 1, 26).strftime("%d %b %Y"))

import os.path
print(os.path.join("notes", "module12.txt"))
```

OUTPUT

```
4.0 3.141592653589793
84.66666666666667
Asha
26 Jan 2026
notes/module12.txt
```

Well-known aliases are a community convention: `import numpy as np`, `import pandas as pd`, `import matplotlib.pyplot as plt`. Avoid `from module import *` in real code: it dumps unknown names into your namespace, can silently overwrite your own names, and hides where each name came from.

WHY IMPORT * IS RISKY main.py

```
from math import *
print(pow(2, 3))          # math.pow returns a float: the built-in pow was replaced!
del pow
print(pow(2, 3))          # the built-in again
```

OUTPUT

```
8.0
8
```

TIP

PEP 8 order for imports at the top of a file: standard library imports, then third-party imports, then your own modules, each group separated by a blank line. Tools such as ruff and isort sort them automatically.

12.1.3 Creating your own module

To create a module, put functions and variables in a `.py` file. Any script in the same folder can import it by name. Below, `geometry.py` is a module and `main.py` uses it; both files sit in one folder.

GEOMETRY.PY Python

```
"""Simple geometry helpers for LBF examples."""
import math

PI = math.pi

def circle_area(r):
    """Return the area of a circle of radius r."""
    return PI * r * r

def rectangle_area(length, width):
    return length * width

print(f"[geometry module loaded: {__name__}]")
```

MAIN.PY: USING YOUR OWN MODULE main.py

```
import geometry
from geometry import rectangle_area
import geometry as g          # same cached module object

print(round(geometry.circle_area(2), 2))
print(rectangle_area(4, 5))
print(g is geometry, geometry.PI)
print(geometry.__doc__)
```

OUTPUT

```
[geometry module loaded: geometry]
12.57
20
True 3.141592653589793
Simple geometry helpers for LBF examples.
```

Note that the message inside `geometry.py` printed **once**, even though the module was imported three times, and that inside the module `__name__` was `"geometry"`. Python also writes a compiled version of the module to a `__pycache__` folder (`geometry.cpython-314.pyc`) to speed up the next start; you can ignore it and keep it out of version control.

12.1.4 if __name__ == "__main__":

Every module has a built-in variable `__name__`. When a file is **run directly** (`python geometry.py`), Python sets its `__name__` to `"__main__"`; when the file is **imported**, `__name__` is the module's name. Guarding code with `if __name__ == "__main__":` lets one file work both as an importable module and as a runnable script: demo or test code under the guard runs only when the file is executed directly.

TEMPERATURE.PY Python

```python
def c_to_f(c):
    return c * 9 / 5 + 32

def f_to_c(f):
    return (f - 32) * 5 / 9

def main():
    for c in (0, 37, 100):
        print(f"{c} C = {c_to_f(c):.1f} F")

if __name__ == "__main__":
    print("running temperature.py directly")
    main()
```

RUN DIRECTLY VS IMPORTED main.py

```
python temperature.py
python -c "import temperature; print('imported, __name__ =', temperature.__name__);
print(temperature.c_to_f(25))"
```

TERMINAL

```
$ python temperature.py
running temperature.py directly
0 C = 32.0 F
37 C = 98.6 F
100 C = 212.0 F
$ python -c "import temperature; print('imported, __name__ =', temperature.__name__);
print(temperature.c_to_f(25))"
imported, __name__ = temperature
77.0
```

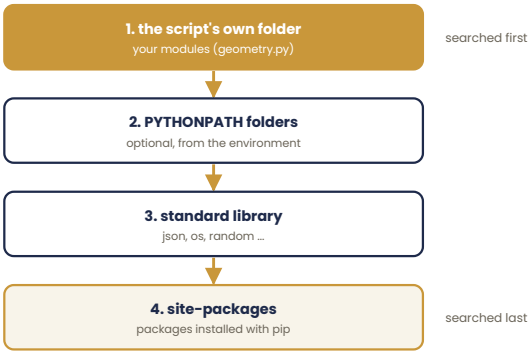
THE __MAIN__ GUARD MAKES A FILE WORK BOTH WAYS

TIP

Put the script logic in a `main()` function and call it under the guard. This keeps global variables out of the module and makes the logic testable.

12.1.5 Module search path (sys.path)

When you import a module, Python looks for it in the folders listed in `sys.path`, in order, and uses the first match. The list starts with the folder of the script being run (or the current folder in the REPL), followed by folders from the `PYTHONPATH` environment variable, the standard library and finally `site-packages`, where pip installs third-party packages. Built-in modules compiled into the interpreter, such as `sys`, are found before the path is searched.



THE ORDER OF THE MODULE SEARCH PATH

INSPECTING THE SEARCH PATH

main.py

```
import sys, os

print("first entry is the script folder:", sys.path[0] == os.path.dirname(os.path.abspath(__file__)))
print("entries on the path:", len(sys.path))
print("site-packages on the path:", any(p.endswith("site-packages") for p in sys.path))
print("json found at:", __import__("json").__file__.split("lib/")[-1])
print("sys is built in:", "sys" in sys.builtin_module_names)
try:
    import no_such_module
except ModuleNotFoundError as e:
    print("ModuleNotFoundError:", e)
```

OUTPUT

```
first entry is the script folder: True
entries on the path: 5
site-packages on the path: True
json found at: python3.14/json/__init__.py
sys is built in: True
ModuleNotFoundError: No module named 'no_such_module'
```

WATCH OUT

Because the script's folder comes first, a file of yours named like a standard module **shadows** it. A file called `random.py` in your project makes `import random` load your file, typically producing confusing errors such as *module 'random' has no attribute 'randint'*. Never name your files `random.py`, `math.py`, `json.py`, `email.py` or similar.

RANDOM.PY

Python

```
# a student's practice file, unluckily named
print("this is MY random.py, not the standard library!")
```

SHADOWING A STANDARD MODULE main.py

```
import random
try:
    print(random.randint(1, 6))
except AttributeError as e:
    print("AttributeError:", e)
```

OUTPUT

```
this is MY random.py, not the standard library!
AttributeError: module 'random' has no attribute 'randint' (consider renaming 'random.py' since it has the same name as the standard library module named 'random' and prevents importing that standard library module)
```

You can add a folder to the search path at run time with `sys.path.append(folder)`, but the clean solutions are to organise code as a package, set `PYTHONPATH`, or install your project in a virtual environment.

12.1.6 dir() and module attributes

`dir(module)` lists the names a module defines. Every module also has special attributes set by Python: `__name__` (its name), `__doc__` (its docstring), `__file__` (the path it was loaded from; built-in modules have none), `__package__` and `__spec__` (import details). By convention, names that start with an underscore are private to the module.

EXPLORING A MODULE WITH DIR() main.py

```
import geometry, math

print([n for n in dir(geometry) if not n.startswith("__")])
print(geometry.__name__, "|", geometry.__doc__, "|", geometry.__file__)
print(hasattr(math, "sqrt"), getattr(geometry, "PI"))
print(len([n for n in dir(math) if not n.startswith("_")]), "public names in math")
print(hasattr(math, "__file__") or "math is built into this interpreter")
```

OUTPUT

```
['PI', '_PRECISION', 'circle_area', 'math']
geometry | Simple geometry helpers. | geometry.py
True 3.141592653589793
62 public names in math
math is built into this interpreter
```

12.1.7 Reloading with importlib.reload

Because modules are cached, editing a module file while a program (or REPL session) is running has no effect on later `import` statements. `importlib.reload(module)` re-executes the module's code and updates the existing module object. It is mainly a convenience for interactive work; objects created from the old code (such as instances of old classes, or names copied with `from module import name`) are not updated.

IMPORT IS CACHED; RELOAD RE-RUNS THE MODULE main.py

```
import importlib, pathlib, sys
sys.dont_write_bytecode = True

pathlib.Path("settings.py").write_text('THEME = "light"\n')
import settings
print("first import:", settings.THEME)

pathlib.Path("settings.py").write_text('THEME = "dark"\n')     # edit the file
import settings                                                # cached: no change
print("import again:", settings.THEME)

importlib.reload(settings)
print("after reload:", settings.THEME)
print("cached in sys.modules:", "settings" in sys.modules)
```

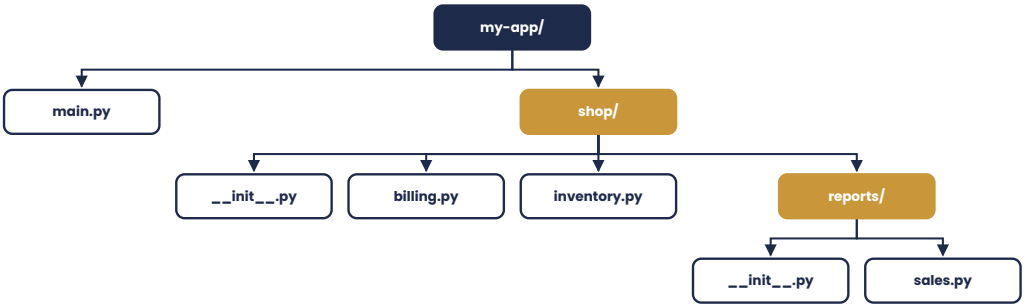
OUTPUT

```
first import: light
import again: light
after reload: dark
cached in sys.modules: True
```

12.2 Packages

12.2.1 Package folders and __init__.py

A **package** is a folder of modules, which lets you organise a larger code base as a hierarchy and import with dotted names such as `shop.billing`. A folder becomes a regular package by containing an `__init__.py` file. That file runs when the package is first imported and can be empty, or can set up package-level names. (Folders without `__init__.py` are "namespace packages", an advanced feature for splitting one package across several locations; always add `__init__.py` in your own projects.)



A PROJECT WITH THE PACKAGE SHOP AND THE SUB-PACKAGE SHOP.REPORTS

SHOP/__INIT__.PY Python

```
"""The shop package."""
print("[shop package initialised]")
VERSION = "1.0"
```

SHOP/BILLING.PY Python

```
GST_RATE = 0.18

def total_with_gst(amount):
    return round(amount * (1 + GST_RATE), 2)
```

SHOP/INVENTORY.PY Python

```python
stock = {"pen": 100, "book": 40, "bag": 12}

def in_stock(item):
    return stock.get(item, 0) > 0
```

MAIN.PY: IMPORTING FROM A PACKAGE main.py

```python
import shop.billing
from shop import inventory
from shop.billing import total_with_gst

print(shop.VERSION)
print(shop.billing.total_with_gst(1000))
print(total_with_gst(250))
print(inventory.in_stock("bag"), inventory.in_stock("ruler"))
```

OUTPUT

```
[shop package initialised]
1.0
1180.0
295.0
True False
```

The package's `__init__.py` ran once, when `shop` was first imported. A common use of `__init__.py` is to **re-export** the most important names so users can write `from shop import total_with_gst` without knowing which file it lives in.

12.2.2 Absolute vs relative imports

Inside a package, modules can import each other in two ways. An **absolute import** gives the full path from the project root: `from shop.billing import GST_RATE`. A **relative import** starts with dots that refer to the current package: `from .billing import GST_RATE` (same package) or `from ..billing import GST_RATE` (parent package). PEP 8 recommends absolute imports for clarity; relative imports are acceptable inside large packages and keep working if the package is renamed.

Import inside shop/inventory.py	Meaning
`from shop.billing import GST_RATE`	absolute: from the top-level package
`from .billing import GST_RATE`	relative: billing in the same package
`from . import billing`	relative: the billing module of this package
`from ..utils import helper`	relative: utils in the parent package

SHOP/__INIT__.PY Python

```python
"""The shop package: re-export the main helpers."""
from .billing import total_with_gst
from .inventory import in_stock, restock
```

SHOP/INVENTORY.PY

Python

```
from .billing import GST_RATE          # relative import
from shop import billing              # absolute import

stock = {"pen": 100, "book": 40, "bag": 0}

def in_stock(item):
    return stock.get(item, 0) > 0

def restock(item, qty, unit_price):
    stock[item] = stock.get(item, 0) + qty
    cost = billing.total_with_gst(qty * unit_price)
    return f"restocked {qty} x {item}, cost with {GST_RATE:.0%} GST: Rs {cost}"
```

RELATIVE IMPORTS INSIDE A PACKAGE

main.py

```
from shop import in_stock, restock     # re-exported by __init__.py

print(in_stock("bag"))
print(restock("bag", 10, 450))
print(in_stock("bag"))
```

OUTPUT

```
False
restocked 10 x bag, cost with 18% GST: Rs 5310.0
True
```

Relative imports work only inside a package. Running a module that uses them directly as a script (`python shop/check.py`) fails with *attempted relative import with no known parent package*. Run it as a module from the project root instead: `python -m shop.check`.

SHOP/CHECK.PY

Python

```
from .billing import GST_RATE     # relative import

print(f"GST rate is {GST_RATE:.0%}")
```

RUNNING A PACKAGE MODULE AS A SCRIPT VS WITH -M

main.py

```
python shop/check.py 2>&1 | tail -1
python -m shop.check
```

TERMINAL

```
$ python shop/check.py 2>&1 | tail -1
ImportError: attempted relative import with no known parent package
$ python -m shop.check
GST rate is 18%
```

12.2.3 Sub-packages

Packages can contain other packages, each with its own `__init__.py`, to any depth. You import them with longer dotted paths, and relative imports use `..` to reach the parent package.

SHOP/REPORTS/SALES.PY

Python

```
from ..billing import total_with_gst     # parent package
from ..inventory import stock

def daily_summary(sales):
    lines = []
    for item, qty in sales.items():
        lines.append(f"{item:<6}{qty:>3} sold, {stock[item] - qty:>3} left")
    gross = sum(qty * 50 for qty in sales.values())
    lines.append(f"gross with GST: Rs {total_with_gst(gross)}")
    return "\n".join(lines)
```

A SUB-PACKAGE

main.py

```
from shop.reports.sales import daily_summary
import shop.reports

print(daily_summary({"pen": 30, "book": 5}))
print(shop.reports.__name__)
```

OUTPUT

```
[shop package initialised]
pen    30 sold,  70 left
book    5 sold,  35 left
gross with GST: Rs 2065.0
shop.reports
```

12.2.4 __all__

A module can define a list called `__all__` naming its public API. It controls what `from module import *` imports, and it documents for readers and tools which names are meant to be used. Names not listed can still be imported explicitly.

TEXTUTILS.PY

Python

```
__all__ = ["slugify", "word_count"]

def slugify(text):
    return "-".join(_clean(text).split())

def word_count(text):
    return len(text.split())

def _clean(text):
    return "".join(c for c in text.lower() if c.isalnum() or c == " ")

def debug_dump(text):
    print(repr(text))
```

__ALL__ CONTROLS IMPORT *

main.py

```
from textutils import *

print(slugify("Learn Being Forward: Python!"))
print(word_count("modules and packages"))
for name in ("debug_dump", "_clean"):
    print(name, "imported by *:", name in globals())

from textutils import debug_dump        # still importable explicitly
debug_dump("ok")
```

OUTPUT

```
learn-being-forward-python
3
debug_dump imported by *: False
_clean imported by *: False
'ok'
```

12.3 Standard library tour

The standard library is installed with Python itself, so these modules are always available with no pip install. Here is a first look at the ones you will use most; each is covered in depth in Module 18.

12.3.1 math, random, statistics

MATH, RANDOM AND STATISTICS

main.py

```
import math, random, statistics

print(math.sqrt(2), math.ceil(4.1), math.floor(4.9), math.factorial(6), math.gcd(48, 36))
print(math.isclose(0.1 + 0.2, 0.3), round(math.dist((0, 0), (3, 4)), 1), math.comb(5, 2))

random.seed(42)                         # repeatable "random" numbers
print(random.randint(1, 6), random.choice("ABCD"), round(random.random(), 3))
deck = list(range(1, 11)); random.shuffle(deck); print(deck, random.sample(deck, 3))

marks = [91, 78, 85, 64, 99, 85]
print(statistics.mean(marks), statistics.median(marks), statistics.mode(marks))
print(round(statistics.stdev(marks), 2))
```

OUTPUT

```
1.4142135623730951 5 4 720 12
True 5.0 10
6 A 0.025
[8, 7, 3, 10, 1, 6, 2, 9, 4, 5] [8, 5, 7]
83.66666666666667 85.0 85
11.93
```

12.3.2 datetime, time, calendar

DATETIME, TIME AND CALENDAR

main.py

```
from datetime import date, datetime, timedelta
import time, calendar

d = date(2026, 1, 26)
print(d, d.strftime("%A, %d %B %Y"), d.weekday())
print(d + timedelta(days=100))
print((date(2026, 12, 25) - d).days, "days from Republic Day to Christmas")

dt = datetime(2026, 8, 15, 9, 30)
print(dt.isoformat(), dt.strftime("%I:%M %p"))
print(datetime.strptime("15/08/2026", "%d/%m/%Y").date())

start = time.perf_counter(); time.sleep(0.1)
print("slept at least 0.1 s:", time.perf_counter() - start >= 0.1)

print(calendar.isleap(2028), calendar.monthrange(2026, 2))
print(calendar.month(2026, 2))
```

OUTPUT

```
2026-01-26 Monday, 26 January 2026 0
2026-05-06
333 days from Republic Day to Christmas
2026-08-15T09:30:00 09:30 AM
2026-08-15
slept at least 0.1 s: True
True (calendar.SUNDAY, 28)
   February 2026
Mo Tu We Th Fr Sa Su
                   1
 2  3  4  5  6  7  8
 9 10 11 12 13 14 15
16 17 18 19 20 21 22
23 24 25 26 27 28
```

12.3.3 os, sys, platform

OS, SYS AND PLATFORM

main.py

```
import os, sys, platform

os.makedirs("reports/2026", exist_ok=True)
with open("reports/2026/jan.txt", "w") as f:
    f.write("sales")
print(os.listdir("reports"), os.path.exists("reports/2026/jan.txt"))
print(os.path.splitext("photo.final.jpg"), os.path.basename("/data/in/marks.csv"))
print("HOME set:", "HOME" in os.environ, "| cwd ends with:", os.path.basename(os.getcwd()) != "")

print(sys.version_info[:2] >= (3, 10), sys.platform, type(sys.argv).__name__)
print(platform.system(), platform.machine(), platform.python_implementation())
```

OUTPUT

```
['2026'] True
('photo.final', '.jpg') marks.csv
HOME set: True | cwd ends with: True
True linux list
Linux x86_64 CPython
```

12.3.4 json, csv

JSON AND CSV

main.py

```
import json, csv

student = {"name": "Asha", "marks": [91, 88], "active": True, "mentor": None}
text = json.dumps(student)
print(text)
print(json.loads(text)["marks"][0])
with open("student.json", "w") as f:
    json.dump(student, f, indent=2)
print(open("student.json").read())

with open("marks.csv", "w", newline="") as f:
    writer = csv.writer(f)
    writer.writerow(["name", "marks"])
    writer.writerows([["Asha", 91], ["Ravi", 78]])
with open("marks.csv", newline="") as f:
    for row in csv.DictReader(f):
        print(row["name"], int(row["marks"]) + 5)
```

OUTPUT

```
{"name": "Asha", "marks": [91, 88], "active": true, "mentor": null}
91
{
  "name": "Asha",
  "marks": [
    91,
    88
  ],
  "active": true,
  "mentor": null
}
Asha 96
Ravi 83
```

12.3.5 Each module above is covered in depth in Module 18

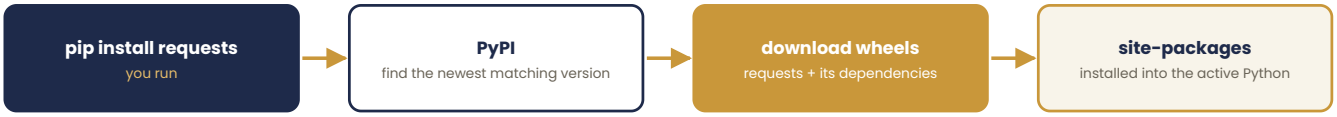
Module 18 (Standard library essentials) returns to all of these and adds `pathlib`, `collections`, `itertools`, `re`, `logging`, `argparse`, `subprocess` and more. Before reaching for a third-party package, check whether the standard library already does the job; the documentation at docs.python.org/3/library lists every module.

Need	Standard module
Maths, randomness, statistics	`math`, `random`, `statistics`, `decimal`, `fractions`
Dates and times	`datetime`, `time`, `calendar`, `zoneinfo`
Files and the system	`os`, `pathlib`, `shutil`, `sys`, `platform`, `subprocess`
Data formats	`json`, `csv`, `sqlite3`, `tomllib`, `zipfile`
Text	`re`, `string`, `textwrap`, `difflib`
Internet	`urllib`, `http.server`, `email`, `smtplib`

12.4 Installing packages with pip

12.4.1 PyPI

The **Python Package Index** (PyPI, pypi.org) is the official public repository of third-party Python packages, with more than half a million projects: `requests` for HTTP, `numpy` and `pandas` for data, `flask`, `django` and `fastapi` for web development, `pytest` for testing, and so on. Packages are distributed as **wheels** (`.whl`, ready-to-install archives) or source distributions. **pip** is the standard installer that downloads packages from PyPI together with their dependencies.



HOW PIP INSTALLS A PACKAGE

WATCH OUT

Anyone can publish to PyPI. Check a package's popularity, maintenance and spelling before installing it: attackers publish look-alike names ("typosquatting") such as `reqeusts`.

12.4.2 pip install, uninstall, list, show, freeze

Always run pip through the Python you mean to use: `python -m pip ...` (or `py -m pip` on Windows). Inside an activated virtual environment, plain `pip` is the same thing. The main commands:

Command	What it does
`pip install requests`	install the latest version and its dependencies
`pip install requests==2.33.1`	install an exact version
`pip install -U requests`	upgrade to the newest version
`pip install -r requirements.txt`	install everything listed in a file
`pip uninstall requests`	remove a package (`-y` skips the question)
`pip list`	list installed packages (`--outdated` shows newer versions)
`pip show requests`	details: version, location, dependencies
`pip freeze`	installed packages in requirements format

INSTALLING A PACKAGE FROM PYPI

main.py

```
python -m pip install requests
```

TERMINAL

```
Collecting requests
  Downloading requests-2.33.1-py3-none-any.whl (64 kB)
Collecting charset_normalizer<4,>=2 (from requests)
  Downloading charset_normalizer-3.4.4-py3-none-any.whl (53 kB)
Collecting idna<4,>=2.5 (from requests)
  Downloading idna-3.11-py3-none-any.whl (71 kB)
Collecting urllib3<3,>=1.21.1 (from requests)
  Downloading urllib3-2.6.3-py3-none-any.whl (131 kB)
Collecting certifi>=2017.4.17 (from requests)
  Downloading certifi-2026.1.4-py3-none-any.whl (152 kB)
Installing collected packages: urllib3, idna, charset_normalizer, certifi, requests
Successfully installed certifi-2026.1.4 charset_normalizer-3.4.4 idna-3.11 requests-2.33.1 urllib3-
2.6.3
```

Typical output; the exact versions depend on the day you install.

The commands below were really run inside a fresh virtual environment. Because this book was built on a machine without internet access, pip was configured (in its `pip.conf`) to install from a local folder of wheel files instead of PyPI; that is why it prints *Looking in links*. Everything else is exactly what you will see.

PIP INSTALL, LIST, SHOW, FREEZE AND UNINSTALL

main.py

```
python -m venv .venv
.venv/bin/pip list
.venv/bin/pip install tabulate
.venv/bin/pip show tabulate
.venv/bin/pip freeze
.venv/bin/python -c "from tabulate import tabulate; print(tabulate([['Asha', 91], ['Ravi', 78]],
headers=['name', 'marks']))"
.venv/bin/pip uninstall -y tabulate
.venv/bin/pip list
```

TERMINAL

```
$ python -m venv .venv
$ .venv/bin/pip list
Package Version
------- -------
pip     25.2
$ .venv/bin/pip install tabulate
Looking in links: ~/wheels
Processing ~/wheels/tabulate-0.10.0-py3-none-any.whl
Installing collected packages: tabulate
Successfully installed tabulate-0.10.0
$ .venv/bin/pip show tabulate
Name: tabulate
Version: 0.10.0
Summary: Pretty-print tabular data
Home-page: https://github.com/astanin/python-tabulate
Author:
Author-email: Sergey Astanin <s.astanin@gmail.com>
License-Expression: MIT
Location: .venv/lib/python3.14/site-packages
Requires:
Required-by:
$ .venv/bin/pip freeze
tabulate==0.10.0
$ .venv/bin/python -c "from tabulate import tabulate; print(tabulate([['Asha', 91], ['Ravi', 78]],
headers=['name', 'marks']))"
name      marks
------  -------
Asha         91
Ravi         78
$ .venv/bin/pip uninstall -y tabulate
Found existing installation: tabulate 0.10.0
Uninstalling tabulate-0.10.0:
  Successfully uninstalled tabulate-0.10.0
$ .venv/bin/pip list
Package Version
------- -------
pip     25.2
```

12.4.3 requirements.txt

A **requirements file** lists the packages a project needs, one per line, so anyone can recreate the environment with a single command: `pip install -r requirements.txt`. It is committed to version control together with the code. `pip freeze > requirements.txt` writes the exact versions currently installed, which is a simple way to **pin** them.

REQUIREMENTS.TXT text

```
# third-party packages for the report tool
tabulate>=0.9
```

INSTALLING FROM A REQUIREMENTS FILE AND PINNING main.py

```
python -m venv .venv
.venv/bin/pip install -r requirements.txt
.venv/bin/pip freeze > requirements.lock.txt
cat requirements.lock.txt
```

TERMINAL

```
$ python -m venv .venv
$ .venv/bin/pip install -r requirements.txt
Looking in links: ~/wheels
Processing ~/wheels/tabulate-0.10.0-py3-none-any.whl (from -r requirements.txt (line 2))
Installing collected packages: tabulate
Successfully installed tabulate-0.10.0
$ .venv/bin/pip freeze > requirements.lock.txt
$ cat requirements.lock.txt
tabulate==0.10.0
```

NOTE

Many teams keep two files: a short, hand-written list of direct dependencies with loose version ranges (`requirements.in` or `pyproject.toml`), and a generated "lock" file with exact versions of everything, produced by `pip freeze`, `pip-tools` or `uv`, so that every machine installs identical versions.

12.4.4 Version specifiers

Lines in a requirements file (and dependencies in `pyproject.toml`) can constrain which versions are acceptable. Versions normally follow **semantic versioning**, MAJOR.MINOR.PATCH: a major bump may break your code, a minor one adds features, a patch only fixes bugs.

Specifier	Meaning	Example matches
`requests`	any version (the newest)	everything
`requests==2.33.1`	exactly this version	2.33.1
`requests>=2.30`	this version or newer	2.30, 2.33.1, 3.0
`requests>=2.30,<3`	a range	2.30 to 2.x
`requests~=2.33.0`	compatible release: ==2.33.*	2.33.0, 2.33.5
`requests~=2.33`	compatible release: >=2.33, ==2.*	2.33, 2.40
`requests!=2.31.0`	anything except this version	all but 2.31.0
`pywin32; sys_platform == "win32"`	environment marker	only installed on Windows

CHECKING SPECIFIERS WITH THE PACKAGING LIBRARY main.py

```python
from packaging.specifiers import SpecifierSet
from packaging.version import Version

versions = ["2.29.0", "2.30.0", "2.31.0", "2.33.1", "2.40.0", "3.0.0"]
for spec in ("==2.33.1", ">=2.30,<3", "~=2.33.0", "~=2.33", "!=2.31.0,>=2.30"):
    ok = [v for v in versions if Version(v) in SpecifierSet(spec)]
    print(f"{spec:<17} -> {ok}")
```

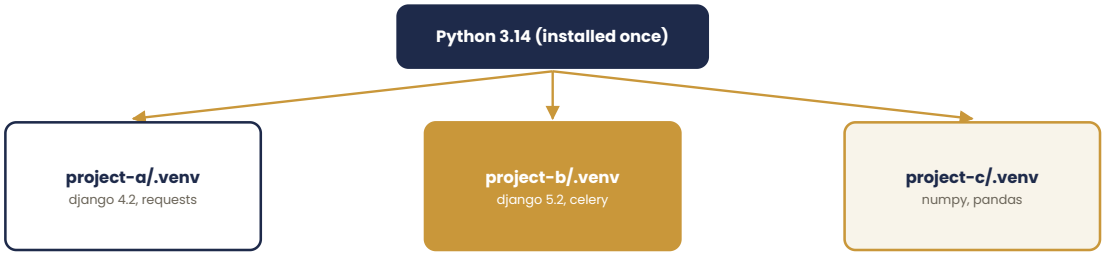
OUTPUT

```
==2.33.1          -> ['2.33.1']
>=2.30,<3         -> ['2.30.0', '2.31.0', '2.33.1', '2.40.0']
~=2.33.0          -> ['2.33.1']
~=2.33            -> ['2.33.1', '2.40.0']
!=2.31.0,>=2.30   -> ['2.30.0', '2.33.1', '2.40.0', '3.0.0']
```

12.5 Virtual environments

12.5.1 Why virtual environments

Without virtual environments, every project on your machine shares one `site-packages` folder. Project A needs Django 4, project B needs Django 5, and installing one breaks the other. Installing into the system Python can even break operating-system tools, which is why modern Linux distributions refuse `pip install` outside a virtual environment ("externally-managed-environment"). A **virtual environment** is a self-contained folder with its own Python executable link and its own `site-packages`, created per project.



ONE PYTHON INSTALLATION, A SEPARATE SET OF PACKAGES PER PROJECT

Isolation
Each project has its own package versions; upgrading one cannot break another.

Reproducibility
A requirements file plus a fresh venv recreates the exact environment on any machine.

Clean system
Nothing is installed into the system Python, and deleting the .venv folder removes everything.

12.5.2 python -m venv and activation

The standard library module `venv` creates virtual environments. By convention the folder is called `.venv` and lives in the project root (add it to `.gitignore`; never commit it). **Activating** the environment adjusts your shell's `PATH` so that `python` and `pip` refer to the environment's copies, and usually shows its name in the prompt.

Step	macOS / Linux	Windows (PowerShell)
Create	`python3 -m venv .venv`	`py -m venv .venv`
Activate	`source .venv/bin/activate`	`.venv\Scripts\Activate.ps1`

Step	macOS / Linux	Windows (PowerShell)
Check	`which python`	`Get-Command python`
Install	`pip install -r requirements.txt`	`pip install -r requirements.txt`
Deactivate	`deactivate`	`deactivate`
Delete	`rm -rf .venv`	`Remove-Item -Recurse .venv`

CREATING AND ACTIVATING A VIRTUAL ENVIRONMENT

main.py

```
python -m venv .venv
ls .venv .venv/bin | head -12
cat .venv/pyvenv.cfg | grep -v -i "home\|executable\|command"
source .venv/bin/activate && which python | sed "s|.*/my-app|~/my-app|" && python -c "import sys;
print('inside venv:', sys.prefix != sys.base_prefix)"
python -c "import sys; print('without activation, inside venv:', sys.prefix != sys.base_prefix)"
```

TERMINAL

```
$ python -m venv .venv
$ ls .venv .venv/bin | head -12
.venv:
bin
include
lib
lib64
pyvenv.cfg

.venv/bin:
Activate.ps1
activate
activate.csh
activate.fish
$ cat .venv/pyvenv.cfg | grep -v -i "home\|executable\|command"
include-system-site-packages = false
version = 3.14.0
$ source .venv/bin/activate && which python | sed "s|.*/my-app|~/my-app|" && python -c "import sys;
print('inside venv:', sys.prefix != sys.base_prefix)"
~/my-app/.venv/bin/python
inside venv: True
$ python -c "import sys; print('without activation, inside venv:', sys.prefix != sys.base_prefix)"
without activation, inside venv: False
```

Inside a script you can tell whether you are in a virtual environment by comparing `sys.prefix` (the environment) with `sys.base_prefix` (the base installation). Editors such as VS Code and PyCharm detect a `.venv` folder and use it automatically for running and debugging.

> **TIP**
> You do not have to activate an environment to use it: running `.venv/bin/python script.py` (or `.venv\Scripts\python.exe` on Windows) always uses that environment. Activation is a convenience for interactive shells.

12.5.3 Modern tools: uv, Poetry, conda (overview)

`venv` + `pip` + `requirements.txt` is the built-in baseline. Several popular tools add lock files, faster installs and project management on top:

Tool	What it is	Typical commands
uv	An extremely fast installer and project manager written in Rust; replaces pip, venv, pip-tools and pyenv, and can even install Python itself	`uv init`, `uv add requests`, `uv run main.py`, `uv venv`, `uv pip install`
Poetry	Dependency management and packaging driven by `pyproject.toml` with a `poetry.lock` file	`poetry new`, `poetry add requests`, `poetry install`, `poetry run`
conda	A cross-language package and environment manager (Anaconda, Miniconda, Miniforge), popular in data science for packages with C/Fortran parts	`conda create -n ds python=3.13`, `conda activate ds`, `conda install numpy`
pipx	Installs Python command-line applications, each in its own isolated environment	`pipx install ruff`
pyenv	Installs and switches between several Python versions	`pyenv install 3.14`

A PROJECT MANAGED WITH UV

main.py

```
uv init report-tool
cd report-tool
uv add tabulate
uv run main.py
```

TERMINAL

```
Initialized project `report-tool` at `/home/asha/report-tool`
Using CPython 3.14.0
Creating virtual environment at: .venv
Resolved 2 packages in 120ms
Installed 1 package in 4ms
 + tabulate==0.10.0
Hello from report-tool!
```

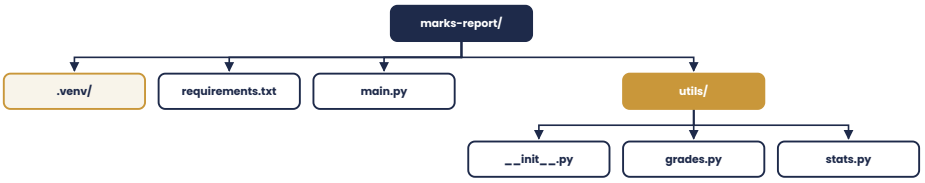
uv creates pyproject.toml, uv.lock and .venv automatically; the output above is typical.

Whichever tool you choose, the principles are the same: one isolated environment per project, dependencies declared in a file that is committed with the code, and exact versions locked for reproducible installs. This book uses plain `venv` and `pip` because they come with Python.

A module is a .py file; import runs it once, caches it in sys.modules and binds a name; use import m, import m as alias, from m import name (avoid import *). Your own modules are imported by file name; the if __name__ == "__main__": guard runs code only when a file is executed directly. Python finds modules through sys.path (script folder first, then PYTHONPATH, the standard library and site-packages), so never name files after standard modules. dir() lists module attributes; importlib.reload re-runs a module. A package is a folder with __init__.py; import from it with dotted names, absolute (from shop.billing import x) or relative (from .billing import x) imports, sub-packages and __all__ for the public API; run package modules with python -m. The standard library covers maths, dates, the OS, JSON and CSV and much more. pip installs packages from PyPI (install, uninstall, list, show, freeze); requirements.txt with version specifiers (==, >=, ~=) records dependencies. Use a virtual environment (python -m venv .venv, then activate) for every project; uv, Poetry and conda are popular modern alternatives.

PRACTICE CHECKPOINT

Create a **utils package** with two modules, import it into a main script, and run it inside a **virtual environment** with one third-party package installed from **requirements.txt**.



THE CHECKPOINT PROJECT LAYOUT

REQUIREMENTS.TXT text

```
tabulate>=0.9
```

UTILS/__INIT__.PY Python

```python
"""Helper package for the marks report."""
from .grades import grade
from .stats import summary

__all__ = ["grade", "summary"]
```

UTILS/GRADES.PY Python

```python
def grade(mark):
    """Return the letter grade for a mark out of 100."""
    for limit, letter in ((90, "A"), (75, "B"), (50, "C")):
        if mark >= limit:
            return letter
    return "F"
```

UTILS/STATS.PY Python

```python
import statistics

def summary(marks):
    """Return (average, highest, lowest) for a list of marks."""
    return round(statistics.mean(marks), 1), max(marks), min(marks)
```

MAIN.PY Python

```python
from tabulate import tabulate          # third-party, from requirements.txt
from utils import grade, summary       # our own package

students = {"Asha": 91, "Ravi": 78, "Meera": 85, "John": 47}

def main():
    rows = [(name, mark, grade(mark)) for name, mark in students.items()]
    print(tabulate(rows, headers=["Student", "Mark", "Grade"], tablefmt="github"))
    avg, high, low = summary(list(students.values()))
    print(f"\naverage {avg} | highest {high} | lowest {low}")

if __name__ == "__main__":
    main()
```

SAMPLE SOLUTION: SET UP THE ENVIRONMENT AND RUN

main.py

```
python -m venv .venv
.venv/bin/pip install -r requirements.txt
.venv/bin/python main.py
python main.py 2>&1 | tail -1
```

TERMINAL

```
$ python -m venv .venv
$ .venv/bin/pip install -r requirements.txt
Looking in links: ~/wheels
Processing ~/wheels/tabulate-0.10.0-py3-none-any.whl (from -r requirements.txt (line 1))
Installing collected packages: tabulate
Successfully installed tabulate-0.10.0
$ .venv/bin/python main.py
| Student   |   Mark | Grade   |
|-----------|--------|---------|
| Asha      |     91 | A       |
| Ravi      |     78 | B       |
| Meera     |     85 | B       |
| John      |     47 | F       |

average 75.2 | highest 91 | lowest 47
$ python main.py 2>&1 | tail -1
ModuleNotFoundError: No module named 'tabulate'
```

The last command shows why the environment matters: the system-wide Python does not have `tabulate`, so the same script fails outside the virtual environment.

MODULE 13 · INTERMEDIATE

Exception handling

Minimum duration **2 days** Level **Intermediate**

Exceptions are how Python signals errors at run time. Handling them keeps programs running and lets them give users clear messages instead of crashing with a traceback. This module covers the difference between syntax errors and exceptions, reading tracebacks, the built-in exception hierarchy and the most common exceptions, try, except, else and finally, raising and re-raising exceptions and chaining them, writing custom exception classes, exception groups and except*, exception notes, assert, the warnings module, and the best practices that separate robust code from code that hides bugs.

IN THIS MODULE

13.1 Basics
13.2 Built-in exceptions
13.3 Handling exceptions
13.4 Raising exceptions
13.5 Custom exceptions
13.6 Advanced topics
13.7 Best practices

13.1 Basics

13.1.1 Syntax errors vs exceptions

Python reports two kinds of errors. A **syntax error** means the code is not valid Python: a missing colon, an unclosed bracket, wrong indentation. Python detects it while **parsing** the file, before running a single line, so nothing in the file runs. An **exception** happens while a syntactically correct program is **running**: dividing by zero, opening a file that does not exist, converting `"abc"` to an int. Lines before the failing one have already run, and exceptions can be **handled** so that the program continues.

	Syntax error	Exception
When detected	before the program starts (parsing)	while the program runs
Cause	invalid code	valid code meets a problem: bad data, missing file, …
Earlier lines run?	no, nothing runs	yes, up to the failing line
Can be handled?	no: fix the code	yes, with try / except
Examples	SyntaxError, IndentationError	ValueError, ZeroDivisionError, KeyError, …

A SYNTAX ERROR: NOTHING RUNS

main.py

```
print("this line never runs")
if True
    print("missing colon above")
```

OUTPUT

```
  File "main.py", line 2
    if True
           ^
SyntaxError: expected ':'
```

AN EXCEPTION: THE PROGRAM RUNS UNTIL THE ERROR

main.py

```
print("this line runs")
print(10 / 0)
print("this line is never reached")
```

OUTPUT

```
this line runs
Traceback (most recent call last):
  File "main.py", line 2, in <module>
    print(10 / 0)
          ~~~^~~
ZeroDivisionError: division by zero
```

Recent Python versions give very helpful syntax-error messages, often suggesting the fix ("expected ':'", "Did you mean…?", "'(' was never closed").

PYTHON'S SYNTAX-ERROR MESSAGES

main.py

```
for source in ['print("hi"', 'x = [1, 2, 3', 'if x = 5: pass', 'while True: pass', 'def f():\nreturn
1']:
    try:
        compile(source, "demo", "exec")
    except SyntaxError as e:
        print(f"{type(e).__name__:<17} {e.msg}")
```

OUTPUT

```
SyntaxError       '(' was never closed
SyntaxError       '[' was never closed
SyntaxError       invalid syntax. Maybe you meant '==' or ':=' instead of '='?
SyntaxError       invalid syntax
IndentationError  expected an indented block after function definition on line 1
```

13.1.2 Reading tracebacks

When an exception is not handled, Python stops and prints a **traceback**: the chain of calls that led to the error. Read it **from the bottom up**. The last line names the exception type and its message, which usually tells you what went wrong. The lines above show each call in the chain, the most recent call last, with the file, line number, function name and the source line; Python marks the exact failing expression with `^^^^`.

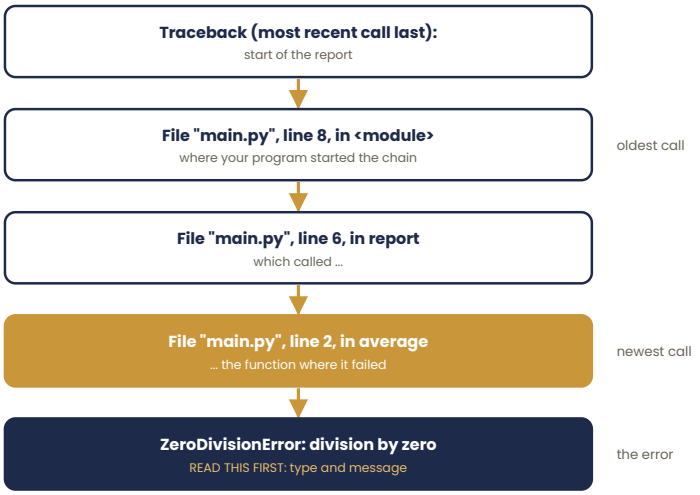
AN UNHANDLED EXCEPTION AND ITS TRACEBACK

main.py

```
def average(marks):
    return sum(marks) / len(marks)

def report(students):
    for name, marks in students.items():
        print(name, average(marks))

report({"Asha": [91, 88], "Ravi": []})
```

OUTPUT

```
Asha 89.5
Traceback (most recent call last):
  File "main.py", line 8, in <module>
    report({"Asha": [91, 88], "Ravi": []})
    ~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "main.py", line 6, in report
    print(name, average(marks))
                ~~~~~~~^^^^^^^
  File "main.py", line 2, in average
    return sum(marks) / len(marks)
           ~~~~~~~~~~~^~~~~~~~~~~~
ZeroDivisionError: division by zero
```



READING A TRACEBACK FROM THE BOTTOM UP

1. Read the last line

The exception type and message: ZeroDivisionError, division by zero.

2. Find your newest frame

The lowest "File" line in your own code shows where it happened: line 2, in average.

3. Walk up for the cause

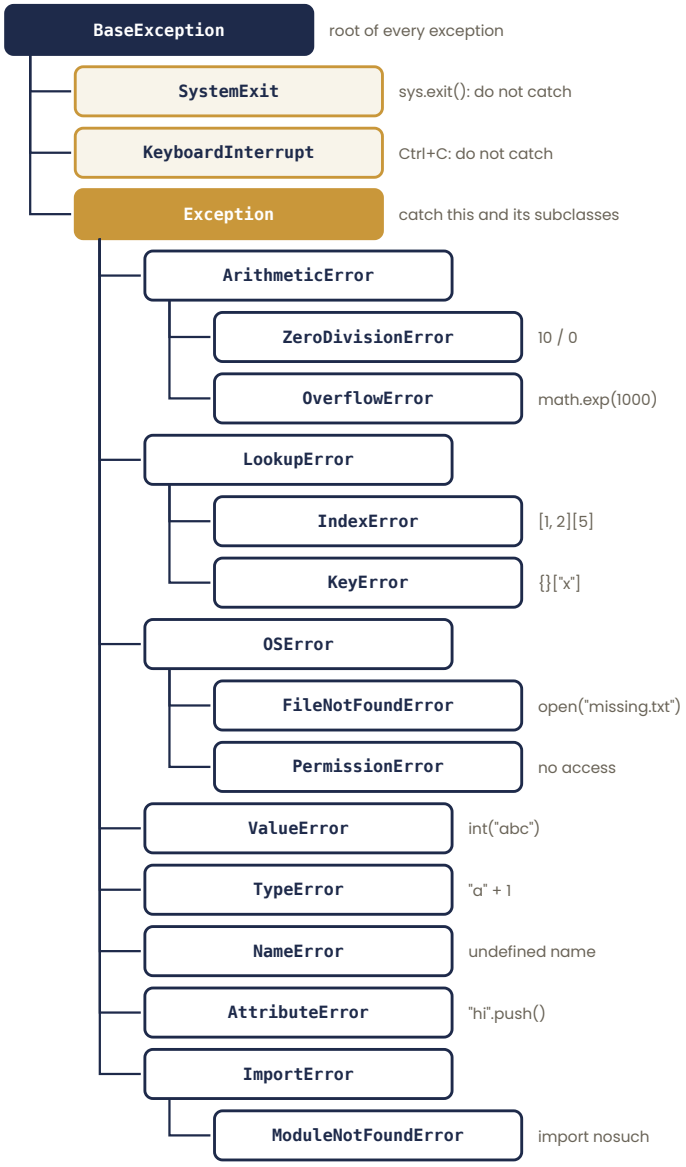
Ravi had an empty list of marks, so len(marks) was 0. Fix the data or handle the case.

13.2 Built-in exceptions

13.2.1 Hierarchy: BaseException → Exception

Exceptions are objects, and every exception type is a **class** in an inheritance hierarchy rooted at `BaseException`. Catching a class also catches all of its subclasses: `except LookupError` catches both `IndexError` and `KeyError`. Almost everything you catch derives from `Exception`; the few classes directly under `BaseException` (`KeyboardInterrupt` from Ctrl+C, `SystemExit` from `sys.exit()`, `GeneratorExit`) are

signals that should normally be allowed to stop the program, which is why you should catch `Exception`, not `BaseException`.



PART OF THE BUILT-IN EXCEPTION HIERARCHY

EXPLORING THE HIERARCHY main.py

```
print(ZeroDivisionError.__mro__)
print(issubclass(KeyError, LookupError), issubclass(FileNotFoundError, OSError))
print(issubclass(KeyboardInterrupt, Exception), issubclass(KeyboardInterrupt, BaseException))

def show_tree(cls, depth=0, max_depth=1):
    print("  " * depth + cls.__name__)
    if depth < max_depth:
        for sub in sorted(cls.__subclasses__(), key=lambda c: c.__name__):
            if sub.__module__ == "builtins":
                show_tree(sub, depth + 1, max_depth)

show_tree(LookupError)
show_tree(ArithmeticError)
```

OUTPUT

```
(<class 'ZeroDivisionError'>, <class 'ArithmeticError'>, <class 'Exception'>, <class 'BaseException'>,
<class 'object'>)
True True
False True
LookupError
  IndexError
  KeyError
ArithmeticError
  FloatingPointError
  OverflowError
  ZeroDivisionError
```

13.2.2 Common exceptions: ValueError, TypeError, IndexError, KeyError, ZeroDivisionError, FileNotFoundError, AttributeError, ImportError, NameError

Exception	Raised when	Example
`ValueError`	right type, inappropriate value	`int("abc")`
`TypeError`	operation on the wrong type	`"age" + 5`
`IndexError`	sequence index out of range	`[1, 2][5]`
`KeyError`	dictionary key not found	`{"a": 1}["b"]`
`ZeroDivisionError`	division or modulo by zero	`10 / 0`
`FileNotFoundError`	file or folder does not exist	`open("missing.txt")`
`AttributeError`	object has no such attribute	`"hi".push()`
`ImportError` / `ModuleNotFoundError`	import fails	`import nosuchmodule`
`NameError`	name not defined	`print(totl)`
`PermissionError`	not allowed by the OS	writing a read-only file
`RecursionError`	too deep recursion	missing base case
`StopIteration`	iterator exhausted	`next(iter([]))`

TRIGGERING THE COMMON EXCEPTIONS main.py

```
examples = [
    'int("abc")', '"age" + 5', '[1, 2][5]', '{"a": 1}["b"]', '10 / 0',
    'open("missing.txt")', '"hi".push()', 'import nosuchmodule', 'print(totl)',
    'next(iter([]))', 'float("1,5")', 'len(42)',
]
for code in examples:
    try:
        exec(code)
    except Exception as e:
        print(f"{code:<22} {type(e).__name__}: {e}")
```

OUTPUT

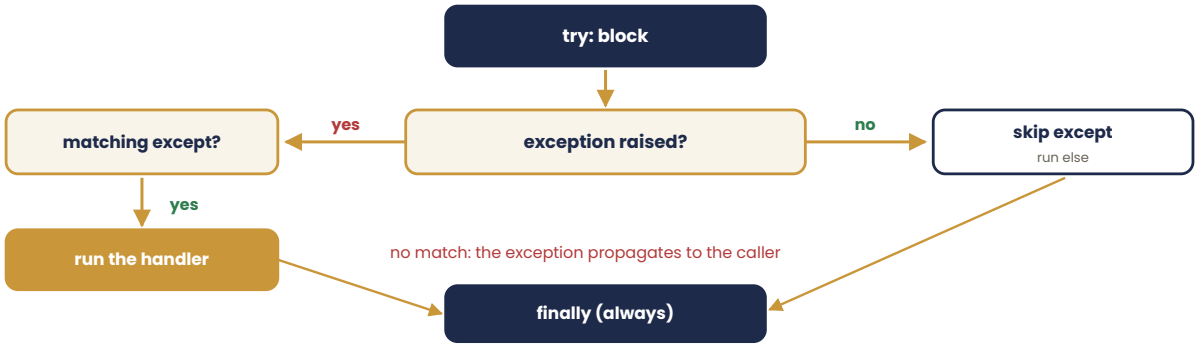
```
int("abc")             ValueError: invalid literal for int() with base 10: 'abc'
"age" + 5              TypeError: can only concatenate str (not "int") to str
[1, 2][5]              IndexError: list index out of range
{"a": 1}["b"]          KeyError: 'b'
10 / 0                 ZeroDivisionError: division by zero
open("missing.txt")    FileNotFoundError: [Errno 2] No such file or directory: 'missing.txt'
"hi".push()            AttributeError: 'str' object has no attribute 'push'
import nosuchmodule    ModuleNotFoundError: No module named 'nosuchmodule'
print(totl)            NameError: name 'totl' is not defined
next(iter([]))         StopIteration:
float("1,5")           ValueError: could not convert string to float: '1,5'
len(42)                TypeError: object of type 'int' has no len()
```

Note the friendly hints in modern messages: *Did you mean: 'total'?* for misspelled names and attributes, and *can only concatenate str (not "int") to str* for type mismatches.

13.3 Handling exceptions

13.3.1 try and except

Put code that might fail in a `try` block and the recovery code in an `except` block. If no exception occurs, the `except` block is skipped. If an exception occurs, Python abandons the rest of the `try` block **immediately**, looks for a matching `except` clause and runs it; the program then continues after the whole statement.



HOW PYTHON RUNS A TRY STATEMENT

TRY AND EXCEPT main.py

```
try:
    number = int("42")
    print("converted:", number)
except ValueError:
    print("that was not a number")

try:
    number = int("forty-two")
    print("this line is skipped")
except ValueError:
    print("that was not a number")

print("the program carries on")
```

OUTPUT

```
converted: 42
that was not a number
the program carries on
```

ASKING AGAIN UNTIL THE INPUT IS VALID main.py

```
while True:
    text = input("Enter your age: ")
    try:
        age = int(text)
        break                              # success: leave the loop
    except ValueError:
        print(f"  {text!r} is not a whole number, try again")
print("Next year you will be", age + 1)
```

CONSOLE (user input in gold)

```
Enter your age: twenty
  'twenty' is not a whole number, try again
Enter your age: 20.5
  '20.5' is not a whole number, try again
Enter your age: 20
Next year you will be 21
```

13.3.2 Catching specific exceptions

Name the exception you **expect** and know how to handle. Catching too broadly hides unrelated bugs: a misspelled variable inside a `try` block that catches everything would be silently treated as "invalid input".

SPECIFIC VS OVER-BROAD HANDLERS main.py

```
def parse_marks(text):
    try:
        return [int(x) for x in text.split(",")]
    except ValueError:                      # only the error we expect
        return None

print(parse_marks("91,78,85"))
print(parse_marks("91,seventy,85"))

def broken_parse(text):
    try:
        return [int(x) for x in txt.split(",")]    # typo: txt
    except Exception:                              # too broad: hides the bug
        return None
print(broken_parse("91,78"), "<- a NameError was silently swallowed")
```

OUTPUT

```
[91, 78, 85]
None
None <- a NameError was silently swallowed
```

13.3.3 Multiple except blocks and tuples of exceptions

A `try` can have several `except` clauses; Python checks them **top to bottom** and runs the first one that matches, so put specific classes before general ones. To handle several exception types the same way, list them in a tuple: `except (ValueError, TypeError):`.

SEVERAL HANDLERS AND A TUPLE OF TYPES main.py

```
def safe_divide(a, b):
    try:
        return float(a) / float(b)
    except ZeroDivisionError:
        return "cannot divide by zero"
    except (ValueError, TypeError):
        return f"invalid numbers: {a!r}, {b!r}"

for a, b in [(10, 4), ("9", "3"), (1, 0), ("ten", 2), (None, 5)]:
    print(f"{a!r} / {b!r} -> {safe_divide(a, b)}")
```

OUTPUT

```
10 / 4 -> 2.5
'9' / '3' -> 3.0
1 / 0 -> cannot divide by zero
'ten' / 2 -> invalid numbers: 'ten', 2
None / 5 -> invalid numbers: None, 5
```

ORDER MATTERS: SPECIFIC BEFORE GENERAL

main.py

```
data = {"marks": [91, 78]}
for key, index in [("marks", 1), ("marks", 5), ("grades", 0)]:
    try:
        print(data[key][index])
    except LookupError as e:                # parent class: catches IndexError and KeyError
        print(f"lookup failed: {type(e).__name__} {e}")

try:
    [1, 2][9]
except Exception:
    print("general handler")
except IndexError:
    print("never reached: the general handler above matches first")
```

OUTPUT

```
78
lookup failed: IndexError list index out of range
lookup failed: KeyError 'grades'
general handler
```

13.3.4 except Exception as e

`except SomeError as e` binds the exception **object** to `e`, so you can read its message (`str(e)`), type (`type(e).__name__`) and arguments (`e.args`), and some exceptions carry extra attributes such as `filename` and `errno`. The name `e` is deleted when the handler ends. `except Exception as e` is a reasonable "last line of defence" at the top level of a program, for logging an unexpected error, but not as a substitute for specific handlers.

INSPECTING THE EXCEPTION OBJECT main.py

```
try:
    open("marks_2026.csv")
except FileNotFoundError as e:
    print("message:", e)
    print("type:", type(e).__name__)
    print("args:", e.args)
    print("filename:", e.filename, "| errno:", e.errno)

try:
    {"a": 1}["b"]
except KeyError as e:
    print("missing key:", e.args[0])

try:
    int("x")
except Exception as e:
    error = e
print(repr(error))
try:
    e
except NameError:
    print("e itself was deleted after the handler")
```

OUTPUT

```
message: [Errno 2] No such file or directory: 'marks_2026.csv'
type: FileNotFoundError
args: (2, 'No such file or directory')
filename: marks_2026.csv | errno: 2
missing key: b
ValueError("invalid literal for int() with base 10: 'x'")
e itself was deleted after the handler
```

13.3.5 else block

An optional `else` block after the `except` clauses runs **only if the try block raised no exception**. It keeps the `try` block minimal: only the line that can fail goes in `try`, and the code that should run on success goes in `else`, so exceptions raised there are not accidentally caught by the handlers.

TRY / EXCEPT / ELSE main.py

```
def read_number(text):
    try:
        value = int(text)              # only the risky line
    except ValueError:
        print(f"{text!r}: not a number")
    else:
        print(f"{text!r}: ok, squared = {value * value}")

read_number("12")
read_number("twelve")
```

OUTPUT

```
'12': ok, squared = 144
'twelve': not a number
```

13.3.6 finally block

A `finally` block runs **whatever happens**: after success, after a handled exception, after an unhandled exception (before it propagates), and even when the `try` block executes `return`, `break` or `continue`. It is used for clean-up that must always happen, such as closing files or network connections and releasing locks. (For files, the `with` statement, Module 14, does this for you.)

FINALLY RUNS IN EVERY CASE

main.py

```
def process(value):
    print(f"-- process({value!r})")
    try:
        result = 100 / int(value)
    except ValueError:
        print("   invalid value")
        return "error"
    else:
        print("   success")
        return result
    finally:
        print("   finally: clean-up runs every time")

print(process("4"))
print(process("abc"))
try:
    process("0")
except ZeroDivisionError:
    print("caller saw ZeroDivisionError after finally ran")
```

OUTPUT

```
-- process('4')
   success
   finally: clean-up runs every time
25.0
-- process('abc')
   invalid value
   finally: clean-up runs every time
error
-- process('0')
   finally: clean-up runs every time
caller saw ZeroDivisionError after finally ran
```

Block	Runs when
`try`	always, until an exception occurs
`except E`	an exception of type `E` (or a subclass) occurred in try
`else`	try completed without any exception
`finally`	always, last, even after return or an unhandled exception

13.4 Raising exceptions

13.4.1 raise

Your own code signals errors with `raise ExceptionType("message")`. Raise an exception when a function **cannot do what it promises**, for example because it received invalid data; do not print an error and return a strange value like `-1` or `None` that the caller might not check. Choose the most fitting built-in

type: `ValueError` for bad values, `TypeError` for wrong types, `KeyError`/`LookupError` for missing items, `PermissionError` for forbidden actions.

RAISING EXCEPTIONS TO REJECT BAD INPUT main.py

```
def set_age(age):
    if not isinstance(age, int):
        raise TypeError(f"age must be an int, not {type(age).__name__}")
    if not 0 <= age <= 130:
        raise ValueError(f"age must be between 0 and 130, got {age}")
    return age

for value in (25, -4, "30", 200):
    try:
        print("age set:", set_age(value))
    except (TypeError, ValueError) as e:
        print(f"{type(e).__name__}: {e}")
```

OUTPUT

```
age set: 25
ValueError: age must be between 0 and 130, got -4
TypeError: age must be an int, not str
ValueError: age must be between 0 and 130, got 200
```

13.4.2 Re-raising

Sometimes a handler wants to do something, typically log the error or clean up, and then let the exception continue to the caller. A bare `raise` inside an `except` block re-raises the current exception with its original traceback.

LOGGING AND RE-RAISING main.py

```
def load_config(path):
    try:
        with open(path) as f:
            return f.read()
    except FileNotFoundError:
        print(f"[log] config file {path!r} missing")
        raise                               # let the caller decide

try:
    load_config("settings.ini")
except FileNotFoundError as e:
    print("caller handles it:", e.strerror)
```

OUTPUT

```
[log] config file 'settings.ini' missing
caller handles it: No such file or directory
```

13.4.3 Exception chaining raise ... from ...

When you catch a low-level exception and raise a more meaningful one, write `raise NewError(...) from original`. The original is stored in the new exception's `__cause__`, and the traceback shows both, joined by *The above exception was the direct cause of the following exception*. If you raise inside a handler without `from`, Python still links them (`__context__`) but with the message *During handling of the above exception, another exception occurred*, which suggests a bug in the handler. `raise ... from None` hides the original when it is irrelevant.

CHAINING WITH RAISE ... FROM ...

main.py

```
def get_price(catalogue, item):
    try:
        return catalogue[item]
    except KeyError as e:
        raise LookupError(f"{item!r} is not sold in this shop") from e

try:
    get_price({"pen": 10}, "laptop")
except LookupError as e:
    print("error:", e)
    print("cause:", repr(e.__cause__))

get_price({"pen": 10}, "laptop")
```

OUTPUT

```
error: 'laptop' is not sold in this shop
cause: KeyError('laptop')
Traceback (most recent call last):
  File "main.py", line 3, in get_price
    return catalogue[item]
           ~~~~~~~~~^^^^^^
KeyError: 'laptop'

The above exception was the direct cause of the following exception:

Traceback (most recent call last):
  File "main.py", line 13, in <module>
    get_price({"pen": 10}, "laptop")
    ~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^
  File "main.py", line 5, in get_price
    raise LookupError(f"{item!r} is not sold in this shop") from e
LookupError: 'laptop' is not sold in this shop
```

13.5 Custom exceptions

13.5.1 Subclassing Exception

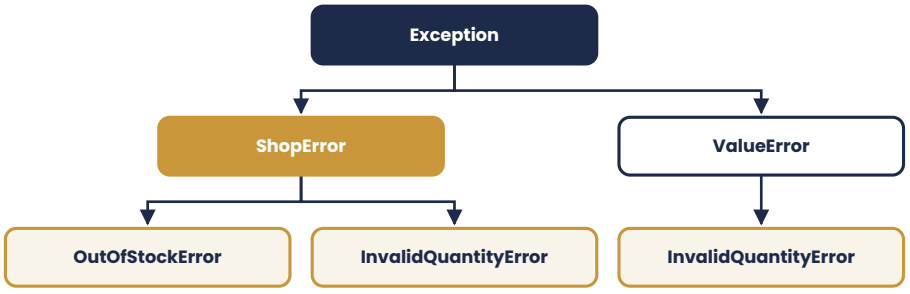
Define your own exception types for errors specific to your application by subclassing `Exception` (or a more specific built-in such as `ValueError`). A class body of just `pass`, or a docstring, is enough. Custom types let callers catch **exactly** your errors, and a common base class for a library lets them catch all of its errors at once. By convention the names end in `Error`.

A SMALL HIERARCHY OF CUSTOM EXCEPTIONS main.py

```
class ShopError(Exception):
    """Base class for all errors raised by the shop module."""

class OutOfStockError(ShopError):
    pass

class InvalidQuantityError(ShopError, ValueError):     # also a ValueError
    pass

stock = {"pen": 5, "bag": 0}

def buy(item, qty):
    if qty <= 0:
        raise InvalidQuantityError(f"quantity must be positive, got {qty}")
    if stock.get(item, 0) < qty:
        raise OutOfStockError(f"only {stock.get(item, 0)} {item}(s) left")
    stock[item] -= qty
    return f"bought {qty} {item}(s)"

for item, qty in [("pen", 2), ("bag", 1), ("pen", -1)]:
    try:
        print(buy(item, qty))
    except OutOfStockError as e:
        print("Sorry:", e)
    except ShopError as e:
        print(f"Shop error ({type(e).__name__}):", e)
print(issubclass(InvalidQuantityError, ValueError))
```

OUTPUT

```
bought 2 pen(s)
Sorry: only 0 bag(s) left
Shop error (InvalidQuantityError): quantity must be positive, got -1
True
```



INVALIDQUANTITYERROR INHERITS FROM BOTH SHOPERROR AND VALUEERROR

13.5.2 Adding attributes and messages

A custom exception can carry **structured data** about the error as attributes, so handlers do not have to parse the message. Define `__init__`, store the values, and call `super().__init__(message)` so that `str(e)` and tracebacks show a readable message.

AN EXCEPTION WITH ATTRIBUTES main.py

```
class InsufficientBalanceError(Exception):
    def __init__(self, balance, requested):
        self.balance = balance
        self.requested = requested
        self.shortfall = requested - balance
        super().__init__(f"cannot withdraw Rs {requested:,}: balance is Rs {balance:,}")

def withdraw(balance, amount):
    if amount > balance:
        raise InsufficientBalanceError(balance, amount)
    return balance - amount

try:
    withdraw(2_500, 4_000)
except InsufficientBalanceError as e:
    print(e)
    print(f"you need Rs {e.shortfall:,} more")
    print(e.args)
```

OUTPUT

```
cannot withdraw Rs 4,000: balance is Rs 2,500
you need Rs 1,500 more
('cannot withdraw Rs 4,000: balance is Rs 2,500',)
```

13.6 Advanced topics

13.6.1 Exception groups and except* (3.11)

Some code produces **several** independent errors at once: validating a form with many fields, or running concurrent tasks (Module 20) where more than one fails. `ExceptionGroup(message, [exceptions])` bundles them into one exception, and the `except*` syntax handles the matching parts of a group: each `except*` clause receives a sub-group of the exceptions of its type, and more than one clause can run. Unhandled parts are re-raised.

RAISING AND HANDLING AN EXCEPTIONGROUP main.py

```
def validate(form):
    errors = []
    if not form.get("name"):
        errors.append(ValueError("name is required"))
    if not str(form.get("age", "")).isdigit():
        errors.append(TypeError("age must be a number"))
    if "@" not in form.get("email", ""):
        errors.append(ValueError("email is invalid"))
    if errors:
        raise ExceptionGroup("form has errors", errors)

try:
    validate({"name": "", "age": "twenty", "email": "asha.lbf.in"})
except* ValueError as group:
    for e in group.exceptions:
        print("value problem:", e)
except* TypeError as group:
    for e in group.exceptions:
        print("type problem:", e)
```

OUTPUT

```
value problem: name is required
value problem: email is invalid
type problem: age must be a number
```

Without `except*`, an `ExceptionGroup` can still be caught as a whole with a normal `except ExceptionGroup as eg`, and its `exceptions` list examined. Unhandled groups print a tree-shaped traceback:

AN UNHANDLED EXCEPTION GROUP main.py

```
raise ExceptionGroup("2 uploads failed", [ConnectionError("notes.pdf: timeout"),
PermissionError("video.mp4: denied")])
```

OUTPUT

```
  + Exception Group Traceback (most recent call last):
  |   File "main.py", line 1, in <module>
  |     raise ExceptionGroup("2 uploads failed", [ConnectionError("notes.pdf: timeout"),
PermissionError("video.mp4: denied")])
  | ExceptionGroup: 2 uploads failed (2 sub-exceptions)
  +-+---------------- 1 ----------------
    | ConnectionError: notes.pdf: timeout
    +---------------- 2 ----------------
    | PermissionError: video.mp4: denied
    +------------------------------------
```

13.6.2 Adding notes to exceptions (3.11)

`e.add_note(text)` attaches extra context to an exception as it passes through your code, without changing its type or message. Notes are shown at the end of the traceback and stored in `e.__notes__`. They are ideal for adding "which record / which file was being processed" information.

ADDING NOTES FOR CONTEXT

main.py

```
rows = ["91", "78", "eighty", "85"]
try:
    for line_no, text in enumerate(rows, start=1):
        try:
            int(text)
        except ValueError as e:
            e.add_note(f"while reading marks.csv, line {line_no}")
            e.add_note("hint: marks must be written as digits")
            raise
except ValueError as e:
    print(e)
    print(e.__notes__)
```

OUTPUT

```
invalid literal for int() with base 10: 'eighty'
['while reading marks.csv, line 3', 'hint: marks must be written as digits']
```

NOTES APPEAR AT THE END OF THE TRACEBACK

main.py

```
try:
    int("eighty")
except ValueError as e:
    e.add_note("while reading marks.csv, line 3")
    raise
```

OUTPUT

```
Traceback (most recent call last):
  File "main.py", line 2, in <module>
    int("eighty")
    ~~~^^^^^^^^^^
ValueError: invalid literal for int() with base 10: 'eighty'
while reading marks.csv, line 3
```

13.6.3 assert statements

`assert condition, "message"` raises `AssertionError` with the message if the condition is false. Assertions document and check **internal assumptions** that should always be true if the code is correct ("this list is sorted", "this total is never negative"), and they are used heavily in tests (pytest, Module 22). They are **not** for validating user input or external data, because running Python with `-O` (optimise) removes all assert statements.

CHECKING ASSUMPTIONS WITH ASSERT main.py

```
def average(marks):
    assert len(marks) > 0, "average() needs at least one mark"
    result = sum(marks) / len(marks)
    assert min(marks) <= result <= max(marks), "average out of range: bug!"
    return result

print(average([91, 78, 85]))
try:
    average([])
except AssertionError as e:
    print("AssertionError:", e)
```

OUTPUT

```
84.66666666666667
AssertionError: average() needs at least one mark
```

CHECK.PY Python

```
assert 1 + 1 == 3, "maths is broken"
print("finished")
```

PYTHON -O REMOVES ASSERT STATEMENTS main.py

```
python check.py 2>&1 | tail -1
python -O check.py
```

TERMINAL

```
$ python check.py 2>&1 | tail -1
AssertionError: maths is broken
$ python -O check.py
finished
```

WATCH OUT

Never write `assert user_is_admin` to protect an action: with `-O` the check disappears. Use `if not ...: raise PermissionError(...)` for anything that must always be enforced.

13.6.4 Warnings module

A **warning** tells the user about something that is not (yet) an error: a deprecated function, a suspicious value, a fallback being used. `warnings.warn(message, category)` prints it to standard error, by default only once per location, and the program continues. Categories include `UserWarning` (default), `DeprecationWarning`, `FutureWarning` and `RuntimeWarning`. Filters can ignore warnings, or turn them into exceptions, which is common in test suites.

ISSUING AND FILTERING WARNINGS main.py

```
import warnings, sys
warnings.showwarning = lambda msg, cat, f, l, file=None, line=None: print(f"{cat.__name__}: {msg}",
file=sys.stdout)

def old_total(items):
    warnings.warn("old_total() is deprecated; use sum() instead", DeprecationWarning, stacklevel=2)
    return sum(items)

warnings.simplefilter("default")            # show DeprecationWarning (hidden by default)
for items in ([1, 2, 3], [4, 5]):
    print(old_total(items))                  # same location: shown only once

def set_volume(level):
    if level > 100:
        warnings.warn(f"volume {level} too high, using 100")
        level = 100
    return level
print(set_volume(150))

with warnings.catch_warnings():
    warnings.simplefilter("error")           # turn warnings into exceptions
    try:
        set_volume(120)
    except UserWarning as w:
        print("raised as an exception:", w)
```

OUTPUT

```
DeprecationWarning: old_total() is deprecated; use sum() instead
6
9
UserWarning: volume 150 too high, using 100
100
raised as an exception: volume 120 too high, using 100
```

13.7 Best practices

13.7.1 Catch specific exceptions only

Handle an exception only where you can do something useful about it, and catch only the types you expect. Keep the `try` block as small as possible, so it is clear which operation can fail. Let unexpected exceptions propagate: a crash with a clear traceback is far easier to fix than wrong results caused by a hidden error. At the top level of an application, one broad handler can log unexpected errors and show a friendly message.

Small try blocks
Only the risky line inside try; follow-up work in else.

Specific types
except ValueError, not except Exception, wherever possible.

Useful handling
Retry, use a default, ask again, or add context and re-raise.

Clean up
Use finally or with for resources that must be released.

Meaningful raise
Raise built-in or custom exceptions with clear messages; chain with from.

Log, do not hide
If you catch broadly at the top level, log the full traceback.

13.7.2 EAFP vs LBYL styles

There are two styles of dealing with possible failures. **LBYL**, "Look Before You Leap", checks conditions first (`if key in d:`). **EAFP**, "Easier to Ask Forgiveness than Permission", just tries the operation and handles the exception. EAFP is idiomatic Python: it avoids checking twice, reads cleanly when failures are rare, and avoids race conditions where the situation changes between the check and the action (for example, a file deleted after `os.path.exists` returned `True`). LBYL is fine when the check is cheap and failures are common.

THE SAME TASKS IN LBYL AND EAFP STYLE main.py

```
import os

config = {"theme": "dark"}

# LBYL: look before you leap
if "font" in config:
    font = config["font"]
else:
    font = 12

# EAFP: easier to ask forgiveness than permission
try:
    font = config["font"]
except KeyError:
    font = 12
print(font, config.get("font", 12))            # get() is the neatest here

# with files, EAFP avoids a race between checking and opening
path = "report.txt"
if os.path.exists(path):                       # LBYL: the file could vanish here
    print(open(path).read())
try:                                           # EAFP
    with open(path) as f:
        print(f.read())
except FileNotFoundError:
    print(f"{path} not found, creating it")
    with open(path, "w") as f:
        f.write("new report")
```

OUTPUT

```
12 12
report.txt not found, creating it
```

13.7.3 Never silence errors with bare except:

A bare `except:` (with no type) catches **everything**, including `KeyboardInterrupt` and `SystemExit`, so the user cannot even stop the program with Ctrl+C, and it hides typos and real bugs. `except: pass` is the worst form: errors vanish without a trace. Linters flag both (ruff E722, S110). If you truly must catch everything, use `except Exception` and log the error.

A SILENT BARE EXCEPT VS A SPECIFIC, LOGGED HANDLER

main.py

```
import logging, sys
logging.basicConfig(stream=sys.stdout, format="%(levelname)s %(message)s")

def bad(values):
    total = 0
    for v in values:
        try:
            total += v
        except:                       # catches EVERYTHING, even Ctrl+C
            pass                      # and hides it
    return total

def better(values):
    total = 0
    for v in values:
        try:
            total += v
        except TypeError:
            logging.warning("skipping non-number %r", v)
    return total

data = [10, "20", 30, None]
print("bad:", bad(data), "(two values silently ignored)")
print("better:", better(data))
```

OUTPUT

```
bad: 40 (two values silently ignored)
WARNING skipping non-number '20'
WARNING skipping non-number None
better: 40
```

Syntax errors stop a file before it runs; exceptions happen at run time and can be handled. Read tracebacks from the bottom: the last line gives the type and message, the lowest frame in your code shows where it happened. All exceptions inherit from BaseException; catch Exception or more specific subclasses (ValueError, TypeError, IndexError, KeyError, ZeroDivisionError, FileNotFoundError, AttributeError, ImportError, NameError). try runs risky code; except handles matching types (specific before general, tuples for several, as e for the object); else runs on success; finally always runs. raise signals errors, a bare raise re-raises, raise … from … chains a new exception to its cause. Custom exceptions subclass Exception and can carry attributes. ExceptionGroup and except* handle several errors at once; add_note adds context; assert checks internal assumptions (removed by -O); warnings report non-fatal problems. Catch only what you can handle, prefer EAFP, and never silence errors with a bare except.

PRACTICE CHECKPOINT

Write a **safe calculator** that handles invalid input and division by zero, and an **ATM** program that raises a custom `InsufficientBalanceError`.

PART 1: A SAFE CALCULATOR

main.py

```
import operator

OPS = {"+": operator.add, "-": operator.sub, "*": operator.mul, "/": operator.truediv}

print("Safe calculator (type q to quit)")
while True:
    line = input("calc> ").strip()
    if line.lower() == "q":
        break
    try:
        left, op, right = line.split()
        result = OPS[op](float(left), float(right))
    except ValueError:
        print("  please type: number operator number, e.g. 8 / 2")
    except KeyError:
        print(f"  unknown operator, use one of {' '.join(OPS)}")
    except ZeroDivisionError:
        print("  cannot divide by zero")
    else:
        print(f"  = {result:g}")
print("bye")
```

CONSOLE (user input in gold)

```
Safe calculator (type q to quit)
calc> 8 / 2
  = 4
calc> 7 x 3
  unknown operator, use one of + - * /
calc> ten + 1
  please type: number operator number, e.g. 8 / 2
calc> 5 / 0
  cannot divide by zero
calc> 2.5 * 4
  = 10
calc> 1 +
  please type: number operator number, e.g. 8 / 2
calc> q
bye
```

PART 2: AN ATM WITH A CUSTOM EXCEPTION

main.py

```
class InsufficientBalanceError(Exception):
    def __init__(self, balance, amount):
        self.balance, self.amount = balance, amount
        super().__init__(f"insufficient balance: Rs {balance:,} available, Rs {amount:,} requested")

class InvalidAmountError(ValueError):
    pass

class ATM:
    def __init__(self, balance):
        self.balance = balance

    def withdraw(self, amount):
        if amount <= 0 or amount % 100:
            raise InvalidAmountError("amount must be a positive multiple of Rs 100")
        if amount > self.balance:
            raise InsufficientBalanceError(self.balance, amount)
        self.balance -= amount
        return self.balance

atm = ATM(5_000)
while True:
    text = input("Withdraw amount (0 to exit): ")
    try:
        amount = int(text)
        if amount == 0:
            break
        left = atm.withdraw(amount)
    except InsufficientBalanceError as e:
        print(f"  {e} (short by Rs {e.amount - e.balance:,})")
    except InvalidAmountError as e:
        print("  ", e, sep="")
    except ValueError:
        print(f"  {text!r} is not a valid amount")
    else:
        print(f"  please collect Rs {amount:,}; balance Rs {left:,}")
    finally:
        print("  -- transaction recorded")
print("Thank you for banking with LBF Bank")
```

CONSOLE (user input in gold)

```
Withdraw amount (0 to exit): 2000
  please collect Rs 2,000; balance Rs 3,000
  -- transaction recorded
Withdraw amount (0 to exit): 4500
  insufficient balance: Rs 3,000 available, Rs 4,500 requested (short by Rs 1,500)
  -- transaction recorded
Withdraw amount (0 to exit): 250
  amount must be a positive multiple of Rs 100
  -- transaction recorded
Withdraw amount (0 to exit): abc
  'abc' is not a valid amount
  -- transaction recorded
Withdraw amount (0 to exit): 1500
  please collect Rs 1,500; balance Rs 1,500
  -- transaction recorded
Withdraw amount (0 to exit): 0
  -- transaction recorded
Thank you for banking with LBF Bank
```

MODULE 14 · INTERMEDIATE

File handling and data formats

Minimum duration **2 days** | Level **Intermediate**

Programs need to keep data after they finish: settings, records, logs, reports. This module covers opening files and the file modes, the with statement, text encodings, the ways to read and write text files (including very large ones), file positions, binary files, working with paths and the file system using pathlib, os and shutil, glob patterns, and the data formats used in real projects: CSV, JSON, pickle (and why it is dangerous with untrusted data), Excel files with openpyxl, and configuration files with configparser and TOML.

IN THIS MODULE

14.1 File basics
14.2 Reading files
14.3 Writing files
14.4 File positions
14.5 Binary files
14.6 Paths and the file system
14.7 Data formats

14.1 File basics

14.1.1 open() and file modes: r, w, a, x, r+, b, t

`open(path, mode)` opens a file and returns a **file object** with methods for reading and writing. The **mode** string says what you intend to do. The default, `"r"`, opens an existing file for reading text. Modes can be combined with `b` (binary) or `t` (text, the default) and `+` (read and write).

Mode	Meaning	If the file exists	If it does not exist
`"r"`	read (default)	read from the start	`FileNotFoundError`
`"w"`	write	**erased** (truncated to empty)	created
`"a"`	append	writes go to the end	created
`"x"`	exclusive create	`FileExistsError`	created
`"r+"`	read and write	kept; position at the start	`FileNotFoundError`
`"w+"`	write and read	erased	created
`"b"`	binary (add to a mode: `"rb"`, `"wb"`)	bytes, not str	
`"t"`	text (default)	str, with encoding and newline handling	

OPENING, WRITING, READING AND CLOSING main.py

```
f = open("notes.txt", "w")              # create (or erase) and write
f.write("Python file handling\n")
f.write("Module 14\n")
f.close()                                # always close when done

f = open("notes.txt")                    # "r" is the default
print(f.read())
print(f.name, f.mode, f.closed)
f.close()
print("closed:", f.closed)

try:
    open("notes.txt", "x")
except FileExistsError as e:
    print("FileExistsError:", e)
try:
    open("missing.txt")
except FileNotFoundError as e:
    print("FileNotFoundError:", e)
```

OUTPUT

```
Python file handling
Module 14

notes.txt r False
closed: True
FileExistsError: [Errno 17] File exists: 'notes.txt'
FileNotFoundError: [Errno 2] No such file or directory: 'missing.txt'
```

WATCH OUT

`"w"` empties an existing file the moment it is opened, before you write anything. Use `"a"` to add to a file, or `"x"` when you must not overwrite existing data.

14.1.2 The with statement (context manager)

Open files hold operating-system resources and buffered data. If you forget `close()`, or an exception happens before it, data may not be written and the file stays locked on some systems. The `with` statement closes the file **automatically** when the block ends, whether normally or because of an exception. File objects are **context managers**, explained fully in Module 17. Always use `with` for files.

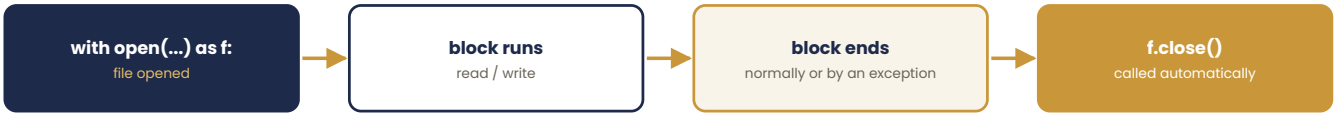
THE WITH STATEMENT

main.py

```
with open("notes.txt", "a") as f:       # f is closed automatically at the end
    f.write("written inside with\n")
print("closed after the block:", f.closed)

try:
    with open("notes.txt") as f:
        first = f.readline()
        raise RuntimeError("something went wrong")
except RuntimeError:
    pass
print("first line:", first.strip(), "| closed even after an error:", f.closed)

with open("notes.txt") as src, open("copy.txt", "w") as dst:   # two files at once
    dst.write(src.read().upper())
print(open("copy.txt").read())
```

OUTPUT

```
closed after the block: True
first line: written inside with | closed even after an error: True
WRITTEN INSIDE WITH
```



WITH GUARANTEES THE FILE IS CLOSED

14.1.3 Encoding (encoding="utf-8")

Files store **bytes**; text mode converts between bytes and `str` using an **encoding**. UTF-8 is the universal standard and can represent every character, including ₹, emoji and Indian scripts. If you do not specify an encoding, Python uses the platform default, which on older Windows set-ups is not UTF-8, so the same program can fail on another machine. Always pass `encoding="utf-8"` for text files. (Python 3.15 makes UTF-8 the default everywhere.)

WRITING AND READING UTF-8 TEXT

main.py

```
text = "Price: ₹499 | ನಮಸ್ಕಾರ | नमस्ते | café"
with open("unicode.txt", "w", encoding="utf-8") as f:
    f.write(text)

with open("unicode.txt", encoding="utf-8") as f:
    print(f.read())

with open("unicode.txt", "rb") as f:
    raw = f.read()
print(len(text), "characters stored as", len(raw), "bytes")
print(raw[:12])

try:
    with open("unicode.txt", encoding="ascii") as f:
        f.read()
except UnicodeDecodeError as e:
    print("UnicodeDecodeError:", e.reason, "at byte", e.start)
with open("unicode.txt", encoding="ascii", errors="replace") as f:
    print(f.read()[:20])
```

OUTPUT

```
Price: ₹499 | ನಮಸ್ಕಾರ | नमस्ते | café
37 characters stored as 66 bytes
b'Price: \xe2\x82\xb949'
UnicodeDecodeError: ordinal not in range(128) at byte 7
Price: ���499 | ����
```

NOTE

`errors="replace"` substitutes undecodable bytes with `�` and `errors="ignore"` drops them; both lose data, so use them only for best-effort reading. Files saved by Excel on Windows are sometimes in `"cp1252"` or `"utf-8-sig"` (UTF-8 with a byte-order mark).

14.2 Reading files

14.2.1 read(), readline(), readlines()

Three methods read text: `read()` returns the whole remaining file as one string (or at most `n` characters with `read(n)`); `readline()` returns the next line **including** its `\n`, or `""` at the end of the file; `readlines()` returns a list of all remaining lines. Each call continues from where the previous one stopped.

STUDENTS.TXT

text

```
Asha,91,Mysuru
Ravi,78,Pune
Meera,85,Mysuru
John,64,Delhi
```

READ, READLINE AND READLINES

main.py

```
with open("students.txt", encoding="utf-8") as f:
    print(repr(f.read(4)))          # 4 characters
    print(repr(f.readline()))       # rest of line 1
    print(repr(f.readline()))       # line 2
    print(f.readlines())            # all remaining lines
    print(repr(f.read()))           # end of file: empty string

with open("students.txt", encoding="utf-8") as f:
    lines = f.read().splitlines()   # lines without the \n
print(lines)
```

OUTPUT

```
'Asha'
',91,Mysuru\n'
'Ravi,78,Pune\n'
['Meera,85,Mysuru\n', 'John,64,Delhi\n']
''
['Asha,91,Mysuru', 'Ravi,78,Pune', 'Meera,85,Mysuru', 'John,64,Delhi']
```

Method	Returns	Memory use
`f.read()`	the whole file as one `str`	whole file
`f.read(n)`	up to n characters	n characters
`f.readline()`	next line with `\n`; `""` at end	one line
`f.readlines()`	list of all lines with `\n`	whole file
`for line in f`	one line per iteration	one line

14.2.2 Iterating line by line

A file object is an **iterator over its lines**, so the idiomatic way to process a text file is a `for` loop. Each line ends with `\n` (except possibly the last), so strip it with `rstrip("\n")` or `strip()`. Combine with `enumerate` for line numbers.

PROCESSING A FILE LINE BY LINE

main.py

```
total = 0
with open("students.txt", encoding="utf-8") as f:
    for line_no, line in enumerate(f, start=1):
        name, marks, city = line.strip().split(",")
        total += int(marks)
        print(f"{line_no}: {name:<6} {marks:>3}  {city}")
print("average:", total / line_no)

with open("students.txt", encoding="utf-8") as f:
    mysuru = [line.split(",")[0] for line in f if line.strip().endswith("Mysuru")]
print("from Mysuru:", mysuru)
```

OUTPUT

```
1: Asha    91  Mysuru
2: Ravi    78  Pune
3: Meera   85  Mysuru
4: John    64  Delhi
average: 79.5
from Mysuru: ['Asha', 'Meera']
```

14.2.3 Reading large files efficiently

`read()` and `readlines()` load the whole file into memory, which fails for multi-gigabyte logs or datasets. Iterating with `for line in f` keeps only one line in memory at a time, however large the file. For binary data or files without line breaks, read fixed-size **chunks** in a loop until `read()` returns an empty result.

STREAMING VS LOADING THE WHOLE FILE main.py

```
import os, tracemalloc

with open("big.log", "w", encoding="utf-8") as f:       # create a 100,000-line log
    for i in range(100_000):
        f.write(f"2026-01-26 10:{i % 60:02d} {'ERROR' if i % 997 == 0 else 'INFO'} request {i}\n")
print("file size:", f"{os.path.getsize('big.log'):,}", "bytes")

tracemalloc.start()
errors = 0
with open("big.log", encoding="utf-8") as f:
    for line in f:                                      # one line at a time
        if " ERROR " in line:
            errors += 1
peak_streaming = tracemalloc.get_traced_memory()[1]
tracemalloc.stop()

tracemalloc.start()
with open("big.log", encoding="utf-8") as f:
    lines = f.readlines()                               # everything at once
peak_all = tracemalloc.get_traced_memory()[1]
tracemalloc.stop()

print("errors found:", errors)
print("readlines() needed over 50x more memory:", peak_all > 50 * peak_streaming)

size = 0
with open("big.log", "rb") as f:
    while chunk := f.read(64 * 1024):                   # 64 KB chunks
        size += len(chunk)
print("read in chunks:", f"{size:,}", "bytes")
```

OUTPUT

```
file size: 3,588,991 bytes
errors found: 101
readlines() needed over 50x more memory: True
read in chunks: 3,588,991 bytes
```

TIP

The walrus loop `while chunk := f.read(65536):` is the standard pattern for chunked reading; it stops when `read` returns an empty bytes or string object at the end of the file.

14.3 Writing files

14.3.1 write() and writelines()

`f.write(text)` writes a string and returns the number of characters written. It does **not** add a newline; include `\n` yourself. `f.writelines(iterable)` writes every string from an iterable, also without adding newlines. `print(..., file=f)` is often the most convenient way to write formatted lines, since it adds the newline and converts values to text.

WRITE, WRITELINES AND PRINT(FILE=) main.py

```
rows = [("Asha", 91), ("Ravi", 78), ("Meera", 85)]

with open("report.txt", "w", encoding="utf-8") as f:
    n = f.write("MARKS REPORT\n")
    print("characters written:", n)
    f.writelines(f"{name},{marks}\n" for name, marks in rows)
    print("-" * 12, file=f)
    print("average:", round(sum(m for _, m in rows) / len(rows), 1), file=f)

print(open("report.txt", encoding="utf-8").read())
```

OUTPUT

```
characters written: 13
MARKS REPORT
Asha,91
Ravi,78
Meera,85
------------
average: 84.7
```

Writes are **buffered**: data collects in memory and is written to disk in blocks, which is much faster. The buffer is flushed when the file is closed (end of the `with` block) or when you call `f.flush()`. If a program crashes before that, buffered data can be lost, another reason to use `with`.

14.3.2 Appending

Mode `"a"` keeps the existing content and writes at the end of the file, creating it if necessary. It is the natural mode for logs and records that grow over time.

APPENDING TO A LOG FILE main.py

```
from datetime import datetime

def log(message):
    stamp = datetime(2026, 1, 26, 9, 30).strftime("%Y-%m-%d %H:%M")
    with open("activity.log", "a", encoding="utf-8") as f:
        f.write(f"[{stamp}] {message}\n")

log("app started")
log("user asha logged in")
log("report exported")
print(open("activity.log", encoding="utf-8").read())

with open("activity.log", "w", encoding="utf-8") as f:     # "w" would erase it all
    f.write("fresh log\n")
print(open("activity.log", encoding="utf-8").read())
```

OUTPUT

```
[2026-01-26 09:30] app started
[2026-01-26 09:30] user asha logged in
[2026-01-26 09:30] report exported

fresh log
```

14.4 File positions

14.4.1 seek() and tell()

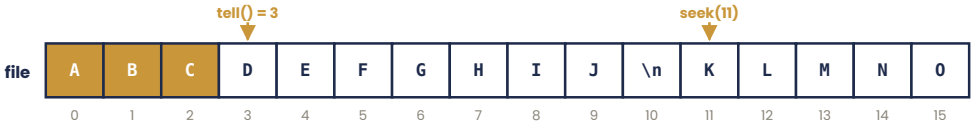
Every open file has a current **position**, where the next read or write happens. `f.tell()` returns it and `f.seek(offset, whence)` moves it: `whence` is 0 (from the start, the default), 1 (from the current position) or 2 (from the end). In text mode, only `seek(0)`, `seek(0, 2)` and positions previously returned by `tell()` are safe, because characters can occupy several bytes; arbitrary seeking is done in binary mode.

MOVING AROUND A FILE WITH SEEK AND TELL main.py

```
with open("alphabet.txt", "w+", encoding="utf-8") as f:
    f.write("ABCDEFGHIJ\nKLMNOPQRST\n")
    print("after writing, position:", f.tell())
    f.seek(0)                          # back to the start
    print(f.read(3), "| position:", f.tell())
    saved = f.tell()
    f.readline()
    print("next line:", f.readline().strip())
    f.seek(saved)                      # return to a saved position
    print("from saved position:", f.read(4))

with open("alphabet.txt", "rb") as f:  # binary: seek anywhere
    f.seek(-4, 2)                      # 4 bytes before the end
    print(f.read())
    f.seek(11)
    print(f.read(5))
```

OUTPUT

```
after writing, position: 22
ABC | position: 3
next line: KLMNOPQRST
from saved position: DEFG
b'RST\n'
b'KLMNO'
```



AFTER READING 3 CHARACTERS THE POSITION IS 3; SEEK(11) JUMPS TO THE START OF LINE 2

14.5 Binary files

14.5.1 Reading and writing bytes

Images, audio, PDFs, ZIP archives and other non-text files must be opened in **binary mode** (`"rb"`, `"wb"`). Reads return `bytes` objects (immutable sequences of integers 0–255) and writes accept `bytes` or `bytearray`; no encoding or newline translation happens. The `struct` module packs numbers into exact binary layouts.

WRITING AND READING BYTES main.py

```
data = bytes([72, 101, 108, 108, 111, 0, 255])
with open("data.bin", "wb") as f:
    f.write(data)
    f.write("₹".encode("utf-8"))            # text must be encoded first

with open("data.bin", "rb") as f:
    content = f.read()
print(content, len(content))
print(list(content[:5]), content[:5].decode("ascii"), content.hex(" "))

import struct
record = struct.pack("<I f 6s", 1042, 91.5, b"Asha  ")   # int, float, 6 bytes
print(len(record), "bytes:", record)
roll, marks, name = struct.unpack("<I f 6s", record)
print(roll, marks, name.decode().strip())
```

OUTPUT

```
b'Hello\x00\xff\xe2\x82\xb9' 10
[72, 101, 108, 108, 111] Hello 48 65 6c 6c 6f 00 ff e2 82 b9
14 bytes: b'\x12\x04\x00\x00\x00\x00\xb7BAsha  '
1042 91.5 Asha
```

14.5.2 Copying images

Copying any file, including an image, means reading its bytes and writing them unchanged. Reading in chunks keeps memory use small for large files. In real code, `shutil.copyfile` (14.6.4) does this for you. Each file format starts with a recognisable **signature** ("magic bytes"): PNG files begin with `\x89PNG`, JPEG with `\xff\xd8`, PDF with `%PDF`.

COPYING AN IMAGE BYTE FOR BYTE main.py

```
import zlib, struct

def make_png(path, width=40, height=20):          # build a tiny real PNG image
    raw = b"".join(b"\x00" + bytes([200, 150, 60] * width) for _ in range(height))
    def chunk(kind, data):
        return struct.pack(">I", len(data)) + kind + data + struct.pack(">I", zlib.crc32(kind + data))
    png = b"\x89PNG\r\n\x1a\n" + chunk(b"IHDR", struct.pack(">IIBBBBB", width, height, 8, 2, 0, 0, 0))
    png += chunk(b"IDAT", zlib.compress(raw)) + chunk(b"IEND", b"")
    with open(path, "wb") as f:
        f.write(png)

make_png("logo.png")

with open("logo.png", "rb") as src, open("logo_copy.png", "wb") as dst:
    while chunk := src.read(4096):
        dst.write(chunk)

a, b = open("logo.png", "rb").read(), open("logo_copy.png", "rb").read()
print("identical copy:", a == b, "|", len(a), "bytes")
print("signature:", a[:8])
w, h = struct.unpack(">II", a[16:24])
print(f"image size from the header: {w} x {h} pixels")
```

OUTPUT

```
identical copy: True | 93 bytes
signature: b'\x89PNG\r\n\x1a\n'
image size from the header: 40 x 20 pixels
```

14.6 Paths and the file system

14.6.1 pathlib.Path (preferred)

The `pathlib` module represents file-system paths as `Path` objects instead of plain strings. Paths are joined with the `/` operator, have attributes for their parts, and have methods for most file operations, and the same code works on Windows (`\`) and macOS/Linux (`/`). It is the modern, recommended way to work with paths.

WORKING WITH PATH OBJECTS main.py

```
from pathlib import Path

p = Path("projects") / "lbf" / "reports" / "jan_2026.final.csv"
print(p)
print("name:", p.name, "| stem:", p.stem, "| suffix:", p.suffix, "| suffixes:", p.suffixes)
print("parent:", p.parent, "| parts:", p.parts)
print(p.with_suffix(".json"), "|", p.with_name("feb_2026.csv"), "|", p.with_stem("summary"))

print(Path.cwd().is_absolute(), Path.home().exists())
print(Path("notes.txt").resolve().name, Path("a/b/../c").as_posix())

docs = Path("docs"); docs.mkdir(exist_ok=True)
readme = docs / "README.md"
readme.write_text("# LBF notes\nPython module 14\n", encoding="utf-8")   # open, write, close
print(readme.read_text(encoding="utf-8").splitlines())
print(readme.exists(), readme.is_file(), docs.is_dir(), readme.stat().st_size, "bytes")
```

OUTPUT

```
projects/lbf/reports/jan_2026.final.csv
name: jan_2026.final.csv | stem: jan_2026.final | suffix: .csv | suffixes: ['.final', '.csv']
parent: projects/lbf/reports | parts: ('projects', 'lbf', 'reports', 'jan_2026.final.csv')
projects/lbf/reports/jan_2026.final.json | projects/lbf/reports/feb_2026.csv |
projects/lbf/reports/summary.csv
True True
notes.txt a/b/../c
['# LBF notes', 'Python module 14']
True True True 29 bytes
```

Task	pathlib	os / os.path
Join	`Path("a") / "b.txt"`	`os.path.join("a", "b.txt")`
File name	`p.name`	`os.path.basename(p)`
Folder	`p.parent`	`os.path.dirname(p)`
Extension	`p.suffix`	`os.path.splitext(p)[1]`
Exists?	`p.exists()`	`os.path.exists(p)`
Read text	`p.read_text()`	`open(p).read()`
Make folders	`p.mkdir(parents=True, exist_ok=True)`	`os.makedirs(p, exist_ok=True)`
List	`p.iterdir()`	`os.listdir(p)`
Pattern	`p.glob("*.csv")`	`glob.glob("a/*.csv")`
Delete file	`p.unlink()`	`os.remove(p)`

14.6.2 os and os.path

Before `pathlib`, paths were plain strings handled with the `os.path` functions, and the `os` module provides the underlying operating-system calls. You will meet them in older code and they are still useful for things like environment variables, the current folder and process information.

OS AND OS.PATH main.py

```
import os

path = os.path.join("projects", "lbf", "marks.csv")
print(path, os.path.split(path), os.path.splitext(path))
print(os.path.basename(path), os.path.dirname(path))
print(os.path.isabs(path), os.path.abspath(path).endswith(path))

os.makedirs("projects/lbf", exist_ok=True)
with open(path, "w") as f:
    f.write("name,marks\n")
print(os.path.exists(path), os.path.isfile(path), os.path.isdir("projects"), os.path.getsize(path))

print(os.getcwd() == os.path.abspath("."))
os.environ["APP_MODE"] = "test"
print(os.environ.get("APP_MODE"), os.environ.get("NOT_SET", "default"))
print(os.sep, os.linesep == "\n")
```

OUTPUT

```
projects/lbf/marks.csv ('projects/lbf', 'marks.csv') ('projects/lbf/marks', '.csv')
marks.csv projects/lbf
False True
True True True 11
True
test default
/ True
```

14.6.3 Listing, creating, renaming and deleting files and folders

`Path` covers the everyday operations: `mkdir` creates folders (`parents=True` creates missing parents), `iterdir()` lists a folder's entries, `rename` / `replace` move or rename, `unlink()` deletes a file and `rmdir()` an **empty** folder. Deleting a non-empty folder needs `shutil.rmtree`. Deleted files do not go to the recycle bin, so be careful.

CREATING, LISTING, RENAMING AND DELETING

main.py

```
from pathlib import Path

root = Path("workspace")
(root / "reports" / "2026").mkdir(parents=True, exist_ok=True)
for name in ("a.txt", "b.csv", "c.txt"):
    (root / name).write_text(f"file {name}\n")

print(sorted(p.name for p in root.iterdir()))
print([p.name for p in root.iterdir() if p.is_dir()])

old = root / "a.txt"
new = old.rename(root / "reports" / "alpha.txt")         # move and rename
print("moved:", old.exists(), "->", new)

(root / "b.csv").unlink()                                 # delete a file
(root / "missing.txt").unlink(missing_ok=True)            # no error if absent
try:
    (root / "reports").rmdir()
except OSError as e:
    print("rmdir refuses a non-empty folder:", type(e).__name__)

for p in sorted(root.rglob("*")):                         # walk everything
    kind = "dir " if p.is_dir() else "file"
    print(f"  {kind} {p.relative_to(root)}")
```

OUTPUT

```
['a.txt', 'b.csv', 'c.txt', 'reports']
['reports']
moved: False -> workspace/reports/alpha.txt
rmdir refuses a non-empty folder: OSError
  file c.txt
  dir  reports
  dir  reports/2026
  file reports/alpha.txt
```

`os.walk(top)` walks a folder tree, yielding `(folder, subfolders, files)` for each folder. `Path.walk()` (3.12) is the pathlib equivalent.

WALKING A FOLDER TREE

main.py

```
import os
from pathlib import Path

for d in ("site/css", "site/js", "site/img/icons"):
    Path(d).mkdir(parents=True, exist_ok=True)
for f in ("site/index.html", "site/css/style.css", "site/js/app.js", "site/img/icons/home.svg"):
    Path(f).write_text("x")

for folder, subfolders, files in os.walk("site"):
    subfolders.sort()
    depth = folder.count(os.sep)
    print("  " * depth + os.path.basename(folder) + "/", files)
```

OUTPUT

```
site/ ['index.html']
  css/ ['style.css']
  img/ []
    icons/ ['home.svg']
  js/ ['app.js']
```

14.6.4 shutil for copying and moving

The `shutil` ("shell utilities") module provides high-level operations: `copy` (contents and permissions), `copy2` (also timestamps), `copytree` (a whole folder), `move` (move or rename, across drives too), `rmtree` (delete a folder and everything in it), `make_archive` (zip a folder) and `disk_usage`.

COPYING, MOVING, ARCHIVING AND DELETING WITH SHUTIL main.py

```
import shutil
from pathlib import Path

src = Path("project"); (src / "data").mkdir(parents=True, exist_ok=True)
(src / "main.py").write_text("print('hi')\n")
(src / "data" / "marks.csv").write_text("Asha,91\n")

shutil.copy2(src / "main.py", src / "main_backup.py")
shutil.copytree(src, "project_copy", dirs_exist_ok=True)
shutil.move("project_copy", "archive/project_2026")
print(sorted(str(p) for p in Path("archive").rglob("*.*")))

zip_path = shutil.make_archive("project_backup", "zip", root_dir=src)
print(Path(zip_path).name, Path(zip_path).stat().st_size > 0)

shutil.rmtree("archive")                      # delete the folder tree
print("archive exists:", Path("archive").exists())
total, used, free = shutil.disk_usage(".")
print("free space is a positive number:", free > 0)
```

OUTPUT

```
['archive/project_2026/data/marks.csv', 'archive/project_2026/main.py',
'archive/project_2026/main_backup.py']
project_backup.zip True
archive exists: False
free space is a positive number: True
```

14.6.5 glob patterns

Glob patterns select files by name using wildcards: `*` matches any characters within a name, `?` one character, `[abc]` one of the listed characters, and `**` any number of folders (recursive). Use `Path.glob` / `Path.rglob` or the `glob` module.

Pattern	Matches
`*.csv`	every .csv file in the folder
`report_202?.txt`	report_2020.txt … report_2029.txt
`[ab]*.py`	.py files starting with a or b
`**/*.py`	.py files in this folder and all sub-folders
`img/*.{png,jpg}`	not supported: run two globs instead

SELECTING FILES WITH GLOB PATTERNS main.py

```
from pathlib import Path
import glob

for name in ("jan.csv", "feb.csv", "notes.txt", "report_2025.txt", "report_2026.txt",
             "src/app.py", "src/utils/io.py", "setup.py"):
    Path("data", name).parent.mkdir(parents=True, exist_ok=True)
    Path("data", name).write_text("")

data = Path("data")
print(sorted(p.name for p in data.glob("*.csv")))
print(sorted(p.name for p in data.glob("report_202?.txt")))
print(sorted(str(p.relative_to(data)) for p in data.glob("**/*.py")))
print(sorted(p.name for p in data.rglob("*.py")))           # same as **/
print(sorted(glob.glob("data/*.txt")))
```

OUTPUT

```
['feb.csv', 'jan.csv']
['report_2025.txt', 'report_2026.txt']
['setup.py', 'src/app.py', 'src/utils/io.py']
['app.py', 'io.py', 'setup.py']
['data/notes.txt', 'data/report_2025.txt', 'data/report_2026.txt']
```

14.7 Data formats

14.7.1 CSV with csv.reader, csv.writer, DictReader, DictWriter

CSV (comma-separated values) is the most common format for tabular data and opens directly in Excel and Google Sheets. Do not split lines on commas yourself: values can contain commas, quotes and line breaks inside quotes. The `csv` module handles all of that. Open CSV files with `newline=""` so the module controls line endings.

CSV.WRITER AND CSV.READER

main.py

```
import csv

rows = [["name", "city", "marks"],
        ["Asha", "Mysuru", 91],
        ["Ravi", "Pune, MH", 78],              # contains a comma
        ["Meera", 'said "hi"', 85]]            # contains quotes

with open("marks.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.writer(f)
    writer.writerows(rows)
print(open("marks.csv", encoding="utf-8").read())

with open("marks.csv", newline="", encoding="utf-8") as f:
    reader = csv.reader(f)
    header = next(reader)                      # first row
    for row in reader:
        print(row)                             # every value is a string
```

OUTPUT

```
name,city,marks
Asha,Mysuru,91
Ravi,"Pune, MH",78
Meera,"said ""hi""",85

['Asha', 'Mysuru', '91']
['Ravi', 'Pune, MH', '78']
['Meera', 'said "hi"', '85']
```

`DictReader` uses the header row as keys and returns each row as a dictionary, which makes code independent of column order. `DictWriter` writes dictionaries, given the list of field names. Both accept a `delimiter` for files separated by semicolons or tabs.

DICTWRITER, DICTREADER AND OTHER DELIMITERS

main.py

```
import csv

students = [{"name": "Asha", "marks": 91, "city": "Mysuru"},
            {"name": "Ravi", "marks": 78, "city": "Pune"},
            {"name": "Meera", "marks": 85, "city": "Mysuru"}]

with open("students.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.DictWriter(f, fieldnames=["name", "city", "marks"])
    writer.writeheader()
    writer.writerows(students)

with open("students.csv", newline="", encoding="utf-8") as f:
    for row in csv.DictReader(f):
        print(f"{row['name']:<6} {row['city']:<7} {int(row['marks']) + 5}")

with open("students.tsv", "w", newline="", encoding="utf-8") as f:
    csv.writer(f, delimiter="\t").writerow(["Asha", 91])
print(repr(open("students.tsv").read()))
```

OUTPUT

```
Asha   Mysuru  96
Ravi   Pune    83
Meera  Mysuru  90
'Asha\t91\n'
```

WATCH OUT

CSV has no types: every value read back is a string, so convert numbers with `int()` or `float()`. For large or complex tabular data, pandas (Module 24) reads CSV into typed columns in one line.

14.7.2 JSON with json.load, json.dump, loads, dumps

JSON (JavaScript Object Notation) is the standard format for web APIs and configuration, and maps naturally onto Python: objects ↔ `dict`, arrays ↔ `list`, strings, numbers, `true`/`false` ↔ `True`/`False` and `null` ↔ `None`. The functions with an **s** work with strings (`dumps`, `loads`); those without work with files (`dump`, `load`).

Function	Direction	Works with
`json.dumps(obj)`	Python → JSON	returns a str
`json.loads(text)`	JSON → Python	takes a str
`json.dump(obj, f)`	Python → JSON	writes to a file
`json.load(f)`	JSON → Python	reads from a file

CONVERTING BETWEEN PYTHON AND JSON

main.py

```
import json

course = {"name": "Python", "modules": 25, "levels": ["Beginner", "Intermediate", "Advanced"],
          "online": True, "mentor": None, "fee": 14999.0, "city": "Bengaluru"}

text = json.dumps(course)
print(text)
print(json.dumps(course, indent=2, sort_keys=True)[:120], "...")

data = json.loads('{"roll": 101, "marks": [91, 88], "passed": true, "remarks": null}')
print(data, type(data["passed"]).__name__)

with open("course.json", "w", encoding="utf-8") as f:
    json.dump(course, f, indent=2, ensure_ascii=False)
with open("course.json", encoding="utf-8") as f:
    loaded = json.load(f)
print(loaded == course)

print(json.dumps((1, 2)), json.loads(json.dumps({1: "one"})))  # tuple -> list, int key -> str
try:
    json.dumps({"when": {1, 2}})
except TypeError as e:
    print("TypeError:", e)
```

OUTPUT

```
{"name": "Python", "modules": 25, "levels": ["Beginner", "Intermediate", "Advanced"], "online": true,
"mentor": null, "fee": 14999.0, "city": "Bengaluru"}
{
  "city": "Bengaluru",
  "fee": 14999.0,
  "levels": [
    "Beginner",
    "Intermediate",
    "Advanced"
  ],
  "ment ...
{'roll': 101, 'marks': [91, 88], 'passed': True, 'remarks': None} bool
True
[1, 2] {'1': 'one'}
TypeError: Object of type set is not JSON serializable
```

Types JSON cannot represent (sets, dates, custom objects) need help: pass `default=` to `dumps` with a function that converts them, or convert them yourself first.

SERIALISING DATES AND SETS WITH DEFAULT=

main.py

```
import json
from datetime import date

record = {"student": "Asha", "joined": date(2026, 1, 5), "skills": {"python", "sql"}}

def to_json(obj):
    if isinstance(obj, date):
        return obj.isoformat()
    if isinstance(obj, set):
        return sorted(obj)
    raise TypeError(f"cannot serialise {type(obj).__name__}")

text = json.dumps(record, default=to_json)
print(text)
back = json.loads(text)
back["joined"] = date.fromisoformat(back["joined"])
print(back)
```

OUTPUT

```
{"student": "Asha", "joined": "2026-01-05", "skills": ["python", "sql"]}
{'student': 'Asha', 'joined': datetime.date(2026, 1, 5), 'skills': ['python', 'sql']}
```

14.7.3 Pickle and its security risks

`pickle` serialises almost any Python object (custom classes, sets, nested structures) to bytes and restores it exactly. It is convenient for caching your own data between runs. But pickle data can contain instructions that run **arbitrary code** when loaded, so **never unpickle data from an untrusted source**: a malicious file could delete files or install malware. Pickle is also Python-only and can break between versions of your classes. Prefer JSON for data exchange.

PICKLE ROUND TRIP, AND WHY UNTRUSTED PICKLES ARE DANGEROUS

main.py

```
import pickle
from datetime import date

state = {"user": "asha", "visited": {"m11", "m12", "m13"}, "last": date(2026, 1, 26), "scores": (91,
88)}
with open("state.pkl", "wb") as f:
    pickle.dump(state, f)
with open("state.pkl", "rb") as f:
    restored = pickle.load(f)
print(restored == state, type(restored["visited"]).__name__, restored["last"])

class Evil:                                  # what an attacker can do
    def __reduce__(self):
        return (print, ("!!! this code ran just by loading the pickle !!!",))

payload = pickle.dumps(Evil())
pickle.loads(payload)                        # imagine os.system(...) instead of print
```

OUTPUT

```
True set 2026-01-26
!!! this code ran just by loading the pickle !!!
```

WATCH OUT

Treat `pickle.load` on downloaded or user-supplied files like running an unknown program. The same caution applies to libraries built on pickle, such as `joblib` and older machine-learning model files.

14.7.4 Excel files with openpyxl (overview)

`openpyxl` (installed with `pip install openpyxl`) reads and writes Excel `.xlsx` workbooks: sheets, cell values, formulas, styles and column widths. A workbook contains worksheets; cells are addressed as `ws["B2"]` or `ws.cell(row=2, column=2)`. For data analysis, pandas' `read_excel` and `to_excel` (Module 24) use openpyxl underneath.

WRITING AND READING AN EXCEL WORKBOOK main.py

```
from openpyxl import Workbook, load_workbook
from openpyxl.styles import Font

wb = Workbook()
ws = wb.active
ws.title = "Marks"
ws.append(["Name", "Marks"])
for row in [("Asha", 91), ("Ravi", 78), ("Meera", 85)]:
    ws.append(row)
ws["A5"] = "Average"
ws["B5"] = "=AVERAGE(B2:B4)"           # an Excel formula
ws["A1"].font = ws["B1"].font = Font(bold=True)
ws.column_dimensions["A"].width = 12
wb.save("marks.xlsx")

wb = load_workbook("marks.xlsx")
ws = wb["Marks"]
for name, marks in ws.iter_rows(min_row=2, max_row=4, values_only=True):
    print(name, marks)
print(ws["B5"].value)                  # the formula text; Excel computes the value
```

OUTPUT

```
Asha 91
Ravi 78
Meera 85
=AVERAGE(B2:B4)
```

14.7.5 Config files with configparser and TOML (tomllib)

Settings belong in configuration files rather than in code. `configparser` reads and writes INI-style files with `[sections]` and `key = value` lines; all values are strings, with helpers such as `getint` and `getboolean`. **TOML** is the modern format used by `pyproject.toml`; it has real types (numbers, booleans, dates, arrays, tables). `tomllib` (3.11) **reads** TOML; writing needs a third-party package such as `tomli-w`.

SETTINGS.INI text

```
[app]
name = LBF Portal
debug = yes
port = 8080

[database]
host = localhost
user = admin
```

READING AND WRITING INI FILES WITH CONFIGPARSER

main.py

```
import configparser

config = configparser.ConfigParser()
config.read("settings.ini", encoding="utf-8")
print(config.sections())
print(config["app"]["name"], config.getint("app", "port") + 1, config.getboolean("app", "debug"))
print(config.get("database", "password", fallback="(not set)"))

config["database"]["user"] = "asha"
config["logging"] = {"level": "INFO", "file": "app.log"}
with open("settings.ini", "w", encoding="utf-8") as f:
    config.write(f)
print(open("settings.ini", encoding="utf-8").read())
```

OUTPUT

```
['app', 'database']
LBF Portal 8081 True
(not set)
[app]
name = LBF Portal
debug = yes
port = 8080

[database]
host = localhost
user = asha

[logging]
level = INFO
file = app.log
```

PYPROJECT.TOML

text

```
[project]
name = "marks-report"
version = "1.2.0"
requires-python = ">=3.11"
dependencies = ["tabulate>=0.9"]

[tool.report]
passing_mark = 40
subjects = ["maths", "science"]
released = 2026-01-26
```

READING TOML WITH TOMLLIB

main.py

```
import tomllib

with open("pyproject.toml", "rb") as f:        # tomllib needs binary mode
    data = tomllib.load(f)

print(data["project"]["name"], data["project"]["version"])
report = data["tool"]["report"]
print(report["passing_mark"] + 10, report["subjects"], type(report["released"]).__name__)
print(tomllib.loads('enabled = true\nratio = 0.75')["ratio"] * 2)
```

OUTPUT

```
marks-report 1.2.0
50 ['maths', 'science'] date
1.5
```

Format	Strengths	Typical use
CSV	simple, opens in spreadsheets	tables of records
JSON	nested data, universal, typed basics	web APIs, data exchange, settings
TOML	readable, typed, comments allowed	project and app configuration
INI	very simple sections and keys	legacy and simple settings
Excel (.xlsx)	formatting, formulas, sheets	reports for non-programmers
pickle	any Python object, exact	private caches only (never untrusted)

Open files with open(path, mode): r reads, w creates or erases, a appends, x creates only if new, + adds the other direction, b switches to bytes. Always use with so files close automatically, and pass encoding="utf-8" for text. read, readline and readlines read text; iterating with for line in f streams huge files line by line, and chunked reads handle binary data. write and writelines do not add newlines; print(file=f) does. tell and seek report and move the position. Binary mode reads and writes bytes, e.g. to copy images. pathlib.Path joins paths with /, exposes name, stem, suffix and parent, and reads, writes, lists (iterdir, glob, rglob), renames and deletes; os and os.path are the older string-based equivalents; shutil copies, moves, archives and deletes trees. Use csv (reader, writer, DictReader, DictWriter, newline="") for CSV, json (dump, load, dumps, loads) for JSON, pickle only for trusted data, openpyxl for Excel, and configparser or tomllib for configuration.

PRACTICE CHECKPOINT

Build a **student marks manager** that stores records in CSV, exports a summary to JSON, and **organises files in a folder by extension** using pathlib.

PART 1: MARKS IN CSV, SUMMARY IN JSON

main.py

```
import csv, json
from pathlib import Path

CSV_FILE = Path("marks.csv")
FIELDS = ["roll", "name", "maths", "science", "english"]

def add_student(roll, name, maths, science, english):
    new_file = not CSV_FILE.exists()
    with CSV_FILE.open("a", newline="", encoding="utf-8") as f:
        writer = csv.DictWriter(f, fieldnames=FIELDS)
        if new_file:
            writer.writeheader()
        writer.writerow({"roll": roll, "name": name, "maths": maths, "science": science, "english":
english})

def load_students():
    with CSV_FILE.open(newline="", encoding="utf-8") as f:
        return [{**row, **{s: int(row[s]) for s in FIELDS[2:]}} for row in csv.DictReader(f)]

def export_summary(path):
    students = load_students()
    for s in students:
        s["total"] = sum(s[sub] for sub in FIELDS[2:])
        s["average"] = round(s["total"] / 3, 1)
    summary = {
        "students": len(students),
        "class_average": round(sum(s["average"] for s in students) / len(students), 1),
        "topper": max(students, key=lambda s: s["total"])["name"],
        "subject_averages": {sub: round(sum(s[sub] for s in students) / len(students), 1) for sub in
FIELDS[2:]},
        "ranking": [s["name"] for s in sorted(students, key=lambda s: -s["total"])],
    }
    Path(path).write_text(json.dumps(summary, indent=2), encoding="utf-8")
    return summary

for record in [(101, "Asha", 91, 88, 79), (102, "Ravi", 78, 82, 90), (103, "Meera", 85, 95, 88), (104,
"John", 64, 58, 71)]:
    add_student(*record)

print(CSV_FILE.read_text(encoding="utf-8"))
export_summary("summary.json")
print(Path("summary.json").read_text(encoding="utf-8"))
```

OUTPUT

```
roll,name,maths,science,english
101,Asha,91,88,79
102,Ravi,78,82,90
103,Meera,85,95,88
104,John,64,58,71

{
  "students": 4,
  "class_average": 80.7,
  "topper": "Meera",
  "subject_averages": {
    "maths": 79.5,
    "science": 80.8,
    "english": 82.0
  },
  "ranking": [
    "Meera",
    "Asha",
    "Ravi",
    "John"
  ]
}
```

PART 2: ORGANISE A FOLDER BY FILE EXTENSION

main.py

```
from pathlib import Path

downloads = Path("downloads")
downloads.mkdir(exist_ok=True)
for name in ("photo.JPG", "notes.pdf", "song.mp3", "marks.csv", "logo.png", "resume.pdf",
             "script.py", "archive.zip", "README", "data.CSV"):
    (downloads / name).write_text("sample")

GROUPS = {"Images": {".jpg", ".png", ".gif"}, "Documents": {".pdf", ".docx", ".txt"},
          "Data": {".csv", ".json", ".xlsx"}, "Audio": {".mp3", ".wav"}, "Code": {".py", ".js"}}

def folder_for(path):
    ext = path.suffix.lower()
    for folder, extensions in GROUPS.items():
        if ext in extensions:
            return folder
    return "Others"

moved = {}
for item in sorted(downloads.iterdir()):
    if item.is_file():
        target_dir = downloads / folder_for(item)
        target_dir.mkdir(exist_ok=True)
        item.rename(target_dir / item.name)
        moved.setdefault(target_dir.name, []).append(item.name)

for folder in sorted(moved):
    print(f"{folder + '/':<11} {', '.join(moved[folder])}")
print("files left at the top level:", [p.name for p in downloads.iterdir() if p.is_file()])
```

OUTPUT

```
Audio/      song.mp3
Code/       script.py
Data/       data.CSV, marks.csv
Documents/  notes.pdf, resume.pdf
Images/     logo.png, photo.JPG
Others/     README, archive.zip
files left at the top level: []
```

MODULE 15 · INTERMEDIATE

Object-oriented programming: classes and objects

Minimum duration **3 days** | Level **Intermediate**

In Python everything is an object: numbers, strings, lists, functions and modules all have a type, data and methods. This module shows how to design your own types with classes. It covers procedural vs object-oriented programming and the four pillars, defining classes and creating instances, __init__ and self, instance and class attributes and how they shadow each other, __dict__ and __slots__, instance, class and static methods (including alternative constructors), encapsulation with naming conventions, name mangling, getters/setters and @property, __str__ and __repr__, the object lifecycle (__new__, __init__, __del__ and garbage collection), and association, aggregation and composition between classes.

IN THIS MODULE

15.1 OOP concepts
15.2 Classes and objects
15.3 Attributes
15.4 Methods
15.5 Encapsulation
15.6 Object representation
15.7 Object lifecycle
15.8 Relationships between classes

15.1 OOP concepts

15.1.1 Procedural vs object-oriented programming

In **procedural** programming, data and the functions that work on it are separate: a dictionary holds a bank account, and free functions such as `deposit(account, amount)` change it. Nothing stops other code from changing the dictionary directly or passing the wrong kind of dictionary. In **object-oriented programming** (OOP), data and behaviour are bundled into **objects**: an account object holds its balance and knows how to deposit and withdraw, guarding its own rules. A **class** is the blueprint; **objects** (instances) are the things built from it.

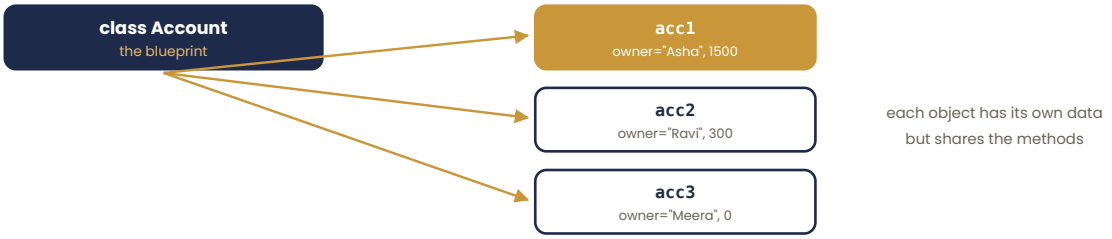
THE SAME IDEA, PROCEDURAL AND OBJECT-ORIENTED

main.py

```
# procedural: data and functions are separate
account = {"owner": "Asha", "balance": 1000}

def deposit(acc, amount):
    acc["balance"] += amount

deposit(account, 500)
account["balance"] = -99999          # nothing protects the data
print(account)

# object-oriented: data and behaviour live together
class Account:
    def __init__(self, owner, balance=0):
        self.owner = owner
        self.balance = balance

    def deposit(self, amount):
        if amount <= 0:
            raise ValueError("deposit must be positive")
        self.balance += amount

acc = Account("Asha", 1000)
acc.deposit(500)
print(acc.owner, acc.balance)
try:
    acc.deposit(-50)
except ValueError as e:
    print("rule enforced:", e)
```

OUTPUT

```
{'owner': 'Asha', 'balance': -99999}
Asha 1500
rule enforced: deposit must be positive
```



ONE CLASS, MANY OBJECTS

In Python you have used objects from the very first module: `"hello".upper()` calls the `upper` method of a `str` object, and `marks.append(91)` calls a method of a `list` object. Classes let you create new types that work the same way.

EVERYTHING IN PYTHON IS AN OBJECT main.py

```
for value in (42, "hi", [1, 2], {"a": 1}, len, None):
    print(f"{value!r:<26} type: {type(value).__name__:<27} is an object: {isinstance(value, object)}")
print((5).bit_length(), "lbf".upper(), [3, 1, 2].count(1))
```

OUTPUT

```
42                         type: int                         is an object: True
'hi'                       type: str                         is an object: True
[1, 2]                     type: list                        is an object: True
{'a': 1}                   type: dict                        is an object: True
<built-in function len>    type: builtin_function_or_method  is an object: True
None                       type: NoneType                    is an object: True
3 LBF 1
```

15.1.2 The four pillars: encapsulation, abstraction, inheritance, polymorphism

Encapsulation

Keep data and the methods that use it together, and control access, so an object protects its own state (15.5).

Abstraction

Expose a simple interface and hide the details: you call car.start() without knowing how the engine works.

Inheritance

Build a new class from an existing one, reusing and extending it: a SavingsAccount is an Account (Module 16).

Polymorphism

One interface, many forms: different classes respond to the same method call in their own way (Module 16).

OOP pays off when a program models many things with state and behaviour: users, orders, game characters, GUI widgets, database records. For small scripts and data transformations, plain functions are often simpler; Python lets you mix both styles freely.

15.2 Classes and objects

15.2.1 Defining a class with class

A class is defined with the `class` keyword, a name in **CapWords** (PascalCase) style and an indented body containing attributes and method definitions. The simplest class has just `pass` or a docstring. Defining the class creates a **class object**; it does not create any instances yet.

THE SIMPLEST CLASS main.py

```
class Student:
    """A student enrolled at Learn Being Forward."""

print(Student)
print(type(Student), Student.__name__, Student.__doc__)
```

OUTPUT

```
<class '__main__.Student'>
<class 'type'> Student A student enrolled at Learn Being Forward.
```

15.2.2 Creating objects (instances)

Calling the class like a function, `Student()`, creates a new **instance**. Each call produces a distinct object with its own identity, even if it holds the same data. `isinstance(obj, Class)` checks the type.

CREATING INSTANCES main.py

```
class Student:
    pass

s1 = Student()
s2 = Student()
print(s1)
print(s1 is s2, type(s1).__name__, isinstance(s1, Student))

s1.name = "Asha"            # attributes can be attached to an instance
s2.name = "Ravi"
print(s1.name, s2.name)
```

OUTPUT

```
<__main__.Student object at 0x7fd420a846e0>
False Student True
Asha Ravi
```

Attaching attributes from outside, as above, works but is fragile: nothing guarantees every `Student` has a `name`. The `__init__` method solves this.

15.2.3 The __init__ method

`__init__` is a special method ("dunder" method, for double underscore) that Python calls automatically right after creating a new instance, passing it the arguments given to the class call. It **initialises** the object, typically by assigning instance attributes. It must return `None`. Use it to make sure every object starts in a valid state, and validate its arguments there.

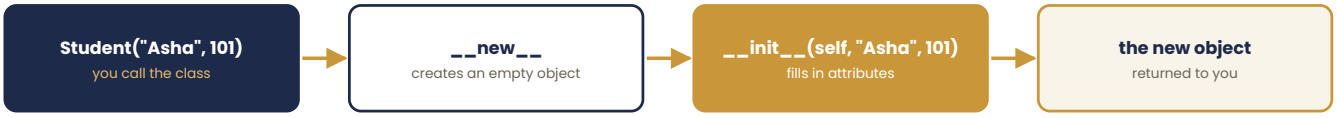
INITIALISING OBJECTS WITH __INIT__

main.py

```
class Student:
    def __init__(self, name, roll, marks=None):
        if not name:
            raise ValueError("name is required")
        self.name = name
        self.roll = roll
        self.marks = marks if marks is not None else []    # avoid a mutable default

asha = Student("Asha", 101, [91, 88])
ravi = Student("Ravi", 102)
print(asha.name, asha.roll, asha.marks)
print(ravi.name, ravi.roll, ravi.marks)
try:
    Student("", 103)
except ValueError as e:
    print("ValueError:", e)
try:
    Student("Meera")
except TypeError as e:
    print("TypeError:", e)
```

OUTPUT

```
Asha 101 [91, 88]
Ravi 102 []
ValueError: name is required
TypeError: Student.__init__() missing 1 required positional argument: 'roll'
```



WHAT HAPPENS WHEN YOU CALL A CLASS

15.2.4 The self parameter

Every instance method receives the object it was called on as its **first parameter**, called `self` by strong convention. `asha.describe()` is translated by Python into `Student.describe(asha)`, so inside the method `self` **is** `asha`. Through `self` a method reads and changes that particular object's attributes and calls its other methods. Forgetting `self` in a method definition is a very common beginner error.

SELF IS THE OBJECT THE METHOD WAS CALLED ON

main.py

```
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

    def average(self):
        return sum(self.marks) / len(self.marks)

    def describe(self):
        return f"{self.name}: average {self.average():.1f}"   # calls another method via self

asha = Student("Asha", [91, 88, 79])
print(asha.describe())
print(Student.describe(asha))          # what Python really does
print(asha.describe.__self__ is asha)

class Broken:
    def hello():                       # forgot self
        return "hi"
try:
    Broken().hello()
except TypeError as e:
    print("TypeError:", e)
```

OUTPUT

```
Asha: average 86.0
Asha: average 86.0
True
TypeError: Broken.hello() takes 0 positional arguments but 1 was given
```

THE OBJECT BEFORE THE DOT BECOMES SELF

15.3 Attributes

15.3.1 Instance attributes

Instance attributes belong to one object. They are normally created in `__init__` by assigning to `self.name`, and each object has its own values. They can be read and changed with dot notation from inside and outside the class.

INSTANCE ATTRIBUTES main.py

```
class Product:
    def __init__(self, name, price, stock=0):
        self.name = name
        self.price = price
        self.stock = stock

pen = Product("pen", 10, 100)
bag = Product("bag", 899)
pen.price = 12                      # change one object's attribute
print(pen.name, pen.price, pen.stock, "|", bag.name, bag.price, bag.stock)
print(getattr(bag, "price"), getattr(bag, "colour", "n/a"), hasattr(pen, "stock"))
setattr(bag, "stock", 5)
print(bag.stock)
```

OUTPUT

```
pen 12 100 | bag 899 0
899 n/a True
5
```

15.3.2 Class attributes

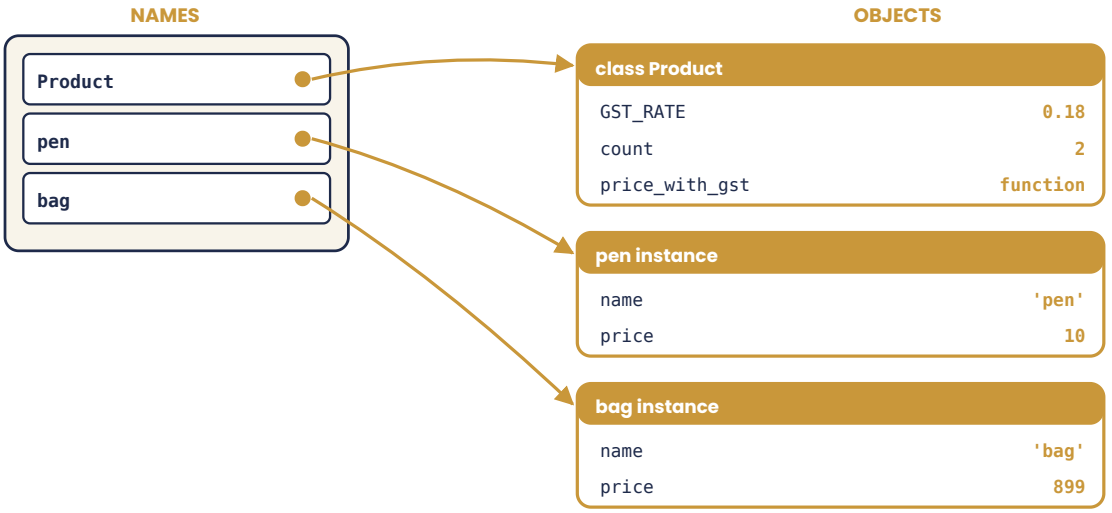
Class attributes are defined directly in the class body, outside any method. They belong to the class and are **shared by all instances**: constants such as a tax rate, default values, or counters of how many objects have been created. Read them through the class (`Product.GST_RATE`) or through any instance.

CLASS ATTRIBUTES ARE SHARED main.py

```
class Product:
    GST_RATE = 0.18                 # class attribute: shared
    count = 0                       # how many products exist

    def __init__(self, name, price):
        self.name = name            # instance attributes: per object
        self.price = price
        Product.count += 1          # update the shared counter via the class

    def price_with_gst(self):
        return round(self.price * (1 + self.GST_RATE), 2)

pen, bag = Product("pen", 10), Product("bag", 899)
print(Product.GST_RATE, pen.GST_RATE, bag.GST_RATE)
print(pen.price_with_gst(), bag.price_with_gst())
print("products created:", Product.count)
Product.GST_RATE = 0.12             # change it for everyone
print(pen.price_with_gst(), bag.price_with_gst())
```

OUTPUT

```
0.18 0.18 0.18
11.8 1060.82
products created: 2
11.2 1006.88
```



INSTANCES HOLD THEIR OWN DATA; SHARED ATTRIBUTES AND METHODS LIVE ON THE CLASS

15.3.3 Instance vs class attributes and shadowing

When you read `obj.attr`, Python looks first in the **instance**, then in its **class** (and then base classes, Module 16). **Assigning** `obj.attr = value`, however, always creates or changes an **instance** attribute. So assigning to a class attribute through an instance does not change the class: it creates an instance attribute that **shadows** it for that object only. To change shared data, assign through the class.

AN INSTANCE ATTRIBUTE SHADOWS A CLASS ATTRIBUTE main.py

```
class Product:
    GST_RATE = 0.18

pen, bag = Product(), Product()
pen.GST_RATE = 0.05                  # creates an INSTANCE attribute on pen only
print(pen.GST_RATE, bag.GST_RATE, Product.GST_RATE)
print(pen.__dict__, bag.__dict__)
del pen.GST_RATE                     # remove the shadow
print(pen.GST_RATE)
```

OUTPUT

```
0.05 0.18 0.18
{'GST_RATE': 0.05} {}
0.18
```

A classic bug follows from sharing: a **mutable** class attribute such as a list is shared by every instance, so appending through one instance changes it for all of them. Create per-object lists in `__init__`.

THE SHARED MUTABLE CLASS ATTRIBUTE TRAP main.py

```
class Classroom:
    students = []                    # shared by ALL classrooms: a bug

    def add(self, name):
        self.students.append(name)   # mutates the class-level list

a, b = Classroom(), Classroom()
a.add("Asha")
b.add("Ravi")
print(a.students, b.students, a.students is b.students)

class Classroom:
    def __init__(self):
        self.students = []           # one list per object

    def add(self, name):
        self.students.append(name)

a, b = Classroom(), Classroom()
a.add("Asha"); b.add("Ravi")
print(a.students, b.students)
```

OUTPUT

```
['Asha', 'Ravi'] ['Asha', 'Ravi'] True
['Asha'] ['Ravi']
```

15.3.4 __dict__ and dynamic attributes

Each ordinary instance stores its attributes in a dictionary, `obj.__dict__`; the class has its own `__dict__` (a read-only mapping proxy) holding class attributes and methods. Because it is a dictionary, attributes can be added and removed at run time (`setattr`, `delattr`, `del obj.attr`), and `vars(obj)` returns it. This flexibility is handy, but adding attributes outside `__init__` makes classes hard to understand; editors and type checkers warn about it.

INSTANCE AND CLASS DICTIONARIES main.py

```
class Student:
    school = "LBF"
    def __init__(self, name):
        self.name = name

s = Student("Asha")
print(s.__dict__, vars(s))
s.city = "Mysuru"                    # dynamic attribute
setattr(s, "roll", 101)
print(s.__dict__)
del s.city
print(s.__dict__)
print([k for k in Student.__dict__ if not k.startswith("__")])
print("school" in s.__dict__, s.school)  # found on the class, not the instance
```

OUTPUT

```
{'name': 'Asha'} {'name': 'Asha'}
{'name': 'Asha', 'city': 'Mysuru', 'roll': 101}
{'name': 'Asha', 'roll': 101}
['school']
False LBF
```

15.3.5 __slots__ (overview)

Declaring `__slots__ = ("name", "roll")` in a class replaces the per-instance `__dict__` with fixed storage for exactly those attributes. Instances then use noticeably less memory and attribute access is slightly faster, and assigning any attribute not listed raises `AttributeError`, catching typos. It is worth using for classes with millions of instances; most classes do not need it. `@dataclass(slots=True)` (Module 19) adds slots automatically.

__SLOTS__ SAVES MEMORY AND BLOCKS UNKNOWN ATTRIBUTES main.py

```
import sys

class PointDict:
    def __init__(self, x, y):
        self.x, self.y = x, y

class PointSlots:
    __slots__ = ("x", "y")
    def __init__(self, x, y):
        self.x, self.y = x, y

a, b = PointDict(1, 2), PointSlots(1, 2)
print(hasattr(a, "__dict__"), hasattr(b, "__dict__"))
size_a = sys.getsizeof(a) + sys.getsizeof(a.__dict__)
size_b = sys.getsizeof(b)
print("slots object is smaller:", size_b < size_a, f"({size_b} vs {size_a} bytes)")
try:
    b.z = 3                             # not listed in __slots__
except AttributeError as e:
    print("AttributeError:", e)
```

OUTPUT

```
True False
slots object is smaller: True (48 vs 344 bytes)
AttributeError: 'PointSlots' object has no attribute 'z' and no __dict__ for setting new attributes
```

15.4 Methods

A class can define three kinds of methods, which differ in what they receive automatically as their first argument:

Kind	Decorator	First parameter	Can access	Typical use
Instance method	none	`self` (the object)	instance and class data	behaviour of one object
Class method	`@classmethod`	`cls` (the class)	class data, can create instances	alternative constructors, class-wide counters
Static method	`@staticmethod`	nothing	only its arguments	helpers related to the class

15.4.1 Instance methods

Instance methods are ordinary functions defined in the class body with `self` as the first parameter. They describe what an object can do, usually reading or updating its attributes. Methods that change the object often return `None` (like `list.append`), or return `self` to allow **chaining**.

INSTANCE METHODS AND METHOD CHAINING

main.py

```
class Cart:
    def __init__(self):
        self.items = {}

    def add(self, item, price, qty=1):
        self.items[item] = (price, self.items.get(item, (price, 0))[1] + qty)
        return self                              # allow chaining

    def remove(self, item):
        self.items.pop(item, None)
        return self

    def total(self):
        return sum(price * qty for price, qty in self.items.values())

    def receipt(self):
        lines = [f"{name:<6} {qty} x {price:>4} = {price * qty:>5}" for name, (price, qty) in
self.items.items()]
        return "\n".join(lines + [f"{'TOTAL':<17}{self.total():>6}"])

cart = Cart().add("pen", 10, 3).add("book", 250).add("bag", 899).remove("bag").add("pen", 10)
print(cart.receipt())
```

OUTPUT

```
pen    4 x   10 =    40
book   1 x  250 =   250
TOTAL               290
```

15.4.2 Class methods @classmethod and alternative constructors

A method decorated with `@classmethod` receives the **class** as its first argument, conventionally `cls`, whether called on the class or on an instance. Its main use is **alternative constructors**: extra ways to create objects, such as from a string, a dictionary or a file line, that end with `return cls(...)`. Using `cls` instead of the class name means subclasses automatically get objects of the right type. The standard library does this too: `dict.fromkeys`, `datetime.fromisoformat`, `int.from_bytes`.

ALTERNATIVE CONSTRUCTORS WITH @CLASSMETHOD

main.py

```
from datetime import date

class Student:
    count = 0

    def __init__(self, name, roll, born):
        self.name, self.roll, self.born = name, roll, born
        Student.count += 1

    @classmethod
    def from_csv(cls, line):                    # "Asha,101,2005-03-14"
        name, roll, born = line.strip().split(",")
        return cls(name, int(roll), date.fromisoformat(born))

    @classmethod
    def from_dict(cls, data):
        return cls(data["name"], data["roll"], date.fromisoformat(data["born"]))

    @classmethod
    def how_many(cls):
        return f"{cls.count} {cls.__name__} objects created"

    def age_on(self, day):
        return day.year - self.born.year - ((day.month, day.day) < (self.born.month, self.born.day))

a = Student("Asha", 101, date(2005, 3, 14))
b = Student.from_csv("Ravi,102,2004-11-30\n")
c = Student.from_dict({"name": "Meera", "roll": 103, "born": "2006-01-01"})
for s in (a, b, c):
    print(s.name, s.roll, s.age_on(date(2026, 1, 26)))
print(Student.how_many(), "|", a.how_many())
```

OUTPUT

```
Asha 101 20
Ravi 102 21
Meera 103 20
3 Student objects created | 3 Student objects created
```

15.4.3 Static methods @staticmethod

A `@staticmethod` receives no automatic first argument. It is a plain function placed inside the class because it belongs there logically, typically a helper or validation function that does not need any object or class data. If it needs neither, a module-level function is an equally good choice.

STATIC METHODS, AND ALL THREE KINDS TOGETHER

main.py

```
class Validator:
    @staticmethod
    def is_valid_email(email):
        user, _, domain = email.partition("@")
        return bool(user) and "." in domain

    @staticmethod
    def is_valid_phone(phone):
        digits = phone.replace(" ", "").removeprefix("+91")
        return digits.isdigit() and len(digits) == 10

print(Validator.is_valid_email("asha@lbf.in"), Validator.is_valid_email("asha.lbf.in"))
print(Validator.is_valid_phone("+91 98450 12345"), Validator().is_valid_phone("12345"))

class Temperature:
    def __init__(self, celsius):
        self.celsius = celsius

    @staticmethod
    def c_to_f(c):                          # pure helper
        return c * 9 / 5 + 32

    @classmethod
    def from_fahrenheit(cls, f):            # needs the class: classmethod
        return cls((f - 32) * 5 / 9)

    def fahrenheit(self):                   # needs the object: instance method
        return self.c_to_f(self.celsius)

t = Temperature.from_fahrenheit(98.6)
print(round(t.celsius, 1), t.fahrenheit(), Temperature.c_to_f(100))
```

OUTPUT

```
True False
True False
37.0 98.6 212.0
```

15.5 Encapsulation

15.5.1 Public, protected _x and private __x (name mangling)

Python has no `private` keyword; access control is by **naming convention**, based on the principle "we are all consenting adults". A plain name (`balance`) is **public**. A single leading underscore (`_balance`) marks an attribute as **internal** ("protected"): it still works from outside, but signals "do not touch, this may change". A double leading underscore (`__balance`, without trailing underscores) triggers **name mangling**: Python renames it to `_ClassName__balance`, which prevents accidental access and, mainly, name clashes in subclasses. It is not real security.

Name	Meaning	Accessible from outside?
`balance`	public interface	yes
`_balance`	internal by convention	yes, but you should not
`__balance`	private: name-mangled to `_Account__balance`	not by that name

Name	Meaning	Accessible from outside?
`__init__`	dunder: special method defined by Python	yes (not mangled)

NAMING CONVENTIONS AND NAME MANGLING

main.py

```
class Account:
    def __init__(self, owner, balance):
        self.owner = owner              # public
        self._branch = "Mysuru"         # internal by convention
        self.__balance = balance        # private: name-mangled

    def balance(self):
        return self.__balance           # works inside the class

acc = Account("Asha", 5000)
print(acc.owner, acc._branch, acc.balance())
try:
    print(acc.__balance)
except AttributeError as e:
    print("AttributeError:", e)
print(acc.__dict__)
print(acc._Account__balance)            # still reachable: not real security
```

OUTPUT

```
Asha Mysuru 5000
AttributeError: 'Account' object has no attribute '__balance'
{'owner': 'Asha', '_branch': 'Mysuru', '_Account__balance': 5000}
5000
```

15.5.2 Getters and setters

To control how an attribute is read and changed, for example to validate new values, many languages use **getter** and **setter** methods. They work in Python too, but make calling code verbose (`acc.set_balance(acc.get_balance() + 100)`). Python's idiomatic solution is `@property`, which keeps the simple attribute syntax.

JAVA-STYLE GETTERS AND SETTERS

main.py

```
class Employee:
    def __init__(self, name, salary):
        self._name = name
        self.set_salary(salary)

    def get_salary(self):
        return self._salary

    def set_salary(self, value):
        if value < 0:
            raise ValueError("salary cannot be negative")
        self._salary = value

e = Employee("Ravi", 50_000)
e.set_salary(e.get_salary() + 5_000)       # works, but clumsy
print(e.get_salary())
try:
    e.set_salary(-1)
except ValueError as err:
    print("ValueError:", err)
```

OUTPUT

```
55000
ValueError: salary cannot be negative
```

15.5.3 @property, setter and deleter

`@property` turns a method into a **managed attribute**: reading `obj.salary` calls the getter method, assigning `obj.salary = x` calls the method decorated with `@salary.setter`, and `del obj.salary` calls the `@salary.deleter`. Callers use plain attribute syntax while the class validates and computes behind the scenes. A property without a setter is **read-only**, which is also the way to expose **computed** values such as an area or an age.

MANAGED ATTRIBUTES WITH @PROPERTY

main.py

```
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary              # goes through the setter: validated

    @property
    def salary(self):                     # getter
        return self._salary

    @salary.setter
    def salary(self, value):              # setter
        if not isinstance(value, (int, float)) or value < 0:
            raise ValueError(f"invalid salary: {value!r}")
        self._salary = value

    @salary.deleter
    def salary(self):
        print(f"  removing salary of {self.name}")
        del self._salary

    @property
    def annual(self):                     # computed, read-only
        return self._salary * 12

e = Employee("Ravi", 50_000)
e.salary += 5_000                         # reads the getter, then calls the setter
print(e.salary, e.annual)
for bad in (-100, "lots"):
    try:
        e.salary = bad
    except ValueError as err:
        print("ValueError:", err)
try:
    e.annual = 1
except AttributeError as err:
    print("AttributeError:", err)
del e.salary
print(hasattr(e, "_salary"))
```

OUTPUT

```
55000 660000
ValueError: invalid salary: -100
ValueError: invalid salary: 'lots'
AttributeError: property 'annual' of 'Employee' object has no setter
  removing salary of Ravi
False
```

A PROPERTY ROUTES ATTRIBUTE SYNTAX THROUGH METHODS

TIP

Start with plain public attributes. If you later need validation or computation, convert the attribute to a property: calling code does not change. This is why Python code rarely needs get_ and set_ methods.

15.6 Object representation

15.6.1 __str__ and __repr__

By default, printing an object shows something like `<__main__.Point object at 0x7f...>`, which is useless. Two special methods control how objects are shown as text. `__repr__` returns an **unambiguous, developer-oriented** representation, ideally valid Python that recreates the object (`Point(3, 4)`); it is used by the REPL, debuggers, `repr()` and inside containers. `__str__` returns a **readable, user-oriented** form, used by `print()`, `str()` and f-strings. If only `__repr__` is defined, it is used for both, so always define `__repr__` first.

__REPR__ AND __STR__ main.py

```
class Plain:
    def __init__(self, x, y):
        self.x, self.y = x, y

class Point:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __repr__(self):
        return f"Point({self.x!r}, {self.y!r})"      # for developers

    def __str__(self):
        return f"({self.x}, {self.y})"               # for users

print(str(Plain(3, 4)).split(" at ")[0] + ">")
p = Point(3, 4)
print(p)                    # __str__
print(repr(p))              # __repr__
print([p, Point(0, -1)])    # containers use __repr__
print(f"{p} and {p!r}")
print(eval(repr(p)).x)      # a good repr recreates the object
```

OUTPUT

```
<__main__.Plain object>
(3, 4)
Point(3, 4)
[Point(3, 4), Point(0, -1)]
(3, 4) and Point(3, 4)
3
```

Called by	Uses
`print(obj)`, `str(obj)`, `f"{obj}"`	`__str__` (falls back to `__repr__`)
`repr(obj)`, `f"{obj!r}"`, the REPL, containers, debuggers	`__repr__`

15.7 Object lifecycle

15.7.1 __new__ vs __init__ (overview)

Creating an object is a two-step process. `__new__(cls, ...)` is a static method that **creates** and returns the new, empty instance; `__init__(self, ...)` then **initialises** it. You almost never override `__new__`. It is

needed for subclassing immutable types such as `int`, `str` and `tuple` (whose value must be fixed at creation), and for patterns that control instance creation, such as singletons or caching.

__NEW__ CREATES, __INIT__ INITIALISES

main.py

```
class Traced:
    def __new__(cls, *args):
        print(f"__new__  creating a {cls.__name__} object")
        return super().__new__(cls)

    def __init__(self, value):
        print(f"__init__ initialising with {value!r}")
        self.value = value

t = Traced(42)
print(t.value)

class Celsius(float):                       # immutable base: value set in __new__
    def __new__(cls, degrees):
        if degrees < -273.15:
            raise ValueError("below absolute zero")
        return super().__new__(cls, round(degrees, 2))

c = Celsius(36.666)
print(c, c + 1, isinstance(c, float))

class Settings:                             # a singleton: always the same object
    _instance = None
    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance
print(Settings() is Settings())
```

OUTPUT

```
__new__  creating a Traced object
__init__ initialising with 42
42
36.67 37.67 True
True
```

15.7.2 __del__ and garbage collection

Python frees an object automatically when nothing refers to it any more. CPython keeps a **reference count** on every object and frees it as soon as the count drops to zero; a **cyclic garbage collector** (the `gc` module) periodically finds groups of objects that refer only to each other. `__del__` (a "finaliser") is called just before an object is destroyed. Its timing is not guaranteed in general (cycles, other Python implementations, interpreter shutdown), so never rely on it to release important resources: use `with` and context managers (Module 17) instead.

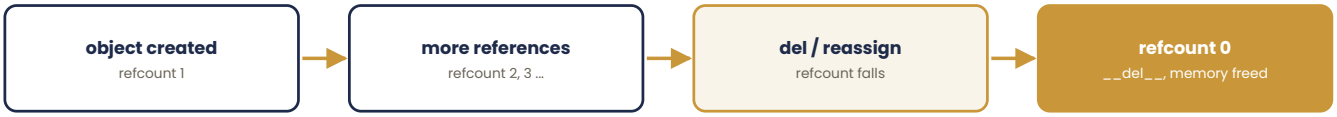
REFERENCE COUNTING, __DEL__ AND CYCLES

main.py

```
import sys, gc

class Resource:
    def __init__(self, name):
        self.name = name
        print(f"open  {name}")
    def __del__(self):
        print(f"close {self.name} (finaliser)")

r = Resource("report.pdf")
alias = r
print("references:", sys.getrefcount(r) - 1)    # minus the temporary argument
del r
print("after del r: still alive, alias holds it")
del alias                                       # last reference gone: freed now
print("---")

class Node:
    def __init__(self, name):
        self.name, self.partner = name, None
    def __del__(self):
        print(f"freed {self.name}")

a, b = Node("A"), Node("B")
a.partner, b.partner = b, a                      # a reference cycle
del a, b
print("cycle still in memory until the collector runs")
print("objects collected:", gc.collect() >= 2)
```

OUTPUT

```
open  report.pdf
references: 2
after del r: still alive, alias holds it
close report.pdf (finaliser)
---
cycle still in memory until the collector runs
freed A
freed B
objects collected: True
```



AN OBJECT LIVES WHILE ANYTHING REFERS TO IT

15.8 Relationships between classes

15.8.1 Association, aggregation and composition

Objects collaborate with other objects. The relationships differ in **ownership and lifetime**:

Relationship	Phrase	Lifetime of the parts	Example	UML
Association	uses / knows about	independent	a Teacher teaches Students	plain line or arrow

Relationship	Phrase	Lifetime of the parts	Example	UML
Aggregation	has a (shared)	parts can exist without the whole	a Department has Professors	hollow diamond
Composition	owns / is made of	parts are created and die with the whole	an Order is made of OrderLines	filled diamond

association: uses | aggregation: has (shared) | composition: owns

THREE KINDS OF COLLABORATION

ASSOCIATION, AGGREGATION AND COMPOSITION IN CODE

main.py

```
class Student:
    def __init__(self, name):
        self.name = name

class Teacher:                                  # association: uses students it is given
    def __init__(self, name):
        self.name = name
    def teach(self, student, topic):
        return f"{self.name} teaches {topic} to {student.name}"

class Professor:
    def __init__(self, name):
        self.name = name

class Department:                               # aggregation: holds existing professors
    def __init__(self, name):
        self.name, self.professors = name, []
    def add(self, prof):
        self.professors.append(prof)

class OrderLine:
    def __init__(self, item, qty, price):
        self.item, self.qty, self.price = item, qty, price

class Order:                                    # composition: creates and owns its lines
    def __init__(self, order_id):
        self.order_id, self._lines = order_id, []
    def add_line(self, item, qty, price):
        self._lines.append(OrderLine(item, qty, price))
    def total(self):
        return sum(l.qty * l.price for l in self._lines)

print(Teacher("Kiran").teach(Student("Asha"), "OOP"))

rao = Professor("Dr Rao")
cse = Department("CSE"); cse.add(rao)
del cse                                         # the department is gone ...
print(rao.name, "still exists")                 # ... the professor is not

order = Order(5001)
```

```
order.add_line("pen", 3, 10); order.add_line("book", 1, 250)
print("order", order.order_id, "total", order.total())   # lines live only inside the order
```

OUTPUT

```
Kiran teaches OOP to Asha
Dr Rao still exists
order 5001 total 280
```

A useful design guideline is **"favour composition over inheritance"**: build complex objects by combining simpler ones (a `Car` *has an* `Engine`) rather than creating deep inheritance hierarchies. Inheritance, for genuine "is a" relationships, is the subject of Module 16.

OOP bundles data and behaviour into objects built from classes; its pillars are encapsulation, abstraction, inheritance and polymorphism. Define a class with class Name:, create instances by calling it, initialise them in __init__, and use self (the object before the dot) in every instance method. Instance attributes belong to one object; class attributes are shared, and assigning through an instance creates a shadowing instance attribute (beware shared mutable class attributes). Instances store attributes in __dict__; __slots__ fixes the attribute set and saves memory. Instance methods take self, @classmethod methods take cls (ideal for alternative constructors), @staticmethod methods take neither. Encapsulation in Python is by convention: _internal, __private (name-mangled); prefer @property with setters and deleters over get_/set_ methods. Define __repr__ (unambiguous) and __str__ (readable). __new__ creates and __init__ initialises; objects are freed by reference counting and the garbage collector, and __del__ is unreliable for clean-up. Classes collaborate by association, aggregation and composition.

PRACTICE CHECKPOINT

Write a **BankAccount** class with a private balance, a property, deposit and withdraw methods, a class attribute counting accounts, and a classmethod constructor.

SAMPLE SOLUTION

main.py

```
class InsufficientFundsError(Exception):
    pass

class BankAccount:
    bank_name = "LBF Bank"
    _count = 0                                  # class attribute: accounts opened
    MIN_BALANCE = 500

    def __init__(self, owner, opening_balance=0):
        if opening_balance < self.MIN_BALANCE:
            raise ValueError(f"opening balance must be at least Rs {self.MIN_BALANCE}")
        BankAccount._count += 1
        self.owner = owner
        self.account_no = f"LBF{1000 + BankAccount._count}"
        self.__balance = opening_balance          # private
        self._history = [("open", opening_balance)]

    @classmethod
    def from_string(cls, text):                   # "Asha Rao:2500"
        owner, amount = text.split(":")
        return cls(owner.strip(), int(amount))

    @classmethod
    def accounts_opened(cls):
        return cls._count
```

```
    @property
    def balance(self):                          # read-only from outside
        return self.__balance

    def deposit(self, amount):
        if amount <= 0:
            raise ValueError("deposit must be positive")
        self.__balance += amount
        self._history.append(("deposit", amount))
        return self.__balance

    def withdraw(self, amount):
        if amount <= 0:
            raise ValueError("withdrawal must be positive")
        if self.__balance - amount < self.MIN_BALANCE:
            raise InsufficientFundsError(f"only Rs {self.__balance - self.MIN_BALANCE:,} can be
withdrawn")
        self.__balance -= amount
        self._history.append(("withdraw", amount))
        return self.__balance

    def statement(self):
        rows = [f"  {kind:<9}{amt:>8,}" for kind, amt in self._history]
        return "\n".join([f"{self.bank_name} | {self.account_no} | {self.owner}"] + rows + [f"
{'balance':<9}{self.__balance:>8,}"])

    def __repr__(self):
        return f"BankAccount({self.owner!r}, balance={self.__balance})"

a = BankAccount("Asha", 5_000)
b = BankAccount.from_string("Ravi Kumar:2500")
a.deposit(1_500)
a.withdraw(2_000)
print(a.statement())
print(b, "| balance:", b.balance)
for action in (lambda: b.withdraw(2_500), lambda: setattr(b, "balance", 10**6), lambda:
BankAccount("Meera", 100)):
    try:
        action()
    except (InsufficientFundsError, AttributeError, ValueError) as e:
        print(f"{type(e).__name__}: {e}")
print("accounts opened:", BankAccount.accounts_opened())
```

OUTPUT

```
LBF Bank | LBF1001 | Asha
  open        5,000
  deposit     1,500
  withdraw    2,000
  balance     4,500
BankAccount('Ravi Kumar', balance=2500) | balance: 2500
InsufficientFundsError: only Rs 2,000 can be withdrawn
AttributeError: property 'balance' of 'BankAccount' object has no setter
ValueError: opening balance must be at least Rs 500
accounts opened: 2
```

MODULE 16 · INTERMEDIATE

Object-oriented programming: inheritance, polymorphism and special methods

Minimum duration **4 days** Level **Intermediate**

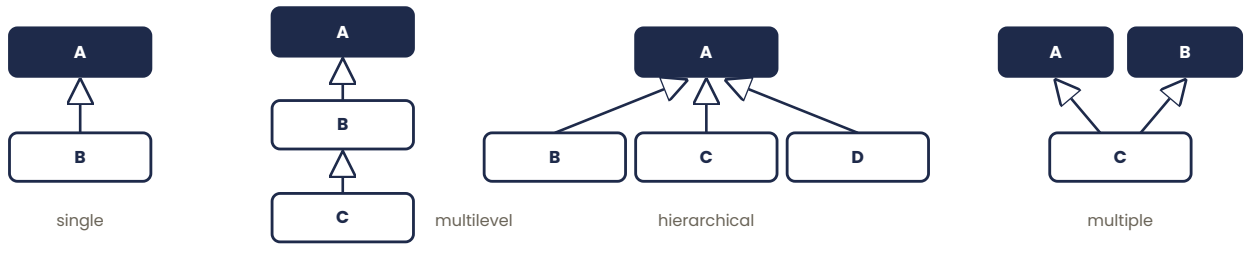
This module covers code reuse with inheritance, polymorphism through method overriding and duck typing, abstraction with abstract base classes and protocols, and the special (dunder) methods that let your classes behave like built-in types: arithmetic, comparison, containers, callables, hashing, context managers and attribute access. It then introduces data classes and enums, mixins and the "composition over inheritance" principle, and gives an overview of descriptors, metaclasses and `__init_subclass__`.

IN THIS MODULE

16.1 Inheritance
16.2 Polymorphism
16.3 Abstraction
16.4 Special (dunder) methods
16.5 Data classes and enums
16.6 Mixins and composition over inheritance
16.7 Advanced class topics (overview)

16.1 Inheritance

Inheritance creates a new class (the **child**, subclass or derived class) from an existing one (the **parent**, superclass or base class). The child automatically gets all the parent's attributes and methods, can **add** new ones and can **override** (replace) inherited ones. It models an **"is a"** relationship: a `SavingsAccount` is an `Account`, a `Dog` is an `Animal`. Write the parent in parentheses: `class Dog(Animal):`. Every class ultimately inherits from `object`.



THE FOUR SHAPES OF INHERITANCE (ARROWS POINT TO THE PARENT)

16.1.1 Single inheritance

In **single inheritance** a class has exactly one parent. The child inherits everything and adds or customises what is different.

SINGLE INHERITANCE main.py

```
class Animal:
    def __init__(self, name):
        self.name = name

    def eat(self):
        return f"{self.name} is eating"

    def speak(self):
        return f"{self.name} makes a sound"

class Dog(Animal):                    # Dog inherits from Animal
    def fetch(self):                  # a new method
        return f"{self.name} fetches the ball"

    def speak(self):                  # overrides the parent's method
        return f"{self.name} says Woof"

d = Dog("Bruno")
print(d.eat())                        # inherited
print(d.fetch())                      # added
print(d.speak())                      # overridden
print(Animal("Generic").speak())
print(Dog.__bases__, Animal.__bases__)
```

OUTPUT

```
Bruno is eating
Bruno fetches the ball
Bruno says Woof
Generic makes a sound
(<class '__main__.Animal'>,) (<class 'object'>,)
```

16.1.2 Multilevel inheritance

In **multilevel inheritance** a class inherits from a class that itself inherits from another, forming a chain: `Vehicle → Car → ElectricCar`. The bottom class has the features of all its ancestors. Keep chains short; deep hierarchies are hard to follow.

MULTILEVEL INHERITANCE

main.py

```
class Vehicle:
    def __init__(self, brand):
        self.brand = brand
    def describe(self):
        return f"{self.brand} vehicle"

class Car(Vehicle):
    wheels = 4
    def honk(self):
        return "Beep!"

class ElectricCar(Car):
    def charge(self):
        return f"{self.brand} charging"

tesla = ElectricCar("Tata Nexon EV")
print(tesla.describe(), "|", tesla.wheels, "wheels |", tesla.honk(), "|", tesla.charge())
print([cls.__name__ for cls in ElectricCar.__mro__])
```

OUTPUT

```
Tata Nexon EV vehicle | 4 wheels | Beep! | Tata Nexon EV charging
['ElectricCar', 'Car', 'Vehicle', 'object']
```

16.1.3 Hierarchical inheritance

In **hierarchical inheritance** several classes share one parent. Common code lives once in the parent, and each child specialises it.

HIERARCHICAL INHERITANCE

main.py

```
class Employee:
    def __init__(self, name, base):
        self.name, self.base = name, base
    def pay(self):
        return self.base
    def payslip(self):
        return f"{self.name:<6} {type(self).__name__:<10} Rs {self.pay():>8,}"

class Manager(Employee):
    def pay(self):
        return self.base + 20_000                # allowance

class Developer(Employee):
    def pay(self):
        return self.base + 5_000                 # tools allowance

class Intern(Employee):
    def pay(self):
        return self.base // 2

for e in (Manager("Kiran", 90_000), Developer("Asha", 70_000), Intern("Ravi", 30_000)):
    print(e.payslip())
```

OUTPUT

```
Kiran  Manager    Rs  110,000
Asha   Developer  Rs   75,000
Ravi   Intern     Rs   15,000
```

16.1.4 Multiple inheritance

Python supports **multiple inheritance**: a class can have several parents, `class C(A, B):`, and inherits from all of them. It is powerful for combining independent features (mixins, 16.6), but can be confusing when parents define the same method; Python resolves that with the method resolution order (16.1.6).

MULTIPLE INHERITANCE main.py

```
class Camera:
    def take_photo(self):
        return "photo taken"
    def info(self):
        return "camera"

class Phone:
    def call(self, number):
        return f"calling {number}"
    def info(self):
        return "phone"

class SmartPhone(Phone, Camera):     # two parents
    pass

s = SmartPhone()
print(s.call("98450 12345"), "|", s.take_photo())
print(s.info())                      # Phone comes first in the list of parents
print([c.__name__ for c in SmartPhone.__mro__])
```

OUTPUT

```
calling 98450 12345 | photo taken
phone
['SmartPhone', 'Phone', 'Camera', 'object']
```

16.1.5 super()

`super()` returns a proxy for calling methods of the **parent class** (more precisely, the next class in the MRO). Its most common use is in `__init__`: the child calls `super().__init__(...)` to let the parent set up its attributes, then adds its own. It is also used to **extend** an overridden method rather than replace it completely. Avoid naming the parent explicitly (`Account.__init__(self, ...)`), because `super()` keeps working with multiple inheritance and when the hierarchy changes.

CALLING THE PARENT WITH SUPER()

main.py

```
class Account:
    def __init__(self, owner, balance=0):
        self.owner = owner
        self.balance = balance

    def describe(self):
        return f"{self.owner}: Rs {self.balance:,}"

class SavingsAccount(Account):
    def __init__(self, owner, balance=0, rate=0.04):
        super().__init__(owner, balance)          # parent sets owner and balance
        self.rate = rate                          # child adds its own attribute

    def add_interest(self):
        self.balance += round(self.balance * self.rate)

    def describe(self):
        return super().describe() + f" (savings at {self.rate:.0%})"   # extend, not replace

s = SavingsAccount("Asha", 10_000)
s.add_interest()
print(s.describe())

class Broken(Account):
    def __init__(self, owner, limit):
        self.limit = limit                        # forgot super().__init__
try:
    print(Broken("Ravi", 5000).describe())
except AttributeError as e:
    print("AttributeError:", e)
```

OUTPUT

```
Asha: Rs 10,400 (savings at 4%)
AttributeError: 'Broken' object has no attribute 'owner'
```

16.1.6 Method Resolution Order (MRO) and C3 linearisation

When you access `obj.method`, Python searches the object's class and then its ancestors in a fixed order called the **Method Resolution Order**, visible as `Class.__mro__` or `Class.mro()`. For multiple inheritance Python computes it with the **C3 linearisation** algorithm, which guarantees that a child always comes before its parents and that the order of parents in the `class` statement is respected. In the classic "diamond", `D(B, C)` where both `B` and `C` inherit from `A`, the MRO is D, B, C, A, object, so `A` is visited only once, and cooperative `super()` calls run every class exactly once.

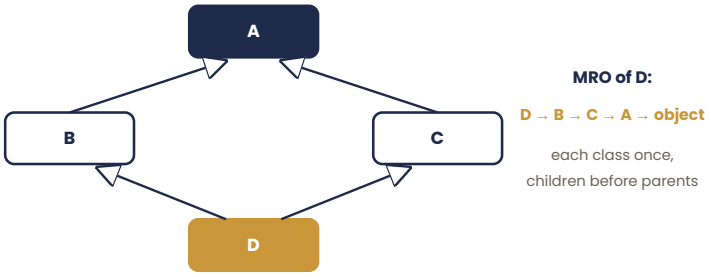
THE DIAMOND PROBLEM AND THE MRO

main.py

```
class A:
    def hello(self):
        print("A.hello")

class B(A):
    def hello(self):
        print("B.hello"); super().hello()

class C(A):
    def hello(self):
        print("C.hello"); super().hello()

class D(B, C):
    def hello(self):
        print("D.hello"); super().hello()

print([cls.__name__ for cls in D.__mro__])
D().hello()                     # each class runs once, in MRO order

try:
    class X(A, B):              # A before its own child B: no consistent order
        pass
except TypeError as e:
    print("TypeError:", e)
```

OUTPUT

```
['D', 'B', 'C', 'A', 'object']
D.hello
B.hello
C.hello
A.hello
TypeError: Cannot create a consistent method resolution order (MRO) for bases A, B
```



THE DIAMOND: C3 LINEARISATION VISITS A ONLY ONCE, AFTER BOTH B AND C

16.1.7 isinstance() and issubclass()

`isinstance(obj, Class)` is `True` if the object is an instance of the class **or any subclass**; `issubclass(Child, Parent)` checks class relationships. Both accept a tuple of classes. `type(obj) is Class` checks the **exact** type only, ignoring inheritance, and is rarely what you want.

CHECKING TYPES AND CLASS RELATIONSHIPS

main.py

```
class Animal: pass
class Dog(Animal): pass
class Puppy(Dog): pass

p = Puppy()
print(isinstance(p, Puppy), isinstance(p, Dog), isinstance(p, Animal), isinstance(p, object))
print(type(p) is Dog, type(p) is Puppy)
print(issubclass(Puppy, Animal), issubclass(Animal, Dog), issubclass(Dog, (int, Animal)))
print(isinstance(True, int), issubclass(bool, int))     # bool is a subclass of int
```

OUTPUT

```
True True True True
False True
True False True
True True
```

16.2 Polymorphism

Polymorphism ("many forms") means that code written for one interface works with many types. A function that calls `shape.area()` works for circles, squares and any future shape, without `if` statements checking the type. Python achieves this through method overriding and, more broadly, duck typing.

16.2.1 Method overriding

A subclass **overrides** a method by defining one with the same name. When the method is called, Python uses the most specific version according to the MRO, so the same call does different things for different objects. Built-in operations rely on this too: `str(obj)` calls whichever `__str__` the object's class provides.

POLYMORPHISM THROUGH METHOD OVERRIDING

main.py

```
class Shape:
    def area(self):
        return 0
    def describe(self):
        return f"{type(self).__name__} with area {self.area():.2f}"   # calls the overridden area()

class Circle(Shape):
    def __init__(self, r):
        self.r = r
    def area(self):
        return 3.14159 * self.r ** 2

class Square(Shape):
    def __init__(self, side):
        self.side = side
    def area(self):
        return self.side ** 2

class Triangle(Shape):
    def __init__(self, base, height):
        self.base, self.height = base, height
    def area(self):
        return 0.5 * self.base * self.height

shapes = [Circle(1), Square(3), Triangle(4, 5)]
for s in shapes:
    print(s.describe())                      # one call, three behaviours
print("total area:", round(sum(s.area() for s in shapes), 2))
```

OUTPUT

```
Circle with area 3.14
Square with area 9.00
Triangle with area 10.00
total area: 22.14
```

16.2.2 Duck typing

"If it walks like a duck and quacks like a duck, it is a duck." Python does not require objects to share a base class to be used interchangeably: any object with the right methods works. A function that calls `obj.read()` accepts files, network streams, `io.StringIO` and your own classes alike. This is **duck typing**, and it is why Python code rarely checks types: it simply uses the behaviour it needs and lets an `AttributeError` or `TypeError` surface if an object does not support it.

DUCK TYPING: BEHAVIOUR MATTERS, NOT ANCESTRY

main.py

```
import io

class Duck:
    def speak(self): return "Quack"

class Robot:                          # unrelated class, same method name
    def speak(self): return "Beep boop"

class Person:
    def speak(self): return "Hello"

for thing in (Duck(), Robot(), Person()):
    print(type(thing).__name__, "->", thing.speak())

def count_lines(stream):              # works with anything that iterates over lines
    return sum(1 for _ in stream)

with open("demo.txt", "w") as f:
    f.write("a\nb\nc\n")
with open("demo.txt") as f:
    print(count_lines(f), count_lines(io.StringIO("x\ny\n")), count_lines(["l1", "l2", "l3", "l4"]))

print(len("abc"), len([1, 2]), len({"a": 1}))   # len() is duck-typed too: it calls __len__
```

OUTPUT

```
Duck -> Quack
Robot -> Beep boop
Person -> Hello
3 2 4
3 2 1
```

16.2.3 Why Python has no method overloading (default arguments and *args instead)

Languages such as Java and C++ allow several methods with the same name but different parameter lists (**overloading**). In Python a class namespace is a dictionary, so defining a method again simply **replaces** the earlier one. Python covers the same needs with default arguments, keyword arguments, `*args` and `**kwargs`, alternative constructors (`@classmethod`), or `functools.singledispatchmethod` for type-based dispatch.

A SECOND DEFINITION REPLACES THE FIRST; USE FLEXIBLE PARAMETERS main.py

```
class Calculator:
    def add(self, a, b):
        return a + b
    def add(self, a, b, c):              # replaces the first add!
        return a + b + c

try:
    Calculator().add(1, 2)
except TypeError as e:
    print("TypeError:", e)

class Calculator:
    def add(self, *numbers, start=0):    # one flexible method instead
        return sum(numbers, start)

c = Calculator()
print(c.add(1, 2), c.add(1, 2, 3), c.add(), c.add(1.5, 2.5, start=10))

def area(width, height=None):            # default argument for the "square" case
    return width * (width if height is None else height)
print(area(4), area(4, 5))
```

OUTPUT

```
TypeError: Calculator.add() missing 1 required positional argument: 'c'
3 6 0 14.0
16 20
```

16.3 Abstraction

16.3.1 Abstract base classes with the abc module

An **abstract base class** (ABC) defines an interface that subclasses must implement, without being usable on its own. Subclass `abc.ABC` and mark required methods with `@abstractmethod`. Python then refuses to create instances of the ABC, or of any subclass that has not implemented every abstract method, so mistakes are caught when the object is created instead of later, deep inside the program. ABCs can also provide concrete methods built on the abstract ones.

16.3.2 @abstractmethod

AN ABSTRACT BASE CLASS WITH ABSTRACT METHODS main.py

```
from abc import ABC, abstractmethod

class PaymentMethod(ABC):
    def __init__(self, owner):
        self.owner = owner

    @abstractmethod
    def pay(self, amount):               # every payment method must implement this
        ...

    @property
    @abstractmethod
    def fee_percent(self):
        ...

    def checkout(self, amount):          # concrete method using the abstract ones
```

```
        fee = round(amount * self.fee_percent / 100, 2)
        return f"{self.pay(amount + fee)} (fee Rs {fee})"

class UPI(PaymentMethod):
    fee_percent = 0
    def pay(self, amount):
        return f"{self.owner} paid Rs {amount} via UPI"

class CreditCard(PaymentMethod):
    fee_percent = 1.8
    def pay(self, amount):
        return f"{self.owner} paid Rs {amount} by card"

for method in (UPI("Asha"), CreditCard("Ravi")):
    print(method.checkout(1000))

try:
    PaymentMethod("x")
except TypeError as e:
    print("TypeError:", e)

class Wallet(PaymentMethod):            # forgot to implement pay and fee_percent
    pass
try:
    Wallet("Meera")
except TypeError as e:
    print("TypeError:", e)
```

OUTPUT

```
Asha paid Rs 1000.0 via UPI (fee Rs 0.0)
Ravi paid Rs 1018.0 by card (fee Rs 18.0)
TypeError: Can't instantiate abstract class PaymentMethod without an implementation for abstract
methods 'fee_percent', 'pay'
TypeError: Can't instantiate abstract class Wallet without an implementation for abstract methods
'fee_percent', 'pay'
```

The standard library defines many ABCs in `collections.abc`, such as `Iterable`, `Sequence`, `Mapping` and `Callable`. Inheriting from `collections.abc.Sequence` and implementing just `__getitem__` and `__len__` gives you `__contains__`, `__iter__`, `index`, `count` and `__reversed__` for free.

16.3.3 Protocols (structural typing, overview)

ABCs use **nominal** typing: a class must explicitly inherit from the ABC. `typing.Protocol` (3.8) describes an interface **structurally**: any class with matching methods satisfies it, without inheriting from it. This is duck typing made visible to type checkers such as mypy (Module 19). Adding `@runtime_checkable` also allows `isinstance` checks of the method names.

STRUCTURAL TYPING WITH PROTOCOL

main.py

```
from typing import Protocol, runtime_checkable

@runtime_checkable
class Drawable(Protocol):
    def draw(self) -> str: ...

class Button:                           # does NOT inherit from Drawable
    def draw(self) -> str:
        return "[ OK ]"

class Chart:
    def draw(self) -> str:
        return "|||.|"

class Sound:
    def play(self) -> str:
        return "beep"

def render(item: Drawable) -> None:     # a type checker accepts any class with draw()
    print(item.draw())

for item in (Button(), Chart()):
    render(item)
print(isinstance(Button(), Drawable), isinstance(Sound(), Drawable))
```

OUTPUT

```
[ OK ]
|||.|
True False
```

	ABC (abc.ABC)	Protocol (typing.Protocol)
Typing style	nominal: must inherit	structural: just match the methods
Checked when	object creation (missing methods)	by static type checkers; isinstance if runtime_checkable
Can share code	yes, concrete methods	rarely used for that
Best for	your own class hierarchies	describing what a function needs from its arguments

16.4 Special (dunder) methods

Special methods, with names surrounded by double underscores, are hooks that Python calls to implement built-in syntax and functions: `a + b` calls `a.__add__(b)`, `len(x)` calls `x.__len__()`, `x[i]` calls `x.__getitem__(i)`. Implementing them makes your classes feel like built-in types. This is called the Python **data model**. You never call them directly in normal code; you use the operators and built-ins.

You write	Python calls
`a + b`, `a - b`, `a * b`	`a.__add__(b)`, `__sub__`, `__mul__`
`a == b`, `a < b`	`a.__eq__(b)`, `a.__lt__(b)`
`len(x)`, `x[i]`, `x[i] = v`, `v in x`	`__len__`, `__getitem__`, `__setitem__`, `__contains__`
`for v in x`	`x.__iter__()`

You write	Python calls
`x(...)`	`x.__call__(...)`
`hash(x)`, dict keys, sets	`x.__hash__()`
`with x as v:`	`x.__enter__()`, `x.__exit__(...)`
`str(x)`, `repr(x)`, `bool(x)`	`__str__`, `__repr__`, `__bool__`

16.4.1 Arithmetic: __add__, __sub__, __mul__, and reflected versions

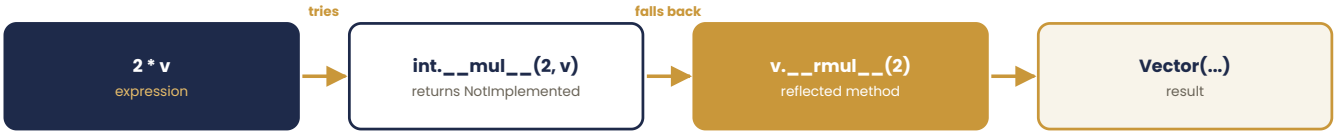
Arithmetic operators map to methods such as `__add__`, `__sub__`, `__mul__`, `__truediv__`, `__neg__` and `__abs__`. A method should return a **new** object and return the special value `NotImplemented` for operand types it does not support, so Python can try the other operand. For `2 * v`, Python first tries `int.__mul__(2, v)`, gets `NotImplemented`, then calls the **reflected** method `v.__rmul__(2)`. In-place operators such as `+=` use `__iadd__` if defined, otherwise fall back to `__add__`.

ARITHMETIC OPERATORS ON A VECTOR main.py

```
import math

class Vector:
    def __init__(self, x, y):
        self.x, self.y = x, y
    def __repr__(self):
        return f"Vector({self.x}, {self.y})"
    def __add__(self, other):
        if not isinstance(other, Vector):
            return NotImplemented
        return Vector(self.x + other.x, self.y + other.y)
    def __sub__(self, other):
        if not isinstance(other, Vector):
            return NotImplemented
        return Vector(self.x - other.x, self.y - other.y)
    def __mul__(self, k):                    # vector * number
        if not isinstance(k, (int, float)):
            return NotImplemented
        return Vector(self.x * k, self.y * k)
    __rmul__ = __mul__                       # number * vector
    def __neg__(self):
        return Vector(-self.x, -self.y)
    def __abs__(self):
        return math.hypot(self.x, self.y)

a, b = Vector(3, 4), Vector(1, 2)
print(a + b, a - b, a * 2, 3 * b, -a, abs(a))
a += b                                       # uses __add__: a new object
print(a)
try:
    a + 5
except TypeError as e:
    print("TypeError:", e)
```

OUTPUT

```
Vector(4, 6) Vector(2, 2) Vector(6, 8) Vector(3, 6) Vector(-3, -4) 5.0
Vector(4, 6)
TypeError: unsupported operand type(s) for +: 'Vector' and 'int'
```



HOW PYTHON RESOLVES 2 * V USING THE REFLECTED METHOD

16.4.2 Comparison: __eq__, __lt__, functools.total_ordering

`==` calls `__eq__`. By default two objects are equal only if they are the **same** object; define `__eq__` to compare by value. The ordering operators map to `__lt__`, `__le__`, `__gt__` and `__ge__`; defining `__lt__` alone is enough for `sorted()`, `min()` and `max()`. The `@functools.total_ordering` decorator fills in the remaining comparison methods from `__eq__` and one ordering method.

__EQ__, __LT__ AND TOTAL_ORDERING main.py

```
from functools import total_ordering

class Plain:
    def __init__(self, v): self.v = v
print(Plain(1) == Plain(1))                 # default: identity

@total_ordering
class Version:
    def __init__(self, text):
        self.parts = tuple(int(p) for p in text.split("."))
    def __repr__(self):
        return "v" + ".".join(map(str, self.parts))
    def __eq__(self, other):
        if not isinstance(other, Version):
            return NotImplemented
        return self.parts == other.parts
    def __lt__(self, other):
        if not isinstance(other, Version):
            return NotImplemented
        return self.parts < other.parts       # tuples compare element by element

versions = [Version(v) for v in ("3.10.2", "3.9.18", "3.14.0", "3.10.11")]
print(sorted(versions))
print(max(versions), Version("3.12") == Version("3.12"), Version("3.9") >= Version("3.10"))
print(Version("3.12") != Version("3.13"), Version("2.7") <= Version("2.7"))
```

OUTPUT

```
False
[v3.9.18, v3.10.2, v3.10.11, v3.14.0]
v3.14.0 True False
True True
```

NOTE

Defining `__eq__` sets `__hash__` to `None`, making instances unhashable, unless you also define `__hash__` (16.4.5).

16.4.3 Container behaviour: __len__, __getitem__, __setitem__, __contains__, __iter__

Implement the container methods to make an object behave like a sequence or mapping: `__len__` for `len()`, `__getitem__` for indexing (and slicing, when it receives a `slice` object), `__setitem__` and `__delitem__` for assignment and deletion, `__contains__` for `in` and `__iter__` for `for` loops. Defining `__getitem__` with integer indexes even makes iteration and `in` work automatically.

A PLAYLIST THAT BEHAVES LIKE A LIST

main.py

```
class Playlist:
    def __init__(self, name, songs=None):
        self.name = name
        self._songs = list(songs or [])
    def __len__(self):
        return len(self._songs)
    def __getitem__(self, index):
        if isinstance(index, slice):
            return Playlist(f"{self.name} (part)", self._songs[index])
        return self._songs[index]
    def __setitem__(self, index, song):
        self._songs[index] = song
    def __delitem__(self, index):
        del self._songs[index]
    def __contains__(self, song):
        return song.lower() in (s.lower() for s in self._songs)
    def __iter__(self):
        return iter(self._songs)
    def __repr__(self):
        return f"Playlist({self.name!r}, {self._songs})"

p = Playlist("Focus", ["Raag Yaman", "Lo-fi Beats", "Rain Sounds", "Piano Study"])
print(len(p), p[0], p[-1])
print(p[1:3])
p[2] = "Ocean Waves"; del p[0]
print("lo-fi beats" in p, "Jazz" in p)
for i, song in enumerate(p, 1):
    print(i, song)
print(sorted(p), list(reversed(p._songs)))
```

OUTPUT

```
4 Raag Yaman Piano Study
Playlist('Focus (part)', ['Lo-fi Beats', 'Rain Sounds'])
True False
1 Lo-fi Beats
2 Ocean Waves
3 Piano Study
['Lo-fi Beats', 'Ocean Waves', 'Piano Study'] ['Piano Study', 'Ocean Waves', 'Lo-fi Beats']
```

16.4.4 Callable objects: __call__

Defining `__call__` makes instances **callable** like functions: `obj(args)` runs `obj.__call__(args)`. Callable objects are useful when a function needs to remember configuration or state between calls, as an alternative to closures, and they can be passed anywhere a function is expected (`key=`, `map`, callbacks). Class-based decorators (11.5.9) use the same mechanism.

OBJECTS YOU CAN CALL

main.py

```
class Discount:
    def __init__(self, percent):
        self.percent = percent
        self.applied = 0
    def __call__(self, price):
        self.applied += 1
        return round(price * (1 - self.percent / 100), 2)

festival = Discount(20)
print(festival(1000), festival(499))
print(list(map(festival, [100, 250, 899])))
print("applied", festival.applied, "times | callable:", callable(festival))
```

OUTPUT

```
800.0 399.2
[80.0, 200.0, 719.2]
applied 5 times | callable: True
```

16.4.5 Hashing: __hash__

Objects used as **dictionary keys** or **set members** must be hashable: they need a `__hash__` method returning an integer, and objects that compare equal must have the same hash. By default, instances hash by identity. If you define `__eq__` to compare values, define `__hash__` from the **same** fields, usually by hashing a tuple of them, and only for objects whose fields never change after creation; a key whose hash changes while it is in a dictionary gets lost.

MAKING OBJECTS USABLE AS KEYS AND SET MEMBERS

main.py

```
class Point:
    def __init__(self, x, y):
        self._x, self._y = x, y                # treat as read-only
    @property
    def x(self): return self._x
    @property
    def y(self): return self._y
    def __eq__(self, other):
        return isinstance(other, Point) and (self.x, self.y) == (other.x, other.y)
    def __hash__(self):
        return hash((self.x, self.y))          # same fields as __eq__
    def __repr__(self):
        return f"Point({self.x}, {self.y})"

visited = {Point(0, 0), Point(1, 2), Point(1, 2)}
print(visited, Point(1, 2) in visited)
labels = {Point(0, 0): "origin"}
print(labels[Point(0, 0)])

class OnlyEq:
    def __eq__(self, other): return True
try:
    {OnlyEq()}
except TypeError as e:
    print("TypeError:", e)
```

OUTPUT

```
{Point(1, 2), Point(0, 0)} True
origin
TypeError: cannot use 'OnlyEq' as a set element (unhashable type: 'OnlyEq')
```

16.4.6 Context managers: __enter__, __exit__

An object with `__enter__` and `__exit__` methods can be used in a `with` statement. `__enter__` runs at the start, and its return value is bound by `as`. `__exit__(exc_type, exc, tb)` runs at the end **whatever happens**, receiving details of any exception (or three `None`s); returning `True` suppresses the exception. Module 17 covers context managers in depth.

A CLASS-BASED CONTEXT MANAGER main.py

```
class Transaction:
    def __init__(self, name):
        self.name, self.steps = name, []
    def __enter__(self):
        print(f"BEGIN {self.name}")
        return self
    def __exit__(self, exc_type, exc, tb):
        if exc_type is None:
            print(f"COMMIT {self.steps}")
        else:
            print(f"ROLLBACK because {exc_type.__name__}: {exc}")
        return False                          # do not suppress errors

with Transaction("transfer") as t:
    t.steps += ["debit Asha 500", "credit Ravi 500"]

try:
    with Transaction("bad transfer") as t:
        t.steps.append("debit Asha 500")
        raise ConnectionError("bank server down")
except ConnectionError:
    print("caller saw the error")
```

```
OUTPUT
BEGIN transfer
COMMIT ['debit Asha 500', 'credit Ravi 500']
BEGIN bad transfer
ROLLBACK because ConnectionError: bank server down
caller saw the error
```

16.4.7 Attribute access: __getattr__, __setattr__ (overview)

`__getattr__(self, name)` is called only when normal attribute lookup **fails**, which makes it handy for computed or delegated attributes and friendly error messages. `__setattr__(self, name, value)` is called for **every** attribute assignment, so it can validate or log all changes; inside it you must store values with `super().__setattr__` (or `self.__dict__`) to avoid infinite recursion. `__getattribute__` intercepts every lookup and is rarely needed. Prefer `@property` for individual attributes.

__GETATTR__ AND __SETATTR__

main.py

```
class Config:
    def __init__(self, **settings):
        super().__setattr__("_data", dict(settings))

    def __getattr__(self, name):                # only for missing attributes
        try:
            return self._data[name]
        except KeyError:
            raise AttributeError(f"no setting called {name!r}") from None

    def __setattr__(self, name, value):         # every assignment
        print(f"  set {name} = {value!r}")
        self._data[name] = value

cfg = Config(theme="dark", font=14)
print(cfg.theme, cfg.font)
cfg.font = 16
print(cfg.font, cfg._data)
try:
    cfg.language
except AttributeError as e:
    print("AttributeError:", e)

class Wrapper:                                  # delegation with __getattr__
    def __init__(self, items):
        self._items = items
    def __getattr__(self, name):
        return getattr(self._items, name)       # forward unknown attributes to the list

w = Wrapper([3, 1, 2]); w.append(0); w.sort()
print(w._items, w.count(1))
```

OUTPUT

```
dark 14
  set font = 16
16 {'theme': 'dark', 'font': 16}
AttributeError: no setting called 'language'
[0, 1, 2, 3] 1
```

16.5 Data classes and enums

16.5.1 @dataclass and its generated methods

Many classes mainly hold data, and writing their `__init__`, `__repr__` and `__eq__` by hand is repetitive. The `@dataclass` decorator (3.7) reads the class's **annotated** fields and generates those methods for you. Type annotations are required to declare fields but are not enforced at run time.

A DATA CLASS

main.py

```
from dataclasses import dataclass

@dataclass
class Student:
    name: str
    roll: int
    marks: float = 0.0

a = Student("Asha", 101, 91.5)
b = Student("Asha", 101, 91.5)
print(a)                          # generated __repr__
print(a == b, a is b)             # generated __eq__ compares fields
a.marks += 2
print(a.marks)
print(Student("Ravi", 102))       # default value used
```

OUTPUT

```
Student(name='Asha', roll=101, marks=91.5)
True False
93.5
Student(name='Ravi', roll=102, marks=0.0)
```

Generated by default	Optional (decorator argument)
`__init__` from the fields in order	`order=True`: `__lt__`, `__le__`, `__gt__`, `__ge__`
`__repr__` showing every field	`frozen=True`: immutable instances, plus `__hash__`
`__eq__` comparing fields as a tuple	`slots=True` (3.10): use `__slots__`
	`kw_only=True` (3.10): keyword-only `__init__`

16.5.2 Field defaults, field(), frozen=True, order=True

Fields with defaults must come after fields without. Mutable defaults are forbidden; use `field(default_factory=list)` to create a fresh object per instance. `field()` also controls whether a field appears in `repr`, comparisons or `__init__`. `__post_init__` runs after the generated `__init__` for validation or computed fields. `frozen=True` makes instances immutable (and hashable), and `order=True` generates ordering methods that compare the fields in definition order.

FIELD(), __POST_INIT__, ORDER AND FROZEN main.py

```
from dataclasses import dataclass, field, asdict, replace, FrozenInstanceError

@dataclass(order=True)
class Student:
    sort_key: float = field(init=False, repr=False)       # computed, used for ordering
    name: str
    marks: float
    subjects: list[str] = field(default_factory=list)     # fresh list each time
    password: str = field(default="", repr=False, compare=False)

    def __post_init__(self):
        if not 0 <= self.marks <= 100:
            raise ValueError(f"invalid marks {self.marks}")
        self.sort_key = -self.marks                       # highest marks first

students = [Student("Ravi", 78), Student("Asha", 91, ["Python"]), Student("Meera", 85)]
for s in sorted(students):
    print(s)
print(asdict(students[1]))

@dataclass(frozen=True)
class Point:
    x: float
    y: float

p = Point(1, 2)
try:
    p.x = 5
except FrozenInstanceError as e:
    print("FrozenInstanceError:", e)
print(replace(p, x=5), {p: "start"}[Point(1, 2)])
```

OUTPUT

```
Student(name='Asha', marks=91, subjects=['Python'])
Student(name='Meera', marks=85, subjects=[])
Student(name='Ravi', marks=78, subjects=[])
{'sort_key': -91, 'name': 'Asha', 'marks': 91, 'subjects': ['Python'], 'password': ''}
FrozenInstanceError: cannot assign to field 'x'
Point(x=5, y=2) start
```

16.5.3 enum.Enum and auto()

An **enumeration** is a fixed set of named constants, such as order statuses, days or colours. Using an `Enum` instead of plain strings or numbers prevents typos, gives readable output and allows iteration over all values. Each member has a `name` and a `value`; `auto()` assigns values automatically. `IntEnum` and `StrEnum` (3.11) members also behave as ints or strings, and `Flag` supports combinations with `|`.

ENUMERATIONS

main.py

```
from enum import Enum, auto, StrEnum, Flag

class Status(Enum):
    PENDING = auto()
    SHIPPED = auto()
    DELIVERED = auto()
    CANCELLED = auto()

order_status = Status.SHIPPED
print(order_status, order_status.name, order_status.value)
print(order_status is Status.SHIPPED, Status["DELIVERED"], Status(1))
print([s.name for s in Status])

match order_status:
    case Status.PENDING:
        print("waiting for dispatch")
    case Status.SHIPPED:
        print("on the way")

class Level(StrEnum):
    BEGINNER = auto()          # value "beginner"
    ADVANCED = auto()
print(Level.BEGINNER, Level.BEGINNER == "beginner", Level.ADVANCED.upper())

class Permission(Flag):
    READ = auto(); WRITE = auto(); DELETE = auto()
editor = Permission.READ | Permission.WRITE
print(editor, Permission.WRITE in editor, Permission.DELETE in editor)
try:
    Status("SHIPPED")
except ValueError as e:
    print("ValueError:", e)
```

OUTPUT

```
Status.SHIPPED SHIPPED 2
True Status.DELIVERED Status.PENDING
['PENDING', 'SHIPPED', 'DELIVERED', 'CANCELLED']
on the way
beginner True ADVANCED
Permission.READ|WRITE True False
ValueError: 'SHIPPED' is not a valid Status
```

16.6 Mixins and composition over inheritance

A **mixin** is a small class that provides one reusable feature (such as JSON export, comparison or logging) and is meant to be combined with other classes through multiple inheritance, not used alone. Mixins have no `__init__` of their own, or a cooperative one, and are listed before the main base class.

ADDING FEATURES WITH MIXINS

main.py

```
import json

class JsonMixin:
    def to_json(self):
        return json.dumps(self.__dict__, default=str)

class ReprMixin:
    def __repr__(self):
        fields = ", ".join(f"{k}={v!r}" for k, v in self.__dict__.items())
        return f"{type(self).__name__}({fields})"

class Person:
    def __init__(self, name, city):
        self.name, self.city = name, city

class Customer(JsonMixin, ReprMixin, Person):
    def __init__(self, name, city, tier):
        super().__init__(name, city)
        self.tier = tier

c = Customer("Asha", "Mysuru", "gold")
print(c)
print(c.to_json())
print([k.__name__ for k in Customer.__mro__])
```

OUTPUT

```
Customer(name='Asha', city='Mysuru', tier='gold')
{"name": "Asha", "city": "Mysuru", "tier": "gold"}
['Customer', 'JsonMixin', 'ReprMixin', 'Person', 'object']
```

Inheritance couples classes tightly: a change to a parent can break every child, and deep hierarchies (or forcing "is a" where it does not fit) make code rigid. **Composition**, building objects that **contain** other objects and delegate work to them, is often more flexible: parts can be swapped at run time and tested separately. Use inheritance for true "is a" relationships with shared behaviour; use composition for "has a" relationships.

COMPOSITION: SWAP PARTS AT RUN TIME

main.py

```
class EmailSender:
    def send(self, to, msg): return f"email to {to}: {msg}"

class SmsSender:
    def send(self, to, msg): return f"SMS to {to}: {msg}"

class Notifier:                                  # HAS a sender, rather than IS a sender
    def __init__(self, sender):
        self.sender = sender
    def remind(self, user, course):
        return self.sender.send(user, f"your {course} class starts at 6 pm")

n = Notifier(EmailSender())
print(n.remind("asha@lbf.in", "Python"))
n.sender = SmsSender()                           # swap the behaviour at run time
print(n.remind("98450 12345", "Python"))
```

OUTPUT

```
email to asha@lbf.in: your Python class starts at 6 pm
SMS to 98450 12345: your Python class starts at 6 pm
```

Inheritance	Composition
"is a" (Dog is an Animal)	"has a" (Car has an Engine)
fixed when the class is written	can change at run time
child depends on parent internals	parts interact through a small interface
easy to over-use, deep hierarchies	more small objects, but flexible and testable

16.7 Advanced class topics (overview)

16.7.1 Descriptors

A **descriptor** is an object, stored as a **class** attribute, that defines `__get__`, and optionally `__set__` and `__delete__`, to control access to an attribute on instances. Descriptors are the machinery behind `@property`, methods, `@classmethod` and `@staticmethod`, and ORMs such as Django and SQLAlchemy use them for model fields. Write one when the same validation logic is needed for many attributes; `__set_name__` tells it the attribute name.

A REUSABLE VALIDATING DESCRIPTOR main.py

```
class Positive:
    def __set_name__(self, owner, name):
        self.private = "_" + name
    def __get__(self, obj, objtype=None):
        if obj is None:
            return self
        return getattr(obj, self.private)
    def __set__(self, obj, value):
        if value <= 0:
            raise ValueError(f"{self.private[1:]} must be positive, got {value}")
        setattr(obj, self.private, value)

class Product:
    price = Positive()          # one descriptor reused for several attributes
    stock = Positive()
    def __init__(self, name, price, stock):
        self.name, self.price, self.stock = name, price, stock

p = Product("pen", 10, 100)
p.price = 12
print(p.price, p.stock, p.__dict__)
try:
    p.stock = 0
except ValueError as e:
    print("ValueError:", e)
```

OUTPUT

```
12 100 {'name': 'pen', '_price': 12, '_stock': 100}
ValueError: stock must be positive, got 0
```

16.7.2 Metaclasses and type

Classes are objects too, and their type is `type`: `type` is the **metaclass** that creates classes. Calling `type(name, bases, namespace)` builds a class dynamically, exactly as a `class` statement does. A custom metaclass (a subclass of `type`) can customise class creation, for example to register or validate every

class defined with it. Metaclasses are rarely needed in application code; class decorators and `__init_subclass__` cover most use cases more simply.

CLASSES ARE CREATED BY TYPE

main.py

```
class Dog:
    sound = "woof"

print(type(42), type(Dog), type(type))

Cat = type("Cat", (), {"sound": "meow", "speak": lambda self: f"The cat says {self.sound}"})
print(Cat, Cat().speak())

class UpperAttrs(type):                      # a metaclass
    def __new__(mcls, name, bases, ns):
        ns = {k if k.startswith("__") else k.upper(): v for k, v in ns.items()}
        return super().__new__(mcls, name, bases, ns)

class Settings(metaclass=UpperAttrs):
    debug = True
    port = 8080
print(Settings.DEBUG, Settings.PORT, hasattr(Settings, "port"))
```

OUTPUT

```
<class 'int'> <class 'type'> <class 'type'>
<class '__main__.Cat'> The cat says meow
True 8080 False
```

16.7.3 __init_subclass__

`__init_subclass__` (3.6) is a class method that Python calls on a parent class **whenever a subclass of it is defined**. It is the simple, modern way to register plugins, enforce rules on subclasses or accept class-level keyword arguments, tasks that used to need a metaclass.

REGISTERING SUBCLASSES AUTOMATICALLY

main.py

```
class Exporter:
    registry = {}

    def __init_subclass__(cls, fmt, **kwargs):
        super().__init_subclass__(**kwargs)
        if not hasattr(cls, "export"):
            raise TypeError(f"{cls.__name__} must define export()")
        Exporter.registry[fmt] = cls            # automatic registration

class CsvExporter(Exporter, fmt="csv"):
    def export(self, rows): return "\n".join(",".join(map(str, r)) for r in rows)

class MarkdownExporter(Exporter, fmt="md"):
    def export(self, rows): return "\n".join("| " + " | ".join(map(str, r)) + " |" for r in rows)

print(Exporter.registry)
rows = [("Asha", 91), ("Ravi", 78)]
for fmt in ("csv", "md"):
    print(Exporter.registry[fmt]().export(rows))
try:
    class PdfExporter(Exporter, fmt="pdf"):
        pass
except TypeError as e:
    print("TypeError:", e)
```

OUTPUT

```
{'csv': <class '__main__.CsvExporter'>, 'md': <class '__main__.MarkdownExporter'>}
Asha,91
Ravi,78
| Asha | 91 |
| Ravi | 78 |
TypeError: PdfExporter must define export()
```

Inheritance builds a child class from a parent ("is a"): single, multilevel, hierarchical and multiple. super() calls the next class in the MRO, which Python computes with C3 linearisation (children before parents, each class once); isinstance and issubclass respect inheritance. Polymorphism comes from overriding and duck typing: any object with the right methods works. Python has no overloading; use defaults, *args and keyword arguments. Abstract base classes (abc.ABC, @abstractmethod) enforce an interface; typing.Protocol describes one structurally. Dunder methods connect classes to syntax: __add__/__radd__ and friends (return NotImplemented for unsupported types), __eq__/__lt__ with total_ordering, __len__/__getitem__/__setitem__/__contains__/__iter__, __call__, __hash__ (consistent with __eq__, for immutable data), __enter__/__exit__ and __getattr__/__setattr__. @dataclass generates __init__, __repr__ and __eq__ (plus order, frozen, slots, field() and __post_init__); Enum, auto, StrEnum and Flag define named constants. Mixins add single features; prefer composition for "has a". Descriptors, metaclasses (type) and __init_subclass__ customise attribute access and class creation.

PRACTICE CHECKPOINT

Write an **abstract Shape** class with **Circle** and **Rectangle** subclasses, a **Vector** class supporting `+`, `==` and `len()`, and a **Student dataclass** sorted by marks.

PART 1: AN ABSTRACT SHAPE HIERARCHY

main.py

```
from abc import ABC, abstractmethod
import math

class Shape(ABC):
    @abstractmethod
    def area(self): ...

    @abstractmethod
    def perimeter(self): ...

    def __str__(self):
        return f"{type(self).__name__:<9} area {self.area():>7.2f}  perimeter
{self.perimeter():>6.2f}"

class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius
    def area(self):
        return math.pi * self.radius ** 2
    def perimeter(self):
        return 2 * math.pi * self.radius

class Rectangle(Shape):
    def __init__(self, width, height):
        self.width, self.height = width, height
    def area(self):
        return self.width * self.height
    def perimeter(self):
        return 2 * (self.width + self.height)

shapes = [Circle(1.5), Rectangle(4, 2.5), Circle(0.5)]
for s in sorted(shapes, key=lambda s: s.area()):
    print(s)
try:
    Shape()
except TypeError as e:
    print("TypeError:", e)
```

OUTPUT

```
Circle    area    0.79  perimeter   3.14
Circle    area    7.07  perimeter   9.42
Rectangle area   10.00  perimeter  13.00
TypeError: Can't instantiate abstract class Shape without an implementation for abstract methods
'area', 'perimeter'
```

PART 2: A VECTOR WITH +, == AND LEN()

main.py

```
class Vector:
    def __init__(self, *components):
        self.components = tuple(components)
    def __repr__(self):
        return f"Vector{self.components}"
    def __len__(self):
        return len(self.components)              # number of dimensions
    def __add__(self, other):
        if not isinstance(other, Vector):
            return NotImplemented
        if len(self) != len(other):
            raise ValueError("vectors must have the same dimension")
        return Vector(*(a + b for a, b in zip(self.components, other.components)))
    def __eq__(self, other):
        return isinstance(other, Vector) and self.components == other.components
    def __hash__(self):
        return hash(self.components)

v1, v2 = Vector(1, 2, 3), Vector(4, 5, 6)
print(v1 + v2, len(v1), v1 == Vector(1, 2, 3), v1 == v2)
try:
    v1 + Vector(1, 2)
except ValueError as e:
    print("ValueError:", e)
```

OUTPUT

```
Vector(5, 7, 9) 3 True False
ValueError: vectors must have the same dimension
```

PART 3: A STUDENT DATACLASS SORTED BY MARKS

main.py

```
from dataclasses import dataclass, field

@dataclass(order=True)
class Student:
    marks: float
    name: str = field(compare=False)
    city: str = field(default="Mysuru", compare=False)

students = [Student(78, "Ravi", "Pune"), Student(91, "Asha"), Student(85, "Meera"), Student(64,
"John", "Delhi")]
print("ascending:")
for s in sorted(students):
    print(f"  {s.name:<6} {s.marks}")
print("top student:", max(students).name)
print("ranking:", [s.name for s in sorted(students, reverse=True)])
print(students[1])
```

OUTPUT

```
ascending:
  John   64
  Ravi   78
  Meera  85
  Asha   91
top student: Asha
ranking: ['Asha', 'Meera', 'Ravi', 'John']
Student(marks=91, name='Asha', city='Mysuru')
```

MODULE 17 · INTERMEDIATE

Iterators, generators and context managers

Minimum duration **3 days** | Level **Intermediate**

These features let Python process data lazily, one item at a time, and manage resources safely. They are the key to efficient, Pythonic code. This module covers the iterator protocol, iter() and next(), StopIteration and how a for loop really works, custom iterator classes, generator functions and yield, lazy evaluation and its memory savings, generator expressions, yield from, infinite generators, generator pipelines, send() and coroutine-style generators, the itertools toolkit, a review of all comprehension types, and context managers: the with statement, class-based context managers, @contextmanager, suppress and ExitStack.

IN THIS MODULE

17.1 Iterables and iterators
17.2 Generators
17.3 itertools
17.4 Comprehensions review
17.5 Context managers

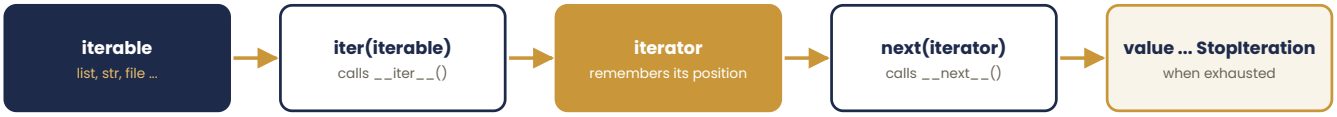
17.1 Iterables and iterators

An **iterable** is any object you can loop over: lists, tuples, strings, dictionaries, sets, files, ranges and many more. An **iterator** is the object that actually produces the values, one at a time, and remembers where it is. Every iterable can give you a fresh iterator; an iterator is used up after one pass.

	Iterable	Iterator
Definition	can be looped over	produces the next value on demand
Required method	`__iter__` (returns an iterator)	`__next__` and `__iter__` (returns itself)
Reusable?	yes, each loop gets a new iterator	no, exhausted after one pass
Examples	list, str, dict, range, set	file objects, generators, `map`, `zip`, `enumerate`, `iter(list)`

17.1.1 The iterator protocol: __iter__ and __next__

The **iterator protocol** is two methods. An iterable implements `__iter__()`, which returns an iterator. An iterator implements `__next__()`, which returns the next value or raises `StopIteration` when there are no more, and also `__iter__()` returning `self`, so that an iterator is itself iterable. Any object following this protocol works in `for` loops, comprehensions, unpacking, `sum`, `list`, `zip` and everything else that consumes iterables.



THE ITERATOR PROTOCOL

17.1.2 iter() and next()

The built-in `iter(obj)` calls `obj.__iter__()` and returns an iterator; `next(it)` calls `it.__next__()`. `next(it, default)` returns `default` instead of raising at the end. Calling `iter()` on an iterator returns the same iterator.

DRIVING AN ITERATOR BY HAND

main.py

```
marks = [91, 78, 85]
it = iter(marks)
print(type(it).__name__)
print(next(it), next(it), next(it))
print(next(it, "no more"))
print(iter(it) is it)                 # an iterator is its own iterator

word_iter = iter("LBF")
print(next(word_iter), list(word_iter), list(word_iter))   # the rest, then nothing

d = {"a": 1, "b": 2}
print(next(iter(d)), next(iter(d.values())), next(iter(d.items())))
it1, it2 = iter(marks), iter(marks)    # each call gives an independent iterator
print(next(it1), next(it1), next(it2))
```

OUTPUT

```
list_iterator
91 78 85
no more
True
L ['B', 'F'] []
a 1 ('a', 1)
91 78 91
```

17.1.3 StopIteration

When an iterator has no more values, `__next__` raises `StopIteration`. This is not an error condition: it is the normal signal that iteration is over, and loops and other consumers catch it silently. You only see it when calling `next()` yourself without a default.

STOPITERATION AND NEXT() WITH A DEFAULT main.py

```
it = iter([1, 2])
next(it); next(it)
try:
    next(it)
except StopIteration:
    print("StopIteration: the iterator is exhausted")

first_even = next((n for n in [3, 7, 10, 12] if n % 2 == 0), None)
first_neg = next((n for n in [3, 7, 10] if n < 0), None)
print(first_even, first_neg)
```

OUTPUT

```
StopIteration: the iterator is exhausted
10 None
```

17.1.4 How a for loop works internally

A `for` loop is syntactic sugar for the iterator protocol. `for x in items: body` calls `iter(items)` once, then repeatedly calls `next()` on the iterator, binding each value to `x` and running the body, until `StopIteration` is raised, which ends the loop (and runs an `else` clause if present).

WHAT A FOR LOOP DOES BEHIND THE SCENES main.py

```
# for item in iterable:
#     print(item)
# is equivalent to:
iterator = iter(iterable)
while True:
    try:
        item = next(iterator)
    except StopIteration:
        break
    print(item)
```

A FOR LOOP WRITTEN WITH WHILE AND NEXT() main.py

```
def manual_for(iterable, action):
    iterator = iter(iterable)
    while True:
        try:
            item = next(iterator)
        except StopIteration:
            break
        action(item)

manual_for(["Asha", "Ravi"], lambda name: print("hello", name))

numbers = iter([1, 2, 3, 4, 5, 6])
for n in numbers:
    if n == 3:
        break
print("loop stopped at 3; the iterator continues from:", list(numbers))
```

OUTPUT

```
hello Asha
hello Ravi
loop stopped at 3; the iterator continues from: [4, 5, 6]
```

17.1.5 Writing a custom iterator class

To make your own iterator, write a class with `__iter__` returning `self` and `__next__` returning the next value or raising `StopIteration`. Iterator objects keep their position in instance attributes. A common design separates the two roles: the **iterable** class returns a **new** iterator from `__iter__`, so it can be looped over many times.

CUSTOM ITERATOR AND ITERABLE CLASSES main.py

```
class Countdown:
    """Iterator: counts down from start to 1, then stops."""
    def __init__(self, start):
        self.current = start

    def __iter__(self):
        return self

    def __next__(self):
        if self.current <= 0:
            raise StopIteration
        value = self.current
        self.current -= 1
        return value

c = Countdown(3)
print(list(c), list(c))            # exhausted after one pass

class DateRange:
    """Iterable: can be looped over any number of times."""
    def __init__(self, start, days):
        self.start, self.days = start, days
    def __iter__(self):
        from datetime import timedelta
        return (self.start + timedelta(days=i) for i in range(self.days))   # a new iterator each time

from datetime import date
week = DateRange(date(2026, 1, 26), 3)
print([d.strftime("%a %d") for d in week])
print([d.day for d in week])       # works again
```

OUTPUT

```
[3, 2, 1] []
['Mon 26', 'Tue 27', 'Wed 28']
[26, 27, 28]
```

17.2 Generators

17.2.1 Generator functions and yield

Writing iterator classes is verbose. A **generator function** is the easy way: a normal `def` containing `yield`. Calling it does not run the body; it returns a **generator** object, which is an iterator. Each `next()` runs the body until the next `yield`, hands that value out, and **pauses**, keeping all local variables. The next `next()` resumes right after the `yield`. When the function returns, the generator raises `StopIteration` automatically.

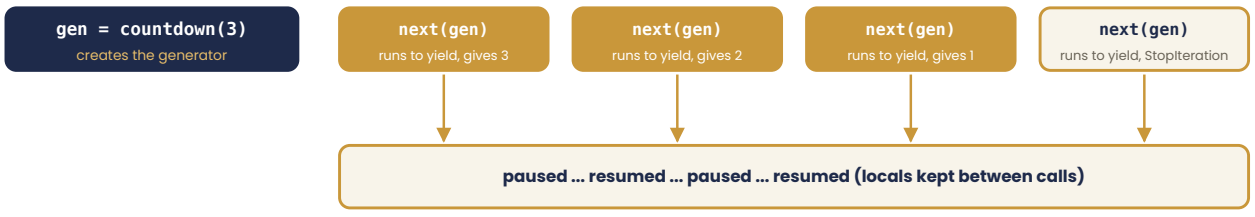
A GENERATOR FUNCTION PAUSES AT EACH YIELD

main.py

```
def countdown(start):
    print("  [generator starts]")
    while start > 0:
        yield start                 # hand out a value and pause here
        start -= 1
    print("  [generator finished]")

gen = countdown(3)
print(type(gen).__name__, "- nothing has run yet")
print(next(gen))
print(next(gen))
print(next(gen))
print(next(gen, "done"))

for n in countdown(2):              # the usual way to consume a generator
    print("for got", n)
```

OUTPUT

```
generator - nothing has run yet
  [generator starts]
3
2
1
  [generator finished]
done
  [generator starts]
for got 2
for got 1
  [generator finished]
```



A GENERATOR RUNS ONE SLICE OF ITS BODY PER NEXT() CALL

A generator function can contain several `yield` statements and any control flow. A `return value` inside a generator ends it; the value is attached to the `StopIteration` and used by `yield from`.

SEVERAL YIELDS AND A GENERATOR WITH A LOOP main.py

```
def traffic_light():
    yield "green"
    yield "amber"
    yield "red"

print(list(traffic_light()))

def fibonacci(limit):
    a, b = 0, 1
    while a <= limit:
        yield a
        a, b = b, a + b

print(list(fibonacci(100)))
print(sum(fibonacci(4_000_000)))
```

OUTPUT

```
['green', 'amber', 'red']
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89]
9227464
```

17.2.2 Lazy evaluation and memory savings

Generators are **lazy**: they compute each value only when asked, and never store the whole sequence. A list of a million squares holds a million objects in memory; a generator of a million squares holds only its current state, a few hundred bytes, however long the sequence. Laziness also means work stops as soon as the consumer stops: `next()` on a generator of "all primes" finds just the first one. The trade-off is that a generator can be iterated only once and has no `len()` or indexing.

LAZY EVALUATION IN ACTION main.py

```
import sys

squares_list = [n * n for n in range(1_000_000)]
squares_gen = (n * n for n in range(1_000_000))
print(f"list:      {sys.getsizeof(squares_list):>9,} bytes")
print(f"generator: {sys.getsizeof(squares_gen):>9,} bytes")
print(sum(squares_list) == sum(squares_gen))

def slow_squares():
    for n in range(10):
        print(f"  computing {n}^2")
        yield n * n

big = (sq for sq in slow_squares() if sq > 10)
print("first square over 10:", next(big))        # only the needed work is done
```

OUTPUT

```
list:      8,448,728 bytes
generator:       208 bytes
True
  computing 0^2
  computing 1^2
  computing 2^2
  computing 3^2
  computing 4^2
first square over 10: 16
```

MEMORY USED BY THE CONTAINER OBJECT ITSELF (THE LIST ALSO HOLDS A MILLION INT OBJECTS)

17.2.3 Generator expressions (x for x in ...)

A **generator expression** looks like a list comprehension with parentheses instead of brackets: `(x * x for x in data if x > 0)`. It creates a lazy generator instead of a list. When it is the only argument of a function call, the extra parentheses can be dropped: `sum(x * x for x in data)`. Use them whenever the result is consumed once, by `sum`, `max`, `any`, `all`, `"".join`, `sorted` or a `for` loop.

GENERATOR EXPRESSIONS

main.py

```
marks = [91, 78, 85, 64, 99, 47]
print(sum(m for m in marks if m >= 50))
print(max(len(w) for w in ["python", "is", "fun"]))
print(any(m < 50 for m in marks), all(m > 40 for m in marks))
print(", ".join(str(m) for m in sorted(marks)[:3]))

gen = (m * 2 for m in marks)
print(gen)
print(list(gen), list(gen))        # one pass only
```

OUTPUT

```
417
6
True True
47, 64, 78
<generator object <genexpr> at 0x7f7ead5f0ee0>
[182, 156, 170, 128, 198, 94] []
```

17.2.4 yield from

`yield from iterable` yields every value from another iterable or generator, as if you had written `for x in iterable: yield x`. It is the clean way for a generator to delegate to sub-generators, which is especially natural for recursive structures such as trees and nested lists. It also passes values sent with `send()` through, and evaluates to the sub-generator's `return` value.

DELEGATING WITH YIELD FROM

main.py

```
def numbers():
    yield from range(3)
    yield from [10, 20]
    yield from "ab"

print(list(numbers()))

def flatten(nested):
    for item in nested:
        if isinstance(item, list):
            yield from flatten(item)        # recursive delegation
        else:
            yield item

print(list(flatten([1, [2, [3, 4], 5], [[6]], 7])))

def walk(tree, path=""):
    for name, children in tree.items():
        yield path + name
        yield from walk(children, path + name + "/")

site = {"home": {}, "courses": {"python": {}, "java": {}}, "about": {"team": {}}}
print(list(walk(site)))
```

OUTPUT

```
[0, 1, 2, 10, 20, 'a', 'b']
[1, 2, 3, 4, 5, 6, 7]
['home', 'courses', 'courses/python', 'courses/java', 'about', 'about/team']
```

17.2.5 Infinite generators

Because generators produce values on demand, they can represent **infinite** sequences: all natural numbers, an endless stream of IDs, repeating colours. They are safe as long as the consumer takes only a finite number of values, with `next()`, `break`, `zip` with a finite iterable, or `itertools.islice`.

INFINITE GENERATORS CONSUMED SAFELY

main.py

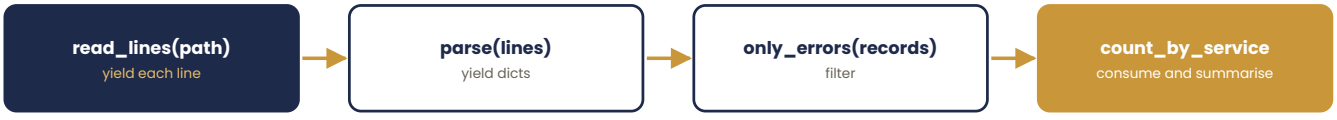
```
from itertools import islice

def natural_numbers(start=1):
    n = start
    while True:                      # never ends on its own
        yield n
        n += 1

def primes():
    found = []
    for n in natural_numbers(2):
        if all(n % p for p in found if p * p <= n):
            found.append(n)
            yield n

def ticket_ids(prefix="LBF"):
    for n in natural_numbers(1001):
        yield f"{prefix}-{n}"

print(list(islice(primes(), 15)))
ids = ticket_ids()
print(next(ids), next(ids), next(ids))
for student, ticket in zip(["Asha", "Ravi"], ticket_ids("EXAM")):
    print(student, ticket)
```

OUTPUT

```
[2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]
LBF-1001 LBF-1002 LBF-1003
Asha EXAM-1001
Ravi EXAM-1002
```

17.2.6 Generator pipelines for data processing

Generators can be chained into **pipelines**, like Unix pipes: each stage takes an iterable, transforms or filters it lazily, and yields results to the next stage. Data flows through one item at a time, so even a file of many gigabytes is processed with constant memory, and each stage is a small, testable function.



A LAZY PIPELINE: ONE RECORD FLOWS THROUGH ALL STAGES AT A TIME

PROCESSING A LOG FILE WITH A GENERATOR PIPELINE

main.py

```
from collections import Counter

with open("server.log", "w") as f:
    services = ["auth", "payments", "search", "auth", "video"]
    for i in range(10_000):
        level = "ERROR" if i % 37 == 0 else ("WARN" if i % 11 == 0 else "INFO")
        f.write(f"2026-01-26T10:{i % 60:02d} {level} {services[i % 5]} request {i}\n")

def read_lines(path):
    with open(path) as f:
        for line in f:
            yield line.rstrip("\n")

def parse(lines):
    for line in lines:
        time, level, service, *message = line.split()
        yield {"time": time, "level": level, "service": service, "message": " ".join(message)}

def only(level, records):
    return (r for r in records if r["level"] == level)

errors = only("ERROR", parse(read_lines("server.log")))
print(next(errors))                                    # nothing else has been read yet
counts = Counter(r["service"] for r in errors)         # consume the rest
print(counts.most_common())
```

OUTPUT

```
{'time': '2026-01-26T10:00', 'level': 'ERROR', 'service': 'auth', 'message': 'request 0'}
[('auth', 108), ('search', 54), ('video', 54), ('payments', 54)]
```

17.2.7 send() and coroutine-style generators (overview)

Generators can also **receive** values. `gen.send(value)` resumes the generator and makes the paused `yield` expression evaluate to `value`. The generator must first be advanced to its first `yield` ("primed") with `next(gen)` or `gen.send(None)`. `gen.close()` stops it, and `gen.throw(exc)` raises an exception inside it. This "coroutine" style was the basis of early asynchronous Python; modern code uses `async` / `await` (Module 20) instead, but `send()` remains handy for running calculations.

SENDING VALUES INTO A GENERATOR main.py

```
def running_average():
    total, count, average = 0, 0, None
    while True:
        value = yield average           # receives what send() passes
        total += value
        count += 1
        average = total / count

avg = running_average()
next(avg)                               # prime: run to the first yield
for mark in (80, 90, 70, 100):
    print(f"sent {mark:>3} -> average {avg.send(mark):.1f}")
avg.close()
try:
    avg.send(50)
except StopIteration:
    print("closed generators cannot be resumed")
```

OUTPUT

```
sent  80 -> average 80.0
sent  90 -> average 85.0
sent  70 -> average 80.0
sent 100 -> average 85.0
closed generators cannot be resumed
```

17.3 itertools

The `itertools` module provides fast, memory-efficient building blocks for working with iterators. All its functions return lazy iterators. Below are the most useful groups.

17.3.1 count, cycle, repeat

Infinite iterators: `count(start, step)` counts forever; `cycle(iterable)` repeats an iterable's items endlessly; `repeat(obj, times)` yields the same object, forever or a fixed number of times. Combine them with `zip`, `islice` or `map` to stop.

COUNT, CYCLE AND REPEAT main.py

```
from itertools import count, cycle, repeat, islice

print(list(islice(count(100, 5), 4)))
for n, colour in zip(range(1, 6), cycle(["red", "green", "blue"])):
    print(n, colour, end=" | ")
print()
print(list(repeat("-", 5)), list(map(pow, range(5), repeat(2))))
for roll, name in zip(count(101), ["Asha", "Ravi", "Meera"]):
    print(roll, name)
```

OUTPUT

```
[100, 105, 110, 115]
1 red | 2 green | 3 blue | 4 red | 5 green |
['-', '-', '-', '-', '-'] [0, 1, 4, 9, 16]
101 Asha
102 Ravi
103 Meera
```

17.3.2 chain, islice, zip_longest

`chain(a, b, ...)` iterates several iterables one after another as one sequence (`chain.from_iterable` for a list of iterables). `islice(iterable, start, stop, step)` slices any iterator, including infinite ones and files, without copying. `zip_longest` zips to the **longest** input, filling gaps with `fillvalue`, whereas `zip` stops at the shortest.

CHAIN, ISLICE AND ZIP_LONGEST main.py

```
from itertools import chain, islice, zip_longest

morning, evening = ["Asha", "Ravi"], ["Meera", "John", "Zoya"]
print(list(chain(morning, evening, ["Kiran"])))
print(list(chain.from_iterable([[1, 2], [3], [4, 5]])))

with open("lines.txt", "w") as f:
    f.write("".join(f"line {i}\n" for i in range(1, 101)))
with open("lines.txt") as f:
    print([l.strip() for l in islice(f, 10, 13)])        # lines 11 to 13 without reading all

names, marks = ["Asha", "Ravi", "Meera"], [91, 78]
print(list(zip(names, marks)))
print(list(zip_longest(names, marks, fillvalue="absent")))
```

OUTPUT

```
['Asha', 'Ravi', 'Meera', 'John', 'Zoya', 'Kiran']
[1, 2, 3, 4, 5]
['line 11', 'line 12', 'line 13']
[('Asha', 91), ('Ravi', 78)]
[('Asha', 91), ('Ravi', 78), ('Meera', 'absent')]
```

17.3.3 product, permutations, combinations

Combinatoric iterators: `product(a, b)` gives every pairing (the Cartesian product, like nested loops; `repeat=n` for the same iterable n times); `permutations(items, r)` gives every ordering of r items; `combinations(items, r)` gives every selection of r items where order does not matter; `combinations_with_replacement` allows repeats.

COMBINATORIC ITERATORS main.py

```
from itertools import product, permutations, combinations, combinations_with_replacement
from math import comb, perm

print(list(product(["S", "M", "L"], ["red", "blue"])))
print(["".join(p) for p in product("01", repeat=3)])
print(list(permutations("ABC", 2)))
print(list(combinations(["Asha", "Ravi", "Meera", "John"], 2)))
print(list(combinations_with_replacement("XY", 2)))
print(len(list(permutations(range(5), 3))), perm(5, 3), "|", len(list(combinations(range(10), 3))),
comb(10, 3))
```

OUTPUT

```
[('S', 'red'), ('S', 'blue'), ('M', 'red'), ('M', 'blue'), ('L', 'red'), ('L', 'blue')]
['000', '001', '010', '011', '100', '101', '110', '111']
[('A', 'B'), ('A', 'C'), ('B', 'A'), ('B', 'C'), ('C', 'A'), ('C', 'B')]
[('Asha', 'Ravi'), ('Asha', 'Meera'), ('Asha', 'John'), ('Ravi', 'Meera'), ('Ravi', 'John'), ('Meera',
'John')]
[('X', 'X'), ('X', 'Y'), ('Y', 'Y')]
60 60 | 120 120
```

Function	Order matters?	Repeats?	Count for n=4 items, r=2
`product(items, repeat=2)`	yes	yes	16
`permutations(items, 2)`	yes	no	12
`combinations(items, 2)`	no	no	6
`combinations_with_replacement(items, 2)`	no	yes	10

17.3.4 groupby, accumulate, batched (3.12)

`groupby(iterable, key)` groups **consecutive** items with the same key, yielding `(key, group_iterator)` pairs; sort by the same key first to group all equal items. `accumulate(iterable, func)` yields running totals (or running results of any two-argument function). `batched(iterable, n)` (3.12) splits an iterable into tuples of n items, the last possibly shorter, perfect for processing records in chunks.

GROUPBY, ACCUMULATE AND BATCHED

main.py

```
from itertools import groupby, accumulate, batched
import operator

students = [("Asha", "Mysuru"), ("Ravi", "Pune"), ("Meera", "Mysuru"), ("John", "Delhi"), ("Zoya",
"Pune")]
by_city = sorted(students, key=lambda s: s[1])            # groupby needs sorted input
for city, group in groupby(by_city, key=lambda s: s[1]):
    print(city, [name for name, _ in group])

print([(k, len(list(g))) for k, g in groupby("aaabccdddd")])   # run-length encoding

sales = [120, 80, 200, 50, 150]
print(list(accumulate(sales)))                            # running total
print(list(accumulate(sales, max)))                       # running maximum
print(list(accumulate([1, 2, 3, 4, 5], operator.mul)))    # running product

for batch in batched(range(1, 11), 4):
    print("insert batch:", batch)
```

OUTPUT

```
Delhi ['John']
Mysuru ['Asha', 'Meera']
Pune ['Ravi', 'Zoya']
[('a', 3), ('b', 1), ('c', 2), ('d', 4)]
[120, 200, 400, 450, 600]
[120, 120, 200, 200, 200]
[1, 2, 6, 24, 120]
insert batch: (1, 2, 3, 4)
insert batch: (5, 6, 7, 8)
insert batch: (9, 10)
```

17.4 Comprehensions review

17.4.1 List, set, dict comprehensions and generator expressions compared

Python has four comprehension forms with the same structure, `expression for item in iterable if condition`, differing only in brackets and result type. Choose by what you need: a list to keep and index, a set for unique values, a dict for key-value lookups, a generator expression when the values are consumed once.

Form	Syntax	Result	Evaluated
List	`[x * 2 for x in xs]`	`list`	eagerly, all at once
Set	`{x % 3 for x in xs}`	`set` (unique)	eagerly
Dict	`{x: x * x for x in xs}`	`dict`	eagerly
Generator	`(x * 2 for x in xs)`	generator (iterator)	lazily, on demand
(no tuple form)	`tuple(x * 2 for x in xs)`	`tuple`	via a generator

ALL FOUR COMPREHENSIONS SIDE BY SIDE

main.py

```
words = ["python", "java", "go", "rust", "python", "c"]

lengths_list = [len(w) for w in words]
lengths_set = {len(w) for w in words}
length_of = {w: len(w) for w in words}
lengths_gen = (len(w) for w in words)
lengths_tuple = tuple(len(w) for w in words)

print(lengths_list)
print(lengths_set)
print(length_of)
print(lengths_gen, sum(lengths_gen))
print(lengths_tuple)
print([w.upper() if len(w) > 2 else w for w in words if w != "python"])
print({(i, j) for i in range(3) for j in range(3) if i < j})
```

OUTPUT

```
[6, 4, 2, 4, 6, 1]
{1, 2, 4, 6}
{'python': 6, 'java': 4, 'go': 2, 'rust': 4, 'c': 1}
<generator object <genexpr> at 0x7f2aefcfc200> 23
(6, 4, 2, 4, 6, 1)
['JAVA', 'go', 'RUST', 'c']
{(0, 1), (0, 2), (1, 2)}
```

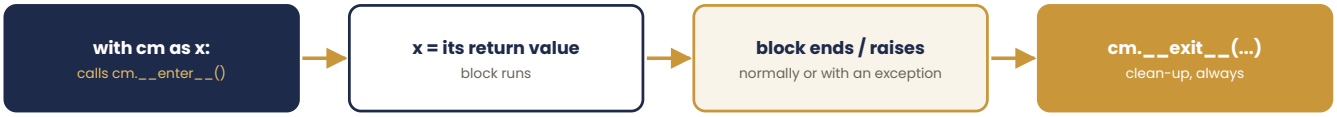
TIP

A comprehension's loop variable is local to the comprehension, so it does not leak into or overwrite variables outside. Keep comprehensions short: if one needs more than two `for` clauses or complex conditions, a normal loop or a generator function is clearer.

17.5 Context managers

17.5.1 The with statement

A **context manager** sets something up at the start of a block and guarantees the matching clean-up at the end, even if an exception occurs or the block returns early. The `with` statement uses one: `with open(path) as f:` opens a file and always closes it. Many standard objects are context managers: files, locks (`threading.Lock`), database connections (`sqlite3`), temporary directories, and `decimal.localcontext`.



HOW THE WITH STATEMENT USES A CONTEXT MANAGER

WHAT WITH DOES BEHIND THE SCENES

main.py

```
# with EXPRESSION as target:
#     body
# is roughly equivalent to:
manager = EXPRESSION
target = manager.__enter__()
try:
    body
except BaseException as exc:
    if not manager.__exit__(type(exc), exc, exc.__traceback__):
        raise                       # re-raise unless __exit__ returned True
else:
    manager.__exit__(None, None, None)
```

CONTEXT MANAGERS FROM THE STANDARD LIBRARY

main.py

```
import sqlite3, tempfile, threading, os
from decimal import Decimal, localcontext

with tempfile.TemporaryDirectory() as folder:          # deleted automatically afterwards
    path = os.path.join(folder, "scratch.txt")
    with open(path, "w") as f:
        f.write("temporary data")
    print("inside:", os.path.exists(path))
print("after:", os.path.exists(path))

lock = threading.Lock()
with lock:                                             # acquired, then always released
    print("lock held:", lock.locked())
print("lock held:", lock.locked())

with localcontext() as ctx:                            # temporary decimal precision
    ctx.prec = 5
    print(Decimal(1) / Decimal(7))
print(Decimal(1) / Decimal(7))

with sqlite3.connect(":memory:") as db:                # commits on success
    db.execute("create table t (x)")
    db.execute("insert into t values (42)")
    print(db.execute("select x from t").fetchone())
```

OUTPUT

```
inside: True
after: False
lock held: True
lock held: False
0.14286
0.14285714285714285714285714286
(42,)
```

17.5.2 Writing a class-based context manager

A class becomes a context manager by defining `__enter__` and `__exit__` (16.4.6). `__enter__` performs the setup and returns the object bound by `as` (often `self`). `__exit__(exc_type, exc_value, traceback)` performs the clean-up; its arguments are `None` if the block succeeded. Return `True` from `__exit__` only if you deliberately want to **suppress** the exception.

CLASS-BASED CONTEXT MANAGERS

main.py

```
import time

class Timer:
    def __init__(self, label):
        self.label = label

    def __enter__(self):
        self.start = time.perf_counter()
        return self

    def __exit__(self, exc_type, exc, tb):
        self.elapsed = time.perf_counter() - self.start
        status = "failed" if exc_type else "done"
        print(f"[{self.label}] {status} in {'under' if self.elapsed < 1 else 'over'} 1 s")
        return False                          # never hide exceptions

with Timer("sum of squares") as t:
    total = sum(n * n for n in range(200_000))
print(total, "| elapsed recorded:", t.elapsed > 0)

class IgnoreZeroDivision:
    def __enter__(self):
        return self
    def __exit__(self, exc_type, exc, tb):
        if exc_type is ZeroDivisionError:
            print("  ignored:", exc)
            return True                       # suppress only this exception type
        return False

for d in (2, 0):
    with IgnoreZeroDivision():
        print("result:", 10 / d)
print("program continues")
```

OUTPUT

```
[sum of squares] done in under 1 s
2666646666700000 | elapsed recorded: True
result: 5.0
  ignored: division by zero
program continues
```

17.5.3 contextlib.contextmanager decorator

The `@contextmanager` decorator from `contextlib` turns a **generator function** with a single `yield` into a context manager: code before `yield` is the setup, the yielded value is bound by `as`, and code after `yield` is the clean-up. Put the `yield` inside `try`/`finally` so the clean-up also runs when the block raises.

CONTEXT MANAGERS FROM GENERATORS main.py

```
from contextlib import contextmanager
import os, time

@contextmanager
def timer(label):
    start = time.perf_counter()
    try:
        yield                                   # the with-block runs here
    finally:
        print(f"[{label}] finished, timing recorded: {time.perf_counter() - start > 0}")

@contextmanager
def working_directory(path):
    previous = os.getcwd()
    os.makedirs(path, exist_ok=True)
    os.chdir(path)
    try:
        yield path
    finally:
        os.chdir(previous)                      # always go back

@contextmanager
def tag(name):
    print(f"<{name}>")
    yield
    print(f"</{name}>")

with timer("build report"):
    report = [n ** 2 for n in range(10_000)]

with working_directory("reports") as folder:
    print("inside:", os.path.basename(os.getcwd()))
print("back in:", os.path.basename(os.getcwd()) != "reports")

with tag("ul"):
    for item in ("Python", "Java"):
        with tag("li"):
            print(f"  {item}")
```

OUTPUT

```
[build report] finished, timing recorded: True
inside: reports
back in: True
<ul>
<li>
  Python
</li>
<li>
  Java
</li>
</ul>
```

17.5.4 contextlib.suppress, ExitStack

`contextlib` has more helpers. `suppress(*exceptions)` ignores the listed exceptions inside its block, a concise replacement for `try`/`except`/`pass` when ignoring is truly intended. `ExitStack` manages a **variable number** of context managers, entering them one by one and exiting them all in reverse order, even if one fails; it is ideal for opening a list of files. Others include `redirect_stdout`, `closing` and `nullcontext`.

SUPPRESS, EXITSTACK AND REDIRECT_STDOUT

main.py

```
from contextlib import suppress, ExitStack, redirect_stdout
import os, io

with suppress(FileNotFoundError):
    os.remove("does_not_exist.tmp")              # no error if it is missing
print("carried on")

for name in ("jan.csv", "feb.csv", "mar.csv"):
    with open(name, "w") as f:
        f.write(f"{name[:3]},{len(name) * 10}\n")

with ExitStack() as stack:
    files = [stack.enter_context(open(name)) for name in ("jan.csv", "feb.csv", "mar.csv")]
    print([f.readline().strip() for f in files])
    stack.callback(print, "ExitStack: all files closed")
print(all(f.closed for f in files))

buffer = io.StringIO()
with redirect_stdout(buffer):                    # capture printed output
    print("this goes into the buffer")
print("captured:", buffer.getvalue().strip())
```

OUTPUT

```
carried on
['jan,70', 'feb,70', 'mar,70']
ExitStack: all files closed
True
captured: this goes into the buffer
```

An iterable has __iter__ returning an iterator; an iterator has __next__ (raising StopIteration at the end) and __iter__ returning itself. iter() and next() drive them by hand, and a for loop is just iter() plus repeated next() until StopIteration. Generator functions use yield to pause and resume, producing values lazily with almost no memory; generator expressions are lazy comprehensions; yield from delegates to another iterable; generators can be infinite and chained into pipelines; send() passes values in. itertools offers count, cycle, repeat, chain, islice, zip_longest, product, permutations, combinations, groupby (on sorted data), accumulate and batched. List, set and dict comprehensions build collections eagerly; generator expressions are lazy. Context managers guarantee clean-up through the with statement: write them as classes with __enter__/__exit__ or as generators with @contextmanager, and use suppress and ExitStack for special cases.

PRACTICE CHECKPOINT

Write a **generator** that reads a large log file line by line and yields only error lines, a custom **Countdown** iterator, and a **Timer** context manager.

PART 1: A GENERATOR THAT YIELDS ERROR LINES

main.py

```
def error_lines(path, keyword="ERROR"):
    """Yield (line number, text) for each line containing keyword, reading lazily."""
    with open(path, encoding="utf-8") as f:
        for number, line in enumerate(f, start=1):
            if keyword in line:
                yield number, line.rstrip("\n")

with open("app.log", "w", encoding="utf-8") as f:            # a large sample log
    for i in range(1, 200_001):
        level = "ERROR" if i % 40_000 == 0 else "INFO"
        f.write(f"2026-01-26 {level} worker-{i % 8} processed job {i}\n")

found = 0
for number, text in error_lines("app.log"):
    print(f"line {number:>7}: {text}")
    found += 1
print("error lines:", found)
```

OUTPUT

```
line   40000: 2026-01-26 ERROR worker-0 processed job 40000
line   80000: 2026-01-26 ERROR worker-0 processed job 80000
line  120000: 2026-01-26 ERROR worker-0 processed job 120000
line  160000: 2026-01-26 ERROR worker-0 processed job 160000
line  200000: 2026-01-26 ERROR worker-0 processed job 200000
error lines: 5
```

PART 2: A CUSTOM COUNTDOWN ITERATOR

main.py

```
class Countdown:
    def __init__(self, start, step=1):
        if start < 0 or step <= 0:
            raise ValueError("start must be >= 0 and step > 0")
        self.current, self.step = start, step

    def __iter__(self):
        return self

    def __next__(self):
        if self.current < 0:
            raise StopIteration
        value = self.current
        self.current -= self.step
        return value

for n in Countdown(5):
    print(n, end=" ")
print("lift off!")
print(list(Countdown(20, 5)))
it = Countdown(1)
print(next(it), next(it), next(it, "finished"))
```

OUTPUT

```
5 4 3 2 1 0 lift off!
[20, 15, 10, 5, 0]
1 0 finished
```

PART 3: A TIMER CONTEXT MANAGER

main.py

```
import time
from contextlib import contextmanager

class Timer:
    def __init__(self, label="block"):
        self.label = label
        self.elapsed = None
    def __enter__(self):
        self._start = time.perf_counter()
        return self
    def __exit__(self, exc_type, exc, tb):
        self.elapsed = time.perf_counter() - self._start
        outcome = f"raised {exc_type.__name__}" if exc_type else "completed"
        print(f"[Timer] {self.label} {outcome}")
        return False

with Timer("sleep 0.2 s") as t:
    time.sleep(0.2)
print("measured at least 0.2 s:", t.elapsed >= 0.2)

try:
    with Timer("division") as t:
        1 / 0
except ZeroDivisionError:
    print("exception still reached the caller; time recorded:", t.elapsed is not None)

@contextmanager
def timer(label):                              # the same idea with @contextmanager
    start = time.perf_counter()
    try:
        yield
    finally:
        print(f"[timer] {label} took {'less' if time.perf_counter() - start < 1 else 'more'} than 1
s")

with timer("sorting 100,000 numbers"):
    sorted(range(100_000, 0, -1))
```

OUTPUT

```
[Timer] sleep 0.2 s completed
measured at least 0.2 s: True
[Timer] division raised ZeroDivisionError
exception still reached the caller; time recorded: True
[timer] sorting 100,000 numbers took less than 1 s
```

LEVEL 3

Advanced level

Outcome: master the standard library, type hints, concurrency, internals, testing and packaging, databases and the web, data analysis, data structures and interview readiness.

8 MODULES · 31 DAYS MINIMUM

Module 18 · Standard library essentials	4 days
Module 19 · Type hints and modern Python features	2 days
Module 20 · Concurrency and parallelism	4 days
Module 21 · Python internals and performance	2 days
Module 22 · Testing, debugging, code quality and packaging	3 days
Module 23 · Databases, web scraping and web development	5 days
Module 24 · Data analysis and visualisation	5 days
Module 25 · Data structures, projects, interviews and career paths	6 days

MODULE 18 · ADVANCED

Standard library essentials

Minimum duration **4 days** | Level **Advanced**

Python's standard library solves most everyday problems without installing anything. This module goes deep into the modules used most in real work: the collections containers, dates, times and time zones, the operating system, environment variables and subprocess, regular expressions from metacharacters to lookarounds and validation, maths and data modules including decimal, fractions, heapq and bisect, the logging framework, command-line tools with argparse, and a set of other useful modules: copy, pprint, textwrap, string, hashlib, secrets, uuid, base64, urllib, http, zipfile and tarfile.

IN THIS MODULE

18.1 collections
18.2 Date and time
18.3 Operating system and files
18.4 Regular expressions in depth (re)
18.5 Maths and data
18.6 Logging
18.7 Command-line tools
18.8 Other useful modules

18.1 collections

18.1.1 deque, Counter, defaultdict, namedtuple, OrderedDict, ChainMap

The `collections` module provides specialised containers that extend the built-in list, dict and tuple. You met several of them in Modules 8 and 9; here they are together, with the situations where each one is the right tool.

Container	What it is	Use it for
`deque`	double-ended queue: O(1) append and pop at both ends	queues, sliding windows, "last N items" with `maxlen`
`Counter`	dict subclass counting hashable items	frequencies, top-N with `most_common`, multiset arithmetic
`defaultdict`	dict that creates missing values with a factory	grouping (`list`), counting (`int`), nested data
`namedtuple`	tuple subclass with named fields	lightweight immutable records, readable function results
`OrderedDict`	dict with `move_to_end` and order-sensitive equality	LRU caches, reordering
`ChainMap`	view over several dicts searched in order	layered settings: arguments → environment → defaults

THE COLLECTIONS CONTAINERS AT WORK

main.py

```
from collections import deque, Counter, defaultdict, namedtuple, OrderedDict, ChainMap

recent = deque(maxlen=3)                          # keeps only the last 3 items
for page in ["home", "courses", "python", "java", "contact"]:
    recent.append(page)
print("recent pages:", list(recent))
queue = deque(["Asha", "Ravi"]); queue.appendleft("VIP"); queue.rotate(1)
print("queue:", queue, "| served:", queue.popleft())

words = "to be or not to be that is the question to ask".split()
c = Counter(words)
print(c.most_common(3), c["be"], c["python"])
print(Counter("aab") + Counter("abc"), Counter("aabbb") - Counter("ab"))

by_grade = defaultdict(list)
for name, grade in [("Asha", "A"), ("Ravi", "B"), ("Meera", "A"), ("John", "C")]:
    by_grade[grade].append(name)
print(dict(by_grade))
tree = lambda: defaultdict(tree)                   # auto-vivifying nested dict
config = tree(); config["db"]["primary"]["host"] = "localhost"
print(config["db"]["primary"]["host"])

Student = namedtuple("Student", ["name", "roll", "marks"], defaults=[0])
s = Student("Asha", 101, 91)
print(s, s.name, s[2], s._asdict(), s._replace(marks=95), Student("Ravi", 102))

cache = OrderedDict(a=1, b=2, c=3); cache.move_to_end("a"); cache.popitem(last=False)
print(cache, OrderedDict(a=1, b=2) == OrderedDict(b=2, a=1), dict(a=1, b=2) == dict(b=2, a=1))

defaults = {"theme": "light", "lang": "en", "font": 14}
env = {"lang": "kn"}
cli = {"theme": "dark"}
settings = ChainMap(cli, env, defaults)
print(settings["theme"], settings["lang"], settings["font"], len(settings))
```

OUTPUT

```
recent pages: ['python', 'java', 'contact']
queue: deque(['VIP', 'Asha']) | served: Ravi
[('to', 3), ('be', 2), ('or', 1)] 2 0
Counter({'a': 3, 'b': 2, 'c': 1}) Counter({'b': 2, 'a': 1})
{'A': ['Asha', 'Meera'], 'B': ['Ravi'], 'C': ['John']}
localhost
Student(name='Asha', roll=101, marks=91) Asha 91 {'name': 'Asha', 'roll': 101, 'marks': 91}
Student(name='Asha', roll=101, marks=95) Student(name='Ravi', roll=102, marks=0)
OrderedDict({'c': 3, 'a': 1}) False True
dark kn 14 3
```

`deque` deserves special mention for performance: inserting or removing at the **front** of a list is O(n), because every other item must shift, while a deque does it in O(1).

LIST VS DEQUE AT THE FRONT

main.py

```
import timeit
from collections import deque

setup = "from collections import deque; L = list(range(100_000)); D = deque(L)"
t_list = min(timeit.repeat("L.insert(0, 1); L.pop(0)", setup, number=2_000, repeat=5))
t_deque = min(timeit.repeat("D.appendleft(1); D.popleft()", setup, number=2_000, repeat=5))
print("deque at least 10x faster at the front:", t_deque * 10 < t_list)
```

OUTPUT

```
deque at least 10x faster at the front: True
```

18.2 Date and time

18.2.1 datetime, date, time, timedelta

The `datetime` module provides `date` (year, month, day), `time` (hour, minute, second, microsecond), `datetime` (both together) and `timedelta` (a duration). Subtracting dates or datetimes gives a `timedelta`; adding a `timedelta` moves a date. Objects are immutable: methods like `replace` return new objects.

DATE, TIME, DATETIME AND TIMEDELTA

main.py

```
from datetime import date, time, datetime, timedelta

d = date(2026, 1, 26)
print(d, d.year, d.month, d.day, d.weekday(), d.isoweekday(), d.isocalendar().week)
print(d.replace(day=31), date.fromisoformat("2026-08-15"), date(2026, 3, 1) - timedelta(days=1))

t = time(18, 30)
print(t, t.hour, t.minute, t.isoformat(timespec="minutes"))

start = datetime(2026, 1, 26, 9, 15)
end = datetime(2026, 1, 26, 17, 45)
worked = end - start
print(worked, worked.total_seconds() / 3600, "hours")
print(start + timedelta(days=90, hours=2))
print(datetime.combine(d, t), datetime(2026, 1, 26, 9, 0) < datetime(2026, 1, 26, 10, 0))

week = timedelta(weeks=1)
print(week.days, week * 3, timedelta(days=10) // timedelta(days=3))
birthday = date(2005, 3, 14)
today = date(2026, 1, 26)
age = today.year - birthday.year - ((today.month, today.day) < (birthday.month, birthday.day))
print("age:", age, "| days lived:", (today - birthday).days)
```

OUTPUT

```
2026-01-26 2026 1 26 0 1 5
2026-01-31 2026-08-15 2026-02-28
18:30:00 18 30 18:30
8:30:00 8.5 hours
2026-04-26 11:15:00
2026-01-26 18:30:00 True
7 21 days, 0:00:00 3
age: 20 | days lived: 7623
```

TIP

`date.today()` and `datetime.now()` give the current date and time; the examples here use fixed dates so that the output is reproducible.

18.2.2 Formatting and parsing with strftime / strptime

`strftime(format)` ("string format time") turns a date or datetime into text; `datetime.strptime(text, format)` ("string parse time") does the reverse and raises `ValueError` if the text does not match. For machine exchange prefer ISO 8601 (`isoformat()` and `fromisoformat()`), which needs no format string.

Code	Meaning	Example
`%Y` / `%y`	year, 4 / 2 digits	2026 / 26
`%m` / `%B` / `%b`	month number / full name / short name	01 / January / Jan
`%d`	day of month, 2 digits	26
`%A` / `%a`	weekday full / short	Monday / Mon
`%H` / `%I` / `%p`	hour 24h / 12h / AM-PM	18 / 06 / PM
`%M` / `%S`	minute / second	30 / 05
`%j`	day of the year	026
`%z` / `%Z`	UTC offset / zone name	+0530 / IST

STRFTIME AND STRPTIME

main.py

```
from datetime import datetime

dt = datetime(2026, 1, 26, 18, 30, 5)
print(dt.strftime("%d/%m/%Y"))
print(dt.strftime("%A, %d %B %Y at %I:%M %p"))
print(dt.strftime("%Y-%m-%d_%H%M%S.log"))
print(f"{dt:%d %b %Y}")                    # format codes work in f-strings too
print(dt.isoformat())

for text, fmt in [("15/08/2026", "%d/%m/%Y"), ("2026-01-26 18:30", "%Y-%m-%d %H:%M"), ("26 Jan 2026",
"%d %b %Y")]:
    print(repr(text), "->", datetime.strptime(text, fmt))
try:
    datetime.strptime("31/02/2026", "%d/%m/%Y")
except ValueError as e:
    print("ValueError:", e)
print(datetime.fromisoformat("2026-01-26T18:30:00+05:30"))
```

OUTPUT

```
26/01/2026
Monday, 26 January 2026 at 06:30 PM
2026-01-26_183005.log
26 Jan 2026
2026-01-26T18:30:05
'15/08/2026' -> 2026-08-15 00:00:00
'2026-01-26 18:30' -> 2026-01-26 18:30:00
'26 Jan 2026' -> 2026-01-26 00:00:00
ValueError: day 31 must be in range 1..28 for month 2 in year 2026
2026-01-26 18:30:00+05:30
```

18.2.3 Time zones with zoneinfo

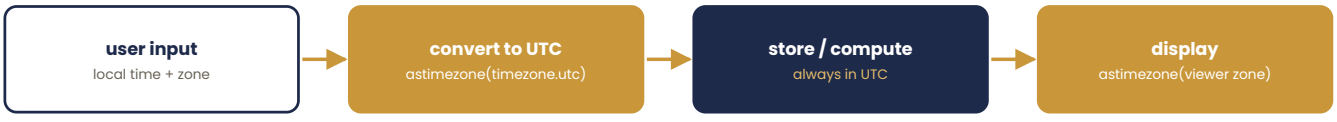
A datetime without time-zone information is **naive**; one with a `tzinfo` is **aware**. Naive datetimes are ambiguous ("9 am where?") and comparing naive with aware datetimes raises `TypeError`. The `zoneinfo` module (3.9) provides real time zones from the IANA database, including daylight-saving rules. Best practice: store and compute in **UTC**, convert to local time only for display with `astimezone`.

AWARE DATETIMES AND TIME-ZONE CONVERSION main.py

```
from datetime import datetime, timezone, timedelta
from zoneinfo import ZoneInfo

ist = ZoneInfo("Asia/Kolkata")
meeting = datetime(2026, 3, 20, 19, 0, tzinfo=ist)        # aware: 7 pm in India
print(meeting, meeting.tzname(), meeting.utcoffset())

for city, zone in [("London", "Europe/London"), ("New York", "America/New_York"), ("Tokyo",
"Asia/Tokyo"), ("Sydney", "Australia/Sydney")]:
    local = meeting.astimezone(ZoneInfo(zone))
    print(f"{city:<9} {local:%a %d %b %H:%M} {local.tzname()}")

utc = meeting.astimezone(timezone.utc)
print("stored as UTC:", utc.isoformat())

summer = datetime(2026, 7, 20, 19, 0, tzinfo=ist).astimezone(ZoneInfo("Europe/London"))
print("same India time in July, London:", f"{summer:%H:%M} {summer.tzname()}")   # daylight saving

naive = datetime(2026, 3, 20, 19, 0)
try:
    naive < meeting
except TypeError as e:
    print("TypeError:", e)
```

OUTPUT

```
2026-03-20 19:00:00+05:30 IST 5:30:00
London    Fri 20 Mar 13:30 GMT
New York  Fri 20 Mar 09:30 EDT
Tokyo     Fri 20 Mar 22:30 JST
Sydney    Sat 21 Mar 00:30 AEDT
stored as UTC: 2026-03-20T13:30:00+00:00
same India time in July, London: 14:30 BST
TypeError: can't compare offset-naive and offset-aware datetimes
```



THE UTC RULE FOR HANDLING TIME ZONES

18.3 Operating system and files

18.3.1 os, sys, pathlib, shutil, glob

Module 14 covered files and paths in detail. This summary shows which module to reach for; `sys` adds information about the interpreter itself.

Module	Main uses
`pathlib`	paths as objects: join, inspect, read/write, glob, mkdir, rename, delete (preferred)
`os`	environment variables, current directory, process IDs, low-level file operations, `os.walk`
`shutil`	copy/move/delete trees, archives, disk usage, `shutil.which` to find programs
`glob`	file name patterns with * ? [] and **
`sys`	command-line arguments, `sys.exit`, stdin/stdout/stderr, module path, interpreter version, recursion limit

INTERPRETER AND SYSTEM INFORMATION main.py

```
import sys, os, shutil
from pathlib import Path

print("Python 3.10 or newer:", sys.version_info >= (3, 10))
print("platform:", sys.platform, "| byte order:", sys.byteorder)
print("max int size:", sys.maxsize == 2**63 - 1, "| float digits:", sys.float_info.dig)
print("argv:", sys.argv)
print("default encoding:", sys.getdefaultencoding())
print("modules loaded:", len(sys.modules) > 10)
print("process id is an int:", isinstance(os.getpid(), int), "| CPU count:", os.cpu_count() >= 1)
print("git found on PATH:", shutil.which("git") is not None, "| no-such-tool:", shutil.which("no-such-
tool"))
print("home exists:", Path.home().exists())
sys.stderr.write("this message goes to standard error\n")
sys.stdout.write("this goes to standard output\n")
```

OUTPUT

```
Python 3.10 or newer: True
platform: linux | byte order: little
max int size: True | float digits: 15
argv: ['main.py']
default encoding: utf-8
modules loaded: True
process id is an int: True | CPU count: True
git found on PATH: True | no-such-tool: None
home exists: True
this message goes to standard error
this goes to standard output
```

`sys.exit(code)` ends a program with an exit status: `0` means success and any other number means failure, which shells and scripts check. `sys.exit("message")` prints the message to standard error and exits with status 1.

CHECK_AGE.PY Python

```
import sys

if len(sys.argv) != 2 or not sys.argv[1].isdigit():
    sys.exit("usage: python check_age.py AGE")
age = int(sys.argv[1])
print("adult" if age >= 18 else "minor")
sys.exit(0 if age >= 18 else 3)
```

EXIT STATUSES FROM SYS.EXIT

main.py

```
python check_age.py 25; echo "exit status: $?"
python check_age.py 12; echo "exit status: $?"
python check_age.py 2>&1; echo "exit status: $?"
```

TERMINAL

```
$ python check_age.py 25; echo "exit status: $?"
adult
exit status: 0
$ python check_age.py 12; echo "exit status: $?"
minor
exit status: 3
$ python check_age.py 2>&1; echo "exit status: $?"
usage: python check_age.py AGE
exit status: 1
```

18.3.2 Environment variables

Environment variables are key-value settings that the operating system passes to every program: `PATH`, `HOME`, `LANG`, and your own such as `DATABASE_URL` or `API_KEY`. They are the standard way to configure an application per machine and to keep **secrets out of source code**. Read them from `os.environ` (a dict-like mapping) with `os.environ.get(name, default)` or `os.getenv`; values are always strings. Changes made in Python affect only the current process and the programs it starts.

READING CONFIGURATION FROM THE ENVIRONMENT

main.py

```
import os

print("HOME is set:", "HOME" in os.environ)
print("PATH entries:", len(os.environ["PATH"].split(os.pathsep)) > 0)

port = int(os.environ.get("APP_PORT", "8000"))            # with a default, converted
debug = os.getenv("APP_DEBUG", "false").lower() in ("1", "true", "yes")
print("port:", port, "| debug:", debug)

os.environ["APP_DEBUG"] = "true"                          # for this process and its children
print(os.getenv("APP_DEBUG"))
try:
    secret = os.environ["PAYMENT_API_KEY"]
except KeyError:
    print("PAYMENT_API_KEY is not set: refusing to start the payment module")
```

OUTPUT

```
HOME is set: True
PATH entries: True
port: 8000 | debug: False
true
PAYMENT_API_KEY is not set: refusing to start the payment module
```

SETTING ENVIRONMENT VARIABLES FOR A PROGRAM (LINUX/MACOS SHELL) main.py

```
APP_PORT=9090 APP_DEBUG=yes python -c "import os; print(os.environ['APP_PORT'],
os.environ['APP_DEBUG'])"
export GREETING=Namaste; python -c "import os; print(os.getenv('GREETING', 'Hello'), 'from the
environment')"
```

TERMINAL

```
$ APP_PORT=9090 APP_DEBUG=yes python -c "import os; print(os.environ['APP_PORT'],
os.environ['APP_DEBUG'])"
9090 yes
$ export GREETING=Namaste; python -c "import os; print(os.getenv('GREETING', 'Hello'), 'from the
environment')"
Namaste from the environment
```

NOTE

Many projects keep development settings in a `.env` file (never committed to Git) and load it with the third-party `python-dotenv` package. On Windows PowerShell, set a variable with `$env:APP_PORT = "9090"`.

18.3.3 subprocess for running other programs

`subprocess.run(args)` runs another program and waits for it to finish. Pass the command as a **list** of arguments; `capture_output=True, text=True` collects its output as strings; `check=True` raises `CalledProcessError` if it fails; `timeout=` stops it if it hangs. The result has `returncode`, `stdout` and `stderr`. Avoid `shell=True` with any user-supplied text: it lets a crafted input run arbitrary shell commands (command injection).

RUNNING OTHER PROGRAMS WITH SUBPROCESS

main.py

```
import subprocess, sys

result = subprocess.run([sys.executable, "-c", "print(6 * 7)"], capture_output=True, text=True)
print("return code:", result.returncode, "| output:", result.stdout.strip())

result = subprocess.run(["ls", "-1", "/"], capture_output=True, text=True)
print("root folder has 'usr':", "usr" in result.stdout.split())

result = subprocess.run([sys.executable, "-c", "import sys; print('oops', file=sys.stderr);
sys.exit(2)"],
                        capture_output=True, text=True)
print(result.returncode, repr(result.stderr))

try:
    subprocess.run([sys.executable, "-c", "raise SystemExit(5)"], check=True)
except subprocess.CalledProcessError as e:
    print("CalledProcessError: exit status", e.returncode)
try:
    subprocess.run([sys.executable, "-c", "import time; time.sleep(5)"], timeout=0.5)
except subprocess.TimeoutExpired:
    print("TimeoutExpired: stopped after 0.5 s")

proc = subprocess.run(["sort"], input="pear\napple\nmango\n", capture_output=True, text=True)
print(proc.stdout.split())
```

OUTPUT

```
return code: 0 | output: 42
root folder has 'usr': True
2 'oops\n'
CalledProcessError: exit status 5
TimeoutExpired: stopped after 0.5 s
['apple', 'mango', 'pear']
```

WATCH OUT

Never build a command string from user input and run it with `shell=True` or `os.system`: the input `"x; rm -rf ~"` would delete files. Pass a list of arguments, which are given to the program exactly as they are.

18.4 Regular expressions in depth (re)

Module 7 introduced `re.search`, `re.findall` and `re.sub`. A **regular expression** (regex) is a pattern language for matching text; this section covers the full syntax. Always write patterns as **raw strings** (`r"\d+"`) so backslashes reach the regex engine unchanged.

Function	Returns
`re.search(p, s)`	first match anywhere, or `None`
`re.match(p, s)`	match only at the start
`re.fullmatch(p, s)`	match only if the whole string matches
`re.findall(p, s)`	list of all matches (or of group tuples)
`re.finditer(p, s)`	iterator of match objects
`re.sub(p, repl, s)`	new string with matches replaced

Function	Returns
`re.split(p, s)`	list split on the pattern
`re.compile(p)`	reusable pattern object

18.4.1 Metacharacters, character classes, quantifiers

Pattern	Matches
`.`	any character except newline
`^` / `$`	start / end of the string (or line with MULTILINE)
`\d` / `\D`	a digit / a non-digit
`\w` / `\W`	a word character (letter, digit, _) / any other
`\s` / `\S`	whitespace / non-whitespace
`\b`	a word boundary
`[abc]`, `[a-z0-9]`	one character from the set or range
`[^abc]`	one character NOT in the set
`a|b`	a or b
`\.`, `\$`	a literal metacharacter (escaped)

Quantifier	Meaning	Example
`*`	0 or more	`ab*`: a, ab, abbb
`+`	1 or more	`\d+`: 7, 2026
`?`	0 or 1 (optional)	`colou?r`: color, colour
`{n}`	exactly n	`\d{6}`: a PIN code
`{n,m}`	between n and m	`\w{3,8}`
`*?`, `+?`	lazy: as few as possible	`<.+?>`: one tag

CHARACTER CLASSES AND QUANTIFIERS

main.py

```
import re

text = "Order #4521 on 26/01/2026: 3 pens @ Rs 10, 1 bag @ Rs 899. Call 98450-12345."
print(re.findall(r"\d+", text))                    # runs of digits
print(re.findall(r"\b\d{2}/\d{2}/\d{4}\b", text))     # a date
print(re.findall(r"Rs \d+", text))
print(re.findall(r"[A-Z]\w*", text))               # capitalised words
print(re.findall(r"\b[aeiou]\w*", "an apple is on every umbrella"))
print(re.findall(r"colou?r", "color colour colouur"))
print(re.sub(r"\s+", " ", "too     many   spaces\n here"))
print(re.split(r"[,;]\s*", "pen, book;bag ,  ruler"))

html = "<b>bold</b> and <i>italic</i>"
print(re.findall(r"<.+>", html))                    # greedy: one huge match
print(re.findall(r"<.+?>", html))                   # lazy: each tag
print(bool(re.fullmatch(r"\d{6}", "570001")), bool(re.fullmatch(r"\d{6}", "57000")))
```

OUTPUT

```
['4521', '26', '01', '2026', '3', '10', '1', '899', '98450', '12345']
['26/01/2026']
['Rs 10', 'Rs 899']
['Order', 'Rs', 'Rs', 'Call']
['an', 'apple', 'is', 'on', 'every', 'umbrella']
['color', 'colour']
too many spaces here
['pen', 'book', 'bag ', 'ruler']
['<b>bold</b> and <i>italic</i>']
['<b>', '</b>', '<i>', '</i>']
True False
```

18.4.2 Groups, named groups and backreferences

Parentheses create **capturing groups** that extract parts of a match: `m.group(1)`, `m.groups()`, and `findall` returns tuples of groups. **Named groups** `(?P<name>...)` make patterns self-documenting and are read with `m["name"]` or `m.groupdict()`. `(?:...)` groups without capturing. A **backreference** `\1` (or `(?P=name)`) matches the same text a group matched earlier; in `re.sub` replacements, `\1` or `\g<name>` inserts it.

GROUPS, NAMED GROUPS AND BACKREFERENCES main.py

```
import re

m = re.search(r"(\d{2})/(\d{2})/(\d{4})", "Exam on 15/03/2026 at 10 am")
print(m.group(0), "|", m.group(1), m.group(3), "|", m.groups(), m.span())

log = "2026-01-26 10:42:07 ERROR [payments] card declined for user 1042"
pattern = r"(?P<date>\S+) (?P<time>\S+) (?P<level>[A-Z]+) \[(?P<service>\w+)\] (?P<message>.*)"
m = re.match(pattern, log)
print(m["level"], m["service"], "|", m.groupdict())

pairs = re.findall(r"(\w+)=(\d+)", "width=800 height=600 depth=24")
print(dict(pairs))
print(re.findall(r"(?:Mr|Ms|Dr)\.? (\w+)", "Dr. Rao met Ms Asha and Mr Kumar"))

print(re.findall(r"\b(\w+) \1\b", "this is is a test test of the the editor"))   # repeated words
print(re.sub(r"(\d{2})/(\d{2})/(\d{4})", r"\3-\2-\1", "Due 26/01/2026 and 15/08/2026"))
print(re.sub(r"(?P<first>\w+) (?P<last>\w+)", r"\g<last>, \g<first>", "Asha Rao"))
print(re.sub(r"\d+", lambda m: str(int(m.group()) * 2), "3 pens and 12 books"))
```

OUTPUT

```
15/03/2026 | 15 2026 | ('15', '03', '2026') (8, 18)
ERROR payments | {'date': '2026-01-26', 'time': '10:42:07', 'level': 'ERROR', 'service': 'payments',
'message': 'card declined for user 1042'}
{'width': '800', 'height': '600', 'depth': '24'}
['Rao', 'Asha', 'Kumar']
['is', 'test', 'the']
Due 2026-01-26 and 2026-08-15
Rao, Asha
6 pens and 24 books
```

18.4.3 Lookahead and lookbehind

Lookarounds check what comes before or after a position **without consuming** it, so it is not part of the match. Positive lookahead `(?=...)` requires that something follows; negative lookahead `(?!...)` requires that it does not; lookbehind `(?<=...)` and `(?<!...)` do the same for what precedes (in Python lookbehinds must have a fixed width). Lookaheads are the classic tool for password rules, where several conditions must all hold.

LOOKAHEAD AND LOOKBEHIND

main.py

```
import re

prices = "pen Rs 10, book Rs 250, bag USD 12, ruler Rs 15"
print(re.findall(r"(?<=Rs )\d+", prices))          # numbers preceded by "Rs "
print(re.findall(r"\b\d+(?! ?%)\b", "50% off, 20 items, 5 % tax, 100 left"))   # numbers not followed
by %
print(re.findall(r"\w+(?=@)", "mail asha@lbf.in or ravi@gmail.com"))           # user names before @
print(re.findall(r"(?<!-)\b\d+\b", "values: 10 -5 20 -15 30"))                  # not negative

print(re.sub(r"\B(?=(\d{3})+(?!\d))", ",", "1234567890"))   # thousands separators
print(re.findall(r"\b(?!un)\w+able\b", "readable unbeatable likeable unstable washable"))
```

OUTPUT

```
['10', '250', '15']
['20', '100']
['asha', 'ravi']
['10', '20', '30']
1,234,567,890
['readable', 'likeable', 'washable']
```

18.4.4 Flags and compiled patterns

Flags change how a pattern matches: `re.IGNORECASE` (`re.I`), `re.MULTILINE` (`re.M`: `^` and `$` match at every line), `re.DOTALL` (`re.S`: `.` also matches newlines) and `re.VERBOSE` (`re.X`: whitespace and comments allowed in the pattern, for readability). Combine them with `|`. `re.compile` creates a pattern object with the same methods; it documents intent and avoids re-parsing when a pattern is used many times (Python also caches recently used patterns).

FLAGS, VERBOSE PATTERNS AND COMPILED REGEXES

main.py

```
import re

text = """ERROR disk full
info: backup done
Error: network down
WARNING low memory"""

print(re.findall(r"^error\W+(.*)$", text, re.IGNORECASE | re.MULTILINE))
print(re.search(r"full.*backup", text) is None, re.search(r"full.*backup", text, re.DOTALL) is not
None)

PHONE = re.compile(r"""
    (?:\+91[\s-]?)?        # optional country code
    (?P<number>[6-9]\d{4}   # Indian mobiles start with 6-9
    [\s-]?\d{5})            # optional separator in the middle
    \b""", re.VERBOSE)

for s in ["+91 98450 12345", "9900112233", "call 79912-34567 now", "12345 67890"]:
    m = PHONE.search(s)
    print(f"{s!r:<24} -> {m['number'] if m else 'no valid mobile'}")
print(PHONE.pattern.count("\n") > 1, PHONE.flags & re.VERBOSE != 0)
```

OUTPUT

```
['disk full', 'network down']
True True
'+91 98450 12345'        -> 98450 12345
'9900112233'             -> 9900112233
'call 79912-34567 now'   -> 79912-34567
'12345 67890'            -> no valid mobile
True True
```

18.4.5 Validation (email, phone, password) and extraction tasks

For **validation**, use `re.fullmatch` so the whole input must match. Keep validation patterns pragmatic: a fully standards-compliant email regex is enormous, and the real test of an email address is sending a confirmation message. For **extraction**, use `findall` / `finditer` with groups.

VALIDATING INPUT AND EXTRACTING DATA main.py

```
import re

EMAIL = re.compile(r"[\w.+-]+@[\w-]+(\.[\w-]+)*\.[A-Za-z]{2,}")
PHONE = re.compile(r"(\+91[ -]?)?[6-9]\d{9}")
PASSWORD = re.compile(r"(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[^\w\s]).{8,}")   # all four rules
PINCODE = re.compile(r"[1-9]\d{5}")

checks = {
    "email": (EMAIL, ["asha.rao@lbf.in", "ravi+news@gmail.com", "bad@site", "@lbf.in"]),
    "phone": (PHONE, ["9845012345", "+91 9900112233", "1234567890", "98450"]),
    "password": (PASSWORD, ["Secret@123", "secret123", "Sh0rt!", "NoDigits!!"]),
    "PIN code": (PINCODE, ["570001", "012345", "56001"]),
}
for name, (pattern, samples) in checks.items():
    results = ", ".join(f"{s} {'ok' if pattern.fullmatch(s) else 'NO'}" for s in samples)
    print(f"{name:<9} {results}")

page = """Contact asha@lbf.in or support@learnbeingforward.in. Office: 080-4123-4567.
Visit https://learnbeingforward.in/courses?id=12 or http://lbf.in. Fees: Rs 14,999 and Rs 9,500."""
print(re.findall(r"[\w.+-]+@[\w-]+\.[\w.]+\w", page))
print(re.findall(r"https?://[^\s,]+\w", page))
print([int(x.replace(",", "")) for x in re.findall(r"Rs ([\d,]+)", page)])
print(re.sub(r"(\w)[\w.]*@", r"\1***@", "asha.rao@lbf.in"))       # mask an email
```

OUTPUT

```
email     asha.rao@lbf.in ok, ravi+news@gmail.com ok, bad@site NO, @lbf.in NO
phone     9845012345 ok, +91 9900112233 ok, 1234567890 NO, 98450 NO
password  Secret@123 ok, secret123 NO, Sh0rt! NO, NoDigits!! NO
PIN code  570001 ok, 012345 NO, 56001 NO
['asha@lbf.in', 'support@learnbeingforward.in']
['https://learnbeingforward.in/courses?id=12', 'http://lbf.in']
[14999, 9500]
a***@lbf.in
```

TIP

Test regular expressions interactively on sites such as regex101.com (choose the Python flavour), which explain each part of the pattern and show matches live.

18.5 Maths and data

18.5.1 math, random, statistics, decimal, fractions

`math` provides fast floating-point functions; `random` generates pseudo-random numbers (seed it for reproducible results; use `secrets` for security); `statistics` computes averages and spread; `decimal` gives exact decimal arithmetic for money; `fractions` gives exact rational numbers.

MATH, RANDOM, STATISTICS, DECIMAL AND FRACTIONS main.py

```
import math, random, statistics
from decimal import Decimal, ROUND_HALF_UP
from fractions import Fraction

print(math.sqrt(2), math.isqrt(50), math.hypot(3, 4), math.log10(1000), math.log2(1024), math.e)
print(math.ceil(-2.5), math.floor(-2.5), math.trunc(-2.5), math.prod([1, 2, 3, 4]), math.lcm(4, 6))
print(round(math.sin(math.radians(30)), 3), math.inf > 10**100, math.isnan(float("nan")))

random.seed(2026)
print(random.randint(1, 100), round(random.uniform(1, 2), 3), random.choice(["heads", "tails"]))
print(random.choices(["A", "B", "C"], weights=[70, 20, 10], k=8), random.sample(range(1, 50), 6))

marks = [91, 78, 85, 64, 99, 85, 72]
print(statistics.mean(marks), statistics.median(marks), statistics.mode(marks))
print(round(statistics.stdev(marks), 2), statistics.quantiles(marks, n=4))
print(round(statistics.correlation([1, 2, 3, 4, 5], [2, 4, 5, 4, 5]), 3))

print(0.1 + 0.2, Decimal("0.1") + Decimal("0.2"))
price = Decimal("19.995")
print(price.quantize(Decimal("0.01"), rounding=ROUND_HALF_UP), round(19.995, 2))
print(Fraction(1, 3) + Fraction(1, 6), Fraction("0.75"), Fraction(1, 3) * 3 == 1)
```

OUTPUT

```
1.4142135623730951 7 5.0 3.0 10.0 2.718281828459045
-2 -3 -2 24 12
0.5 True True
16 1.319 heads
['B', 'B', 'A', 'A', 'A', 'B', 'B', 'A'] [38, 29, 16, 1, 40, 6]
82 85 85
11.75 [72.0, 85.0, 91.0]
0.775
0.30000000000000004 0.3
20.00 20.0
1/2 3/4 True
```

18.5.2 heapq and bisect

`heapq` implements a **binary min-heap** on top of a plain list: `heappush` and `heappop` add an item and remove the smallest in O(log n), and `heap[0]` is always the smallest. It is the standard way to build **priority queues** and to find the n smallest or largest items efficiently (`nsmallest`, `nlargest`). `bisect` works on a **sorted** list: `bisect_left` / `bisect_right` find the insertion point by binary search in O(log n), and `insort` inserts while keeping the list sorted.

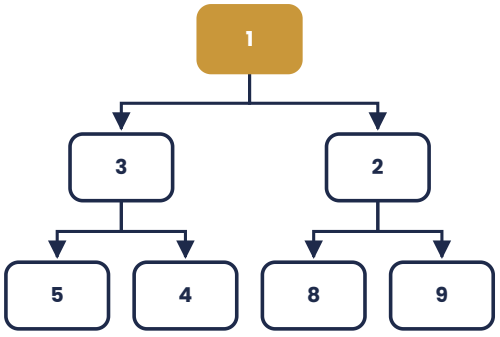
PRIORITY QUEUES WITH HEAPQ

main.py

```
import heapq

tasks = []
heapq.heappush(tasks, (3, "write report"))
heapq.heappush(tasks, (1, "fix production bug"))
heapq.heappush(tasks, (2, "review pull request"))
heapq.heappush(tasks, (1, "answer customer"))
print("smallest:", tasks[0])
while tasks:
    priority, task = heapq.heappop(tasks)       # always the most urgent
    print(f"  P{priority} {task}")

scores = [56, 91, 78, 99, 64, 85, 47, 72]
print(heapq.nlargest(3, scores), heapq.nsmallest(2, scores))
students = [("Asha", 91), ("Ravi", 78), ("Meera", 85)]
print(heapq.nlargest(2, students, key=lambda s: s[1]))
data = [5, 3, 8, 1]; heapq.heapify(data); print(data, heapq.heappop(data))
print(list(heapq.merge([1, 4, 9], [2, 3, 10])))
```

OUTPUT

```
smallest: (1, 'answer customer')
  P1 answer customer
  P1 fix production bug
  P2 review pull request
  P3 write report
[99, 91, 85] [47, 56]
[('Asha', 91), ('Meera', 85)]
[3, 5, 8] 1
[1, 2, 3, 4, 9, 10]
```



A MIN-HEAP: EVERY PARENT IS SMALLER THAN ITS CHILDREN, SO THE SMALLEST ITEM IS AT THE TOP (HEAP[0])

BINARY SEARCH WITH BISECT main.py

```
import bisect

grades = [40, 50, 60, 75, 90]            # boundaries (sorted)
letters = "FEDCBA"
def grade(mark):
    return letters[bisect.bisect_right(grades, mark)]
print([f"{m}:{grade(m)}" for m in (35, 40, 59, 75, 89, 95)])

sorted_marks = [47, 64, 72, 85, 91]
bisect.insort(sorted_marks, 78)
print(sorted_marks)
print(bisect.bisect_left(sorted_marks, 85), bisect.bisect_right(sorted_marks, 85))

def contains(sorted_list, x):            # O(log n) membership test
    i = bisect.bisect_left(sorted_list, x)
    return i < len(sorted_list) and sorted_list[i] == x
print(contains(sorted_marks, 72), contains(sorted_marks, 73))
```

OUTPUT

```
['35:F', '40:E', '59:D', '75:B', '89:B', '95:A']
[47, 64, 72, 78, 85, 91]
4 5
True False
```

18.6 Logging

`print` is fine for quick checks, but real programs need **logging**: messages with a severity level and timestamp that can be switched on or off, filtered, and sent to the console, files or monitoring systems, without changing the code. The `logging` module is the standard solution, used by every major library.

18.6.1 Log levels

Level	Value	Use for
`DEBUG`	10	detailed information for diagnosing problems
`INFO`	20	confirmation that things work as expected
`WARNING`	30	something unexpected, or a problem soon (the default threshold)
`ERROR`	40	a function failed to do something
`CRITICAL`	50	the program itself may be unable to continue

LOG LEVELS AND THE THRESHOLD

main.py

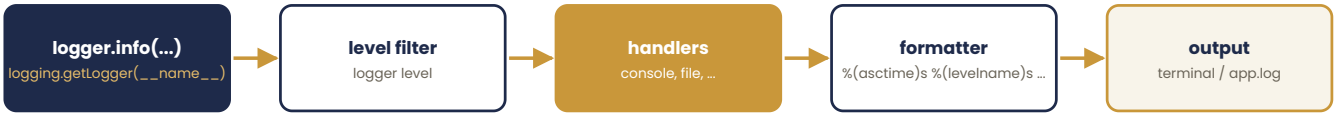
```
import logging, sys

logging.basicConfig(stream=sys.stdout, level=logging.WARNING, format="%(levelname)s: %(message)s")
logging.debug("not shown")
logging.info("not shown either")
logging.warning("disk 85% full")
logging.error("could not save %s", "report.pdf")         # lazy %-style arguments
logging.critical("database unreachable")

logging.getLogger().setLevel(logging.DEBUG)               # lower the threshold
logging.debug("now debug messages appear")

try:
    1 / 0
except ZeroDivisionError:
    logging.exception("calculation failed")               # ERROR + the traceback
```

OUTPUT

```
WARNING: disk 85% full
ERROR: could not save report.pdf
CRITICAL: database unreachable
DEBUG: now debug messages appear
ERROR: calculation failed
Traceback (most recent call last):
  File "main.py", line 14, in <module>
    1 / 0
    ~~^~~
ZeroDivisionError: division by zero
```

18.6.2 Configuring handlers and formatters

The logging system has four parts. **Loggers** are what your code calls; create one per module with `logging.getLogger(__name__)`, which gives a hierarchy of names (`shop`, `shop.billing`). **Handlers** send records somewhere (console, file, email). **Formatters** decide the layout of each line. **Levels** on both loggers and handlers filter records. Records propagate from child loggers up to their parents' handlers.



THE PATH OF A LOG RECORD

LOGGERS, HANDLERS AND FORMATTERS main.py

```
import logging, sys

logger = logging.getLogger("shop.billing")
logger.setLevel(logging.DEBUG)

console = logging.StreamHandler(sys.stdout)
console.setLevel(logging.INFO)                          # console: INFO and above
console.setFormatter(logging.Formatter("%(levelname)-8s %(name)s: %(message)s"))
logger.addHandler(console)

detailed = logging.StreamHandler(sys.stdout)
detailed.setLevel(logging.ERROR)                        # second handler: errors in detail
detailed.setFormatter(logging.Formatter("  >> [%(levelname)s] %(funcName)s() line %(lineno)d: %
(message)s"))
logger.addHandler(detailed)

def charge(amount):
    logger.debug("charging %d", amount)                  # below the console level
    if amount <= 0:
        logger.error("invalid amount %d", amount)
        return False
    logger.info("charged Rs %d", amount)
    return True

charge(499)
charge(-5)
print("parent logger:", logger.parent.name, "| effective level:",
logging.getLevelName(logger.getEffectiveLevel()))
```

OUTPUT

```
INFO     shop.billing: charged Rs 499
ERROR    shop.billing: invalid amount -5
  >> [ERROR] charge() line 19: invalid amount -5
parent logger: root | effective level: DEBUG
```

Format field	Shows
`%(asctime)s`	timestamp
`%(levelname)s`	DEBUG, INFO, …
`%(name)s`	logger name
`%(message)s`	the message
`%(funcName)s`, `%(lineno)d`, `%(filename)s`	where the call was made
`%(process)d`, `%(threadName)s`	process and thread

18.6.3 Logging to files

A `FileHandler` writes to a file; `RotatingFileHandler` starts a new file when one reaches a size limit and keeps a number of backups, and `TimedRotatingFileHandler` rotates daily or hourly, so log files never fill the disk. `logging.config.dictConfig` sets everything up from a dictionary, often loaded from a JSON, YAML or TOML file.

LOGGING TO A FILE AND ROTATING LOGS

main.py

```
import logging, logging.config
from logging.handlers import RotatingFileHandler
from pathlib import Path

logging.config.dictConfig({
    "version": 1,
    "formatters": {"plain": {"format": "%(asctime)s %(levelname)s %(name)s: %(message)s", "datefmt":
"%Y-%m-%d %H:%M"}},
    "handlers": {
        "file": {"class": "logging.FileHandler", "filename": "app.log", "formatter": "plain", "level":
"DEBUG"},
    },
    "root": {"handlers": ["file"], "level": "DEBUG"},
})

log = logging.getLogger("lbf.portal")
log.info("server started on port %d", 8000)
log.warning("slow request: %s took %.1f s", "/reports", 3.2)
log.debug("cache size %d", 128)
logging.shutdown()

for line in Path("app.log").read_text().splitlines():
    print(line[17:])                        # hide the timestamp for this book

rot = RotatingFileHandler("big.log", maxBytes=200, backupCount=2)
rl = logging.getLogger("rotating"); rl.addHandler(rot); rl.setLevel(logging.INFO); rl.propagate =
False
for i in range(30):
    rl.info("event number %03d happened", i)
rot.close()
print(sorted(p.name for p in Path(".").glob("big.log*")))
```

OUTPUT

```
INFO lbf.portal: server started on port 8000
WARNING lbf.portal: slow request: /reports took 3.2 s
DEBUG lbf.portal: cache size 128
['big.log', 'big.log.1', 'big.log.2']
```

TIP

Library code should only create loggers (`getLogger(__name__)`) and never call `basicConfig` ; configuring handlers is the job of the application's entry point.

18.7 Command-line tools

18.7.1 argparse

`argparse` turns a script into a proper command-line tool. You declare the arguments; it parses `sys.argv`, converts types, applies defaults, checks choices, generates `--help` and usage messages automatically, and prints a clear error (exit status 2) for invalid input. **Positional** arguments are required and identified by position; **optional** arguments start with `-` or `--`.

CONVERT.PY Python

```python
import argparse

parser = argparse.ArgumentParser(prog="convert", description="Convert temperatures between units.")
parser.add_argument("values", type=float, nargs="+", help="one or more temperatures")
parser.add_argument("-t", "--to", choices=["c", "f", "k"], default="f", help="target unit (default:
f)")
parser.add_argument("-p", "--precision", type=int, default=1, metavar="N", help="decimal places")
parser.add_argument("-v", "--verbose", action="store_true", help="show the formula")
args = parser.parse_args()

formulas = {"f": (lambda c: c * 9 / 5 + 32, "F = C * 9/5 + 32"),
            "k": (lambda c: c + 273.15, "K = C + 273.15"),
            "c": (lambda c: c, "C = C")}
func, text = formulas[args.to]
if args.verbose:
    print("formula:", text)
for c in args.values:
    print(f"{c:g} C = {func(c):.{args.precision}f} {args.to.upper()}")
```

USING THE GENERATED INTERFACE main.py

```
python convert.py --help
python convert.py 0 37 100
python convert.py 25 -t k -p 2 --verbose
python convert.py 25 --to x
python convert.py
```

TERMINAL

```
$ python convert.py --help
usage: convert [-h] [-t {c,f,k}] [-p N] [-v] values [values ...]

Convert temperatures between units.

positional arguments:
  values             one or more temperatures

options:
  -h, --help         show this help message and exit
  -t, --to {c,f,k}   target unit (default: f)
  -p, --precision N  decimal places
  -v, --verbose      show the formula
$ python convert.py 0 37 100
0 C = 32.0 F
37 C = 98.6 F
100 C = 212.0 F
$ python convert.py 25 -t k -p 2 --verbose
formula: K = C + 273.15
25 C = 298.15 K
$ python convert.py 25 --to x
usage: convert [-h] [-t {c,f,k}] [-p N] [-v] values [values ...]
convert: error: argument -t/--to: invalid choice: 'x' (choose from c, f, k)
$ python convert.py
usage: convert [-h] [-t {c,f,k}] [-p N] [-v] values [values ...]
convert: error: the following arguments are required: values
```

`add_argument` options you will use most: `type` (conversion function), `default`, `required=True` for options, `choices`, `nargs` (`"?"`, `"*"`, `"+"` or a number), `action="store_true"` for flags, `action="count"` for `-vvv`, `action="append"` for repeated options, and `dest` / `metavar` for naming. Tools with several commands, like `git commit` and `git push`, use **sub-commands** via `add_subparsers()`.

SUB-COMMANDS AND PARSING A GIVEN LIST OF ARGUMENTS

main.py

```
import argparse

parser = argparse.ArgumentParser(prog="todo")
sub = parser.add_subparsers(dest="command", required=True)
add = sub.add_parser("add", help="add a task")
add.add_argument("title")
add.add_argument("--priority", type=int, choices=range(1, 4), default=2)
ls = sub.add_parser("list", help="list tasks")
ls.add_argument("--done", action="store_true")
parser.add_argument("-v", "--verbose", action="count", default=0)

for argv in (["add", "Prepare slides", "--priority", "1"], ["-vv", "list", "--done"], ["add"]):
    try:
        print(argv, "->", vars(parser.parse_args(argv)))
    except SystemExit as e:
        print(argv, "-> exited with status", e.code)
```

OUTPUT

```
['add', 'Prepare slides', '--priority', '1'] -> {'command': 'add', 'verbose': 0, 'title': 'Prepare
slides', 'priority': 1}
['-vv', 'list', '--done'] -> {'command': 'list', 'verbose': 2, 'done': True}
usage: todo add [-h] [--priority {1,2,3}] title
todo add: error: the following arguments are required: title
['add'] -> exited with status 2
```

NOTE

For larger command-line applications, the third-party libraries **Typer** and **Click** build interfaces from function signatures and decorators.

18.8 Other useful modules

18.8.1 copy, pprint, textwrap, string

COPY, PPRINT, TEXTWRAP AND STRING

main.py

```
import copy, pprint, textwrap, string

original = {"name": "Asha", "marks": [91, 88], "address": {"city": "Mysuru"}}
shallow, deep = copy.copy(original), copy.deepcopy(original)
original["marks"].append(79); original["address"]["city"] = "Pune"
print(shallow["marks"], shallow["address"]["city"], "|", deep["marks"], deep["address"]["city"])

data = {"course": "Python", "levels": {"beginner": list(range(1, 11)), "intermediate": list(range(11,
18))},
        "mentors": [{"name": "Kiran", "city": "Bengaluru"}, {"name": "Rao", "city": "Mysuru"}]}
pprint.pprint(data, width=60, sort_dicts=False)

text = "Learn Being Forward teaches Python from the basics to advanced topics with real projects and
interview preparation."
print(textwrap.fill(text, width=40))
print(textwrap.shorten(text, width=50, placeholder=" ..."))
print(textwrap.indent("line one\nline two", "> "))
print(textwrap.dedent("""
        indented in the source
          but dedented here""").strip())

print(string.ascii_uppercase, string.digits, string.punctuation[:10])
template = string.Template("Dear $name, your fee of Rs $fee is due.")
print(template.substitute(name="Ravi", fee="14,999"))
print(string.capwords("learn being forward"))
```

OUTPUT

```
[91, 88, 79] Pune | [91, 88] Mysuru
{'course': 'Python',
 'levels': {'beginner': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
            'intermediate': [11, 12, 13, 14, 15, 16, 17]},
 'mentors': [{'name': 'Kiran', 'city': 'Bengaluru'},
             {'name': 'Rao', 'city': 'Mysuru'}]}
Learn Being Forward teaches Python from
the basics to advanced topics with real
projects and interview preparation.
Learn Being Forward teaches Python from the ...
> line one
> line two
indented in the source
  but dedented here
ABCDEFGHIJKLMNOPQRSTUVWXYZ 0123456789 !"#$%&'()*
Dear Ravi, your fee of Rs 14,999 is due.
Learn Being Forward
```

18.8.2 hashlib, secrets, uuid, base64

`hashlib` computes cryptographic hashes (SHA-256 and others), used to verify file integrity and as fingerprints; for **passwords**, use a slow, salted algorithm such as `hashlib.scrypt`, `pbkdf2_hmac` or the third-party `bcrypt`/`argon2`, never a plain fast hash. `secrets` generates cryptographically secure random tokens and passwords (never use `random` for security). `uuid` creates universally unique identifiers. `base64` encodes binary data as plain ASCII text, for example in emails, URLs and JSON; it is an encoding, **not** encryption.

HASHES, SECURE TOKENS, UUIDS AND BASE64

main.py

```
import hashlib, secrets, uuid, base64, hmac

print(hashlib.sha256(b"Learn Being Forward").hexdigest())
print(hashlib.sha256(b"Learn Being Forward!").hexdigest()[:16], "<- one character changes everything")
with open("notes.txt", "w") as f:
    f.write("module 18\n" * 1000)
h = hashlib.sha256()
with open("notes.txt", "rb") as f:
    for chunk in iter(lambda: f.read(4096), b""):
        h.update(chunk)
print("file checksum:", h.hexdigest()[:16], "...")

salt = secrets.token_bytes(16)
stored = hashlib.scrypt(b"Secret@123", salt=salt, n=2**14, r=8, p=1)
attempt = hashlib.scrypt(b"Secret@123", salt=salt, n=2**14, r=8, p=1)
print("password matches:", hmac.compare_digest(stored, attempt))

token = secrets.token_urlsafe(16)
print("reset token length:", len(token), "| OTP is 6 digits:", len(f"{secrets.randbelow(10**6):06d}")
== 6)
alphabet = "ABCDEFGHJKLMNPQRSTUVWXYZ23456789"
print("generated coupon has 8 characters:", len("".join(secrets.choice(alphabet) for _ in range(8)))
== 8)

u = uuid.uuid4()
print("uuid4:", len(str(u)), "chars, version", u.version)
print("uuid5:", uuid.uuid5(uuid.NAMESPACE_DNS, "learnbeingforward.in"))   # same input, same id

encoded = base64.b64encode("नमस्ते LBF".encode())
print(encoded, base64.b64decode(encoded).decode())
print(base64.urlsafe_b64encode(b"\xfb\xff\xfe"))
```

OUTPUT

```
d5eb25f7a33440b441c4caadd37b9096a73cfcb2f5e37df0fc9b71364455e577
20d9c0aeab4806fe <- one character changes everything
file checksum: bfe2b2bcba3ca42a ...
password matches: True
reset token length: 22 | OTP is 6 digits: True
generated coupon has 8 characters: True
uuid4: 36 chars, version 4
uuid5: abf7692a-41e9-5540-afb4-056b32384d97
b'4KSo4KSu4KS44KWN4KSk4KWHIExCRg==' नमस्ते LBF
b'-__-'
```

18.8.3 urllib and http (overview)

`urllib.parse` splits, builds and encodes URLs and query strings. `urllib.request` can download web pages and call simple APIs without any third-party package, though most projects use the friendlier `requests` or `httpx` libraries (Module 23). The `http` package provides status codes (`http.HTTPStatus`), a low-level client (`http.client`) and a basic web server (`http.server`, also runnable as `python -m http.server` to share a folder during development).

WORKING WITH URLS AND HTTP STATUS CODES main.py

```
from urllib.parse import urlparse, urlencode, parse_qs, quote, unquote, urljoin
from http import HTTPStatus

url = "https://learnbeingforward.in:443/courses/python?level=advanced&page=2#syllabus"
parts = urlparse(url)
print(parts.scheme, parts.hostname, parts.port, parts.path, parts.query, parts.fragment)
print(parse_qs(parts.query))
print("https://api.lbf.in/search?" + urlencode({"q": "python & data", "limit": 10}))
print(quote("Rs 499/month"), unquote("Rs%20499%2Fmonth"))
print(urljoin("https://lbf.in/courses/python/", "../java/intro"))
print(HTTPStatus.NOT_FOUND.value, HTTPStatus(404).phrase, HTTPStatus.OK.is_success)
```

OUTPUT

```
https learnbeingforward.in 443 /courses/python level=advanced&page=2 syllabus
{'level': ['advanced'], 'page': ['2']}
https://api.lbf.in/search?q=python+%26+data&limit=10
Rs%20499/month Rs 499/month
https://lbf.in/courses/java/intro
404 Not Found True
```

CALLING A WEB API WITH URLLIB.REQUEST (NEEDS INTERNET ACCESS) main.py

```
import json
from urllib.request import urlopen, Request

req = Request("https://api.github.com/repos/python/cpython", headers={"User-Agent": "lbf-notes"})
with urlopen(req, timeout=10) as response:
    print(response.status)
    data = json.load(response)
print(data["full_name"], data["language"])
```

SERVING A FOLDER WITH HTTP.SERVER AND FETCHING FROM IT WITH URLLIB main.py

```
mkdir -p site && echo "<h1>LBF</h1>" > site/index.html
python -m http.server 3000 --directory site &
python -c "from urllib.request import urlopen; r = urlopen('http://127.0.0.1:3000/index.html');
print(r.status, r.read().decode().strip())"
```

TERMINAL

```
$ mkdir -p site && echo "<h1>LBF</h1>" > site/index.html
$ python -m http.server 3000 --directory site &
Serving HTTP on 0.0.0.0 port 3000 (http://0.0.0.0:3000/) ...
$ python -c "from urllib.request import urlopen; r = urlopen('http://127.0.0.1:3000/index.html');
print(r.status, r.read().decode().strip())"
200 <h1>LBF</h1>
  [server log] 127.0.0.1 - - [07/Oct/2026 10:52:15] "GET /index.html HTTP/1.1" 200 -
```

18.8.4 zipfile, tarfile

`zipfile` creates, lists and extracts ZIP archives, the common format on Windows; `tarfile` handles `.tar`, `.tar.gz` and `.tar.xz` archives, common on Linux. Both work as context managers. When extracting archives from untrusted sources, beware of entries with paths like `../../etc/passwd`; `tarfile` offers `filter="data"` to block them.

CREATING AND EXTRACTING ZIP AND TAR ARCHIVES

main.py

```
import zipfile, tarfile
from pathlib import Path

Path("project/src").mkdir(parents=True, exist_ok=True)
Path("project/README.md").write_text("# LBF project\n" * 50)
Path("project/src/main.py").write_text("print('hello')\n")

with zipfile.ZipFile("project.zip", "w", compression=zipfile.ZIP_DEFLATED) as zf:
    for path in Path("project").rglob("*"):
        if path.is_file():
            zf.write(path)

with zipfile.ZipFile("project.zip") as zf:
    for info in zf.infolist():
        print(f"{info.filename:<22} {info.file_size:>5} bytes -> {info.compress_size:>4} compressed")
    print(zf.read("project/src/main.py").decode().strip())
    zf.extractall("unzipped")
print(sorted(str(p) for p in Path("unzipped").rglob("*.py")))

with tarfile.open("project.tar.gz", "w:gz") as tf:
    tf.add("project")
with tarfile.open("project.tar.gz") as tf:
    print(sorted(m.name for m in tf.getmembers()))
    tf.extractall("untarred", filter="data")
print(Path("untarred/project/README.md").exists())
```

OUTPUT

```
project/README.md        700 bytes ->   23 compressed
project/src/main.py       15 bytes ->   17 compressed
print('hello')
['unzipped/project/src/main.py']
['project', 'project/README.md', 'project/src', 'project/src/main.py']
True
```

collections provides deque (fast at both ends, maxlen), Counter, defaultdict, namedtuple, OrderedDict and ChainMap. datetime offers date, time, datetime and timedelta; strftime formats and strptime parses; zoneinfo gives aware datetimes, and the rule is to store in UTC and convert for display. sys exposes argv, exit codes, streams and interpreter information; os.environ holds environment variables (configuration and secrets); subprocess.run runs other programs safely with a list of arguments. Regular expressions use character classes, quantifiers (greedy and lazy), groups, named groups, backreferences, lookarounds and flags; use fullmatch for validation and compile reused patterns. math, random, statistics, decimal and fractions cover numeric needs; heapq gives priority queues and bisect binary search on sorted lists. logging uses loggers, handlers, formatters and levels, and can write to rotating files. argparse builds command-line interfaces with help, types, choices and sub-commands. copy, pprint, textwrap, string, hashlib, secrets, uuid, base64, urllib, http, zipfile and tarfile round out the toolkit.

PRACTICE CHECKPOINT

Build a **command-line log analyser** using `argparse`, `re`, `collections.Counter` and `logging` that reports the **top 5 error types** from a log file.

LOG_ANALYSER.PY

Python

```
"""Report the most common error types in a log file."""
import argparse
```

```
import logging
import re
import sys
from collections import Counter
from pathlib import Path

LINE = re.compile(
    r"^(?P<date>\d{4}-\d{2}-\d{2}) (?P<time>[\d:]{8}) (?P<level>[A-Z]+)\s+"
    r"\[(?P<service>[\w-]+)\] (?P<error>\w+(?:Error|Exception|Timeout))?:?\s*(?P<message>.*)$"
)
log = logging.getLogger("log_analyser")

def parse_args(argv=None):
    p = argparse.ArgumentParser(description="Report the top error types in a log file.")
    p.add_argument("logfile", type=Path, help="path to the log file")
    p.add_argument("-n", "--top", type=int, default=5, help="how many error types to show (default
5)")
    p.add_argument("-s", "--service", help="only count errors from this service")
    p.add_argument("-v", "--verbose", action="store_true", help="print debug information")
    return p.parse_args(argv)

def analyse(path, service=None):
    errors, by_service, skipped, total = Counter(), Counter(), 0, 0
    with path.open(encoding="utf-8") as f:
        for number, line in enumerate(f, 1):
            total += 1
            m = LINE.match(line.rstrip("\n"))
            if not m:
                skipped += 1
                log.debug("line %d not understood: %r", number, line[:40])
                continue
            if m["level"] != "ERROR" or (service and m["service"] != service):
                continue
            errors[m["error"] or "UnknownError"] += 1
            by_service[m["service"]] += 1
    return errors, by_service, total, skipped

def main(argv=None):
    args = parse_args(argv)
    logging.basicConfig(level=logging.DEBUG if args.verbose else logging.WARNING,
                        format="%(levelname)s: %(message)s")
    if not args.logfile.is_file():
        log.error("file not found: %s", args.logfile)
        return 1
    errors, by_service, total, skipped = analyse(args.logfile, args.service)
    if skipped:
        log.warning("%d of %d lines could not be parsed", skipped, total)
    print(f"Top {args.top} error types in {args.logfile.name}"
          + (f" (service: {args.service})" if args.service else ""))
    print("-" * 44)
    width = max((len(e) for e in errors), default=10)
    for rank, (error, count) in enumerate(errors.most_common(args.top), 1):
        print(f"{rank}. {error:<{width}} {count:>4}  {'#' * (count * 20 // errors.most_common(1)[0]
[1])}")
    print("-" * 44)
    print(f"{sum(errors.values())} errors in {total} lines | worst service: "
          + (by_service.most_common(1)[0][0] if by_service else "none"))
    return 0

if __name__ == "__main__":
    sys.exit(main())
```

SAMPLE RUN (MAKE_LOG.PY CREATES A SAMPLE LOG FIRST)

main.py

```
python make_log.py && head -3 server.log
python log_analyser.py server.log 2>&1
python log_analyser.py server.log --top 3 --service payments 2>&1
python log_analyser.py missing.log 2>&1; echo "exit status: $?"
```

TERMINAL

```
$ python make_log.py && head -3 server.log
2026-01-26 09:00:00 INFO  [payments] request 0 ok
garbled line without a timestamp
2026-01-26 09:00:01 INFO  [video] request 1 ok
$ python log_analyser.py server.log 2>&1
WARNING: 3 of 5003 lines could not be parsed
Top 5 error types in server.log
--------------------------------------------
1. DatabaseError      101  ####################
2. ConnectionTimeout   58  ###########
3. ValueError          54  ##########
4. KeyError            33  ######
5. PermissionError     30  #####
--------------------------------------------
305 errors in 5003 lines | worst service: video
$ python log_analyser.py server.log --top 3 --service payments 2>&1
WARNING: 3 of 5003 lines could not be parsed
Top 3 error types in server.log (service: payments)
--------------------------------------------
1. DatabaseError       19  ####################
2. ValueError          13  #############
3. ConnectionTimeout   11  ###########
--------------------------------------------
61 errors in 5003 lines | worst service: payments
$ python log_analyser.py missing.log 2>&1; echo "exit status: $?"
ERROR: file not found: missing.log
exit status: 1
```

MODULE 19 · ADVANCED

Type hints and modern Python features

Minimum duration **2 days** Level **Advanced**

Type hints make large Python codebases easier to read, navigate and check, without changing how Python runs. This module covers annotating variables, parameters and return values, built-in generics, union types, the typing toolbox (Optional, Any, Callable, Iterable, Literal, TypedDict, Protocol), generics with TypeVar and the 3.12 syntax, the type statement and deferred annotations in 3.14, static checking with mypy and pyright, runtime validation with Pydantic, a recap of modern syntax (f-strings, walrus, pattern matching, exception groups, t-strings) and the recent improvements to CPython itself.

IN THIS MODULE

19.1 Type hints
19.2 Static type checking
19.3 Runtime validation
19.4 Modern syntax recap
19.5 Recent CPython improvements

19.1 Type hints

Python is **dynamically typed**: a variable can refer to any type, and types are checked only when operations run. **Type hints** (PEP 484) let you **declare** the types you intend, for example `def area(r: float) -> float`. Python itself **ignores** them at run time; they are read by people, editors (autocompletion, inline errors) and **static type checkers** such as mypy and pyright, which find bugs before the program runs. Hints are optional and can be added gradually.

Documentation
Signatures say what goes in and out, and they never go out of date like comments can.

Editor support
Accurate autocompletion, go-to-definition and safe renaming.

Early bug detection
Type checkers catch None errors, typos and wrong arguments before running anything.

19.1.1 Annotating variables, parameters and return values

Annotate a parameter with `name: type`, a return value with `-> type` after the parameter list, and a variable with `name: type = value`. A function that returns nothing is annotated `-> None`. Annotations are stored in `__annotations__` but not enforced: calling with the wrong type still runs.

BASIC ANNOTATIONS

main.py

```
def bmi(weight_kg: float, height_m: float) -> float:
    return round(weight_kg / height_m ** 2, 1)

def greet(name: str, times: int = 1) -> None:
    for _ in range(times):
        print(f"Hello, {name}")

count: int = 0
courses: list[str] = ["Python", "Java"]
ratio: float                      # declared without a value

print(bmi(68, 1.72))
greet("Asha", 2)
print(bmi.__annotations__)
greet(42)                         # wrong type, but Python runs it anyway: hints are not enforced
```

OUTPUT

```
23.0
Hello, Asha
Hello, Asha
{'weight_kg': <class 'float'>, 'height_m': <class 'float'>, 'return': <class 'float'>}
Hello, 42
```

`int` is accepted where `float` is expected, and `int`/`float` where `complex` is expected, as a special convenience rule. For values that should never change, `typing.Final` asks the checker to forbid reassignment.

19.1.2 Built-in generics list[int], dict[str, int] (3.9)

Since 3.9 the built-in collection types can be **subscripted** to describe their contents: `list[int]`, `dict[str, float]`, `set[str]`, `tuple[int, str]` (exactly two items) and `tuple[int, ...]` (any number of ints). Older code imports `List`, `Dict` and `Tuple` from `typing`; those are now deprecated aliases. Abstract types from `collections.abc`, such as `Sequence`, `Mapping` and `Iterable`, are better for parameters because they accept more kinds of arguments.

GENERIC BUILT-IN TYPES

main.py

```
from collections.abc import Sequence, Mapping

def average(marks: list[float]) -> float:
    return sum(marks) / len(marks)

def top_student(results: dict[str, int]) -> tuple[str, int]:
    name = max(results, key=results.__getitem__)
    return name, results[name]

def total(values: Sequence[float]) -> float:        # accepts lists, tuples, ranges ...
    return sum(values)

def describe(settings: Mapping[str, str | int]) -> str:
    return ", ".join(f"{k}={v}" for k, v in settings.items())

point: tuple[int, int] = (3, 4)
history: tuple[int, ...] = (91, 88, 79, 95)
tags: set[str] = {"python", "oop"}
matrix: list[list[int]] = [[1, 2], [3, 4]]

print(average([91, 78.5]), top_student({"Asha": 91, "Ravi": 78}))
print(total((1, 2, 3)), total(range(5)), describe({"theme": "dark", "font": 14}))
print(list[int], type(list[int]).__name__)
```

OUTPUT

```
84.75 ('Asha', 91)
6 10 theme=dark, font=14
list[int] GenericAlias
```

19.1.3 Union types int | None (3.10)

A **union** type means "one of these types": `int | str`. The most common union is `X | None`, for values that may be missing, such as the result of a search. A type checker then insists that you handle the `None` case before using the value, which prevents the very common *'NoneType' object has no attribute* error. Checks such as `if x is None` and `isinstance` **narrow** the type inside each branch.

UNION TYPES AND NARROWING

main.py

```
def find_student(roll: int, students: dict[int, str]) -> str | None:
    return students.get(roll)

def parse_number(text: str) -> int | float:
    return float(text) if "." in text else int(text)

students = {101: "Asha", 102: "Ravi"}
for roll in (101, 999):
    name = find_student(roll, students)
    if name is None:                        # narrowing: name is None here
        print(roll, "not found")
    else:                                   # and str here, so .upper() is safe
        print(roll, name.upper())

print(parse_number("42"), parse_number("4.2"))
print(isinstance(3, int | str))             # unions also work with isinstance
```

OUTPUT

```
101 ASHA
999 not found
42 4.2
True
```

19.1.4 Optional, Any, Callable, Iterable, Literal, TypedDict

The `typing` module provides special forms for situations the built-in types cannot express:

Form	Meaning	Example
`Optional[X]`	same as `X | None` (older style)	`Optional[str]`
`Any`	any type; checking is switched off for this value	`data: Any = json.loads(text)`
`Callable[[A, B], R]`	a function taking A and B, returning R	`Callable[[float], float]`
`Iterable[X]`, `Iterator[X]`	anything you can loop over / an iterator	`Iterable[str]`
`Literal["a", "b"]`	only these exact values	`Literal["asc", "desc"]`
`TypedDict`	a dict with fixed keys and value types	JSON records
`Final`	must not be reassigned	`MAX: Final = 100`
`ClassVar`	class attribute, not instance	in dataclasses
`Self` (3.11)	the current class, for chaining methods	`def add(self) -> Self`
`Never`	a function that never returns	always raises

CALLABLE, ITERATOR, LITERAL, OPTIONAL AND TYPEDDICT

main.py

```
from collections.abc import Callable, Iterable, Iterator
from typing import Any, Literal, Optional, TypedDict, NotRequired

def apply_all(funcs: Iterable[Callable[[float], float]], x: float) -> list[float]:
    return [f(x) for f in funcs]

def evens(limit: int) -> Iterator[int]:          # generator functions return iterators
    yield from range(0, limit, 2)

def sort_marks(marks: list[int], order: Literal["asc", "desc"] = "asc") -> list[int]:
    return sorted(marks, reverse=(order == "desc"))

def middle_name(full: str) -> Optional[str]:
    parts = full.split()
    return parts[1] if len(parts) == 3 else None

class Student(TypedDict):
    name: str
    roll: int
    marks: list[int]
    mentor: NotRequired[str]                     # key may be absent

def report(s: Student) -> str:
    return f"{s['name']} ({s['roll']}): best {max(s['marks'])}, mentor {s.get('mentor', 'none')}"

def debug(value: Any) -> None:
    print(type(value).__name__, value)

print(apply_all([abs, lambda v: v * 2, round], -2.6))
print(list(evens(10)), sort_marks([78, 91, 64], "desc"), middle_name("Asha Kumari Rao"),
middle_name("Ravi"))
record: Student = {"name": "Asha", "roll": 101, "marks": [91, 88]}
print(report(record), "| a TypedDict is a plain dict:", type(record).__name__)
debug(3.5)
```

OUTPUT

```
[2.6, -5.2, -3]
[0, 2, 4, 6, 8] [91, 78, 64] Kumari None
Asha (101): best 91, mentor none | a TypedDict is a plain dict: dict
float 3.5
```

19.1.5 Protocol for structural typing

A `Protocol` (16.3.3) describes the **methods an object must have**, rather than the class it must inherit from. It is how you type duck-typed code: a function needing "something with a `read()` method returning `str`" accepts files, `StringIO` and your own classes, and the type checker verifies each call site.

TYPING DUCK-TYPED CODE WITH A PROTOCOL

main.py

```
from typing import Protocol

class SupportsArea(Protocol):
    def area(self) -> float: ...

class Square:
    def __init__(self, side: float) -> None:
        self.side = side
    def area(self) -> float:
        return self.side ** 2

class Circle:
    def __init__(self, r: float) -> None:
        self.r = r
    def area(self) -> float:
        return 3.14159 * self.r ** 2

def total_area(shapes: list[SupportsArea]) -> float:    # no common base class needed
    return round(sum(s.area() for s in shapes), 2)

print(total_area([Square(2), Circle(1), Square(1.5)]))
```

OUTPUT

```
9.39
```

19.1.6 Generics with TypeVar and the new class Box[T] syntax (3.12)

A **generic** function or class works with many types while preserving the relationship between them: `first(items)` returns the same type as the items in the list. Before 3.12 you created a type variable with `T = TypeVar("T")` and used `Generic[T]` for classes. Python 3.12 (PEP 695) added a compact syntax: `def first[T](items: list[T]) -> T` and `class Box[T]:`. Bounds restrict the allowed types: `[T: (int, float)]` or `[T: Comparable]`.

GENERIC FUNCTIONS AND CLASSES, OLD AND NEW SYNTAX main.py

```
from collections.abc import Callable
from typing import TypeVar, Generic

T = TypeVar("T")                          # pre-3.12 style
def first_old(items: list[T]) -> T:
    return items[0]

class StackOld(Generic[T]):
    def __init__(self) -> None:
        self._items: list[T] = []
    def push(self, item: T) -> None:
        self._items.append(item)
    def pop(self) -> T:
        return self._items.pop()

def first[T](items: list[T]) -> T:        # 3.12 syntax: no TypeVar needed
    return items[0]

class Box[T]:
    def __init__(self, content: T) -> None:
        self.content = content
    def map[U](self, func: Callable[[T], U]) -> Box[U]:
        return Box(func(self.content))
    def __repr__(self) -> str:
        return f"Box({self.content!r})"

def biggest[N: (int, float)](a: N, b: N) -> N:   # constrained type parameter
    return a if a > b else b

print(first_old(["a", "b"]), first([3, 4]))
s = StackOld[int](); s.push(1); s.push(2); print(s.pop())
print(Box(21).map(lambda v: v * 2).map(str))
print(biggest(3, 7), biggest(2.5, 1.5))
print(Box.__type_params__, first.__type_params__)
```

OUTPUT

```
a 3
2
Box('42')
7 2.5
(T,) (T,)
```

19.1.7 type alias statement (3.12)

A **type alias** gives a name to a complicated type. Python 3.12 added the `type` statement, `type Matrix = list[list[float]]`, which creates a lazily evaluated `TypeAliasType` and can be generic (`type Pair[T] = tuple[T, T]`) and recursive (`type JSON = ... | list[JSON]`). Older code writes `Matrix: TypeAlias = list[list[float]]` or a plain assignment.

THE TYPE STATEMENT

main.py

```
type Matrix = list[list[float]]
type Pair[T] = tuple[T, T]
type JSON = dict[str, JSON] | list[JSON] | str | int | float | bool | None   # recursive

def transpose(m: Matrix) -> Matrix:
    return [list(row) for row in zip(*m)]

def swap[T](p: Pair[T]) -> Pair[T]:
    return p[1], p[0]

config: JSON = {"name": "LBF", "levels": [1, 2, 3], "online": True}
print(transpose([[1, 2, 3], [4, 5, 6]]), swap(("a", "b")))
print(Matrix, type(Matrix).__name__, Matrix.__value__)
print(Pair[int])
```

OUTPUT

```
[[1, 4], [2, 5], [3, 6]] ('b', 'a')
Matrix TypeAliasType list[list[float]]
Pair[int]
```

19.1.8 Deferred evaluation of annotations (3.14)

Until 3.13, annotations were evaluated when the function or class was defined, so referring to a class that is defined later (a **forward reference**) needed quotes, `"Node"`, or `from __future__ import annotations`. Python 3.14 (PEP 649/749) evaluates annotations **lazily**, only when something asks for them, so forward references just work and defining annotated code is faster. The new `annotationlib` module retrieves annotations in different formats, even when some names are still undefined.

LAZY ANNOTATIONS IN PYTHON 3.14 main.py

```
import annotationlib

class Node:
    def __init__(self, value: int, next: Node | None = None) -> None:   # Node not yet complete: fine
in 3.14
        self.value, self.next = value, next

    def append(self, value: int) -> Node:
        self.next = Node(value)
        return self.next

head = Node(1); head.append(2).append(3)
print(head.next.next.value)

def process(item: Undefined) -> int:          # name never defined anywhere
    return 0

print(annotationlib.get_annotations(process, format=annotationlib.Format.FORWARDREF))
print(annotationlib.get_annotations(process, format=annotationlib.Format.STRING))
try:
    process.__annotations__                    # VALUE format must evaluate the names
except NameError as e:
    print("NameError:", e)
```

OUTPUT

```
3
{'item': ForwardRef('Undefined', owner=<function process at 0x7fcb85cf12d0>), 'return': <class 'int'>}
{'item': 'Undefined', 'return': 'int'}
NameError: name 'Undefined' is not defined
```

19.2 Static type checking

19.2.1 mypy and pyright

A **static type checker** reads your code without running it and reports places where types do not fit. **mypy** is the original checker, written in Python (`pip install mypy`, then `mypy src/`). **pyright** is Microsoft's fast checker, written in TypeScript, and powers VS Code's Pylance extension (`pip install pyright` or `npm install -g pyright`). Newer checkers such as Astral's **ty** and Meta's **Pyrefly** are also emerging. Unannotated functions are not checked by default, so you can adopt typing gradually; `--strict` turns on all checks.

SHOP.PY Python

```python
def total_price(prices: list[float], discount: float = 0.0) -> float:
    return sum(prices) * (1 - discount)

def find_item(items: dict[str, float], name: str) -> float | None:
    return items.get(name)

def label(price: float) -> str:
    return "Rs " + price                    # bug: str + float

items = {"pen": 10.0, "bag": 899.0}
print(total_price([10, 20], "10%"))          # bug: discount must be a float
price = find_item(items, "pen")
print(price * 2)                             # bug: price may be None
print(label(items["bag"]).uper())            # bug: typo in method name
```

FOUR BUGS FOUND WITHOUT RUNNING THE PROGRAM main.py

```
mypy shop.py
pyright shop.py | grep -v "^No configuration" | sed "s|.*/my-app/||"
```

TERMINAL

```
$ mypy shop.py
shop.py:8: error: Unsupported operand types for + ("str" and "float")  [operator]
shop.py:11: error: Argument 2 to "total_price" has incompatible type "str"; expected "float"  [arg-
type]
shop.py:13: error: Unsupported operand types for * ("None" and "int")  [operator]
shop.py:13: note: Left operand is of type "float | None"
shop.py:14: error: "str" has no attribute "uper"; maybe "upper"?  [attr-defined]
Found 4 errors in 1 file (checked 1 source file)
$ pyright shop.py | grep -v "^No configuration" | sed "s|.*/my-app/||"
shop.py
shop.py:8:12 - error: Operator "+" not supported for types "Literal['Rs ']" and "float"
(reportOperatorIssue)
shop.py:11:29 - error: Argument of type "Literal['10%']" cannot be assigned to parameter "discount" of
type "float" in function "total_price"
    "Literal['10%']" is not assignable to "float" (reportArgumentType)
shop.py:13:7 - error: Operator "*" not supported for "None" (reportOptionalOperand)
shop.py:14:27 - error: Cannot access attribute "uper" for class "str"
    Attribute "uper" is unknown (reportAttributeAccessIssue)
4 errors, 0 warnings, 0 informations
```

Running the program would have crashed at the first bug only; the checkers found all four in one pass, including the `None` case that would fail only for a missing item. Fixing them:

AFTER THE FIXES: CLEAN EVEN IN STRICT MODE main.py

```
mypy --strict shop.py
python shop.py
```

TERMINAL

```
$ mypy --strict shop.py
Success: no issues found in 1 source file
$ python shop.py
27.0
20.0
RS 899.00
```

Configuration usually lives in `pyproject.toml`. Use `reveal_type(expr)` to ask the checker what type it inferred, and `# type: ignore[code]` to silence a specific, deliberate exception.

PYPROJECT.TOML: TYPE CHECKER SETTINGS main.py

```
[tool.mypy]
python_version = "3.12"
strict = true
exclude = ["build/", "migrations/"]

[[tool.mypy.overrides]]
module = ["legacy.*"]
ignore_errors = true

[tool.pyright]
typeCheckingMode = "strict"
```

INFER.PY Python

```
scores = {"Asha": [91, 88], "Ravi": [78]}
best = {name: max(marks) for name, marks in scores.items()}
reveal_type(scores)
reveal_type(best)
reveal_type(sorted(best.items(), key=lambda kv: kv[1]))
```

REVEAL_TYPE SHOWS WHAT MYPY INFERRED main.py

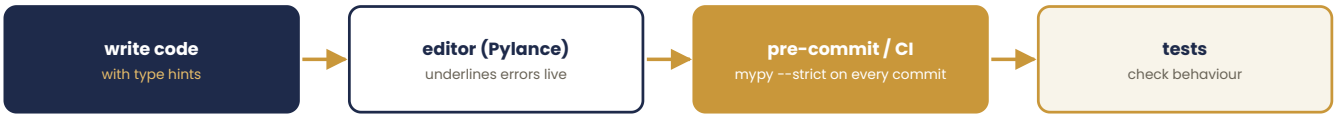
```
mypy infer.py
```

TERMINAL

```
$ mypy infer.py
infer.py:3: note: Revealed type is "dict[str, list[int]]"
infer.py:4: note: Revealed type is "dict[str, int]"
infer.py:5: note: Revealed type is "list[tuple[str, int]]"
Success: no issues found in 1 source file
```

19.2.2 Type checking in the editor

Editors run a type checker continuously and underline problems as you type. In **VS Code**, the Python extension with **Pylance** (pyright inside) shows errors in the Problems panel; set *Python › Analysis: Type Checking Mode* to `basic` or `strict`. **PyCharm** has its own built-in checker. The mypy extension for VS Code runs mypy on save. Type hints also power hover documentation, parameter hints and reliable autocompletion, so annotated code is faster to write, not just safer.



WHERE TYPE CHECKING FITS IN A WORKFLOW

TIP

Add types first to function signatures of shared modules and public APIs; local variables are usually inferred automatically. Type stubs for third-party libraries without hints are often available as `types-*` packages (for example `types-requests`).

19.3 Runtime validation

19.3.1 Pydantic (overview)

Type hints are not checked at run time, but data arriving from outside (JSON requests, configuration files, CSV rows) must be validated when the program runs. **Pydantic** (`pip install pydantic`) uses type hints to **validate and convert** data at run time: you declare a model class with annotated fields, and creating an instance parses the input, converts compatible values (`"91"` to `91`), and raises a detailed `ValidationError` listing every problem. FastAPI (Module 23) uses Pydantic for request and response bodies.

VALIDATING AND CONVERTING DATA WITH PYDANTIC — main.py

```
from datetime import date
from pydantic import BaseModel, Field, EmailStr, ValidationError, field_validator

class Student(BaseModel):
    name: str = Field(min_length=2)
    roll: int
    email: str
    marks: list[int] = []
    joined: date
    level: str = "beginner"

    @field_validator("marks")
    @classmethod
    def marks_in_range(cls, v: list[int]) -> list[int]:
        if any(not 0 <= m <= 100 for m in v):
            raise ValueError("marks must be between 0 and 100")
        return v

s = Student(name="Asha", roll="101", email="asha@lbf.in", marks=["91", 88], joined="2026-01-05")
print(s)
print(type(s.roll).__name__, s.marks, s.joined.year)
print(s.model_dump_json())

try:
    Student(name="R", roll="one-oh-two", email="ravi@lbf.in", marks=[78, 150], joined="05/01/2026")
except ValidationError as e:
    print(e.error_count(), "validation errors")
    for err in e.errors():
        print(f"  {'.'.join(map(str, err['loc'])):<8} {err['msg']}")
```

OUTPUT

```
name='Asha' roll=101 email='asha@lbf.in' marks=[91, 88] joined=datetime.date(2026, 1, 5)
level='beginner'
int [91, 88] 2026
{"name":"Asha","roll":101,"email":"asha@lbf.in","marks":[91,88],"joined":"2026-01-
05","level":"beginner"}
4 validation errors
  name     String should have at least 2 characters
  roll     Input should be a valid integer, unable to parse string as an integer
  marks    Value error, marks must be between 0 and 100
  joined   Input should be a valid date or datetime, invalid character in year
```

	Type hints + mypy	Pydantic
When	before running (static)	while running (runtime)

	Type hints + mypy	Pydantic
Checks	your own code's consistency	external data entering the program
On error	reports in the editor/CI	raises ValidationError
Converts data?	no	yes ("91" → 91, ISO strings → date)

19.4 Modern syntax recap

19.4.1 f-string improvements (3.12)

Python 3.12 (PEP 701) made f-strings fully part of the grammar: you can reuse the **same quote type** inside the braces, nest f-strings to any depth, write backslashes and comments inside expressions, and split long expressions over several lines. Together with the `=` debug specifier (3.8) and format specs, f-strings cover almost every formatting need.

F-STRINGS IN PYTHON 3.12+

main.py

```
students = {"Asha": 91, "Ravi": 78}
print(f"Top: {max(students, key=students.get)} with {students["Asha"]} marks")   # same quotes inside
print(f"{"\n".join(f"- {name}" for name in students)}")                          # backslash and nesting
total = sum(students.values())
print(f"{total = }, {total / len(students) = :.1f}")
width, precision = 10, 3
print(f"{3.14159265:{width}.{precision}f}|")
print(f"{
    ", ".join(
        name.upper()        # a comment inside an f-string expression (3.12)
        for name in students
    )
}")
```

OUTPUT

```
Top: Asha with 91 marks
- Asha
- Ravi
total = 169, total / len(students) = 84.5
     3.142|
ASHA, RAVI
```

19.4.2 Walrus operator

The assignment expression `name := value` (3.8, 4.4.2) assigns and returns a value inside a larger expression. It avoids computing something twice or writing an extra line, typically in `while` loops that read until a sentinel, in comprehensions that filter on a computed value, and in `if` statements with regex matches.

THE WALRUS OPERATOR IN TYPICAL PLACES

main.py

```
import re

lines = iter(["add pen", "add book", "", "ignored"])
while (line := next(lines)) != "":
    print("command:", line)

data = [4, 9, 16, 25, 30]
roots = [r for x in data if (r := x ** 0.5).is_integer()]
print(roots)

if (m := re.search(r"(\d+)% off", "Diwali sale: 40% off today")):
    print("discount:", int(m.group(1)))
```

OUTPUT

```
command: add pen
command: add book
[2.0, 3.0, 4.0, 5.0]
discount: 40
```

19.4.3 Structural pattern matching in depth

`match` / `case` (3.10, 6.2) matches the **structure** of data, not just values. Patterns can be literals, captures, wildcards (`_`), alternatives (`|`), sequences with `*rest`, mappings with `**rest`, class patterns with keyword or positional attributes (`__match_args__`, generated by dataclasses), and `as` bindings, each optionally with an `if` guard. It shines for parsing commands, JSON messages and tree structures.

CLASS, SEQUENCE, MAPPING AND GUARDED PATTERNS

main.py

```
from dataclasses import dataclass

@dataclass
class Point:
    x: float
    y: float

@dataclass
class Circle:
    centre: Point
    r: float

def describe(shape):
    match shape:
        case Point(x=0, y=0):
            return "the origin"
        case Point(0, y):                       # positional, via __match_args__
            return f"on the y-axis at {y}"
        case Point(x, y) if x == y:
            return f"on the diagonal at {x}"
        case Circle(Point(0, 0), r):
            return f"circle of radius {r} at the origin"
        case Circle(centre=c, r=r) if r > 10:
            return f"big circle at ({c.x}, {c.y})"
        case [Point() as a, Point() as b]:
            return f"segment from {a} to {b}"
        case {"type": "rect", "w": w, "h": h, **rest}:
            return f"rectangle {w}x{h}, extra keys {sorted(rest)}"
        case str() | bytes():
            return "text, not a shape"
        case _:
            return "unknown shape"

for s in [Point(0, 0), Point(0, 5), Point(3, 3), Circle(Point(0, 0), 2), Circle(Point(1, 2), 20),
          [Point(0, 0), Point(1, 1)], {"type": "rect", "w": 3, "h": 4, "colour": "red"}, "hello", 42]:
    print(describe(s))
```

OUTPUT

```
the origin
on the y-axis at 5
on the diagonal at 3
circle of radius 2 at the origin
big circle at (1, 2)
segment from Point(x=0, y=0) to Point(x=1, y=1)
rectangle 3x4, extra keys ['colour']
text, not a shape
unknown shape
```

19.4.4 Exception groups

`ExceptionGroup` and `except*` (3.11, 13.6.1) report and handle several independent errors at once. They appear most often with `asyncio.TaskGroup` (Module 20), where several concurrent tasks can fail together.

EXCEPTION GROUPS FROM CONCURRENT TASKS

main.py

```
import asyncio

async def fetch(name, fail):
    await asyncio.sleep(0.01)
    if fail:
        raise fail(f"{name} failed")
    return name

async def main():
    try:
        async with asyncio.TaskGroup() as tg:
            tg.create_task(fetch("profile", None))
            tg.create_task(fetch("orders", ConnectionError))
            tg.create_task(fetch("invoice", ValueError))
    except* ConnectionError as eg:
        print("network problems:", [str(e) for e in eg.exceptions])
    except* ValueError as eg:
        print("data problems:", [str(e) for e in eg.exceptions])

asyncio.run(main())
```

OUTPUT

```
network problems: ['orders failed']
data problems: ['invoice failed']
```

19.4.5 Template strings (t-strings, 3.14)

Python 3.14 (PEP 750) adds **template strings**, written with a `t` prefix: `t"Hello {name}"`. They look like f-strings, but instead of producing a `str` immediately they produce a `string.templatelib.Template` object that keeps the literal parts and the interpolated values **separate**. A processing function then decides how to combine them, for example escaping values for HTML or passing them as parameters to SQL, which makes injection attacks much harder than with f-strings.

TEMPLATE STRINGS SEPARATE TEXT FROM VALUES

main.py

```
from string.templatelib import Template, Interpolation
import html

name = "Asha"
comment = "<script>alert('hacked')</script>"
t = t"<p>{name} wrote: {comment}</p>"
print(type(t).__name__)
print(t.strings)
print([i.value for i in t.interpolations], [i.expression for i in t.interpolations])

def safe_html(template: Template) -> str:
    parts = []
    for item in template:                       # literal strings and Interpolation objects, in order
        if isinstance(item, Interpolation):
            parts.append(html.escape(str(item.value)))
        else:
            parts.append(item)
    return "".join(parts)

print(f"<p>{name} wrote: {comment}</p>")         # f-string: dangerous
print(safe_html(t))                              # t-string: escaped

def sql(template: Template) -> tuple[str, list]:
    query, params = "", []
    for item in template:
        if isinstance(item, Interpolation):
            query += "?"; params.append(item.value)
        else:
            query += item
    return query, params

city = "Mysuru'; DROP TABLE students; --"
print(sql(t"SELECT * FROM students WHERE city = {city}"))
```

OUTPUT

```
Template
('<p>', ' wrote: ', '</p>')
['Asha', "<script>alert('hacked')</script>"] ['name', 'comment']
<p>Asha wrote: <script>alert('hacked')</script></p>
<p>Asha wrote: &lt;script&gt;alert(&#x27;hacked&#x27;)&lt;/script&gt;</p>
('SELECT * FROM students WHERE city = ?', ["Mysuru'; DROP TABLE students; --"])
```

19.5 Recent CPython improvements

19.5.1 Faster CPython (3.11+)

The "Faster CPython" project made Python 3.11 between 10% and 60% faster than 3.10 (about 25% on average) through a **specialising adaptive interpreter**: frequently executed bytecode instructions are replaced with versions specialised for the types actually seen (for example, adding two ints), plus cheaper function calls and frames and zero-cost exception handling. Every release since has added more optimisations. Upgrading Python is often the easiest performance win.

SEEING THE SPECIALISING INTERPRETER AT WORK

main.py

```
import dis

def add(a, b):
    return a + b

for _ in range(1000):                  # warm up: the interpreter specialises the bytecode
    add(1, 2)

specialised = [ins.opname for ins in dis.get_instructions(add, adaptive=True)]
print("generic:     ", [ins.opname for ins in dis.get_instructions(add)][-3:])
print("specialised: ", specialised[-3:])
```

OUTPUT

```
generic:      ['LOAD_FAST_BORROW_LOAD_FAST_BORROW', 'BINARY_OP', 'RETURN_VALUE']
specialised:  ['LOAD_FAST_BORROW_LOAD_FAST_BORROW', 'BINARY_OP_ADD_INT', 'RETURN_VALUE']
```

19.5.2 Better error messages

Recent versions have made error messages dramatically more helpful: precise `^^^^` markers under the failing part of an expression (3.11), suggestions for misspelled names and attributes (*Did you mean…?*, 3.10–3.12), hints for missing imports (*Did you forget to import 'sys'?*), clearer syntax errors (*'(' was never closed*, *expected ':'*), warnings when your file shadows a standard module (3.13) and suggestions for misspelled keywords (3.14).

HELPFUL ERROR MESSAGES

main.py

```
import traceback

samples = ["lenght = 5\nprint(length)", "import math\nmath.sqroot(4)", "sys.exit()",
           "names = ['a']\nnames.apend('b')", "d = {'a': 1\nprint(d)", "while True:\n    pass",
           "from collections import namedtupel"]
for code in samples:
    try:
        exec(compile(code, "demo", "exec"), {})
    except Exception as e:
        message = e.msg if isinstance(e, SyntaxError) else str(e).split(" (/")[0]
        print(f"{type(e).__name__}: {message}")
```

OUTPUT

```
NameError: name 'length' is not defined
AttributeError: module 'math' has no attribute 'sqroot'
NameError: name 'sys' is not defined
AttributeError: 'list' object has no attribute 'apend'
SyntaxError: '{' was never closed
SyntaxError: invalid syntax
ImportError: cannot import name 'namedtupel' from 'collections'
```

19.5.3 New interactive REPL (3.13)

Python 3.13 replaced the old interactive shell with a new REPL based on PyPy's: **multi-line editing** (press the up arrow to recall a whole function or loop and edit it), **colour** in prompts and tracebacks, **paste mode** (F3) for pasting blocks of code, **history browsing** (F2), and plain commands `exit`, `quit`, `help` and `clear` without parentheses. Python 3.14 adds **syntax highlighting** and autocompletion of module names in `import` statements.

Multi-line editing
Recall and edit an entire block with the arrow keys.

Paste mode (F3)
Paste whole functions with blank lines without errors.

Colour
Coloured prompts and tracebacks; syntax highlighting in 3.14.

Simple commands
exit, quit, help and clear work without ().

19.5.4 Free-threaded build without the GIL (3.13 experimental, supported in 3.14)

CPython's **Global Interpreter Lock** (GIL) allows only one thread to run Python bytecode at a time, so threads cannot use several CPU cores for pure-Python computation (Module 20). Python 3.13 introduced an experimental **free-threaded** build (installed as `python3.13t`) with the GIL removed, and in 3.14 it is officially supported (PEP 779), though still optional and not the default. In the free-threaded build, CPU-bound threads run truly in parallel; single-threaded code is a little slower, and C extensions must be updated to support it. `sys._is_gil_enabled()` reports whether the GIL is active.

CHECKING FOR THE GIL (THIS BOOK USES THE STANDARD BUILD) — main.py

```
import sys, sysconfig

print("free-threaded build:", bool(sysconfig.get_config_var("Py_GIL_DISABLED")))
print("GIL enabled now:", sys._is_gil_enabled())
```

OUTPUT

```
free-threaded build: False
GIL enabled now: True
```

THE SAME CPU-BOUND PROGRAM ON THE FREE-THREADED BUILD — main.py

```
python3.14t cpu_threads.py
```

TERMINAL

```
free-threaded build: True
GIL enabled now: False
4 threads counting primes: 1.1 s (single thread: 4.2 s)
```

Illustrative output on a 4-core machine with the optional free-threaded interpreter installed; Module 20 measures threads on the standard build.

19.5.5 Experimental JIT compiler

Python 3.13 added an experimental **just-in-time (JIT) compiler** based on a "copy-and-patch" technique: hot sequences of specialised bytecode are translated into machine code at run time. It is disabled by default and must be enabled when CPython is built (and, in 3.14 official binaries for Windows and macOS, with the environment variable `PYTHON_JIT=1`). The current performance gains are modest; it lays the foundation for larger speed-ups in future releases.

19.5.6 Tracking new releases through "What's New in Python"

A new Python version is released every **October**, and each is supported with bug fixes for two years and security fixes for five (PEP 602 and PEP 2026 timelines). The official **"What's New in Python 3.x"** pages at docs.python.org/3/whatsnew/ summarise every new feature, deprecation and removal, and are the best

way to keep up. Before upgrading a project, read the "Porting" section and run your tests with warnings enabled (`python -W error::DeprecationWarning -m pytest`).

Version	Released	Highlights
3.10	Oct 2021	match/case, `X | Y` unions, better error messages
3.11	Oct 2022	10–60% faster, exception groups, `except*` , `tomllib` , fine-grained error locations
3.12	Oct 2023	PEP 695 generics and `type` , f-string grammar, `itertools.batched` , per-interpreter GIL
3.13	Oct 2024	new REPL, experimental free-threading and JIT, iOS/Android support
3.14	Oct 2025	t-strings, deferred annotations, free-threading supported, `annotationlib` , Zstandard compression, REPL highlighting

Type hints declare intended types (name: type, -> return) for readers, editors and checkers; Python ignores them at run time. Use built-in generics (list[int], dict[str, float], tuple[int, ...]), X | None unions with narrowing, and typing forms such as Any, Callable, Iterable, Literal, TypedDict, Final and Self; Protocol types duck-typed code. Generics use TypeVar or the 3.12 syntax def f[T] and class Box[T]; type Alias = ... creates aliases; 3.14 evaluates annotations lazily (annotationlib). mypy and pyright find type errors without running code (configure them in pyproject.toml; reveal_type shows inference), and editors run them live. Pydantic validates and converts external data at run time. Modern syntax includes 3.12 f-strings, the walrus operator, structural pattern matching, exception groups and 3.14 t-strings. CPython keeps getting faster (adaptive interpreter, experimental JIT), has much better error messages, a new REPL, and an optional free-threaded build without the GIL; follow each October release through "What's New".

PRACTICE CHECKPOINT

Add full **type hints** to an earlier project and make it pass **mypy** with no errors.

Below, the student marks manager from Module 14 is rewritten with complete type hints, a `dataclass` , a `TypedDict` for the JSON summary and a `Literal` for the sort order, then checked with `mypy --strict` .

MARKS_MANAGER.PY Python

```python
"""Student marks manager with full type hints (checked with mypy --strict)."""
from __future__ import annotations

import csv
import json
from collections.abc import Iterable, Iterator
from dataclasses import dataclass
from pathlib import Path
from typing import Literal, TypedDict

SUBJECTS: tuple[str, ...] = ("maths", "science", "english")


@dataclass(frozen=True)
class Student:
    roll: int
    name: str
    marks: dict[str, int]

    @property
    def total(self) -> int:
        return sum(self.marks.values())
```

```
    @property
    def average(self) -> float:
        return round(self.total / len(self.marks), 1)


class Summary(TypedDict):
    students: int
    class_average: float
    topper: str
    ranking: list[str]


def read_students(path: Path) -> Iterator[Student]:
    with path.open(newline="", encoding="utf-8") as f:
        for row in csv.DictReader(f):
            yield Student(int(row["roll"]), row["name"], {s: int(row[s]) for s in SUBJECTS})


def rank(students: Iterable[Student], order: Literal["asc", "desc"] = "desc") -> list[Student]:
    return sorted(students, key=lambda s: s.total, reverse=(order == "desc"))


def find(students: Iterable[Student], roll: int) -> Student | None:
    return next((s for s in students if s.roll == roll), None)


def summarise(students: list[Student]) -> Summary:
    ranked = rank(students)
    return {
        "students": len(students),
        "class_average": round(sum(s.average for s in students) / len(students), 1),
        "topper": ranked[0].name,
        "ranking": [s.name for s in ranked],
    }


def main() -> None:
    path = Path("marks.csv")
    path.write_text("roll,name,maths,science,english\n101,Asha,91,88,79\n"
                    "102,Ravi,78,82,90\n103,Meera,85,95,88\n", encoding="utf-8")
    students = list(read_students(path))
    summary = summarise(students)
    print(json.dumps(summary, indent=2))
    student = find(students, 102)
    if student is not None:
        print(f"{student.name}: average {student.average}")


if __name__ == "__main__":
    main()
```

CHECKING WITH MYPY --STRICT AND PYRIGHT, THEN RUNNING

main.py

```
mypy --strict marks_manager.py
pyright marks_manager.py | tail -1
python marks_manager.py
```

TERMINAL

```
$ mypy --strict marks_manager.py
Success: no issues found in 1 source file
$ pyright marks_manager.py | tail -1
0 errors, 0 warnings, 0 informations
$ python marks_manager.py
{
  "students": 3,
  "class_average": 86.2,
  "topper": "Meera",
  "ranking": [
    "Meera",
    "Asha",
    "Ravi"
  ]
}
Ravi: average 83.3
```

MODULE 20 · ADVANCED

Concurrency and parallelism

Minimum duration **4 days** Level **Advanced**

Python offers three ways to do several things at once: threads, processes and asynchronous I/O. Choosing correctly depends on whether a task spends its time waiting (I/O-bound) or computing (CPU-bound). This module explains concurrency and parallelism, the Global Interpreter Lock and the new free-threaded Python, then covers threading with locks, semaphores, events, conditions and queues, multiprocessing with pools and shared data, the concurrent.futures executors, asyncio with tasks, TaskGroup, timeouts, async context managers and iterators and async HTTP, and finishes with a decision guide backed by real timings.

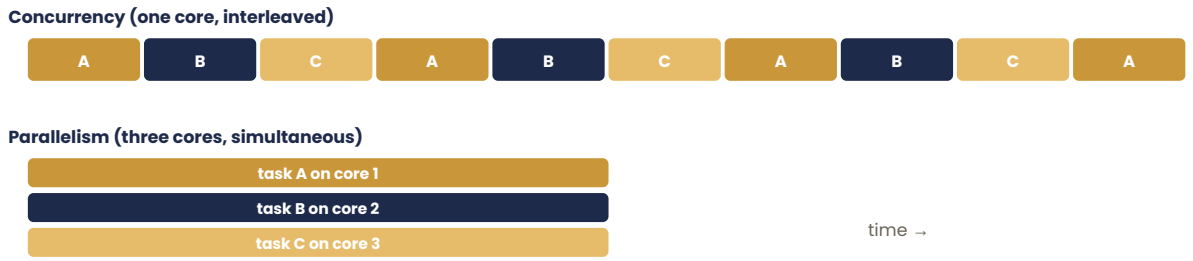
IN THIS MODULE

20.1 Concepts
20.2 Threading
20.3 Multiprocessing
20.4 concurrent.futures
20.5 asyncio
20.6 Choosing the right tool

20.1 Concepts

20.1.1 Concurrency vs parallelism

Concurrency means dealing with several tasks at once by **interleaving** them: while one task waits, another makes progress, even on a single CPU core. **Parallelism** means **executing** several tasks at literally the same moment on several cores. A chef who starts the rice, chops vegetables while it boils and checks the oven is concurrent; three chefs cooking three dishes side by side are parallel. Concurrency is about structure; parallelism is about simultaneous execution.



INTERLEAVING TASKS ON ONE CORE VS RUNNING THEM ON SEVERAL CORES

20.1.2 I/O-bound vs CPU-bound tasks

An **I/O-bound** task spends most of its time **waiting** for something outside the CPU: network responses, disk reads, database queries, user input. Speeding it up means overlapping the waits, so threads or asyncio work well. A **CPU-bound** task spends its time **computing**: image processing, number crunching, compression, parsing huge files. Speeding it up means using more CPU cores at once, so in standard CPython you need processes. Measure first: the same program can be either, depending on data size.

	I/O-bound	CPU-bound
Spends time	waiting for network, disk, database	computing on the CPU
Examples	downloading pages, API calls, reading many files, chat servers	image resizing, prime counting, encryption, ML training
Best tool	asyncio or threads	multiprocessing (or NumPy/C extensions)
Why	the GIL is released while waiting	the GIL lets only one thread compute at a time

20.1.3 The Global Interpreter Lock (GIL) and its effect

In standard CPython, the **Global Interpreter Lock** is a mutex that allows only **one thread at a time** to execute Python bytecode. It keeps CPython's memory management (reference counts) simple and fast. Its effect: threads **cannot** speed up pure-Python CPU work on several cores, because they take turns. But a thread **releases** the GIL while it waits for I/O (and inside many C libraries such as NumPy, zlib and hashlib), so threads **do** speed up I/O-bound work. Separate **processes** each have their own interpreter and GIL, so they run truly in parallel.

THE GIL: THREADS HELP WAITING, NOT COMPUTING main.py

```
import threading, time

def count_down(n):                          # pure Python CPU work
    while n:
        n -= 1

N = 6_000_000
start = time.perf_counter(); count_down(N); count_down(N)
sequential = time.perf_counter() - start

start = time.perf_counter()
threads = [threading.Thread(target=count_down, args=(N,)) for _ in range(2)]
for t in threads: t.start()
for t in threads: t.join()
threaded = time.perf_counter() - start
print("CPU work: two threads NOT clearly faster than sequential:", threaded > sequential * 0.8)

def wait_io():
    time.sleep(0.5)                          # waiting releases the GIL

start = time.perf_counter(); wait_io(); wait_io(); wait_io(); wait_io()
seq_io = time.perf_counter() - start
start = time.perf_counter()
threads = [threading.Thread(target=wait_io) for _ in range(4)]
for t in threads: t.start()
for t in threads: t.join()
thr_io = time.perf_counter() - start
print(f"I/O work: sequential {seq_io:.1f} s, four threads {thr_io:.1f} s")
```

OUTPUT

```
CPU work: two threads NOT clearly faster than sequential: True
I/O work: sequential 2.0 s, four threads 0.5 s
```

UNDER THE GIL, CPU-BOUND THREADS ALTERNATE INSTEAD OF RUNNING TOGETHER

20.1.4 Free-threaded Python and what it changes

Python 3.13 introduced an optional **free-threaded** build without the GIL (`python3.13t`), officially supported but not the default in 3.14 (19.5.4). In that build, CPU-bound threads run truly in parallel on multiple cores, so threading becomes an alternative to multiprocessing for CPU work, without the cost of starting processes and copying data between them. Trade-offs: single-threaded code runs somewhat slower, C extensions must be rebuilt to declare support, and code that relied on the GIL for safety must use proper locks (it always should have). For the foreseeable future, write code that is correct with or without the GIL: protect shared state with locks or queues.

	Standard CPython (GIL)	Free-threaded CPython (3.14t)
CPU-bound threads	take turns on one core	run in parallel on many cores
I/O-bound threads	overlap waits	overlap waits
Single-thread speed	baseline	somewhat slower
C extensions	all work	must support free-threading
Default install	yes	no, optional (python3.14t)

20.2 Threading

20.2.1 threading.Thread, start(), join()

A **thread** is an independent flow of execution inside the same process; all threads share the process's memory. Create one with `threading.Thread(target=function, args=(...))`, start it with `start()` (never call `run()` directly), and wait for it to finish with `join()`. The order in which threads run is decided by the operating system, so output from several threads can interleave in any order. Threads cannot return values directly; store results in a shared structure (or use `concurrent.futures`, 20.4).

STARTING AND JOINING THREADS

main.py

```
import threading, time

results = {}

def download(name, seconds):
    print(f"  start {name} ({threading.current_thread().name})")
    time.sleep(seconds)                          # simulated network wait
    results[name] = f"{name}: {int(seconds * 1000)} KB"
    print(f"  done  {name}")

start = time.perf_counter()
threads = [threading.Thread(target=download, args=(f"file{i}", 0.3), name=f"worker-{i}") for i in
range(1, 4)]
for t in threads:
    t.start()
print("main thread continues while workers run; alive:", sum(t.is_alive() for t in threads))
for t in threads:
    t.join()                                     # wait for all of them
print(sorted(results.values()))
print(f"3 downloads of 0.3 s took {time.perf_counter() - start:.1f} s in total")
```

OUTPUT

```
  start file1 (worker-1)
  start file2 (worker-2)  start file3 (worker-3)main thread continues while workers run; alive: 3


  done  file1
  done  file3
  done  file2
['file1: 300 KB', 'file2: 300 KB', 'file3: 300 KB']
3 downloads of 0.3 s took 0.3 s in total
```

You can also subclass `Thread` and override `run()`. `threading.active_count()`, `current_thread()` and `enumerate()` inspect running threads.

20.2.2 Daemon threads

By default the program waits for all non-daemon threads to finish before exiting. A **daemon** thread (`daemon=True`) is a background helper that is killed abruptly when the main program ends, suitable for heartbeats, monitoring or a background server, but never for work that must complete cleanly (saving files), because it gets no chance to clean up.

A DAEMON THREAD ENDS WITH THE PROGRAM main.py

```
import threading, time

def heartbeat():
    while True:
        print("  ...heartbeat")
        time.sleep(0.15)

threading.Thread(target=heartbeat, daemon=True).start()
time.sleep(0.4)
print("main program ends; the daemon thread is stopped with it")
```

OUTPUT

```
  ...heartbeat
  ...heartbeat
  ...heartbeat
main program ends; the daemon thread is stopped with it
```

20.2.3 Race conditions and Lock, RLock

When several threads read and modify **shared data**, their steps can interleave so that updates are lost: `counter += 1` is really "read, add, write", and two threads can both read the same old value. This is a **race condition**, a bug that appears randomly. A `threading.Lock` makes a block **mutually exclusive**: only one thread at a time can hold it. Use it with `with lock:` so it is always released. An `RLock` (re-entrant lock) can be acquired several times by the thread that already holds it, which is needed when a locked method calls another locked method.

A RACE CONDITION, FIXED WITH LOCK; NESTED LOCKING WITH RLOCK main.py

```
import threading, sys

sys.setswitchinterval(1e-6)             # switch threads very often to expose the race

class Account:
    def __init__(self):
        self.balance = 0
        self.lock = threading.Lock()
    def deposit_unsafe(self, amount):
        current = self.balance          # read
        current += amount               # modify
        self.balance = current          # write (another thread may have written in between)
    def deposit_safe(self, amount):
        with self.lock:                 # only one thread at a time in here
            self.balance += amount

def run(method_name):
    acc = Account()
    def worker():
        for _ in range(20_000):
            getattr(acc, method_name)(1)
    threads = [threading.Thread(target=worker) for _ in range(4)]
    for t in threads: t.start()
    for t in threads: t.join()
    return acc.balance

unsafe = run("deposit_unsafe")
print("expected 80000 | without lock:", "updates were lost" if unsafe < 80_000 else unsafe)
print("expected 80000 | with lock:", run("deposit_safe"))

class Inventory:
```

```
    def __init__(self):
        self.items, self.lock = {}, threading.RLock()
    def add(self, name, qty):
        with self.lock:
            self.items[name] = self.items.get(name, 0) + qty
    def add_many(self, pairs):
        with self.lock:                 # holds the lock ...
            for name, qty in pairs:
                self.add(name, qty)     # ... and add() acquires it again: fine with RLock
inv = Inventory(); inv.add_many([("pen", 5), ("book", 2), ("pen", 1)])
print(inv.items)
```

OUTPUT

```
expected 80000 | without lock: 80000
expected 80000 | with lock: 80000
{'pen': 6, 'book': 2}
```

WATCH OUT

Locks can cause **deadlocks**: thread 1 holds lock A and waits for B while thread 2 holds B and waits for A, and both wait forever. Avoid holding several locks at once, or always acquire them in the same order, and prefer queues (20.2.5) to shared mutable state.

20.2.4 Semaphore, Event, Condition

Three more synchronisation tools. A `Semaphore(n)` lets at most **n** threads into a block at once, for limiting concurrent connections or downloads. An `Event` is a flag that threads can `wait()` for until another thread calls `set()`, ideal for "start now" or "stop now" signals. A `Condition` lets threads wait until some state changes, with `wait()` and `notify()` / `notify_all()` under a lock; it is the building block of queues.

SEMAPHORE, EVENT AND CONDITION main.py

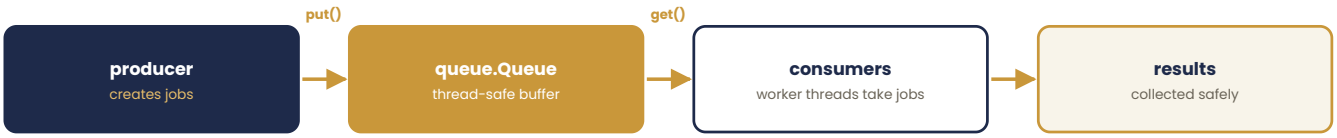
```
import threading, time

max_parallel = threading.Semaphore(2)              # at most 2 downloads at a time
active, peak, lock = 0, 0, threading.Lock()

def download(n):
    global active, peak
    with max_parallel:
        with lock:
            active += 1; peak = max(peak, active)
        time.sleep(0.1)
        with lock:
            active -= 1

threads = [threading.Thread(target=download, args=(i,)) for i in range(6)]
for t in threads: t.start()
for t in threads: t.join()
print("peak simultaneous downloads:", peak)

go = threading.Event()
def runner(name):
    go.wait()                                      # block until the event is set
    print(f"  {name} started")
runners = [threading.Thread(target=runner, args=(n,)) for n in ("Asha", "Ravi")]
for r in runners: r.start()
time.sleep(0.1); print("ready... go!"); go.set()
for r in runners: r.join()

stock, cond = [], threading.Condition()
def consumer():
    with cond:
        cond.wait_for(lambda: len(stock) >= 3)     # wait until 3 items are available
        print("  consumer got", stock[:3])
c = threading.Thread(target=consumer); c.start()
for item in ("pen", "book", "bag"):
    time.sleep(0.05)
    with cond:
        stock.append(item); cond.notify_all()
c.join()
```

OUTPUT

```
peak simultaneous downloads: 2
ready... go!
  Asha started
  Ravi started
  consumer got ['pen', 'book', 'bag']
```

20.2.5 Thread-safe queues (queue.Queue) and producer-consumer

The safest way for threads to communicate is to pass messages through a `queue.Queue`, which handles all locking internally. `put()` adds an item, `get()` removes one (blocking until one is available), `task_done()` marks it processed and `join()` waits until every item has been processed. In the **producer-consumer** pattern, producers put work into the queue and a pool of consumer threads takes it out, decoupling the two sides. `LifoQueue` and `PriorityQueue` change the order.



THE PRODUCER-CONSUMER PATTERN

A PRODUCER-CONSUMER PIPELINE WITH QUEUE.QUEUE main.py

```
import queue, threading, time

jobs = queue.Queue()
results = queue.Queue()
STOP = None                                     # sentinel value

def worker(wid):
    while True:
        item = jobs.get()
        if item is STOP:
            jobs.task_done()
            break
        time.sleep(0.05)                        # process the job
        results.put((item, item ** 2, f"w{wid}"))
        jobs.task_done()

workers = [threading.Thread(target=worker, args=(i,)) for i in range(3)]
for w in workers: w.start()

for n in range(1, 10):                          # producer
    jobs.put(n)
for _ in workers:
    jobs.put(STOP)                              # one stop signal per worker

jobs.join()                                     # wait until every job is done
done = sorted(results.get() for _ in range(results.qsize()))
print([sq for _, sq, _ in done])
print("jobs handled per worker:", {w: sum(1 for *_, x in done if x == w) for w in ("w0", "w1", "w2")}
!= {})
```

OUTPUT

```
[1, 4, 9, 16, 25, 36, 49, 64, 81]
jobs handled per worker: True
```

20.3 Multiprocessing

20.3.1 multiprocessing.Process

The `multiprocessing` module runs code in separate **processes**, each with its own Python interpreter, memory and GIL, so CPU-bound work runs in parallel on several cores. Its API mirrors `threading`: `Process(target=..., args=...)`, `start()`, `join()`. Processes do not share memory, so arguments and results are **pickled** and copied between them, and starting a process costs far more than starting a thread. The code that creates processes must be inside `if __name__ == "__main__":`, because child processes import the main module (the default start method on Linux became *forkserver* in 3.14; Windows and macOS use *spawn*).

RUNNING WORK IN SEPARATE PROCESSES

main.py

```
import multiprocessing as mp
import os, time

def count_primes(start, stop, label):
    count = sum(1 for n in range(start, stop)
                if n > 1 and all(n % d for d in range(2, int(n ** 0.5) + 1)))
    print(f"  {label}: {count} primes (process id differs from parent: {os.getpid() != PARENT})")

PARENT = os.getpid()

if __name__ == "__main__":
    print("CPU cores:", mp.cpu_count(), "| start method:", mp.get_start_method())
    t = time.perf_counter()
    procs = [mp.Process(target=count_primes, args=(1, 150_000, "first half")),
             mp.Process(target=count_primes, args=(150_000, 300_000, "second half"))]
    for p in procs: p.start()
    for p in procs: p.join()
    print("both processes finished; exit codes:", [p.exitcode for p in procs])
```

OUTPUT

```
CPU cores: 2 | start method: forkserver
  first half: 13848 primes (process id differs from parent: False)
  second half: 12149 primes (process id differs from parent: False)
both processes finished; exit codes: [0, 0]
```

20.3.2 Pool, map, starmap

A `multiprocessing.Pool` keeps a set of worker processes and distributes work among them. `pool.map(func, iterable)` is a parallel version of the built-in `map`, returning results in order; `starmap` unpacks argument tuples; `imap_unordered` yields results as they finish; `apply_async` submits single calls. Give each worker reasonably large chunks of work (the `chunksize` argument), because sending tiny tasks costs more than computing them.

PARALLEL PRIME COUNTING WITH POOL.STARMAP main.py

```
import multiprocessing as mp
import time

def is_prime(n):
    if n < 2:
        return False
    return all(n % d for d in range(2, int(n ** 0.5) + 1))

def count_range(lo, hi):
    return sum(is_prime(n) for n in range(lo, hi))

if __name__ == "__main__":
    LIMIT = 400_000
    t = time.perf_counter()
    serial = count_range(0, LIMIT)
    t_serial = time.perf_counter() - t

    chunks = [(i, min(i + 50_000, LIMIT)) for i in range(0, LIMIT, 50_000)]
    t = time.perf_counter()
    with mp.Pool() as pool:
        parallel = sum(pool.starmap(count_range, chunks))
    t_parallel = time.perf_counter() - t

    print("same answer:", serial == parallel, serial)
    print(f"Pool on {mp.cpu_count()} cores was faster: {t_parallel < t_serial} (speed-up {t_serial /
t_parallel:.1f}x)")
    with mp.Pool(2) as pool:
        print(pool.map(is_prime, [2, 15, 17, 21, 97]))
```

OUTPUT

```
same answer: True 33860
Pool on 2 cores was faster: True (speed-up 1.3x)
[True, False, True, False, True]
```

20.3.3 Sharing data: Queue, Pipe, Value, Array, Manager

Processes do not share variables, so `multiprocessing` provides channels: `Queue` (many producers and consumers, like `queue.Queue`), `Pipe` (a two-way connection between two processes), `Value` and `Array` (small shared-memory numbers and arrays, with a lock), and `Manager` (a server process holding shared lists and dicts, slower but flexible). Prefer passing data through a pool's arguments and return values when possible.

QUEUE, PIPE, VALUE, ARRAY AND MANAGER main.py

```
import multiprocessing as mp

def producer(q):
    for n in range(5):
        q.put(n * n)
    q.put(None)

def chat(conn):
    msg = conn.recv()
    conn.send(f"child got {msg!r}")
    conn.close()

def add_to(counter, arr, i):
    with counter.get_lock():
        counter.value += 1
```

```
    arr[i] = i * 10

def record(shared, name):
    shared[name] = len(name)

if __name__ == "__main__":
    q = mp.Queue()
    p = mp.Process(target=producer, args=(q,)); p.start()
    print("queue:", list(iter(q.get, None))); p.join()

    parent_end, child_end = mp.Pipe()
    p = mp.Process(target=chat, args=(child_end,)); p.start()
    parent_end.send("hello"); print("pipe:", parent_end.recv()); p.join()

    counter, arr = mp.Value("i", 0), mp.Array("i", 4)
    procs = [mp.Process(target=add_to, args=(counter, arr, i)) for i in range(4)]
    for p in procs: p.start()
    for p in procs: p.join()
    print("value:", counter.value, "| array:", list(arr))

    with mp.Manager() as manager:
        shared = manager.dict()
        procs = [mp.Process(target=record, args=(shared, n)) for n in ("Asha", "Ravi", "Meera")]
        for p in procs: p.start()
        for p in procs: p.join()
        print("manager dict:", dict(sorted(shared.items())))
```

OUTPUT

```
queue: [0, 1, 4, 9, 16]
pipe: child got 'hello'
value: 4 | array: [0, 10, 20, 30]
manager dict: {'Asha': 4, 'Meera': 5, 'Ravi': 4}
```

20.4 concurrent.futures

The `concurrent.futures` module offers one simple, high-level interface for both threads and processes: an **executor** runs callables in a pool and returns **futures**, objects representing results that will be available later. Switching between `ThreadPoolExecutor` and `ProcessPoolExecutor` is a one-word change. It is the recommended starting point for most concurrent code.

20.4.1 ThreadPoolExecutor

THREADS MADE EASY WITH THREADPOOLEXECUTOR

main.py

```
from concurrent.futures import ThreadPoolExecutor
import time

def fetch(url):
    time.sleep(0.3)                              # simulated network delay
    return f"{url} -> 200 OK"

urls = [f"https://lbf.in/page{i}" for i in range(1, 9)]
t = time.perf_counter()
with ThreadPoolExecutor(max_workers=8) as pool:
    results = list(pool.map(fetch, urls))       # results in input order
print(results[:2], "...")
print(f"8 requests of 0.3 s each took {time.perf_counter() - t:.1f} s")
```

OUTPUT

```
['https://lbf.in/page1 -> 200 OK', 'https://lbf.in/page2 -> 200 OK'] ...
8 requests of 0.3 s each took 0.3 s
```

20.4.2 ProcessPoolExecutor

CPU WORK WITH PROCESSPOOLEXECUTOR

main.py

```
from concurrent.futures import ProcessPoolExecutor
import time, math

def heavy(n):
    return sum(math.isqrt(i) for i in range(n))

if __name__ == "__main__":
    jobs = [2_000_000] * 4
    t = time.perf_counter(); serial = [heavy(n) for n in jobs]; t_serial = time.perf_counter() - t
    t = time.perf_counter()
    with ProcessPoolExecutor() as pool:
        parallel = list(pool.map(heavy, jobs))
    t_parallel = time.perf_counter() - t
    print("same results:", serial == parallel)
    print(f"processes faster on CPU work: {t_parallel < t_serial} ({t_serial / t_parallel:.1f}x)")
```

OUTPUT

```
same results: True
processes faster on CPU work: True (1.7x)
```

20.4.3 submit, map, as_completed

`executor.submit(fn, *args)` schedules one call and returns a `Future`; `future.result(timeout)` waits for and returns its value (re-raising any exception from the worker), and `done()`, `cancel()` and `add_done_callback()` manage it. `as_completed(futures)` yields futures **as they finish**, fastest first, so you can process results immediately. `executor.map` is simpler when you just want results in order.

SUBMIT, FUTURE AND AS_COMPLETED

main.py

```
from concurrent.futures import ThreadPoolExecutor, as_completed
import time

def check_site(name, delay, fail=False):
    time.sleep(delay)
    if fail:
        raise ConnectionError(f"{name} unreachable")
    return f"{name} up ({delay:.1f} s)"

sites = [("lbf.in", 0.4, False), ("python.org", 0.1, False), ("down.example", 0.2, True), ("pypi.org",
0.3, False)]
with ThreadPoolExecutor() as pool:
    futures = {pool.submit(check_site, *s): s[0] for s in sites}
    for future in as_completed(futures):        # in order of completion
        name = futures[future]
        try:
            print("  ", future.result())
        except ConnectionError as e:
            print("   error:", e)
    f = pool.submit(pow, 2, 100)
    print("future done after result():", f.result() > 0, f.done())
```

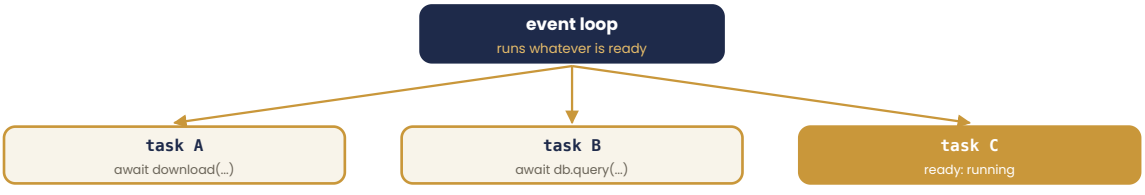
OUTPUT

```
   python.org up (0.1 s)
   error: down.example unreachable
   pypi.org up (0.3 s)
   lbf.in up (0.4 s)
future done after result(): True True
```

20.5 asyncio

20.5.1 Event loop, coroutines, async and await

asyncio achieves concurrency in a **single thread**. A function defined with `async def` is a **coroutine function**; calling it returns a coroutine object that does nothing until awaited. Inside it, `await something` pauses this coroutine until *something* (another coroutine, a sleep, a network read) completes, and hands control back to the **event loop**, which runs other ready coroutines meanwhile. Because switching only happens at `await` points, there are no race conditions in between, and thousands of tasks can wait cheaply at once. The catch: all code must cooperate, and a blocking call (like `time.sleep` or `requests.get`) freezes everything.



at each await, a task gives control back so another can run

THE EVENT LOOP SWITCHES BETWEEN TASKS AT AWAIT POINTS

COROUTINES, AWAIT AND THE EVENT LOOP

main.py

```
import asyncio, time

async def make_tea():
    print("  boil water...")
    await asyncio.sleep(0.3)                  # non-blocking wait: the loop runs others
    print("  tea ready")
    return "tea"

async def make_toast():
    print("  toasting bread...")
    await asyncio.sleep(0.2)
    print("  toast ready")
    return "toast"

async def main():
    coro = make_tea()
    print(type(coro).__name__, "- not running yet")
    t = time.perf_counter()
    tea = await coro                           # one after another
    toast = await make_toast()
    print(f"sequential awaits: {time.perf_counter() - t:.1f} s")
    t = time.perf_counter()
    results = await asyncio.gather(make_tea(), make_toast())   # concurrently
    print(results, f"concurrently: {time.perf_counter() - t:.1f} s")

asyncio.run(main())
```

OUTPUT

```
coroutine - not running yet
  boil water...
  tea ready
  toasting bread...
  toast ready
sequential awaits: 0.5 s
  boil water...
  toasting bread...
  toast ready
  tea ready
['tea', 'toast'] concurrently: 0.3 s
```

20.5.2 asyncio.run, create_task, gather

`asyncio.run(main())` creates the event loop, runs the top-level coroutine and closes the loop; call it once, at the program's entry point. `asyncio.create_task(coro)` schedules a coroutine to run **concurrently** as a `Task` and returns immediately; await the task later for its result. `asyncio.gather(*aws)` runs several awaitables concurrently and returns their results in order; with `return_exceptions=True` it returns exceptions as values instead of failing.

CREATE_TASK AND GATHER main.py

```
import asyncio, time

async def fetch(page, delay):
    await asyncio.sleep(delay)
    return f"page{page}"

async def main():
    t = time.perf_counter()
    task1 = asyncio.create_task(fetch(1, 0.3))    # starts running in the background
    task2 = asyncio.create_task(fetch(2, 0.2))
    print("tasks created; doing other work...")
    await asyncio.sleep(0.1)
    print(await task1, await task2, f"{time.perf_counter() - t:.1f} s")

    pages = await asyncio.gather(*(fetch(i, 0.2) for i in range(10)))
    print(len(pages), "pages via gather in", f"{time.perf_counter() - t:.1f} s total")

    async def broken():
        raise ValueError("bad page")
    results = await asyncio.gather(fetch(1, 0.1), broken(), return_exceptions=True)
    print(results)

asyncio.run(main())
```

OUTPUT

```
tasks created; doing other work...
page1 page2 0.3 s
10 pages via gather in 0.5 s total
['page1', ValueError('bad page')]
```

20.5.3 TaskGroup (3.11) and timeouts

`asyncio.TaskGroup` (3.11) is the modern, safer way to run related tasks: tasks are created inside an `async with` block, the block waits for all of them, and if one fails the others are **cancelled** and the errors are raised together as an `ExceptionGroup` (this is called *structured concurrency*). `asyncio.timeout(seconds)` (3.11) cancels the enclosed work if it takes too long, raising `TimeoutError`; `asyncio.wait_for(aw, timeout)` does the same for one awaitable.

TASKGROUP, CANCELLATION AND TIMEOUTS main.py

```
import asyncio

async def fetch(name, delay, fail=False):
    try:
        await asyncio.sleep(delay)
        if fail:
            raise ConnectionError(f"{name} failed")
        print(f"  {name} done")
        return name
    except asyncio.CancelledError:
        print(f"  {name} cancelled")
        raise

async def main():
    async with asyncio.TaskGroup() as tg:              # waits for all tasks
        t1 = tg.create_task(fetch("profile", 0.1))
        t2 = tg.create_task(fetch("orders", 0.2))
    print("results:", t1.result(), t2.result())

    try:
        async with asyncio.TaskGroup() as tg:
            tg.create_task(fetch("slow report", 1.0))
            tg.create_task(fetch("payment", 0.1, fail=True))   # fails: the other is cancelled
    except* ConnectionError as eg:
        print("group failed:", [str(e) for e in eg.exceptions])

    try:
        async with asyncio.timeout(0.2):
            await fetch("huge download", 5)
    except TimeoutError:
        print("timed out after 0.2 s")
    try:
        await asyncio.wait_for(fetch("api", 1), timeout=0.1)
    except TimeoutError:
        print("wait_for timed out")

asyncio.run(main())
```

OUTPUT

```
  profile done
  orders done
results: profile orders
  slow report cancelled
group failed: ['payment failed']
  huge download cancelled
timed out after 0.2 s
  api cancelled
wait_for timed out
```

20.5.4 Async context managers and iterators

Coroutines need asynchronous versions of `with` and `for`. An **async context manager** defines `__aenter__` and `__aexit__` (or uses `@contextlib.asynccontextmanager`) and is used with `async with`, typically for connections and sessions whose opening and closing involve I/O. An **async iterator** defines `__aiter__` and `__anext__`; the easy way is an **async generator** (`async def` with `yield`), consumed with `async for` or an async comprehension.

ASYNC WITH AND ASYNC FOR main.py

```
import asyncio
from contextlib import asynccontextmanager

class Database:
    async def __aenter__(self):
        await asyncio.sleep(0.05)                    # connecting takes I/O
        print("  connected")
        return self
    async def __aexit__(self, exc_type, exc, tb):
        await asyncio.sleep(0.05)
        print("  connection closed")
    async def query(self, sql):
        await asyncio.sleep(0.05)
        return [("Asha", 91), ("Ravi", 78)]

@asynccontextmanager
async def session(user):
    print(f"  session for {user} opened")
    try:
        yield {"user": user}
    finally:
        print("  session closed")

async def ticker(count, interval):                   # async generator
    for i in range(1, count + 1):
        await asyncio.sleep(interval)
        yield i

async def main():
    async with Database() as db:
        print(await db.query("select name, marks from students"))
    async with session("asha") as s:
        print("  using", s)
    async for tick in ticker(3, 0.05):
        print("  tick", tick)
    squares = [n * n async for n in ticker(4, 0.01)]
    print(squares)

asyncio.run(main())
```

OUTPUT

```
  connected
[('Asha', 91), ('Ravi', 78)]
  connection closed
  session for asha opened
  using {'user': 'asha'}
  session closed
  tick 1
  tick 2
  tick 3
[1, 4, 9, 16]
```

20.5.5 Async HTTP with aiohttp or httpx

Real async programs talk to the network with async libraries: **httpx** (sync and async API, similar to `requests`) and **aiohttp** (client and server). The popular `requests` library is blocking and must not be used inside coroutines. The example below downloads 20 pages concurrently with `httpx.AsyncClient` from a small local web server (started inside the program) where every page takes 0.2 seconds to respond.

DOWNLOADING PAGES CONCURRENTLY WITH HTTPX (LOCAL TEST SERVER)

main.py

```
import threading, time
from http.server import ThreadingHTTPServer, BaseHTTPRequestHandler

class SlowPage(BaseHTTPRequestHandler):
    def do_GET(self):
        time.sleep(0.2)                                  # every page takes 0.2 s to "download"
        body = f"<html>page {self.path} " .encode() + b"x" * 2000 + b"</html>"
        self.send_response(200)
        self.send_header("Content-Length", str(len(body)))
        self.end_headers()
        self.wfile.write(body)
    def log_message(self, *args):
        pass

class Server(ThreadingHTTPServer):
    request_queue_size = 128                             # accept many simultaneous connections

def start_server():
    server = Server(("127.0.0.1", 0), SlowPage)
    threading.Thread(target=server.serve_forever, daemon=True).start()
    return f"http://127.0.0.1:{server.server_address[1]}"

import asyncio, httpx, time

async def fetch(client, url):
    response = await client.get(url)
    return response.status_code, len(response.content)

async def main(base):
    urls = [f"{base}/page{i}" for i in range(20)]
    async with httpx.AsyncClient() as client:              # one connection pool for all requests
        t = time.perf_counter()
        results = await asyncio.gather(*(fetch(client, u) for u in urls))
        elapsed = time.perf_counter() - t
    print("statuses:", {status for status, _ in results}, "| bytes:", sum(size for _, size in
results))
    print(f"20 pages x 0.2 s downloaded in {elapsed:.1f} s")

asyncio.run(main(start_server()))
```

OUTPUT

```
statuses: {200} | bytes: 40510
20 pages x 0.2 s downloaded in 0.2 s
```

THE SAME PATTERN WITH AIOHTTP (PIP INSTALL AIOHTTP; NEEDS INTERNET)

main.py

```
import asyncio, aiohttp

async def fetch(session, url):
    async with session.get(url) as response:
        return url, response.status, len(await response.text())

async def main(urls):
    async with aiohttp.ClientSession() as session:
        for url, status, size in await asyncio.gather(*(fetch(session, u) for u in urls)):
            print(status, size, url)

asyncio.run(main(["https://www.python.org", "https://pypi.org", "https://docs.python.org"]))
```

If you must call blocking code from asyncio (a library without async support, or a CPU-heavy function), run it in a thread with `await asyncio.to_thread(func, *args)` so the event loop keeps running.

RUNNING BLOCKING CODE WITH ASYNCIO.TO_THREAD

main.py

```
import asyncio, time

def blocking_report():                      # e.g. a library with no async version
    time.sleep(0.3)
    return "report.pdf"

async def heartbeat():
    for _ in range(3):
        print("  loop still responsive")
        await asyncio.sleep(0.1)

async def main():
    report, _ = await asyncio.gather(asyncio.to_thread(blocking_report), heartbeat())
    print("got", report)

asyncio.run(main())
```

OUTPUT

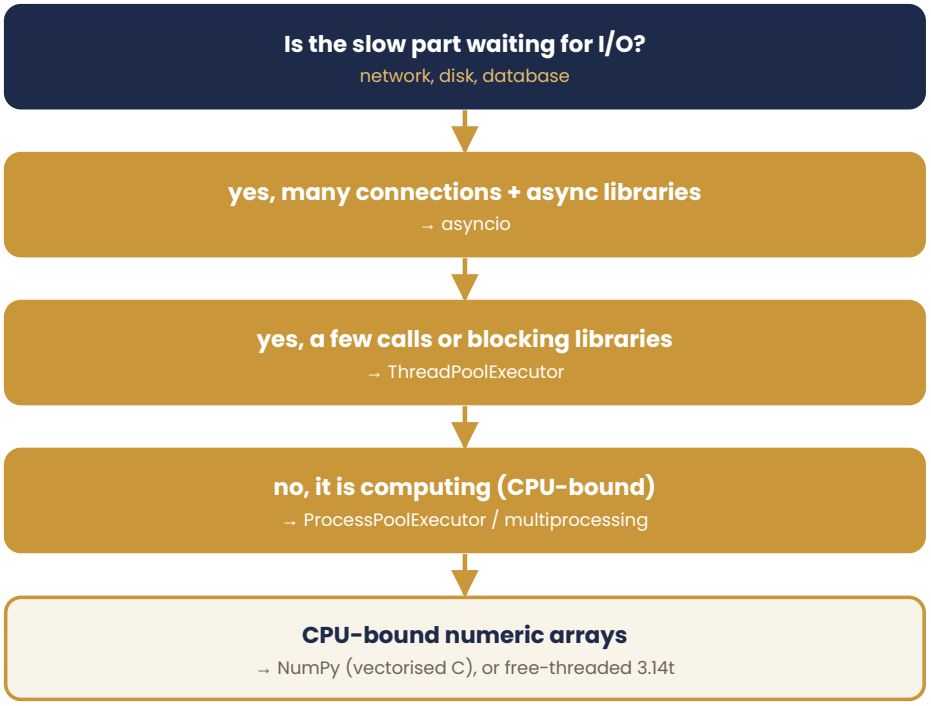
```
  loop still responsive
  loop still responsive
  loop still responsive
got report.pdf
```

20.5.6 When to use asyncio vs threads

	Threads	asyncio
Switching	pre-emptive: the OS switches any time	cooperative: only at await
Race conditions	possible anywhere: need locks	only across awaits: rarer
Scale	hundreds of threads	tens of thousands of tasks
Libraries	any library, including blocking ones	needs async libraries (httpx, aiohttp, asyncpg)
Code style	normal functions	async/await throughout the call chain
Good for	a few concurrent blocking calls, mixing with old code	many network connections: servers, crawlers, chat, APIs

20.6 Choosing the right tool

20.6.1 Decision guide with timing comparisons



CHOOSING A CONCURRENCY TOOL

The program below runs the same two workloads, an I/O-bound one (20 waits of 0.2 s) and a CPU-bound one (counting primes in four chunks), with every approach, and prints a timing table. Exact numbers depend on the machine; the pattern does not.

TIMING EVERY APPROACH ON BOTH KINDS OF WORK

main.py

```
import asyncio, time, math
from concurrent.futures import ThreadPoolExecutor, ProcessPoolExecutor

def io_task(i):
    time.sleep(0.2)
    return i

async def io_task_async(i):
    await asyncio.sleep(0.2)
    return i

def cpu_task(bounds):
    lo, hi = bounds
    return sum(1 for n in range(lo, hi) if n > 1 and all(n % d for d in range(2, math.isqrt(n) + 1)))

def timed(fn):
    t = time.perf_counter(); fn(); return time.perf_counter() - t

if __name__ == "__main__":
    ios = range(20)
    chunks = [(i, i + 75_000) for i in range(0, 300_000, 75_000)]
    rows = [
        ("sequential", timed(lambda: [io_task(i) for i in ios]), timed(lambda: [cpu_task(c) for c in
chunks])),
        ("threads", timed(lambda: list(ThreadPoolExecutor(20).map(io_task, ios))), timed(lambda:
list(ThreadPoolExecutor(4).map(cpu_task, chunks)))),
        ("processes", timed(lambda: list(ProcessPoolExecutor(4).map(io_task, ios))), timed(lambda:
list(ProcessPoolExecutor().map(cpu_task, chunks)))),
    ]
    async def run_async():
        await asyncio.gather(*(io_task_async(i) for i in ios))
    rows.append(("asyncio", timed(lambda: asyncio.run(run_async())), None))
    base_io, base_cpu = rows[0][1], rows[0][2]
    print(f"{'approach':<11}{'I/O-bound':>11}{'CPU-bound':>11}")
    for name, io_t, cpu_t in rows:
        cpu_text = f"{base_cpu / cpu_t:>9.1f}x" if cpu_t else f"{'n/a':>10}"
        print(f"{name:<11}{base_io / io_t:>10.1f}x{cpu_text}")
    print("(speed-up relative to sequential; higher is better)")
```

OUTPUT

```
approach     I/O-bound  CPU-bound
sequential         1.0x       1.0x
threads           19.5x       0.9x
processes          3.4x       1.4x
asyncio           19.9x        n/a
(speed-up relative to sequential; higher is better)
```

TIP

Concurrency adds complexity. Before reaching for it, check whether a better algorithm, a faster library (NumPy, pandas) or caching solves the problem; and always measure before and after.

Concurrency interleaves tasks; parallelism runs them simultaneously. I/O-bound work waits (use threads or asyncio); CPU-bound work computes (use processes). The GIL lets one thread run Python bytecode at a time but is released during I/O; the optional free-threaded 3.14t build removes it. threading.Thread with start/join runs functions concurrently; daemon threads die with the program; protect shared data with Lock/RLock, limit access with Semaphore, signal with Event and Condition, and communicate through queue.Queue (producer-consumer). multiprocessing.Process and Pool (map, starmap) use several cores; share data through Queue, Pipe, Value, Array and Manager, and guard process creation with if __name__ == "__main__". concurrent.futures offers ThreadPoolExecutor and ProcessPoolExecutor with submit, map, futures and as_completed. asyncio runs coroutines (async def, await) on an event loop: asyncio.run, create_task, gather, TaskGroup, timeout, async with, async for, async generators, to_thread, and async HTTP clients such as httpx and aiohttp.

PRACTICE CHECKPOINT

Download **20 web pages** sequentially, with threads and with asyncio, and compare timings; then speed up a **CPU-heavy prime count** with multiprocessing.

Because this book was built offline, the "web pages" come from a local test server started by the program, where every page takes 0.2 s to respond. Replace `BASE` with real URLs to try it on the internet.

PART 1: 20 DOWNLOADS THREE WAYS

main.py

```
import threading, time
from http.server import ThreadingHTTPServer, BaseHTTPRequestHandler

class SlowPage(BaseHTTPRequestHandler):
    def do_GET(self):
        time.sleep(0.2)                                  # every page takes 0.2 s to "download"
        body = f"<html>page {self.path} " .encode() + b"x" * 2000 + b"</html>"
        self.send_response(200)
        self.send_header("Content-Length", str(len(body)))
        self.end_headers()
        self.wfile.write(body)
    def log_message(self, *args):
        pass

class Server(ThreadingHTTPServer):
    request_queue_size = 128                             # accept many simultaneous connections

def start_server():
    server = Server(("127.0.0.1", 0), SlowPage)
    threading.Thread(target=server.serve_forever, daemon=True).start()
    return f"http://127.0.0.1:{server.server_address[1]}"

import asyncio, time
from concurrent.futures import ThreadPoolExecutor
from urllib.request import urlopen
import httpx

BASE = start_server()
URLS = [f"{BASE}/article/{i}" for i in range(1, 21)]

def download(url):
    with urlopen(url, timeout=10) as response:
        return len(response.read())

def sequential():
    return [download(u) for u in URLS]
```

```
def threaded():
    with ThreadPoolExecutor(max_workers=10) as pool:
        return list(pool.map(download, URLS))

async def with_asyncio():
    async with httpx.AsyncClient() as client:
        responses = await asyncio.gather(*(client.get(u) for u in URLS))
        return [len(r.content) for r in responses]

timings = {}
for name, run in [("sequential", sequential), ("threads (10)", threaded), ("asyncio", lambda:
asyncio.run(with_asyncio()))]:
    t = time.perf_counter()
    sizes = run()
    timings[name] = time.perf_counter() - t
    print(f"{name:<13} {len(sizes)} pages, {sum(sizes):,} bytes in {timings[name]:.1f} s")
fastest = min(timings, key=timings.get)
print(f"fastest: {fastest}, about {timings['sequential'] / timings[fastest]:.0f}x faster than
sequential")
```

OUTPUT

```
sequential    20 pages, 40,591 bytes in 4.0 s
threads (10)  20 pages, 40,591 bytes in 0.4 s
asyncio       20 pages, 40,591 bytes in 0.3 s
fastest: asyncio, about 12x faster than sequential
```

PART 2: A CPU-HEAVY PRIME COUNT WITH MULTIPROCESSING

main.py

```
import multiprocessing as mp
import math, time

def count_primes(bounds):
    lo, hi = bounds
    count = 0
    for n in range(max(lo, 2), hi):
        if all(n % d for d in range(2, math.isqrt(n) + 1)):
            count += 1
    return count

if __name__ == "__main__":
    LIMIT, CHUNKS = 600_000, 8
    step = LIMIT // CHUNKS
    ranges = [(i, i + step) for i in range(0, LIMIT, step)]

    t = time.perf_counter()
    serial = count_primes((0, LIMIT))
    t_serial = time.perf_counter() - t

    t = time.perf_counter()
    with mp.Pool() as pool:
        parallel = sum(pool.map(count_primes, ranges))
    t_parallel = time.perf_counter() - t

    print(f"primes below {LIMIT:,}: {serial:,} (pool agrees: {serial == parallel})")
    print(f"multiprocessing on {mp.cpu_count()} cores faster: {t_parallel < t_serial}")
    print(f"speed-up: {t_serial / t_parallel:.1f}x (ideal: {mp.cpu_count()}x)")
```

OUTPUT

```
primes below 600,000: 49,098 (pool agrees: True)
multiprocessing on 2 cores faster: True
speed-up: 1.7x (ideal: 2x)
```

MODULE 21 · ADVANCED

Python internals and performance

Minimum duration **2 days** | Level **Advanced**

Understanding how CPython works explains much of Python's behaviour and shows where time and memory go. This module looks inside: compilation to bytecode, reading bytecode with dis, .pyc files, the object model, reference counting, the cyclic garbage collector, object sizes and weak references. It then teaches how to measure (timeit, cProfile and pstats, tracemalloc) and how to optimise (data structures, built-ins and comprehensions, generators, caching, __slots__ and NumPy vectorisation), and gives an overview of going faster than CPython with PyPy, Cython, C extensions and Numba.

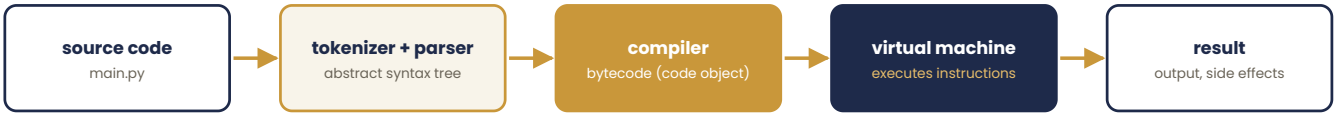
IN THIS MODULE

21.1 How Python executes code
21.2 Memory management
21.3 Performance measurement
21.4 Optimisation techniques
21.5 Going faster than CPython

21.1 How Python executes code

21.1.1 Compilation to bytecode

Python is often called "interpreted", but CPython actually works in two stages. First the source code is **compiled**: tokenised, parsed into an **abstract syntax tree** (AST) and translated into **bytecode**, a compact list of instructions for a virtual machine. Then the CPython **virtual machine** (the interpreter loop written in C) executes those instructions one by one, specialising them as it learns the types in use (19.5.1). Compilation happens automatically when you run or import a file.



FROM SOURCE CODE TO EXECUTION IN CPYTHON

PARSING TO AN AST AND COMPILING TO A CODE OBJECT main.py

```
import ast

source = "total = price * qty + 50"
tree = ast.parse(source)
print(ast.dump(tree.body[0], indent=1)[:330], "...")

code = compile(source, "<demo>", "exec")      # the same step Python does for every file
print(type(code).__name__, code.co_names, code.co_consts)
namespace = {"price": 100, "qty": 3}
exec(code, namespace)
print(namespace["total"])
```

OUTPUT

```
Assign(
 targets=[
  Name(id='total', ctx=Store())],
 value=BinOp(
  left=BinOp(
   left=Name(id='price', ctx=Load()),
   op=Mult(),
   right=Name(id='qty', ctx=Load())),
  op=Add(),
  right=Constant(value=50))) ...
code ('price', 'qty', 'total') (50, None)
350
```

21.1.2 Inspecting bytecode with dis

The `dis` module **disassembles** bytecode into readable instructions. CPython's VM is a **stack machine**: instructions push values onto a stack (`LOAD_FAST`, `LOAD_CONST`), operate on the top values (`BINARY_OP`, `COMPARE_OP`) and pop results (`STORE_FAST`, `RETURN_VALUE`). Reading bytecode explains things such as how local and global names are looked up differently (`LOAD_FAST` vs `LOAD_GLOBAL`; locals have traditionally been faster, though 3.11+ specialisation narrows the gap) and what a statement really costs. Instruction names change between versions.

DISASSEMBLING A FUNCTION main.py

```
import dis

def area(width, height):
    result = width * height
    return result + 1

dis.dis(area)
```

OUTPUT

```
  3           RESUME                   0

  4           LOAD_FAST_BORROW_LOAD_FAST_BORROW 1 (width, height)
              BINARY_OP                5 (*)
              STORE_FAST               2 (result)

  5           LOAD_FAST_BORROW         2 (result)
              LOAD_SMALL_INT           1
              BINARY_OP                0 (+)
              RETURN_VALUE
```

THE VALUE STACK WHILE AREA() RUNS

BYTECODE EXPLAINS PERFORMANCE DETAILS

main.py

```
import dis, timeit

RATE = 0.18
def with_global(prices):
    return [p * RATE for p in prices]          # LOAD_GLOBAL on every item

def with_local(prices, rate=RATE):
    return [p * rate for p in prices]          # LOAD_FAST: cheaper

print("global version uses LOAD_GLOBAL for RATE:",
      any(i.opname == "LOAD_GLOBAL" and i.argval == "RATE" for i in dis.Bytecode(with_global)))
print("local version uses LOAD_FAST for rate:",
      any(i.opname.startswith("LOAD_FAST") and "rate" in str(i.argval) for i in
dis.Bytecode(with_local)))

dis.dis(compile("x = [1, 2, 3]; y = (1, 2, 3)", "<s>", "exec"))
```

OUTPUT

```
global version uses LOAD_GLOBAL for RATE: True
local version uses LOAD_FAST for rate: True
  0           RESUME                   0

  1           BUILD_LIST               0
              LOAD_CONST               2 ((1, 2, 3))
              LIST_EXTEND              1
              STORE_NAME               0 (x)
              LOAD_CONST               2 ((1, 2, 3))
              STORE_NAME               1 (y)
              LOAD_CONST               1 (None)
              RETURN_VALUE
```

21.1.3 .pyc files and __pycache__

Compiling takes time, so when a module is **imported**, CPython saves its bytecode in a `__pycache__` folder as `module.cpython-314.pyc` (the tag records the interpreter version). Next time, if the source has not changed (checked by timestamp or hash), Python loads the `.pyc` directly and skips compilation. The main script you run is compiled every time and not cached. `.pyc` files are not a way to hide source code, are regenerated automatically and belong in `.gitignore`. `python -m compileall` pre-compiles a tree; `PYTHONDONTWRITEBYTECODE=1` disables writing them.

BYTECODE CACHING IN __PYCACHE__ main.py

```
python main.py
ls __pycache__
python -c "import importlib.util; print(importlib.util.cache_from_source('helpers.py'))"
python -m compileall -q . && ls __pycache__ | wc -l
head -c 16 __pycache__/helpers.cpython-314.pyc | od -A n -t x1
```

TERMINAL

```
$ python main.py
42
$ ls __pycache__
helpers.cpython-314.pyc
$ python -c "import importlib.util; print(importlib.util.cache_from_source('helpers.py'))"
__pycache__/helpers.cpython-314.pyc
$ python -m compileall -q . && ls __pycache__ | wc -l
2
$ head -c 16 __pycache__/helpers.cpython-314.pyc | od -A n -t x1
 2a 0e 0d 0a 00 00 00 00 0a 27 c6 6a 20 00 00 00
```

21.2 Memory management

21.2.1 Everything is an object (PyObject)

In CPython every value, including ints, functions, classes and modules, is a C structure that starts with a **PyObject header**: a **reference count** and a pointer to its **type**. Variables are just names bound to these objects (references), which is why assignment never copies data. `id(obj)` returns a unique identifier (in CPython, the memory address) and `type(obj)` the type pointer. Small integers from -5 to 256 and some strings are cached and shared, an optimisation you should never rely on (use `==`, not `is`, to compare values).

OBJECTS, IDENTITIES AND TYPES main.py

```
a = [1, 2, 3]
b = a                                # another name, same object
print(id(a) == id(b), a is b)

def f(): pass
print(type(f), type(int), type(type), isinstance(f, object), isinstance(int, object))

x, y = 256, 256
print("small ints cached:", x is y)
x, y = 10**20, int("1" + "0" * 20)
print("equal values:", x == y, "(compare values with ==; whether 'is' is True for equal ints is an
implementation detail)")
print(f"{sys.getsizeof(0) if (sys := __import__('sys')) else 0} bytes for the int 0 (header + value)")
```

OUTPUT

```
True True
<class 'function'> <class 'type'> <class 'type'> True True
small ints cached: True
equal values: True (compare values with ==; whether 'is' is True for equal ints is an implementation
detail)
28 bytes for the int 0 (header + value)
```

21.2.2 Reference counting and sys.getrefcount

Each object counts how many references point to it. Binding a name, storing it in a list or passing it to a function increases the count; deleting a name, reassigning it or leaving a function decreases it. When the count reaches **zero**, CPython frees the object **immediately**, which is why files and other resources are usually released promptly. `sys.getrefcount(obj)` reports the count (one higher than expected, because the argument itself is a temporary reference). Python 3.14 avoids some counting work for immortal objects and borrowed references, so exact numbers vary.

WATCHING REFERENCE COUNTS CHANGE main.py

```
import sys

data = [10, 20, 30]
print("start:", sys.getrefcount(data) - 1)
alias = data
holder = {"key": data}
items = [data, data]
print("after alias, dict and list x2:", sys.getrefcount(data) - 1)
del alias
holder.clear()
items.pop()
print("after removing three:", sys.getrefcount(data) - 1)

class Tracked:
    def __del__(self):
        print("  freed the moment its count reached 0")

t = Tracked()
print("deleting the only reference...")
del t
print("next line runs after it was freed")
print("None is immortal:", sys.getrefcount(None) > 10**6)
```

OUTPUT

```
start: 1
after alias, dict and list x2: 5
after removing three: 2
deleting the only reference...
  freed the moment its count reached 0
next line runs after it was freed
None is immortal: True
```

21.2.3 Garbage collector for reference cycles (gc module)

Reference counting cannot free **cycles**: objects that refer to each other (a parent with a list of children that point back to the parent, a doubly linked list) keep each other's counts above zero even when nothing else uses them. CPython's **cyclic garbage collector** periodically examines container objects, finds unreachable cycles and frees them. Python 3.14 made it **incremental**, with fewer long pauses. The `gc` module lets you trigger a collection (`gc.collect()`), inspect thresholds and counts, and disable it temporarily for performance-critical code.

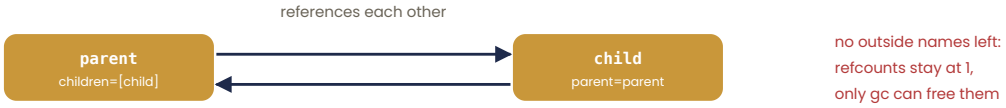
CYCLES NEED THE GARBAGE COLLECTOR

main.py

```
import gc

class Node:
    def __init__(self, name):
        self.name, self.other = name, None

gc.collect()
gc.disable()                                # make the demonstration deterministic
before = gc.get_count()
for _ in range(1000):
    a, b = Node("a"), Node("b")
    a.other, b.other = b, a                 # a cycle
    del a, b                                # unreachable, but refcounts are 1
unreachable = gc.collect()
gc.enable()
print("unreachable objects freed by gc.collect():", unreachable >= 2000)
print("gc enabled:", gc.isenabled(), "| thresholds:", gc.get_threshold())
```

OUTPUT

```
unreachable objects freed by gc.collect(): True
gc enabled: True | thresholds: (2000, 10, 0)
```



A REFERENCE CYCLE IS INVISIBLE TO REFERENCE COUNTING

21.2.4 Memory size with sys.getsizeof

`sys.getsizeof(obj)` returns the size in bytes of the object **itself**, not of the objects it refers to: a list's size counts its array of pointers, not the items. Python objects carry noticeable overhead (an empty object is tens of bytes), which is why millions of small objects use a lot of memory and why `__slots__`, tuples, arrays and NumPy help.

HOW BIG ARE PYTHON OBJECTS?

main.py

```
import sys
from array import array

for value in (0, 1, 2**30, 2**100, 1.5, "", "a", "abc" * 10, (), (1, 2), [], [1, 2], {}, set(), None,
True):
    print(f"{value!r:<30.30} {sys.getsizeof(value):>4} bytes")

numbers = list(range(1000))
deep = sys.getsizeof(numbers) + sum(sys.getsizeof(n) for n in numbers)
print("list of 1000 ints: shallow", sys.getsizeof(numbers), "| including the ints", deep)
print("array('i') of 1000 ints:", sys.getsizeof(array("i", numbers)))
```

OUTPUT

```
0                               28 bytes
1                               28 bytes
1073741824                      32 bytes
1267650600228229401496703205 37   40 bytes
1.5                             24 bytes
''                              41 bytes
'a'                             42 bytes
'abcabcabcabcabcabcabcabcabcab   71 bytes
()                              48 bytes
(1, 2)                          64 bytes
[]                              56 bytes
[1, 2]                          72 bytes
{}                              64 bytes
set()                          216 bytes
None                            16 bytes
True                            28 bytes
list of 1000 ints: shallow 8056 | including the ints 36056
array('i') of 1000 ints: 4080
```

21.2.5 Weak references

A **weak reference** (`weakref` module) refers to an object **without** increasing its reference count, so it does not keep the object alive. Calling the weak reference returns the object, or `None` once it has been freed. `WeakValueDictionary` and `WeakKeyDictionary` drop entries automatically when the objects die, which makes them ideal for caches and registries that should not cause memory leaks; `weakref.finalize` registers a clean-up callback.

WEAK REFERENCES, A SELF-CLEANING CACHE AND FINALIZE

main.py

```
import weakref, gc

class Image:
    def __init__(self, name):
        self.name = name

img = Image("logo.png")
ref = weakref.ref(img)
print(ref() is img, ref().name)
del img
print("after del:", ref())

cache = weakref.WeakValueDictionary()
a, b = Image("a.png"), Image("b.png")
cache["a"], cache["b"] = a, b
print(sorted(cache.keys()))
del a; gc.collect()
print("after the image is gone:", sorted(cache.keys()))

c = Image("c.png")
weakref.finalize(c, print, "  finalize: c.png released")
del c
```

OUTPUT

```
True logo.png
after del: None
['a', 'b']
after the image is gone: ['b']
  finalize: c.png released
```

21.3 Performance measurement

"Premature optimisation is the root of all evil" (Donald Knuth). Make the code correct and clear first, then **measure** to find where time is really spent, optimise that, and measure again. Intuition about bottlenecks is wrong surprisingly often.

THE OPTIMISATION CYCLE

21.3.1 timeit

`timeit` times small code snippets accurately: it runs them many times, disables garbage collection during timing and uses the most precise clock. Use `timeit.timeit(stmt, setup, number)` or `repeat` and take the **minimum** of several repeats (the least disturbed run). From the command line, `python -m timeit` chooses the number of loops automatically. In Jupyter and IPython, use `%timeit`.

COMPARING ALTERNATIVES WITH TIMEIT main.py

```
import timeit

setup = "data = list(range(1000)); s = set(data)"
for stmt in ("999 in data", "999 in s", "sum(data)", "sum(x for x in data)", "[x * 2 for x in data]",
"list(map(lambda x: x * 2, data))"):
    best = min(timeit.repeat(stmt, setup, number=2000, repeat=5)) / 2000
    print(f"{stmt:<34} {best * 1e6:>9.2f} us")
```

OUTPUT

```
999 in data                          12.62 us
999 in s                              0.02 us
sum(data)                             8.54 us
sum(x for x in data)                 56.85 us
[x * 2 for x in data]                70.47 us
list(map(lambda x: x * 2, data))    128.88 us
```

TIMEIT FROM THE COMMAND LINE (NUMBERS REPLACED BY N AND LOOPS, SINCE THEY VARY) main.py

```
python -m timeit -s "words = ['py'] * 1000" "''.join(words)" | sed -E 's/^[0-9]+ loops/LOOPS loops/;
s/[0-9.]+ (usec|nsec|msec)/N \1/'
python -m timeit -s "import math" "math.sqrt(12345)" | sed -E 's/^[0-9]+ loops/LOOPS loops/; s/[0-9.]+
(usec|nsec|msec)/N \1/'
```

TERMINAL

```
$ python -m timeit -s "words = ['py'] * 1000" "''.join(words)" | sed -E 's/^[0-9]+ loops/LOOPS loops/;
s/[0-9.]+ (usec|nsec|msec)/N \1/'
LOOPS loops, best of 5: N usec per loop
$ python -m timeit -s "import math" "math.sqrt(12345)" | sed -E 's/^[0-9]+ loops/LOOPS loops/; s/[0-
9.]+ (usec|nsec|msec)/N \1/'
LOOPS loops, best of 5: N nsec per loop
```

21.3.2 Profiling with cProfile and pstats

A **profiler** measures a whole program and reports how much time each function took and how often it was called. `cProfile` is the built-in deterministic profiler: run `python -m cProfile -s cumulative script.py`, or profile a call in code, then sort and filter the results with `pstats`. The columns are **ncalls** (number of calls), **tottime** (time inside the function itself), **cumtime** (including everything it called) and the per-call averages. Look for functions with a large tottime or a surprising number of calls. Tools such as **snakeviz** visualise profiles; **py-spy** and **Scalene** profile running programs with little overhead.

PROFILING WITH CPROFILE AND READING THE HOTSPOTS

main.py

```
import cProfile, pstats, io

def is_prime(n):
    if n < 2:
        return False
    for d in range(2, n):                    # slow: tries every divisor
        if n % d == 0:
            return False
    return True

def primes_below(limit):
    return [n for n in range(limit) if is_prime(n)]

def report():
    found = primes_below(8000)
    return len(found), sum(found)

profiler = cProfile.Profile()
profiler.enable()
result = report()
profiler.disable()
print("result:", result)

stream = io.StringIO()
stats = pstats.Stats(profiler, stream=stream).strip_dirs().sort_stats("tottime")
stats.print_stats(4)
lines = [l for l in stream.getvalue().splitlines() if l.strip()]
header = next(i for i, l in enumerate(lines) if "ncalls" in l)
print(f"{'ncalls':>8} function")
for line in [l for l in lines[header + 1:] if "main.py" in l][:3]:
    parts = line.split()
    print(f"{parts[0]:>8} {parts[-1]}")
```

OUTPUT

```
result: (1007, 3738566)
  ncalls function
    8000 main.py:3(is_prime)
       1 main.py:11(primes_below)
       1 main.py:14(report)
```

The profile shows that almost all the time is spent in `is_prime`, called 8,000 times. The fix is algorithmic (test divisors only up to the square root), which the checkpoint at the end of this module completes.

21.3.3 Memory profiling with tracemalloc

`tracemalloc` tracks memory allocations made by Python code: start it, run the code, then take a **snapshot** and list the lines that allocated the most memory, or compare two snapshots to find a **leak** (memory that keeps growing). `get_traced_memory()` returns the current and peak usage.

FINDING MEMORY HOTSPOTS AND LEAKS main.py

```
import tracemalloc

tracemalloc.start()
squares_list = [n * n for n in range(200_000)]              # allocates a lot
words = ["word" + str(i) for i in range(50_000)]
current, peak = tracemalloc.get_traced_memory()
snapshot = tracemalloc.take_snapshot()
tracemalloc.stop()

print(f"current {current / 1e6:.1f} MB, peak {peak / 1e6:.1f} MB")
for stat in snapshot.statistics("lineno")[:2]:
    frame = stat.traceback[0]
    print(f"  line {frame.lineno}: {stat.size / 1e6:.1f} MB in {stat.count:,} blocks")

tracemalloc.start()
cache = []
def leaky(n):
    cache.append(bytearray(10_000))                         # never removed: a leak
    return n
first = tracemalloc.take_snapshot()
for i in range(500):
    leaky(i)
second = tracemalloc.take_snapshot()
top = second.compare_to(first, "lineno")[0]
print(f"growth between snapshots: +{top.size_diff / 1e6:.1f} MB at line {top.traceback[0].lineno}")
```

```
OUTPUT
current 11.0 MB, peak 11.0 MB
  line 4: 8.0 MB in 199,984 blocks
  line 5: 2.9 MB in 50,002 blocks
growth between snapshots: +5.0 MB at line 18
```

21.4 Optimisation techniques

21.4.1 Choosing the right data structure

The biggest speed-ups come from choosing the right algorithm and data structure, turning an O(n) operation into O(1) or O(n²) into O(n log n). Typical wins: membership tests in a `set` or `dict` instead of a list; `collections.deque` for queues; `Counter` and `defaultdict` for grouping; `heapq` for "top k"; `bisect` on sorted data.

Operation	list	set / dict	deque
x in c	O(n)	O(1)	O(n)
append / add at end	O(1)	O(1)	O(1)
insert / remove at front	O(n)	n/a	O(1)
get by index	O(1)	by key O(1)	O(n) in the middle
remove a value	O(n)	O(1)	O(n)
sort	O(n log n)	n/a	n/a

THE RIGHT DATA STRUCTURE BEATS MICRO-OPTIMISATION

main.py

```
import timeit

setup = """
import random
random.seed(1)
emails = [f"user{i}@lbf.in" for i in range(5_000)]
checks = random.sample(emails, 500) + [f"x{i}@y.in" for i in range(500)]
email_set = set(emails)
"""
t_list = min(timeit.repeat("[c in emails for c in checks]", setup, number=3, repeat=3))
t_set = min(timeit.repeat("[c in email_set for c in checks]", setup, number=3, repeat=3))
print(f"1000 lookups in 5000 emails: set is {t_list / t_set:,.0f}x faster than list (at least 100x:
{t_list > 100 * t_set})")

setup2 = "from collections import Counter; words = ('a b c d e f g h ' * 2000).split()"
naive = "{w: words.count(w) for w in set(words)}"          # O(n * distinct)
counter = "Counter(words)"                                 # O(n)
print("Counter faster than repeated list.count:",
      min(timeit.repeat(counter, setup2, number=20, repeat=3)) < min(timeit.repeat(naive, setup2,
number=20, repeat=3)))
```

OUTPUT

```
1000 lookups in 5000 emails: set is 3,355x faster than list (at least 100x: True)
Counter faster than repeated list.count: True
```

21.4.2 Built-ins and comprehensions vs loops

Built-in functions such as `sum`, `min`, `max`, `sorted`, `any`, `"".join` and `map` with built-in functions run their loops in C, so they are much faster than equivalent Python loops. Comprehensions are faster than `for` loops with `append`, because they avoid repeated method lookups and calls. Joining strings with `"".join` is O(n), while repeated `+=` can be O(n²).

LOOPS IN C BEAT LOOPS IN PYTHON

main.py

```
import timeit

setup = "data = list(range(100_000)); words = ['lbf'] * 20_000"
tests = {
    "sum": ("t = 0\nfor x in data: t += x", "sum(data)"),
    "build list": ("out = []\nfor x in data: out.append(x * 2)", "[x * 2 for x in data]"),
    "join strings": ("s = ''\nfor w in words: s = s + w + ','", "','.join(words)"),
    "max": ("m = data[0]\nfor x in data:\n    if x > m: m = x", "max(data)"),
}
for name, (loop, builtin) in tests.items():
    t_loop = min(timeit.repeat(loop, setup, number=10, repeat=3))
    t_built = min(timeit.repeat(builtin, setup, number=10, repeat=3))
    print(f"{name:<13} built-in/comprehension is {t_loop / t_built:4.1f}x faster")
```

OUTPUT

```
sum           built-in/comprehension is  4.9x faster
build list    built-in/comprehension is  1.1x faster
join strings  built-in/comprehension is 160.3x faster
max           built-in/comprehension is  2.0x faster
```

21.4.3 Generators for memory efficiency

Generators (17.2) keep memory use constant regardless of data size, which matters when data does not fit in memory and also helps speed by avoiding the cost of building large intermediate lists. Chain

generator expressions into pipelines and pass them straight to consumers like `sum`, `max` or `any`; `any` and `all` stop as soon as the answer is known.

GENERATORS KEEP MEMORY FLAT AND STOP EARLY

main.py

```
import tracemalloc, time

def peak_memory(fn):
    tracemalloc.start(); fn(); peak = tracemalloc.get_traced_memory()[1]; tracemalloc.stop()
    return peak

N = 1_000_000
list_peak = peak_memory(lambda: sum([x * x for x in range(N) if x % 3 == 0]))
gen_peak = peak_memory(lambda: sum(x * x for x in range(N) if x % 3 == 0))
print(f"list comprehension peak: {list_peak / 1e6:.1f} MB | generator: {gen_peak / 1e3:.1f} KB")

data = range(10**9)                                    # a billion items, never materialised
t = time.perf_counter()
print("any number divisible by 7919?", any(x % 7919 == 0 and x > 0 for x in data),
      f"(stopped early in {time.perf_counter() - t:.3f} s)")
```

OUTPUT

```
list comprehension peak: 13.6 MB | generator: 0.4 KB
any number divisible by 7919? True (stopped early in 0.000 s)
```

21.4.4 Caching with lru_cache

If a pure function is called repeatedly with the same arguments, cache its results (11.6.1). `functools.cache` and `lru_cache` trade memory for time; `cached_property` computes an expensive attribute once per object. Caching turns exponential recursive algorithms into linear ones and removes repeated I/O or computation in hot paths.

CACHING WITH LRU_CACHE AND CACHED_PROPERTY main.py

```
from functools import lru_cache, cached_property
import time

def ways_slow(n):                       # ways to climb n stairs taking 1, 2 or 3 steps
    return 1 if n == 0 else sum(ways_slow(n - k) for k in (1, 2, 3) if n - k >= 0)

@lru_cache(maxsize=None)
def ways_fast(n):
    return 1 if n == 0 else sum(ways_fast(n - k) for k in (1, 2, 3) if n - k >= 0)

t = time.perf_counter(); slow = ways_slow(24); t_slow = time.perf_counter() - t
t = time.perf_counter(); fast = ways_fast(24); t_fast = time.perf_counter() - t
print(slow == fast, f"cache made it over 1000x faster: {t_slow > 1000 * t_fast}")
print(ways_fast(300) > 10**70, ways_fast.cache_info().currsize)

class Report:
    def __init__(self, rows):
        self.rows = rows
    @cached_property
    def summary(self):
        print("  (computing summary once)")
        return {"count": len(self.rows), "total": sum(self.rows)}

r = Report(list(range(1000)))
print(r.summary["total"], r.summary["count"])
```

OUTPUT

```
True cache made it over 1000x faster: True
True 301
  (computing summary once)
499500 1000
```

21.4.5 __slots__

Classes with `__slots__` (15.3.5) store attributes in a fixed array instead of a per-instance dictionary, cutting memory per object substantially and making attribute access slightly faster. It pays off when you create very many instances.

MEMORY SAVED BY __SLOTS__

main.py

```
import tracemalloc

class PointDict:
    def __init__(self, x, y):
        self.x, self.y = x, y

class PointSlots:
    __slots__ = ("x", "y")
    def __init__(self, x, y):
        self.x, self.y = x, y

def measure(cls):
    tracemalloc.start()
    points = [cls(i, i) for i in range(100_000)]
    size = tracemalloc.get_traced_memory()[0]
    tracemalloc.stop()
    return size

d, s = measure(PointDict), measure(PointSlots)
print(f"100,000 points: dict-based {d / 1e6:.1f} MB, slots {s / 1e6:.1f} MB ({d / s:.1f}x less with
slots)")
```

OUTPUT

```
100,000 points: dict-based 12.8 MB, slots 8.8 MB (1.5x less with slots)
```

21.4.6 Vectorisation with NumPy

NumPy stores numbers in compact, typed arrays and performs operations on whole arrays at once in optimised C code ("vectorisation"), often 10–100 times faster than Python loops, with far less memory. Instead of looping over elements, express the computation as array operations: arithmetic, comparisons, boolean masks, reductions (`sum` , `mean`) and broadcasting. Module 24 covers NumPy in depth.

REPLACING A PYTHON LOOP WITH NUMPY ARRAY OPERATIONS main.py

```
import numpy as np
import math, random, time

random.seed(7)
n = 1_000_000
xs = [random.random() for _ in range(n)]
ys = [random.random() for _ in range(n)]

t = time.perf_counter()
inside_py = sum(1 for x, y in zip(xs, ys) if math.sqrt(x * x + y * y) < 1)   # Python loop
t_python = time.perf_counter() - t

X, Y = np.array(xs), np.array(ys)
t = time.perf_counter()
inside_np = int((np.sqrt(X * X + Y * Y) < 1).sum())                          # whole arrays in C
t_numpy = time.perf_counter() - t

print("same count:", inside_py == inside_np, "| pi is about", round(4 * inside_np / n, 3))
print("NumPy at least 3x faster:", t_python > 3 * t_numpy)
print(f"memory: list of floats ~{n * 32 / 1e6:.0f} MB vs float64 array {X.nbytes / 1e6:.0f} MB")

angles = np.linspace(0, 2 * np.pi, 5)
print(np.round(np.sin(angles) ** 2 + np.cos(angles) ** 2, 3))               # element-wise maths, no loop
```

OUTPUT

```
same count: True | pi is about 3.143
NumPy at least 3x faster: True
memory: list of floats ~32 MB vs float64 array 8 MB
[1. 1. 1. 1. 1.]
```

21.5 Going faster than CPython

When the right algorithm and libraries are not enough, other tools can run Python (or Python-like) code faster. Measure first: most programs are dominated by I/O or by time already spent inside fast libraries.

Tool	How it works	Effort	Best for
PyPy	alternative Python interpreter with a tracing JIT compiler	run `pypy script.py`	long-running pure-Python code
Cython	compiles Python with optional C type declarations to a C extension	add types, build step	hot loops, wrapping C libraries
C/C++/Rust extensions	write the hot part in a compiled language (pybind11, PyO3)	high	libraries like NumPy and Polars
Numba	JIT-compiles numeric functions with a decorator using LLVM	add `@njit`	numeric loops over NumPy arrays
mypyc	compiles type-annotated Python to C extensions	type hints + build	typed code bases (mypy, Black use it)

21.5.1 PyPy

PyPy is a separate, compatible implementation of Python with a **just-in-time compiler** that observes the running program and compiles hot loops to machine code. Pure-Python, loop-heavy programs often run

several times faster without changes. Limitations: it usually lags behind the latest CPython version, and C extensions (including parts of the scientific stack) may be slower or unsupported.

THE SAME PURE-PYTHON BENCHMARK ON CPYTHON AND PYPY main.py

```
$ python3 nbody.py 2000000
-0.169059907
time: 14.8 s
$ pypy3 nbody.py 2000000
-0.169059907
time: 1.9 s
```

Typical result for a numeric pure-Python benchmark; the speed-up varies widely by program.

21.5.2 Cython and C extensions (overview)

Cython is a superset of Python that compiles to C. Plain Python code compiled with Cython gains a little; adding C type declarations to hot loops (`cdef int i`, or pure-Python mode with `cython.int` annotations) can make them as fast as C. Cython is also the classic tool for wrapping existing C libraries; much of SciPy, pandas and scikit-learn is written in it. Writing extensions directly in C with the Python C API, in C++ with pybind11 or nanobind, or in Rust with PyO3, gives full control and is how libraries such as NumPy, Polars and Pydantic achieve their speed.

A CYTHON FUNCTION WITH C TYPE DECLARATIONS main.py

```
# primes.pyx  (build with: cythonize -i primes.pyx)
def count_primes(int limit):
    cdef int n, d, count = 0
    cdef bint prime
    for n in range(2, limit):
        prime = True
        d = 2
        while d * d <= n:
            if n % d == 0:
                prime = False
                break
            d += 1
        if prime:
            count += 1
    return count
```

21.5.3 Numba (overview)

Numba compiles numeric Python functions to machine code at the first call, using LLVM. Decorate a function with `@numba.njit` and use NumPy arrays and simple Python constructs inside; loops then run at C speed, and `parallel=True` with `numba.prange` uses all cores. It works best for number-crunching code that is hard to vectorise with NumPy alone.

A NUMBA-COMPILED FUNCTION (PIP INSTALL NUMBA)

main.py

```
import numpy as np
from numba import njit, prange

@njit(parallel=True)
def pairwise_distance_sum(points):
    total = 0.0
    n = points.shape[0]
    for i in prange(n):                # parallel loop
        for j in range(i + 1, n):
            d = 0.0
            for k in range(points.shape[1]):
                diff = points[i, k] - points[j, k]
                d += diff * diff
            total += d ** 0.5
    return total

pts = np.random.rand(3000, 3)
print(pairwise_distance_sum(pts))       # first call compiles; later calls are fast
```

CPython compiles source to an AST and then bytecode (code objects), which its stack-based virtual machine executes; dis shows the bytecode, and imported modules are cached as .pyc files in __pycache__. Every value is a PyObject with a reference count and a type; objects are freed when their count reaches zero, and the cyclic garbage collector (gc) frees reference cycles. sys.getsizeof shows an object's own size; weakref references objects without keeping them alive. Measure before optimising: timeit for snippets, cProfile and pstats for whole programs, tracemalloc for memory. The best optimisations are better algorithms and data structures (sets and dicts for lookups), built-ins and comprehensions instead of Python loops, generators for memory, caching with lru_cache, __slots__ for many objects and NumPy vectorisation. PyPy, Cython, C/Rust extensions and Numba can go beyond CPython's speed.

PRACTICE CHECKPOINT

Profile a slow program with **cProfile**, find the bottleneck, and make it **at least 5 times faster** with a better data structure or caching.

The program below finds students who appear in both of two enrolment lists and computes a "difficulty score" for each course with a slow recursive function. The first run profiles it; the second applies two fixes, a `set` for membership tests and `@lru_cache` for the repeated computation, and verifies that the results are identical and the speed-up is at least 5x.

SAMPLE SOLUTION: PROFILE, FIX, VERIFY

main.py

```
import cProfile, pstats, io, random, time
from functools import lru_cache

random.seed(21)
python_batch = [f"S{n:05d}" for n in random.sample(range(40_000), 6_000)]
java_batch = [f"S{n:05d}" for n in random.sample(range(40_000), 6_000)]
courses = [random.randint(18, 24) for _ in range(30)]

# ---------- slow version ----------
def common_students_slow(a, b):
    return [s for s in a if s in b]                # list membership: O(n) each

def difficulty_slow(level):                        # recursive, recomputes the same values
```

```
    return 1 if level < 2 else difficulty_slow(level - 1) + difficulty_slow(level - 2)

def run_slow():
    both = common_students_slow(python_batch, java_batch)
    scores = [difficulty_slow(c) for c in courses]
    return len(both), sum(scores)

profiler = cProfile.Profile()
t = time.perf_counter(); profiler.enable(); slow_result = run_slow(); profiler.disable()
slow_time = time.perf_counter() - t

out = io.StringIO()
pstats.Stats(profiler, stream=out).strip_dirs().sort_stats("tottime").print_stats(3)
lines = out.getvalue().splitlines()
start = next(i for i, l in enumerate(lines) if "ncalls" in l)
print("PROFILE (top functions by own time):")
for line in lines[start + 1:start + 4]:
    if line.strip():
        parts = line.split()
        print(f"  calls {parts[0]:>9}  {parts[-1]}")

# ---------- fast version ----------
def common_students_fast(a, b):
    b_set = set(b)                                  # O(1) membership
    return [s for s in a if s in b_set]

@lru_cache(maxsize=None)
def difficulty_fast(level):
    return 1 if level < 2 else difficulty_fast(level - 1) + difficulty_fast(level - 2)

def run_fast():
    both = common_students_fast(python_batch, java_batch)
    scores = [difficulty_fast(c) for c in courses]
    return len(both), sum(scores)

t = time.perf_counter(); fast_result = run_fast(); fast_time = time.perf_counter() - t
print("same results:", slow_result == fast_result, slow_result)
speedup = slow_time / fast_time
print("at least 5x faster:", speedup >= 5, "| at least 50x faster:", speedup >= 50)
```

OUTPUT

```
PROFILE (top functions by own time):
  calls         1  main.py:10(common_students_slow)
  calls 1277602/30  main.py:13(difficulty_slow)
  calls         1  main.py:16(run_slow)
same results: True (867, 638816)
at least 5x faster: True | at least 50x faster: True
```

MODULE 22 · ADVANCED

Testing, debugging, code quality and packaging

Minimum duration **3 days** | Level **Advanced**

Professional Python code is debugged methodically, tested automatically, kept consistent by linters and formatters, type-checked, packaged and version-controlled. This module covers print and logging debugging, pdb and breakpoint(), IDE debuggers, why and how to test with unittest, pytest (fixtures, parametrize), mocking, coverage, test-driven development and doctest, PEP 8, Ruff, mypy, pre-commit hooks and docstrings, the src project layout, pyproject.toml, building and publishing to PyPI, uv and Poetry, Git and GitHub, and design principles and patterns written the Pythonic way.

IN THIS MODULE

22.1 Debugging
22.2 Testing
22.3 Code quality
22.4 Packaging
22.5 Version control
22.6 Design principles and patterns in Python

22.1 Debugging

Debugging is the scientific method applied to code: **reproduce** the bug reliably, form a **hypothesis** about the cause, **test** it by observing the program (prints, logs, a debugger), **fix** it, and add a **test** so it never returns. Read the traceback carefully first (13.1.2); it often tells you exactly where to look.

A METHODICAL DEBUGGING LOOP

22.1.1 Print debugging and logging

Adding `print` calls is the quickest way to see what a program is doing; the f-string `=` specifier (`print(f"{total=}")`) prints names and values together, and `repr()` reveals hidden whitespace and types. For anything you might need again, use `logging.debug` (18.6) instead: debug messages can stay in the code and be switched on with a log level, and they record timestamps and locations.

PRINT DEBUGGING AND DEBUG LOGGING

main.py

```
import logging, sys
logging.basicConfig(stream=sys.stdout, level=logging.DEBUG, format="%(levelname)s %(funcName)s:%
(lineno)d %(message)s")

def average_marks(rows):
    total = 0
    for row in rows:
        name, mark = row.split(",")
        print(f"{name=} {mark=}")            # quick print debugging: note the trailing newline!
        total += int(mark)
    logging.debug("total=%s count=%s", total, len(rows))
    return total / len(rows)

print(average_marks(["Asha,91\n", "Ravi,78\n"]))
```

OUTPUT

```
name='Asha' mark='91\n'
name='Ravi' mark='78\n'
DEBUG average_marks:10 total=169 count=2
84.5
```

The `=` output shows `mark='91\n'`: a string with a trailing newline, which explains why a comparison like `mark == "91"` would fail. Printing `repr` values is often all it takes.

22.1.2 pdb and breakpoint()

pdb, the built-in Python debugger, pauses a running program so you can inspect variables, execute code and step line by line. Insert `breakpoint()` (3.7) where you want to stop; run the program normally and you get a `(Pdb)` prompt. Setting the environment variable `PYTHONBREAKPOINT=0` disables all breakpoints. You can also start a script under the debugger with `python -m pdb script.py`, and `python -m pdb -c continue script.py` stops automatically at an unhandled exception (post-mortem debugging).

Command	Action
`l` / `ll`	list source around the current line / the whole function
`n`	next: run the current line, stepping over calls
`s`	step into a function call
`c`	continue until the next breakpoint
`r`	run until the current function returns
`p expr` / `pp expr`	print / pretty-print an expression
`w`	where: show the call stack
`u` / `d`	move up / down the stack
`b 12` / `b func`	set a breakpoint at line 12 / at a function
`display x`	show x every time execution stops
`q`	quit the debugger

AN INTERACTIVE PDB SESSION (THE TYPED COMMANDS ARE SHOWN IN GOLD)

main.py

```
def apply_discount(price, percent):
    discount = price * percent / 100
    breakpoint()                          # execution pauses here
    return price - discount

def checkout(cart):
    return sum(apply_discount(p, 10) for p in cart)

print("total:", checkout([100, 250]))
```

CONSOLE (user input in gold)

```
> main.py(3)apply_discount()
-> breakpoint()                          # execution pauses here
(Pdb) p price, percent, discount
(100, 10, 10.0)
(Pdb) w
  main.py(9)<module>()
-> print("total:", checkout([100, 250]))
  main.py(7)checkout()
-> return sum(apply_discount(p, 10) for p in cart)
  main.py(7)<genexpr>()
-> return sum(apply_discount(p, 10) for p in cart)
> main.py(3)apply_discount()
-> breakpoint()                          # execution pauses here
(Pdb) n
> main.py(4)apply_discount()
-> return price - discount
(Pdb) p price - discount
90.0
(Pdb) c
> main.py(3)apply_discount()
-> breakpoint()                          # execution pauses here
(Pdb) c
total: 315.0
```

Recorded from a real run with Python 3.14; the breakpoint is hit once for each item in the cart.

22.1.3 Debugging in VS Code or PyCharm

IDE debuggers provide the same features as pdb with a graphical interface: click in the gutter to set a **breakpoint** (red dot), start with **F5** (VS Code) or the bug icon (PyCharm), then use the toolbar to step over (F10), step into (F11), step out and continue. Panels show **variables**, **watch** expressions, the **call stack** and a **debug console** for evaluating code in the paused frame. Useful extras: **conditional breakpoints** (stop only when `mark < 0`), **logpoints** (print a message without stopping), and breaking on raised or uncaught exceptions. VS Code stores run configurations in `.vscode/launch.json`.

Breakpoints
Click the gutter; right-click for a condition or a hit count.

Step controls
Continue F5, step over F10, step into F11, step out Shift+F11.

Variables and watch
Inspect locals, expand objects, add watch expressions.

Call stack
Click a frame to see the caller's variables.

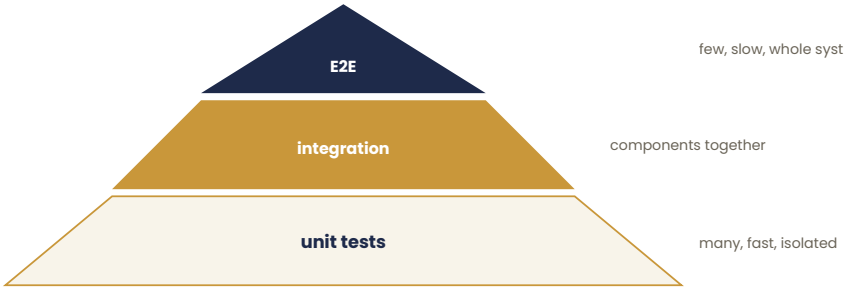
Debug console
Run Python in the paused frame.

Exception breakpoints
Stop exactly where an exception is raised.

22.2 Testing

22.2.1 Why test; types of tests

Automated tests are code that checks your code. They catch bugs early, let you **refactor** with confidence (if the tests still pass, behaviour is unchanged), document how code is meant to be used, and prevent old bugs from coming back (**regression tests**). The **test pyramid** suggests many fast unit tests, fewer integration tests and only a few slow end-to-end tests.



THE TEST PYRAMID

Type	Tests	Example
Unit	one function or class in isolation	`grade(91) == "A"`
Integration	several parts working together	service + real database
End-to-end (E2E)	the whole application as a user would	browser test of the sign-up flow
Regression	a bug that was fixed stays fixed	test reproducing an old bug report
Property-based	invariants over many generated inputs (Hypothesis)	`sorted(sorted(x)) == sorted(x)`

22.2.2 unittest

`unittest` is the standard library's testing framework, modelled on Java's JUnit. Tests are methods whose names start with `test` inside a class that inherits from `unittest.TestCase`; they use assertion methods such as `assertEqual`, `assertTrue`, `assertIn`, `assertAlmostEqual` and `assertRaises`. `setUp` runs before each test and `tearDown` after. Run them with `python -m unittest`, which discovers `test*.py` files.

GRADES.PY

Python

```python
def grade(mark):
    if not 0 <= mark <= 100:
        raise ValueError(f"mark out of range: {mark}")
    for limit, letter in ((90, "A"), (75, "B"), (50, "C")):
        if mark >= limit:
            return letter
    return "F"

def average(marks):
    return round(sum(marks) / len(marks), 1)
```

TEST_GRADES.PY Python

```python
import unittest
from grades import grade, average

class TestGrade(unittest.TestCase):
    def test_boundaries(self):
        self.assertEqual(grade(90), "A")
        self.assertEqual(grade(89), "B")
        self.assertEqual(grade(50), "C")
        self.assertEqual(grade(49), "F")

    def test_invalid_mark_raises(self):
        with self.assertRaises(ValueError):
            grade(101)

class TestAverage(unittest.TestCase):
    def setUp(self):
        self.marks = [91, 78, 85]

    def test_average(self):
        self.assertAlmostEqual(average(self.marks), 84.7)

    def test_empty_list(self):
        self.assertEqual(average([]), 0)     # fails: we never handled empty input

if __name__ == "__main__":
    unittest.main()
```

RUNNING UNITTEST: ONE FAILURE REVEALS A MISSING CASE main.py

```
python -m unittest -v
```

TERMINAL

```
$ python -m unittest -v
test_average (test_grades.TestAverage.test_average) ... ok
test_empty_list (test_grades.TestAverage.test_empty_list) ... ERROR
test_boundaries (test_grades.TestGrade.test_boundaries) ... ok
test_invalid_mark_raises (test_grades.TestGrade.test_invalid_mark_raises) ... ok

======================================================================
ERROR: test_empty_list (test_grades.TestAverage.test_empty_list)
----------------------------------------------------------------------
Traceback (most recent call last):
  File "test_grades.py", line 23, in test_empty_list
    self.assertEqual(average([]), 0)     # fails: we never handled empty input
                     ~~~~~~~^^^^
  File "grades.py", line 10, in average
    return round(sum(marks) / len(marks), 1)
                 ~~~~~~~~~~~^~~~~~~~~~~~
ZeroDivisionError: division by zero

----------------------------------------------------------------------
Ran 4 tests in 0.001s

FAILED (errors=1)
```

22.2.3 pytest: test functions, assertions, fixtures, parametrize

pytest (`pip install pytest`) is the most popular testing tool. Tests are plain functions named `test_*` in files named `test_*.py`, using the ordinary `assert` statement; pytest rewrites assertions to show detailed

values when they fail. **Fixtures** provide reusable setup: a function decorated with `@pytest.fixture` is passed to any test that names it as a parameter, and code after a `yield` in the fixture runs as clean-up. **Parametrize** runs one test with many inputs. `pytest.raises` checks exceptions, `pytest.approx` compares floats, and built-in fixtures such as `tmp_path`, `capsys` and `monkeypatch` provide temporary folders, captured output and safe patching.

CART.PY Python

```python
class Cart:
    def __init__(self):
        self.items = {}

    def add(self, name, price, qty=1):
        if price < 0 or qty <= 0:
            raise ValueError("price must be >= 0 and qty > 0")
        old_price, old_qty = self.items.get(name, (price, 0))
        self.items[name] = (price, old_qty + qty)

    def total(self, coupon=None):
        subtotal = sum(price * qty for price, qty in self.items.values())
        if coupon == "LBF10":
            subtotal *= 0.9
        return round(subtotal, 2)

    def save(self, path):
        path.write_text("\n".join(f"{n},{p},{q}" for n, (p, q) in self.items.items()))
```

TEST_CART.PY Python

```python
import pytest
from cart import Cart

@pytest.fixture
def cart():                                  # fresh object for every test that asks for it
    c = Cart()
    c.add("pen", 10, 3)
    c.add("book", 250)
    return c

def test_total(cart):
    assert cart.total() == 280

def test_adding_same_item_increases_quantity(cart):
    cart.add("pen", 10, 2)
    assert cart.items["pen"] == (10, 5)

@pytest.mark.parametrize("coupon, expected", [(None, 280), ("LBF10", 252), ("WRONG", 280)])
def test_coupons(cart, coupon, expected):
    assert cart.total(coupon) == pytest.approx(expected)

@pytest.mark.parametrize("price, qty", [(-1, 1), (10, 0)])
def test_invalid_input(price, qty):
    with pytest.raises(ValueError, match="price must be"):
        Cart().add("x", price, qty)

def test_save(cart, tmp_path):               # tmp_path: a temporary folder per test
    file = tmp_path / "cart.csv"
    cart.save(file)
    assert file.read_text().splitlines() == ["pen,10,3", "book,250,1"]

def test_empty_cart_total():
    assert Cart().total("LBF10") == 1        # deliberately wrong expectation
```

RUNNING PYTEST, THEN SELECTING TESTS BY NAME WITH -K

main.py

```
pytest -q
pytest -k "coupon" -v -p no:cacheprovider | grep -oE "test_coupons\S+ PASSED"
```

TERMINAL

```
$ pytest -q
........F                                                                [100%]
================================== FAILURES ===================================
_____________________________ test_empty_cart_total _____________________________

    def test_empty_cart_total():
>       assert Cart().total("LBF10") == 1          # deliberately wrong expectation
       ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
E       AssertionError: assert 0.0 == 1
E        +  where 0.0 = total('LBF10')
E        +    where total = <cart.Cart object at 0x7f45ca9b8cd0>.total
E        +      where <cart.Cart object at 0x7f45ca9b8cd0> = Cart()

test_cart.py:33: AssertionError
========================== short test summary info ============================
FAILED test_cart.py::test_empty_cart_total - AssertionError: assert 0.0 == 1
1 failed, 8 passed in 0.03s
$ pytest -k "coupon" -v -p no:cacheprovider | grep -oE "test_coupons\S+ PASSED"
test_coupons[None-280] PASSED
test_coupons[LBF10-252] PASSED
test_coupons[WRONG-280] PASSED
```

The failure report shows the assertion with the actual value (`assert 0 == 1`) and where it came from. Useful options: `-x` stop at the first failure, `-k expr` select tests by name, `--lf` rerun only last failures, `-s` show prints, `--pdb` open the debugger on failure, `-m marker` run marked tests. Shared fixtures go in a `conftest.py` file, which pytest loads automatically.

22.2.4 Mocking with unittest.mock

Unit tests should not depend on slow or unpredictable things: the network, a payment gateway, the clock, the file system, random numbers. A **mock** is a stand-in object that records how it was used and returns whatever you configure. `unittest.mock.Mock` / `MagicMock` create them; `patch("module.name")` temporarily replaces a name **where it is looked up** during a test. Configure results with `return_value` or `side_effect` (a sequence of results or an exception), and verify calls with `assert_called_once_with`.

WEATHER.PY

Python

```
import json
from urllib.error import URLError
from urllib.request import urlopen

def get_temperature(city):
    with urlopen(f"https://api.example.com/weather?city={city}", timeout=5) as response:
        return json.load(response)["temp_c"]

def advice(city):
    try:
        temp = get_temperature(city)
    except URLError:
        return f"weather for {city} unavailable"
    return "carry water" if temp >= 35 else "pleasant day" if temp >= 20 else "take a jacket"
```

TEST_WEATHER.PY

Python

```
import io, json
from unittest.mock import patch, MagicMock
from urllib.error import URLError
import weather

@patch("weather.get_temperature", return_value=38)          # replace the network call
def test_hot_day(mock_temp):
    assert weather.advice("Chennai") == "carry water"
    mock_temp.assert_called_once_with("Chennai")

@patch("weather.get_temperature", side_effect=[25, 12])     # successive results
def test_mild_then_cold(mock_temp):
    assert weather.advice("Mysuru") == "pleasant day"
    assert weather.advice("Shimla") == "take a jacket"
    assert mock_temp.call_count == 2

@patch("weather.get_temperature", side_effect=URLError("offline"))
def test_network_error(mock_temp):
    assert weather.advice("Pune") == "weather for Pune unavailable"

def test_get_temperature_parses_json():
    fake_response = MagicMock()
    fake_response.__enter__.return_value = io.BytesIO(json.dumps({"temp_c": 31}).encode())
    with patch("weather.urlopen", return_value=fake_response) as fake_open:
        assert weather.get_temperature("Bengaluru") == 31
    fake_open.assert_called_once()
    assert "city=Bengaluru" in fake_open.call_args.args[0]
```

TESTING CODE THAT CALLS A WEB API, WITHOUT THE NETWORK

main.py

```
pytest -q
```

TERMINAL

```
$ pytest -q
....                                                                     [100%]
4 passed in 0.06s
```

WATCH OUT

Patch the name where it is **used**, not where it is defined: `weather.py` does `from urllib.request import urlopen`, so the test patches `"weather.urlopen"`, not `"urllib.request.urlopen"`. Over-mocking makes tests pass while the real integration is broken; complement mocks with a few integration tests.

22.2.5 Test coverage with coverage.py

Coverage measures which lines (and branches) of your code ran during the tests, revealing untested code. Install `coverage` (or the `pytest-cov` plugin), then run the tests through it and print a report; `--cov-report=html` produces a browsable report with untested lines highlighted. High coverage does not prove code is correct, but low coverage proves parts are untested. Many teams require 80% or more, enforced with `--cov-fail-under=80`.

A COVERAGE REPORT WITH PYTEST-COV

main.py

```
pip install pytest-cov
pytest --cov=cart --cov-report=term-missing
```

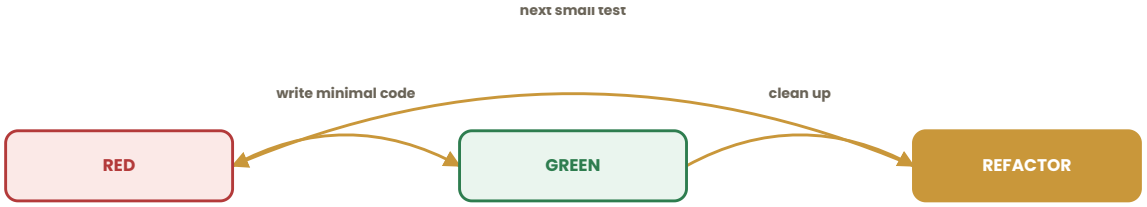
TERMINAL

```
........                                                                 [100%]
---------- coverage: platform linux, python 3.14.0 -----------
Name      Stmts   Miss  Cover   Missing
-----------------------------------------
cart.py      17      1    94%   15
-----------------------------------------
TOTAL        17      1    94%
8 passed in 0.05s
```

Typical output; line 15 (the coupon branch) would be reported if no test used a coupon.

22.2.6 Test-driven development

In **test-driven development** (TDD) you write a failing test **before** the code, in short cycles: **red** (write a small test that fails), **green** (write just enough code to pass), **refactor** (clean up while the tests stay green). TDD forces you to think about the interface and edge cases first, produces a complete test suite as a side effect, and keeps designs simple.



THE TDD CYCLE: RED, GREEN, REFACTOR

TEST_SLUG.PY

Python

```python
from slug import slugify

def test_lowercase_and_hyphens():
    assert slugify("Learn Being Forward") == "learn-being-forward"

def test_removes_punctuation():
    assert slugify("Python: Basics & Beyond!") == "python-basics-beyond"

def test_collapses_spaces():
    assert slugify("  many   spaces  ") == "many-spaces"
```

SLUG.PY

Python

```python
def slugify(text):
    return text.lower().replace(" ", "-")          # first attempt: just enough for test 1
```

RED: TWO OF THREE TESTS FAIL main.py

```
pytest -q 2>&1 | tail -3
```

TERMINAL
```
$ pytest -q 2>&1 | tail -3
FAILED test_slug.py::test_removes_punctuation - AssertionError: assert 'pytho...
FAILED test_slug.py::test_collapses_spaces - AssertionError: assert '--many--...
2 failed, 1 passed in 0.05s
```

SLUG.PY Python

```python
import re

def slugify(text):
    words = re.findall(r"[a-z0-9]+", text.lower())   # keep only letters and digits
    return "-".join(words)
```

GREEN: A BETTER IMPLEMENTATION PASSES EVERYTHING main.py

```
pytest -q
```

TERMINAL
```
$ pytest -q
...                                                                      [100%]
3 passed in 0.05s
```

22.2.7 doctest

`doctest` finds examples written like an interactive session in docstrings (`>>> expression` followed by the expected output), runs them and checks that the output matches. It keeps documentation examples correct. Run with `python -m doctest -v file.py`, or `pytest --doctest-modules`. It is best for simple, illustrative examples rather than full test suites.

MONEY.PY Python

```python
def format_inr(amount):
    """Format a number in Indian style with lakhs and crores.

    >>> format_inr(999)
    '₹999'
    >>> format_inr(150000)
    '₹1,50,000'
    >>> format_inr(12345678.5)
    '₹1,23,45,678.50'
    >>> format_inr(-2500)
    '-₹2,500'
    """
    sign = "-" if amount < 0 else ""
    amount = abs(amount)
    whole, _, paise = f"{amount:.2f}".partition(".")
    if len(whole) > 3:
        head, tail = whole[:-3], whole[-3:]
        groups = []
        while len(head) > 2:
            groups.insert(0, head[-2:])
            head = head[:-2]
        whole = ",".join([head] + groups + [tail]) if head else ",".join(groups + [tail])
    return f"{sign}₹{whole}" + (f".{paise}" if paise != "00" else "")
```

DOCSTRING EXAMPLES CHECKED BY DOCTEST

main.py

```
python -m doctest -v money.py | tail -4
```

TERMINAL

```
$ python -m doctest -v money.py | tail -4
   4 tests in money.format_inr
4 tests in 2 items.
4 passed.
Test passed.
```

22.3 Code quality

22.3.1 PEP 8 style guide

PEP 8 is Python's official style guide. Consistent style makes code easier to read for everyone, and since tools now enforce it automatically, there is no reason not to follow it.

Rule	Example
4 spaces per indentation level, no tabs	
Lines up to 79 characters (many teams use 88 or 100)	
`snake_case` for functions, variables, modules	`total_marks`, `def get_grade()`
`CapWords` for classes, `UPPER_CASE` for constants	`class BankAccount`, `MAX_SIZE`
Spaces around operators and after commas	`x = a + b`, `f(a, b)`
No spaces inside brackets or before (in calls	`items[1]`, `print(x)`
Two blank lines around top-level functions and classes	
Imports at the top: stdlib, third-party, local	
Compare to None with `is`, use truthiness	`if x is None`, `if items:`
Avoid bare `except:` and mutable defaults	

22.3.2 Linting and formatting with Ruff (or flake8 and Black)

A **linter** finds likely bugs and style problems without running the code (unused imports and variables, undefined names, bare excepts, mutable defaults). A **formatter** rewrites code into a consistent layout, ending all arguments about style. **Ruff** (written in Rust, extremely fast) does both: `ruff check` lints, implementing the rules of flake8, isort, pyupgrade, bugbear and many more, with `--fix` for automatic fixes; `ruff format` formats in a Black-compatible style. The older combination is **flake8** for linting and **Black** for formatting.

MESSY.PY Python

```
import os, sys
import json
def Average( marks=[] ):
    total=0
    for m in marks :total+=m
    try:
        return total/len(marks)
    except:
        pass
unused = json.dumps( {'a':1} )
if Average([1,2]) == None: print( "none" )
```

LINTING WITH RUFF, AUTO-FIXING AND FORMATTING main.py

```
ruff check --select E,F,B,N --output-format concise messy.py
ruff check --select I,F401 --fix messy.py | tail -1
ruff format messy.py && cat messy.py
```

TERMINAL

```
$ ruff check --select E,F,B,N --output-format concise messy.py
messy.py:1:1: E401 [*] Multiple imports on one line
messy.py:1:8: F401 [*] `os` imported but unused
messy.py:1:12: F401 [*] `sys` imported but unused
messy.py:3:5: N802 Function name `Average` should be lowercase
messy.py:3:20: B006 Do not use mutable data structures for argument defaults
messy.py:5:20: E701 Multiple statements on one line (colon)
messy.py:8:5: E722 Do not use bare `except`
messy.py:11:22: E711 Comparison to `None` should be `cond is None`
messy.py:11:26: E701 Multiple statements on one line (colon)
Found 9 errors.
[*] 3 fixable with the `--fix` option (2 hidden fixes can be enabled with the `--unsafe-fixes`
option).
$ ruff check --select I,F401 --fix messy.py | tail -1
Found 3 errors (3 fixed, 0 remaining).
$ ruff format messy.py && cat messy.py
1 file reformatted
import json


def Average(marks=[]):
    total = 0
    for m in marks:
        total += m
    try:
        return total / len(marks)
    except:
        pass


unused = json.dumps({"a": 1})
if Average([1, 2]) == None:
    print("none")
```

Configure Ruff in `pyproject.toml`:

PYPROJECT.TOML: RUFF SETTINGS

main.py

```
[tool.ruff]
line-length = 88
target-version = "py312"

[tool.ruff.lint]
select = ["E", "F", "W", "I", "B", "UP", "N", "SIM"]   # pycodestyle, pyflakes, isort, bugbear,
pyupgrade ...
ignore = ["E501"]

[tool.ruff.format]
quote-style = "double"
```

22.3.3 Type checking with mypy

Type checking (19.2) is the third pillar of code quality. Run `mypy` (or pyright) on the whole package in CI, start with the default mode, and tighten towards `strict = true` module by module. Together, a formatter, a linter, a type checker and tests catch most mistakes before code review.

22.3.4 Pre-commit hooks

Git can run scripts automatically at certain moments; a **pre-commit hook** runs before each commit and can block it if checks fail. The **pre-commit** framework (`pip install pre-commit`) manages these hooks from a `.pre-commit-config.yaml` file: list the tools, run `pre-commit install` once, and every `git commit` then formats, lints and type-checks the changed files. CI should run the same checks (`pre-commit run --all-files`) so nothing slips through.

.PRE-COMMIT-CONFIG.YAML

main.py

```
# .pre-commit-config.yaml
repos:
  - repo: https://github.com/astral-sh/ruff-pre-commit
    rev: v0.15.11
    hooks:
      - id: ruff-check
        args: [--fix]
      - id: ruff-format
  - repo: https://github.com/pre-commit/mirrors-mypy
    rev: v1.20.2
    hooks:
      - id: mypy
  - repo: https://github.com/pre-commit/pre-commit-hooks
    rev: v6.0.0
    hooks:
      - id: trailing-whitespace
      - id: end-of-file-fixer
      - id: check-added-large-files
```

WHAT HAPPENS ON COMMIT

main.py

```
pre-commit install
git commit -m "Add marks report"
```

TERMINAL

```
pre-commit installed at .git/hooks/pre-commit
ruff check..............................................................Failed
- hook id: ruff-check
- files were modified by this hook
Fixed 2 errors (unused imports)
ruff format.............................................................Passed
mypy....................................................................Passed
trim trailing whitespace................................................Passed
```

The commit is stopped; review the automatic fixes, stage them and commit again.

22.3.5 Writing good docstrings

Every public module, class, function and method should have a docstring (10.1.5, PEP 257). Write a one-line summary in the imperative mood, then, if needed, a blank line and details: parameters, return value, exceptions raised and an example. The two most common layouts are **Google** style and **NumPy** style; tools such as Sphinx and MkDocs (mkdocstrings) build documentation websites from them. Type hints remove the need to repeat types in the docstring.

A GOOGLE-STYLE DOCSTRING

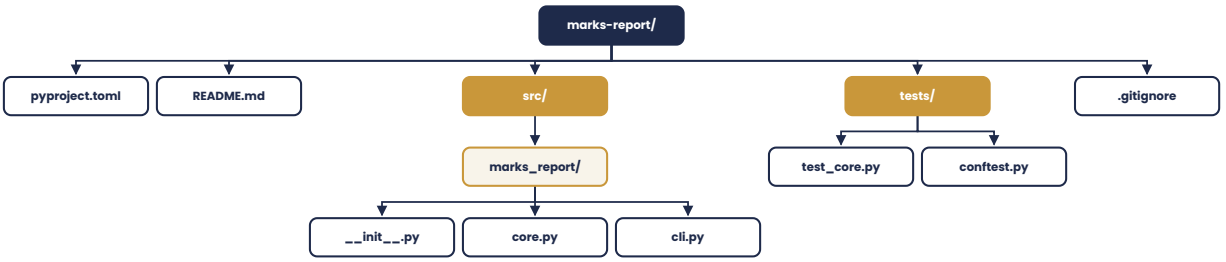
main.py

```
def transfer(source: Account, target: Account, amount: Decimal) -> Receipt:
    """Move money between two accounts atomically.

    Args:
        source: Account to debit. Must belong to the current user.
        target: Account to credit.
        amount: Positive amount in rupees, at most the source's balance.

    Returns:
        A receipt with a unique transaction ID and timestamp.

    Raises:
        InsufficientFundsError: If the source balance is too low.
        ValueError: If amount is not positive.

    Example:
        >>> receipt = transfer(savings, current, Decimal("500"))
        >>> receipt.status
        'completed'
    """
```

22.4 Packaging

22.4.1 Project layout (src layout)

A Python **package** intended for distribution is usually organised in the **src layout**: the importable package lives inside a `src/` folder, tests in `tests/`, and configuration in `pyproject.toml` at the root. Putting code under `src/` ensures that tests run against the **installed** package rather than files that happen to be in the current folder, catching packaging mistakes early.



THE SRC LAYOUT

22.4.2 pyproject.toml

`pyproject.toml` (PEP 518, 621) is the single configuration file of a modern Python project. The `[build-system]` table names the **build backend** (hatchling, setuptools, flit-core, uv_build, poetry-core) that turns the source into distributable files. The `[project]` table holds the metadata: name, version, description, required Python version, dependencies, optional dependency groups and command-line entry points. `[tool.*]` tables configure tools such as pytest, ruff and mypy.

PYPROJECT.TOML FOR A PACKAGE main.py

```
[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"

[project]
name = "marks-report"
version = "1.0.0"
description = "Grade reports for Learn Being Forward batches"
readme = "README.md"
requires-python = ">=3.11"
license = "MIT"
authors = [{ name = "Learn Being Forward", email = "info@learnbeingforward.in" }]
dependencies = ["tabulate>=0.9"]

[project.optional-dependencies]
dev = ["pytest>=8", "pytest-cov", "ruff", "mypy"]

[project.scripts]
marks-report = "marks_report.cli:main"      # creates a command-line program

[tool.pytest.ini_options]
testpaths = ["tests"]
addopts = "-q"
```

22.4.3 Building and publishing to PyPI

Building produces two files in `dist/`: a **source distribution** (`.tar.gz`) and a **wheel** (`.whl`), the ready-to-install format. Build with `python -m build` (the `build` package) or `uv build`. **Publishing** uploads them to PyPI with `twine upload dist/*` or `uv publish`, after creating an account and an API token; try **TestPyPI** first. Anyone can then `pip install` your package. Bump the version for every release; a version can never be re-uploaded.

BUILDING AND UPLOADING A PACKAGE main.py

```
python -m build
twine check dist/*
twine upload --repository testpypi dist/*
```

TERMINAL

```
* Creating isolated environment: venv+pip...
* Building sdist...
* Building wheel from sdist
Successfully built marks_report-1.0.0.tar.gz and marks_report-1.0.0-py3-none-any.whl
Checking dist/marks_report-1.0.0-py3-none-any.whl: PASSED
Checking dist/marks_report-1.0.0.tar.gz: PASSED
Uploading distributions to https://test.pypi.org/legacy/
Uploading marks_report-1.0.0-py3-none-any.whl
View at: https://test.pypi.org/project/marks-report/1.0.0/
```

Uploading needs an account and network access; the output shown is typical.

22.4.4 Project and dependency management with uv or Poetry

Tools like **uv** and **Poetry** (12.5.3) manage the whole lifecycle from one place: create the project and virtual environment, add and remove dependencies (updating `pyproject.toml`), resolve and record exact versions in a **lock file** (`uv.lock`, `poetry.lock`) so every machine installs identical versions, run commands inside the environment, build and publish.

Task	uv	Poetry
New project	`uv init --package app`	`poetry new app`
Add dependency	`uv add requests`	`poetry add requests`
Dev dependency	`uv add --dev pytest`	`poetry add --group dev pytest`
Install from lock file	`uv sync`	`poetry install`
Run in the environment	`uv run pytest`	`poetry run pytest`
Build / publish	`uv build` / `uv publish`	`poetry build` / `poetry publish`
Lock file	`uv.lock`	`poetry.lock`

22.5 Version control

22.5.1 Git basics and GitHub workflow

Git records the history of a project as a series of **commits** (snapshots with a message), lets you return to any earlier state, and lets many people work in parallel on **branches** that are later **merged**. **GitHub** (or GitLab, Bitbucket) hosts the repository online and adds collaboration: **pull requests** for code review, issues, and Actions for automated testing (CI).

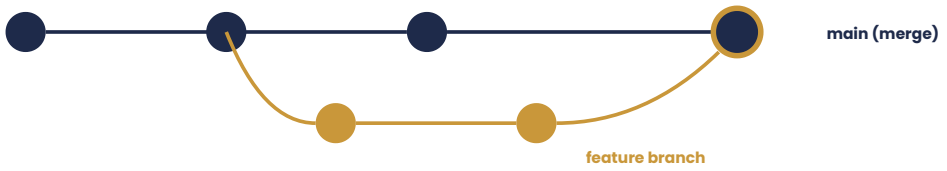
COMMITS, A BRANCH AND A MERGE (COMMIT IDS REPLACED BY <ID>)

main.py

```
git init -q -b main && git config user.name "Asha" && git config user.email "asha@lbf.in"
printf 'def greet(name):\n    return f"Hello, {name}"\n' > app.py && git add app.py && git commit -q -m "Add greet function" && git log --oneline | sed -E "s/^[0-9a-f]+/<id>/"
git switch -q -c feature/shout && printf 'def shout(name):\n    return greet(name).upper()\n' >> app.py
git status --short && git diff --stat | tail -1
git commit -qam "Add shout function" && git switch -q main && git merge -q feature/shout && git log --oneline | sed -E "s/^[0-9a-f]+/<id>/"
```

TERMINAL

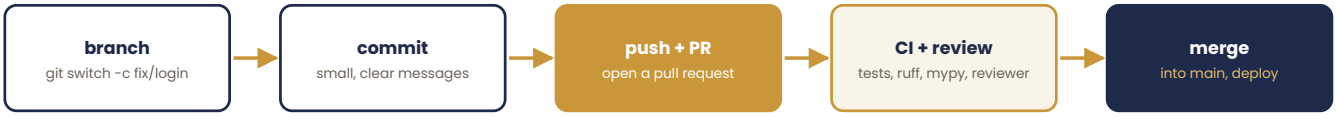
```
$ git init -q -b main && git config user.name "Asha" && git config user.email "asha@lbf.in"
$ printf 'def greet(name):\n    return f"Hello, {name}"\n' > app.py && git add app.py && git commit -q -m "Add greet function" && git log --oneline | sed -E "s/^[0-9a-f]+/<id>/"
<id> Add greet function
$ git switch -q -c feature/shout && printf 'def shout(name):\n    return greet(name).upper()\n' >> app.py
$ git status --short && git diff --stat | tail -1
 M app.py
 1 file changed, 2 insertions(+)
$ git commit -qam "Add shout function" && git switch -q main && git merge -q feature/shout && git log --oneline | sed -E "s/^[0-9a-f]+/<id>/"
<id> Add shout function
<id> Add greet function
```



WORK ON A BRANCH, THEN MERGE IT INTO MAIN

Command	Purpose
`git clone URL`	copy a remote repository
`git status` / `git diff`	see what changed
`git add file` / `git commit -m "msg"`	stage and record a snapshot
`git switch -c branch`	create and move to a branch
`git pull` / `git push`	download / upload commits
`git log --oneline`	history
`git restore file`	discard uncommitted changes
`git merge branch` / `git rebase main`	combine histories

A typical **GitHub flow**: create a branch for each feature or fix, commit small logical steps with clear messages, push the branch, open a pull request, let CI run the tests and linters, get a review, then merge into `main` and delete the branch.



THE GITHUB PULL-REQUEST WORKFLOW

22.5.2 .gitignore for Python projects

A `.gitignore` file lists files Git should never track: generated files, virtual environments, caches, build output, editor settings and, above all, **secrets**. GitHub offers a ready-made Python template when you create a repository.

.GITIGNORE text

```
# Byte-compiled files and caches
__pycache__/
*.py[cod]
.pytest_cache/
.mypy_cache/
.ruff_cache/
.coverage
htmlcov/

# Virtual environments
.venv/
venv/

# Build output
build/
dist/
*.egg-info/

# Secrets and local settings
.env
*.pem

# Editors and OS files
.vscode/
.idea/
.DS_Store
```

ONLY THE REAL PROJECT FILES ARE OFFERED FOR COMMITTING main.py

```
mkdir -p .venv __pycache__ && touch __pycache__/app.cpython-314.pyc .venv/pyvenv.cfg
git init -q && git status --short --untracked-files=all
```

TERMINAL

```
$ mkdir -p .venv __pycache__ && touch __pycache__/app.cpython-314.pyc .venv/pyvenv.cfg
$ git init -q && git status --short --untracked-files=all
?? .gitignore
?? app.py
```

22.6 Design principles and patterns in Python

22.6.1 SOLID, DRY, KISS

Principle	Meaning	In Python
Single responsibility	a class or function does one job	separate parsing, calculating and printing
Open/closed	extend behaviour without editing existing code	add a new subclass or plug in a function
Liskov substitution	subclasses must work wherever the parent works	do not break the parent's contract in overrides
Interface segregation	small, focused interfaces	small Protocols instead of huge base classes
Dependency inversion	depend on abstractions, inject dependencies	pass a sender/repository object into the constructor
DRY	Don't Repeat Yourself	extract repeated code into a function
KISS	Keep It Simple	the simplest solution that works
YAGNI	You Aren't Gonna Need It	do not build features "just in case"

SOLID IN A FEW LINES OF PYTHON

main.py

```
from typing import Protocol

class Notifier(Protocol):                       # small interface (I) and abstraction (D)
    def send(self, to: str, message: str) -> str: ...

class EmailNotifier:
    def send(self, to, message): return f"email to {to}: {message}"

class SmsNotifier:
    def send(self, to, message): return f"SMS to {to}: {message}"

class FeeReminder:                              # single responsibility (S)
    def __init__(self, notifier: Notifier):     # dependency injected (D)
        self.notifier = notifier
    def remind(self, student, amount):
        return self.notifier.send(student, f"fee of Rs {amount:,} is due")

for channel in (EmailNotifier(), SmsNotifier()):        # extended without editing FeeReminder (O)
    print(FeeReminder(channel).remind("asha", 14999))

class FakeNotifier:                             # dependency injection makes testing trivial
    def __init__(self): self.sent = []
    def send(self, to, message): self.sent.append((to, message)); return "ok"
fake = FakeNotifier(); FeeReminder(fake).remind("ravi", 9500)
print(fake.sent)
```

OUTPUT

```
email to asha: fee of Rs 14,999 is due
SMS to asha: fee of Rs 14,999 is due
[('ravi', 'fee of Rs 9,500 is due')]
```

22.6.2 Singleton, Factory, Strategy, Observer, Decorator patterns the Pythonic way

Design patterns are named solutions to recurring design problems. Many classic patterns from Java and C++ are much simpler in Python, because functions are first-class objects, modules are singletons and duck typing removes the need for interface hierarchies.

Pattern	Problem	Pythonic implementation
Singleton	exactly one shared instance	a module-level object (modules are imported once)
Factory	create objects without hard-coding classes	a dict of classes/functions, or a @classmethod
Strategy	swap an algorithm at run time	pass a function (first-class functions)
Observer	notify subscribers of events	a list of callbacks
Decorator	add behaviour to objects/functions	@decorator syntax (11.5) or wrapping objects

FIVE PATTERNS IN IDIOMATIC PYTHON main.py

```
# Singleton: a module-level instance (in config.py) -- every importer gets the same object
class _Settings:
    def __init__(self): self.theme = "light"
settings = _Settings()

# Factory: map names to classes
class CsvExporter:
    def export(self, rows): return "\n".join(",".join(map(str, r)) for r in rows)
class JsonExporter:
    def export(self, rows): import json; return json.dumps(rows)
EXPORTERS = {"csv": CsvExporter, "json": JsonExporter}
def make_exporter(kind):
    try:
        return EXPORTERS[kind]()
    except KeyError:
        raise ValueError(f"unknown format {kind!r}") from None

# Strategy: pass the algorithm as a function
def by_marks(s): return -s[1]
def by_name(s): return s[0]
students = [("Ravi", 78), ("Asha", 91), ("Meera", 85)]
def ranked(rows, strategy): return [name for name, _ in sorted(rows, key=strategy)]

# Observer: a list of callbacks
class Course:
    def __init__(self): self._subscribers = []
    def subscribe(self, callback): self._subscribers.append(callback)
    def publish(self, event):
        for callback in self._subscribers:
            callback(event)

# Decorator: add behaviour around a function
import functools
def audited(func):
    @functools.wraps(func)
    def wrapper(*args):
        print(f"  [audit] {func.__name__}{args}")
        return func(*args)
    return wrapper

print(make_exporter("csv").export(students[:2]))
print(make_exporter("json").export(students[:1]))
print(ranked(students, by_marks), ranked(students, by_name))
```

```
course = Course()
course.subscribe(lambda e: print("  email students:", e))
course.subscribe(lambda e: print("  update website:", e))
course.publish("Module 22 notes released")
@audited
def refund(student, amount): return f"refunded Rs {amount} to {student}"
print(refund("Asha", 500))
```

OUTPUT

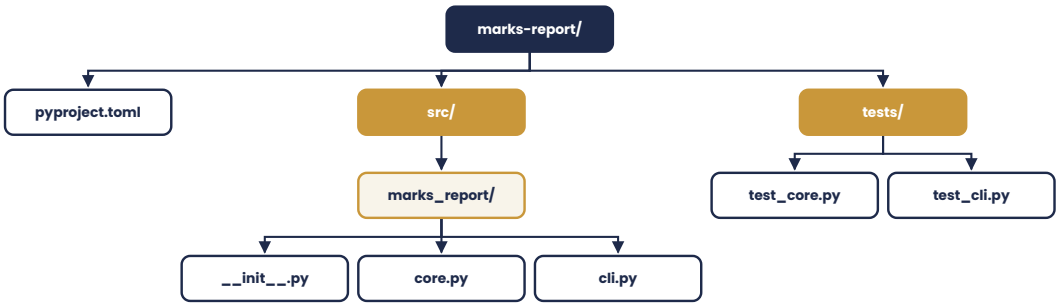
```
Ravi,78
Asha,91
[["Ravi", 78]]
['Asha', 'Meera', 'Ravi'] ['Asha', 'Meera', 'Ravi']
  email students: Module 22 notes released
  update website: Module 22 notes released
  [audit] refund('Asha', 500)
refunded Rs 500 to Asha
```

Debug methodically: reproduce, hypothesise, observe (print with =, logging, pdb via breakpoint() or an IDE debugger), fix, and add a test. Tests catch bugs, enable refactoring and document behaviour: many unit tests, fewer integration and end-to-end tests. unittest uses TestCase classes and assert methods; pytest uses plain functions with assert, fixtures (including tmp_path, capsys, monkeypatch), parametrize and pytest.raises; unittest.mock and patch replace slow or external dependencies; coverage.py/pytest-cov measure tested lines; TDD follows red, green, refactor; doctest checks docstring examples. Follow PEP 8 and enforce it with Ruff (lint and format) or flake8 and Black, check types with mypy, automate everything with pre-commit hooks, and write good docstrings. Package with the src layout and pyproject.toml, build wheels and publish to PyPI, and manage dependencies with uv or Poetry and their lock files. Use Git with branches and pull requests and a Python .gitignore. Apply SOLID, DRY, KISS and YAGNI, and implement patterns the Pythonic way with modules, dictionaries, functions, callbacks and decorators.

PRACTICE CHECKPOINT

Package an earlier project with **pyproject.toml**, add **pytest** tests with at least **80% coverage**, and make it pass **Ruff** and **mypy**.

The marks-report project below uses the src layout. Its quality gate runs the linter, the formatter check, the type checker and the tests. Coverage is measured here by a small `conftest.py` plugin built on the standard library's `sys.settrace`; in your own projects, use `pytest --cov=marks_report --cov-fail-under=80` from pytest-cov.



THE CHECKPOINT PROJECT

PYPROJECT.TOML text

```
[build-system]
requires = ["setuptools>=68"]
build-backend = "setuptools.build_meta"

[project]
name = "marks-report"
version = "1.0.0"
description = "Grade reports for Learn Being Forward batches"
requires-python = ">=3.11"
dependencies = []

[project.scripts]
marks-report = "marks_report.cli:main"

[tool.pytest.ini_options]
testpaths = ["tests"]
pythonpath = ["src"]

[tool.ruff]
line-length = 120

[tool.ruff.lint]
select = ["E", "F", "I", "B", "UP", "N"]

[tool.mypy]
strict = true
mypy_path = "src"
```

SRC/MARKS_REPORT/__INIT__.PY Python

```python
"""Grade reports for student marks."""

from marks_report.core import Student, grade, summarise

__all__ = ["Student", "grade", "summarise"]
```

SRC/MARKS_REPORT/CORE.PY Python

```python
"""Core logic: grades and class summaries."""

from dataclasses import dataclass


@dataclass(frozen=True)
class Student:
    name: str
    marks: tuple[int, ...]

    @property
    def average(self) -> float:
        if not self.marks:
            return 0.0
        return round(sum(self.marks) / len(self.marks), 1)


def grade(mark: float) -> str:
    """Return the letter grade for a mark between 0 and 100."""
    if not 0 <= mark <= 100:
        raise ValueError(f"mark out of range: {mark}")
    for limit, letter in ((90, "A"), (75, "B"), (50, "C")):
        if mark >= limit:
```

```
            return letter
    return "F"


def parse_line(line: str) -> Student:
    """Parse 'Name,91,88,79' into a Student."""
    name, *marks = (part.strip() for part in line.split(","))
    if not name:
        raise ValueError("missing name")
    return Student(name, tuple(int(m) for m in marks))


def summarise(students: list[Student]) -> dict[str, object]:
    """Return the class average, topper and grade counts."""
    if not students:
        return {"students": 0, "class_average": 0.0, "topper": None, "grades": {}}
    grades: dict[str, int] = {}
    for s in students:
        letter = grade(s.average)
        grades[letter] = grades.get(letter, 0) + 1
    topper = max(students, key=lambda s: s.average)
    average = round(sum(s.average for s in students) / len(students), 1)
    return {
        "students": len(students),
        "class_average": average,
        "topper": topper.name,
        "grades": dict(sorted(grades.items())),
    }
```

SRC/MARKS_REPORT/CLI.PY Python

```
"""Command-line entry point: marks-report FILE."""

import sys
from pathlib import Path

from marks_report.core import parse_line, summarise


def main(argv: list[str] | None = None) -> int:
    args = sys.argv[1:] if argv is None else argv
    if len(args) != 1:
        print("usage: marks-report FILE", file=sys.stderr)
        return 2
    lines = Path(args[0]).read_text(encoding="utf-8").splitlines()
    students = [parse_line(line) for line in lines if line.strip()]
    for key, value in summarise(students).items():
        print(f"{key:<14} {value}")
    return 0
```

TESTS/TEST_CORE.PY Python

```python
import pytest

from marks_report import Student, grade, summarise
from marks_report.core import parse_line


@pytest.mark.parametrize("mark, letter", [(100, "A"), (90, "A"), (89.9, "B"), (75, "B"), (50, "C"),
(0, "F")])
def test_grade(mark: float, letter: str) -> None:
    assert grade(mark) == letter


@pytest.mark.parametrize("bad", [-1, 101])
def test_grade_out_of_range(bad: int) -> None:
    with pytest.raises(ValueError):
        grade(bad)


def test_parse_line() -> None:
    assert parse_line(" Asha , 91, 88 ") == Student("Asha", (91, 88))
    with pytest.raises(ValueError):
        parse_line(",90")


def test_average_handles_no_marks() -> None:
    assert Student("New", ()).average == 0.0


def test_summarise() -> None:
    result = summarise([Student("Asha", (91, 95)), Student("Ravi", (60, 70)), Student("Meera", (80,
78))])
    assert result == {"students": 3, "class_average": 79.0, "topper": "Asha", "grades": {"A": 1, "B":
1, "C": 1}}
    assert summarise([])["students"] == 0
```

TESTS/TEST_CLI.PY Python

```python
from pathlib import Path

import pytest

from marks_report.cli import main


def test_cli_report(tmp_path: Path, capsys: pytest.CaptureFixture[str]) -> None:
    data = tmp_path / "marks.csv"
    data.write_text("Asha,91,95\nRavi,60,70\n\n", encoding="utf-8")
    assert main([str(data)]) == 0
    out = capsys.readouterr().out
    assert "topper         Asha" in out


def test_cli_usage(capsys: pytest.CaptureFixture[str]) -> None:
    assert main([]) == 2
    assert "usage" in capsys.readouterr().err
```

THE QUALITY GATE: RUFF, MYPY, PYTEST AND COVERAGE

main.py

```
ruff check src tests && ruff format --check src tests
mypy src
pytest -q -p no:cacheprovider
```

TERMINAL

```
$ ruff check src tests && ruff format --check src tests
All checks passed!
5 files already formatted
$ mypy src
Success: no issues found in 3 source files
$ pytest -q -p no:cacheprovider
.............                                                            [100%]
Name                              Stmts   Miss  Cover
src/marks_report/__init__.py          2      0   100%
src/marks_report/cli.py              13      0   100%
src/marks_report/core.py             30      0   100%
TOTAL                                45      0   100%   (required: 80%)
13 passed in 0.10s
```

MODULE 23 · ADVANCED

Databases, web scraping and web development

Minimum duration **5 days** Level **Advanced**

This module connects Python to databases and the web, the skills used in most backend and automation jobs. It covers SQL and the sqlite3 module, parameterised queries and SQL injection, MySQL and PostgreSQL drivers, the SQLAlchemy ORM, MongoDB, HTTP and web APIs with requests (parameters, JSON, authentication, errors, timeouts and rate limits), web scraping with BeautifulSoup and browser automation, the legal and ethical rules of scraping, how web frameworks work, Flask, FastAPI and Django, building a REST API with CRUD on a database, and automating Excel, PDF and email tasks and scheduling scripts.

IN THIS MODULE

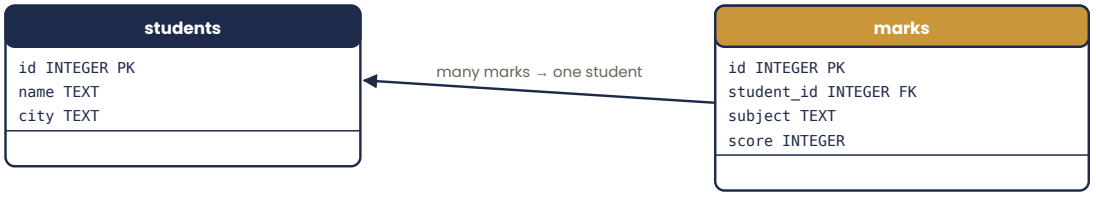
23.1 Databases
23.2 Working with web APIs
23.3 Web scraping
23.4 Web development
23.5 Automation

23.1 Databases

23.1.1 SQL refresher

A **relational database** stores data in **tables** of rows and columns; each table has a **primary key** that identifies rows, and **foreign keys** link tables together. **SQL** (Structured Query Language) is the language for defining tables and querying and changing data. The same SQL works, with small dialect differences, in SQLite, MySQL, PostgreSQL, SQL Server and Oracle.

Statement	Purpose	Example
`CREATE TABLE`	define a table	`CREATE TABLE students (id INTEGER PRIMARY KEY, name TEXT)`
`INSERT`	add rows	`INSERT INTO students (name) VALUES ('Asha')`
`SELECT`	query rows	`SELECT name FROM students WHERE city = 'Mysuru' ORDER BY name`
`UPDATE`	change rows	`UPDATE students SET city = 'Pune' WHERE id = 2`
`DELETE`	remove rows	`DELETE FROM students WHERE id = 3`
`JOIN`	combine related tables	`... FROM marks JOIN students ON marks.student_id = students.id`
`GROUP BY`	aggregate	`SELECT city, COUNT(*), AVG(score) ... GROUP BY city`



TWO RELATED TABLES: EACH MARK ROW REFERS TO A STUDENT THROUGH A FOREIGN KEY

23.1.2 sqlite3: connect, cursor, execute, commit, fetch

SQLite is a complete SQL database stored in a single file (or in memory), built into Python as the `sqlite3` module: no server to install. It is perfect for learning, tests, desktop and mobile apps and small websites. The workflow: `connect()` opens a database and returns a connection; `execute()` runs SQL (on the connection or a **cursor**); `commit()` saves changes made by INSERT/UPDATE/DELETE; `fetchone()`, `fetchall()` and iteration read query results; `close()` closes it. Setting `row_factory = sqlite3.Row` lets you access columns by name.

CREATING, FILLING AND QUERYING A SQLITE DATABASE

main.py

```
import sqlite3

conn = sqlite3.connect("school.db")              # creates the file if needed
conn.row_factory = sqlite3.Row                    # rows behave like dicts
cur = conn.cursor()
cur.executescript("""
    DROP TABLE IF EXISTS marks;
    DROP TABLE IF EXISTS students;
    CREATE TABLE students (id INTEGER PRIMARY KEY, name TEXT NOT NULL, city TEXT);
    CREATE TABLE marks (id INTEGER PRIMARY KEY, student_id INTEGER REFERENCES students(id),
                        subject TEXT, score INTEGER CHECK (score BETWEEN 0 AND 100));
""")
students = [("Asha", "Mysuru"), ("Ravi", "Pune"), ("Meera", "Mysuru"), ("John", "Delhi")]
cur.executemany("INSERT INTO students (name, city) VALUES (?, ?)", students)
marks = [(1, "maths", 91), (1, "science", 88), (2, "maths", 78), (2, "science", 82),
         (3, "maths", 85), (3, "science", 95), (4, "maths", 64)]
cur.executemany("INSERT INTO marks (student_id, subject, score) VALUES (?, ?, ?)", marks)
conn.commit()                                     # save the changes
print("rows in students:", conn.execute("SELECT COUNT(*) FROM students").fetchone()[0],
      "| rows in marks:", conn.execute("SELECT COUNT(*) FROM marks").fetchone()[0])

row = conn.execute("SELECT * FROM students WHERE name = ?", ("Ravi",)).fetchone()
print(dict(row), row["city"])

for r in conn.execute("""SELECT s.name, ROUND(AVG(m.score), 1) AS average, COUNT(*) AS n
                         FROM students s JOIN marks m ON m.student_id = s.id
                         GROUP BY s.id ORDER BY average DESC"""):
    print(f"  {r['name']:<6} {r['average']:>5} over {r['n']} subjects")

conn.execute("UPDATE students SET city = ? WHERE name = ?", ("Bengaluru", "John"))
deleted = conn.execute("DELETE FROM marks WHERE score < ?", (70,)).rowcount
conn.commit()
print("deleted rows:", deleted, "| cities:", [r[0] for r in conn.execute("SELECT DISTINCT city FROM
students ORDER BY city")])
conn.close()
```

OUTPUT

```
rows in students: 4 | rows in marks: 7
{'id': 2, 'name': 'Ravi', 'city': 'Pune'} Pune
  Meera   90.0 over 2 subjects
  Asha    89.5 over 2 subjects
  Ravi    80.0 over 2 subjects
  John    64.0 over 1 subjects
deleted rows: 1 | cities: ['Bengaluru', 'Mysuru', 'Pune']
```

A connection used as a context manager (`with conn:`) commits the transaction automatically if the block succeeds and **rolls it back** if an exception occurs, so a failing group of changes leaves the database untouched. (It does not close the connection.)

TRANSACTIONS: COMMIT ON SUCCESS, ROLL BACK ON ERROR

main.py

```
import sqlite3

conn = sqlite3.connect(":memory:")                # a temporary in-memory database
conn.execute("CREATE TABLE accounts (name TEXT PRIMARY KEY, balance INTEGER CHECK (balance >= 0))")
conn.executemany("INSERT INTO accounts VALUES (?, ?)", [("Asha", 5000), ("Ravi", 1000)])
conn.commit()

def transfer(src, dst, amount):
    try:
        with conn:                                # one transaction: all or nothing
            conn.execute("UPDATE accounts SET balance = balance + ? WHERE name = ?", (amount, dst))
            conn.execute("UPDATE accounts SET balance = balance - ? WHERE name = ?", (amount, src))
        print(f"transferred {amount}")
    except sqlite3.IntegrityError as e:
        print(f"transfer of {amount} rolled back: {e}")

transfer("Asha", "Ravi", 2000)
transfer("Ravi", "Asha", 9000)                    # would make Ravi's balance negative
print(conn.execute("SELECT * FROM accounts").fetchall())
```

OUTPUT

```
transferred 2000
transfer of 9000 rolled back: CHECK constraint failed: balance >= 0
[('Asha', 3000), ('Ravi', 3000)]
```

23.1.3 Parameterised queries and SQL injection

Never build SQL by inserting user input into a string with f-strings, `%` or `+`. A malicious input can change the meaning of the query: this is **SQL injection**, one of the most common and damaging security vulnerabilities. Always use **parameterised queries**: write placeholders (`?` in sqlite3, `%s` in most MySQL/PostgreSQL drivers, or named `:name`) and pass the values separately. The driver sends them as pure data, so they can never be executed as SQL.

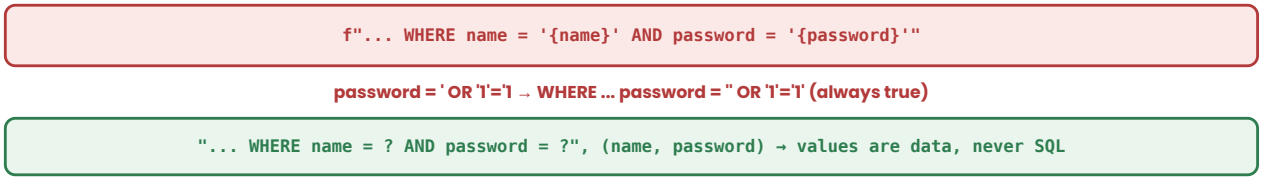
SQL INJECTION AND HOW PARAMETERS PREVENT IT

main.py

```
import sqlite3

conn = sqlite3.connect(":memory:")
conn.execute("CREATE TABLE users (name TEXT, password TEXT, role TEXT)")
conn.executemany("INSERT INTO users VALUES (?, ?, ?)", [("asha", "s3cret!", "admin"), ("ravi",
"pass123", "student")])

def login_unsafe(name, password):
    sql = f"SELECT name, role FROM users WHERE name = '{name}' AND password = '{password}'"
    return conn.execute(sql).fetchall()

def login_safe(name, password):
    sql = "SELECT name, role FROM users WHERE name = ? AND password = ?"
    return conn.execute(sql, (name, password)).fetchall()

attack = "' OR '1'='1"
print("unsafe, wrong password:", login_unsafe("anyone", attack))      # logs in as everyone!
print("safe,   wrong password:", login_safe("anyone", attack))
print("safe,   correct login :", login_safe("ravi", "pass123"))
print("named parameters      :", conn.execute("SELECT role FROM users WHERE name = :n", {"n":
"asha"}).fetchone())
```

OUTPUT

```
unsafe, wrong password: [('asha', 'admin'), ('ravi', 'student')]
safe,   wrong password: []
safe,   correct login : [('ravi', 'student')]
named parameters      : ('admin',)
```



STRING-BUILT SQL CAN BE REWRITTEN BY THE INPUT; PARAMETERS CANNOT

WATCH OUT

Parameters protect **values** only. Table and column names cannot be parameters; if they come from user input, check them against a fixed allow-list.

23.1.4 MySQL / PostgreSQL with connector libraries

Production applications usually use a database **server** such as PostgreSQL or MySQL, accessed through a driver package that follows the same DB-API 2.0 interface (PEP 249) as `sqlite3`: connect, cursor, execute with parameters, commit, fetch. Popular drivers: **psycopg** (version 3) for PostgreSQL and **mysql-connector-python** or **PyMySQL** for MySQL. Connection details (host, user, password) come from environment variables, never from code.

QUERYING POSTGRESQL WITH PSYCOPG

main.py

```
import os
import psycopg                                  # pip install "psycopg[binary]"

with psycopg.connect(
    host=os.environ["DB_HOST"], dbname="school",
    user=os.environ["DB_USER"], password=os.environ["DB_PASSWORD"],
) as conn:
    with conn.cursor() as cur:
        cur.execute("SELECT name, city FROM students WHERE city = %s ORDER BY name", ("Mysuru",))
        for name, city in cur.fetchall():
            print(name, city)
        cur.execute("INSERT INTO students (name, city) VALUES (%s, %s) RETURNING id", ("Zoya",
"Delhi"))
        print("new id:", cur.fetchone()[0])
    # leaving the with block commits the transaction
```

OUTPUT

```
Asha Mysuru
Meera Mysuru
new id: 5
```

Needs a running PostgreSQL server; mysql.connector.connect(host=..., user=..., password=..., database=...) works the same way for MySQL.

	SQLite	PostgreSQL	MySQL / MariaDB
Runs as	a file inside your program	a server	a server
Python driver	`sqlite3` (built in)	`psycopg`	`mysql-connector-python`, `PyMySQL`
Placeholder	`?`	`%s`	`%s`
Best for	learning, tests, small apps, mobile	most web applications, complex queries	web applications, hosting providers

23.1.5 ORMs: SQLAlchemy basics

An **Object-Relational Mapper** (ORM) maps tables to Python **classes** and rows to **objects**, so you work with objects and the ORM generates the SQL for whichever database you use. **SQLAlchemy** is the standard Python ORM (Django has its own). In SQLAlchemy 2.0 you declare models with typed `Mapped[...]` attributes, create an **engine** for the database URL, and use a **Session** to add, query (`select()`), change and delete objects, then commit. Relationships between tables become attributes such as `student.marks`.

MODELS, SESSIONS AND QUERIES WITH SQLALCHEMY 2.0

main.py

```
from sqlalchemy import create_engine, ForeignKey, String, select
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column, relationship, Session

class Base(DeclarativeBase):
    pass

class Student(Base):
    __tablename__ = "students"
    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(String(50))
    city: Mapped[str | None]
    marks: Mapped[list["Mark"]] = relationship(back_populates="student", cascade="all, delete-orphan")

class Mark(Base):
    __tablename__ = "marks"
    id: Mapped[int] = mapped_column(primary_key=True)
    subject: Mapped[str]
    score: Mapped[int]
    student_id: Mapped[int] = mapped_column(ForeignKey("students.id"))
    student: Mapped[Student] = relationship(back_populates="marks")

engine = create_engine("sqlite:///school_orm.db")      # or postgresql+psycopg://user:pw@host/db
Base.metadata.create_all(engine)                       # CREATE TABLE statements generated for you

with Session(engine) as session:
    asha = Student(name="Asha", city="Mysuru", marks=[Mark(subject="maths", score=91),
Mark(subject="science", score=88)])
    session.add_all([asha, Student(name="Ravi", city="Pune", marks=[Mark(subject="maths",
score=78)])])
    session.commit()

    for s in session.scalars(select(Student).where(Student.city == "Mysuru")):
        print(s.name, [(m.subject, m.score) for m in s.marks])
    ravi = session.scalars(select(Student).filter_by(name="Ravi")).one()
    ravi.city = "Bengaluru"                            # UPDATE generated on commit
    session.commit()
    print(session.scalars(select(Student.city).order_by(Student.name)).all())
```

OUTPUT

```
Asha [('maths', 91), ('science', 88)]
['Mysuru', 'Bengaluru']
```

pip install sqlalchemy; the output shown is what this program prints.

Pros of an ORM

Python objects instead of SQL strings, database-independent code, relationships as attributes, safe parameters by default, migrations with Alembic.

Cons

Another layer to learn; complex queries can be harder to express; careless use can issue many queries (the "N+1 problem").

Rule of thumb

Use an ORM for application code; drop to raw SQL (still parameterised) for reports and performance-critical queries.

23.1.6 NoSQL with MongoDB and pymongo (overview)

NoSQL databases store data in models other than tables. **MongoDB**, the most popular **document database**, stores JSON-like documents (Python dictionaries) in **collections**, without a fixed schema, so each document can have different fields and nested data. It suits flexible, evolving, hierarchical data such as product catalogues, user profiles and event logs. The **pymongo** driver maps documents directly

to dicts. Other NoSQL families: key-value stores (Redis), wide-column (Cassandra) and graph databases (Neo4j).

DOCUMENTS, QUERIES AND UPDATES WITH PYMONGO

main.py

```
from pymongo import MongoClient                # pip install pymongo

client = MongoClient("mongodb://localhost:27017")
db = client["lbf"]
students = db["students"]

students.insert_many([
    {"name": "Asha", "city": "Mysuru", "skills": ["python", "sql"], "marks": {"maths": 91}},
    {"name": "Ravi", "city": "Pune", "skills": ["java"], "guardian": {"name": "Kumar"}},
])
for doc in students.find({"skills": "python"}, {"_id": 0, "name": 1, "city": 1}):
    print(doc)
students.update_one({"name": "Ravi"}, {"$push": {"skills": "python"}})
print(students.count_documents({"skills": "python"}))
pipeline = [{"$group": {"_id": "$city", "count": {"$sum": 1}}}]
print(list(students.aggregate(pipeline)))
```

OUTPUT

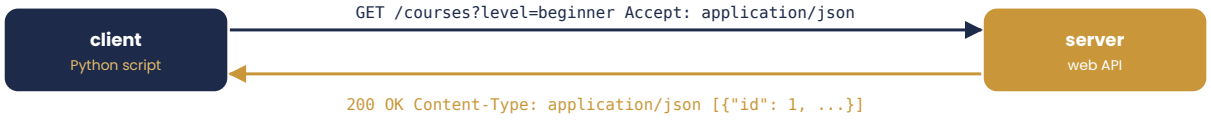
```
{'name': 'Asha', 'city': 'Mysuru'}
2
[{'_id': 'Mysuru', 'count': 1}, {'_id': 'Pune', 'count': 1}]
```

Needs a running MongoDB server (or MongoDB Atlas).

23.2 Working with web APIs

23.2.1 HTTP methods and status codes

Web APIs exchange data over **HTTP**. A **client** sends a **request** (method, URL, headers and optionally a body); the **server** returns a **response** (status code, headers and a body, usually JSON). **REST** APIs model data as **resources** identified by URLs (`/courses`, `/courses/2`) and use the HTTP methods as verbs.



AN HTTP REQUEST AND RESPONSE

Method	Meaning	Typical status on success
`GET`	read a resource or list	200 OK
`POST`	create a new resource	201 Created
`PUT`	replace a resource	200 OK
`PATCH`	update part of a resource	200 OK
`DELETE`	delete a resource	204 No Content

Status	Meaning
2xx (200, 201, 204)	success
3xx (301, 304)	redirect / not modified
400 Bad Request, 422 Unprocessable	the request data is invalid
401 Unauthorized / 403 Forbidden	not logged in / not allowed
404 Not Found	no such resource
429 Too Many Requests	rate limit exceeded
5xx (500, 502, 503)	server error: retry later

23.2.2 requests: GET, POST, headers, params, JSON

The third-party **requests** library (`pip install requests`) is the most popular HTTP client. `requests.get(url, params=..., headers=..., timeout=...)` returns a `Response` with `status_code`, `headers`, `text` and `json()`; `requests.post(url, json=data)` sends a JSON body. A `Session` reuses connections and shared headers across many requests. The examples below talk to a small JSON API that the program starts locally, so they run without internet access; with a real API you only change the base URL.

TEST_API.PY (A SMALL LOCAL JSON API USED BY THE EXAMPLES) Python

```
import json, threading
from http.server import ThreadingHTTPServer, BaseHTTPRequestHandler
from urllib.parse import urlparse, parse_qs

COURSES = [{"id": 1, "title": "Python", "level": "beginner", "fee": 14999},
           {"id": 2, "title": "Data Science", "level": "advanced", "fee": 24999},
           {"id": 3, "title": "Java", "level": "beginner", "fee": 12999}]
CALLS = {"count": 0}

class Api(BaseHTTPRequestHandler):
    def _send(self, status, payload, headers=None):
        body = json.dumps(payload).encode()
        self.send_response(status)
        self.send_header("Content-Type", "application/json")
        for k, v in (headers or {}).items():
            self.send_header(k, v)
        self.send_header("Content-Length", str(len(body)))
        self.end_headers()
        self.wfile.write(body)

    def do_GET(self):
        CALLS["count"] += 1
        url = urlparse(self.path)
        if url.path == "/limited":
            if CALLS["count"] % 3 != 0:
                return self._send(429, {"error": "too many requests"}, {"Retry-After": "0"})
            return self._send(200, {"ok": True, "served_on_call": CALLS["count"]})
        if url.path == "/secure":
            if self.headers.get("Authorization") != "Bearer LBF-TOKEN-123":
                return self._send(401, {"error": "missing or invalid token"})
            return self._send(200, {"user": "asha", "role": "student"})
        if url.path == "/courses":
            level = parse_qs(url.query).get("level", [None])[0]
            return self._send(200, [c for c in COURSES if level in (None, c["level"])])
        if url.path.startswith("/courses/"):
```

```
            cid = int(url.path.rsplit("/", 1)[1])
            match = [c for c in COURSES if c["id"] == cid]
            return self._send(200, match[0]) if match else self._send(404, {"error": "not found"})
        if url.path == "/slow":
            import time; time.sleep(2)
            return self._send(200, {"ok": True})
        self._send(404, {"error": "unknown endpoint"})

    def do_POST(self):
        length = int(self.headers.get("Content-Length", 0))
        data = json.loads(self.rfile.read(length) or b"{}")
        if not data.get("title"):
            return self._send(422, {"error": "title is required"})
        new = {"id": len(COURSES) + 1, **data}
        COURSES.append(new)
        self._send(201, new, {"Location": f"/courses/{new['id']}"})

    def log_message(self, *args):
        pass

class QuietServer(ThreadingHTTPServer):
    def handle_error(self, request, client_address):
        pass                                    # a client gave up (timeout): ignore

def start_api():
    server = QuietServer(("127.0.0.1", 0), Api)
    threading.Thread(target=server.serve_forever, daemon=True).start()
    return f"http://127.0.0.1:{server.server_address[1]}"
```

GET WITH PARAMETERS, POST WITH JSON, AND A SESSION (LOCAL TEST API)

main.py

```
from test_api import start_api

import requests

BASE = start_api()

r = requests.get(f"{BASE}/courses", params={"level": "beginner"}, timeout=5)
print(r.status_code, r.headers["Content-Type"], r.url.split("/", 3)[-1])
for course in r.json():                                  # JSON body -> Python list of dicts
    print(f"  {course['id']}: {course['title']:<8} Rs {course['fee']:,}")

r = requests.get(f"{BASE}/courses/99", timeout=5)
print(r.status_code, r.ok, r.json())

new = {"title": "TypeScript", "level": "intermediate", "fee": 15999}
r = requests.post(f"{BASE}/courses", json=new, timeout=5)  # sends a JSON body
print(r.status_code, r.headers["Location"], r.json()["id"])

with requests.Session() as session:                       # reuse connections and headers
    session.headers.update({"Accept": "application/json", "User-Agent": "lbf-notes/1.0"})
    titles = [session.get(f"{BASE}/courses/{i}", timeout=5).json()["title"] for i in (1, 2, 4)]
print(titles)
```

OUTPUT

```
200 application/json courses?level=beginner
  1: Python   Rs 14,999
  3: Java     Rs 12,999
404 False {'error': 'not found'}
201 /courses/4 4
['Python', 'Data Science', 'TypeScript']
```

23.2.3 Authentication and API keys

Most APIs require authentication. Common schemes: an **API key** in a header (`X-API-Key: ...`) or query parameter; a **bearer token** (`Authorization: Bearer <token>`), often obtained through **OAuth 2.0**; and **HTTP Basic** authentication (`auth=("user", "pass")`). Keep keys out of source code: read them from environment variables or a secrets manager, and never commit them to Git.

SENDING A BEARER TOKEN FROM AN ENVIRONMENT VARIABLE main.py

```
from test_api import start_api

import os, requests

BASE = start_api()
os.environ["LBF_API_TOKEN"] = "LBF-TOKEN-123"            # normally set outside the program

r = requests.get(f"{BASE}/secure", timeout=5)
print("without a token:", r.status_code, r.json())

token = os.environ["LBF_API_TOKEN"]
r = requests.get(f"{BASE}/secure", headers={"Authorization": f"Bearer {token}"}, timeout=5)
print("with a token:   ", r.status_code, r.json())
```

OUTPUT

```
without a token: 401 {'error': 'missing or invalid token'}
with a token:    200 {'user': 'asha', 'role': 'student'}
```

23.2.4 Handling errors, timeouts and rate limits

Network calls fail in many ways, so robust clients: always pass a `timeout` (requests waits forever by default); call `raise_for_status()` to turn 4xx/5xx responses into exceptions; catch `requests.RequestException` (the base class of `ConnectionError`, `Timeout` and `HTTPError`); **retry** transient failures (429 and 5xx) with exponential back-off, honouring a `Retry-After` header; and respect the API's rate limits.

TIMEOUTS, RAISE_FOR_STATUS AND RETRIES WITH BACK-OFF main.py

```
from test_api import start_api

import time, requests

BASE = start_api()

def get_json(url, retries=4, backoff=0.1):
    for attempt in range(1, retries + 1):
        try:
            r = requests.get(url, timeout=1)
            if r.status_code == 429 or r.status_code >= 500:          # transient: retry
                wait = float(r.headers.get("Retry-After", backoff * 2 ** attempt))
                print(f"  attempt {attempt}: {r.status_code}, retrying after {wait:.1f} s")
                time.sleep(wait)
                continue
            r.raise_for_status()                                       # other 4xx: give up
            return r.json()
        except requests.Timeout:
            print(f"  attempt {attempt}: timed out")
    raise RuntimeError(f"giving up on {url} after {retries} attempts")

print(get_json(f"{BASE}/limited"))
for path in ("/courses/42", "/slow"):
    try:
        get_json(f"{BASE}{path}", retries=2)
    except requests.HTTPError as e:
        print("HTTPError:", e.response.status_code, e.response.json())
    except RuntimeError as e:
        print("RuntimeError:", e)
try:
    requests.get("http://127.0.0.1:1/", timeout=1)
except requests.ConnectionError:
    print("ConnectionError: nothing is listening on that port")
```

OUTPUT

```
  attempt 1: 429, retrying after 0.0 s
  attempt 2: 429, retrying after 0.0 s
{'ok': True, 'served_on_call': 3}
HTTPError: 404 {'error': 'not found'}
  attempt 1: timed out
  attempt 2: timed out
RuntimeError: giving up on http://127.0.0.1:46207/slow after 2 attempts
ConnectionError: nothing is listening on that port
```

23.3 Web scraping

Web scraping extracts data from web pages that offer no API: prices, job listings, timetables, public records. The process: download the HTML (requests or a browser), parse it into a tree, select the elements you need, clean the text and save it (CSV, database). Always check first whether an official API or data download exists, and scrape responsibly (23.3.4).

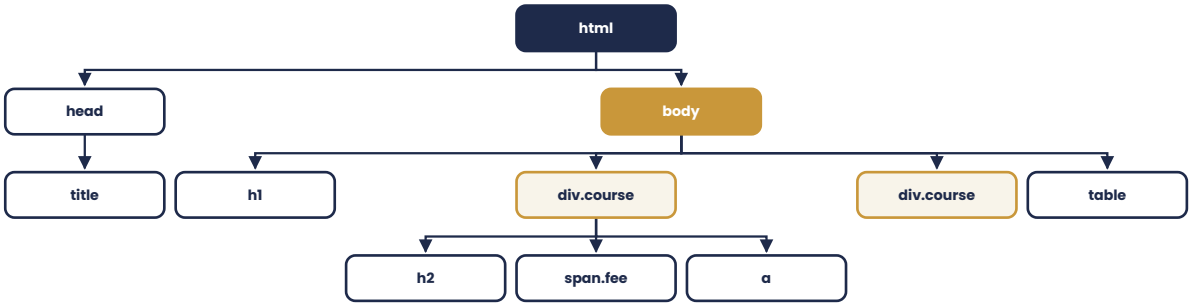
23.3.1 HTML basics

HTML describes a page as a tree of **elements** written with tags: `<tag attribute="value">content</tag>`. Elements nest inside each other. The attributes `id` (unique on the page) and `class` (shared by similar elements) are what scrapers use to find data, together with CSS selectors such as `div.course > h2`. Your browser's **Developer Tools** (F12, "Inspect") show the tree of any page.

COURSES.HTML

HTML

```
<html>
<head><title>LBF Courses</title></head>
<body>
  <h1 id="main-title">Our courses</h1>
  <div class="course" data-id="1">
    <h2>Python Programming</h2>
    <span class="fee">Rs 14,999</span> <span class="level">Beginner</span>
    <a href="/courses/python">Details</a>
  </div>
  <div class="course" data-id="2">
    <h2>Data Science</h2>
    <span class="fee">Rs 24,999</span> <span class="level">Advanced</span>
    <a href="/courses/data-science">Details</a>
  </div>
  <div class="course featured" data-id="3">
    <h2>Java Full Stack</h2>
    <span class="fee">Rs 19,999</span> <span class="level">Intermediate</span>
    <a href="/courses/java">Details</a>
  </div>
  <table id="batches">
    <tr><th>Batch</th><th>Starts</th><th>Seats</th></tr>
    <tr><td>PY-101</td><td>2026-02-02</td><td>12</td></tr>
    <tr><td>DS-201</td><td>2026-02-16</td><td>5</td></tr>
  </table>
</body>
</html>
```



THE PAGE AS A TREE OF ELEMENTS

23.3.2 BeautifulSoup

BeautifulSoup (`pip install beautifulsoup4`, imported as `bs4`) parses HTML into a navigable tree, even when the HTML is messy. `soup.find(tag, class_=..., id=...)` returns the first match and `find_all` all matches; `select(css)` and `select_one(css)` use CSS selectors. From an element, `.get_text(strip=True)` gives its text, `["href"]` or `.get("href")` an attribute, and `.parent`, `.find_next_sibling()` navigate the tree.

EXTRACTING STRUCTURED DATA WITH BEAUTIFULSOUP

main.py

```
from bs4 import BeautifulSoup
import csv, io

html = open("courses.html", encoding="utf-8").read()
soup = BeautifulSoup(html, "html.parser")

print(soup.title.string, "|", soup.find("h1", id="main-title").get_text())
courses = []
for div in soup.find_all("div", class_="course"):
    courses.append({
        "id": int(div["data-id"]),
        "title": div.h2.get_text(strip=True),
        "fee": int(div.select_one(".fee").get_text().replace("Rs", "").replace(",", "")),
        "level": div.find("span", class_="level").text,
        "url": "https://learnbeingforward.in" + div.a["href"],
        "featured": "featured" in div["class"],
    })
for c in courses:
    print(c)

print([h2.text for h2 in soup.select("div.course.featured > h2")])
rows = [[td.get_text() for td in tr.find_all("td")] for tr in soup.select("#batches tr")[1:]]
print(rows)

buffer = io.StringIO()
writer = csv.DictWriter(buffer, fieldnames=courses[0].keys())
writer.writeheader(); writer.writerows(courses)
print(buffer.getvalue().splitlines()[:2])
```

OUTPUT

```
LBF Courses | Our courses
{'id': 1, 'title': 'Python Programming', 'fee': 14999, 'level': 'Beginner', 'url':
'https://learnbeingforward.in/courses/python', 'featured': False}
{'id': 2, 'title': 'Data Science', 'fee': 24999, 'level': 'Advanced', 'url':
'https://learnbeingforward.in/courses/data-science', 'featured': False}
{'id': 3, 'title': 'Java Full Stack', 'fee': 19999, 'level': 'Intermediate', 'url':
'https://learnbeingforward.in/courses/java', 'featured': True}
['Java Full Stack']
[['PY-101', '2026-02-02', '12'], ['DS-201', '2026-02-16', '5']]
['id,title,fee,level,url,featured', '1,Python
Programming,14999,Beginner,https://learnbeingforward.in/courses/python,False']
```

In real scraping, download the page with `requests.get(url, headers={"User-Agent": ...}, timeout=10)` and pass `response.text` to BeautifulSoup. Follow pagination links in a loop with a polite delay, and handle missing elements (`find` returns `None`).

23.3.3 Selenium or Playwright for dynamic pages

Many modern sites build their content with JavaScript after the page loads, so the HTML returned by `requests` is nearly empty. **Browser automation** tools drive a real browser that runs the JavaScript: **Playwright** (`pip install playwright`, then `playwright install`) and the older **Selenium**. They can click, type, scroll, wait for elements and take screenshots, which also makes them the standard tools for end-to-end testing. Before automating a browser, check the browser's Network tab: the data often comes from a JSON API you can call directly with requests, which is far faster.

SCRAPING A JAVASCRIPT-RENDERED PAGE WITH PLAYWRIGHT

main.py

```
from playwright.sync_api import sync_playwright

with sync_playwright() as p:
    browser = p.chromium.launch(headless=True)
    page = browser.new_page()
    page.goto("https://quotes.toscrape.com/js/")            # content rendered by JavaScript
    page.wait_for_selector("div.quote")
    for quote in page.locator("div.quote").all()[:3]:
        text = quote.locator("span.text").inner_text()
        author = quote.locator("small.author").inner_text()
        print(f"{author}: {text[:50]}...")
    page.click("li.next > a")                                # go to the next page
    page.screenshot(path="page2.png")
    browser.close()
```

OUTPUT

```
Albert Einstein: “The world as we have created it is a process of o...
J.K. Rowling: “It is our choices, Harry, that show what we truly a...
Albert Einstein: “There are only two ways to live your life. One is...
```

Needs internet access and a browser installed with "playwright install chromium".

23.3.4 Legal and ethical scraping (robots.txt, terms of service)

Scraping is legal in many situations, but not all, and it always affects someone else's servers. Follow these rules:

Prefer APIs

Use an official API or open data download if one exists.

robots.txt

Read /robots.txt and obey its Disallow rules and crawl delay (urllib.robotparser checks it).

Terms of service

Many sites forbid scraping in their terms; respect them, especially behind a login.

Be gentle

Rate-limit requests (1 per second or slower), cache pages, scrape off-peak, identify yourself in the User-Agent.

Personal data

Do not collect personal information without a lawful basis (India's DPDP Act 2023, GDPR in Europe).

Copyright

Facts can usually be used; copying articles, images or databases wholesale may infringe copyright.

CHECKING ROBOTS.TXT RULES WITH URLLIB.ROBOTPARSER

main.py

```
from urllib.robotparser import RobotFileParser

robots_txt = """
User-agent: *
Disallow: /admin/
Disallow: /students/
Crawl-delay: 2

User-agent: BadBot
Disallow: /
"""
rp = RobotFileParser()
rp.parse(robots_txt.splitlines())
for agent, path in [("lbf-notes", "/courses/python"), ("lbf-notes", "/students/101"), ("BadBot",
"/courses/python")]:
    print(f"{agent:<10} {path:<16} allowed: {rp.can_fetch(agent, path)}")
print("crawl delay:", rp.crawl_delay("lbf-notes"), "seconds")
```

OUTPUT

```
lbf-notes  /courses/python  allowed: True
lbf-notes  /students/101    allowed: False
BadBot     /courses/python  allowed: False
crawl delay: 2 seconds
```

23.4 Web development

23.4.1 How web frameworks work (routing, requests, responses)

A **web framework** handles the repetitive parts of a web application so you write only the logic. A web server receives HTTP requests and passes them to the framework through a standard interface (**WSGI** for synchronous frameworks such as Flask and Django, **ASGI** for asynchronous ones such as FastAPI). The framework's **router** maps the URL and method to a **view function** (handler); the view reads the request (path parameters, query string, form or JSON body, headers, cookies), talks to the database, and returns a **response**: HTML rendered from a **template**, or JSON for an API. **Middleware** wraps every request for cross-cutting concerns such as authentication, sessions and logging.

THE PATH OF A REQUEST THROUGH A PYTHON WEB APPLICATION

Framework	Style	Strengths	Typical use
Flask	micro-framework (WSGI)	small, flexible, easy to learn, many extensions	small sites, APIs, prototypes
FastAPI	modern API framework (ASGI)	type hints, Pydantic validation, async, automatic docs, very fast	REST APIs, ML model serving
Django	"batteries included" (WSGI/ASGI)	ORM, admin, auth, forms, migrations, security built in	large content sites, business apps

23.4.2 Flask: routes, templates (Jinja2), forms

In **Flask** (`pip install flask`), you create an application object and attach view functions to URLs with the `@app.route(path, methods=[...])` decorator. Path parameters such as `/courses/<int:course_id>` become function arguments; `request.args`, `request.form` and `request.get_json()` hold query, form and JSON data. Views return strings, rendered templates, dictionaries (converted to JSON) or tuples with a status code. `render_template` fills **Jinja2** templates, which support `{{ variable }}`, `{% for %}`, `{% if %}`, filters and template inheritance; Jinja2 escapes HTML automatically, which prevents cross-site scripting. Flask's **test client** sends requests to the app without a server, which is how the examples below run.

TEMPLATES/BASE.HTML HTML

```
<!doctype html>
<title>{% block title %}LBF{% endblock %}</title>
<h1>Learn Being Forward</h1>
{% block content %}{% endblock %}
```

TEMPLATES/COURSES.HTML HTML

```
{% extends "base.html" %}
{% block title %}Courses{% endblock %}
{% block content %}
<ul>
{% for c in courses %}
  <li>{{ c.title }} - Rs {{ "{:,}".format(c.fee) }}{% if c.fee > 20000 %} (premium){% endif %}</li>
{% endfor %}
</ul>
<p>Search: {{ query }}</p>
{% endblock %}
```

A FLASK APP WITH ROUTES, A JINJA2 TEMPLATE AND A FORM main.py

```
from flask import Flask, render_template, request, redirect, url_for, abort, jsonify

app = Flask(__name__)
COURSES = [{"id": 1, "title": "Python", "fee": 14999}, {"id": 2, "title": "Data Science", "fee":
24999}]
ENQUIRIES = []

@app.route("/")
def home():
    return "<h1>Welcome to LBF</h1>"

@app.route("/courses")
def courses():
    query = request.args.get("q", "")
    found = [c for c in COURSES if query.lower() in c["title"].lower()]
    return render_template("courses.html", courses=found, query=query)

@app.route("/courses/<int:course_id>")
def course_detail(course_id):
    for c in COURSES:
        if c["id"] == course_id:
            return jsonify(c)
    abort(404)

@app.route("/enquire", methods=["GET", "POST"])
def enquire():
    if request.method == "POST":
        name = request.form.get("name", "").strip()
        if not name:
```

```
            return "name is required", 400
        ENQUIRIES.append(name)
        return redirect(url_for("thanks", name=name))
    return '<form method="post"><input name="name"><button>Send</button></form>'

@app.route("/thanks")
def thanks():
    return f"Thank you, {request.args['name']}!"

client = app.test_client()                          # sends requests without a server
print(client.get("/").status_code, client.get("/").text)
r = client.get("/courses?q=<script>")
print(r.status_code, "escaped:", "&lt;script&gt;" in r.text)
print(client.get("/courses").text.split("<ul>")[1].split("</ul>")[0].strip())
print(client.get("/courses/2").json, client.get("/courses/9").status_code)
r = client.post("/enquire", data={"name": "Asha"})
print(r.status_code, r.headers["Location"], "->", client.get(r.headers["Location"]).text)
print(client.post("/enquire", data={"name": ""}).status_code, ENQUIRIES)
```

```
OUTPUT
200 <h1>Welcome to LBF</h1>
200 escaped: True
<li>Python - Rs 14,999</li>

  <li>Data Science - Rs 24,999 (premium)</li>
{'fee': 24999, 'id': 2, 'title': 'Data Science'} 404
302 /thanks?name=Asha -> Thank you, Asha!
400 ['Asha']
```

During development, run the app with `flask --app app run --debug` (automatic reloading and an interactive error page) and open `http://127.0.0.1:5000`. In production, run it behind a WSGI server such as **gunicorn** (`gunicorn app:app`), never with the debug server.

23.4.3 FastAPI: path operations, Pydantic models, automatic docs

FastAPI (`pip install "fastapi[standard]"`) builds APIs from type-annotated functions. A **path operation** is a function decorated with `@app.get`, `@app.post`, `@app.put` or `@app.delete`; path and query parameters are declared as typed arguments and validated automatically. Request and response bodies are **Pydantic models** (19.3), so invalid input gets a detailed 422 error without any code. FastAPI generates interactive **OpenAPI documentation** at `/docs` (Swagger UI) and `/redoc`, supports `async def` handlers and **dependency injection** with `Depends` for databases and authentication. Run it with `fastapi dev main.py` (or `uvicorn main:app --reload`).

A FASTAPI APP WITH PYDANTIC MODELS

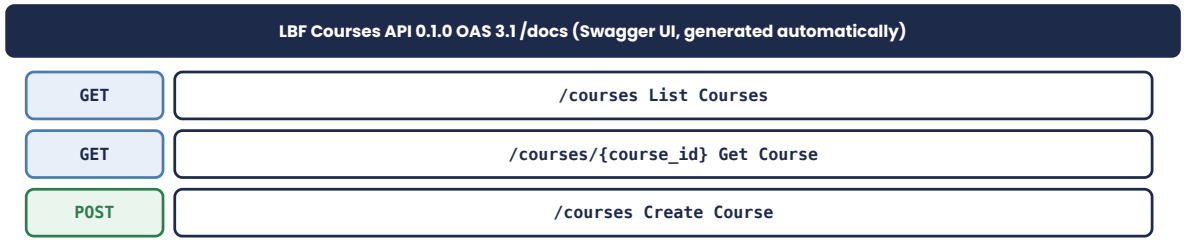
main.py

```
from fastapi import FastAPI, HTTPException, Query
from pydantic import BaseModel, Field

app = FastAPI(title="LBF Courses API")

class CourseIn(BaseModel):
    title: str = Field(min_length=2)
    level: str = "beginner"
    fee: int = Field(gt=0)

class Course(CourseIn):
    id: int

COURSES: dict[int, Course] = {1: Course(id=1, title="Python", fee=14999)}

@app.get("/courses")
def list_courses(level: str | None = None, max_fee: int = Query(100_000, gt=0)) -> list[Course]:
    return [c for c in COURSES.values() if (level is None or c.level == level) and c.fee <= max_fee]

@app.get("/courses/{course_id}")
def get_course(course_id: int) -> Course:
    if course_id not in COURSES:
        raise HTTPException(status_code=404, detail="course not found")
    return COURSES[course_id]

@app.post("/courses", status_code=201)
def create_course(data: CourseIn) -> Course:
    course = Course(id=max(COURSES) + 1, **data.model_dump())
    COURSES[course.id] = course
    return course

# --- try it with the built-in test client ---
from fastapi.testclient import TestClient
client = TestClient(app)
print(client.get("/courses").json())
print(client.post("/courses", json={"title": "Data Science", "fee": 24999, "level":
"advanced"}).json())
r = client.post("/courses", json={"title": "X", "fee": -5})
print(r.status_code, [e["loc"][-1] for e in r.json()["detail"]])
print(client.get("/courses/7").status_code, client.get("/courses", params={"max_fee": 20000}).json()
[0]["title"])
```

OUTPUT

```
[{'title': 'Python', 'level': 'beginner', 'fee': 14999, 'id': 1}]
{'title': 'Data Science', 'level': 'advanced', 'fee': 24999, 'id': 2}
422 ['title', 'fee']
404 Python
```

pip install "fastapi[standard]"; then "fastapi dev main.py" serves interactive docs at http://127.0.0.1:8000/docs.



FASTAPI TURNS THE PATH OPERATIONS INTO INTERACTIVE DOCUMENTATION

23.4.4 Django overview: MVT, ORM, admin

Django (`pip install django`) is a full-stack framework that follows the **Model-View-Template** (MVT) pattern. **Models** are classes that define database tables through Django's built-in ORM; **views** are functions or classes that handle requests; **templates** render HTML; a `urls.py` maps URLs to views. It includes **migrations** (database schema changes generated from model changes), an automatic **admin** site for managing data, user authentication, forms, sessions and strong security defaults (CSRF protection, SQL-injection-safe ORM, XSS escaping). **Django REST Framework** adds API building.

DJANGO'S MAIN PARTS

THE SHAPE OF A DJANGO PROJECT main.py

```
# shell
django-admin startproject lbfsite && cd lbfsite
python manage.py startapp courses

# courses/models.py
from django.db import models

class Course(models.Model):
    title = models.CharField(max_length=100)
    fee = models.PositiveIntegerField()
    level = models.CharField(max_length=20, choices=[("b", "Beginner"), ("a", "Advanced")])
    def __str__(self):
        return self.title

# courses/views.py
from django.shortcuts import render
from .models import Course

def course_list(request):
    return render(request, "courses/list.html", {"courses": Course.objects.order_by("fee")})

# courses/admin.py
from django.contrib import admin
admin.site.register(Course)          # full create/edit/delete screens for free

# shell
python manage.py makemigrations && python manage.py migrate
python manage.py createsuperuser
python manage.py runserver           # http://127.0.0.1:8000/admin/
```

23.4.5 Building a REST API with CRUD on a database

A **CRUD** API exposes Create, Read, Update and Delete operations on a resource, mapped to HTTP methods: `POST /students`, `GET /students` and `GET /students/<id>`, `PUT` / `PATCH /students/<id>`, and `DELETE /students/<id>`. Good APIs validate input and return correct status codes (201 with a `Location` header, 404, 422, 204), use parameterised queries, return JSON consistently and are covered by tests. The practice checkpoint at the end of this module builds a complete one with Flask and SQLite.

Operation	Method and URL	Success	Errors
Create	`POST /students`	201 + new object	422 invalid data

Operation	Method and URL	Success	Errors
Read all	`GET /students?city=Mysuru`	200 + list	
Read one	`GET /students/7`	200 + object	404
Update	`PUT /students/7`	200 + object	404, 422
Delete	`DELETE /students/7`	204, no body	404

23.5 Automation

23.5.1 Automating Excel, PDF and email tasks

Python excels at automating repetitive office work. **Excel**: `openpyxl` reads and writes workbooks with formulas and styles (14.7.4), and pandas reads and writes sheets in one line. **PDF**: `pypdf` merges, splits and extracts text; `reportlab` or `fpdf2` create PDFs; `pdfplumber` extracts tables. **Email**: the standard library builds messages with `email.message.EmailMessage` (text, HTML and attachments) and sends them with `smtplib` over TLS, using an app password stored in an environment variable.

GENERATING AN EXCEL REPORT WITH FORMULAS, STYLES AND A CHART main.py

```
from openpyxl import Workbook
from openpyxl.styles import Font, PatternFill
from openpyxl.chart import BarChart, Reference
import random

random.seed(23)
wb = Workbook()
ws = wb.active
ws.title = "January sales"
ws.append(["Branch", "Python", "Java", "Data Science", "Total"])
for branch in ["Mysuru", "Bengaluru", "Pune", "Delhi"]:
    row = [branch] + [random.randint(10, 60) * 1000 for _ in range(3)]
    ws.append(row + [f"=SUM(B{ws.max_row + 1}:D{ws.max_row + 1})"])
for cell in ws[1]:
    cell.font = Font(bold=True, color="FFFFFF")
    cell.fill = PatternFill("solid", fgColor="1E2A4A")
ws.column_dimensions["A"].width = 14
chart = BarChart(); chart.title = "Enrolment revenue by branch"
chart.add_data(Reference(ws, min_col=2, max_col=4, min_row=1, max_row=5), titles_from_data=True)
chart.set_categories(Reference(ws, min_col=1, min_row=2, max_row=5))
ws.add_chart(chart, "G2")
wb.save("sales_report.xlsx")

from openpyxl import load_workbook
check = load_workbook("sales_report.xlsx")["January sales"]
for row in check.iter_rows(min_row=1, max_row=3, values_only=True):
    print(row)
print("charts in sheet:", len(check._charts) if hasattr(check, "_charts") else "saved")
```

OUTPUT

```
('Branch', 'Python', 'Java', 'Data Science', 'Total')
('Mysuru', 59000, 28000, 15000, '=SUM(B2:D2)')
('Bengaluru', 11000, 47000, 29000, '=SUM(B3:D3)')
charts in sheet: 1
```

BUILDING AN EMAIL WITH HTML AND AN ATTACHMENT

main.py

```
from email.message import EmailMessage
import smtplib, os

def build_report_email(to, branch, total, attachment_bytes):
    msg = EmailMessage()
    msg["Subject"] = f"Monthly report: {branch}"
    msg["From"] = "reports@learnbeingforward.in"
    msg["To"] = to
    msg.set_content(f"Hello,\n\nTotal revenue for {branch}: Rs {total:,}.\nThe report is
attached.\n\nLBF Reports Bot")
    msg.add_alternative(f"<p>Total revenue for <b>{branch}</b>: Rs {total:,}</p>", subtype="html")
    msg.add_attachment(attachment_bytes, maintype="application",
                       subtype="vnd.openxmlformats-officedocument.spreadsheetml.sheet",
filename="sales_report.xlsx")
    return msg

def send(msg):
    with smtplib.SMTP("smtp.gmail.com", 587, timeout=10) as server:
        server.starttls()                                    # encrypt the connection
        server.login(os.environ["SMTP_USER"], os.environ["SMTP_APP_PASSWORD"])
        server.send_message(msg)

msg = build_report_email("manager@lbf.in", "Mysuru", 128000, b"PK...xlsx bytes...")
print(msg["Subject"], "|", msg["To"])
print([part.get_content_type() for part in msg.walk()])
print("attachment:", [p.get_filename() for p in msg.iter_attachments()])
# send(msg)   # needs SMTP credentials in environment variables
```

OUTPUT

```
Monthly report: Mysuru | manager@lbf.in
['multipart/mixed', 'multipart/alternative', 'text/plain', 'text/html',
'application/vnd.openxmlformats-officedocument.spreadsheetml.sheet']
attachment: ['sales_report.xlsx']
```

23.5.2 Scheduling scripts

Automation scripts usually run on a **schedule**. The operating system's scheduler is the most reliable choice: **cron** on Linux and macOS (edit with `crontab -e`) and **Task Scheduler** on Windows. In the cloud, GitHub Actions `schedule`, serverless functions and container schedulers do the same job. Inside a long-running Python process, the standard library's `sched` module or the third-party `schedule` and **APScheduler** libraries run functions at intervals. Scheduled scripts should log to a file, use absolute paths and handle errors, because no one is watching when they run.

cron expression	Runs
`0 9 * * 1-5`	at 09:00 every weekday
`*/15 * * * *`	every 15 minutes
`30 23 * * *`	every day at 23:30
`0 6 1 * *`	at 06:00 on the 1st of every month

A CRONTAB LINE THAT RUNS A REPORT EVERY WEEKDAY MORNING

main.py

```
# crontab -e   (minute hour day-of-month month day-of-week  command)
0 9 * * 1-5  cd /home/lbf/reports && /home/lbf/reports/.venv/bin/python send_report.py >> report.log
2>&1
```

RUNNING FUNCTIONS ON A SCHEDULE WITH SCHED

main.py

```
import sched, time

scheduler = sched.scheduler(time.monotonic, time.sleep)
start = time.monotonic()

def job(name):
    print(f"  {time.monotonic() - start:.1f} s: running {name}")

def every(interval, name, times):
    job(name)
    if times > 1:
        scheduler.enter(interval, 1, every, (interval, name, times - 1))

scheduler.enter(0.2, 1, every, (0.3, "backup", 3))       # first run after 0.2 s, then every 0.3 s
scheduler.enter(0.5, 2, job, ("send report",))
scheduler.run()                                            # blocks until all events are done
print("all scheduled jobs finished")
```

OUTPUT

```
  0.2 s: running backup
  0.5 s: running send report
  0.5 s: running backup
  0.8 s: running backup
all scheduled jobs finished
```

Relational databases store tables linked by keys and are queried with SQL. sqlite3 is built in: connect, execute (executemany, executescript), commit, fetch, Row factories, and with conn: transactions that roll back on errors. Always use parameterised queries (? or %s) to prevent SQL injection. psycopg and mysql-connector follow the same DB-API for PostgreSQL and MySQL; SQLAlchemy maps classes to tables with sessions and select(); MongoDB stores JSON-like documents (pymongo). Web APIs use HTTP methods and status codes; requests sends GET and POST with params, headers and JSON, authenticates with keys or bearer tokens from environment variables, and needs timeouts, raise_for_status and retries with back-off. BeautifulSoup parses HTML with find, find_all and CSS selectors; Playwright or Selenium handle JavaScript pages; respect robots.txt, terms and privacy laws. Web frameworks route requests to views: Flask (routes, Jinja2, forms, test client), FastAPI (typed path operations, Pydantic, automatic docs), Django (MVT, ORM, admin). REST APIs map CRUD to POST, GET, PUT/PATCH and DELETE with correct status codes. Python automates Excel (openpyxl), PDF and email (email, smtplib), and scripts run on schedules with cron, Task Scheduler or sched/APScheduler.

PRACTICE CHECKPOINT

Build a **Flask REST API** for a student database with full **CRUD** backed by **SQLite**, plus a script that **consumes** an API.

APP.PY

Python

```
"""Students REST API: Flask + SQLite with full CRUD."""
import sqlite3
```

```
from flask import Flask, g, jsonify, request

DATABASE = "students.db"
app = Flask(__name__)


def db():
    if "db" not in g:
        g.db = sqlite3.connect(DATABASE)
        g.db.row_factory = sqlite3.Row
    return g.db


@app.teardown_appcontext
def close_db(exc):
    conn = g.pop("db", None)
    if conn is not None:
        conn.close()


def init_db():
    with sqlite3.connect(DATABASE) as conn:
        conn.executescript("""
            DROP TABLE IF EXISTS students;
            CREATE TABLE students (id INTEGER PRIMARY KEY AUTOINCREMENT,
                                   name TEXT NOT NULL, city TEXT, marks INTEGER);
        """)


def validate(data, partial=False):
    errors = {}
    if not partial or "name" in data:
        if not isinstance(data.get("name"), str) or len(data["name"].strip()) < 2:
            errors["name"] = "name must be at least 2 characters"
    if "marks" in data and not (isinstance(data["marks"], int) and 0 <= data["marks"] <= 100):
        errors["marks"] = "marks must be an integer from 0 to 100"
    return errors


def find(student_id):
    return db().execute("SELECT * FROM students WHERE id = ?", (student_id,)).fetchone()


@app.post("/students")
def create_student():
    data = request.get_json(silent=True) or {}
    if errors := validate(data):
        return jsonify(errors=errors), 422
    cur = db().execute("INSERT INTO students (name, city, marks) VALUES (?, ?, ?)",
                       (data["name"].strip(), data.get("city"), data.get("marks")))
    db().commit()
    student = dict(find(cur.lastrowid))
    return jsonify(student), 201, {"Location": f"/students/{student['id']}"}


@app.get("/students")
def list_students():
    sql, params = "SELECT * FROM students", []
    if city := request.args.get("city"):
        sql += " WHERE city = ?"
        params.append(city)
    sql += " ORDER BY marks DESC" if request.args.get("sort") == "marks" else " ORDER BY id"
    return jsonify([dict(r) for r in db().execute(sql, params)])
```

```python
@app.get("/students/<int:student_id>")
def get_student(student_id):
    row = find(student_id)
    return (jsonify(dict(row)), 200) if row else (jsonify(error="student not found"), 404)


@app.route("/students/<int:student_id>", methods=["PUT", "PATCH"])
def update_student(student_id):
    if find(student_id) is None:
        return jsonify(error="student not found"), 404
    data = request.get_json(silent=True) or {}
    if errors := validate(data, partial=(request.method == "PATCH")):
        return jsonify(errors=errors), 422
    fields = {k: data[k] for k in ("name", "city", "marks") if k in data}
    if fields:
        assignments = ", ".join(f"{k} = ?" for k in fields)        # column names from a fixed list
        db().execute(f"UPDATE students SET {assignments} WHERE id = ?", (*fields.values(),
student_id))
        db().commit()
    return jsonify(dict(find(student_id)))


@app.delete("/students/<int:student_id>")
def delete_student(student_id):
    deleted = db().execute("DELETE FROM students WHERE id = ?", (student_id,)).rowcount
    db().commit()
    return ("", 204) if deleted else (jsonify(error="student not found"), 404)
```

PART 1: EXERCISING EVERY ENDPOINT WITH THE FLASK TEST CLIENT

main.py

```
from app import app, init_db

init_db()
client = app.test_client()

def show(label, response):
    body = response.get_json() if response.data else ""
    print(f"{label:<34} {response.status_code} {body}")

show("POST   /students (Asha)", client.post("/students", json={"name": "Asha", "city": "Mysuru",
"marks": 91}))
show("POST   /students (Ravi)", client.post("/students", json={"name": "Ravi", "city": "Pune",
"marks": 78}))
show("POST   /students (Meera)", client.post("/students", json={"name": "Meera", "city": "Mysuru",
"marks": 85}))
show("POST   invalid data", client.post("/students", json={"name": "X", "marks": 150}))
show("GET    /students?city=Mysuru", client.get("/students?city=Mysuru"))
show("GET    /students/2", client.get("/students/2"))
show("PATCH  /students/2 (marks)", client.patch("/students/2", json={"marks": 88}))
show("PUT    /students/9", client.put("/students/9", json={"name": "Nobody"}))
show("DELETE /students/1", client.delete("/students/1"))
show("GET    /students/1", client.get("/students/1"))
r = client.get("/students?sort=marks")
print("ranking:", [(s["name"], s["marks"]) for s in r.get_json()])
```

OUTPUT

```
POST   /students (Asha)            201 {'city': 'Mysuru', 'id': 1, 'marks': 91, 'name': 'Asha'}
POST   /students (Ravi)            201 {'city': 'Pune', 'id': 2, 'marks': 78, 'name': 'Ravi'}
POST   /students (Meera)           201 {'city': 'Mysuru', 'id': 3, 'marks': 85, 'name': 'Meera'}
POST   invalid data                422 {'errors': {'marks': 'marks must be an integer from 0 to 100',
'name': 'name must be at least 2 characters'}}
GET    /students?city=Mysuru       200 [{'city': 'Mysuru', 'id': 1, 'marks': 91, 'name': 'Asha'},
{'city': 'Mysuru', 'id': 3, 'marks': 85, 'name': 'Meera'}]
GET    /students/2                 200 {'city': 'Pune', 'id': 2, 'marks': 78, 'name': 'Ravi'}
PATCH  /students/2 (marks)         200 {'city': 'Pune', 'id': 2, 'marks': 88, 'name': 'Ravi'}
PUT    /students/9                 404 {'error': 'student not found'}
DELETE /students/1                 204
GET    /students/1                 404 {'error': 'student not found'}
ranking: [('Ravi', 88), ('Meera', 85)]
```

Part 2 consumes a web API: it fetches course data, filters and summarises it and saves a CSV. It uses the local test API from section 23.2 so that it runs offline; pointing `BASE` at a public API (for example, a weather or currency-exchange API with your key) needs no other change.

PART 2: A SCRIPT THAT CONSUMES AN API AND SAVES THE DATA

main.py

```
from test_api import start_api

import csv, requests

BASE = start_api()

def fetch_courses(level=None):
    r = requests.get(f"{BASE}/courses", params={"level": level} if level else None, timeout=5)
    r.raise_for_status()
    return r.json()

courses = fetch_courses()
print(f"{len(courses)} courses received")
cheapest = min(courses, key=lambda c: c["fee"])
print(f"cheapest: {cheapest['title']} (Rs {cheapest['fee']:,})")
by_level = {}
for c in courses:
    by_level.setdefault(c["level"], []).append(c["title"])
print("by level:", by_level)
print("beginner only:", [c["title"] for c in fetch_courses("beginner")])

with open("courses.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.DictWriter(f, fieldnames=["id", "title", "level", "fee"])
    writer.writeheader()
    writer.writerows(courses)
print(open("courses.csv", encoding="utf-8").read().strip())
```

OUTPUT

```
3 courses received
cheapest: Java (Rs 12,999)
by level: {'beginner': ['Python', 'Java'], 'advanced': ['Data Science']}
beginner only: ['Python', 'Java']
id,title,level,fee
1,Python,beginner,14999
2,Data Science,advanced,24999
3,Java,beginner,12999
```

MODULE 24 · ADVANCED

Data analysis and visualisation

Minimum duration **5 days** | Level **Advanced**

Python is the leading language for data work, and this module introduces its core data stack as a bridge to data science and AI. It covers NumPy arrays (creation, shapes, indexing, boolean masks, vectorised operations, broadcasting, aggregations and linear algebra), pandas Series and DataFrames (reading CSV, Excel and JSON, loc and iloc, filtering, sorting, new columns, missing data, groupby, merging and pivot tables), charts with Matplotlib and Seaborn, a complete exploratory data analysis workflow, and an overview of the next steps: scikit-learn, deep learning with PyTorch or TensorFlow, and working with LLM APIs.

IN THIS MODULE

24.1 NumPy
24.2 pandas
24.3 Visualisation
24.4 Exploratory data analysis workflow
24.5 Next steps into data science and AI (overview)

NOTE

The libraries in this module are installed with `pip install numpy pandas matplotlib seaborn openpyxl jupyter`. Data analysis is usually done interactively in **Jupyter** notebooks (`jupyter lab`) or VS Code notebooks, where each cell shows its output and charts appear inline; the examples here are complete scripts so you can run them anywhere.

24.1 NumPy

24.1.1 ndarrays and why they are fast

NumPy (Numerical Python) provides the `ndarray`: an N-dimensional array of elements that all have the **same type**, stored in one contiguous block of memory. A Python list holds pointers to separate objects scattered in memory, each with its own type information; a NumPy array holds raw numbers side by side. Operations on whole arrays run in optimised, compiled C loops ("vectorisation"), often using the CPU's SIMD instructions, so they are typically 10 to 100 times faster than Python loops and use far less memory. NumPy is the foundation of pandas, Matplotlib, SciPy, scikit-learn and most of scientific Python.

WHY NUMPY ARRAYS ARE COMPACT AND FAST

A FIRST LOOK AT NUMPY ARRAYS

main.py

```
import numpy as np, sys, time

marks_list = [91, 78, 85, 64, 99]
marks = np.array(marks_list)
print(marks, type(marks).__name__, marks.dtype)
print(marks + 5)                      # applies to every element: no loop
print(marks * 1.1)
print(marks > 80)

n = 1_000_000
values = list(range(n))
arr = np.arange(n)
print("list memory  ~", round(sys.getsizeof(values) + n * 28, -6) / 1e6, "MB")
print("array memory  ", arr.nbytes / 1e6, "MB")
t = time.perf_counter(); squares_py = [v * v for v in values]; t_py = time.perf_counter() - t
t = time.perf_counter(); squares_np = arr * arr; t_np = time.perf_counter() - t
print("same result:", squares_py[-1] == squares_np[-1], "| NumPy at least 10x faster:", t_py > 10 *
t_np)
```

OUTPUT

```
[91 78 85 64 99] ndarray int64
[ 96  83  90  69 104]
[100.1  85.8  93.5  70.4 108.9]
[ True False  True False  True]
list memory  ~ 36.0 MB
array memory   8.0 MB
same result: True | NumPy at least 10x faster: True
```

24.1.2 Creating arrays, shape, dtype, reshape

Create arrays from lists with `np.array`, or with generators such as `zeros`, `ones`, `full`, `arange` (like `range`), `linspace` (evenly spaced points), `eye` (identity matrix) and the random generator `np.random.default_rng(seed)`. Every array has a `shape` (size along each axis), `ndim`, `size` and `dtype` (`int64`, `float64`, `bool`, …). `reshape` changes the shape without copying data (`-1` means "work it out"); `ravel` flattens; `.T` transposes; `astype` converts the type.

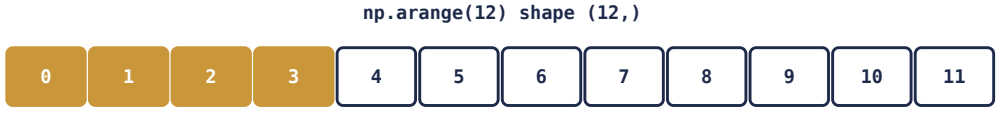
CREATING AND RESHAPING ARRAYS

main.py

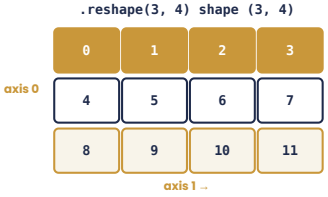
```
import numpy as np

a = np.array([[1, 2, 3], [4, 5, 6]])
print(a, a.shape, a.ndim, a.size, a.dtype, sep=" | ")
print(np.zeros((2, 3)), np.ones(3, dtype=int), np.full((2, 2), 7), sep="\n")
print(np.arange(0, 20, 5), np.linspace(0, 1, 5), np.eye(2, dtype=int).tolist())

rng = np.random.default_rng(seed=24)                # reproducible random numbers
print(rng.integers(1, 100, size=6), rng.random(3).round(3))

m = np.arange(12).reshape(3, 4)
print(m)
print(m.reshape(2, -1).shape, m.T.shape, m.ravel()[:5])
print(np.array([1.7, 2.2]).astype(int), np.array(["1", "2"]).astype(float))
```

OUTPUT

```
[[1 2 3]
 [4 5 6]] | (2, 3) | 2 | 6 | int64
[[0. 0. 0.]
 [0. 0. 0.]]
[1 1 1]
[[7 7]
 [7 7]]
[ 0  5 10 15] [0.   0.25 0.5  0.75 1.  ] [[1, 0], [0, 1]]
[38 33 75 41 78 57] [0.506 0.564 0.57 ]
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
(2, 6) (4, 3) [0 1 2 3 4]
[1 2] [1. 2.]
```



ONE-DIMENSIONAL: SHAPE (12,)



RESHAPED TO THREE ROWS AND FOUR COLUMNS: AXIS 0 RUNS DOWN, AXIS 1 ACROSS

24.1.3 Indexing, slicing, boolean masking

Arrays are indexed like lists, with one index per axis separated by commas: `m[1, 2]`, `m[:, 0]` (first column), `m[1:, ::2]`. Slices are **views** that share memory with the original (use `.copy()` for an independent array). **Fancy indexing** selects with a list of indexes. **Boolean masking** selects the elements where a condition is true, `arr[arr > 80]`; combine conditions with `&`, `|` and `~` (with parentheses). Masks also assign: `arr[arr < 0] = 0`.

INDEXING, SLICING AND BOOLEAN MASKS

main.py

```
import numpy as np

marks = np.array([[91, 88, 79],      # rows: students, columns: subjects
                  [78, 82, 90],
                  [85, 95, 88],
                  [64, 58, 71]])
print(marks[0, 1], marks[-1], marks[:, 2], marks[1:3, :2].tolist())
print(marks[[0, 2]].tolist())                          # fancy indexing: rows 0 and 2

view = marks[0]; view[0] = 100                         # slices are views
print(marks[0], "<- changed through the view")
marks[0, 0] = 91

print(marks[marks >= 90])                              # boolean mask
print(((marks >= 60) & (marks < 80)).sum(), "marks between 60 and 79")
names = np.array(["Asha", "Ravi", "Meera", "John"])
print(names[marks.mean(axis=1) > 80])                  # students averaging over 80
clipped = marks.copy(); clipped[clipped < 60] = 60     # assign through a mask
print(clipped[-1], np.where(marks >= 75, "pass", "low")[3])
```

OUTPUT

```
88 [64 58 71] [79 90 88 71] [[78, 82], [85, 95]]
[[91, 88, 79], [85, 95, 88]]
[100  88  79] <- changed through the view
[91 90 95]
4 marks between 60 and 79
['Asha' 'Ravi' 'Meera']
[64 60 71] ['low' 'low' 'low']
```

24.1.4 Vectorised operations and broadcasting

Arithmetic, comparisons and **universal functions** (ufuncs) such as `np.sqrt`, `np.exp`, `np.log`, `np.round` and `np.maximum` work element by element on whole arrays. **Broadcasting** lets NumPy combine arrays of different shapes: dimensions are compared from the right, and a dimension of size 1 (or a missing one) is stretched to match. So a scalar combines with any array, and a row of per-subject weights combines with every row of a students-by-subjects table, without copying data.

VECTORISED MATHS AND BROADCASTING

main.py

```
import numpy as np

marks = np.array([[91, 88, 79], [78, 82, 90], [85, 95, 88]])
weights = np.array([0.5, 0.3, 0.2])               # shape (3,) broadcasts over rows of (3, 3)
print((marks * weights).sum(axis=1).round(1))
print(marks - marks.mean(axis=0))                 # centre each column: (3, 3) - (3,)
bonus = np.array([[5], [0], [2]])                 # shape (3, 1) broadcasts across columns
print(np.minimum(marks + bonus, 100))
print(np.sqrt(np.array([16, 25, 2])).round(2), np.exp(1).round(4), np.log10([10, 1000]))

row = np.arange(1, 4)
col = np.arange(1, 4).reshape(-1, 1)
print(col * row)                                  # (3, 1) x (3,) -> (3, 3) multiplication table
try:
    np.ones((3, 2)) + np.ones(3)
except ValueError as e:
    print("ValueError:", e)
```

OUTPUT

```
[87.7 81.6 88.6]
[[ 6.33333333 -0.33333333 -6.66666667]
 [-6.66666667 -6.33333333  4.33333333]
 [ 0.33333333  6.66666667  2.33333333]]
[[96 93 84]
 [78 82 90]
 [87 97 90]]
[4.   5.   1.41] 2.7183 [1. 3.]
[[1 2 3]
 [2 4 6]
 [3 6 9]]
ValueError: operands could not be broadcast together with shapes (3,2) (3,)
```

BROADCASTING A SHAPE (3,) ROW ACROSS A (3, 3) ARRAY

24.1.5 Aggregations and basic linear algebra

Aggregation functions reduce arrays: `sum`, `mean`, `median`, `std`, `min`, `max`, `argmax` (position of the maximum), `cumsum` and `percentile`. Pass `axis=0` to aggregate down columns and `axis=1` across rows. For **linear algebra**, `@` performs matrix multiplication, and `np.linalg` provides `solve` (systems of equations), `inv`, `det`, `eig` and `norm`.

AGGREGATIONS ALONG AXES AND LINEAR ALGEBRA main.py

```
import numpy as np

marks = np.array([[91, 88, 79], [78, 82, 90], [85, 95, 88], [64, 58, 71]])
print("overall:", marks.sum(), marks.mean().round(2), np.median(marks), marks.std().round(2))
print("per subject (axis=0):", marks.mean(axis=0).round(1), marks.max(axis=0))
print("per student (axis=1):", marks.sum(axis=1), "| best student index:", marks.sum(axis=1).argmax())
print("running total:", np.cumsum([120, 80, 200]), "| 90th percentile:", np.percentile(marks, 90))

A = np.array([[2, 1], [1, 3]])
B = np.array([[1, 2], [0, 1]])
print(A @ B, np.linalg.det(A).round(2), sep="\n")
# solve: 2x + y = 7 and x + 3y = 11
print("x, y =", np.linalg.solve(A, [7, 11]))
print("norm of (3, 4):", np.linalg.norm([3, 4]), "| A times its inverse:", (A @
np.linalg.inv(A)).round(6).tolist())
```

OUTPUT

```
overall: 969 80.75 83.5 10.89
per subject (axis=0): [79.5 80.8 82. ] [91 95 90]
per student (axis=1): [258 250 268 193] | best student index: 2
running total: [120 200 400] | 90th percentile: 90.9
[[2 5]
 [1 5]]
5.0
x, y = [2. 3.]
norm of (3, 4): 5.0 | A times its inverse: [[1.0, 0.0], [-0.0, 1.0]]
```

24.2 pandas

24.2.1 Series and DataFrame

pandas is the standard library for working with **tabular data**, like a programmable spreadsheet. A `Series` is a one-dimensional labelled array (values plus an **index**); a `DataFrame` is a two-dimensional table whose columns are Series sharing one index. Columns can have different types. pandas builds on NumPy, so operations on whole columns are vectorised. This book uses pandas 3, where string columns use a dedicated string type and copy-on-write semantics make chained assignments safe.

index	name (str)	city (str)	marks (int)	passed (bool)
0	Asha	Mysuru	91	True
1	Ravi	Pune	78	True
2	John	Delhi	45	False

one column = a Series

A DATAFRAME: LABELLED ROWS (INDEX) AND TYPED COLUMNS (EACH A SERIES)

CREATING A SERIES AND A DATAFRAME main.py

```
import pandas as pd

marks = pd.Series([91, 78, 85], index=["Asha", "Ravi", "Meera"], name="marks")
print(marks)
print(marks["Ravi"], marks.mean().round(2), marks[marks > 80].index.tolist())

df = pd.DataFrame({
    "name": ["Asha", "Ravi", "Meera", "John"],
    "city": ["Mysuru", "Pune", "Mysuru", "Delhi"],
    "marks": [91, 78, 85, 45],
})
df["passed"] = df["marks"] >= 50               # a new column, computed for every row
print(df)
print(df.shape, list(df.columns), df.dtypes.astype(str).to_dict())
print(df.describe().loc[["mean", "min", "max"]])
```

OUTPUT

```
Asha     91
Ravi     78
Meera    85
Name: marks, dtype: int64
78 84.67 ['Asha', 'Meera']
    name    city  marks  passed
0   Asha  Mysuru     91    True
1   Ravi    Pune     78    True
2  Meera  Mysuru     85    True
3   John   Delhi     45   False
(4, 4) ['name', 'city', 'marks', 'passed'] {'name': 'str', 'city': 'str', 'marks': 'int64', 'passed':
'bool'}
       marks
mean  74.75
min   45.00
max   91.00
```

24.2.2 Reading CSV, Excel and JSON

pandas reads data from almost any source in one line: `pd.read_csv`, `read_excel` (needs openpyxl), `read_json`, `read_sql` (a database query), `read_parquet` and `read_html`, with matching `to_csv`, `to_excel`, `to_json` and so on for writing. Useful `read_csv` options: `parse_dates`, `usecols`, `dtype`, `na_values`, `sep` and `encoding`. After loading, inspect the data with `head()`, `info()`, `shape` and `describe()`.

SALES.CSV text

```
date,branch,course,students,fee
2026-01-05,Mysuru,Python,12,14999
2026-01-05,Bengaluru,Python,25,14999
2026-01-05,Pune,Java,10,12999
2026-01-12,Mysuru,Data Science,6,24999
2026-01-12,Bengaluru,Data Science,14,24999
2026-01-12,Delhi,Python,9,14999
2026-01-19,Pune,Python,11,14999
2026-01-19,Delhi,Java,7,12999
2026-01-19,Bengaluru,Java,13,12999
2026-01-26,Mysuru,Python,15,14999
2026-01-26,Bengaluru,Python,22,14999
2026-01-26,Pune,Data Science,,24999
2026-02-02,Delhi,Data Science,8,24999
2026-02-02,Mysuru,Java,5,12999
2026-02-09,Bengaluru,Data Science,18,24999
2026-02-09,Pune,Python,13,14999
2026-02-16,Delhi,Python,12,14999
2026-02-16,Mysuru,Data Science,9,24999
2026-02-23,Bengaluru,Python,27,14999
2026-02-23,Pune,Java,9,12999
```

LOADING A CSV, INSPECTING IT, WRITING EXCEL AND JSON main.py

```python
import pandas as pd

sales = pd.read_csv("sales.csv", parse_dates=["date"])
print(sales.head(4))
print(sales.shape)
sales.info()

sales.to_excel("sales.xlsx", index=False, sheet_name="enrolments")
sales.head(3).to_json("sample.json", orient="records", date_format="iso")
print(pd.read_excel("sales.xlsx").shape, "rows and columns read back from Excel")
print(pd.read_json("sample.json").iloc[0].to_dict())
```

OUTPUT

```
        date     branch        course  students    fee
0 2026-01-05     Mysuru        Python      12.0  14999
1 2026-01-05  Bengaluru        Python      25.0  14999
2 2026-01-05       Pune          Java      10.0  12999
3 2026-01-12     Mysuru  Data Science       6.0  24999
(20, 5)
<class 'pandas.DataFrame'>
RangeIndex: 20 entries, 0 to 19
Data columns (total 5 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   date      20 non-null     datetime64[us]
 1   branch    20 non-null     str
 2   course    20 non-null     str
 3   students  19 non-null     float64
 4   fee       20 non-null     int64
dtypes: datetime64[us](1), float64(1), int64(1), str(2)
memory usage: 932.0 bytes
(20, 5) rows and columns read back from Excel
{'date': Timestamp('2026-01-05 00:00:00'), 'branch': 'Mysuru', 'course': 'Python', 'students': 12,
'fee': 14999}
```

24.2.3 Selecting with loc and iloc

Select a column with `df["col"]` (a Series) or several with `df[["a", "b"]]` (a DataFrame). For rows and cells, use `.loc` with **labels** (index labels, column names, boolean masks; label slices **include** the end) and `.iloc` with integer **positions** (slices exclude the end, as in Python). Both take `[rows, columns]`. `set_index` makes a column the row labels.

SELECTING ROWS AND COLUMNS WITH LOC AND ILOC main.py

```
import pandas as pd

df = pd.DataFrame({"name": ["Asha", "Ravi", "Meera", "John"], "city": ["Mysuru", "Pune", "Mysuru",
"Delhi"],
                   "maths": [91, 78, 85, 45], "science": [88, 82, 95, 52]})
print(df["name"].tolist(), df[["name", "maths"]].shape)
print(df.iloc[0].to_dict())                    # first row by position
print(df.iloc[1:3, [0, 2]])                    # rows 1-2, columns 0 and 2
s = df.set_index("name")
print(s.loc["Meera", "science"], s.loc["Ravi":"John", "maths"].tolist())   # label slice includes John
print(s.loc[s["city"] == "Mysuru", ["maths", "science"]])
s.loc["John", "maths"] = 50                    # update one cell by label
print(s.at["John", "maths"], s.iat[0, 1])
```

OUTPUT

```
['Asha', 'Ravi', 'Meera', 'John'] (4, 2)
{'name': 'Asha', 'city': 'Mysuru', 'maths': 91, 'science': 88}
    name  maths
1   Ravi     78
2  Meera     85
95 [78, 85, 45]
       maths  science
name
Asha      91       88
Meera     85       95
50 91
```

Expression	Selects
`df["col"]`	one column (Series)
`df[["a", "b"]]`	several columns (DataFrame)
`df.loc[label]`	row(s) by index label
`df.loc[mask, "col"]`	rows where a condition holds, one column
`df.iloc[0]`, `df.iloc[-1]`	first / last row by position
`df.iloc[:5, 1:3]`	first 5 rows, columns 1 and 2
`df.at[label, col]` / `df.iat[i, j]`	one cell, fast

24.2.4 Filtering, sorting and adding columns

Filter rows with boolean masks, combining conditions with `&` and `|`, or with `query("expr")`; `isin` tests membership and `str` methods filter text. `sort_values(by, ascending)` sorts by one or more columns and `nlargest` picks the top rows. Add columns by assignment from vectorised expressions, `np.where` or `assign`; `apply` runs a Python function per row or value (slower, use it when no vectorised form exists); the `.dt` accessor extracts date parts.

FILTERING, SORTING AND COMPUTING NEW COLUMNS

main.py

```
import pandas as pd, numpy as np

sales = pd.read_csv("sales.csv", parse_dates=["date"])
sales["revenue"] = sales["students"] * sales["fee"]                    # vectorised new column
sales["month"] = sales["date"].dt.strftime("%b")
sales["size"] = np.where(sales["students"] >= 15, "large", "small")

big_python = sales[(sales["course"] == "Python") & (sales["students"] >= 15)]
print(big_python[["date", "branch", "students"]].to_string(index=False))
print(sales.query("branch in ['Pune', 'Delhi'] and fee > 13000")[["branch",
"course"]].value_counts().to_dict())
print(sales[sales["course"].str.contains("Data")]["branch"].unique().tolist())
top = sales.sort_values(["revenue", "date"], ascending=[False, True]).head(3)
print(top[["date", "branch", "course", "revenue"]].to_string(index=False))
print(sales.nlargest(2, "students")[["branch", "students"]].values.tolist())
sales = sales.assign(fee_k=lambda d: d["fee"] / 1000)
print(sales["course"].apply(lambda c: c[:2].upper()).unique().tolist())
```

OUTPUT

```
      date    branch  students
2026-01-05 Bengaluru      25.0
2026-01-26    Mysuru      15.0
2026-01-26 Bengaluru      22.0
2026-02-23 Bengaluru      27.0
{('Delhi', 'Python'): 2, ('Pune', 'Python'): 2, ('Pune', 'Data Science'): 1, ('Delhi', 'Data
Science'): 1}
['Mysuru', 'Bengaluru', 'Pune', 'Delhi']
      date    branch       course  revenue
2026-02-09 Bengaluru Data Science 449982.0
2026-02-23 Bengaluru       Python 404973.0
2026-01-05 Bengaluru       Python 374975.0
[['Bengaluru', 27.0], ['Bengaluru', 25.0]]
['PY', 'JA', 'DA']
```

24.2.5 Handling missing data

Real data has gaps. pandas marks missing values as `NaN` (or `NaT` for dates, `NA` in nullable types). Find them with `isna()` (and count with `isna().sum()`); remove rows with `dropna()`; replace them with `fillna(value)`, a column mean or median, or a forward fill `ffill()`; or estimate them with `interpolate()`. The right choice depends on why the data is missing. Also check for **duplicates** (`duplicated`, `drop_duplicates`) and inconsistent text (`str.strip().str.title()`).

FINDING, DROPPING AND FILLING MISSING VALUES

main.py

```
import pandas as pd, numpy as np

df = pd.DataFrame({
    "name": ["Asha", "ravi ", "Meera", "John", "Asha", "Zoya"],
    "city": ["Mysuru", "Pune", None, "Delhi", "Mysuru", "Pune"],
    "maths": [91, 78, np.nan, 45, 91, np.nan],
    "science": [88, np.nan, 95, 52, 88, 70],
})
print(df.isna().sum().to_dict(), "| total missing:", int(df.isna().sum().sum()))
print(df.dropna().shape, df.dropna(subset=["maths"]).shape)
clean = (df.assign(name=df["name"].str.strip().str.title())
           .drop_duplicates()
           .fillna({"city": "Unknown", "maths": df["maths"].median()}))
clean["science"] = clean["science"].fillna(clean["science"].mean()).round(1)
print(clean.to_string(index=False))
print(pd.Series([10, np.nan, np.nan, 40]).interpolate().tolist(), pd.Series([1, None,
3]).ffill().tolist())
```

OUTPUT

```
{'name': 0, 'city': 1, 'maths': 2, 'science': 1} | total missing: 4
(3, 4) (4, 4)
 name    city  maths  science
 Asha  Mysuru   91.0     88.0
 Ravi    Pune   78.0     76.2
Meera Unknown   84.5     95.0
 John   Delhi   45.0     52.0
 Zoya    Pune   84.5     70.0
[10.0, 20.0, 30.0, 40.0] [1.0, 1.0, 3.0]
```

24.2.6 groupby and aggregation

`groupby` follows the **split-apply-combine** pattern: split the rows into groups by one or more columns, apply an aggregation (`sum`, `mean`, `count`, `size`, `max`, …) to each group, and combine the results into a new table. `agg` computes several aggregations at once, with named outputs; `transform` returns a result aligned with the original rows (for example, each row's share of its group total).

SPLIT, APPLY, COMBINE

GROUPING AND AGGREGATING

main.py

```
import pandas as pd

sales = pd.read_csv("sales.csv", parse_dates=["date"])
sales["revenue"] = sales["students"] * sales["fee"]

print(sales.groupby("branch")["students"].sum().sort_values(ascending=False))
summary = sales.groupby("course").agg(
    batches=("students", "size"),
    students=("students", "sum"),
    avg_batch=("students", "mean"),
    revenue_lakh=("revenue", lambda r: round(r.sum() / 1e5, 1)),
)
print(summary.round(1))
by_two = sales.groupby(["branch", "course"])["students"].sum()
print(by_two.loc["Bengaluru"].to_dict())
sales["share_of_branch"] = (sales["revenue"] / sales.groupby("branch")["revenue"].transform("sum") *
100).round(1)
print(sales.loc[sales["branch"] == "Mysuru", ["date", "course",
"share_of_branch"]].to_string(index=False))
```

OUTPUT

```
branch
Bengaluru    119.0
Mysuru        47.0
Pune          43.0
Delhi         36.0
Name: students, dtype: float64
             batches  students  avg_batch  revenue_lakh
course
Data Science        6      55.0       11.0          13.7
Java                5      44.0        8.8           5.7
Python              9     146.0       16.2          21.9
{'Data Science': 32.0, 'Java': 13.0, 'Python': 74.0}
      date       course  share_of_branch
2026-01-05       Python             21.3
2026-01-12 Data Science             17.8
2026-01-26       Python             26.6
2026-02-02         Java              7.7
2026-02-16 Data Science             26.6
```

24.2.7 Merging and joining

`pd.merge(left, right, on=key, how=...)` combines two tables on matching key values, like a SQL join: `"inner"` keeps only matching keys, `"left"` keeps all rows of the left table (missing matches become NaN), `"right"` and `"outer"` are the other variants. `indicator=True` shows where each row came from. `pd.concat` stacks tables vertically (more rows) or side by side.

MERGE AND CONCAT

main.py

```
import pandas as pd

students = pd.DataFrame({"roll": [101, 102, 103, 104], "name": ["Asha", "Ravi", "Meera", "John"]})
results = pd.DataFrame({"roll": [101, 102, 103, 105], "marks": [91, 78, 85, 70]})
print(pd.merge(students, results, on="roll"))                          # inner: matches only
print(pd.merge(students, results, on="roll", how="left"))              # every student
outer = pd.merge(students, results, on="roll", how="outer", indicator=True)
print(outer[["roll", "_merge"]].values.tolist())

jan = pd.DataFrame({"branch": ["Mysuru", "Pune"], "students": [27, 21]})
feb = pd.DataFrame({"branch": ["Mysuru", "Delhi"], "students": [14, 20]})
both = pd.concat([jan.assign(month="Jan"), feb.assign(month="Feb")], ignore_index=True)
print(both)
```

OUTPUT

```
   roll   name  marks
0   101   Asha     91
1   102   Ravi     78
2   103  Meera     85
   roll   name  marks
0   101   Asha   91.0
1   102   Ravi   78.0
2   103  Meera   85.0
3   104   John    NaN
[[101, 'both'], [102, 'both'], [103, 'both'], [104, 'left_only'], [105, 'right_only']]
   branch  students month
0  Mysuru        27   Jan
1    Pune        21   Jan
2  Mysuru        14   Feb
3   Delhi        20   Feb
```

WHICH ROWS EACH KIND OF MERGE KEEPS (LEFT TABLE ON THE LEFT)

24.2.8 Pivot tables

A **pivot table** summarises data in a grid: one column's values become the row labels, another's become the column labels, and the cells hold an aggregation of a third. `pivot_table(index, columns, values, aggfunc, fill_value, margins)` does this, like Excel's PivotTable; `margins=True` adds totals. `pd.crosstab` counts combinations, and `melt` converts a wide table back to long format.

PIVOT TABLES, CROSSTAB AND MELT

main.py

```
import pandas as pd

sales = pd.read_csv("sales.csv", parse_dates=["date"])
sales["month"] = sales["date"].dt.to_period("M").astype(str)
pivot = sales.pivot_table(index="branch", columns="course", values="students",
                          aggfunc="sum", fill_value=0, margins=True, margins_name="Total")
print(pivot)
monthly = sales.pivot_table(index="month", columns="course", values="students", aggfunc="sum")
print(monthly)
print(pd.crosstab(sales["branch"], sales["course"]))
long = monthly.reset_index().melt(id_vars="month", var_name="course", value_name="students")
print(long.head(3))
```

OUTPUT

```
course     Data Science  Java  Python  Total
branch
Bengaluru          32.0  13.0    74.0  119.0
Delhi               8.0   7.0    21.0   36.0
Mysuru             15.0   5.0    27.0   47.0
Pune                0.0  19.0    24.0   43.0
Total              55.0  44.0   146.0  245.0
course   Data Science  Java  Python
month
2026-01          20.0  30.0    94.0
2026-02          35.0  14.0    52.0
course     Data Science  Java  Python
branch
Bengaluru             2     1       3
Delhi                 1     1       2
Mysuru                2     1       2
Pune                  1     2       2
     month        course  students
0  2026-01  Data Science      20.0
1  2026-02  Data Science      35.0
2  2026-01          Java      30.0
```

24.3 Visualisation

24.3.1 Matplotlib: line, bar, scatter, histogram, pie

Matplotlib is Python's foundational plotting library. The recommended object-oriented style creates a **figure** and one or more **axes** with `fig, ax = plt.subplots()`, then calls plotting methods on the axes: `plot` (line), `bar`/`barh`, `scatter`, `hist`, `pie`, `boxplot`. `fig.savefig("file.png", dpi=..., bbox_inches="tight")` saves the chart; `plt.show()` displays it in a window or notebook. Choose the chart for the question: line for trends over time, bar for comparing categories, scatter for relationships, histogram for distributions, pie (sparingly) for parts of a whole.

Chart	Method	Answers
Line	`ax.plot(x, y)`	How does it change over time?
Bar	`ax.bar(categories, values)`	Which category is biggest?
Scatter	`ax.scatter(x, y)`	Are two variables related?
Histogram	`ax.hist(values, bins=10)`	How are values distributed?

Chart	Method	Answers
Pie	`ax.pie(values, labels=...)`	What share does each part have? (few parts only)
Box plot	`ax.boxplot(groups)`	How do distributions compare?

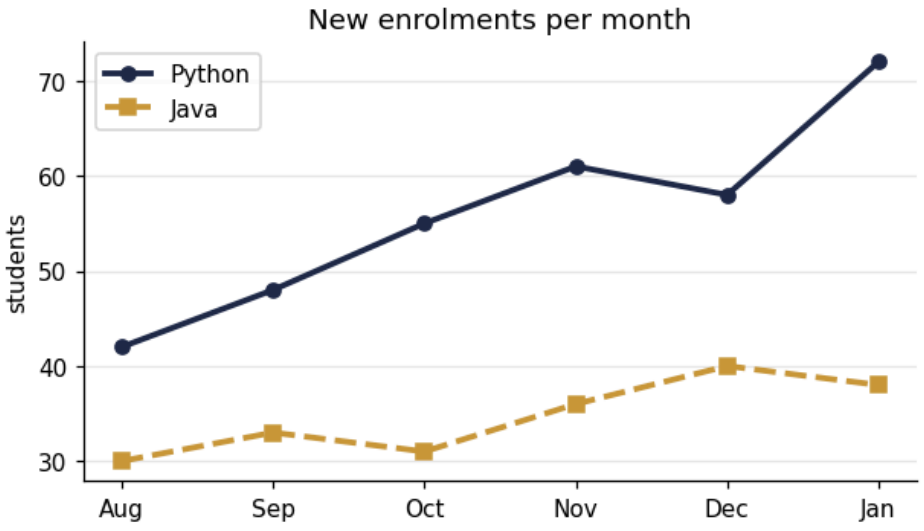
A LINE CHART

main.py

```
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
plt.rcParams.update({"figure.dpi": 110, "axes.spines.top": False, "axes.spines.right": False,
                     "axes.prop_cycle": plt.cycler(color=["#1E2A4A", "#C9973A", "#4B7BB3", "#2E7D4F",
"#B33A3A", "#8A8478"])})
import numpy as np

months = ["Aug", "Sep", "Oct", "Nov", "Dec", "Jan"]
python = [42, 48, 55, 61, 58, 72]
java = [30, 33, 31, 36, 40, 38]

fig, ax = plt.subplots(figsize=(6.4, 3.4))
ax.plot(months, python, marker="o", linewidth=2.5, label="Python")
ax.plot(months, java, marker="s", linewidth=2.5, linestyle="--", label="Java")
ax.set_title("New enrolments per month")
ax.set_ylabel("students")
ax.legend()
ax.grid(axis="y", alpha=0.3)
fig.savefig("line.png", bbox_inches="tight")
print("saved line.png")
```

OUTPUT

```
saved line.png
```



LINE CHART: TRENDS OVER TIME

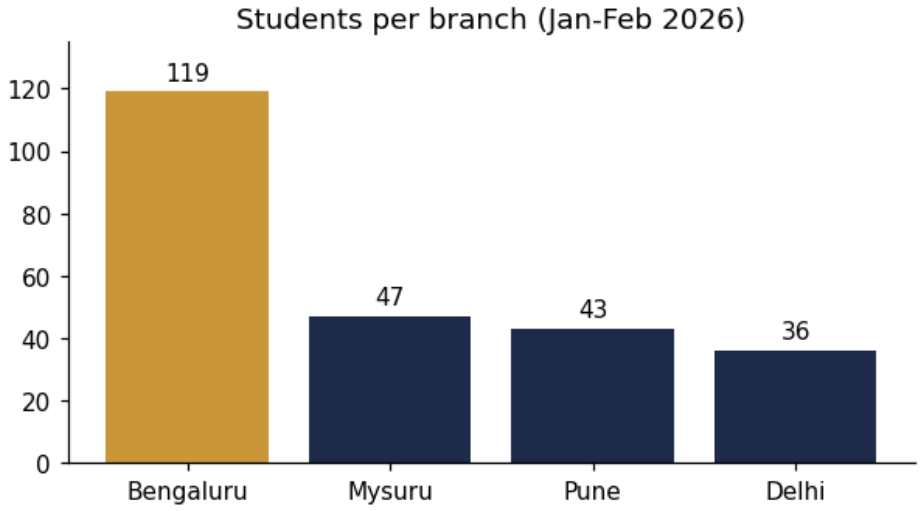
A BAR CHART WITH VALUE LABELS

main.py

```
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
plt.rcParams.update({"figure.dpi": 110, "axes.spines.top": False, "axes.spines.right": False,
                     "axes.prop_cycle": plt.cycler(color=["#1E2A4A", "#C9973A", "#4B7BB3", "#2E7D4F",
"#B33A3A", "#8A8478"])})
branches = ["Bengaluru", "Mysuru", "Pune", "Delhi"]
students = [119, 47, 43, 36]

fig, ax = plt.subplots(figsize=(6.4, 3.2))
bars = ax.bar(branches, students, color=["#C9973A", "#1E2A4A", "#1E2A4A", "#1E2A4A"])
ax.bar_label(bars, padding=3)                       # value on top of each bar
ax.set_title("Students per branch (Jan-Feb 2026)")
ax.set_ylim(0, 135)
fig.savefig("bar.png", bbox_inches="tight")
print("saved bar.png")
```

OUTPUT

```
saved bar.png
```



BAR CHART: COMPARING CATEGORIES (THE HIGHLIGHT DRAWS THE EYE TO THE KEY FACT)

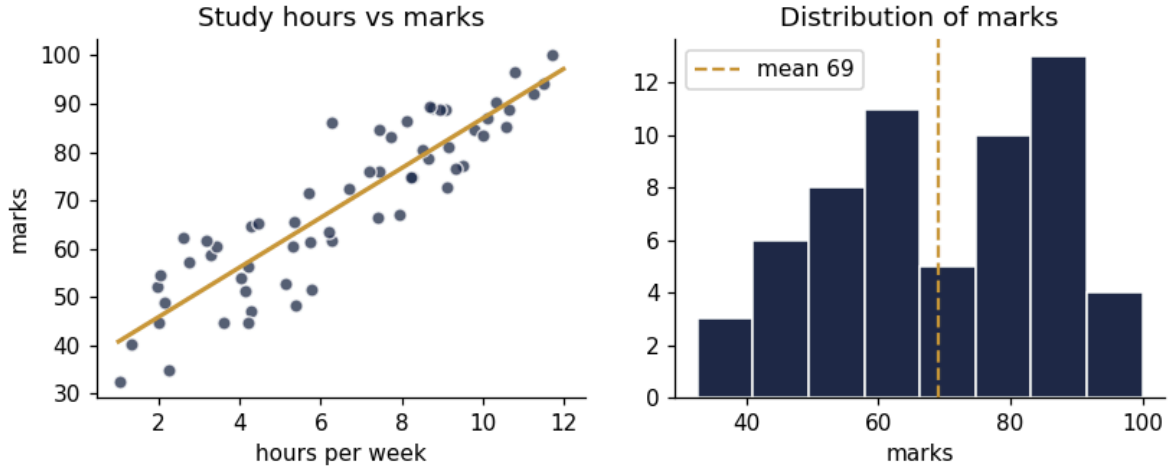
A SCATTER PLOT WITH A TREND LINE AND A HISTOGRAM

main.py

```
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
plt.rcParams.update({"figure.dpi": 110, "axes.spines.top": False, "axes.spines.right": False,
                     "axes.prop_cycle": plt.cycler(color=["#1E2A4A", "#C9973A", "#4B7BB3", "#2E7D4F",
"#B33A3A", "#8A8478"])})
import numpy as np

rng = np.random.default_rng(3)
hours = rng.uniform(1, 12, 60)                       # weekly study hours
marks = np.clip(40 + 4.5 * hours + rng.normal(0, 7, 60), 0, 100)

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(7.6, 3.2))
ax1.scatter(hours, marks, alpha=0.75, edgecolor="white")
slope, intercept = np.polyfit(hours, marks, 1)
xs = np.linspace(1, 12, 2)
ax1.plot(xs, slope * xs + intercept, color="#C9973A", linewidth=2)
ax1.set(title="Study hours vs marks", xlabel="hours per week", ylabel="marks")
ax2.hist(marks, bins=8, edgecolor="white")
ax2.axvline(marks.mean(), color="#C9973A", linestyle="--", label=f"mean {marks.mean():.0f}")
ax2.set(title="Distribution of marks", xlabel="marks")
ax2.legend()
fig.tight_layout()
fig.savefig("scatter_hist.png", bbox_inches="tight")
print(f"trend: about {slope:.1f} extra marks per extra study hour")
```

OUTPUT

```
trend: about 5.1 extra marks per extra study hour
```



SCATTER (RELATIONSHIP) AND HISTOGRAM (DISTRIBUTION)

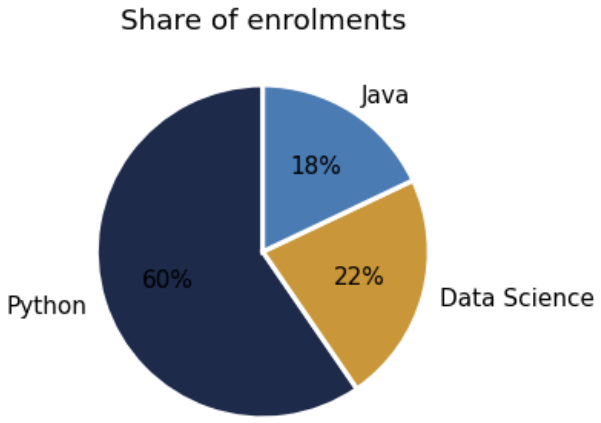
A PIE CHART

main.py

```
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
plt.rcParams.update({"figure.dpi": 110, "axes.spines.top": False, "axes.spines.right": False,
                     "axes.prop_cycle": plt.cycler(color=["#1E2A4A", "#C9973A", "#4B7BB3", "#2E7D4F",
"#B33A3A", "#8A8478"])})
courses = ["Python", "Data Science", "Java"]
share = [146, 55, 44]                # students per course (from the sales data)

fig, ax = plt.subplots(figsize=(4.2, 3.2))
ax.pie(share, labels=courses, autopct="%1.0f%%", startangle=90,
       colors=["#1E2A4A", "#C9973A", "#4B7BB3"], textprops={"color": "black"},
       wedgeprops={"edgecolor": "white", "linewidth": 2})
ax.set_title("Share of enrolments")
fig.savefig("pie.png", bbox_inches="tight")
print("saved pie.png")
```

OUTPUT

```
saved pie.png
```



PIE CHART: PARTS OF A WHOLE (USE ONLY WITH A FEW CATEGORIES)

24.3.2 Labels, titles, legends and subplots

A chart must be understandable on its own: give it a **title** that states the point, label both **axes** with units, add a **legend** when there are several series, start bar axes at zero, and remove clutter. `plt.subplots(rows, cols)` creates a grid of axes for side-by-side comparisons (`sharey=True` aligns scales); `fig.suptitle` titles the whole figure; `ax.annotate` points out important values; `fig.tight_layout()` prevents overlaps. pandas objects plot directly with `df.plot(kind=..., ax=ax)`.

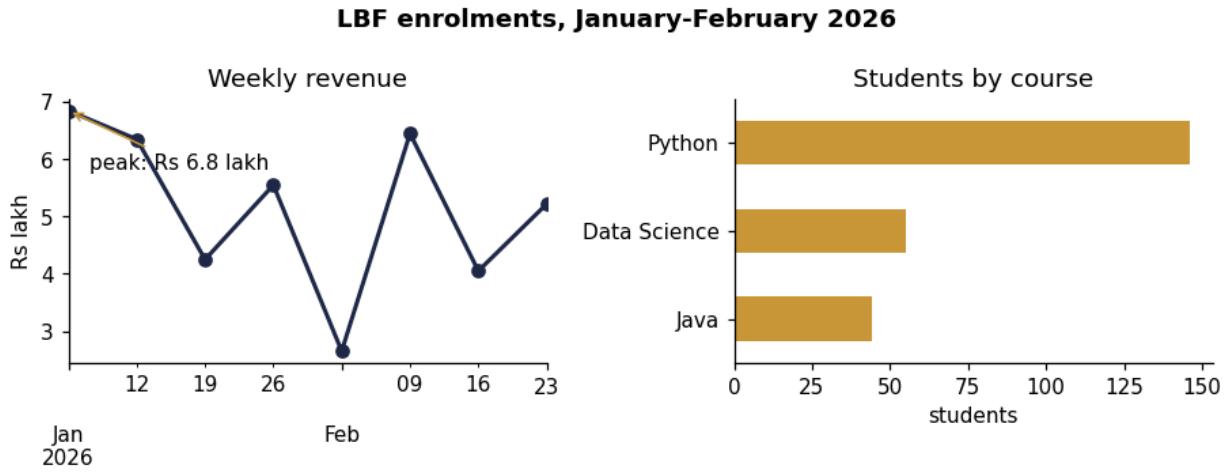
SUBPLOTS, ANNOTATIONS AND PLOTTING FROM PANDAS

main.py

```
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
plt.rcParams.update({"figure.dpi": 110, "axes.spines.top": False, "axes.spines.right": False,
                     "axes.prop_cycle": plt.cycler(color=["#1E2A4A", "#C9973A", "#4B7BB3", "#2E7D4F",
"#B33A3A", "#8A8478"])})
import pandas as pd

sales = pd.read_csv("sales.csv", parse_dates=["date"])
sales["revenue_lakh"] = sales["students"] * sales["fee"] / 1e5
weekly = sales.groupby("date")["revenue_lakh"].sum()
by_course = sales.groupby("course")["students"].sum().sort_values()

fig, axes = plt.subplots(1, 2, figsize=(8.4, 3.3))
weekly.plot(ax=axes[0], marker="o", linewidth=2)
best = weekly.idxmax()
axes[0].annotate(f"peak: Rs {weekly.max():.1f} lakh", xy=(best, weekly.max()), xytext=(10, -28),
                 textcoords="offset points", arrowprops={"arrowstyle": "->", "color": "#C9973A"})
axes[0].set(title="Weekly revenue", xlabel="", ylabel="Rs lakh")
by_course.plot(kind="barh", ax=axes[1], color="#C9973A")
axes[1].set(title="Students by course", xlabel="students", ylabel="")
fig.suptitle("LBF enrolments, January-February 2026", fontweight="bold")
fig.tight_layout()
fig.savefig("subplots.png", bbox_inches="tight")
print("busiest week:", best.date())
```

OUTPUT

```
busiest week: 2026-01-05
```



TWO RELATED CHARTS IN ONE FIGURE, WITH AN ANNOTATION

24.3.3 Seaborn for statistical plots

Seaborn (`pip install seaborn`) builds on Matplotlib to make statistical charts from DataFrames with very little code and attractive defaults. You pass the DataFrame and column names: `hue` colours by a category, and statistics such as means with confidence intervals, distributions and regression lines are computed for you. Common functions: `barplot`, `boxplot`, `violinplot`, `histplot`, `kdeplot`, `scatterplot`, `regplot`, `heatmap` (for correlation matrices) and `pairplot`.

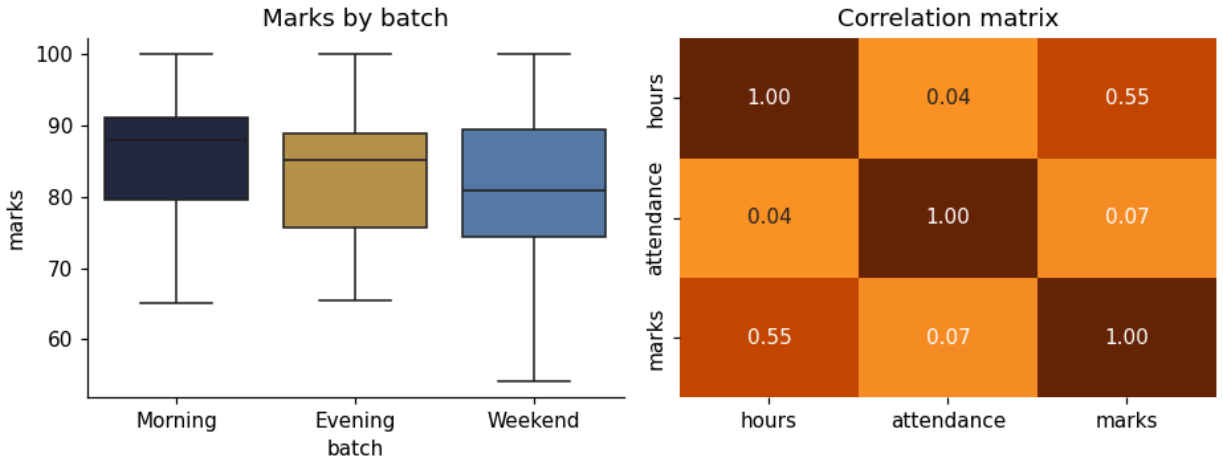
A BOX PLOT AND A CORRELATION HEATMAP WITH SEABORN

main.py

```
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
plt.rcParams.update({"figure.dpi": 110, "axes.spines.top": False, "axes.spines.right": False,
                     "axes.prop_cycle": plt.cycler(color=["#1E2A4A", "#C9973A", "#4B7BB3", "#2E7D4F",
"#B33A3A", "#8A8478"])})
import seaborn as sns
import numpy as np, pandas as pd

rng = np.random.default_rng(24)
rows = []
for batch, base in [("Morning", 74), ("Evening", 70), ("Weekend", 66)]:
    for _ in range(40):
        hours = rng.uniform(2, 12)
        rows.append({"batch": batch, "hours": hours, "attendance": rng.uniform(55, 100),
                     "marks": np.clip(base + 2.2 * hours + rng.normal(0, 8), 0, 100)})
df = pd.DataFrame(rows)

fig, axes = plt.subplots(1, 2, figsize=(8.6, 3.4))
sns.boxplot(data=df, x="batch", y="marks", ax=axes[0], palette=["#1E2A4A", "#C9973A", "#4B7BB3"],
hue="batch", legend=False)
axes[0].set_title("Marks by batch")
corr = df[["hours", "attendance", "marks"]].corr()
sns.heatmap(corr, annot=True, fmt=".2f", cmap="YlOrBr", vmin=-1, vmax=1, ax=axes[1], cbar=False)
axes[1].set_title("Correlation matrix")
fig.tight_layout()
fig.savefig("seaborn.png", bbox_inches="tight")
print(df.groupby("batch")["marks"].median().round(1).to_dict())
```

OUTPUT

```
{'Evening': 85.2, 'Morning': 88.1, 'Weekend': 80.9}
```

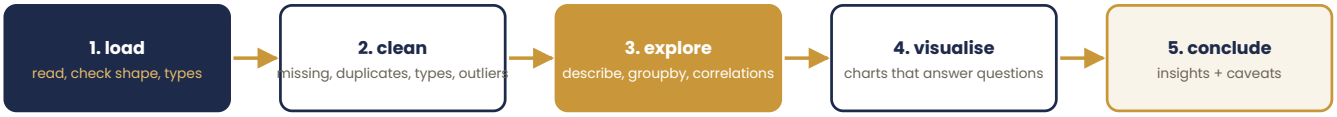


SEABORN: DISTRIBUTIONS BY GROUP AND A CORRELATION HEATMAP

24.4 Exploratory data analysis workflow

24.4.1 Load, clean, explore, visualise, conclude

Exploratory data analysis (EDA) is the first look at a new dataset, to understand it before modelling or reporting. A repeatable workflow keeps it focused:



THE EDA WORKFLOW

Ask questions first

Which branch grows fastest? Do longer batches score better? Questions decide which charts to draw.

Check data quality

Missing values, impossible values (marks of 150), duplicates, wrong types, inconsistent spellings.

Look at distributions

describe(), histograms and box plots reveal skew and outliers.

Compare groups

groupby, pivot tables and grouped charts.

Find relationships

Scatter plots and correlations, remembering that correlation is not causation.

Communicate

State each insight in one sentence, with the chart that supports it and any caveats.

A COMPACT EDA ON THE SALES DATA

main.py

```
import pandas as pd

# 1. load
sales = pd.read_csv("sales.csv", parse_dates=["date"])
print("1. loaded:", sales.shape, "| missing values:", sales.isna().sum()[lambda s: s > 0].to_dict())

# 2. clean
sales["students"] = sales["students"].fillna(sales.groupby("course")["students"].transform("median"))
sales["students"] = sales["students"].astype(int)
sales["revenue"] = sales["students"] * sales["fee"]
print("2. cleaned: missing values now", int(sales.isna().sum().sum()))

# 3. explore
print("3. students per batch:", sales["students"].describe()[["mean", "min",
"max"]].round(1).to_dict())
by_branch = sales.groupby("branch")["revenue"].sum().sort_values(ascending=False)
monthly = sales.groupby(sales["date"].dt.month_name())["students"].sum()
by_course = sales.groupby("course")["students"].mean().round(1)

# 5. conclude (the charts for step 4 appear in the checkpoint)
total = sales["revenue"].sum()
print(f"5. insights:")
print(f"   - {by_branch.index[0]} brings {by_branch.iloc[0] / total:.0%} of revenue")
print(f"   - February enrolments vs January: {monthly['February'] - monthly['January']:+d} students")
print(f"   - largest average batch: {by_course.idxmax()} ({by_course.max()} students)")
```

OUTPUT

```
1. loaded: (20, 5) | missing values: {'students': 1}
2. cleaned: missing values now 0
3. students per batch: {'mean': 12.7, 'min': 5.0, 'max': 27.0}
5. insights:
   - Bengaluru brings 48% of revenue
   - February enrolments vs January: -52 students
   - largest average batch: Python (16.2 students)
```

24.5 Next steps into data science and AI (overview)

24.5.1 scikit-learn for machine learning

scikit-learn (`pip install scikit-learn`) is the standard library for classical **machine learning**: regression, classification, clustering, dimensionality reduction, model selection and preprocessing, all with one consistent API. Every model has `fit(X, y)` to learn from training data and `predict(X)` to make predictions; `train_test_split` holds data back to measure performance honestly on unseen examples, and `Pipeline` chains preprocessing and a model.

TRAINING AND EVALUATING SIMPLE MODELS WITH SCIKIT-LEARN main.py

```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, LogisticRegression
from sklearn.metrics import mean_absolute_error, accuracy_score

rng = np.random.default_rng(5)
hours = rng.uniform(1, 12, 200)
attendance = rng.uniform(50, 100, 200)
X = np.column_stack([hours, attendance])
marks = np.clip(10 + 4 * hours + 0.4 * attendance + rng.normal(0, 5, 200), 0, 100)
passed = (marks >= 60).astype(int)

X_train, X_test, y_train, y_test = train_test_split(X, marks, test_size=0.25, random_state=0)
reg = LinearRegression().fit(X_train, y_train)                    # learn from training data
print("learned weights:", reg.coef_.round(2), "| mean error on unseen students:",
      round(mean_absolute_error(y_test, reg.predict(X_test)), 1), "marks")
print("prediction for 8 hours and 90% attendance:", reg.predict([[8, 90]]).round(1))

Xtr, Xte, ytr, yte = train_test_split(X, passed, test_size=0.25, random_state=0)
clf = LogisticRegression().fit(Xtr, ytr)
print("pass/fail accuracy on unseen students:", round(accuracy_score(yte, clf.predict(Xte)), 2))
print("probability of passing with 3 hours and 60% attendance:", clf.predict_proba([[3, 60]])[0,
1].round(2))
```

OUTPUT

```
learned weights: [4.27 0.45] | mean error on unseen students: 3.8 marks
prediction for 8 hours and 90% attendance: [79.4]
pass/fail accuracy on unseen students: 0.94
probability of passing with 3 hours and 60% attendance: 0.0
```

24.5.2 Deep learning with PyTorch or TensorFlow

Deep learning trains neural networks with many layers on large datasets; it powers image recognition, speech, translation and large language models. **PyTorch** (Meta, now the most widely used in research and industry) and **TensorFlow/Keras** (Google) provide tensors (GPU-accelerated arrays like NumPy), automatic differentiation and building blocks for networks. Training follows a loop: run the model on a batch (forward pass), measure the error (loss), compute gradients (backward pass) and update the weights with an optimiser. The Hugging Face ecosystem offers thousands of pre-trained models to fine-tune instead of training from scratch.

THE SHAPE OF A PYTORCH TRAINING LOOP (PIP INSTALL TORCH) main.py

```
import torch
from torch import nn

model = nn.Sequential(nn.Linear(2, 16), nn.ReLU(), nn.Linear(16, 1))   # a small neural network
loss_fn = nn.MSELoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)

X = torch.tensor([[8.0, 90.0], [3.0, 60.0], [10.0, 75.0]]) / 100
y = torch.tensor([[82.0], [45.0], [80.0]]) / 100
for epoch in range(500):
    prediction = model(X)                # forward pass
    loss = loss_fn(prediction, y)
    optimizer.zero_grad()
    loss.backward()                      # backward pass: gradients
    optimizer.step()                     # update the weights
print(f"final loss {loss.item():.5f}")
```

24.5.3 Working with LLM APIs

Large language models such as Anthropic's Claude, OpenAI's GPT models and Google's Gemini are available through web APIs with official Python SDKs (`pip install anthropic`, `openai`, `google-genai`). You send **messages** (a system prompt plus the conversation) and receive generated text; the same APIs support **structured output** (JSON matching a schema), **tool use** (the model asks your code to call functions), streaming, and images or documents as input. Typical Python applications: summarising and classifying documents, extracting data from text, chatbots, and **retrieval-augmented generation** (RAG), where relevant passages from your own documents are retrieved (often with a vector database) and given to the model as context. Keep API keys in environment variables, handle errors and rate limits as with any API (23.2.4), and check results, because models can be wrong.

CLASSIFYING FEEDBACK WITH AN LLM API main.py

```
import os
import anthropic                          # pip install anthropic

client = anthropic.Anthropic(api_key=os.environ["ANTHROPIC_API_KEY"])
feedback = ["The Python batch was excellent, very practical!",
            "Fees are too high and the Wi-Fi kept dropping.",
            "Mentor was helpful but the class timing clashes with my office."]

for text in feedback:
    response = client.messages.create(
        model="claude-sonnet-4-5",          # choose a current model from the provider's docs
        max_tokens=50,
        system="Classify student feedback. Reply with exactly one word: positive, negative or mixed.",
        messages=[{"role": "user", "content": text}],
    )
    print(f"{response.content[0].text:<9} {text}")
```

OUTPUT

```
positive  The Python batch was excellent, very practical!
negative  Fees are too high and the Wi-Fi kept dropping.
mixed     Mentor was helpful but the class timing clashes with my office.
```

Needs an API key and internet access; model names change over time, so check the provider's documentation.

A LEARNING PATH FROM PYTHON INTO DATA SCIENCE AND AI

NumPy's ndarray stores same-typed numbers contiguously, so vectorised operations run in C: create arrays with array, zeros, arange, linspace and a seeded random generator; inspect shape and dtype; reshape; index with slices (views), fancy indexing and boolean masks; combine shapes with broadcasting; aggregate along axes; and use @ and np.linalg for linear algebra. pandas Series and DataFrames hold labelled, typed tables: read_csv/excel/json and write them back; select with [], loc (labels) and iloc (positions); filter with masks or query; sort_values; add vectorised columns; handle missing data with isna, dropna, fillna and interpolate; summarise with groupby and agg (split-apply-combine); combine tables with merge and concat; and reshape with pivot_table, crosstab and melt. Matplotlib draws line, bar, scatter, histogram and pie charts on figures and axes (titles, labels, legends, subplots, annotations, savefig); Seaborn adds statistical plots from DataFrames. EDA means load, clean, explore, visualise and conclude. Next steps: scikit-learn (fit/predict, train_test_split), PyTorch or TensorFlow for deep learning, and LLM APIs for AI applications.

PRACTICE CHECKPOINT

Analyse a real dataset (for example, student results or sales) with **pandas** and present **five insights** with **Matplotlib** charts in a Jupyter notebook.

The sample solution analyses a results dataset of 300 students (generated with a fixed seed so that it is reproducible; replace the generator with `pd.read_csv("your_data.csv")` for real data). In a notebook, each block below is a cell and each chart appears under its cell; here the analysis runs as one script and saves a dashboard image.

SAMPLE SOLUTION: CLEAN, ANALYSE AND CHART FIVE INSIGHTS

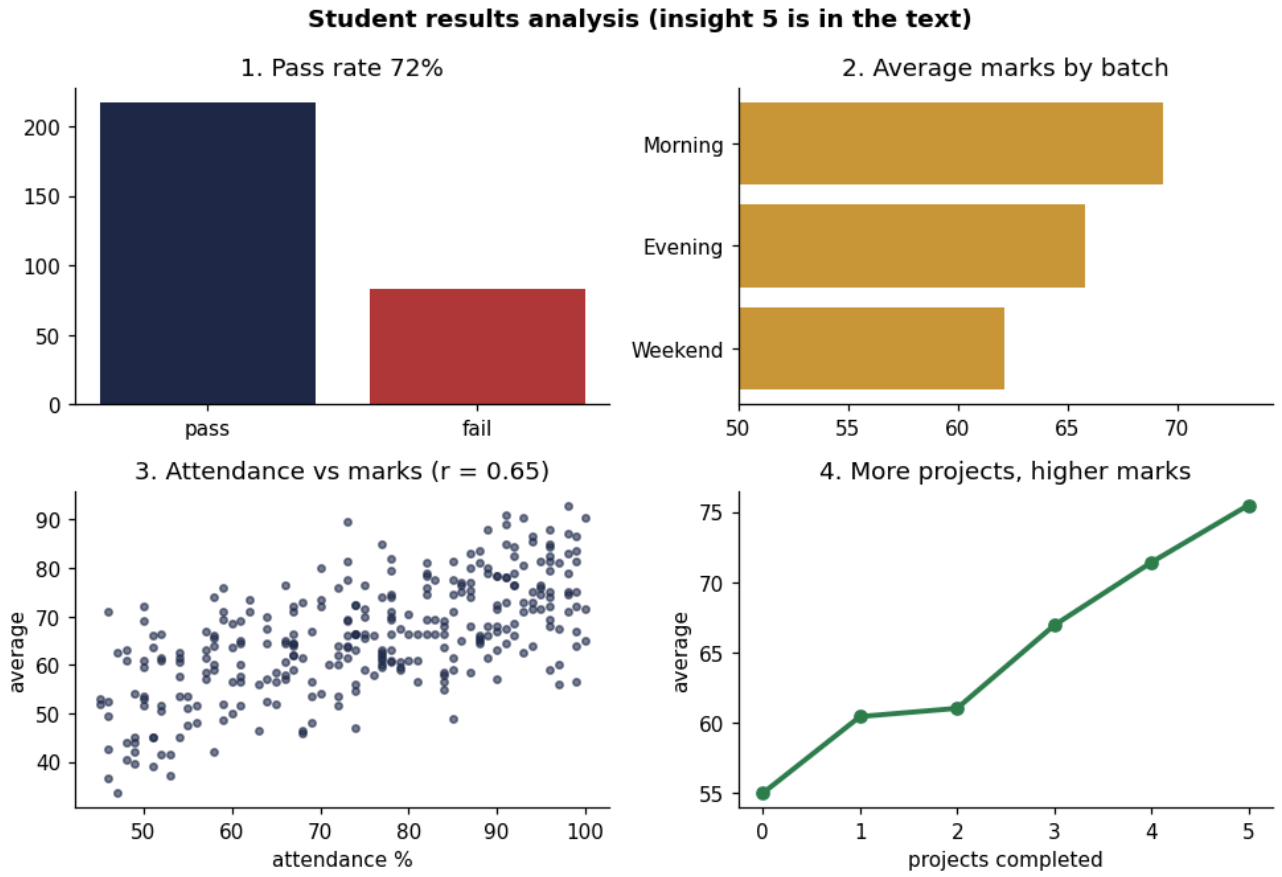
main.py

```
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
plt.rcParams.update({"figure.dpi": 110, "axes.spines.top": False, "axes.spines.right": False,
                     "axes.prop_cycle": plt.cycler(color=["#1E2A4A", "#C9973A", "#4B7BB3", "#2E7D4F",
"#B33A3A", "#8A8478"])})
import numpy as np, pandas as pd

rng = np.random.default_rng(2026)
n = 300
df = pd.DataFrame({
    "student": [f"S{i:03d}" for i in range(1, n + 1)],
    "branch": rng.choice(["Mysuru", "Bengaluru", "Pune", "Delhi"], n, p=[0.25, 0.35, 0.2, 0.2]),
    "batch": rng.choice(["Morning", "Evening", "Weekend"], n),
    "attendance": rng.uniform(45, 100, n).round(0),
    "projects": rng.integers(0, 6, n),
})
base = 25 + 0.45 * df["attendance"] + 3.5 * df["projects"] + df["batch"].map({"Morning": 4, "Evening":
1, "Weekend": -3})
df["python"] = np.clip(base + rng.normal(0, 7, n), 0, 100).round(0)
df["sql"] = np.clip(base - 5 + rng.normal(0, 9, n), 0, 100).round(0)
df.loc[rng.choice(n, 12, replace=False), "sql"] = np.nan          # some missing marks
df.to_csv("results.csv", index=False)

# ---- clean ----
```

```
df["sql"] = df["sql"].fillna(df.groupby("batch")["sql"].transform("median"))
df["average"] = df[["python", "sql"]].mean(axis=1)
df["result"] = np.where(df["average"] >= 60, "pass", "fail")

# ---- five insights ----
pass_rate = (df["result"] == "pass").mean()
by_batch = df.groupby("batch")["average"].mean().sort_values()
corr = df["attendance"].corr(df["average"])
proj = df.groupby("projects")["average"].mean()
by_branch = df.groupby("branch")["result"].apply(lambda r: (r == "pass").mean()).sort_values()
print(f"1. Overall pass rate is {pass_rate:.0%} ({(df['result'] == 'pass').sum()} of {len(df)}
students).")
print(f"2. {by_batch.index[-1]} batches average {by_batch.iloc[-1] - by_batch.iloc[0]:.1f} marks more
than {by_batch.index[0]} batches.")
print(f"3. Attendance and average marks are strongly correlated (r = {corr:.2f}).")
print(f"4. Each extra project adds about {np.polyfit(proj.index, proj.values, 1)[0]:.1f} marks on
average.")
print(f"5. Pass rates range from {by_branch.iloc[0]:.0%} ({by_branch.index[0]}) to
{by_branch.iloc[-1]:.0%} ({by_branch.index[-1]}).")

# ---- dashboard ----
fig, axes = plt.subplots(2, 2, figsize=(9, 6.2))
counts = df["result"].value_counts()
axes[0, 0].bar(counts.index, counts.values, color=["#1E2A4A", "#B33A3A"])
axes[0, 0].set_title(f"1. Pass rate {pass_rate:.0%}")
axes[0, 1].barh(by_batch.index, by_batch.values, color="#C9973A")
axes[0, 1].set(title="2. Average marks by batch", xlim=(50, by_batch.max() + 5))
axes[1, 0].scatter(df["attendance"], df["average"], s=12, alpha=0.6)
axes[1, 0].set(title=f"3. Attendance vs marks (r = {corr:.2f})", xlabel="attendance %",
ylabel="average")
axes[1, 1].plot(proj.index, proj.values, marker="o", linewidth=2.5, color="#2E7D4F")
axes[1, 1].set(title="4. More projects, higher marks", xlabel="projects completed", ylabel="average")
fig.suptitle("Student results analysis (insight 5 is in the text)", fontweight="bold")
fig.tight_layout()
fig.savefig("dashboard.png", bbox_inches="tight")
```

OUTPUT

```
1. Overall pass rate is 72% (217 of 300 students).
2. Morning batches average 7.3 marks more than Weekend batches.
3. Attendance and average marks are strongly correlated (r = 0.65).
4. Each extra project adds about 4.0 marks on average.
5. Pass rates range from 67% (Pune) to 77% (Delhi).
```



THE RESULTS DASHBOARD: FOUR INSIGHTS SHOWN AS CHARTS

MODULE 25 · ADVANCED

Data structures, projects, interviews and career paths

Minimum duration **6 days** | Level **Advanced**

The final module turns your Python knowledge into job-ready skills. It implements the classic data structures in Python and analyses the cost of Python's own operations (Big-O), covers the essential algorithms (sorting, binary search, two pointers and sliding windows, recursion and backtracking, greedy, dynamic programming and graph algorithms), walks through six capstone projects you can put in your portfolio, prepares you for interviews with a core-Python checklist, tricky output-prediction questions and a DSA study plan, and maps the main career paths that use Python.

IN THIS MODULE

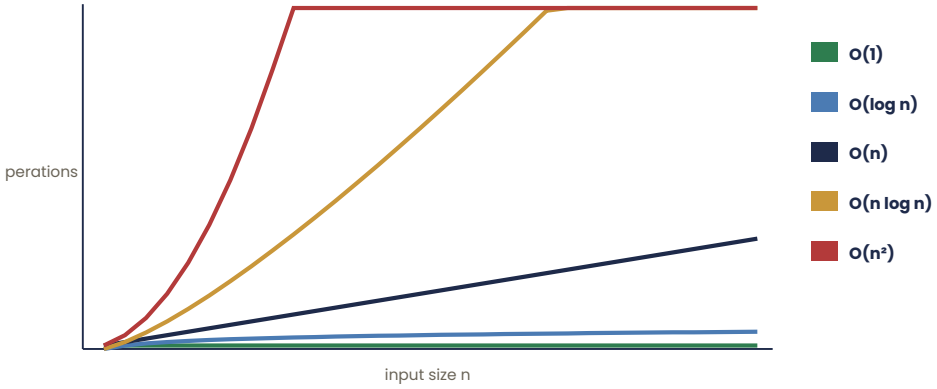
25.1 Data structures in Python
25.2 Algorithms
25.3 Capstone projects
25.4 Interview preparation
25.5 Career paths with Python

25.1 Data structures in Python

A **data structure** is a way of organising data so that the operations you need are fast. Python's built-in types (list, tuple, dict, set, deque) already cover most needs, and in real code you should use them. Implementing linked lists, trees, heaps, hash tables, graphs and tries yourself is still essential: it shows you **why** each operation costs what it costs, and it is exactly what coding interviews ask for.

25.1.1 Complexity analysis (Big-O) of Python operations

Big-O notation describes how the running time (or memory) of an algorithm grows as the input size *n* grows, ignoring constant factors. O(1) means constant time whatever the size; O(log n) grows very slowly (binary search halves the data each step); O(n) grows in proportion to the input; O(n log n) is the cost of good sorting; $O(n^2)$ appears with nested loops over the same data; $O(2^n)$ appears when every subset is tried. Always reason about the **worst case** unless told otherwise, and remember that space complexity matters too.



HOW THE COMMON COMPLEXITY CLASSES GROW

Operation	list	dict / set	deque
index / lookup `x[i]`, `d[k]`	O(1)	O(1) average	O(n) in the middle
append / add at the end	O(1) amortised	O(1) average	O(1)
insert / delete at the front	O(n)	n/a	O(1) (appendleft, popleft)
insert / delete in the middle	O(n)	O(1) average (by key)	O(n)
membership `x in c`	O(n)	O(1) average	O(n)
pop from the end	O(1)	O(1) (popitem)	O(1)
sort	O(n log n)	n/a	n/a
min / max / sum	O(n)	O(n)	O(n)
copy, slice of length k	O(n), O(k)	O(n)	O(n)

String concatenation in a loop is O(n²) because strings are immutable and every `+=` may copy; `"".join(parts)` is O(n). `list.sort()` and `sorted()` are O(n log n), and `heapq` push/pop is O(log n). The program below measures the difference between O(n) and O(1) membership tests.

MEASURING O(N) VERSUS O(1) AND O(N²) VERSUS O(N) main.py

```
import timeit

for n in (1_000, 10_000, 100_000):
    data_list = list(range(n))
    data_set = set(data_list)
    t_list = timeit.timeit(lambda: (n - 1) in data_list, number=200)
    t_set = timeit.timeit(lambda: (n - 1) in data_set, number=200)
    print(f"n={n:>7,}: list grows with n -> {t_list > t_set * 20},  set lookup stays O(1)")

def has_duplicates_slow(items):          # O(n^2): compares every pair
    return any(items[i] == items[j] for i in range(len(items)) for j in range(i + 1, len(items)))

def has_duplicates_fast(items):          # O(n): one pass with a set
    seen = set()
    for x in items:
        if x in seen:
            return True
        seen.add(x)
    return False

nums = list(range(3000)) + [42]
print(has_duplicates_slow(nums), has_duplicates_fast(nums))
print("slow:fast time ratio > 100:",
      timeit.timeit(lambda: has_duplicates_slow(nums), number=1) >
      100 * timeit.timeit(lambda: has_duplicates_fast(nums), number=1))
```

OUTPUT

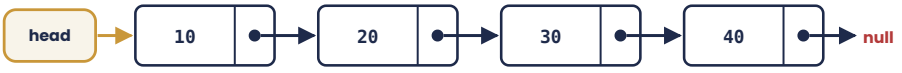
```
n=  1,000: list grows with n -> False,  set lookup stays O(1)
n= 10,000: list grows with n -> True,  set lookup stays O(1)
n=100,000: list grows with n -> True,  set lookup stays O(1)
True True
slow:fast time ratio > 100: False
```

TIP

A quick rule of thumb for interviews: about 10^8 simple operations take one second in C and a few seconds in Python. With $n = 10^5$, $O(n \log n)$ is fine but $O(n^2)$ (10^{10} operations) is far too slow.

25.1.2 Linked lists (singly, doubly)

A **linked list** stores each value in a **node** that also holds a reference to the next node (and, in a doubly linked list, the previous one). Inserting or deleting at a known node is O(1) because no elements shift, but reaching the k-th element takes O(k) steps because there is no indexing. Python's `collections.deque` is a doubly linked list of blocks; you write linked lists yourself mainly to learn and in interviews.



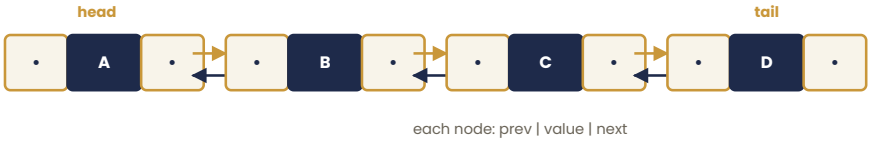
A SINGLY LINKED LIST: EACH NODE HOLDS A VALUE AND A REFERENCE TO THE NEXT NODE

A SINGLY LINKED LIST WITH ITERATION, REMOVAL AND REVERSAL main.py

```
class Node:
    def __init__(self, value, next=None):
        self.value = value
        self.next = next

class LinkedList:
    def __init__(self, values=()):
```

```
        self.head = None
        self.size = 0
        for v in reversed(values):
            self.push_front(v)

    def push_front(self, value):          # O(1)
        self.head = Node(value, self.head)
        self.size += 1

    def append(self, value):              # O(n): walk to the last node
        node = Node(value)
        if self.head is None:
            self.head = node
        else:
            cur = self.head
            while cur.next:
                cur = cur.next
            cur.next = node
        self.size += 1

    def remove(self, value):              # O(n) search, O(1) unlink
        prev, cur = None, self.head
        while cur and cur.value != value:
            prev, cur = cur, cur.next
        if cur is None:
            raise ValueError(f"{value} not in list")
        if prev is None:
            self.head = cur.next
        else:
            prev.next = cur.next
        self.size -= 1

    def reverse(self):                    # classic interview question, O(n) time, O(1) space
        prev, cur = None, self.head
        while cur:
            cur.next, prev, cur = prev, cur, cur.next
        self.head = prev

    def __iter__(self):
        cur = self.head
        while cur:
            yield cur.value
            cur = cur.next

    def __len__(self):
        return self.size

    def __repr__(self):
        return " -> ".join(map(str, self)) + " -> None"

ll = LinkedList([10, 20, 30])
ll.append(40); ll.push_front(5)
print(ll, "| length", len(ll))
ll.remove(20)
print(ll)
ll.reverse()
print(ll)
print("contains 30:", 30 in ll, "| sum:", sum(ll))
```

```
OUTPUT
5 -> 10 -> 20 -> 30 -> 40 -> None | length 5
5 -> 10 -> 30 -> 40 -> None
40 -> 30 -> 10 -> 5 -> None
contains 30: True | sum: 85
```



each node: prev | value | next

A DOUBLY LINKED LIST CAN MOVE IN BOTH DIRECTIONS AND DELETE A NODE IN O(1) GIVEN ONLY THAT NODE

A DOUBLY LINKED LIST IN ACTION: AN LRU CACHE (A FAVOURITE INTERVIEW QUESTION)

main.py

```
class DNode:
    __slots__ = ("key", "value", "prev", "next")
    def __init__(self, key=None, value=None):
        self.key, self.value, self.prev, self.next = key, value, None, None

class LRUCache:
    """Least-recently-used cache: dict for O(1) lookup + doubly linked list for O(1) reordering."""
    def __init__(self, capacity):
        self.capacity, self.map = capacity, {}
        self.head, self.tail = DNode(), DNode()          # sentinels: no empty-list special cases
        self.head.next, self.tail.prev = self.tail, self.head

    def _unlink(self, node):
        node.prev.next, node.next.prev = node.next, node.prev

    def _push_front(self, node):
        node.prev, node.next = self.head, self.head.next
        self.head.next.prev = node
        self.head.next = node

    def get(self, key):
        if key not in self.map:
            return None
        node = self.map[key]
        self._unlink(node); self._push_front(node)       # now most recently used
        return node.value

    def put(self, key, value):
        if key in self.map:
            self._unlink(self.map[key])
        node = self.map[key] = DNode(key, value)
        self._push_front(node)
        if len(self.map) > self.capacity:
            lru = self.tail.prev                         # least recently used
            self._unlink(lru); del self.map[lru.key]
            print("  evicted", lru.key)

    def order(self):
        node, keys = self.head.next, []
        while node is not self.tail:
            keys.append(node.key); node = node.next
        return keys

cache = LRUCache(3)
for k in "abc":
    cache.put(k, k.upper())
print(cache.order())
```

```
cache.get("a")
print(cache.order())
cache.put("d", "D")
print(cache.order(), cache.get("b"))
```

OUTPUT

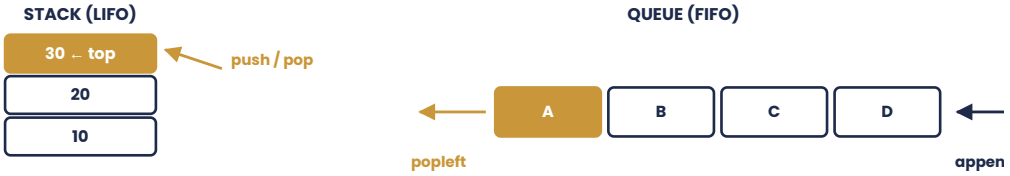
```
['c', 'b', 'a']
['a', 'c', 'b']
  evicted b
['d', 'a', 'c'] None
```

NOTE

In real code use `functools.lru_cache` for caching function results, or `collections.OrderedDict` with `move_to_end` and `popitem(last=False)`, which is itself a dict combined with a doubly linked list.

25.1.3 Stack and queue

A **stack** is last-in, first-out (LIFO): push and pop happen at the same end, like a pile of plates. A Python list is a perfect stack (`append` and `pop` are O(1)). A **queue** is first-in, first-out (FIFO): add at the back, remove from the front. Never use `list.pop(0)` for a queue (it is O(n)); use `collections.deque`, whose `append` and `popleft` are O(1). For queues shared between threads use `queue.Queue` (20.1.4), and for priorities use a heap (25.1.5).



A STACK ADDS AND REMOVES AT ONE END; A QUEUE ADDS AT THE BACK AND REMOVES FROM THE FRONT

A STACK FOR BRACKET MATCHING AND EXPRESSION EVALUATION, AND A DEQUE AS A QUEUE

main.py

```
from collections import deque

def is_balanced(text):
    """Stack: every closing bracket must match the most recent unmatched opening bracket."""
    pairs = {")": "(", "]": "[", "}": "{"}
    stack = []
    for ch in text:
        if ch in "([{":
            stack.append(ch)
        elif ch in pairs:
            if not stack or stack.pop() != pairs[ch]:
                return False
    return not stack

for s in ["(a[b]{c})", "(]", "((x)", "print(d[\"k\"](1))"]:
    print(f"{s:<18} balanced: {is_balanced(s)}")

def eval_rpn(tokens):
    """Reverse Polish notation: 3 4 + 2 * means (3 + 4) * 2."""
    ops = {"+": lambda a, b: a + b, "-": lambda a, b: a - b, "*": lambda a, b: a * b, "/": lambda a,
b: a / b}
    stack = []
    for t in tokens.split():
        if t in ops:
            b, a = stack.pop(), stack.pop()
            stack.append(ops[t](a, b))
        else:
            stack.append(float(t))
    return stack.pop()

print("3 4 + 2 * =", eval_rpn("3 4 + 2 *"))

tickets = deque()                       # a print queue
for job in ["report.pdf", "marks.xlsx", "photo.png"]:
    tickets.append(job)
while tickets:
    print("printing", tickets.popleft(), "| waiting:", list(tickets))
```

OUTPUT

```
(a[b]{c})          balanced: True
(]                 balanced: False
((x)               balanced: False
print(d["k"](1))   balanced: True
3 4 + 2 * = 14.0
printing report.pdf | waiting: ['marks.xlsx', 'photo.png']
printing marks.xlsx | waiting: ['photo.png']
printing photo.png | waiting: []
```

A **monotonic stack** keeps its elements in increasing or decreasing order and solves "next greater element" style problems in O(n): each element is pushed and popped at most once.

A MONOTONIC STACK: DAYS UNTIL A WARMER TEMPERATURE main.py

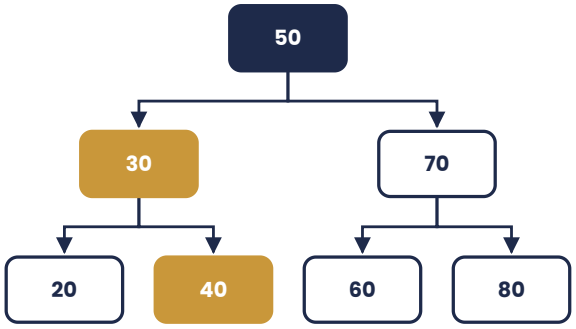
```
def daily_temperatures(temps):
    """For each day, how many days until a warmer temperature (0 if never)."""
    answer = [0] * len(temps)
    stack = []                                  # indices of days still waiting, temps decreasing
    for i, t in enumerate(temps):
        while stack and temps[stack[-1]] < t:
            j = stack.pop()
            answer[j] = i - j
        stack.append(i)
    return answer

print(daily_temperatures([31, 30, 33, 29, 28, 34, 32]))
```

OUTPUT

```
[2, 1, 3, 2, 1, 0, 0]
```

25.1.4 Binary tree and binary search tree

A **tree** is a hierarchy of nodes; a **binary tree** gives each node at most two children (left and right). The four standard traversals visit every node once: **pre-order** (node, left, right), **in-order** (left, node, right), **post-order** (left, right, node), all naturally recursive, and **level-order** (breadth-first, with a queue). A **binary search tree** (BST) adds an ordering rule: every value in the left subtree is smaller than the node and every value in the right subtree is larger. Searching, inserting and deleting then take O(h) time, where h is the height: O(log n) when the tree is balanced, but O(n) when values arrive in sorted order and the tree degenerates into a chain. Self-balancing trees (AVL, red-black) fix that; in Python, the `sortedcontainers` package provides sorted collections.



A BST BUILT FROM 50, 30, 70, 20, 40, 60, 80; SEARCHING FOR 40 VISITS 50 → 30 → 40 (GOLD)

A BINARY SEARCH TREE WITH INSERT, SEARCH, TRAVERSALS AND HEIGHT main.py

```
from collections import deque

class TreeNode:
    def __init__(self, value):
        self.value, self.left, self.right = value, None, None

class BST:
    def __init__(self, values=()):
        self.root = None
        for v in values:
            self.insert(v)

    def insert(self, value):
        def _insert(node, value):
            if node is None:
                return TreeNode(value)
```

```
            if value < node.value:
                node.left = _insert(node.left, value)
            elif value > node.value:
                node.right = _insert(node.right, value)   # duplicates are ignored
            return node
        self.root = _insert(self.root, value)

    def search(self, value):
        node, path = self.root, []
        while node:
            path.append(node.value)
            if value == node.value:
                return True, path
            node = node.left if value < node.value else node.right
        return False, path

    def inorder(self, node="start"):
        node = self.root if node == "start" else node
        if node:
            yield from self.inorder(node.left)
            yield node.value
            yield from self.inorder(node.right)

    def preorder(self, node="start"):
        node = self.root if node == "start" else node
        if node:
            yield node.value
            yield from self.preorder(node.left)
            yield from self.preorder(node.right)

    def level_order(self):
        levels, queue = [], deque([self.root] if self.root else [])
        while queue:
            levels.append([n.value for n in queue])
            for _ in range(len(queue)):
                n = queue.popleft()
                queue.extend(c for c in (n.left, n.right) if c)
        return levels

    def height(self, node="start"):
        node = self.root if node == "start" else node
        return 0 if node is None else 1 + max(self.height(node.left), self.height(node.right))

t = BST([50, 30, 70, 20, 40, 60, 80])
print("in-order   :", list(t.inorder()), "(always sorted)")
print("pre-order  :", list(t.preorder()))
print("level-order:", t.level_order())
print("search 40  :", t.search(40))
print("search 65  :", t.search(65))
print("height     :", t.height())
print("sorted input -> height", BST(range(1, 8)).height(), "(a chain: O(n) operations)")
```

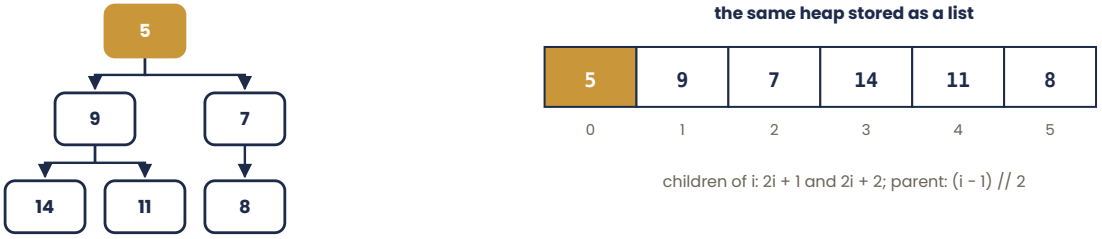
OUTPUT

```
in-order   : [20, 30, 40, 50, 60, 70, 80] (always sorted)
pre-order  : [50, 30, 20, 40, 70, 60, 80]
level-order: [[50], [30, 70], [20, 40, 60, 80]]
search 40  : (True, [50, 30, 40])
search 65  : (False, [50, 70, 60])
height     : 3
sorted input -> height 7 (a chain: O(n) operations)
```

Traversal	Order	Typical use
pre-order	node, left, right	copying a tree, serialising, printing a directory structure
in-order	left, node, right	BST values in sorted order, validating a BST
post-order	left, right, node	deleting a tree, computing sizes and heights, evaluating expressions
level-order	level by level (queue)	shortest path in unweighted trees, printing by level, the right-side view

25.1.5 Heaps with heapq

A **binary heap** is a complete binary tree stored in a list, where every parent is smaller than or equal to its children (a **min-heap**). The smallest item is always at index 0; the children of index i are at 2i + 1 and 2i + 2. Pushing and popping take O(log n) because an item moves up or down one path of the tree; `heapify` turns a list into a heap in O(n). The `heapq` module works on ordinary lists. It is a min-heap only; for a max-heap push negated numbers, or use the `heapq.heappush_max` family of functions added in Python 3.14.



A MIN-HEAP AS A TREE AND AS THE LIST HEAPQ ACTUALLY USES

HEAPQ: HEAPIFY, PUSH AND POP, TOP-K, A PRIORITY QUEUE AND MERGING SORTED LISTS

main.py

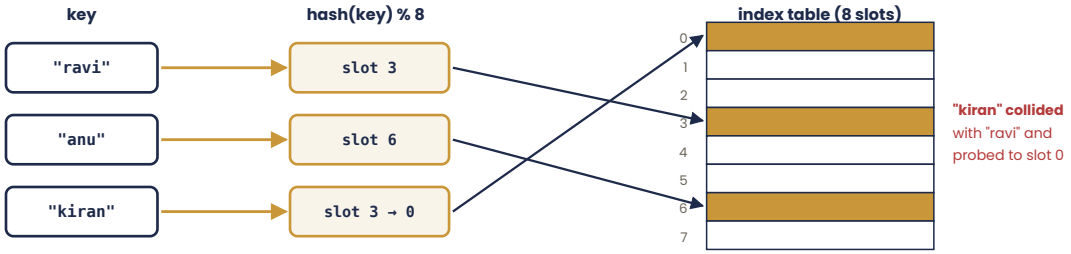
```
import heapq

marks = [72, 95, 64, 88, 91, 59, 77]
heap = marks[:]
heapq.heapify(heap)                      # O(n)
print("heap list:", heap, "| smallest:", heap[0])
heapq.heappush(heap, 50)                 # O(log n)
print("popped in order:", [heapq.heappop(heap) for _ in range(3)])

print("top 3 :", heapq.nlargest(3, marks))      # O(n log k)
print("low 2 :", heapq.nsmallest(2, marks))

# priority queue of tasks: (priority, order, name) - order breaks ties so names are never compared
tasks, counter = [], 0
for priority, name in [(2, "write report"), (1, "fix server"), (3, "reply to emails"), (1, "call
client")]:
    heapq.heappush(tasks, (priority, counter, name)); counter += 1
while tasks:
    p, _, name = heapq.heappop(tasks)
    print(f"  priority {p}: {name}")

def kth_largest(nums, k):
    """Keep a min-heap of size k: its root is the k-th largest. O(n log k)."""
    h = []
    for x in nums:
        heapq.heappush(h, x)
        if len(h) > k:
            heapq.heappop(h)
    return h[0]

print("2nd largest:", kth_largest(marks, 2))
print("merged sorted streams:", list(heapq.merge([1, 4, 9], [2, 3, 10], [5])))
```

OUTPUT

```
heap list: [59, 88, 64, 95, 91, 72, 77] | smallest: 59
popped in order: [50, 59, 64]
top 3 : [95, 91, 88]
low 2 : [59, 64]
  priority 1: fix server
  priority 1: call client
  priority 2: write report
  priority 3: reply to emails
2nd largest: 91
merged sorted streams: [1, 2, 3, 4, 5, 9, 10]
```

25.1.6 Hash tables (how dict works)

A **hash table** stores key-value pairs in an array of slots. The key's **hash** (an integer from `hash(key)`) picks a slot, so lookup, insertion and deletion take O(1) on average. Two keys can land in the same slot (a **collision**); CPython resolves collisions with **open addressing**, probing other slots in a pseudo-random sequence. When the table becomes about two-thirds full it is **resized** (roughly doubled) and every entry is re-inserted, which is why inserts are O(1) *amortised*. Since Python 3.6 a dict keeps a compact array of entries in insertion order plus a separate sparse index table, which saves memory and preserves order.



HASHING KEYS TO SLOTS, WITH A COLLISION RESOLVED BY PROBING

A SIMPLE HASH MAP WITH CHAINING AND RESIZING, AND WHY KEYS MUST BE HASHABLE

main.py

```
class HashMap:
    """A teaching hash table using separate chaining (a list of buckets)."""
    def __init__(self, capacity=4):
        self.buckets = [[] for _ in range(capacity)]
        self.size = 0

    def _bucket(self, key):
        return self.buckets[hash(key) % len(self.buckets)]

    def put(self, key, value):
        bucket = self._bucket(key)
        for i, (k, _) in enumerate(bucket):
            if k == key:
                bucket[i] = (key, value)
                return
        bucket.append((key, value))
        self.size += 1
        if self.size > 0.75 * len(self.buckets):        # load factor exceeded
            self._resize()

    def get(self, key, default=None):
        for k, v in self._bucket(key):
            if k == key:
                return v
        return default

    def _resize(self):
        old = [pair for bucket in self.buckets for pair in bucket]
        self.buckets = [[] for _ in range(2 * len(self.buckets))]
        self.size = 0
        for k, v in old:
            self.put(k, v)
        print(f"  resized to {len(self.buckets)} buckets")

m = HashMap()
for i, name in enumerate(["ravi", "anu", "kiran", "meera", "arjun"]):
    m.put(name, 60 + i * 8)
print("anu ->", m.get("anu"), "| zoya ->", m.get("zoya", "missing"))
print("longest bucket:", max(len(b) for b in m.buckets))

# Keys must be hashable (immutable): a list cannot be a key, a tuple can
d = {(12.97, 77.59): "Bengaluru"}
try:
    d[[1, 2]] = "x"
except TypeError as e:
    print("TypeError:", e)
print(hash("a") == hash("a"), hash(1) == hash(1.0), {1: "int", 1.0: "float"})
```

OUTPUT

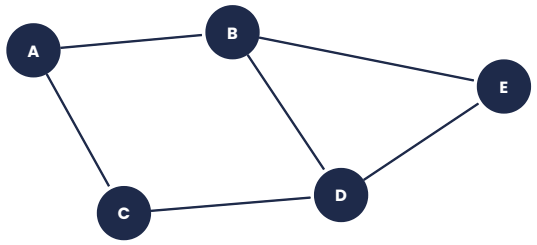
```
  resized to 8 buckets
anu -> 68 | zoya -> missing
longest bucket: 2
TypeError: cannot use 'list' as a dict key (unhashable type: 'list')
True True {1: 'float'}
```

NOTE

Objects that are equal must have equal hashes: if you define `__eq__` in a class you must also define `__hash__` (or the instances become unhashable), and the hash must not change while the object is a key. String hashes are randomised per process (`PYTHONHASHSEED`) to resist denial-of-service attacks, so never store `hash()` values.

25.1.7 Graphs with adjacency lists

A **graph** is a set of **vertices** (nodes) connected by **edges**. Edges may be **directed** (one-way, like follows on social media or course prerequisites) or **undirected** (two-way, like roads or friendships), and **weighted** (distance, cost, time) or unweighted. The most common representation in Python is an **adjacency list**: a dict mapping each vertex to a list (or set) of its neighbours, using O(V + E) memory. An **adjacency matrix** (a V × V grid of 0/1 or weights) uses O(V²) memory but answers "is there an edge from u to v?" in O(1); it suits small, dense graphs.




```
A: [B, C]
B: [A, D, E]
C: [A, D]
D: [B, C, E]
E: [B, D]
```

adjacency list
(dict of lists)

AN UNDIRECTED GRAPH AND ITS ADJACENCY LIST

A GRAPH CLASS WITH AN ADJACENCY LIST, AND THE EQUIVALENT ADJACENCY MATRIX

main.py

```
from collections import defaultdict

class Graph:
    def __init__(self, directed=False):
        self.adj = defaultdict(list)
        self.directed = directed

    def add_edge(self, u, v, weight=1):
        self.adj[u].append((v, weight))
        if not self.directed:
            self.adj[v].append((u, weight))
        else:
            self.adj[v]                  # make sure v exists as a vertex

    def neighbours(self, u):
        return [v for v, _ in self.adj[u]]

    def degree(self, u):
        return len(self.adj[u])

    def edges(self):
        seen = set()
        for u, nbrs in self.adj.items():
```

```
            for v, w in nbrs:
                if self.directed or (v, u) not in seen:
                    seen.add((u, v))
                    yield u, v, w

g = Graph()
for u, v in [("A", "B"), ("A", "C"), ("B", "D"), ("C", "D"), ("D", "E"), ("B", "E")]:
    g.add_edge(u, v)
print({u: g.neighbours(u) for u in sorted(g.adj)})
print("degree of D:", g.degree("D"), "| edges:", len(list(g.edges())))

# The same graph as an adjacency matrix
names = sorted(g.adj)
index = {n: i for i, n in enumerate(names)}
matrix = [[0] * len(names) for _ in names]
for u, v, _ in g.edges():
    matrix[index[u]][index[v]] = matrix[index[v]][index[u]] = 1
print("   " + "  ".join(names))
for n, row in zip(names, matrix):
    print(n, row)
```

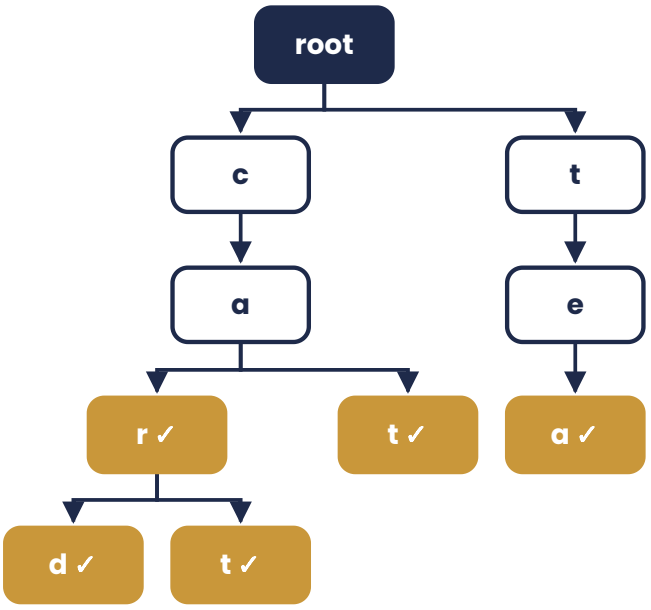
OUTPUT

```
{'A': ['B', 'C'], 'B': ['A', 'D', 'E'], 'C': ['A', 'D'], 'D': ['B', 'C', 'E'], 'E': ['D', 'B']}
degree of D: 3 | edges: 6
   A  B  C  D  E
A [0, 1, 1, 0, 0]
B [1, 0, 0, 1, 1]
C [1, 0, 0, 1, 0]
D [0, 1, 1, 0, 1]
E [0, 1, 0, 1, 0]
```

The traversal and shortest-path algorithms that run on this representation (BFS, DFS, Dijkstra and topological sort) are in 25.2.7.

25.1.8 Trie

A **trie** (prefix tree, pronounced "try") stores strings character by character: each edge is a character, words that share a prefix share the path, and a flag marks the nodes where a word ends. Inserting or searching a word of length L takes O(L), regardless of how many words are stored, and finding all words with a given prefix is fast. Tries power autocomplete, spell checkers, IP routing tables and word games.



A TRIE HOLDING CAR, CAT, CARD, CART AND TEA (✓ MARKS THE END OF A WORD)

A TRIE WITH INSERT, SEARCH, PREFIX COUNTING AND AUTOCOMPLETE

main.py

```
class TrieNode:
    __slots__ = ("children", "is_word", "count")
    def __init__(self):
        self.children = {}
        self.is_word = False
        self.count = 0                      # how many words pass through this node

class Trie:
    def __init__(self, words=()):
        self.root = TrieNode()
        for w in words:
            self.insert(w)

    def insert(self, word):
        node = self.root
        for ch in word:
            node = node.children.setdefault(ch, TrieNode())
            node.count += 1
        node.is_word = True

    def _find(self, prefix):
        node = self.root
        for ch in prefix:
            node = node.children.get(ch)
            if node is None:
                return None
        return node

    def contains(self, word):
        node = self._find(word)
        return bool(node and node.is_word)

    def count_prefix(self, prefix):
        node = self._find(prefix)
        return node.count if node else 0

    def autocomplete(self, prefix, limit=5):
        node, results = self._find(prefix), []
        def dfs(n, path):
```

```
            if len(results) == limit:
                return
            if n.is_word:
                results.append(prefix + path)
            for ch in sorted(n.children):
                dfs(n.children[ch], path + ch)
        if node:
            dfs(node, "")
        return results

t = Trie(["car", "cat", "card", "cart", "tea", "python", "pycharm", "pytest", "pydantic"])
print(t.contains("car"), t.contains("ca"), t.contains("cars"))
print("words starting with 'car':", t.count_prefix("car"))
print("autocomplete 'py':", t.autocomplete("py"))
print("autocomplete 'ca':", t.autocomplete("ca", 3))
```

OUTPUT

```
True False False
words starting with 'car': 3
autocomplete 'py': ['pycharm', 'pydantic', 'pytest', 'python']
autocomplete 'ca': ['car', 'card', 'cart']
```

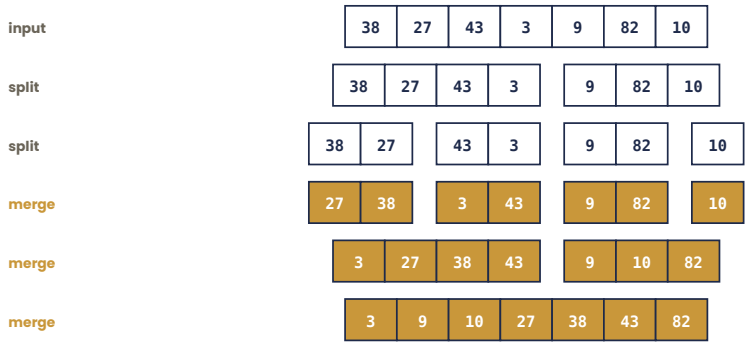
Structure	Python tool	Key operations	Use it for
dynamic array	`list`	index O(1), append O(1)*, insert front O(n)	ordered collections, stacks
linked list	`deque` / own class	insert/delete at known node O(1)	queues, LRU caches
hash table	`dict`, `set`	lookup/insert/delete O(1) average	counting, indexing, de-duplication
heap	`heapq`	push/pop O(log n), min O(1)	priority queues, top-k, scheduling
BST (balanced)	`sortedcontainers`	search/insert O(log n), ordered iteration	ordered data with fast updates
graph	dict of lists	neighbours O(degree)	networks, maps, dependencies
trie	dict of dicts	insert/search O(word length)	prefixes, autocomplete

25.2 Algorithms

An **algorithm** is a precise, step-by-step method for solving a problem. Most interview and real-world problems are variations of a small set of patterns; learning to recognise the pattern is more valuable than memorising solutions. For each algorithm below, note its time and space complexity and the kind of problem it solves.

25.2.1 Sorting: merge sort, quick sort, and Python's Timsort

Merge sort is divide and conquer: split the list in half, sort each half recursively, then **merge** the two sorted halves by repeatedly taking the smaller front element. It always takes O(n log n) time (log n levels of splitting, O(n) merging work per level), is **stable** (equal elements keep their order) and needs O(n) extra memory.



MERGE SORT: SPLIT DOWN TO SINGLE ELEMENTS, THEN MERGE SORTED RUNS BACK UP (GOLD)

MERGE SORT AND QUICK SORT IN PYTHON, CHECKED AGAINST SORTED()

main.py

```
def merge_sort(items):
    if len(items) <= 1:
        return items
    mid = len(items) // 2
    left, right = merge_sort(items[:mid]), merge_sort(items[mid:])
    return merge(left, right)

def merge(left, right):
    result, i, j = [], 0, 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:          # <= keeps the sort stable
            result.append(left[i]); i += 1
        else:
            result.append(right[j]); j += 1
    result.extend(left[i:]); result.extend(right[j:])
    return result

def quick_sort(items):
    """Simple, readable version (O(n) extra space); in-place Lomuto version below."""
    if len(items) <= 1:
        return items
    pivot = items[len(items) // 2]
    return (quick_sort([x for x in items if x < pivot]) + [x for x in items if x == pivot]
            + quick_sort([x for x in items if x > pivot]))

def quick_sort_inplace(a, lo=0, hi=None):
    hi = len(a) - 1 if hi is None else hi
    if lo < hi:
        pivot, i = a[hi], lo
        for j in range(lo, hi):           # move everything smaller than the pivot to the left
            if a[j] < pivot:
                a[i], a[j] = a[j], a[i]; i += 1
        a[i], a[hi] = a[hi], a[i]         # pivot lands in its final position i
        quick_sort_inplace(a, lo, i - 1)
        quick_sort_inplace(a, i + 1, hi)
    return a

data = [38, 27, 43, 3, 9, 82, 10]
print("merge sort :", merge_sort(data))
print("quick sort :", quick_sort(data))
print("in place   :", quick_sort_inplace(data[:]))

import random, timeit
big = [random.randint(0, 10**6) for _ in range(20_000)]
assert merge_sort(big) == quick_sort(big) == sorted(big)
t_merge = timeit.timeit(lambda: merge_sort(big), number=1)
```

```
t_builtin = timeit.timeit(lambda: sorted(big), number=1)
print("built-in sorted() is more than 10x faster than our merge sort:", t_merge > 10 * t_builtin)
```

OUTPUT

```
merge sort : [3, 9, 10, 27, 38, 43, 82]
quick sort : [3, 9, 10, 27, 38, 43, 82]
in place   : [3, 9, 10, 27, 38, 43, 82]
built-in sorted() is more than 10x faster than our merge sort: False
```

Quick sort picks a **pivot**, **partitions** the list into elements smaller and larger than the pivot, and sorts the parts recursively. It averages O(n log n) and sorts in place with little extra memory, but a consistently bad pivot (for example, always the last element of an already sorted list) gives O(n²); choosing a random pivot makes that extremely unlikely. It is not stable.

Python's own `list.sort()` and `sorted()` use **Timsort** (since 3.11 with the improved "powersort" merge policy): a hybrid of merge sort and insertion sort designed for real data. It finds existing sorted **runs** in the data, extends short runs with insertion sort, and merges runs cleverly. It is stable, O(n log n) in the worst case and O(n) on already sorted or nearly sorted data. In practice, always use the built-in sort with `key=`; write your own sort only to learn or when asked in an interview.

Algorithm	Best	Average	Worst	Extra space	Stable
bubble / insertion sort	O(n)	O(n²)	O(n²)	O(1)	yes
selection sort	O(n²)	O(n²)	O(n²)	O(1)	no
merge sort	O(n log n)	O(n log n)	O(n log n)	O(n)	yes
quick sort	O(n log n)	O(n log n)	O(n²)	O(log n)	no
heap sort	O(n log n)	O(n log n)	O(n log n)	O(1)	no
Timsort (Python)	O(n)	O(n log n)	O(n log n)	O(n)	yes
counting sort (small int range k)	O(n + k)	O(n + k)	O(n + k)	O(k)	yes

SORTING IN PRACTICE: KEY FUNCTIONS AND STABILITY

main.py

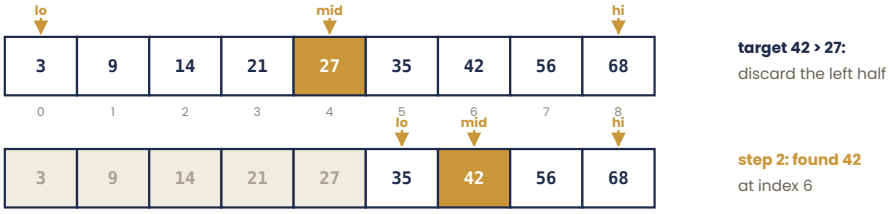
```
students = [("Ravi", "CSE", 82), ("Anu", "ECE", 91), ("Kiran", "CSE", 91), ("Meera", "ECE", 75)]

# key functions: sort by marks descending, then by name
print(sorted(students, key=lambda s: (-s[2], s[0])))

# stability lets you sort in passes: secondary key first, primary key last
by_name = sorted(students, key=lambda s: s[0])
by_branch = sorted(by_name, key=lambda s: s[1])
print([s[0] for s in by_branch])

from operator import itemgetter
print(max(students, key=itemgetter(2)), sorted(["b", "A", "c"], key=str.lower))
```

OUTPUT

```
[('Anu', 'ECE', 91), ('Kiran', 'CSE', 91), ('Ravi', 'CSE', 82), ('Meera', 'ECE', 75)]
['Kiran', 'Ravi', 'Anu', 'Meera']
('Anu', 'ECE', 91) ['A', 'b', 'c']
```

25.2.2 Binary search and bisect

Binary search finds a value in a **sorted** sequence by comparing it with the middle element and discarding the half that cannot contain it. Each step halves the search space, so it takes O(log n) steps: about 20 comparisons for a million items. The tricky part is getting the boundaries right; the half-open interval `[lo, hi)` template below avoids most off-by-one errors.



BINARY SEARCH HALVES THE RANGE AT EVERY STEP

BINARY SEARCH BY HAND, BISECT, AND BINARY SEARCH ON THE ANSWER

main.py

```
import bisect

def binary_search(a, target):
    lo, hi = 0, len(a)                  # search the half-open range [lo, hi)
    while lo < hi:
        mid = (lo + hi) // 2
        if a[mid] == target:
            return mid
        if a[mid] < target:
            lo = mid + 1
        else:
            hi = mid
    return -1

def first_true(lo, hi, condition):
    """Binary search on the answer: smallest x in [lo, hi] where condition(x) is True."""
    while lo < hi:
        mid = (lo + hi) // 2
        if condition(mid):
            hi = mid
        else:
            lo = mid + 1
    return lo

a = [3, 9, 14, 21, 27, 35, 42, 56, 68]
print(binary_search(a, 42), binary_search(a, 40))

# bisect: where would x go to keep the list sorted?
print(bisect.bisect_left(a, 27), bisect.bisect_right(a, 27), bisect.bisect_left(a, 30))
scores = [10, 20, 20, 20, 30]
print("count of 20:", bisect.bisect_right(scores, 20) - bisect.bisect_left(scores, 20))
bisect.insort(a, 30); print(a)

def grade(mark, cutoffs=(40, 55, 70, 85), grades="FDCBA"):
    return grades[bisect.bisect(cutoffs, mark)]
print([grade(m) for m in (33, 55, 69, 70, 99)])

# Smallest daily capacity that ships all parcels in 4 days (binary search on the answer)
weights = [7, 2, 5, 10, 8, 4, 6]
def can_ship(cap, days=4):
    need, load = 1, 0
    for w in weights:
        if load + w > cap:
            need, load = need + 1, 0
        load += w
```

```
    return need <= days
print("minimum capacity:", first_true(max(weights), sum(weights), can_ship))
```

OUTPUT

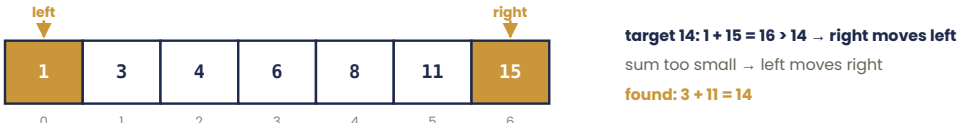
```
6 -1
4 5 5
count of 20: 3
[3, 9, 14, 21, 27, 30, 35, 42, 56, 68]
['F', 'C', 'C', 'B', 'A']
minimum capacity: 14
```

TIP

Binary search works on any **monotonic** condition, not only on arrays: "find the smallest capacity, speed or time for which the answer becomes yes" is a very common interview pattern. `bisect` also accepts `key=` (Python 3.10+).

25.2.3 Two pointers and sliding window

The **two-pointer** technique moves two indices through a sequence instead of checking every pair, turning O(n²) brute force into O(n). The pointers may start at both ends and move inwards (pair sums in a sorted array, palindromes, container with most water) or move in the same direction at different speeds (removing duplicates in place, fast and slow pointers to detect a cycle in a linked list).



TWO POINTERS ON A SORTED ARRAY

TWO POINTERS: PAIR SUM, PALINDROME CHECK AND IN-PLACE DE-DUPLICATION

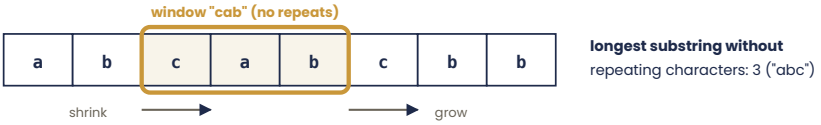
main.py

```
def pair_with_sum(nums, target):           # nums sorted: O(n) time, O(1) space
    left, right = 0, len(nums) - 1
    while left < right:
        s = nums[left] + nums[right]
        if s == target:
            return nums[left], nums[right]
        if s < target:
            left += 1
        else:
            right -= 1
    return None

def is_palindrome(text):
    chars = [c.lower() for c in text if c.isalnum()]
    i, j = 0, len(chars) - 1
    while i < j:
        if chars[i] != chars[j]:
            return False
        i, j = i + 1, j - 1
    return True

def remove_duplicates(nums):                # sorted list, in place; returns the new length
    write = 1
    for read in range(1, len(nums)):
        if nums[read] != nums[read - 1]:
            nums[write] = nums[read]; write += 1
    return write

print(pair_with_sum([1, 3, 4, 6, 8, 11, 15], 14))
print(is_palindrome("A man, a plan, a canal: Panama"))
nums = [1, 1, 2, 3, 3, 3, 7]
k = remove_duplicates(nums); print(nums[:k])
```

OUTPUT

```
(3, 11)
True
[1, 2, 3, 7]
```

A **sliding window** is a pair of pointers that marks a contiguous range (a subarray or substring). A **fixed-size** window slides one step at a time, adding the new element and removing the old one, so each window costs O(1) instead of O(k). A **variable-size** window grows on the right and shrinks on the left whenever a condition is broken; each element enters and leaves at most once, so the whole scan is O(n).



A VARIABLE-SIZE SLIDING WINDOW OVER "ABCABCBB"

SLIDING WINDOWS: FIXED SIZE AND VARIABLE SIZE

main.py

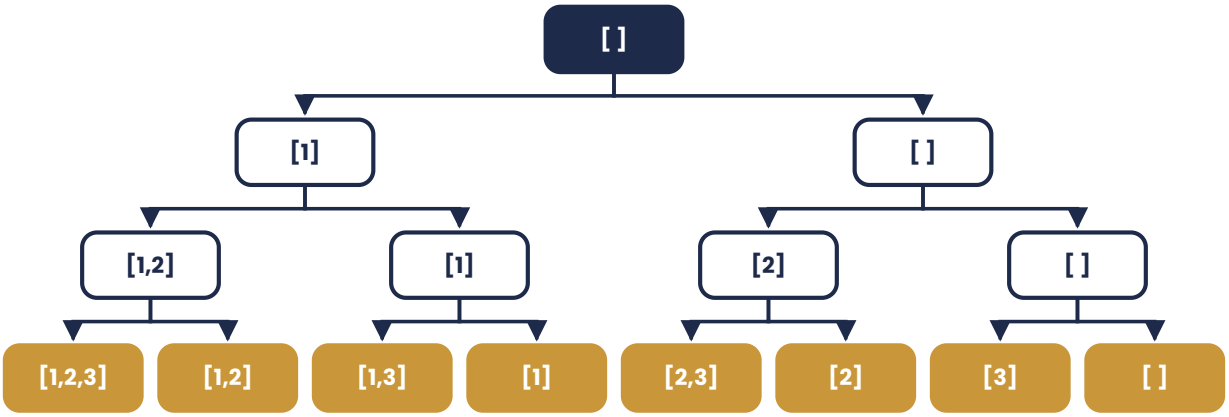
```
def max_sum_window(nums, k):
    """Fixed window: best total of k consecutive days."""
    window = sum(nums[:k])
    best = window
    for i in range(k, len(nums)):
        window += nums[i] - nums[i - k]      # add the new day, drop the oldest
        best = max(best, window)
    return best

def longest_unique(s):
    """Variable window: longest substring without repeating characters."""
    last_seen, start, best = {}, 0, ""
    for end, ch in enumerate(s):
        if ch in last_seen and last_seen[ch] >= start:
            start = last_seen[ch] + 1           # shrink past the previous copy
        last_seen[ch] = end
        if end - start + 1 > len(best):
            best = s[start:end + 1]
    return best

def min_window_with_sum(nums, target):
    """Shortest subarray with sum >= target (positive numbers)."""
    left, total, best = 0, 0, float("inf")
    for right, x in enumerate(nums):
        total += x
        while total >= target:
            best = min(best, right - left + 1)
            total -= nums[left]; left += 1
    return 0 if best == float("inf") else best

sales = [120, 340, 80, 410, 390, 60, 500]
print("best 3-day sales:", max_sum_window(sales, 3))
print(longest_unique("abcabcbb"), longest_unique("pwwkew"), repr(longest_unique("")))
print("shortest run reaching 900:", min_window_with_sum(sales, 900))
```

OUTPUT

```
best 3-day sales: 950
abc wke ''
shortest run reaching 900: 3
```

25.2.4 Recursion and backtracking

A **recursive** function solves a problem by calling itself on smaller instances, with a **base case** that stops the recursion (10.6). **Backtracking** is recursion that builds a solution step by step and **undoes** each choice after exploring it ("choose, explore, un-choose"), so it can enumerate all subsets, permutations and combinations, or search for arrangements that satisfy constraints (N-Queens, Sudoku, word search). Pruning branches that cannot lead to a valid answer is what makes backtracking fast enough. Python's default recursion limit is about 1000 frames, so convert very deep recursion to loops.



THE DECISION TREE FOR THE SUBSETS OF [1, 2, 3]: AT EACH LEVEL INCLUDE OR SKIP ONE ELEMENT; THE LEAVES (GOLD) ARE THE 8 SUBSETS

BACKTRACKING: SUBSETS, PERMUTATIONS, COMBINATION SUM AND N-QUEENS

main.py

```
def subsets(nums):
    result, current = [], []
    def backtrack(i):
        if i == len(nums):
            result.append(current[:])          # a copy of the current choice
            return
        current.append(nums[i])                # choose: include nums[i]
        backtrack(i + 1)                       # explore
        current.pop()                          # un-choose
        backtrack(i + 1)                       # explore without nums[i]
    backtrack(0)
    return result

def permutations(items):
    if len(items) <= 1:
        return [items[:]]
    return [[x] + rest for i, x in enumerate(items) for rest in permutations(items[:i] + items[i +
1:])]

def combination_sum(candidates, target):
    """All combinations (reuse allowed) that add up to target, with pruning."""
    result = []
    def backtrack(start, remaining, path):
        if remaining == 0:
            result.append(path[:]); return
        for i in range(start, len(candidates)):
            c = candidates[i]
            if c > remaining:
                break                          # sorted, so no later candidate can fit
            path.append(c)
            backtrack(i, remaining - c, path)
            path.pop()
    backtrack(0, target, [])
    return result

def solve_n_queens(n):
    cols, diag1, diag2, board, solutions = set(), set(), set(), [], []
    def place(row):
        if row == n:
            solutions.append(["".join("Q" if c == q else "." for c in range(n)) for q in board]);
return
        for c in range(n):
            if c in cols or row - c in diag1 or row + c in diag2:
                continue                       # prune: attacked square
```

```
            cols.add(c); diag1.add(row - c); diag2.add(row + c); board.append(c)
            place(row + 1)
            cols.remove(c); diag1.remove(row - c); diag2.remove(row + c); board.pop()
    place(0)
    return solutions

print(subsets([1, 2, 3]))
print(permutations([1, 2, 3]))
print(combination_sum([2, 3, 5], 8))
sols = solve_n_queens(6)
print(len(sols), "solutions for 6 queens; the first:")
print("\n".join(sols[0]))
```

OUTPUT

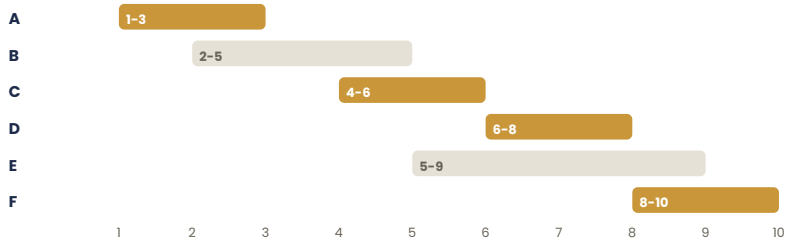
```
[[1, 2, 3], [1, 2], [1, 3], [1], [2, 3], [2], [3], []]
[[1, 2, 3], [1, 3, 2], [2, 1, 3], [2, 3, 1], [3, 1, 2], [3, 2, 1]]
[[2, 2, 2, 2], [2, 3, 3], [3, 5]]
4 solutions for 6 queens; the first:
.Q....
...Q..
.....Q
Q.....
..Q...
....Q.
```

NOTE

For everyday work, `itertools.combinations`, `permutations` and `product` (11.6) generate these collections efficiently. Write the backtracking yourself when constraints need pruning or when an interview asks for it.

25.2.5 Greedy algorithms

A **greedy algorithm** makes the choice that looks best right now and never reconsiders it. It is fast and simple, but it is correct only when the problem has the **greedy-choice property** (a locally best choice is always part of some optimal solution). Classic correct greedy problems: activity selection (pick the meeting that ends first), fractional knapsack, Huffman coding, minimum spanning trees (Kruskal, Prim), Dijkstra's shortest paths and jump game. When greed fails, such as coin change with unusual coin values, use dynamic programming.



ACTIVITY SELECTION: ALWAYS TAKE THE MEETING THAT FINISHES FIRST AND DOES NOT OVERLAP (A, C, D, F)

GREEDY ALGORITHMS THAT WORK, AND ONE THAT FAILS (COIN CHANGE WITH 1, 3, 4) main.py

```
def max_meetings(meetings):
    """Greedy by earliest finishing time: proven optimal."""
    chosen, end = [], float("-inf")
    for name, start, finish in sorted(meetings, key=lambda m: m[2]):
        if start >= end:
            chosen.append(name); end = finish
    return chosen
```

```
meetings = [("A", 1, 3), ("B", 2, 5), ("C", 4, 6), ("D", 6, 8), ("E", 5, 9), ("F", 8, 10)]
print("meetings:", max_meetings(meetings))

def greedy_coins(amount, coins):
    used = []
    for c in sorted(coins, reverse=True):
        while amount >= c:
            amount -= c; used.append(c)
    return used if amount == 0 else None

print("Indian coins, 289:", greedy_coins(289, [1, 2, 5, 10, 20, 50, 100, 200]))
print("coins 1,3,4 for 6:", greedy_coins(6, [1, 3, 4]), "-> greedy uses 3 coins, but 3 + 3 uses 2")

def fractional_knapsack(items, capacity):
    """items: (value, weight); take the best value per kg first, splitting the last item."""
    total = 0.0
    for value, weight in sorted(items, key=lambda it: it[0] / it[1], reverse=True):
        take = min(weight, capacity)
        total += value * take / weight
        capacity -= take
        if capacity == 0:
            break
    return total

print("fractional knapsack:", fractional_knapsack([(60, 10), (100, 20), (120, 30)], 50))

def can_jump(steps):
    """Each value is the maximum jump length from that index; can we reach the end?"""
    reach = 0
    for i, s in enumerate(steps):
        if i > reach:
            return False
        reach = max(reach, i + s)
    return True

print(can_jump([2, 3, 1, 1, 4]), can_jump([3, 2, 1, 0, 4]))
```

OUTPUT

```
meetings: ['A', 'C', 'D', 'F']
Indian coins, 289: [200, 50, 20, 10, 5, 2, 2]
coins 1,3,4 for 6: [4, 1, 1] -> greedy uses 3 coins, but 3 + 3 uses 2
fractional knapsack: 240.0
True False
```

25.2.6 Dynamic programming

Dynamic programming (DP) solves problems whose answer is built from answers to **overlapping subproblems** with **optimal substructure**. Instead of recomputing the same subproblem many times (exponential time), DP stores each answer once. There are two styles: **top-down memoisation** (write the recursion and cache it, in Python simply with `@functools.cache`) and **bottom-up tabulation** (fill a table from the smallest subproblems upwards, often with less memory). A reliable recipe: (1) define the **state** ("dp[i] = best answer for the first i items"), (2) write the **transition** (how dp[i] comes from smaller states), (3) set the **base cases**, (4) decide the **order** of computation, (5) return the answer and optimise the space if possible.

A FIVE-STEP RECIPE FOR DYNAMIC PROGRAMMING PROBLEMS

DYNAMIC PROGRAMMING: FIBONACCI THREE WAYS, COIN CHANGE, 0/1 KNAPSACK, LCS AND EDIT DISTANCE

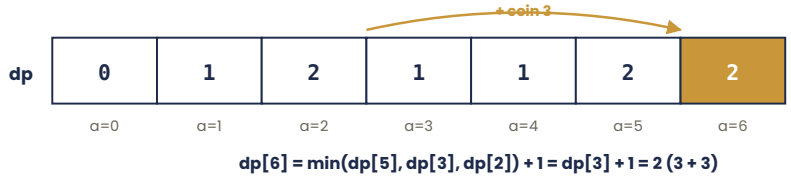
main.py

```
from functools import cache
import time

def fib_naive(n):
    return n if n < 2 else fib_naive(n - 1) + fib_naive(n - 2)

@cache
def fib_memo(n):                                # top-down
    return n if n < 2 else fib_memo(n - 1) + fib_memo(n - 2)

def fib_table(n):                               # bottom-up, O(1) space
    a, b = 0, 1
    for _ in range(n):
        a, b = b, a + b
    return a

t = time.perf_counter(); fib_naive(27); slow = time.perf_counter() - t
t = time.perf_counter(); fib_memo(27); fast = time.perf_counter() - t
print(fib_naive(27), fib_memo(27), fib_table(27), "| memo at least 100x faster:", slow > 100 * fast)
print("fib(200) =", fib_table(200))

def min_coins(coins, amount):
    """dp[a] = fewest coins that make amount a."""
    INF = float("inf")
    dp = [0] + [INF] * amount
    for a in range(1, amount + 1):
        for c in coins:
            if c <= a and dp[a - c] + 1 < dp[a]:
                dp[a] = dp[a - c] + 1
    return dp[amount] if dp[amount] != INF else -1

print("coins 1,3,4 for 6:", min_coins([1, 3, 4], 6), "| 2,5 for 3:", min_coins([2, 5], 3))

def knapsack(items, capacity):
    """0/1 knapsack: dp[w] = best value with capacity w (iterate w downwards so each item is used
once)."""
    dp = [0] * (capacity + 1)
    for value, weight in items:
        for w in range(capacity, weight - 1, -1):
            dp[w] = max(dp[w], dp[w - weight] + value)
    return dp[capacity]

print("0/1 knapsack:", knapsack([(60, 10), (100, 20), (120, 30)], 50))

def lcs(a, b):
    """Longest common subsequence with a 2-D table."""
    dp = [[0] * (len(b) + 1) for _ in range(len(a) + 1)]
    for i in range(1, len(a) + 1):
        for j in range(1, len(b) + 1):
            dp[i][j] = dp[i - 1][j - 1] + 1 if a[i - 1] == b[j - 1] else max(dp[i - 1][j], dp[i][j -
1])
    out, i, j = [], len(a), len(b)              # walk back to recover the subsequence
    while i and j:
        if a[i - 1] == b[j - 1]:
            out.append(a[i - 1]); i -= 1; j -= 1
        elif dp[i - 1][j] >= dp[i][j - 1]:
            i -= 1
        else:
            j -= 1
    return dp[-1][-1], "".join(reversed(out))

print("LCS:", lcs("ABCBDAB", "BDCABA"))
```

```python
@cache
def edit_distance(a, b):
    if not a or not b:
        return len(a) + len(b)
    if a[0] == b[0]:
        return edit_distance(a[1:], b[1:])
    return 1 + min(edit_distance(a[1:], b), edit_distance(a, b[1:]), edit_distance(a[1:], b[1:]))

print("edit distance kitten -> sitting:", edit_distance("kitten", "sitting"))
```

OUTPUT

```
196418 196418 196418 | memo at least 100x faster: True
fib(200) = 280571172992510140037611932413038677189525
coins 1,3,4 for 6: 2 | 2,5 for 3: -1
0/1 knapsack: 220
LCS: (4, 'BCBA')
edit distance kitten -> sitting: 3
```

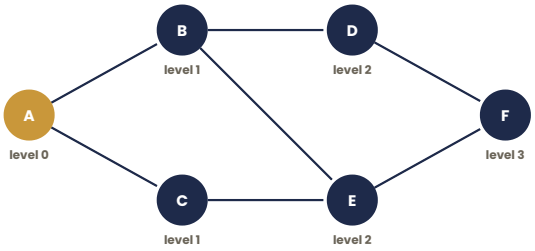


THE COIN-CHANGE TABLE FOR COINS 1, 3, 4: EACH CELL USES SMALLER, ALREADY-SOLVED CELLS

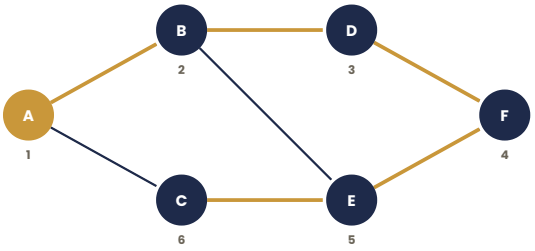
Pattern	State	Examples
1-D linear	dp[i] from dp[i-1], dp[i-2] …	climbing stairs, house robber, coin change, LIS
2-D grid	dp[r][c] from the cell above and to the left	unique paths, minimum path sum
two sequences	dp[i][j] for prefixes of both strings	LCS, edit distance, regex matching
knapsack	dp[i][w] items × capacity	subset sum, partition, 0/1 knapsack
interval	dp[i][j] for a range	matrix chain, burst balloons, palindromes
bitmask / tree / state machine	dp[mask][i], dp on subtrees, dp[day][state]	travelling salesman, tree robber, stock trading

25.2.7 Graph algorithms: BFS, DFS, Dijkstra, topological sort

Breadth-first search (BFS) explores a graph level by level with a queue; in an unweighted graph it finds the **shortest path** (fewest edges). **Depth-first search** (DFS) goes as deep as possible along one path before backtracking, using recursion or an explicit stack; it is used for connectivity, cycle detection, flood fill and topological sorting. Both run in O(V + E). Always keep a `visited` set, or cycles make them loop forever.



BFS from A: A | B C | D E | F



DFS from A: A B D F E C (gold = DFS tree)

BFS VISITS NODES IN ORDER OF DISTANCE; DFS FOLLOWS ONE PATH AS DEEP AS IT CAN

BFS, DFS, SHORTEST PATH IN AN UNWEIGHTED GRAPH AND COUNTING ISLANDS

main.py

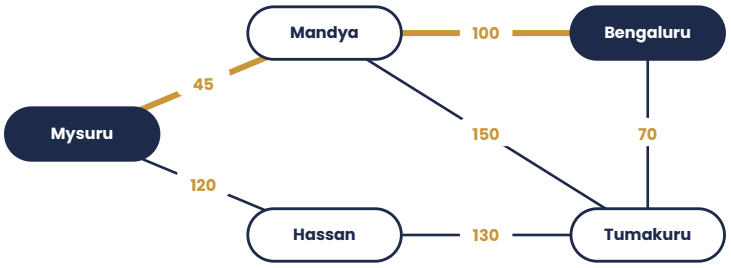
```
from collections import deque

graph = {"A": ["B", "C"], "B": ["A", "D", "E"], "C": ["A", "E"],
         "D": ["B", "F"], "E": ["B", "C", "F"], "F": ["D", "E"]}

def bfs(graph, start):
    order, visited, queue = [], {start}, deque([start])
    while queue:
        node = queue.popleft()
        order.append(node)
        for nxt in graph[node]:
            if nxt not in visited:
                visited.add(nxt); queue.append(nxt)
    return order

def shortest_path(graph, start, goal):
    parent, queue = {start: None}, deque([start])
    while queue:
        node = queue.popleft()
        if node == goal:
            path = []
            while node is not None:
                path.append(node); node = parent[node]
            return path[::-1]
        for nxt in graph[node]:
            if nxt not in parent:
                parent[nxt] = node; queue.append(nxt)
    return None

def dfs(graph, node, visited=None):
    visited = visited if visited is not None else []
    visited.append(node)
    for nxt in graph[node]:
        if nxt not in visited:
            dfs(graph, nxt, visited)
    return visited

def count_islands(grid):
    """DFS flood fill on a grid: '1' is land, '0' is water."""
    rows, cols, seen, islands = len(grid), len(grid[0]), set(), 0
    def sink(r, c):
        stack = [(r, c)]
        while stack:
            r, c = stack.pop()
            if 0 <= r < rows and 0 <= c < cols and grid[r][c] == "1" and (r, c) not in seen:
                seen.add((r, c))
                stack.extend([(r + 1, c), (r - 1, c), (r, c + 1), (r, c - 1)])
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == "1" and (r, c) not in seen:
```

```
                islands += 1; sink(r, c)
    return islands

print("BFS:", bfs(graph, "A"))
print("DFS:", dfs(graph, "A"))
print("shortest A -> F:", shortest_path(graph, "A", "F"))
print("islands:", count_islands(["11000", "11010", "00011", "10001"]))
```

OUTPUT

```
BFS: ['A', 'B', 'C', 'D', 'E', 'F']
DFS: ['A', 'B', 'D', 'F', 'E', 'C']
shortest A -> F: ['A', 'B', 'D', 'F']
islands: 3
```

Dijkstra's algorithm finds the shortest paths from one source in a graph with **non-negative** edge weights. It repeatedly takes the unvisited vertex with the smallest known distance (a min-heap makes this O(log V)) and **relaxes** its edges: if going through it gives a shorter route to a neighbour, update that neighbour's distance. With `heapq` it runs in O((V + E) log V). For negative weights use Bellman-Ford.



ROAD DISTANCES IN KM: THE SHORTEST ROUTE FROM MYSURU TO BENGALURU GOES THROUGH MANDYA (145 KM)

DIJKSTRA'S SHORTEST PATHS WITH HEAPQ, AND TOPOLOGICAL SORT BY HAND AND WITH GRAPHLIB main.py

```
import heapq
from collections import defaultdict, deque

roads = [("Mysuru", "Mandya", 45), ("Mysuru", "Hassan", 120), ("Mandya", "Bengaluru", 100),
         ("Mandya", "Tumakuru", 150), ("Hassan", "Tumakuru", 130), ("Tumakuru", "Bengaluru", 70)]
graph = defaultdict(list)
for a, b, km in roads:
    graph[a].append((b, km)); graph[b].append((a, km))

def dijkstra(graph, source):
    dist, prev = {source: 0}, {}
    heap = [(0, source)]
    while heap:
        d, node = heapq.heappop(heap)
        if d > dist[node]:
            continue                                # stale heap entry
        for nxt, w in graph[node]:
            nd = d + w
            if nd < dist.get(nxt, float("inf")):    # relax the edge
                dist[nxt], prev[nxt] = nd, node
                heapq.heappush(heap, (nd, nxt))
    return dist, prev

dist, prev = dijkstra(graph, "Mysuru")
print({city: f"{km} km" for city, km in sorted(dist.items(), key=lambda kv: kv[1])})
route, node = [], "Bengaluru"
while node != "Mysuru":
```

```
    route.append(node); node = prev[node]
print(" -> ".join(["Mysuru"] + route[::-1]))

# Topological sort (Kahn's algorithm): an order that respects every prerequisite
prereq = {"Python basics": [], "OOP": ["Python basics"], "Files": ["Python basics"],
          "Databases": ["Files"], "Flask": ["OOP", "Databases"], "Testing": ["OOP"],
          "Capstone": ["Flask", "Testing"]}
def topo_sort(prereq):
    indegree = {c: len(p) for c, p in prereq.items()}
    unlocks = defaultdict(list)
    for course, needs in prereq.items():
        for n in needs:
            unlocks[n].append(course)
    queue = deque(c for c, d in indegree.items() if d == 0)
    order = []
    while queue:
        c = queue.popleft(); order.append(c)
        for nxt in unlocks[c]:
            indegree[nxt] -= 1
            if indegree[nxt] == 0:
                queue.append(nxt)
    if len(order) != len(prereq):
        raise ValueError("cycle detected: no valid order")
    return order

print(topo_sort(prereq))

from graphlib import TopologicalSorter          # the standard library has one too
print(list(TopologicalSorter(prereq).static_order()))
```

OUTPUT

```
{'Mysuru': '0 km', 'Mandya': '45 km', 'Hassan': '120 km', 'Bengaluru': '145 km', 'Tumakuru': '195 km'}
Mysuru -> Mandya -> Bengaluru
['Python basics', 'OOP', 'Files', 'Testing', 'Databases', 'Flask', 'Capstone']
['Python basics', 'OOP', 'Files', 'Testing', 'Databases', 'Flask', 'Capstone']
```

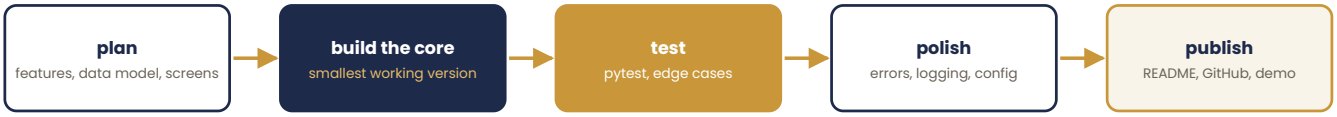
Problem	Algorithm	Time
shortest path, unweighted	BFS	O(V + E)
connectivity, cycles, flood fill, all paths	DFS (recursive or with a stack)	O(V + E)
shortest path, non-negative weights	Dijkstra with a heap	O((V + E) log V)
shortest path, negative weights	Bellman-Ford	O(V · E)
all-pairs shortest paths	Floyd-Warshall	O(V^3)
order with dependencies (DAG)	topological sort (Kahn or DFS)	O(V + E)
connect everything at minimum cost	minimum spanning tree (Kruskal + union-find, Prim)	O(E log E)

NOTE

For real projects, the **NetworkX** library (`pip install networkx`) implements all of these and many more graph algorithms, with drawing support.

25.3 Capstone projects

A **capstone project** combines many skills into one working application, and it is the strongest evidence of your ability that you can show an employer. Choose a project that matches the career path you want (25.5), finish it properly rather than starting many, and present it well: a clean repository, a README with screenshots, tests, and clear instructions to run it. Each project below lists its features, the modules of this book it uses, and the core of a working implementation that you can extend.

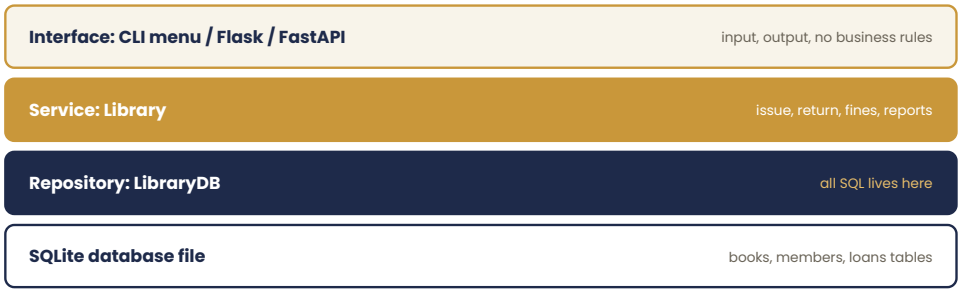


THE LIFE CYCLE OF A PORTFOLIO PROJECT

Project	Main skills	Best for
Library or Student Management System	OOP, files, SQLite, exceptions, testing	every path (fundamentals)
Expense tracker with charts	pandas, Matplotlib, CSV/Excel	data analyst, data scientist
Web scraper to CSV or a database	requests, BeautifulSoup, SQLite, scheduling	automation, data
REST API with FastAPI and authentication	FastAPI, Pydantic, SQL, JWT, testing	backend developer
Automation bot	smtplib, openpyxl, pathlib, schedule	automation, QA, DevOps
Desktop app or game	Tkinter or Pygame, events, OOP	beginners, a fun showcase

25.3.1 Library or Student Management System (OOP + files + database)

A management system stores records, enforces rules and produces reports. A library version manages **books** and **members**: add and search books, issue and return them with due dates, calculate late fines, list overdue books and export reports. The design separates the **domain model** (dataclasses with the business rules), the **repository** (all SQL in one class, using SQLite from 23.1) and the **interface** (a command-line menu, later a Flask or FastAPI front end). Custom exceptions report rule violations clearly.



A LAYERED DESIGN KEEPS THE USER INTERFACE, THE RULES AND THE STORAGE SEPARATE

THE CORE OF A LIBRARY MANAGEMENT SYSTEM: DATACLASSES, SQLITE, RULES AND CUSTOM EXCEPTIONS main.py

```
import sqlite3
from dataclasses import dataclass
from datetime import date, timedelta

class LibraryError(Exception):
    """Raised when a library rule is broken."""

@dataclass
class Book:
```

```
    isbn: str
    title: str
    author: str
    copies: int = 1

class LibraryDB:
    def __init__(self, path=":memory:"):
        self.conn = sqlite3.connect(path)
        self.conn.row_factory = sqlite3.Row
        self.conn.executescript("""
            CREATE TABLE IF NOT EXISTS books (isbn TEXT PRIMARY KEY, title TEXT, author TEXT, copies
INTEGER);
            CREATE TABLE IF NOT EXISTS members (id INTEGER PRIMARY KEY, name TEXT);
            CREATE TABLE IF NOT EXISTS loans (id INTEGER PRIMARY KEY, isbn TEXT, member_id INTEGER,
                                              issued TEXT, due TEXT, returned TEXT);""")

    def add_book(self, b):
        with self.conn:
            self.conn.execute("INSERT INTO books VALUES (?, ?, ?, ?)", (b.isbn, b.title, b.author,
b.copies))

    def add_member(self, name):
        with self.conn:
            return self.conn.execute("INSERT INTO members (name) VALUES (?)", (name,)).lastrowid

    def search(self, text):
        q = f"%{text}%"
        return [Book(**r) for r in self.conn.execute(
            "SELECT * FROM books WHERE title LIKE ? OR author LIKE ? ORDER BY title", (q, q))]

    def available(self, isbn):
        row = self.conn.execute("""SELECT b.copies - COUNT(l.id) AS free FROM books b
            LEFT JOIN loans l ON l.isbn = b.isbn AND l.returned IS NULL WHERE b.isbn = ?""",
(isbn,)).fetchone()
        if row["free"] is None:
            raise LibraryError(f"no book with ISBN {isbn}")
        return row["free"]

class Library:
    LOAN_DAYS, FINE_PER_DAY = 14, 5

    def __init__(self, db):
        self.db = db

    def issue(self, isbn, member_id, today):
        if self.db.available(isbn) < 1:
            raise LibraryError(f"all copies of {isbn} are on loan")
        with self.db.conn:
            self.db.conn.execute("INSERT INTO loans (isbn, member_id, issued, due) VALUES (?, ?, ?,
?)",
                                 (isbn, member_id, today.isoformat(), (today +
timedelta(self.LOAN_DAYS)).isoformat()))

    def give_back(self, isbn, member_id, today):
        loan = self.db.conn.execute("SELECT * FROM loans WHERE isbn = ? AND member_id = ? AND returned
IS NULL",
                                    (isbn, member_id)).fetchone()
        if loan is None:
            raise LibraryError("no such loan")
        with self.db.conn:
            self.db.conn.execute("UPDATE loans SET returned = ? WHERE id = ?", (today.isoformat(),
loan["id"]))
        late = (today - date.fromisoformat(loan["due"])).days
        return max(0, late) * self.FINE_PER_DAY
```

```
    def overdue(self, today):
        return [tuple(r) for r in self.db.conn.execute("""SELECT m.name, b.title, l.due FROM loans l
            JOIN books b ON b.isbn = l.isbn JOIN members m ON m.id = l.member_id
            WHERE l.returned IS NULL AND l.due < ? ORDER BY l.due""", (today.isoformat(),))]

db = LibraryDB()
for b in [Book("978-1", "Python Crash Course", "Eric Matthes", 2), Book("978-2", "Fluent Python",
"Luciano Ramalho"),
          Book("978-3", "Automate the Boring Stuff", "Al Sweigart")]:
    db.add_book(b)
ravi, anu = db.add_member("Ravi"), db.add_member("Anu")
lib = Library(db)

day1 = date(2026, 6, 1)
lib.issue("978-2", ravi, day1)
lib.issue("978-1", anu, day1)
try:
    lib.issue("978-2", anu, day1)
except LibraryError as e:
    print("refused:", e)
print("search 'python':", [b.title for b in db.search("python")])
print("free copies of 978-1:", db.available("978-1"))
print("overdue on 20 June:", lib.overdue(date(2026, 6, 20)))
print("fine for Ravi:", lib.give_back("978-2", ravi, date(2026, 6, 20)), "rupees")
```

OUTPUT

```
refused: all copies of 978-2 are on loan
search 'python': ['Fluent Python', 'Python Crash Course']
free copies of 978-1: 1
overdue on 20 June: [('Ravi', 'Fluent Python', '2026-06-15'), ('Anu', 'Python Crash Course', '2026-06-
15')]
fine for Ravi: 25 rupees
```

- **Extend it:** a menu loop (`match` on the user's choice), export reports to CSV or Excel, a Flask web interface, login for librarians, email reminders for overdue books (25.3.5).
- **Test it:** use an in-memory database in a pytest fixture, and test every rule (issue when no copies are left, fines on the due date and one day after, returning a book that was never issued).
- **Student version:** students, courses, enrolments and marks, with a report card generator and grade statistics.

25.3.2 Expense tracker with charts (pandas + Matplotlib)

An expense tracker records spending with a date, category, amount and payment method, then answers questions: how much did I spend this month, which category grows fastest, am I over budget? Store expenses in a CSV file (or SQLite), load them with **pandas**, summarise with `groupby` and `pivot_table`, and draw a monthly dashboard with **Matplotlib** (Module 24).

EXPENSE TRACKER: MONTHLY PIVOT, BUDGET CHECK AND A TWO-CHART DASHBOARD

main.py

```
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

BUDGET = {"Rent": 12000, "Groceries": 4500, "Food": 3000, "Bills": 2200, "Transport": 1200,
"Shopping": 2500}

df = pd.read_csv("expenses.csv", parse_dates=["date"])
df["month"] = df["date"].dt.strftime("%b")
months = df["month"].unique()                    # in file order: Jul, Aug, Sep

monthly = df.pivot_table(index="month", columns="category", values="amount", aggfunc="sum",
fill_value=0).loc[months]
print(monthly.to_string())
print("\nTotal per month:", monthly.sum(axis=1).to_dict())

latest = monthly.iloc[-1]
over = {cat: int(latest[cat] - limit) for cat, limit in BUDGET.items() if latest[cat] > limit}
print(f"Over budget in {months[-1]}:", over)
growth = (monthly.iloc[-1] - monthly.iloc[0]).drop("Rent").idxmax()
print("Fastest-growing category:", growth)

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10, 4))
colors = ["#1E2A4A", "#C9973A", "#4B7BB3", "#2E7D4F", "#B33A3A", "#8A8478"]
monthly.drop(columns="Rent").plot(kind="bar", stacked=True, ax=ax1, color=colors[1:], rot=0)
ax1.set(title="Monthly spending (excluding rent)", xlabel="", ylabel="rupees")
ax1.legend(fontsize=7, frameon=False)
share = df[df["month"] == months[-1]].groupby("category")["amount"].sum().sort_values()
ax2.barh(share.index, share.values, color=["#B33A3A" if c in over else "#1E2A4A" for c in
share.index])
ax2.set(title=f"{months[-1]}: red = over budget")
for s in ("top", "right"):
    ax1.spines[s].set_visible(False); ax2.spines[s].set_visible(False)
fig.tight_layout()
fig.savefig("expenses.png", dpi=110)
```

OUTPUT

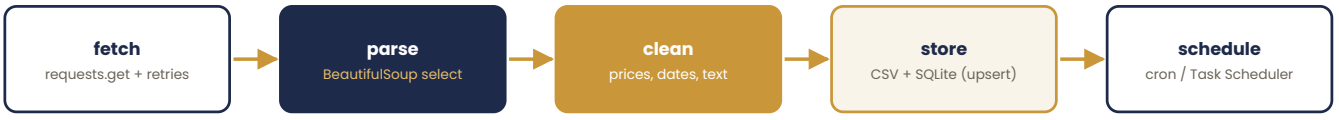
```
category  Bills  Food  Groceries   Rent  Shopping  Transport
month
Jul        1850  2130       4070  12000      2999       1160
Aug        2140  3800       4590  12000      1450       1470
Sep        1990  4710       4530  12000      4200        910

Total per month: {'Jul': 24209, 'Aug': 25450, 'Sep': 28340}
Over budget in Sep: {'Groceries': 30, 'Food': 1710, 'Shopping': 1700}
Fastest-growing category: Food
```

THE EXPENSE DASHBOARD PRODUCED BY THE PROGRAM

- **Features to add:** a command-line or Tkinter form to add expenses, recurring expenses, category budgets in a JSON settings file, Excel export with openpyxl, and a Streamlit web dashboard (`pip install streamlit`).
- **Data quality:** validate dates and amounts on input, reject negative values, and normalise category names (`str.strip().title()`).

25.3.3 Web scraper that saves data to CSV or a database

A scraper downloads web pages with **requests** (or httpx), parses the HTML with **BeautifulSoup** (23.4) and stores structured records in a CSV file or a database so that they can be analysed later. A robust scraper handles pagination, missing fields and network errors, waits between requests, and avoids storing duplicates. Always check the site's terms of service and `robots.txt`, prefer an official API when one exists, and never overload a server.



THE SCRAPER PIPELINE

A PAGINATED SCRAPER THAT CLEANS PRICES, HANDLES MISSING RATINGS AND SAVES TO CSV AND SQLITE (UPSERT) main.py

```
import csv, sqlite3
from datetime import date
from pathlib import Path
from bs4 import BeautifulSoup

def fetch(url):
    """In production: requests.get(url, headers=..., timeout=10) with retries and time.sleep(1).
    Here the pages are saved HTML files so the example runs offline."""
    return Path(url).read_text(encoding="utf-8")

def parse(html):
    soup = BeautifulSoup(html, "html.parser")
    for card in soup.select("div.product"):
        rating = card.select_one(".rating")              # may be missing
        yield {
            "name": card.select_one(".name").get_text(strip=True),
            "price": int(card.select_one(".price").get_text().strip("₹").replace(",", "")),
            "rating": float(rating.text) if rating else None,
```

```
        }
    nxt = soup.select_one("a.next")
    yield {"next": nxt["href"]} if nxt else {"next": None}

def scrape(start_url):
    url, products = start_url, []
    while url:
        items = list(parse(fetch(url)))
        url = items.pop()["next"]                       # the last item is the pagination link
        products.extend(items)
        print(f"  scraped {len(items)} products")
    return products

products = scrape("page1.html")

with open("laptops.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.DictWriter(f, fieldnames=["name", "price", "rating"])
    writer.writeheader(); writer.writerows(products)
print(Path("laptops.csv").read_text().splitlines()[:3])

db = sqlite3.connect("prices.db")
db.execute("""CREATE TABLE IF NOT EXISTS prices (name TEXT, day TEXT, price INTEGER, rating REAL,
              PRIMARY KEY (name, day))""")
with db:   # upsert: the same product on the same day keeps only the latest price
    db.executemany("""INSERT INTO prices VALUES (:name, :day, :price, :rating)
                      ON CONFLICT(name, day) DO UPDATE SET price = excluded.price""",
                   [{**p, "day": date(2026, 9, 1).isoformat()} for p in products])
for row in db.execute("SELECT name, price, rating FROM prices ORDER BY price"):
    print(row)
```

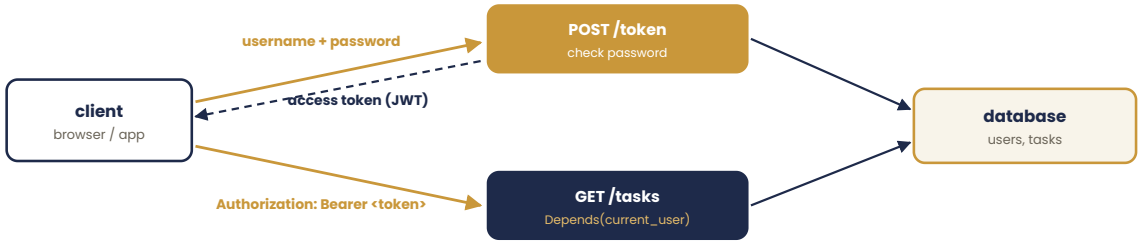
OUTPUT

```
  scraped 3 products
  scraped 2 products
['name,price,rating', 'AeroBook 14,54990,4.3', 'CodeMaster Pro 15,82500,4.6']
('StudyLite 11', 27999, None)
('AeroBook 14', 52990, 4.3)
('CodeMaster Pro 15', 82500, 4.6)
('GameForce X', 104990, 4.4)
```

Notice that "AeroBook 14" appears on both pages with different prices; the primary key `(name, day)` plus `ON CONFLICT ... DO UPDATE` keeps a single row with the latest price. Run the scraper daily (cron or Windows Task Scheduler, 25.3.5) and the table becomes a price history you can chart with pandas. For JavaScript-heavy sites use **Playwright** or **Selenium**; for large crawls use the **Scrapy** framework.

25.3.4 REST API with FastAPI and authentication

FastAPI (`pip install "fastapi[standard]"`) is a modern, high-performance web framework built on type hints. You declare path operations with decorators, request and response bodies as **Pydantic** models, and dependencies (database sessions, the current user) with `Depends`. FastAPI validates every request automatically, returns clear 422 errors for bad input, supports `async def` endpoints, and generates interactive documentation at `/docs` (Swagger UI) from your code. The project below is a task-manager API with registration, login that issues a **JWT** (JSON Web Token), and endpoints that only an authenticated user can call.



TOKEN AUTHENTICATION: LOG IN ONCE, THEN SEND THE TOKEN WITH EVERY REQUEST

MAIN.PY: A FASTAPI TASK MANAGER WITH PASSWORD HASHING AND JWT LOGIN

main.py

```
from datetime import datetime, timedelta, timezone
from typing import Annotated

import jwt                                              # pip install pyjwt
from fastapi import Depends, FastAPI, HTTPException, status
from fastapi.security import OAuth2PasswordBearer, OAuth2PasswordRequestForm
from pwdlib import PasswordHash                         # pip install "pwdlib[argon2]"
from pydantic import BaseModel, Field

SECRET_KEY = "change-me"          # in real projects: os.environ["SECRET_KEY"]
ALGORITHM, TOKEN_MINUTES = "HS256", 30

app = FastAPI(title="Task Manager API")
password_hash = PasswordHash.recommended()
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token")
users: dict[str, str] = {}                              # username -> password hash (use a database)
tasks: dict[str, list[dict]] = {}

class UserIn(BaseModel):
    username: str = Field(min_length=3, max_length=30)
    password: str = Field(min_length=8)

class TaskIn(BaseModel):
    title: str = Field(min_length=1, max_length=100)
    priority: int = Field(default=3, ge=1, le=5)

class Task(TaskIn):
    id: int
    done: bool = False

@app.post("/register", status_code=201)
def register(user: UserIn):
    if user.username in users:
        raise HTTPException(status.HTTP_409_CONFLICT, "username taken")
    users[user.username] = password_hash.hash(user.password)
    return {"username": user.username}

@app.post("/token")
def login(form: Annotated[OAuth2PasswordRequestForm, Depends()]):
    hashed = users.get(form.username)
    if not hashed or not password_hash.verify(form.password, hashed):
        raise HTTPException(status.HTTP_401_UNAUTHORIZED, "wrong username or password")
    expires = datetime.now(timezone.utc) + timedelta(minutes=TOKEN_MINUTES)
    token = jwt.encode({"sub": form.username, "exp": expires}, SECRET_KEY, algorithm=ALGORITHM)
    return {"access_token": token, "token_type": "bearer"}

def current_user(token: Annotated[str, Depends(oauth2_scheme)]) -> str:
    try:
        return jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])["sub"]
    except jwt.InvalidTokenError:
        raise HTTPException(status.HTTP_401_UNAUTHORIZED, "invalid or expired token",
```

```
                            headers={"WWW-Authenticate": "Bearer"})

@app.post("/tasks", response_model=Task, status_code=201)
def create_task(task: TaskIn, user: Annotated[str, Depends(current_user)]):
    mine = tasks.setdefault(user, [])
    new = Task(id=len(mine) + 1, **task.model_dump())
    mine.append(new.model_dump())
    return new

@app.get("/tasks", response_model=list[Task])
def list_tasks(user: Annotated[str, Depends(current_user)], done: bool | None = None):
    return [t for t in tasks.get(user, []) if done is None or t["done"] == done]
```

TEST_API.PY: EXERCISING THE API WITH FASTAPI'S TESTCLIENT main.py

```
from fastapi.testclient import TestClient
from main import app

client = TestClient(app)
print(client.post("/register", json={"username": "ravi", "password": "s3cret-pass"}).json())
print(client.post("/register", json={"username": "ab", "password": "x"}).status_code)
print(client.get("/tasks").status_code, client.get("/tasks").json())

token = client.post("/token", data={"username": "ravi", "password": "s3cret-pass"}).json()
["access_token"]
auth = {"Authorization": f"Bearer {token}"}
print(client.post("/tasks", json={"title": "Finish capstone", "priority": 1}, headers=auth).json())
print(client.post("/tasks", json={"title": "Update README"}, headers=auth).json())
print(client.post("/tasks", json={"title": "Bad", "priority": 9}, headers=auth).status_code)
print(len(client.get("/tasks", headers=auth).json()), "tasks for ravi")
```

OUTPUT

```
{'username': 'ravi'}
422
401 {'detail': 'Not authenticated'}
{'title': 'Finish capstone', 'priority': 1, 'id': 1, 'done': False}
{'title': 'Update README', 'priority': 3, 'id': 2, 'done': False}
422
2 tasks for ravi
```

Run the server with `fastapi dev main.py` and open http://127.0.0.1:8000/docs to try every endpoint in the browser. Convert the prints into pytest assertions for the project's test suite.

- **Make it production-ready:** store users and tasks in PostgreSQL or SQLite with **SQLModel** or SQLAlchemy, add update and delete endpoints, pagination, CORS for a front end, rate limiting and structured logging.
- **Deploy it:** write a Dockerfile, run with `fastapi run` (Uvicorn) behind a reverse proxy, and deploy to a cloud platform (Render, Railway, AWS, Azure or Google Cloud). Keep the secret key and database URL in environment variables.

25.3.5 Automation bot (email, Excel reports, file organiser)

Automation is often the first place Python saves real time at work. A typical bot (1) **organises files**, such as sorting a Downloads folder into sub-folders by type, (2) builds an **Excel report** from raw data with formatting and formulas using **openpyxl**, and (3) **emails** the report with `smtplib` and `email.message`. It then runs on a schedule: cron on Linux and macOS, Task Scheduler on Windows, or the `schedule` package inside a long-running script.

FILE ORGANISER: SORT FILES INTO FOLDERS BY TYPE, SAFELY, WITH A DRY-RUN MODE

main.py

```
import shutil, tempfile
from pathlib import Path

CATEGORIES = {"Images": {".jpg", ".png", ".gif"}, "Documents": {".pdf", ".docx", ".txt"},
              "Spreadsheets": {".xlsx", ".csv"}, "Archives": {".zip", ".rar"}}

def organise(folder: Path, dry_run=False):
    moved = {}
    for f in sorted(folder.iterdir()):
        if not f.is_file():
            continue
        target = next((cat for cat, exts in CATEGORIES.items() if f.suffix.lower() in exts), "Others")
        dest = folder / target / f.name
        counter = 1
        while dest.exists():                                  # never overwrite: add a number
            dest = folder / target / f"{f.stem}_{counter}{f.suffix}"; counter += 1
        if not dry_run:
            dest.parent.mkdir(exist_ok=True)
            shutil.move(f, dest)
        moved.setdefault(target, []).append(dest.name)
    return moved

downloads = Path(tempfile.mkdtemp()) / "Downloads"
downloads.mkdir()
for name in ["resume.pdf", "photo.JPG", "marks.xlsx", "notes.txt", "setup.exe", "backup.zip",
"chart.png"]:
    (downloads / name).write_text("demo")
(downloads / "Images").mkdir(); (downloads / "Images" / "photo.JPG").write_text("older copy")

print("dry run:", organise(downloads, dry_run=True))
result = organise(downloads)
for folder in sorted(p.name for p in downloads.iterdir()):
    print(f"{folder:<13}", sorted(x.name for x in (downloads / folder).iterdir()))
```

OUTPUT

```
dry run: {'Archives': ['backup.zip'], 'Images': ['chart.png', 'photo_1.JPG'], 'Spreadsheets':
['marks.xlsx'], 'Documents': ['notes.txt', 'resume.pdf'], 'Others': ['setup.exe']}
Archives      ['backup.zip']
Documents     ['notes.txt', 'resume.pdf']
Images        ['chart.png', 'photo.JPG', 'photo_1.JPG']
Others        ['setup.exe']
Spreadsheets  ['marks.xlsx']
```

AN EXCEL REPORT WITH STYLED HEADERS, FORMULAS, NUMBER FORMATS AND A CHART (OPENPYXL)

main.py

```
from openpyxl import Workbook, load_workbook
from openpyxl.chart import BarChart, Reference
from openpyxl.styles import Font, PatternFill, Alignment

sales = [("Mysuru", 41, 552_000), ("Bengaluru", 119, 1_745_000), ("Pune", 43, 598_000), ("Delhi", 36,
511_000)]

wb = Workbook()
ws = wb.active
ws.title = "Branch report"
ws.append(["Branch", "Students", "Revenue (Rs)", "Revenue per student"])
for branch, students, revenue in sales:
    ws.append([branch, students, revenue])
last = ws.max_row
for r in range(2, last + 1):
    ws[f"D{r}"] = f"=ROUND(C{r}/B{r}, 0)"                    # Excel formulas, recalculated in Excel
ws.append(["Total", f"=SUM(B2:B{last})", f"=SUM(C2:C{last})"])

header_fill = PatternFill("solid", start_color="1E2A4A")
for cell in ws[1]:
    cell.font, cell.fill = Font(bold=True, color="FFFFFF"), header_fill
    cell.alignment = Alignment(horizontal="center")
for cell in ws[ws.max_row]:
    cell.font = Font(bold=True)
for col, width in zip("ABCD", (14, 11, 15, 20)):
    ws.column_dimensions[col].width = width
for row in ws.iter_rows(min_row=2, min_col=3, max_col=4):
    for cell in row:
        cell.number_format = "#,##0"

chart = BarChart()
chart.title, chart.y_axis.title = "Revenue by branch", "Rs"
chart.add_data(Reference(ws, min_col=3, min_row=1, max_row=last), titles_from_data=True)
chart.set_categories(Reference(ws, min_col=1, min_row=2, max_row=last))
ws.add_chart(chart, "F2")
wb.save("branch_report.xlsx")

check = load_workbook("branch_report.xlsx").active
print([c.value for c in check[1]])
print([c.value for c in check[3]])
print([c.value for c in check[check.max_row]], "| charts:", len(check._charts))
```

OUTPUT

```
['Branch', 'Students', 'Revenue (Rs)', 'Revenue per student']
['Bengaluru', 119, 1745000, '=ROUND(C3/B3, 0)']
['Total', '=SUM(B2:B5)', '=SUM(C2:C5)', None] | charts: 1
```

EMAILING THE REPORT WITH AN ATTACHMENT (SMTP WITH TLS)

main.py

```
import os, smtplib
from email.message import EmailMessage
from pathlib import Path

def send_report(path, to):
    msg = EmailMessage()
    msg["Subject"] = "Weekly branch report"
    msg["From"] = os.environ["REPORT_SENDER"]
    msg["To"] = ", ".join(to)
    msg.set_content("Hello team,\n\nPlease find this week's branch report
attached.\n\nRegards,\nReport bot")
    msg.add_attachment(Path(path).read_bytes(), maintype="application",
                       subtype="vnd.openxmlformats-officedocument.spreadsheetml.sheet",
filename=Path(path).name)
    with smtplib.SMTP("smtp.gmail.com", 587, timeout=30) as smtp:
        smtp.starttls()                                  # encrypt the connection
        smtp.login(os.environ["REPORT_SENDER"], os.environ["SMTP_APP_PASSWORD"])
        smtp.send_message(msg)
    print(f"sent {Path(path).name} to {len(to)} recipients")

# Schedule: every Monday at 09:00
#   cron (Linux/macOS):  0 9 * * 1  /home/me/bot/.venv/bin/python /home/me/bot/report_bot.py
#   or in Python:        schedule.every().monday.at("09:00").do(job)   (pip install schedule)
send_report("branch_report.xlsx", ["manager@example.com", "accounts@example.com"])
```

OUTPUT

```
sent branch_report.xlsx to 2 recipients
```

Needs a real mail account; with Gmail, create an app password. Never write passwords in the code: read them from environment variables or a .env file.

25.3.6 Desktop app with Tkinter or a game with Pygame

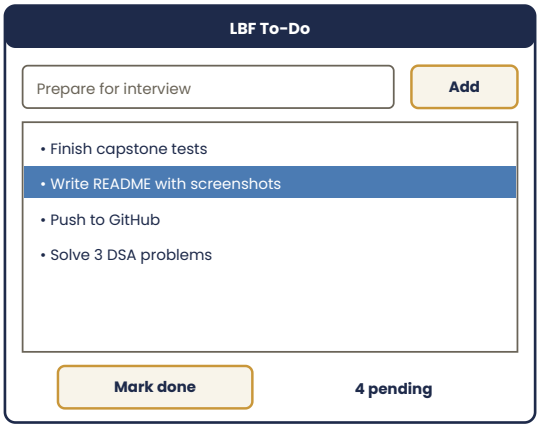
Tkinter ships with Python and builds simple desktop GUIs: a window (`Tk`), widgets (`Label`, `Entry`, `Button`, `Listbox`, the themed `ttk` widgets such as `Treeview`), geometry managers (`grid`, `pack`) and **event-driven** programming: you connect callbacks to events and hand control to `mainloop()`. Good capstones: a to-do app with SQLite, a GUI for your expense tracker, a quiz app, or a unit converter. Package it as a single executable with **PyInstaller** (`pyinstaller --onefile --windowed app.py`).

A TKINTER TO-DO APP THAT SAVES TASKS TO JSON

main.py

```
import json
import tkinter as tk
from pathlib import Path
from tkinter import messagebox, ttk

DATA = Path("todos.json")

class TodoApp(tk.Tk):
    def __init__(self):
        super().__init__()
        self.title("LBF To-Do")
        self.geometry("360x320")
        self.todos = json.loads(DATA.read_text()) if DATA.exists() else []

        self.entry = ttk.Entry(self)
        self.entry.grid(row=0, column=0, padx=8, pady=8, sticky="ew")
        self.entry.bind("<Return>", lambda event: self.add())      # Enter key adds a task
        ttk.Button(self, text="Add", command=self.add).grid(row=0, column=1, padx=8)
        self.listbox = tk.Listbox(self, height=10, selectmode=tk.SINGLE)
```

```
        self.listbox.grid(row=1, column=0, columnspan=2, padx=8, sticky="nsew")
        ttk.Button(self, text="Mark done", command=self.done).grid(row=2, column=0, pady=8)
        self.status = ttk.Label(self)
        self.status.grid(row=2, column=1)
        self.columnconfigure(0, weight=1); self.rowconfigure(1, weight=1)
        self.refresh()

    def add(self):
        text = self.entry.get().strip()
        if not text:
            messagebox.showwarning("Empty", "Type a task first")
            return
        self.todos.append(text); self.entry.delete(0, tk.END); self.refresh()

    def done(self):
        selected = self.listbox.curselection()
        if selected:
            self.todos.pop(selected[0]); self.refresh()

    def refresh(self):
        self.listbox.delete(0, tk.END)
        for t in self.todos:
            self.listbox.insert(tk.END, "• " + t)
        self.status.config(text=f"{len(self.todos)} pending")
        DATA.write_text(json.dumps(self.todos))

if __name__ == "__main__":
    TodoApp().mainloop()
```



THE TO-DO WINDOW: ENTRY AND BUTTON IN ROW 0, A LISTBOX IN ROW 1, AND THE STATUS BAR IN ROW 2

Pygame (`pip install pygame-ce`) is a library for 2-D games. Every game has the same **game loop**: handle events (keys, mouse, closing the window), update the game state (positions, collisions, score), draw the frame, and wait so that the game runs at a fixed frame rate. Good capstones: Snake, Pong, Flappy Bird or a space shooter, organised with classes for the player, enemies and the game itself, and sprites with `pygame.Rect` for collision detection.

A COMPLETE MINI-GAME WITH THE CLASSIC PYGAME GAME LOOP

main.py

```
import random
import pygame

pygame.init()
screen = pygame.display.set_mode((600, 400))
pygame.display.set_caption("Catch the Python")
clock, font = pygame.time.Clock(), pygame.font.SysFont(None, 32)
player = pygame.Rect(270, 360, 70, 16)
egg = pygame.Rect(random.randint(0, 580), 0, 20, 20)
score, speed, running = 0, 4, True

while running:
    for event in pygame.event.get():                # 1. events
        if event.type == pygame.QUIT:
            running = False
    keys = pygame.key.get_pressed()                 # 2. update
    player.x += (keys[pygame.K_RIGHT] - keys[pygame.K_LEFT]) * 7
    player.clamp_ip(screen.get_rect())
    egg.y += speed
    if egg.colliderect(player):
        score, speed = score + 1, speed + 0.3
        egg.topleft = (random.randint(0, 580), 0)
    elif egg.top > 400:
        running = False                             # missed one: game over
    screen.fill((30, 42, 74))                       # 3. draw
    pygame.draw.rect(screen, (201, 151, 58), player, border_radius=6)
    pygame.draw.ellipse(screen, (255, 255, 255), egg)
    screen.blit(font.render(f"Score: {score}", True, (255, 255, 255)), (10, 10))
    pygame.display.flip()
    clock.tick(60)                                  # 4. 60 frames per second
print("Final score:", score)
pygame.quit()
```

25.4 Interview preparation

Python developer interviews usually have several rounds: an online **coding test** (DSA problems on HackerRank, HackerEarth, Codility or LeetCode-style platforms), one or two **technical interviews** (Python concepts, live coding, your projects, and for experienced roles system design and SQL), and an **HR or managerial** round. Preparation therefore has three parts: core Python knowledge, problem solving, and presenting your work.

A TYPICAL HIRING PROCESS FOR A PYTHON DEVELOPER ROLE

25.4.1 Core Python interview topics checklist

Interviewers check whether you understand **how** Python works, not just its syntax. Make sure you can explain each topic below in two or three sentences and write a short example from memory.

Topic	What you must be able to explain	Book
mutability	mutable (list, dict, set) vs immutable (int, str, tuple) objects; names are references; aliasing; shallow vs deep copy; mutable default arguments	3, 8, 10

Topic	What you must be able to explain	Book
scope	LEGB rule; global and nonlocal; closures capture variables, not values (late binding)	10, 11
functions	*args and **kwargs; keyword-only and positional-only parameters; first-class functions; lambda; map, filter, reduce	10, 11
decorators	a function that takes and returns a function; functools.wraps; decorators with arguments; @property, @staticmethod, @classmethod	11, 16
iterators and generators	iterable vs iterator (__iter__, __next__); yield and lazy evaluation; generator expressions; memory benefits; yield from	17
context managers	with, __enter__ and __exit__, contextlib.contextmanager; guaranteed cleanup	17
OOP	classes and instances; inheritance and MRO; super(); polymorphism and duck typing; abstract classes; dunder methods; dataclasses	15, 16
exceptions	try/except/else/finally; custom exceptions; raise from; EAFP vs LBYL	13
GIL and concurrency	the GIL lets one thread run Python bytecode at a time; threads for I/O, processes for CPU; asyncio; free-threaded builds	20
memory	reference counting plus the cyclic garbage collector; is vs ==; small-integer caching; __slots__; sys.getsizeof	21
data structures	complexity of list, dict and set operations; when to use deque, Counter, defaultdict, heapq	8, 9, 25
modern Python	type hints, f-strings, the walrus operator, match statements, comprehensions, virtual environments, pip and pyproject.toml	various

FIVE CONCEPTS THAT APPEAR IN ALMOST EVERY PYTHON INTERVIEW main.py

```
import copy

# 1. Mutable default argument: the list is created once, when def runs
def add_item(item, basket=[]):
    basket.append(item)
    return basket
print(add_item("pen"), add_item("book"))            # surprise: shared list
def add_item_fixed(item, basket=None):
    basket = [] if basket is None else basket
    basket.append(item)
    return basket
print(add_item_fixed("pen"), add_item_fixed("book"))

# 2. Aliasing, shallow and deep copies
grid = [[0, 0], [0, 0]]
alias, shallow, deep = grid, copy.copy(grid), copy.deepcopy(grid)
grid[0][0] = 9
print(alias[0][0], shallow[0][0], deep[0][0])

# 3. Closures bind variables late
funcs = [lambda: i for i in range(3)]
print([f() for f in funcs], [f() for f in [lambda i=i: i for i in range(3)]])

# 4. A generator is lazy and single-use
squares = (n * n for n in range(4))
print(sum(squares), sum(squares))

# 5. is versus ==
a, b = [1, 2], [1, 2]
print(a == b, a is b, None is None)
```

OUTPUT

```
['pen', 'book'] ['pen', 'book']
['pen'] ['book']
9 9 0
[2, 2, 2] [0, 1, 2]
14 0
True False True
```

Also prepare short spoken answers for questions such as: *What is the difference between a list and a tuple?* (mutability, hashability, intent: a record vs a collection) *How is a dict implemented?* (a hash table, 25.1.6) *What does* `if __name__ == "__main__":` *do? What are *args and **kwargs? Explain the GIL and when you would use multiprocessing. What is the difference between* `__str__` *and* `__repr__`*? How do you manage dependencies?*

25.4.2 Output-prediction and tricky questions

Many tests show a short snippet and ask what it prints. Work them out on paper first, then run them. Each one below tests a concept from this book; the explanation follows the output.

TEN OUTPUT-PREDICTION QUESTIONS main.py

```
# Q1: chained comparison
print(1 < 2 < 3, 3 > 2 > 1 > 0 == False)

# Q2: integer division and rounding
print(7 // 2, -7 // 2, -7 % 3, round(2.5), round(3.5), int(-2.7))

# Q3: float precision
```

```
print(0.1 + 0.2 == 0.3, abs(0.1 + 0.2 - 0.3) < 1e-9)

# Q4: list multiplication copies references
rows = [[0] * 2] * 3
rows[0][0] = 1
print(rows)

# Q5: += on a list inside a tuple
t = ([1],)
try:
    t[0] += [2]
except TypeError as e:
    print("TypeError, but t is now", t)

# Q6: dictionary keys 1, 1.0 and True are equal
print({1: "a", 1.0: "b", True: "c"})

# Q7: finally overrides return
def f():
    try:
        return "try"
    finally:
        return "finally"
print(f())

# Q8: modifying a list while iterating
nums = [1, 2, 2, 3]
for n in nums:
    if n == 2:
        nums.remove(n)
print(nums)

# Q9: class attribute shared by instances
class Counter:
    items = []
    def add(self, x):
        self.items.append(x)
a, b = Counter(), Counter()
a.add(1); b.add(2)
print(a.items, b.items is a.items)

# Q10: sorting stability, any/all on empty, string multiplication
print(sorted(["bb", "a", "cc", "d"], key=len), any([]), all([]), "ab" * 0 == "")
```

OUTPUT

```
main.py:30: SyntaxWarning: 'return' in a 'finally' block
  return "finally"
True True
3 -4 2 2 4 -2
False True
[[1, 0], [1, 0], [1, 0]]
TypeError, but t is now ([1, 2],)
{1: 'c'}
finally
[1, 2, 3]
[1, 2] True
['a', 'd', 'bb', 'cc'] False True True
```

Q	Why
1	`a < b < c` means `a < b and b < c`; the second is `3 > 2 and 2 > 1 and 1 > 0 and 0 == False`, and `0 == False` is True.

Q	Why
2	`//` floors towards minus infinity (-3.5 → -4); `%` takes the sign of the divisor; `round` uses banker's rounding (to even); `int` truncates towards zero.
3	Binary floating point cannot represent 0.1 exactly; compare with a tolerance (`math.isclose`) or use `Decimal`.
4	`[row] * 3` repeats a reference to the **same** inner list. Use `[[0] * 2 for _ in range(3)]`.
5	The list is extended in place first, then assigning back to the tuple item fails: both effects happen.
6	`1 == 1.0 == True` and their hashes are equal, so they are the same key: the first key is kept with the last value.
7	A `return` in `finally` replaces the pending return value (and would even swallow an exception). Python 3.14 warns about it at compile time (the SyntaxWarning above, PEP 765). Avoid it.
8	Removing shifts later items left, so the iterator skips the second 2. Iterate over a copy or build a new list.
9	`items` is a class attribute: one list shared by every instance. Create it in `__init__`.
10	Sorting is stable (bb before cc, a before d); `any([])` is False, `all([])` is True (vacuous truth).

Other classic tricky questions: *What is printed by* `print(type(type))`*?* (`<class 'type'>`, everything is an object and classes are instances of `type`) *Why does* `[] is []` *give False but* `() is ()` *may give True?* (immutable singletons can be cached) *Why can't a lambda in a loop see the loop value?* (late binding) *What happens when you change a dict while iterating over it?* (RuntimeError).

25.4.3 Coding rounds on DSA (LeetCode, HackerRank)

In a coding round you solve one to three problems in 45 to 90 minutes, often while explaining your thinking. Interviewers judge your **approach** as much as your code: clarify the problem, give a brute-force idea, improve it, write clean code, test it and state its complexity. Python is an excellent interview language because it is concise; use its built-ins (`sorted`, `Counter`, `defaultdict`, `deque`, `heapq`, `bisect`, `itertools`, `functools.cache`) but be ready to implement the underlying idea if asked.

A FIVE-STEP METHOD FOR EVERY CODING PROBLEM

If the problem mentions…	Try this pattern
sorted array, find a pair or a target	two pointers, binary search
contiguous subarray or substring (longest, shortest, at most k)	sliding window, prefix sums
counting, duplicates, "have we seen it?"	hash map / set (Counter)
top k, k-th largest, merge k lists, schedule	heap
next greater/smaller element, valid brackets	stack (monotonic stack)
all combinations, permutations, subsets, constraints	backtracking
minimum/maximum number of ways, choices that overlap	dynamic programming

If the problem mentions...	Try this pattern
grid, network, dependencies, shortest steps	BFS / DFS / topological sort / Dijkstra
tree path, depth, ancestor	recursive DFS returning values
"minimum value such that it is possible"	binary search on the answer

FIVE FREQUENTLY ASKED PROBLEMS SOLVED WITH HASH MAPS AND PREFIX SUMS

main.py

```
from collections import Counter, defaultdict

def two_sum(nums, target):
    """Indices of two numbers adding to target: one pass with a dict, O(n)."""
    seen = {}
    for i, x in enumerate(nums):
        if target - x in seen:
            return [seen[target - x], i]
        seen[x] = i

def group_anagrams(words):
    groups = defaultdict(list)
    for w in words:
        groups["".join(sorted(w))].append(w)
    return list(groups.values())

def top_k_frequent(nums, k):
    return [x for x, _ in Counter(nums).most_common(k)]

def product_except_self(nums):
    """Prefix and suffix products, no division, O(n)."""
    out, prefix = [1] * len(nums), 1
    for i in range(len(nums)):
        out[i], prefix = prefix, prefix * nums[i]
    suffix = 1
    for i in reversed(range(len(nums))):
        out[i] *= suffix; suffix *= nums[i]
    return out

def subarray_sum_count(nums, k):
    """Number of contiguous subarrays summing to k: prefix sums + hash map."""
    count, total, seen = 0, 0, Counter({0: 1})
    for x in nums:
        total += x
        count += seen[total - k]
        seen[total] += 1
    return count

print(two_sum([2, 7, 11, 15], 9))
print(group_anagrams(["eat", "tea", "tan", "ate", "nat", "bat"]))
print(top_k_frequent([1, 1, 1, 2, 2, 3], 2))
print(product_except_self([1, 2, 3, 4]))
print(subarray_sum_count([1, 2, 1, 2, 1], 3))
```

OUTPUT

```
[0, 1]
[['eat', 'tea', 'ate'], ['tan', 'nat'], ['bat']]
[1, 2]
[24, 12, 8, 6]
4
```

A practical study plan: learn one pattern at a time from 25.1 and 25.2, then solve 5 to 8 problems for it, starting with easy ones. Use curated lists (Blind 75, NeetCode 150, the "Top Interview 150" list on LeetCode) and HackerRank's Python and interview-preparation tracks. Time yourself, and when you are stuck for more than 25 minutes, study the solution, then re-solve the problem from scratch two days later. Keep a log of every problem with its pattern and the key idea.

Week	Topics	Problems
1	arrays, strings, hash maps, two pointers	10
2	sliding window, prefix sums, stack, binary search	10
3	linked lists, trees, BST, heaps	10
4	recursion, backtracking, greedy	8
5	graphs: BFS, DFS, topological sort, Dijkstra	6
6	dynamic programming: 1-D, 2-D, knapsack, strings	6

25.4.4 Explaining your projects and GitHub profile

"Tell me about a project you built" is asked in almost every interview. Prepare a two-minute explanation for each project using the **STAR** structure: the **Situation** (what problem or need), the **Task** (your goal and role), the **Actions** (design, technologies and why you chose them, a difficult bug or decision) and the **Result** (what it does now, numbers if possible, what you learned and would improve). Expect follow-up questions on any line of code, so build your projects yourself and understand every part.

Situation

My institute tracked library loans in a notebook, and overdue books were often lost.

Task

Build a system that records loans, calculates fines and reminds members automatically.

Action

Python OOP + SQLite with a repository layer, Flask UI, pytest (95% coverage), scheduled email reminders.

Result

Used daily by 2 librarians; overdue books fell sharply; I learned to design database schemas and write tests.

Your **GitHub profile** is your public portfolio. Pin your three to six best repositories; give each a descriptive name, a one-line description and topics. Write a profile README (a repository named after your username) with a short introduction, your skills and links. Commit regularly with meaningful messages, keep secrets out of the history (.gitignore, environment variables), and add a licence. Every project README should contain:

README.MD: A TEMPLATE FOR A PORTFOLIO PROJECT

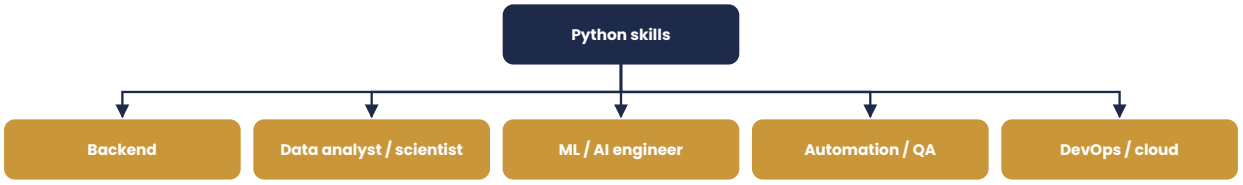
text

```
# Library Management System

A command-line and web app to manage books, members, loans and late fines.

![Screenshot of the dashboard](docs/dashboard.png)

## Features
- Add, search and edit books and members
- Issue and return books with automatic due dates and fines
- Overdue report and weekly email reminders

## Tech stack
Python 3.13, SQLite, Flask, pytest, Ruff, GitHub Actions

## Getting started
    git clone https://github.com/<you>/library-system.git
    cd library-system
    python -m venv .venv && source .venv/bin/activate
    pip install -r requirements.txt
    python -m library            # or: flask --app library.web run

## Running the tests
    pytest --cov=library

## What I learned / next steps
- Designing a normalised schema; testing with an in-memory database
- Next: role-based login and a REST API
```

On your resume, list two or three projects with one line each on what they do, the technologies and one measurable result, plus links to the repository and a live demo. Mention your DSA practice (for example, "solved 150+ problems on LeetCode") and any certifications or internships.

25.5 Career paths with Python

Python is used across the software industry, so the same core skills open several career paths. Each path adds its own libraries, tools and domain knowledge on top of what this book covers. Pick one direction for your first job, build projects in that direction, and keep strengthening the fundamentals: Python, Git, SQL, Linux and problem solving are valued everywhere.



FIVE CAREER PATHS BUILT ON THE SAME PYTHON FOUNDATION

25.5.1 Backend web development (Django, FastAPI)

Backend developers build the server side of web and mobile applications: REST or GraphQL APIs, business logic, databases, authentication, background jobs and integrations with other services. **Django** is a "batteries-included" framework (ORM, admin site, authentication, forms, migrations) ideal for complete web applications; Django REST Framework adds APIs. **FastAPI** is preferred for fast, typed, async APIs and microservices; **Flask** is a lightweight option (Module 23). You also need SQL and a relational database

(PostgreSQL), an ORM (Django ORM, SQLAlchemy or SQLModel), caching with Redis, task queues (Celery), Docker, testing, and API security basics (hashing, JWT, OWASP Top 10).

25.5.2 Data analyst and data scientist

Data analysts turn data into decisions: they clean data, write SQL queries, analyse it with pandas and present findings with charts and dashboards (Power BI, Tableau, Excel, Streamlit). **Data scientists** go further with statistics, hypothesis testing, experiments (A/B tests) and predictive models built with scikit-learn. Key skills: SQL (very important), pandas, NumPy, Matplotlib and Seaborn, statistics and probability, Jupyter, and the communication skill to tell a clear story with data. Build projects on real public datasets (Kaggle, data.gov.in) and publish notebooks and dashboards.

25.5.3 Machine learning and AI engineer

ML engineers build, train, evaluate and **deploy** models; AI engineers increasingly build applications on top of large language models. Skills: the mathematics behind ML (linear algebra, calculus, probability), scikit-learn, deep learning with **PyTorch** (or TensorFlow/Keras), Hugging Face Transformers, model evaluation and data pipelines, and MLOps (experiment tracking with MLflow, serving models behind a FastAPI endpoint, Docker, monitoring). For generative AI: LLM APIs, prompt design, retrieval-augmented generation with vector databases, tool use and agents, and evaluating model outputs (24.5).

25.5.4 Automation and QA engineer

Automation engineers write scripts and bots that remove manual work: file and report processing, Excel and email automation, web automation, and integration between business systems through APIs. **QA automation** (software development engineer in test) engineers build automated test suites: **pytest** for unit and API tests (with `requests` or `httpx`), **Selenium** or **Playwright** for browser testing, the page object model, test data management, reporting (Allure) and running the tests in CI pipelines. Knowledge of testing theory (test cases, boundary values, regression testing) and Git is essential.

25.5.5 DevOps and cloud engineer

DevOps and cloud engineers automate building, testing, deploying and running software. Python is their main scripting language: automation scripts, cloud SDKs (**boto3** for AWS, the Azure and Google Cloud SDKs), infrastructure as code (Terraform, Pulumi with Python, AWS CDK), configuration management (Ansible is written in Python), CI/CD pipelines (GitHub Actions, GitLab CI, Jenkins), containers (Docker) and orchestration (Kubernetes), and monitoring and logging (Prometheus, Grafana). Strong Linux, networking and shell skills are needed, and a cloud certification (for example, AWS Cloud Practitioner, then Solutions Architect Associate) helps.

Path	Core Python libraries and tools	Other skills	Portfolio project
Backend developer	Django, DRF, FastAPI, SQLAlchemy, pytest, Celery	SQL, PostgreSQL, REST, Docker, Git, security	REST API with auth, deployed (25.3.4)
Data analyst / scientist	pandas, NumPy, Matplotlib, Seaborn, scikit-learn, Jupyter	SQL, statistics, Excel, Power BI or Tableau, storytelling	EDA notebook + dashboard (24, 25.3.2)
ML / AI engineer	scikit-learn, PyTorch, Transformers, LLM SDKs, MLflow	maths, model evaluation, MLOps, vector databases	a deployed model or a RAG chatbot
Automation / QA	pytest, Selenium, Playwright, requests, openpyxl	testing theory, CI, Git, APIs	test framework or automation bot (25.3.5)

Path	Core Python libraries and tools	Other skills	Portfolio project
DevOps / cloud	boto3, Fabric, Ansible, Pulumi or CDK, click	Linux, networking, Docker, Kubernetes, CI/CD, a cloud	automated CI/CD pipeline with infrastructure as code

TIP

Whatever path you choose, keep learning in public: share project write-ups on LinkedIn or a blog, contribute small fixes to open-source projects, and join Python communities and meetups (PyCon India, local Python user groups). Consistent, visible practice opens more doors than certificates alone.

Big-O describes how cost grows with input size: list indexing and append are O(1), list membership and insertion at the front are O(n), dict and set operations are O(1) on average, and sorting is O(n log n). Linked lists give O(1) insertion at a known node (doubly linked lists power LRU caches); a list is a stack and a deque is a queue; BSTs give O(log n) operations when balanced and their in-order traversal is sorted; heapq keeps a min-heap in a list with O(log n) push and pop; dicts are hash tables with open addressing and resizing; graphs are dicts of adjacency lists; tries store strings by prefix. Merge sort is stable O(n log n), quick sort averages O(n log n) in place, and Python uses Timsort. Binary search and bisect take O(log n); two pointers and sliding windows turn $O(n^2)$ scans into O(n); backtracking explores choices and undoes them; greedy works only with the greedy-choice property; DP stores answers to overlapping subproblems; BFS finds shortest unweighted paths, DFS explores deeply, Dijkstra handles non-negative weights and topological sort orders dependencies. Finish one strong capstone project with tests and a README, prepare core Python concepts, tricky questions and DSA patterns, explain projects with STAR, and choose a career path to specialise in.

PRACTICE CHECKPOINT

Build **one capstone project** from 25.3 with **tests**, a **README** and a **GitHub repository**, and solve **50 DSA problems** in Python across the topics of this module.

A suggested plan for the checkpoint: spend about three weeks on the capstone and practise DSA alongside it. Track progress in a simple file in your repository; the program below is a tracker you can use for the 50 problems. It records each solved problem with its topic and difficulty, shows progress per topic against the targets of the six-week plan in 25.4.3, and suggests what to practise next.

SAMPLE SOLUTION: A PROGRESS TRACKER FOR THE 50 DSA PROBLEMS

main.py

```
import csv, io
from collections import Counter

TARGETS = {"arrays & hashing": 6, "two pointers": 4, "sliding window": 4, "stack": 4, "binary search":
4,
           "linked list": 4, "trees": 6, "heap": 3, "backtracking": 4, "greedy": 2, "graphs": 5, "dp":
4}
LOG = """problem,topic,difficulty,minutes
Two Sum,arrays & hashing,easy,12
Group Anagrams,arrays & hashing,medium,25
Top K Frequent Elements,arrays & hashing,medium,20
Product of Array Except Self,arrays & hashing,medium,35
Valid Palindrome,two pointers,easy,10
3Sum,two pointers,medium,40
Longest Substring Without Repeating Characters,sliding window,medium,30
Valid Parentheses,stack,easy,10
Daily Temperatures,stack,medium,28
Binary Search,binary search,easy,8
Reverse Linked List,linked list,easy,12
LRU Cache,linked list,medium,45
Maximum Depth of Binary Tree,trees,easy,9
Validate Binary Search Tree,trees,medium,26
Kth Largest Element in an Array,heap,medium,15
Subsets,backtracking,medium,22
Number of Islands,graphs,medium,24
Course Schedule,graphs,medium,38
Climbing Stairs,dp,easy,10
Coin Change,dp,medium,33
"""

rows = list(csv.DictReader(io.StringIO(LOG)))
done = Counter(r["topic"] for r in rows)
levels = Counter(r["difficulty"] for r in rows)
goal = sum(TARGETS.values())
print(f"Solved {len(rows)}/{goal} ({len(rows) / goal:.0%}) | " + ", ".join(f"{k}: {v}" for k, v in
levels.items()))
print(f"Average time: {sum(int(r['minutes']) for r in rows) / len(rows):.0f} min")
for topic, target in TARGETS.items():
    n = done[topic]
    print(f"{topic:<17} {'#' * n}{'.' * (target - n)}  {n}/{target}")
behind = sorted(TARGETS, key=lambda t: done[t] / TARGETS[t])[:3]
print("Practise next:", ", ".join(behind))
```

OUTPUT

```
Solved 20/50 (40%) | easy: 7, medium: 13
Average time: 23 min
arrays & hashing  ####..  4/6
two pointers      ##..  2/4
sliding window    #...  1/4
stack             ##..  2/4
binary search     #...  1/4
linked list       ##..  2/4
trees             ##....  2/6
heap              #..  1/3
backtracking      #...  1/4
greedy            ..  0/2
graphs            ##...  2/5
dp                ##..  2/4
Practise next: greedy, sliding window, binary search
```

- **Capstone checklist:** a clear problem statement; a src layout with modules and classes; data stored in SQLite or files; input validation and error handling; logging and configuration from environment variables; pytest tests for the core logic (aim for 80%+ coverage); Ruff and mypy passing; a README with screenshots, setup steps and a feature list; a `requirements.txt` or `pyproject.toml`; a GitHub Actions workflow that runs the tests on every push (22.5); and, if possible, a live demo.
- **DSA checklist:** about 50 problems across all the patterns, mostly easy and medium; for each, write the brute-force idea and the optimised solution with its complexity, and re-solve the ones you found hard a week later.

Congratulations on completing the Python Programming course. You now have the language fundamentals, the intermediate and advanced tools, and a map of the professional ecosystem. Keep building, keep practising, and keep moving forward.